

Introduction to Computer Science

Introduction to Computer Science

SENIOR CONTRIBUTING AUTHOR

DR. JEAN-CLAUDE FRANCHITTI, NYU COURANT INSTITUTE



openstax™

OpenStax
Rice University
6100 Main Street MS-375
Houston, Texas 77005

To learn more about OpenStax, visit https://openstax.org.
Individual print copies and bulk orders can be purchased through our website.

©2026 Rice University. Textbook content produced by OpenStax is licensed under a Creative Commons Attribution Non-Commercial ShareAlike 4.0 International License (CC BY-NC-SA 4.0). Under this license, any user of this textbook or the textbook contents herein can share, remix, and build upon the content for noncommercial purposes only. Any adaptations must be shared under the same type of license. In any case of sharing the original or adapted material, whether in whole or in part, the user must provide proper attribution as follows:

- If you noncommercially redistribute this textbook in a digital format (including but not limited to PDF and HTML), then you must retain on every page the following attribution:
  "Access for free at openstax.org."
- If you noncommercially redistribute this textbook in a print format, then you must include on every physical page the following attribution:
  "Access for free at openstax.org."
- If you noncommercially redistribute part of this textbook, then you must retain in every digital format page view (including but not limited to PDF and HTML) and on every physical printed page the following attribution:
  "Access for free at openstax.org."
- If you use this textbook as a bibliographic reference, please include https://openstax.org/details/books/introduction-computer-science in your citation.

For questions regarding this licensing, please contact support@openstax.org.

Trademarks
The OpenStax name, OpenStax logo, OpenStax book covers, OpenStax CNX name, OpenStax CNX logo, OpenStax Tutor name, Openstax Tutor logo, Connexions name, Connexions logo, Rice University name, and Rice University logo are not subject to the license and may not be reproduced without the prior and express written consent of Rice University.

Kendall Hunt and the Kendall Hunt Logo are trademarks of Kendall Hunt. The Kendall Hunt mark is registered in the United States, Canada, and the European Union. These trademarks may not be used without the prior and express written consent of Kendall Hunt.

COLOR PAPERBACK BOOK ISBN-13	**978-1-711471-83-9**
B&W PAPERBACK BOOK ISBN-13	**978-1-711471-82-2**
DIGITAL VERSION ISBN-13	**978-1-961584-58-7**
ORIGINAL PUBLICATION YEAR	**2024**

2 3 4 5 6 7 8 9 10 CJP 26 24

OpenStax

OpenStax provides free, peer-reviewed, openly licensed textbooks for introductory college and Advanced Placement® courses and low-cost, personalized courseware that helps students learn. A nonprofit ed tech initiative based at Rice University, we're committed to helping students access the tools they need to complete their courses and meet their educational goals.

Rice University

OpenStax is an initiative of Rice University. As a leading research university with a distinctive commitment to undergraduate education, Rice University aspires to path-breaking research, unsurpassed teaching, and contributions to the betterment of our world. It seeks to fulfill this mission by cultivating a diverse community of learning and discovery that produces leaders across the spectrum of human endeavor.



Philanthropic Support

OpenStax is grateful for the generous philanthropic partners who advance our mission to improve educational access and learning for everyone. To see the impact of our supporter community and our most updated list of partners, please visit openstax.org/foundation.

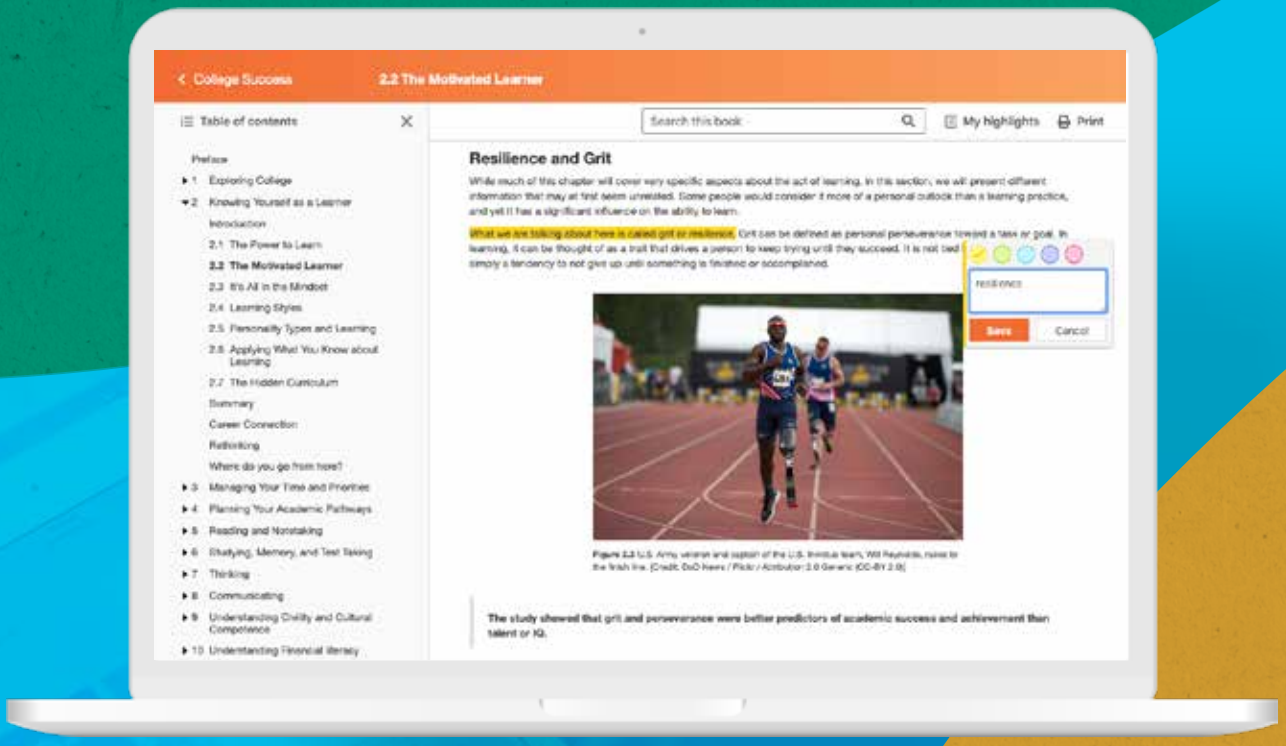
Arnold Ventures
Chan Zuckerberg Initiative
Chegg, Inc.
Arthur and Carlyse Ciocca Charitable Foundation
Digital Promise
Ann and John Doerr
Bill & Melinda Gates Foundation
Girard Foundation
Google Inc.
The William and Flora Hewlett Foundation
The Hewlett-Packard Company
Intel Inc.
Rusty and John Jaggers
The Calvin K. Kazanjian Economics Foundation
Charles Koch Foundation
Leon Lowenstein Foundation, Inc.
The Maxfield Foundation
Burt and Deedee McMurtry
Michelson 20MM Foundation
National Science Foundation
The Open Society Foundations
Jumee Yhu and David E. Park III
Brian D. Patterson USA-International Foundation
The Bill and Stephanie Sick Fund
Steven L. Smith & Diana T. Go
Stand Together
Robin and Sandy Stuart Foundation
The Stuart Family Foundation
Tammy and Guillermo Treviño
Valhalla Charitable Foundation
White Star Education Foundation
Schmidt Futures
William Marsh Rice University

openstax™

Study where you want, what you want, when you want.

When you access your book in our web view, you can use our new online **highlighting and note-taking** features to create your own study guides.

Our books are free and flexible, forever.
Get started at **openstax.org/details/books/introduction-computer-science**



Access. The future of education.
openstax.org

RICE

CONTENTS

Preface 1

PART 1 PROBLEM SOLVING AND ALGORITHMS

1 Introduction to Computer Science 9

Introduction 9
1.1 Computer Science 10
1.2 Computer Science across the Disciplines 20
1.3 Computer Science and the Future of Society 25
Chapter Review 33

2 Computational Thinking and Design Reusability 39

Introduction 39
2.1 Computational Thinking 40
2.2 Architecting Solutions with Adaptive Design Reuse in Mind 52
2.3 Evolving Architectures into Useable Products 71
Chapter Review 81

3 Data Structures and Algorithms 89

Introduction 89
3.1 Introduction to Data Structures and Algorithms 89
3.2 Algorithm Design and Discovery 97
3.3 Formal Properties of Algorithms 104
3.4 Algorithmic Paradigms 110
3.5 Sample Algorithms by Problem 116
3.6 Computer Science Theory 123
Chapter Review 128

PART 2 REALIZATIONS OF ALGORITHMS

4 Linguistic Realization of Algorithms: Low-Level Programming Languages 143

Introduction 143
4.1 Models of Computation 144
4.2 Building C Programs 156
4.3 Parallel Programming Models 173
4.4 Applications of Programming Models 179
Chapter Review 182

5 Hardware Realizations of Algorithms: Computer Systems Design 193

Introduction 193

5.1 Computer Systems Organization 194
5.2 Computer Levels of Abstraction 198
5.3 Machine-Level Information Representation 205
5.4 Machine-Level Program Representation 211
5.5 Memory Hierarchy 217
5.6 Processor Architectures 226
Chapter Review 230

6 Infrastructure Abstraction Layer: Operating Systems 239

Introduction 239
6.1 What Is an Operating System? 240
6.2 Fundamental OS Concepts 246
6.3 Processes and Concurrency 257
6.4 Memory Management 267
6.5 File Systems 274
6.6 Reliability and Security 279
Chapter Review 285

PART 3 DESIGNING AND DEVELOPING SOFTWARE SOLUTIONS

7 High-Level Programming Languages 297

Introduction 297
7.1 Programming Language Foundations 298
7.2 Programming Language Constructs 311
7.3 Alternative Programming Models 331
7.4 Programming Language Implementation 340
Chapter Review 347

8 Data Management 359

Introduction 359
8.1 Data Management Focus 360
8.2 Data Management Systems 366
8.3 Relational Database Management Systems 372
8.4 Nonrelational Database Management Systems 390
8.5 Data Warehousing, Data Lakes, and Business Intelligence 397
8.6 Data Management for Shallow and Deep Learning Applications 402
8.7 Informatics and Data Management 412
Chapter Review 414

9 Software Engineering 429

Introduction 429
9.1 Software Engineering Fundamentals 430
9.2 Software Engineering Process 440

9.3 Special Topics 468
Chapter Review 490

10 Enterprise and Solution Architectures Management 501

Introduction 501
10.1 Patterns Management 502
10.2 Enterprise Architecture Management Frameworks 510
10.3 Solution Architecture Management 545
Chapter Review 550

PART 4 BUILDING MODERN END-TO-END SOLUTIONS TO BUSINESS AND SOCIAL PROBLEMS

11 Web Applications Development 559

Introduction 559
11.1 Modern Web Applications Architectures 560
11.2 Sample Responsive WAD with Bootstrap and Django 578
11.3 Sample Responsive WAD with Bootstrap/React and Node 604
11.4 Sample Responsive WAD with Bootstrap/React and Django 619
11.5 Sample Native WAD with React Native and Node or Django 624
11.6 Sample Ethereum Blockchain Web 2.0/Web 3.0 Application 637
Chapter Review 650

12 Cloud-Native Applications Development 659

Introduction 659
12.1 Introduction to Cloud-Native Applications 660
12.2 Cloud-Based and Cloud-Native Applications Deployment Technologies 684
12.3 Example PaaS and FaaS Deployments of Cloud-Native Applications 705
Chapter Review 744

13 Hybrid Multicloud Digital Solutions Development 755

Introduction 755
13.1 Hybrid Multicloud Solutions and Cloud Mashups 756
13.2 Big Cloud IaaS Mainstream Capabilities 760
13.3 Big Cloud PaaS Mainstream Capabilities 768
13.4 Towards Intelligent Autonomous Networked Super Systems 784
Chapter Review 799

PART 5 HUMAN-CENTERED RESPONSIBLE COMPUTING

14 Cyber Resources Qualities and Cyber Computing Governance 811

Introduction 811
14.1 Cyber Resources Management Frameworks 812
14.2 Cybersecurity Deep Dive 833

14.3 Governing the Use of Cyber Resources 893
Chapter Review 900

A Appendix A: Network Design Application of Algorithms 911

Index 917

Preface

About OpenStax

OpenStax is part of Rice University, which is a 501(c)(3) nonprofit charitable corporation. As an educational initiative, it's our mission to improve educational access and learning for everyone. Through our partnerships with philanthropic organizations and our alliance with other educational resource companies, we're breaking down the most common barriers to learning. Because we believe that everyone should and can have access to knowledge.

About OpenStax Resources

Customization

Introduction to Computer Science is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 (CC BY NC-SA) license, which means that you can non-commercially distribute, remix, and build upon the content, as long as you provide attribution to OpenStax and its content contributors, and distribute all derivatives under the same license.

Because our books are openly licensed, you are free to use the entire book or select only the sections that are most relevant to the needs of your course. Feel free to remix the content by assigning your students certain chapters and sections in your syllabus, in the order that you prefer. You can even provide a direct link in your syllabus to the sections in the web view of your book.

Instructors also have the option of creating a customized version of their OpenStax book. Visit the Instructor Resources section of your book page on OpenStax.org for more information.

Art Attribution

In *Introduction to Computer Science,* art contains attribution to its title, creator or rights holder, host platform, and license within the caption. For any art that is openly licensed, non-commercial users or organizations may reuse the art as long as they provide the same attribution to its original source. (Commercial entities should contact OpenStax to discuss reuse rights and permissions.) To maximize readability and content flow, some art does not include attribution in the text. If you reuse art from this text that does not have attribution provided, use the following attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license.

Errata

All OpenStax textbooks undergo a rigorous review process. However, like any professional-grade textbook, errors sometimes occur. In addition, the wide range of evidence, standards, practices, data, and legal circumstances in computer science change frequently, and portions of the text may become out of date. Since our books are web-based, we can make updates periodically when deemed pedagogically necessary. If you have a correction to suggest, submit it through the link on your book page on OpenStax.org. Subject matter experts review all errata suggestions. OpenStax is committed to remaining transparent about all updates, so you will also find a list of past and pending errata changes on your book page on OpenStax.org.

Format

You can access this textbook for free in web view or PDF through OpenStax.org, and for a low cost in print. The web view is the recommended format because it is the most accessible—including being WCAG 2.2 AA compliant – and most current. Print versions are available for individual purchase, or they may be ordered through your campus bookstore.

About *Introduction to Computer Science*

Introduction to Computer Science provides a comprehensive foundation in core computer science concepts and principles, aligning with the scope and sequence of most introductory computer science courses. The textbook serves as an engaging entry point for students pursuing diverse fields of study and employment,

including computer science, business, engineering, data science, social sciences, and related disciplines. By addressing a broad learner audience—ranging from computer science majors to non-majors—the book offers a thorough introduction to computational thinking and its applications across multiple domains.

Introduction to Computer Science is designed to be both interactive and practical, focusing on real-world applications that showcase how core computer science concepts can be used to solve complex problems. Students will explore foundational topics, such as algorithms, data structures, computer systems organization, and software development, using an array of engaging, hands-on activities. The textbook integrates meaningful learning experiences through chapter-based scenarios, problem-solving exercises, and project-based assessments that encourage students to apply what they learn in authentic contexts.

Features such as embedded coding exercises, industry insights, and explorations of emerging technology trends provide a holistic approach to learning that extends beyond theory. With a forward-looking perspective, Introduction to Computer Science prepares students to engage with advanced topics in computer science, such as machine learning, cybersecurity, and cloud computing, ensuring they have a solid foundation for continued study and future professional success.

Coverage and Scope

The authors and contributors consulted with other educators and industry professionals from a range of institutions and organizations in order to ensure that *Introduction to Computer Science* meets the diverse needs of both computer science majors and non-majors. The book is structured into five main parts, each focusing on critical areas of the discipline:

- **Part 1: Problem Solving and Algorithms** This section introduces students to the foundations of computer science, focusing on computational thinking, problem-solving techniques, and algorithm design. Topics include data structures, formal properties of algorithms, and algorithmic paradigms. Through practical examples and exercises, students will develop the skills needed to construct and analyze algorithms and understand their applications across various domains.
- **Part 2: Realizations of Algorithms** In this part, students explore how algorithms are realized in hardware and software, starting with low-level programming languages and moving into hardware design and computer systems organization. Students will learn about models of computation, machine-level representation, processor architectures, and memory hierarchy. This foundational understanding enables students to see the connections between abstract algorithms and their physical implementations.
- **Part 3: Designing and Developing Software Solutions** This section covers the principles of software development, high-level programming languages, and data management. Students will learn the fundamentals of software engineering and gain hands-on experience with both relational and non-relational database systems. Emphasis is placed on designing robust software solutions and managing complex data structures, ensuring students are well-prepared for future roles in software development and engineering.
- **Part 4: Building Modern End-to-End Solutions to Business and Social Problems** Students apply their knowledge to design and build web and cloud-native applications. This part includes examples of modern web architectures, responsive design techniques, and cloud-based solutions using PaaS and FaaS technologies. Additionally, students will explore the development of hybrid multi-cloud digital solutions, providing them with experience in addressing complex business and social challenges using modern computing technologies.
- **Part 5: Human-Centered Responsible Computing** The final section delves into the ethical and societal implications of computing. Topics include cybersecurity, governance of cyber resources, and responsible computing practices. Students will learn to navigate the complexities of cybersecurity and governance while considering the broader impacts of technology on society.

Each core concept is designed to build on the previous one, ensuring a coherent learning experience that provides students with a clear view of the field. The book's approach enables students to not only understand

the principles of computer science but also see how they can be applied to address practical, real-world problems.

Pedagogical Foundation and Features

The *Introduction to Computer Science* textbook is designed to engage students through a combination of practical, real-world applications and thought-provoking scenarios that promote critical thinking and a deeper understanding of core concepts. The pedagogical approach is centered on making computer science relevant and accessible for students from diverse backgrounds, whether they are pursuing a degree in computer science or exploring how computational thinking can be applied to their respective fields. To support this vision, the textbook incorporates several key features:

- **Concepts in Practice** features present how computer science concepts are applied in real-world contexts by both professionals and non-professionals. Each box profiles personas and practical applications that demonstrate how core topics, such as algorithms, data management, and software engineering, are utilized across various industries. The purpose is to inspire students—particularly non-majors—by showing them the value of computer science in solving everyday challenges and to foster a greater appreciation for the discipline.
- **Global Issues in Technology** features help students think globally about the societal impact of technology. These boxes highlight how technology affects communities and economies around the world and may introduce topics such as digital equity, environmental sustainability, and global data security. Students are encouraged to consider the broader implications of technological advancements and to think critically about their potential to drive positive change or create new challenges in global contexts.
- **Industry Spotlight** boxes focus on specific industry challenges and how technology progress or application can help solve them. *Industry Spotlight* features introduce students to various sectors—such as healthcare, finance, education, and law—providing a glimpse into how computer science can drive innovation and efficiency. By connecting theoretical concepts to industry-specific problems, these features encourage students to explore the wide-ranging applications of computer science and understand its value across different fields.
- **Link to Learning** features provide a very brief introduction to online resources—videos, interactives, collections, maps, and other engaging resources that are pertinent to students' exploration of the topic at hand.
- **Technology in Everyday Life** features connect computer science principles to students' personal experiences and the world around them. These boxes explore how technology intersects with daily life or current events, making computer science concepts more relatable and relevant. Some features may prompt students to think creatively and propose their own ideas for applying computer science solutions to familiar scenarios.
- **Think It Through** scenarios present students with thought-provoking dilemmas or complex problems related to the use of technology. Students are asked to reflect on ethical questions, problem-solving strategies, and real-world decision-making processes. These features emphasize that not all problems have straightforward answers and encourage students to weigh the pros and cons of different approaches. By navigating these scenarios, students learn to develop judgment skills that are crucial in business and technology environments.

Overall, these features are integrated throughout the material to foster active learning, critical thinking, and an appreciation for the practical applications of computer science. By connecting theory to practice and encouraging students to explore real-world issues, *Introduction to Computer Science* provides a meaningful and supportive learning experience that equips students with the knowledge and skills necessary for success in their academic and professional journeys.

Answers to Questions in the Book

The end-of-chapter Review, Conceptual Questions, Practice Exercises, Problem Sets, Thought Provokers, and

Labs are intended for homework assignments or classroom discussion; thus, student-facing answers are not provided in the book. Answers and sample answers are provided in the Instructor Answer Guide, for instructors to share with students at their discretion, as is standard for such resources.

About the Author

Senior Contributing Author



Senior contributing author: Dr. Jean-Claude Franchitti

Dr. Jean-Claude Franchitti is a Clinical Associate Professor of Computer Science and the Associate Director of Graduate Studies for the CS Master's program in Information Systems at NYU Courant Institute. He earned his M.S. in Electrical Engineering (1985) and his M.S. and PhD. in Computer Science from the University of Colorado at Boulder (1988, 1993). He is the founder and CEO of Archemy, Inc., and has over 40 years of experience in a myriad of industries, and over 30 years of teaching and corporate training experience. He held executive positions in large US-based corporations and leading business technology consulting firms such as Computer Sciences Corporation. He has been involved in many large business technology strategy and modernization projects and has a proven record of delivering large scale business solutions. He was the original designer and developer of jcrew.com and the suite of products now known as IBM InfoSphere DataStage. He also created the Agile Enterprise Architecture Management (AEAM) methodology and a corresponding framework that are patented components of the Archemy business evolution platform. He has developed partnerships with many companies at New York University to incubate new methodologies, transition research into business solutions, help recruit skilled graduates, and increase the companies' visibility. Dr. Franchitti has been a reviewer member on several industry standards committees including OMG, ODMG, and X3H2. Dr. Franchitti taught at CU-Boulder, Denver University, Columbia University, NYU SCPS, before joining NYU Courant Institute in 1997. He has extensive experience with corporate training and developed and delivered training and mentoring programs for the top corporate education providers. He conducted research as part of several NSF- and DARPA-funded research programs.

Dr. Franchitti's teaching and research interests include machine learning, artificial intelligence, data management systems and software engineering with an emphasis on large-scale software architectures and business solutions. He has published articles in numerous refereed publications including the Proceedings of Third Int. Conf. on Cooperative Information Systems, Proceedings of the Sixth International Workshop on Persistent Object Systems, and the 16th International Conference on Software Engineering. Dr. Franchitti received an award for Outstanding Service from NYU's School of Continuing and Professional Studies.

Contributing Authors

Amal Alhosban, University of Michigan–Flint

Mark Buckler, Grand Canyon University

Joanna Gilberti, Archemy, Inc.

Scott Gray, Nashua Community College

Matthew Hertz, University at Buffalo

Andrew Hurd, Empire State University

Kevin Lin, University of Washington

Sai Mukkavilli, Georgia Southwestern State University

Phuc (Brian) Nguyen, UC Irvine

Shahab Tayeb, Fresno State

Zdeněk Troníček, Tarleton State University

Kevin Wortman, Cal State Fullerton

Mohamed Zahran, New York University

Reviewers

Reni Abraham, Houston Community College

Shakil Akhtar, Clayton State University

Kiavash Bahreini, Florida International University

Tammie Bolling, Pellissippi State Community College

Phillip Bradford, UConn

Quiana Bradshaw, Campbellsville University

Christopher Bunton, Austin Community College

Komal Chhibber, South Mountain Community College

Chen-Fu Chiang, SUNY Polytechnic Institute

Gabriel de la Cruz, North Idaho College

Gabriel Ferrer, Hendrix College

David Fogarty, New York University

Yuexing Hao, Massachusetts Institute of Technology

Nazli Hardy, Millersville University

Angela Heath, Baptist Health System

Rania Hodhod, Columbus State University

Benita Hubbard, Southern New Hampshire University

Sumit Jha, Florida International University

Mohamed E. Khalefa, SUNY Old Westbury

Steven Ko, Simon Fraser University

Jessica Kong, University of Washington

Alex Krasnok, Florida International University

Blaise Liffick, Millersville University

Sue McCrory, Missouri State University

Morgan McKie, Florida International University

Sandeep Mitra, SUNY Brockport

Mourya Narasareddygari, Rider University

Saty Raghavachary, University of Southern California

Muhammad Rahman, Clayton State University

Amir Rahmati, Stony Brook University

Caryl Rahn, Florida International University

Jerry Reed, Valencia College

Jordan Ringenberg, University of Findlay

Eman Saleh, University of Georgia

Vincent Sanchez, Florida International University

John Schriner, Queensborough Community College

Tiffanie R. Smith, Lincoln University

Hann So, De Anza College

Jayesh Soni, Florida International University

Derrick Stevens, Mohawk Valley Community College

Kathleen Tamerlano, Cuyahoga Community College

Chintan Thakkar, Rasmussen University

Jingnan Xie, Millersville University

Ning Xie, Florida International University

Additional Resources

Student and Instructor Resources

We have compiled additional resources for both students and instructors, including Getting Started Guides, an instructor's answer guide, test bank, and image slides. Instructor resources require a verified instructor account, which you can apply for when you log in or create your account on OpenStax.org. Take advantage of these resources to supplement your OpenStax book.

Instructor's answer guide. Each component of the instructor's guide is designed to provide maximum guidance for delivering the content in an interesting and dynamic manner.

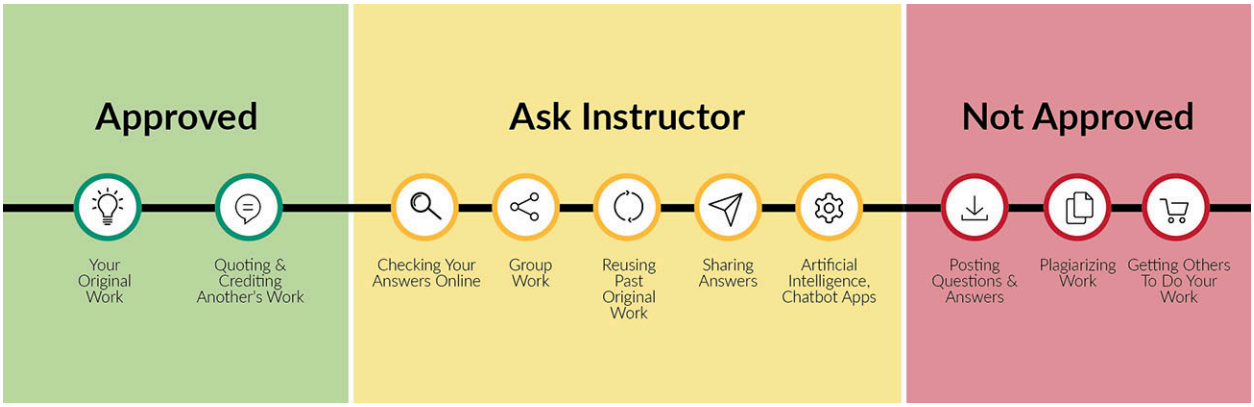
Test bank. With hundreds of assessment items, instructors can customize tests to support a variety of course objectives. The test bank includes review questions (multiple-choice, identification, fill-in-the-blank, true/false), short answer questions, and long answer questions to assess students on a variety of levels. The test bank is available in Word format.

PowerPoint lecture slides. The PowerPoint slides provide learning objectives, images and descriptions, feature focuses, and discussion questions as a starting place for instructors to build their lectures.

Academic Integrity

Academic integrity builds trust, understanding, equity, and genuine learning. While students may encounter significant challenges in their courses and their lives, doing their own work and maintaining a high degree of authenticity will result in meaningful outcomes that will extend far beyond their college career. Faculty, administrators, resource providers, and students should work together to maintain a fair and positive experience.

We realize that students benefit when academic integrity ground rules are established early in the course. To that end, OpenStax has created an interactive to aid with academic integrity discussions in your course.



Visit our academic integrity slider (https://view.genial.ly/61e08a7af6db870d591078c1/interactive-image-defining-academic-integrity-interactive-slider). Click and drag icons along the continuum to align these practices with your institution and course policies. You may then include the graphic on your syllabus, present it in your first course meeting, or create a handout for students. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

At OpenStax we are also developing resources supporting authentic learning experiences and assessment. Please visit this book's page for updates. For an in-depth review of academic integrity strategies, we highly recommend visiting the International Center of Academic Integrity (ICAI) website at https://academicintegrity.org/.

Community Hubs

OpenStax partners with the Institute for the Study of Knowledge Management in Education (ISKME) to offer Community Hubs on OER Commons—a platform for instructors to share community-created resources that support OpenStax books, free of charge. Through our Community Hubs, instructors can upload their own materials or download resources to use in their own courses, including additional ancillaries, teaching material, multimedia, and relevant course content. We encourage instructors to join the hubs for the subjects most relevant to your teaching and research as an opportunity both to enrich your courses and to engage with other faculty. To reach the Community Hubs, visit www.oercommons.org/hubs/openstax.

Technology partners

As allies in making high-quality learning materials accessible, our technology partners offer optional low-cost tools that are integrated with OpenStax books. To access the technology options for your text, visit your book page on OpenStax.org.



1 Introduction to Computer Science

Figure 1.1 Computing is everywhere, affecting everyone, for better and for worse. (credit: modification of "Whereas design is expansive, engineering is narrowing" by Jessie Huynh/Critically Conscious Computing, CC0)

Chapter Outline

1.1 Computer Science
1.2 Computer Science across the Disciplines
1.3 Computer Science and the Future of Society

Introduction

This textbook will introduce you to the exciting and complex world of computer science. In this chapter, you'll review the history of computer science, learn about its use in different fields, and explore how computer science will impact the future of society. Computer science is a powerful tool, and computer scientists have used their vast knowledge of technology to create and implement technology that has transformed societies around the world.

This book will also introduce the computational thinking aspects of problem-solving and analytical thinking that enable the study of algorithms, which are step-by-step instructions for solving specific problems or carrying out computations. Therefore, this book also covers algorithms and their realization via programming languages, computer systems architectures, networks, and operating systems. The book subsequently delves into computer science areas that enable the design and development of software solutions using high-level programming languages (i.e., coding languages designed to be more intuitive for humans), architectural styles and related models, data management systems, and software engineering. Finally, the book demonstrates how to leverage computer science realizations and areas to build modern end-to-end solutions to business and social problems. In particular, the book focuses on modern web applications development, cloud-native applications development, and hybrid Cloud/on-premise digital solutions. The various chapters emphasize how to achieve software solution qualities such as performance and scalability. The last chapter explains how to secure software applications and their applications in the context of various cyber threats. It also explains how to make the right decisions about using computers and information in society to navigate social, ethical, economic, and political issues that could result from the misuse of technology. To conclude this textbook, we'll introduce you to cybersecurity and help you understand why responsible computing is essential to promote

ethical behavior in computer science. The book is designed to help students grasp the full meaning of computer science as a tool that can help them think, build meaningful solutions to complex problems, and motivate their careers in information technology (IT).

You're already familiar with computer science. Whenever you use a laptop, tablet, cell phone, credit card reader, and other technology, you interact with items made possible by computer science. Computer science is a challenging field, and the outputs of computer science offer many benefits for society. At the same time, we have to be cautious about how we use computer science to ensure it impacts society in ethical ways. To help you understand this, the next section will explain how computer science came to be and discuss the field's potential.

1.1 Computer Science

Learning Objectives

By the end of this section, you will be able to:

- Discuss the history that led to the creation of computer science as a field
- Define computer science
- Assess what computer science can do, as well as what it should not do

The field of **computer science (CS)** is the study of **computing**, which includes all phenomena related to computers, such as the Internet. With foundations in engineering and mathematics, computer science focuses on studying algorithms. An **algorithm** is a sequence of precise instructions that enables computing. This includes components computers use to process information. By studying and applying algorithms, computer science creates applications and solutions that impact all areas of society. For example, computer science developed the programs that enable online shopping, texting with friends, streaming music, and other technological processes.

While computers are common today, they weren't always this pervasive. For those whose lives have been shaped by computer technology, it can sometimes seem like computer technology is ahistorical: computing often focuses on rapid innovation and improvement, wasting no time looking back and reflecting on the past. Yet the foundations of computer science defined over 50, and as much as 100, years ago very much shape what is possible with computing today.

The Early History of Computing

The first computing devices were not at all like the computers we know today. They were physical calculation devices such as the abacus, which first appeared in many societies across the world thousands of years ago. They allowed people to tally, count, or add numbers (Figure 1.2). Today, abaci are still used in some situations, such as helping small children learn basic arithmetic, keeping score in games, and as a calculating tool for people with visual impairments. However, abaci are not common today because of the invention of number systems such as the Arabic number system (0, 1, 2, 3, . . .), which included zero and place values that cannot be computed with abaci. The concept of an algorithm was also invented around this time. Algorithms use inputs and a finite number of steps to carry out arithmetic operations like addition, subtraction, multiplication, and division, and produce outputs used in computing. Today's computers still rely on the same foundations of numbers, calculations, and algorithms, except at the scale of billions of numbers and billions of calculations per second.

To introduce a concrete example of an algorithm, let us consider binary search algorithm, which is used to locate a number in a sorted array of integers efficiently. The algorithm operates by repeatedly dividing the search interval in half to perform the search. If the number being searched is less than the integer in the middle of the interval, the interval is narrowed to the lower half. In the alternative, the interval is narrowed to the upper half. The algorithm repeatedly checks until the number is found or the interval is empty.

Algorithms may sound complicated, but they can be quite simple. For example, recipes to prepare food are algorithms with precise directions for ingredient amounts, the process to combine these, and the temperatures and cooking methods needed to transform the combined ingredients into a specific dish. The dish is the output produced by following the algorithm of a recipe.

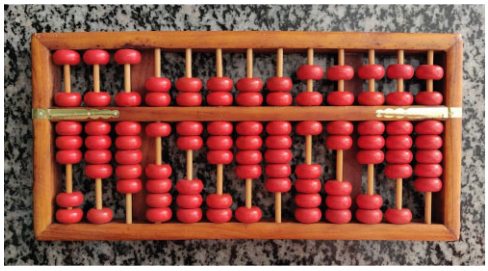


Figure 1.2 An abacus is one of the first calculators. (credit: "Traditional Chinese abacus illustrating the suspended bead use" by Jccsvq/Wikimedia Commons, CC0)

The next major development in the evolution of computing occurred in 1614 when John Napier, a Scottish mathematician, developed logarithms, which express exponents by denoting the power that a number must be raised to obtain another value. Logarithms provided a shortcut for making tedious calculations and became the foundation for multiple analog calculating machines invented during the 1600s.

Scientists continued to explore different ways to speed up or automate calculations. In the 1820s, English mathematician Charles Babbage invented the Difference Engine with the goal of preventing human errors in manual calculations. The Difference Engine provided a means to automate the calculations of polynomial functions and astronomical calculations.

Babbage followed the Difference Engine with his invention of the Analytical Engine. With assistance from Ada Lovelace, the Analytical Engine was program-controlled and included features like an integrated memory and an arithmetic logic unit. Lovelace used punched cards to create sequencing instructions that could be read by the Analytical Engine to automatically perform any calculation included in the programming code. With her work on the Analytical Engine, Lovelace became the world's first computer programmer.

The next major development in computing occurred in the late 1800s when Herman Hollerith, an employee of the U.S. Census Office, developed a machine that could punch cards and count them. In 1890, Hollerith's invention was used to tabulate and prepare statistics for the U.S. census.

By the end of the 1800s and leading into the early 1900s, calculators, adding machines, typewriters, and related machines became more commonplace, setting the stage for the invention of the computer. In the 1940s, multiple computers became available, including IBM's Harvard Mark 1. These were the forerunners to the advent of the digital computer in the 1950s, which changed everything and evolved into the computers and related technology we have today.

Around this time, computer science emerged as an academic discipline rooted in the principles of mathematics, situated primarily in elite institutions, and funded by demand from the military for use in missile guidance systems, airplanes, and other military applications. As computers could execute programs faster than humans, computer science replaced human-powered calculation with computer-powered problem-solving methods. In this way, the earliest academic computer scientists envisioned computer science as a discipline that was far more intellectual and cognitive compared to the manual calculation work that preceded it.

Richard Bellman was a significant contributor to this effort. A mathematics professor at Princeton and later at Stanford in the 1940s, Bellman later went to work for the Rand Corporation, where he studied the theory of multistage decision processes. In 1953, Bellman invented dynamic programming,[1] which is a mathematical optimization methodology and a technique for computer programming. With dynamic programming, complex problems are divided into more manageable subproblems. Each subproblem is solved, and the results are stored, ultimately resulting in a solution to the overall complex problem.[2] With this approach, Bellman helped revolutionize computer programming and enable computer science to become a robust field.

What Is Computer Science?

The term *computer science* was popularized by George E. Forsythe in 1961. A mathematician who founded Stanford University's computer science department, Forsythe defined computer science as "the theory of programming, numerical analysis, data processing, and the design of computer systems." He also argued that computer science was distinguished from other disciplines by the emphasis on algorithms, which are essential for effective computer programming.[3]

Computer science is not only about the study of how computers work, but also everything surrounding computers, including the people who design computers, the people who write programs that run on computers, the people who test the programs to ensure correctness, and the people who are directly and indirectly affected by computers. In this way, computer science is as much about people and how they work with computers as it is about just computers.

Not everyone agrees with this definition. Some people argue that computer science is more about computers or software than the people it affects. However, even if we were to study just the "things" of computer science, the people are still there. When someone designs a computer system, they are thinking about what kinds of programs people might want to run. Typically, effort is made to design the computer system so it is more efficient at running certain kinds of programs. A computer optimized for calculating missile trajectories, for example, won't be optimized for running social media apps.

Many computing innovations were initially developed for military research and communication purposes, including the predecessor to the Internet, the ARPANET (Figure 1.3).

1 S. Golomb, "Richard E. Bellman 1920–1984," n.d. https://www.nae.edu/189177/RICHARD-E-BELLMAN-19201984
2 Geeks for Geeks, "Dynamic Programming or DP," 2024. https://www.geeksforgeeks.org/dynamic-programming/
3 D. E. Knuth, "George Forsythe and the Development of Computer Science," *Communications of the ACM*, vol. 15, no.8, pp. 722–723. 1972. https://dl.acm.org/doi/pdf/10.1145/361532.361538



Figure 1.3 The ARPANET, circa 1974, was an early predecessor to the Internet. It allowed computers at Pentagon-funded research facilities to communicate over phone lines. (credit: modification of "Arpanet 1974" by Yngvar/Wikipedia, Public Domain)

What Is a Computer?

While computer science is about much more than just computers, it helps to know a bit more about computers because they are an important component of computer science. All computers are made of physical, real-world material that we refer to as **hardware**. Hardware—which has four components, including processor, memory, network, and storage—is the computer component that enables computations. The **processor** can be regarded as the computer's "brain," as it follows instructions from algorithms and processes data. The **memory** is a means of addressing information in a computer by storing it in consistent locations, while the **network** refers to the various technological devices that are connected and share information. The hardware and physical components of a computer that permanently house a computer's data are called **storage**.

One way to understand computers is from a hardware perspective: computers leverage digital electronics and the physics of materials used to develop transistors. For example, many of today's computers rely on the physical properties of a brittle, crystalline metalloid called silicon, which makes it suitable for representing information. The batteries that power many of today's smartphones and mobile devices rely on lithium, a soft, silvery metal mostly harvested from minerals in Australia, Zimbabwe, and Brazil, as well as from continental brine deposits in Chile, Argentina, and Bolivia. Computer engineers combine these substances to build circuitry and information pathways at the microscopic scale to form the physical basis for modern computers.

However, the physical basis of computers was not always silicon. The Electronic Numerical Integrator and Computer (ENIAC) was completed in 1945, making it one of the earliest digital computers. The ENIAC operated on different physical principles. Instead of silicon, the ENIAC used the technology of a **vacuum tube**, a physical device like a light bulb that was used as memory in early digital computers. When the "light" in the vacuum tube is off, the vacuum tube represents the number 0. When the "light" is on, the vacuum tube represents the number 1. When thousands of vacuum tubes are combined in a logical way, we suddenly have memory. The ENIAC is notable in computer history because it was the first general-purpose computer, meaning that it could run not just a single program but rather any program specified by a programmer. The ENIAC was often run and programmed by women programmers (Figure 1.4). Despite its age and differences in hardware properties, it shares a fundamental and surprising similarity with modern computers. Anything that can be computed on

today's computers can also be computed by the ENIAC given the right circumstances—just trillions of times more slowly.

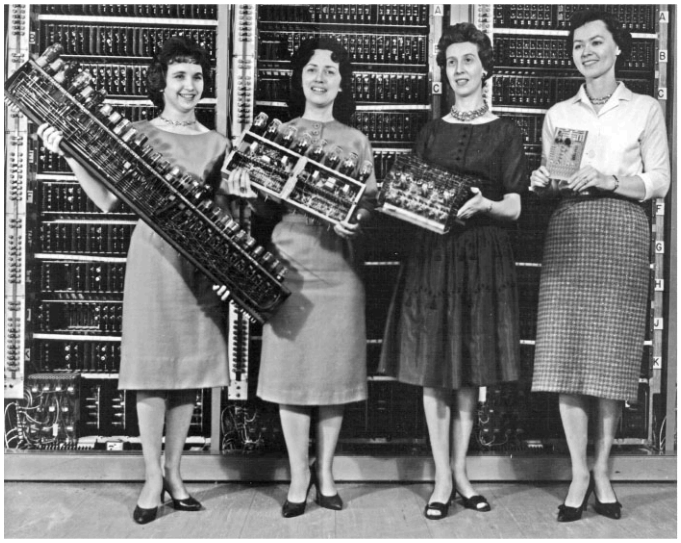


Figure 1.4 This image depicts women programmers holding boards used in computers such as the ENIAC, many of which were designed expressly for ballistics and ordinance guidance research. Today, these room-size computers can be reproduced at a literally microscopic scale—basically invisible to the human eye. (credit: modification of "Women holding parts of the first four Army computers" by U.S. Army/Wikimedia Commons, Public Domain)

How is this possible? The algorithmic principles that determine how results are computed makes up **software**. Almost all computers, from the ENIAC to today's computers, are considered **Turing-complete** (or Computationally Universal, as opposed to specialized computing devices such as scientific calculators) because they share the same fundamental model for computing results and every computer has the ability to run any algorithm. Alan Mathison Turing was an English mathematician who was highly influential in the development of theoretical computer science, which focuses on the mathematical processes behind software, and provided a formalization of the concepts of algorithm and computation with the Turing machine. A Turing-complete computer stores data in memory (either using vacuum tubes or silicon) and manipulates that data according to a **computer program**, which is an algorithm that can be run on a computer. These programs are represented using symbols and instructions written in a **programming language** consisting of symbols and instructions that can be interpreted by the computer. Programs are also stored in memory, which allows programmers to modify and improve programs by changing the instructions.

While both hardware and software are important to the practical operation of computers, computer science's historical roots in mathematics also emphasize a third perspective. Whereas software focuses on the program details for solving problems with computers, theoretical computer science focuses on the mathematical processes behind software. The idea of Turing-completeness is a foundational concept in theoretical computer science, which considers how computers in general—not just the ENIAC or today's computers, but even tomorrow's computers that we haven't yet invented—can solve problems. This theoretical perspective expands computer science knowledge by contributing ideas about (1) whether a problem can be computed by a Turing-complete computer at all, (2) how that problem might be computed using an algorithm, and (3) how quickly or efficiently a computer can run such an algorithm. The answers to these questions suggest the limits of what we can achieve with computers from a technical perspective: Using mathematical ideas, is it possible to use a computer to compute all problems? If the answer to a problem is yes, how much of a computing resource is needed to get the answer?

Clearly both humans and computers have their strengths and limitations. An example of a problem that humans can solve but computers struggle with is interpreting subtle emotions or making moral judgments in complex social situations. While computers can process data and recognize patterns, they cannot fully understand the nuances of human emotions or ethics, which often involve context, empathy, and experience.

On the flip side, there are tasks that neither computers nor humans can perform, such as accurately predicting chaotic systems like long-term weather patterns. Despite advancements in **artificial intelligence (AI)**, computer functions that perform tasks, such as visual perception and decision-making processes that usually are performed by human intelligence, these problems remain beyond our collective reach due to the inherent unpredictability and complexity of certain natural systems.

Theoretical computer science is often emphasized in undergraduate computer science programs because academic computer science emerged from mathematics, often to the detriment of perspectives that center on the social and technical values embodied by applications of computer technology. These perspectives, however, are gradually changing. Just as the design of ARPANET shaped the design of the Internet, computer scientists are also learning that the physical aspects of computer hardware determine what can be efficiently computed. For example, many of today's artificial intelligence technologies rely on highly specialized computer hardware that is fundamentally different at the physical level compared to the general-purpose programmable silicon that has been the traditional focus of computer science. Organizations that develop **human-computer interaction (HCI)**, a subfield of computer science that emphasizes the social aspects of computation, now host annual conferences that bring together thousands of researchers in academia and professionals in the industry. Computer science education is another subfield that emphasizes the cognitive, social, and communal aspects of learning computer science. Although these human-centered subfields are not yet in every computer science department, their increasing representation reflects computer scientists' growing desire to serve not only more engaged students, but also a more engaged public in making sense of the values of computer technologies.

The Capabilities and Limitations of Computer Science

Computers can be understood as sources, tools, and opportunities for changing social conditions. Many people have used computer science to achieve diverse goals beyond this dominant vision for computer science. For example, consider computers in education.

Around the same time that the ARPANET began development in the late 1960s, Wally Feurzeig, Seymour Papert, and Cynthia Solomon designed the LOGO programming language to enable new kinds of computer-mediated expression and communication. Compared to contemporary programming languages such as FORTRAN (FORmula TRANslation System) that emphasized computation toward scientific and engineering applications, LOGO is well known for its use of turtle graphics, whereby programs were used to control the actions of a digital turtle using instructions such as moving forward some number of units and turning left or right some number of degrees. Papert argued that this turtle programming enabled *body-syntonic reasoning*, a kind of experience that could help students more effectively learn concepts in mathematics such as angles, distance, and geometric shapes by instructing the turtle to draw them, and physics by constructing their own understandings via reasoning through the physical motion of turtle programs by showing concepts of velocity, repeated commands to move forward the same amount; acceleration, by making the turtle move forward in increasing amounts; and even friction, by having the turtle slow down by moving forward by decreasing amounts. In this way, computers could not only be used to further education in computer science, but also offer new, more dynamic ways to learn other subjects. Papert's ideas have been expanded beyond the realm of mathematics and physics to areas such as the social sciences, where interactive data visualization can help students identify interesting correlations and patterns that precipitated social change and turning points in history while also learning new data fluencies and the limits of data-based approaches.[4]

Yet despite these roots in aspirations for computers as a medium for learning anything and everything, the study of computer science education emerged in the 1970s as a field narrowly concerned with producing more effective software engineers. Higher-education computer science faculty, motivated by the demand for

4 B. Naimipour, M. Guzdial, and T. Shreiner. 2019. Helping Social Studies Teachers to Design Learning Experiences Around Data: Participatory Design for New Teacher-Centric Programming Languages. In *Proceedings of the 2019 ACM Conference on International Computing Education Research (ICER '19)*. Association for Computing Machinery, New York, NY, USA, 313. DOI: https://doi.org/10.1145/3291279.3341211

software engineers, designed their computer science curricula to teach the concepts that early computer companies such as IBM desperately needed. These courses had an emphasis on efficiency, performance, and scalability, because a university computer science education was only intended to produce software engineers. We live with the consequences of this design even today: the structure of this textbook inherits the borders between concepts originally imagined in the 1970s when university computer science education was only intended to prepare students for software development jobs. We now know that there are many more roles for computer scientists to play in society—not only software engineers, but also data analysts, product managers, entrepreneurs, political advisors or politicians, environmental engineers, social activists, and scientists across every field from accounting to zoology.

Although the role of computers expanded with the introduction of the Internet in the late 1990s, Papert's vision for computation as a learning medium has been challenging to implement, at least partly because of funding constraints. But as computers evolve, primary and secondary education in the United States is striving for ways to help teachers use computers to more effectively teach all things—not just computers for their own sake, but using computers to learn everything.

Computers and Racial Justice

Our histories so far have centered the interests of White American men in computer science. But there are also countless untold, marginalized histories of people of other backgrounds, races, ethnicities, and genders in computing. The book and movie *Hidden Figures* shares the stories of important Black women who were not only human computers, but also some of the first computer scientists for the early digital computers that powered human spaceflight at NASA (Figure 1.5).



Figure 1.5 Katherine Johnson, a Black computer scientist, recalculated the computations done by early digital computers for space flight planning at NASA. Her contributions were portrayed in the book and movie *Hidden Figures*. (credit: "Katherine Johnson at NASA, in 1966" by NASA/Wikimedia Commons, Public Domain)

In one chapter of *Black Software*, Charlton McIlwain shares stories from "The Vanguard" of Black men and women who made a mark on computer science in its early years from the 1950s through the 1990s through the rise of personal computing and the Internet, but whose histories have largely been erased by the dominant Silicon Valley narratives. Their accomplishments include leading computer stores and developing early Internet social media platforms, news, and blog websites. For example, Roy L. Clay Sr., a member of the Silicon Valley Engineering Hall of Fame, helped Hewlett-Packard develop its first computer lab and create the company's first computers. Later, Clay provided information to venture capitalists that motivated them to invest in start-ups such as Intel and Compaq.[5] In another example, Mark Dean was an engineer for IBM whose work was instrumental in helping IBM develop the Industry Standard Architecture (ISA) bus, which created a method of connecting a computer's processor with other components and enabling them to communicate. This led to the creation of PCs, with Dean owning three of the nine patents used to create the original PC.[6]

Yet their efforts were often hampered by the way that computer science failed to center, or even accommodate, Black people. Historically, American Indians and Hispanic people did not have the same access as even Black Americans to computers and higher education. Kamal Al-Mansour, a technical contract negotiator at the NASA Jet Propulsion Lab, worked on space projects while Ronald Reagan was president. He recounts:

"It was conflicting . . . doing a gig . . . supporting missiles in the sky, (while) trying to find my own identity and culture . . . JPL was somewhat hostile . . . and I would come home each day [thinking] What did I accomplish that benefited people like me? And the answer every day would be 'Nothing.'"[7]

Al-Mansour would go on to start a new company, AfroLink, finding purpose in creating software that centered on Black and African history and culture. This story of computer technologies in service of African American communities is reflected in the creation of the Afronet (an early social media for connecting Black technologists) and the NetNoir (a website that sought to popularize Black culture). These examples serve as early indicators of the ways that Black technologists invented computer technologies for Black people in the United States. Yet *Black Software* also raises challenging political implications of the historical exclusion of Black technologists. Black culture on the Internet has greatly influenced mainstream media and culture in the United States, but these Black cultural products are ultimately driving attention and money to dominant platforms such as X and TikTok rather than those that directly benefit Black people, content creators, and entrepreneurs. Computer technologies risk reproducing social inequities through the ways in which they distribute benefits and harms.

The digital divide has emerged as a significant issue, as many aspects of society -- including education, employment, and social mobility -- become tied to computing, computer science, and connectivity. The divide refers to the uneven and unequal access and distribution of technology across populations from different geographies, socioeconomic statuses, races, ethnicities, and other differentiators. While technological access generally improves over time, communities within the United States and around the world have different levels of access to high-speed Internet, cell towers, and functioning school computers. Unreliable electricity can also play a significant role in computer and Internet usage. And beyond systemic infrastructure-based differences, individual product or service access can create a divide within communities. For example, if powerful AI-based search and optimization tools are only accessible through high-priced subscriptions, specific populations can be limited in benefiting from those tools.

5 J. Dreyfuss, "Blacks in Silicon Valley," 2011. https://www.theroot.com/blacks-in-silicon-valley-1790868140
6 IBMers, "Mark Dean," n.d. https://www.ibm.com/history/mark-dean
7 C. D. McIlwain, (2019). *Black software: The Internet and racial justice, from the AfroNet to Black Lives Matter*, New York: Oxford University Press.

GLOBAL ISSUES IN TECHNOLOGY

H-1B Visas Address Worker Shortages

According to the U.S. Bureau of Labor Statistics (BLS), by 2033, the number of jobs available for computer and information research scientists is expected to increase by 26%. This is much faster job growth than the average expected in total for all occupations. BLS predicts that this will result in about 3,400 job openings per year in technology, including computer science.[8]

To fill some of these jobs, U.S. employers likely will continue to rely on H-1B visas. This visa enables employers to recruit well-educated professionals from other countries. These professionals temporarily reside in the United States and work in specialty occupations, like computer science, that require a minimum education of a bachelor's degree or its equivalent.[9] To participate in the visa program, employers must register and file a petition to hire H-1B visa holders. Each year, the U.S. Citizenship and Immigration Services accepts applications from individuals from other countries who compete for a pool of 65,000 visa numbers, as well as an additional pool of 20,000 master's exemption visa numbers awarded that year and valid for a period of three years. At the end of three years, employers can petition to have each worker's visa extended for a period of three additional years.[10] This program helps U.S. employers fill vacancies in many fields, including computer science while providing job opportunities for highly skilled workers around the world.

Computers and Global Development

Computer technology, like any other cutting-edge technology, changes the balance of power in society. But access to new technologies is rarely ever equal. Computer science has improved the quality of life for many people who have access to computer technology and the means of controlling it to serve their interests. But for everyone else in the world, particularly people living in the Global South, computer technologies need context-sensitive designs to meet their needs. In the 1990s, for instance, consumer access to the Internet was primarily based on "dial-up" systems that ran on top of public telephone network systems. Yet many parts of the world, even today, lack telephone coverage, let alone Internet connectivity. Research in computers for global development aims to improve the quality of life for people all over the world by designing computer solutions for low-income and underserved populations across the world—not just those living in the wealthiest countries.

Computer technologies for global development require designing around unique resource constraints such as a lack of reliable power, limited or nonexistent Internet connectivity, and low literacy. Computer scientists employ a variety of methods drawing from the social sciences to produce effective solutions. However, designing for diverse communities is difficult, particularly when the designers have little direct experience with the people they wish to serve. In *The Charisma Machine*, Morgan Ames criticizes the One Laptop Per Child (OLPC) project, a nonprofit initiative announced in 2005 by the Massachusetts Institute of Technology Media Lab. The project attempted to bring computer technology in the form of small, sturdy, and cheap laptops that were powered by a hand crank to children in the Global South. Based on her fieldwork in Paraguay, Ames argues that the project failed to achieve its goals for a variety of reasons, such as electricity infrastructure problems, hardware reliability issues, software frustrations, and a lack of curricular materials. Ames argues that "charismatic technologies are deceptive: they make both technological adoption and social change appear straightforward instead of as a difficult process fraught with choices and politics." When the computers did

8 U.S. Bureau of Labor Statistics, "Computer and Information Research Scientists: Job Outlook," 2024. https://www.bls.gov/ooh/computer-and-information-technology/computer-and-information-research-scientists.htm#tab-6
9 U.S. Citizenship and Immigration Services, "H-1B Specialty Occupations," 2024. https://www.uscis.gov/working-in-the-united-states/h-1b-specialty-occupations
10 American Immigration Council, "The H-1B Visa Program and Its Impact on the U.S. Economy," 2024. https://www.americanimmigrationcouncil.org/research/h1b-visa-program-fact-sheet

work, OLPC's vision for education never truly materialized because children often used the computers for their own entertainment rather than the learning experiences the designers intended. Though Ames's account of the OLPC project (Figure 1.6) itself has been criticized for presenting an oversimplified narrative, it still represents a valuable argument for the risks and potential pitfalls associated with designing technologies for global development: technology does not act on its own but is embedded in a complicated social context and history.



Figure 1.6 In 2005, MIT's Media Lab started the OLPC initiative to bring laptops to children in the Global South. An unexpected outcome they discovered was that designing technologies for global communities is not as straightforward as designers may initially believe. (credit: "One Laptop per Child" by OLE Nepal Cover/Flickr, CC BY 2.0)

THINK IT THROUGH

Internet Commerce

Many products and companies offer services or products over the Internet. While online shopping provides additional sales opportunities for businesses, while offering consumers a convenient shopping option, it is not without risks. For example, online businesses and their shoppers may be victims of data breaches and identity theft. Other risks include fake reviews that motivate consumers to make a purchase, phishing that leads to hacking, and fake online stores that take consumers' money without delivering a product. What can we do to mitigate the risks and dangers of online shopping?

Addressing these risks is not as simple as practicing humility and including communities in the design process. Many challenges in computing for global development are sociopolitical or technopolitical rather than purely technical. For example, carrying out a pilot test to evaluate the effectiveness of a design can appear as favoritism toward the pilot group participants. These issues and social tensions are especially exacerbated in the Global South, where the legacies of imperialism and racial hierarchies continue to produce or expand social inequities and injustices.

The identities of people creating computer technologies for global development are ultimately just as important as the technologies they create. In *Design Justice*, Sasha Costanza-Chock reiterates the call for computer scientists to "build with, not for," the communities they wish to improve. In this way, *Design Justice* seeks to address the social justice tensions raised when asking the question, "Who does technology ultimately benefit?" by centering the ingenuity of the marginalized "user" rather than the dominant "designer."

In some cases, underdeveloped countries can quickly catch up without spending the money that was invested to develop the original technologies. For example, we can set up ad hoc networks quickly today and at a

portion of the cost in Middle Eastern and African countries using technology that was developed (at a high cost) in the United States and Europe over the past several decades. This means that sometimes, progress in one part of the world can be shared with another part of the world, enabling that area to quickly progress and advance technologically.

LINK TO LEARNING

The Design Justice Network (https://openstax.org/r/76DesignJust) is an organization that aims to advance the principles of design justice and to include people who are marginalized in the technology design process.

1.2 Computer Science across the Disciplines

Learning Objectives

By the end of this section, you will be able to:

- Differentiate between discovery and invention
- Describe how science, mathematics, and engineering each play a role in computer science
- Discuss how data science, computational science, and information science each relate to computer science
- Explain why the various areas of computer science are synergistic

Computer science is an incredibly diverse field not because of what it can achieve on its own but because of how it contributes to every other field of human knowledge and expertise. From its early days, it was understood that there would be cross-collaboration between computer scientists and colleagues in other disciplines. Today, almost all modern technologies either depend on computer technologies or benefit significantly from them. Computer technologies and the study of computer science have reshaped almost all facets of life today for everyone.

Data Science

Across business, financial, governmental, scientific, and nonprofit workplaces, millions of people are programming, and most of the time, they don't even know it! A **spreadsheet** is an example of a data-centric programming environment where data is organized into cells in a table. Instead of presenting programs as primarily about algorithms, spreadsheets present programs as primarily about data. Spreadsheets are often used for data analysis by offering a way to organize, share, and communicate ideas about data. Spreadsheets are uniquely effective and accessible because they allow for the visual organization of data in whichever structure makes the most sense to the user. Instead of hiding data behind code, spreadsheets make data as transparent and up to date as possible.

Although spreadsheets make computation accessible for millions of people across the world, they have several shortcomings. Unless limits are removed, many popular spreadsheet software products such as Microsoft Excel may have a limitation to the number of rows of data they can store that is less than the data of modern computers. One such example occurred in October 2020 when Public Health England failed to report 15,841 positive cases of COVID-19 in the United Kingdom due to mismanaged row limits in the spreadsheet used. This shortcoming attests not only to the technical limit on the number of rows supported by spreadsheets, but also to the design limitations of software that fails to communicate data loss, irregularities, or errors to users. Errors in spreadsheet software data entry can often go unnoticed because spreadsheets do not enforce data types. Cells can contain any content: numbers, currencies, names, percentages, labels, and legends. The meaning of a cell is determined largely by the user rather than the software. Spreadsheets are an expressive and accessible technology for data analysis, but this creative power that spreadsheets afford to users is the

very same power that limits spreadsheets as a data management and large-scale data analysis tool. The more data and the more people involved in a spreadsheet, the greater the potential for spreadsheet problems.

The interdisciplinary field that applies computing to managing data and extracting information from data is called **data science**. Data scientists are practitioners who combine computing and data analysis skills with the domain knowledge specific to their field or business. The demand for data scientists is becoming increasingly important as more and more research and business contexts involve analyzing **big data**, or very large datasets that are not easily processed using spreadsheets. These datasets often involve high-volume measurements of user interactions on the Internet at a very fine grain, such as tracking a customer's web browser history across an online storefront. Data scientists can then analyze browser patterns using machine learning methods in order to recommend related products, target advertisements to specific customers over social media, and reengage customers over email or other means. Machine learning (ML) is a subset of artificial intelligence that relies on algorithms and data to make it possible for artificial intelligence to learn, actually mimicking the way humans learn. For example, ML is used to identify fraudulent versus legitimate banking transactions. Once a computer learns how to distinguish fraudulent transactions, it can be alert and call attention to suspicious banking activity.

GLOBAL ISSUES IN TECHNOLOGY

Targeted Advertising

Although data scientists can produce immense value for business and research alike, their work also raises significant social concerns. For example, web browser history tracking enables companies to target advertising of products to people and also allows for targeting of political advertisements. In *Antisocial Media*, Siva Vaidhyanathan argues that the "impact of Facebook on democracy is corrosive [because political campaigns] can issue small, cheap advertisements via platforms like Facebook and Instagram that may target "groups as small as twenty, and then disappear, so they are never examined or debated." This undermines the process of discussions among voters in democracies like the United States, as well as countries like Germany, United Kingdom, and Spain, which spent the most on targeted political advertising on Facebook in the spring of 2019.[11] Given their lack of transparency, such ads are a questionable practice.

Another interesting topic worth mentioning here is targeted advertising toward children.[12] It is important to consider the ethical implications of using the data collected through tracking, especially when it comes to targeting at-risk populations. It raises questions about the accountability of platforms and advertisers in safeguarding users' rights and ensuring transparency in how data is used for these purposes.

Computational Science

Beyond data science, computer science can also fundamentally change how science is researched and developed. The field of **computational science** refers to the application of computing concepts and technologies to advance scientific research and practical applications of scientific knowledge in a wide range of fields, including civil engineering, finance, and medicine (among many others). For example, algorithms and computer software play a key role in enabling numerical weather prediction (Figure 1.7) or the use of mathematical models to forecast weather based on current conditions in order to assist peoples' everyday lives and contribute to our understanding of the climate models, climate changes, and climate catastrophes. These algorithms may rely on a large amount of computer hardware power that might not be available in a single system, so the work may need to be distributed across many computers. Computational science studies methods for realizing these algorithms and computer software.

11 Statista, "European Elections: Countries that spent the most on targeted political advertising on Facebook from March 1 to May 26, 2019*," 2019. https://www.statista.com/statistics/1037329/targeted-political-ad-spend-on-facebook-by-eu-countries/
12 Maya Brownstein. Harvard study is first to estimate annual ad revenue attributable to young users of these platforms. January 2, 2024. https://news.harvard.edu/gazette/story/2024/01/social-media-platforms-make-11b-in-ad-revenue-from-u-s-teens/

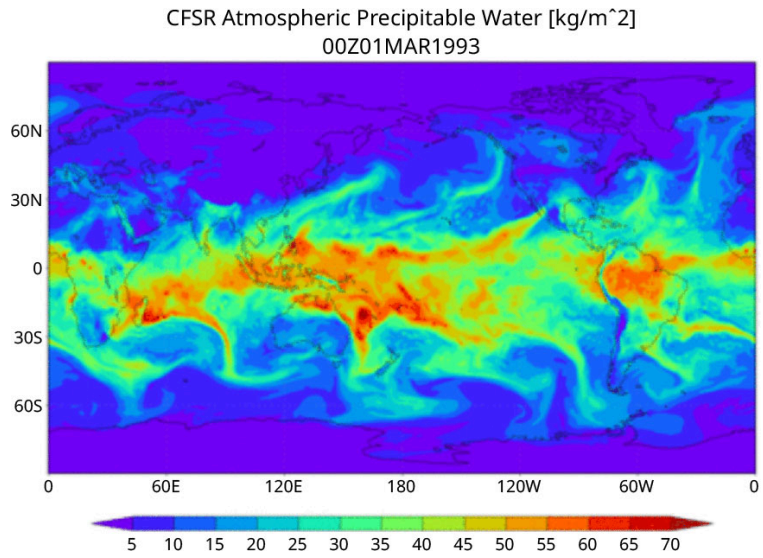


Figure 1.7 Meteorologists collect data from a variety of sources and use the data, algorithms, and computers to predict the weather. (data source: Climate Forecast System, National Centers for Environmental Information, National Oceanic and Atmospheric Administration, https://www.ncei.noaa.gov/products/weather-climate-models/climate-forecast-system; credit: modification of "CFSR Atmospheric Precipitable Water" by NOAA/ncei.noaa.gov, Public Domain)

INDUSTRY SPOTLIGHT

Computer Science and Climate Change

Computer science fights climate change and limits the impacts of climate catastrophes by enabling technologies for decarbonization through power consumption optimization and advancing renewable energy sources. Numerical weather forecasting not only supports our everyday lives but also helps climate scientists determine the precise locations for wind turbines and simulate how they should be designed to enable the greatest energy production. To support the power grid, data science methods can help predict peak power consumption, optimize power sources to produce exactly the right amount of power needed, and adjust power storage to reduce the amount of energy that needs to be generated from nonrenewable sources. Computer models and algorithms assist energy engineers in optimizing building air conditioning and power demands so that they efficiently serve the people living, working, and playing within them.

Although computer science has been used to support scientific discovery, the theory of knowledge of computer science has historically been considered quite different from that of the natural sciences, such as biology, physics, and chemistry. Computer science does not study natural objects, so to most, it would not be considered a natural science but rather an applied science. Unlike natural sciences such as biology, physics, and chemistry, which emphasize the *discovery* of natural phenomena, computer science often emphasizes *invention* or *engineering*.

However, computer science is today deeply interdisciplinary and involves methods from across science, mathematics, and engineering. Computer scientists design, analyze, and evaluate computational structures, systems, and processes.

- Mathematics plays a key role in theoretical computer science, which emphasizes how a computational problem can be defined in mathematical terms and whether that mathematical problem can be efficiently solved with a computer.
- Engineering plays a key role in **software engineering**, which emphasizes how problems can be solved with computers as well as the practices and processes that can help people design more effective software solutions.

- Science plays a key role in human-computer interaction, which emphasizes experimentation and evaluation of the interface (boundary) between humans and computers, often toward designing better computer systems.

Information Science

Not only is computation interdisciplinary, but other disciplines are also becoming more and more computational. In *The Invisible Future*, Nobel Laureate biologist David Baltimore defines DNA in computational terms. He states that biology is an information science because DNA encodes for the outputs of biological systems. The interdisciplinary field studying information technologies and systems as they relate to people, organizations, and societies is called **information science**. The role of information in natural sciences can also be found in the physics of quantum waves that carry information about physical effects, in the chemical equations that specify information about chemical reactions, in the information flows that drive the evolution of economies and political organizations, and in the information processes underlying social, management, and communication sciences.[13]

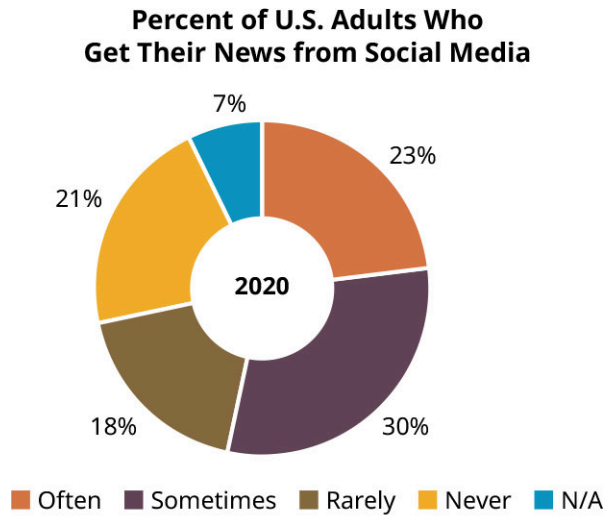
CONCEPTS IN PRACTICE

Computer Science and DNA

Research into DNA sequencing and indexing is opening new ways of helping medical providers offer personalized treatments for patients. Large-scale genome sequencing of not only the human genome, but also the DNA signatures for viruses has made it possible for medical providers to take human fluid samples and analyze them for the presence of infectious diseases. This research requires computer science concepts, including specialized medical computer devices to sequence the billions of nucleotides that form a DNA sequence, data structures and algorithms to efficiently process and identify DNA signatures, and the miniaturization of computer hardware so that this technology is accessible (both in terms of price and physical size) in more and more care centers.

Although information science has its roots in information classification, categorization, and management in the context of library systems, information science today is a broad field that encompasses the many diverse ways information shapes society. For example, today's social media networks provide more personable and instantaneous information communication compared to traditional news outlets—billions of people around the world are using social media to engage with information about the world. For many people, social media may be the primary way that they learn about and make sense of the world (Figure 1.8). Yet, we've already seen risks associated with information technologies such as the Internet. In today's "information age," information has more power than ever before to reshape society. Information scientists, data scientists, computational scientists—and, therefore, computer scientists—have a social responsibility: "One cannot reap the reward when things go right but downplay the responsibility when things go wrong."[14]

13 P. J. Denning, "Computing is a natural science," *Commun. ACM*, vol. 50, no. 7, pp. 13–18, July 2007. https://doi.org/10.1145/1272516.1272529.
14 R. Benjamin, "Race after technology: Abolitionist tools for the new Jim code," 2019, Polity.



Source: Survey of U.S. adults conducted Aug. 31-Sept. 7, 2020.
"News Use Across Social Media Platforms in 2020"

Figure 1.8 While Americans used to primarily get their news from newspapers, as technology has advanced, their primary source of news media has shifted. As of 2020, 53% of American adults surveyed stated they got their news from social media at least some of the time. (data source: Elisa Shearer and Amy Mitchell, Pew Research Center. "About Half of Americans Get News on Social Media at Least Sometimes." From Survey of U.S. adults conducted Aug. 31–Sept. 7, 2020. In: E. Shearer, A. Mitchell, "News Use Across Social Media Platforms in 2020," Jan 12, 2021.; attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Despite the centrality of information to decision-making and social change, dominant approaches to computer science tend to focus on computational structures, systems, and processes (such as algorithms) that describe one kind of information by focusing on the *what* or *how* of solving problems with computers, but less often the *why* or *who* questions. Information science broadly centers people, organizations, and society in the study of information technologies.

Computer Science Is an Interdisciplinary Field

In presenting data science, computational science, and information science, we've introduced the idea that computer science can shape other disciplines. But we've also raised questions about what computer science is today. If computer science is the study of all "phenomena surrounding computers," it could also involve data science, computational science, bioinformatics, cheminformatics, computational social science, medical informatics, and information science. As you will learn in Chapter 13 Hybrid Multicloud Digital Solutions Development, another aspect of computer science is responsible computing, which includes the appropriate management of cyber resources as well as robust cybersecurity. It is difficult to define computer science today because it is so widely used by people across the world in diverse capacities. Definitions are about defining boundaries and excluding practices, which may be helpful for understanding the practices of a certain culture or group that is "doing" computer science, but it can never truly represent everyone and all the things that people are doing with computer science. However, computer science's historical roots in mathematics shape the way it categorizes subfields:

- Theoretical computer science
  - Theory of computation
  - Information representation
  - Data structures and algorithms
  - Programming language and formal methods
- Computer systems
  - Architecture
  - Artificial intelligence
  - Networks

- Security
- Databases
- Distributed computing
- Graphics

- Applied computer science
  - Scientific computing
  - Human-computer interaction
  - Software engineering

In this hierarchy, theoretical computer science and computer systems are treated separately from applied computer science and human-computer interaction, suggesting that the mathematics of computing are pure and separate from social questions. Yet we've seen several examples that question this paradigm and instead point to a structure where human-computer interaction is infused throughout the study of computer science and all its subfields.

Today, computer science is a field that is just as much about people as it is about computer technology because each of these subfields is motivated by the real-world problems that people ultimately want to solve. The subfields of artificial intelligence and machine learning have applications that directly influence human decision-making, ranging from advertisement targeting to language translation to self-driving cars. Effective computational solutions to research or business problems require combining specific knowledge with computer science concepts from a combination of areas. For example, the computational science application of weather prediction combines knowledge about various subfields of computer science (algorithms, distributed computing, computer systems) with knowledge about climate systems. Theoretical computer scientists are increasingly interested in asking questions such as, "How do we design, analyze, and evaluate algorithms or information systems for fairness? How do we even define fairness in a computer system that strips away the complexities of the real world? What ideas or information are encoded in the data? And what are the limits of our approaches?" Computer science is a complex field, and its synergistic nature means that when computer science is used in an interdisciplinary manner that shapes other disciplines, its impact on society is much greater than when each discipline functions on its own.

1.3 Computer Science and the Future of Society

Learning Objectives

By the end of this section, you will be able to:

- Discuss how computer scientists develop foundational technologies
- Discuss how computer scientists evaluate the negative consequences of technologies
- Discuss how computer scientists design technologies for social good

As noted earlier, computer science is a powerful tool, and computer scientists have vast technological knowledge that continues transforming society. Computer scientists have an obligation to be ethical and good stewards of technology with an emphasis on responsible computing. Written code influences daily life, from what we see on social media to the news stories that pop up in a Google search and even who may or may not receive a job interview. When computer scientists don't consider the ramifications of their code, there can be unintended consequences for people around the world. The Y2K problem, also known as the "millennium bug," is a good example of shortsighted decisions that allowed computer scientists to only store the last two digits of the year instead of four. This made sense at a time when memory was expensive on both mainframe computers and early versions of personal computers. The Y2K problem was subsequently coined by John Hamre, the United States Deputy Secretary of Defense, as the "electronic equivalent of the El Niño."[15] The future of computer science will highly affect the future of the world. Although we often think of computer technologies as changing the way the world works, it is actually people and their vision for the future that are amplified by computing. The relationship between computer science and people is about how computer technologies can bias society and how the choices made through computer systems can both promote and discourage social inequities. Computer technologies can encode either value or both values in their designs. In this section, we'll introduce three ways that computer science can shape the future of society: developing foundational technologies, evaluating negative consequences of technologies, and designing technologies for social good.

Developing Foundational Technologies

We've seen how foundational technologies like artificial intelligence, algorithms, and mathematical models enable important applications in data science, computational science, and information science.

As noted previously, artificial intelligence (AI) is the development of computer functions to perform tasks, such as visual perception and decision-making processes, that usually are performed by human intelligence. AI refers to a subfield of CS that is interested in solving problems that require the application of machine learning to human cognitive processes to achieve goals. AI research seeks to develop algorithm architectures that can make progress toward solving problems. One such example is **image recognition**, or the problem of identifying objects in an image. This problem is quite difficult for programmers to solve using traditional programming methods. Imagine having to define very precise rules or instructions that could identify an object in an image regardless of its position, size, lighting conditions, or perspective. As humans, we have an intuitive sense of the qualities of an object. However, representing this human intelligence in a machine requiring strict rules or instructions is a much harder task. AI methods for image recognition involve designing algorithm architectures that can generalize across all the possible ways that an object can appear in an image.

INDUSTRY SPOTLIGHT

Agricultural Robots

Agricultural robots help large-scale industrial farmers produce crops more efficiently and support sustainability efforts. One agricultural robot is now being used to improve fertilizer and pesticide treatments by taking pictures of plants as a farmer drives a tractor over the field. Artificial intelligence techniques are used to recognize and identify the lettuce plants and weed plants in the image. For each identified lettuce or weed plant, the robot makes a personalized decision about the best chemical treatment for the plant in real time as the tractor moves to the next row of crops. This ability to personalize chemical treatments improves yields and plant quality for large-scale industrial agriculture by producing more crops with fewer chemicals.

15 "Looking at the Y2K bug," portal on CNN.com. Archived 7 February 2006 at the Wayback Machine. https://web.archive.org/web/20060207191845/http://www.cnn.com/TECH/specials/y2k/

Recent approaches to AI for image recognition draw on a family of methods called neural networks instead of having programmers craft rules or instructions by hand to form an algorithm. In humans, the neural network is a complex network in the human brain that consists of neurons, or nerve cells, connected by synapses that send messages and electrical signals to all parts of the body, enabling us to think, move, feel, and function.

In computer science, a **neural network** (Figure 1.9) is an AI algorithm architecture that emphasizes connections between artificial nerve cells whose behavior and values change in response to stimulus or input. These neural networks are not defined by individual neurons but by the combination of all the neurons in the network. Typically, artificial neurons are arranged in a hierarchy that aims to capture the structure of an image. Although the first level of neurons might respond to individual pixels, later levels of artificial neurons might respond in aggregate to the arrangement of several artificial neurons in the preceding layer. This is similar to how the human visual system responds to edges at the lower levels, then responds in aggregate to the specific arrangement of several edges in later levels, and ultimately identifies these aggregated arrangements as objects.

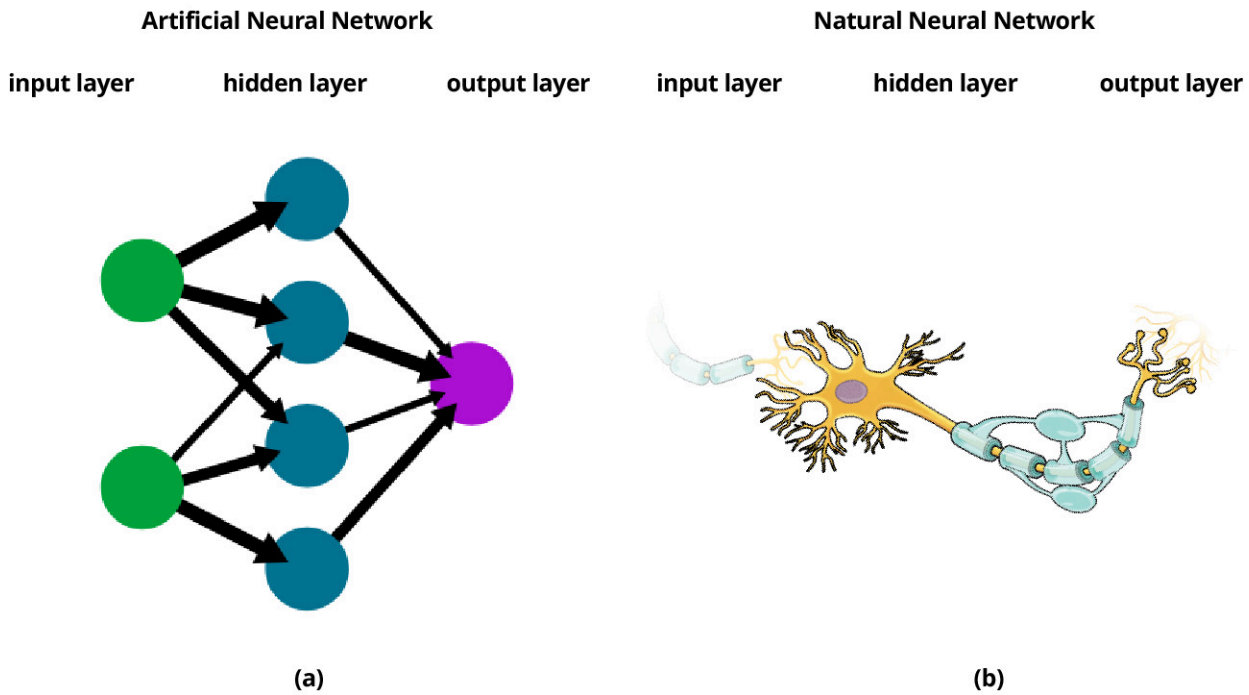


Figure 1.9 (a) An artificial neural network consists of three key layers: the input layer, where raw data enters the system; the hidden layer, where information is processed and patterns are identified; and the output layer, where results are presented. (b) A natural neural network, such as those in the human body, mirrors this structure. The input layer represents sensory receptors, like those in the retina. The hidden layer corresponds to the synapse, where partial processing of the sensory data occurs. Finally, the output layer represents the information sent to the brain for final processing and interpretation. (credit a: modification of "Neural network example" by Wiso/Wikipedia, Public Domain; credit b: modification of work from *Psychology 2e*. attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The idea of neural networks, however, is not as new as it might seem. Artificial neural networks were first imagined in the mid-1900s alongside contemporary research efforts in the cognitive sciences. The ideas of multilayered, hierarchical networks of neurons and the mathematical optimization methods for learning were all there, but these early efforts were limited by the computational processing power available at the time. In addition, the large datasets that drive neural network learning were not nearly as available as they are today with the Internet. Developments in foundational technologies such as computer architecture and computer networks paved the way for the more recent developments in neural network technologies. The broad area of computer systems investigates these architectures and networks that enable new algorithms and software. Without these technologies, neural networks would not be nearly as popular and revolutionary as they are today. Yet the relationship between computer systems and AI development is not one-directional. Today, computer scientists are using neural networks to help design new, more efficient computer systems. The

development of foundational computer technologies not only creates opportunities for direct and indirect applications, but also supports the development of other computer technologies.

Just as we saw how technological fixes embodied a powerful belief about the relationship between computer solutions and social good, a similar cultural belief exists about the relationship between foundational technologies and their social values. The belief that technologies are inherently neutral and that it is the people using technology who ultimately make it "good" or "bad" is considered **social determination of technology**.

THINK IT THROUGH

Social Determination of Technology

Do you agree with the social determination of technology? Is it possible for technology—before it is used by people to solve certain problems—to encode social values? Try to come up with an example that would support this belief. What about an example that refutes this belief? What are the social implications of agreeing or disagreeing with the social determination of technology?

Today's neural networks are designed to identify patterns and reproduce existing data. It is widely accepted that many big datasets can encode social preferences and values, particularly when the data is collected from users on the Internet. A social determination of technology accepts this explanation of AI bias and leaves the design of AI algorithms and techniques as neutral: the bias in an AI system is attributed to the social values of the data rather than the design of the AI algorithms. Critics of social determination point out that the way AI algorithms learn from big data represents a social value, one that encodes a default preference for reproducing the biases inherent in big data. This applies whether the AI application is about fair housing, medical imaging, ad targeting, drone strikes, or another topic. This is an issue that computer scientists must consider as they practice responsible computing and strive to ensure that data is gathered and handled as ethically as possible.

Evaluating Negative Consequences of Technology

Today's AI technologies work by reproducing existing patterns rather than imagining radically different futures. As much as neural networks are inspired by the human brain, it would be a stretch to suggest that AI systems have any semblance of general intelligence. Though these systems might be quite effective at identifying lettuce plants from weed plants in an image, their capacity for humanlike intelligence is limited by design. A neural network learns to recognize similar patterns that appear across millions or billions of sample images and represent these patterns with millions or billions of numbers. Mathematical optimization methods are used to choose the numeric values that best encode correlations across the sample images. However, current approaches lack a deeper, conceptual representation of objects. One criticism of very large neural networks is that there are often more numeric values than there are sample images—the network can effectively memorize the details of a million sample images by encoding them in a billion numbers. Many of today's neural networks recognize objects in images not by relying on some intrinsic idea or concept of objects but by memorizing every single configuration of edges as they appear in the sample images.

This limitation can lead to peculiar outcomes for image recognition systems. Often, neural network approaches for image recognition have certain examples of images where objects are misidentified in unusual ways: a person's face might be recognized in a piece of toast or in a bunch of clouds in the sky. In these examples, the pattern of edges might coincidentally trigger the neural network values so that it misidentifies objects. These are among the more human-understandable examples; there are many other odd situations that are less explainable. An **adversarial attack** is a sample input (e.g., an image) that is designed to cause a system to behave problematically. Researchers have found that even tweaking the color of just a single point in an image can cause a chain reaction in the neural network, leading it to severely misidentify objects. The

adversary can choose the color of the point in such a way as to almost entirely control the output of some neural networks: changing a single specific point in an image of a dog might cause the system to recognize the object as a car, airplane, human, or almost anything that the adversary so desires. Moreover, these adversarial attacks can often be engineered to cause the neural network to report extremely high confidence in its wrong answers. Self-driving cars that use neural networks for image recognition can be at risk of real-world adversarial attacks when specially designed stickers are placed on signs that cause the system to recognize a red light as a green light (Figure 1.10). By studying adversarial attacks, researchers can design neural networks that are more robust and resilient to these attacks.

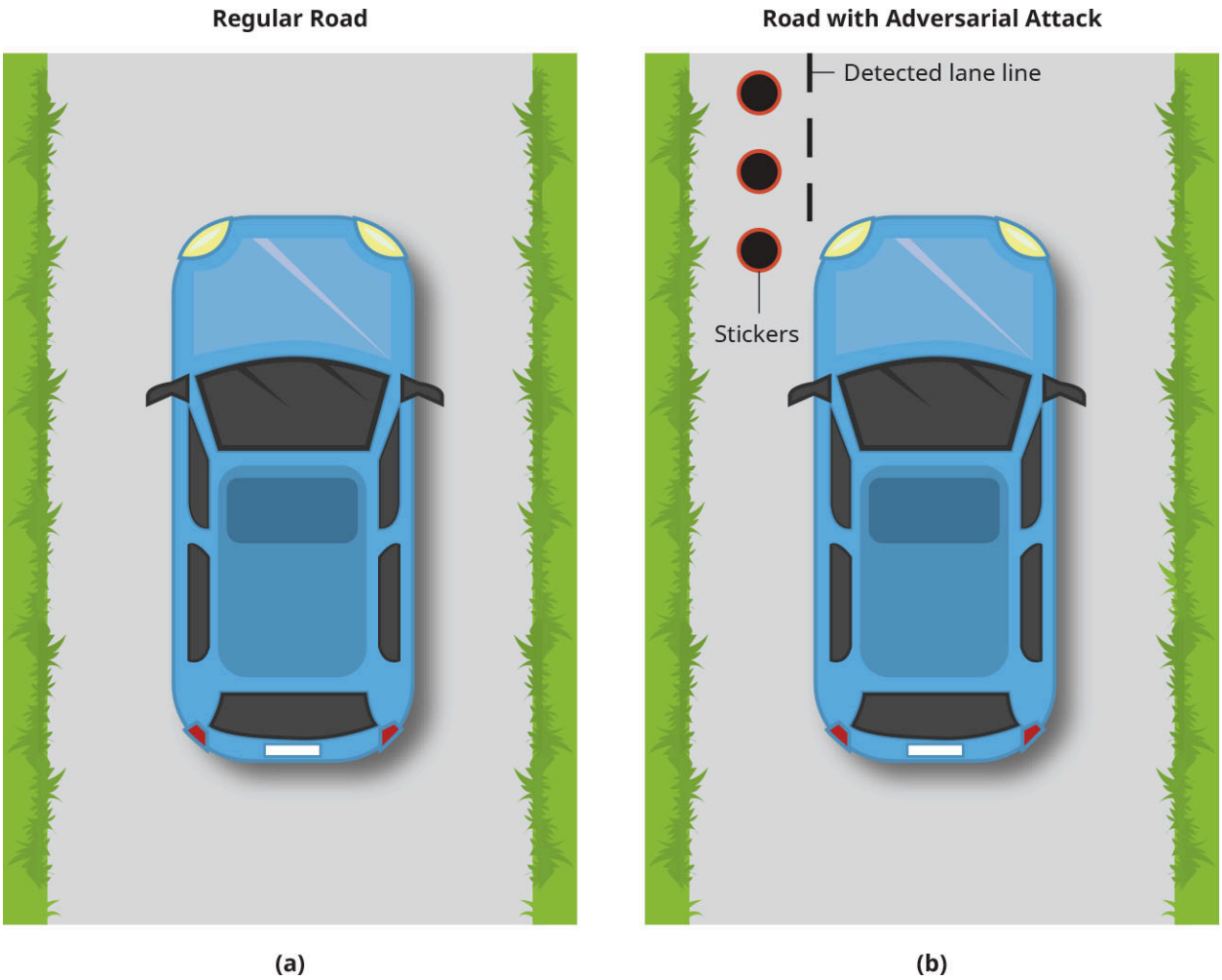


Figure 1.10 (a) Autopilot functions in self-driving cars generally identify roads and lanes using artificial intelligence to "see" road markings. (b) Researchers were able to trick these cars into seeing new lanes by using as few as three small stickers, to confuse the neural networks and force the cars to change lanes. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

In general, research is an important part of computer science. Through research, computer scientists analyze ways that technology can be used and gain insight and answers to address issues and improve various aspects of society. Research enables computer scientists to make advancements like the design of new algorithms, development of new hardware and software, and applications for emerging technologies such as AI.

One important use of research is to investigate adversarial attacks to gather answers needed for computer scientists to improve foundational technologies by evaluating the negative consequences of technologies. Computer technologies offer a unique medium for learning things (not just learning computer science), connecting with each other, and enhancing the lives of people all around the world. Yet, in each of these examples, we also raised concerns about how these technologies unfolded and affected people's lives in both positive and negative ways. While we can rarely, if ever, paint any one technology as purely "good" or "bad,"

computer scientists are interested in studying questions around how technologies are designed to center social values. Social scientists are not solely responsible for answering questions about technology, but computer scientists can also contribute important knowledge and methods toward understanding computer technologies.

Designing Technologies for Social Good

Computer science can advance social good by benefiting many people in many different areas, including public health, agricultural sustainability, climate sustainability, and education.

Computer technologies accelerate medical treatments for public and personal health from initial research and development to clinical trials to large-scale production and distribution. In January 2020, Chinese officials posted the genetic sequence of the coronavirus SARS-CoV-2. This helped pharmaceutical companies to begin developing potential vaccines for the virus at a significantly faster rate than for any other virus in the past (Figure 1.11).

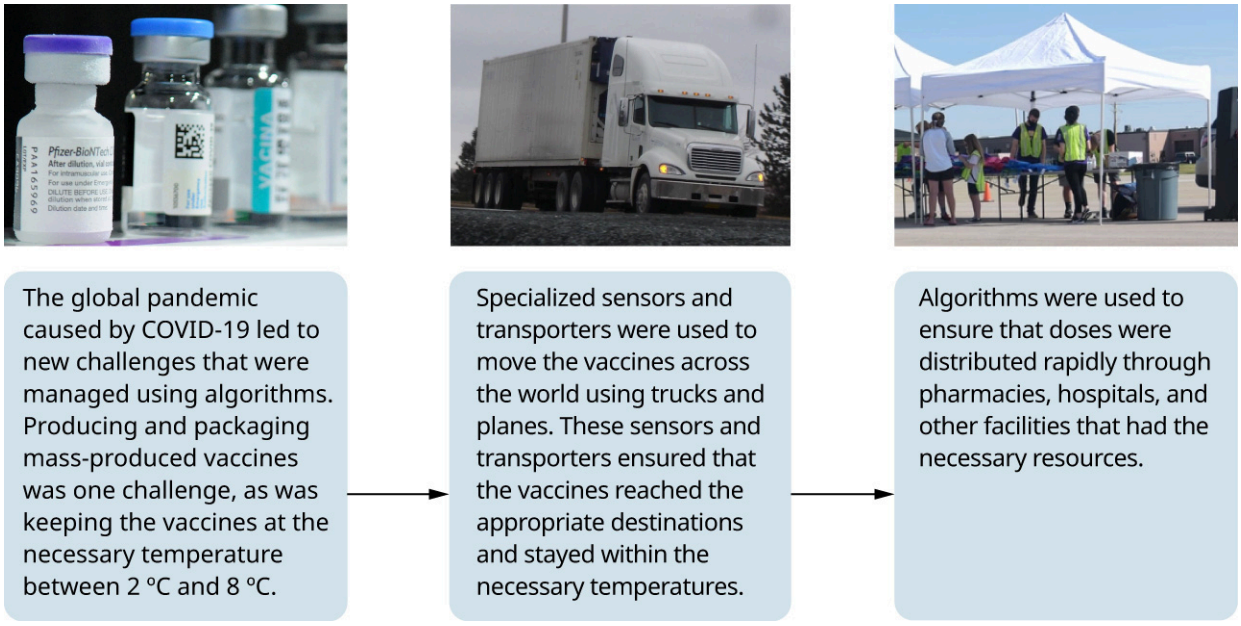


Figure 1.11 The SARS-CoV-2 outbreak that began in 2020 displayed how quickly computer science could be harnessed by governments, medical facilities, and scientists to decode the virus, develop treatments, and distribute vaccinations around the world. What would have been a very difficult feat to manage manually was simplified through the use of algorithms and computer technology. (credit left: modification of "COVID-19 vaccines" by Agência Brasília/Flickr, CC BY 2.0; credit center: modification of "T04" by Sarah Taylor/Flickr, CC BY 2.0; credit right: modification of “Back2School Brigade prepares families for upcoming school year” by Thomas Karol/DVIDS, Public Domain)

Computational science enables the miracles of modern medicine. Viral sequences can be digitized and rapidly shared between researchers across the world via the Internet. Computer algorithms and models can simulate the human immune system responses to particular treatments within hours rather than years. The first treatments can then be produced at a small scale using computer-engineered cells in less than a month from the initial sequencing. To ensure the treatments are safe and effective, clinical trials are held at disease transmission “hot spots” predicted using data science methods drawing on data aggregated and monitored from across the world. Once a treatment is proven safe and effective, it is mass-produced with the help of computer-controlled robots and automated assembly lines. Algorithms manage the inventory supply and demand and control the transportation of treatments on trucks and planes guided by computer navigation systems. Web apps and services notify people throughout the process.

Yet the use of computer technology throughout modern medicine is anything but politically neutral. Computers, algorithms, and mathematical models solve the problems that their creators wish to solve and encode the assumptions of their target populations. Supply and demand data for the data models are

determined by various factors, at least partly in response to the money and relationships between countries that control the technology, the Global North, and countries that don't, the Global South. Within local communities, the uptake of medical treatments is often inequitable, reflecting and reinforcing historical inequities and disparities in public health. Computer technology alone often doesn't address these issues. In fact, without people thinking about these issues, computer technologies can often amplify disparities. Consider datasets, which can be biased if they overrepresent or underrepresent specific groups of people. If decisions are made on the basis of biased data, people in the groups that are not represented fairly may receive inequitable treatment. For example, if a local government agency is working with a biased dataset, political leaders may make decisions that result in certain citizens receiving inadequate funding or services. This is an example of why responsible computing, which we will cover in Chapter 14 Cyber Resources Qualities and Cyber Computing Governance, is so important.

These problematic histories are not only aggravated in medicine and public health, but also reflected in housing. Redlining refers to the inequitable access to basic public services based on residents' neighborhoods and communities, which includes the practice of withholding financial services from areas with a large underrepresented population. In the United States, these communities reflect the histories of racial segregation and racial wealth inequalities. Fair housing laws are intended to prevent property owners from discriminating against buyers or renters because of race, color, ability, national origin, and other protected classes. But computer technologies also present new kinds of challenges. Microtargeted ads on social media platforms contribute to not only political polarization, but also discrimination in housing. This can be a particular problem when combined with redlining. Even if the ad targeting is not explicitly designed to discriminate, microtargeted ads can still reinforce historical redlining by incorporating data such as zip codes or neighborhoods. This may result in digital redlining, which is the practice of using technology, such as targeted ads, to promote discrimination. In 2021, a Facebook user filed a class-action lawsuit that argued nine companies in the Washington, D.C., area deliberately excluded people over the age of 50 from seeing their advertisements for housing because they wanted to attract younger people to live in their apartments.[16] This is an example of an issue in technology that should be addressed by responsible computing with an emphasis on ethical behavior.

With good intentions and attention to personal biases, technologies can be designed for social good. For example, a hypothetical algorithm for fair housing could evenly distribute new housing to people across protected classes and marginalized identities, such as older populations. Of course, algorithmic control and automated decision-making is challenged to consider the underlying conditions behind social problems. Still, algorithms can be important tools to enable us to distribute outcomes more fairly from a statistical perspective, and this can be an important step in addressing the larger societal systems and inequities that produce social problems.

LINK TO LEARNING

Review the Parable of the Polygons (https://openstax.org/r/76polygons) by Vi Hart and Nicky Case. In it, the authors show how a segregated world where people simply prefer living near other people who are like themselves (a "small individual bias") can re-create and reproduce "large collective bias."

As part of responsible computing, computer scientists must be aware of **technological fix**, which refers to the idea that technologies can solve social problems, but is now often used to critique blind faith in technological solutions to human problems. Unless the process is handled responsibly, the "fix" may cause more problems than it resolves. When considering how to address social and political problems, computer scientists must take care to ensure that they select the appropriate technology to address specific problems.

16 C. Silva, "Facebook ads have a problem. It's called digital redlining," 2022. https://mashable.com/article/facebook-digital-redlining-ads-protected-traits-section-230

To address social problems and advance social good, recall that human-centered computing emphasizes people rather than technologies in the design of computer solutions. A human-centered approach to fair housing might begin by centering local communities directly affected by redlining. Rather than replacing or disrupting the people and organizations already working on a problem, a human-centered approach would center them in the design process as experts. A human-centered approach requires that the designer ask why they are not already working with people in the community impacted by their work.

LINK TO LEARNING

Anatomy of an AI System (https://openstax.org/r/76AIanatomy) illustrates how an AI system like the Amazon Echo does not just involve computer technology, but also involves a vast and deeply interconnected web of human labor, data, and physical resources that are often taken for granted. Evaluation of the negative consequences of technology does not end at the technology itself, but also considers its broad-reaching impacts and implications for people around the world.

Chapter Review

Key Terms

adversarial attack sample input (e.g., an image) that is designed to cause a system to behave problematically
algorithm sequence of precise instructions
artificial intelligence (AI) development of computer functions to perform tasks, such as visual perception and decision-making processes, that usually are performed by human intelligence
big data very large datasets that aren't easily processed using spreadsheets
computational science application of computing concepts and technologies to advance scientific research and practical applications of science knowledge
computer program algorithms that can be run on a computer
computer science (CS) study of the phenomena surrounding computers
computing all phenomena related to computers
data science interdisciplinary field that applies computing toward managing data and extracting information from data
hardware physical, real-world materials that enable computation
human-computer interaction (HCI) subfield of computer science that emphasizes the social aspects of computation
image recognition problem of identifying objects in an image
information science interdisciplinary field studying information technologies and systems as they relate to people, organizations, and societies
memory means of addressing information in a computer by storing it in consistent locations
network various technological devices that are connected and share information
neural network AI algorithm architecture that emphasizes connections between artificial "neurons" whose behavior and values change in response to stimulus or input
processor computer's "brain," that follows instructions from algorithms and processes data
programming language language consisting of symbols and instructions that can be interpreted by a computer
social determination of technology belief that technologies are inherently neutral, and that it is the people who use a technology who ultimately make it "good" or "bad"
software algorithmic principles that determine how results are computed
software engineering subfield of computer science that emphasizes how problems can be solved with computers as well as the practices and processes that can help people design more effective software solutions
spreadsheet data-centric programming environment where data is organized into cells in a table
storage hardware and physical components of a computer that permanently house a computer's data
technological fix idea that technologies can solve social problems, but now often used to critique blind faith in technological solutions to human problems
theoretical computer science mathematical processes behind software
Turing-complete fundamental model for computing results and every computer has the ability to run any algorithm
vacuum tube physical device that works like a light bulb used as memory in early digital computers

Summary

1.1 Computer Science

- Computer science is pervasive in our daily lives, business and industry, scientific research and development, and social change.
- Computer science (CS) is the study of computing, which includes all phenomena related to computers,

such as the Internet. With foundations in engineering and mathematics, computer science focuses on studying algorithms, which are instructions that enable computing. This includes computer hardware and software and the way these are used to process information. Three perspectives on computers include the hardware perspective, software perspective, and theoretical perspective. These perspectives each emphasize different aspects of computation, and they're often centered in undergraduate computer science because of the history of computer science, but there are other perspectives on computer science.

- People have used computer science to advance many more diverse goals beyond making war or making money. Computing was imagined as: a new medium for helping people learn everything; a new technology that could enable anti-racism; a means of enabling global development for peoples across the world. Yet these visions and promises are still taking hold in a world largely focused on the dominant history of computer science.

1.2 Computer Science across the Disciplines

- By contributing tools and resources to handle tasks and improve operations, computer science enables many other fields and areas of research or development.
- Data science is an interdisciplinary field that applies computing to managing data and extracting information from data. Many millions of people engage in data science work by using spreadsheets. Still, data science also often emphasizes larger-scale problems involving big data that are hard to manage using spreadsheets alone.
- Computational science refers to applying computing concepts and technologies to advance scientific research and practical applications of science knowledge. Computer science's emphasis on creating things can help other sciences by, for example, contributing new models or simulations that can enable the discovery of new kinds of scientific knowledge previously inaccessible to scientists.
- Information science is an interdisciplinary field studying information technologies and systems as they relate to people, organizations, and societies. As computing is now so central to information management and information exchange, information science has significant overlap with computer science. Still, it tends to emphasize the social value of information, whereas computer science has (historically) emphasized algorithms and computation over people or information.
- Today, computer science is an interdisciplinary field that contributes to all other fields. Effective computational solutions to research or business problems require combining domain-specific knowledge with computer science concepts from a combination of areas.

1.3 Computer Science and the Future of Society

- Computer science is shaping the future of society. There are three ways in which computer science can shape the future of society: developing foundational technologies, evaluating negative consequences of technologies, and designing technologies for social good.
- As one example of developing foundational technologies, computer science's rapid development of artificial intelligence technologies (and the current trend around neural networks) has enabled many new applications like image recognition. These developments often do not occur in isolation: the popular use of neural networks, for example, depended on new computer architectures and advancements in the Internet (computer networks). Technologies can encode social values: neural networks are designed to learn from big data, so they encode a preference for the contemporary social realities that produced the data.
- As one example of evaluating negative consequences, computer science considers the philosophical and practical limitations of neural networks. Research into adversarial attacks can enable computer scientists to develop more robust neural networks that are safer and more effective.
- As one example of designing technologies for social good, computer science contributes to the research, development, mass production, delivery, and logistics of modern medicine from beginning to end. Yet applications for social good are often embedded in broader social and political dynamics that computer science has difficulty addressing. Even though computer technologies can be designed for social good, they can cause harm when their design processes fail to center on human values and diverse users.

Review Questions

1. What is computer science?
2. What two subjects does computer science combine the foundations of?
   a. math and engineering
   b. math and physics
   c. physics and engineering
   d. math and chemistry
3. What can execute an algorithm?
4. What enables the ENIAC (one of the first digital computers, invented in 1945) to be able to compute anything that can run on modern computers?
   a. Both the ENIAC and modern computers have memory.
   b. Both the ENIAC and modern computers share the same hardware.
   c. Both the ENIAC and modern computers are considered Turing-complete.
   d. Both the ENIAC and modern computers run the same software.
5. What invention was credited as the first calculator?
   a. punch cards
   b. abacus
   c. Difference Machine
   d. ENIAC
6. What term is considered an algorithm that can be run on a computer?
   a. artificial intelligence
   b. algorithm
   c. computer program
   d. programming language
7. Why is computer science often not considered a science?
   a. Computer science does not study natural objects.
   b. Computer science emphasizes the discovery of natural phenomena.
   c. Computer science is spreadsheet-based.
   d. Computer science focuses on computational structures.
8. What is the definition of data science?
   a. a subfield of computer science that emphasizes the social aspects of computation
   b. an interdisciplinary field studying information technologies and systems as they relate to people, organizations, and societies
   c. a subfield of computer science that emphasizes how problems can be solved with computers as well as the practices and processes that can help people design more effective software solutions
   d. an interdisciplinary field that applies computing toward managing data and extracting information from data
9. What term is used to describe a subfield of computer science that emphasizes how a computational problem can be defined in mathematical terms and whether that mathematical problem can be efficiently solved with a computer?
   a. computational science
   b. theoretical computer science
   c. information science
   d. data science
10. What subfield of computer science relates information technology to people and society?

a. computational science
b. theoretical computer science
c. information science
d. data science

11. How does computational science contribute new methods to the study of the sciences?

12. How do information science and computer science compare?

13. What does it mean to say that the various areas of computer science are synergistic?

14. What term is defined as an approach that emphasizes people rather than technologies in the design of computer solutions?
a. human-centered computing
b. neural network
c. social determination of technology
d. technological fix

15. A software application takes an image as an input and analyzes it. This is an example of what?
a. illustrative processing
b. image recognition
c. image generation
d. analytical modeling

16. What are adversarial attacks and why is it important to study them?

17. What is the relationship between artificial intelligence, image recognition, and neural networks?

18. How do neural networks recognize objects in images?

Conceptual Questions

1. Give two definitions of computer science. How do they compare?

2. Explain the concept of theoretical computer science.

3. What is body-syntonic reasoning and how has it affected education?

4. In terms of data science, what is a spreadsheet and why can it be said that by using spreadsheets, people are programming without realizing it?

5. How do discovery and invention differ and how are these involved in computer science?

6. Describe in your own words the difference between data (as in data science) and information (as in information science). How does computer science shape both fields?

7. Give a real-life example that refutes social determination of technology. Your example does not need to involve computing, but it should involve some technology designed by humans.

8. Artificial intelligence approaches are typically used to solve problems that requires specific kinds of "intelligence." Describe a real-life computational problem or application that *does not* need artificial intelligence.

9. Image recognition systems that can detect objects in images enable self-driving cars and many large-scale manufacturing or agricultural operations. Give another example where image recognition could be used as part of a larger system to automate decisions at scale.

Practice Exercises

1. Research where the term *debugging* originated from and why it refers to finding problems in our

programs today.

2. Research some examples of computational models and how they are a part of everyday life.
3. Research how computational models relate to mathematics.
4. Research some examples of how artificial intelligence is used across multiple industries. Summarize at least two different industries and how AI is currently being used.
5. Research ethical issues related to artificial intelligence and provide an example of how artificial intelligence can be misused for unethical and even criminal ways.

Problem Set A

1. Research examples of how modeling and simulations have led to new inventions and discoveries.
2. Research and provide a summary of the difference between artificial intelligence and machine learning.

Problem Set B

1. Research how artificial intelligence and machine learning can improve the accuracy of computational models and lead to cutting-edge technology inventions in the future. Provide a specific example of how AI and ML have been used in this way so far.
2. Research and provide a summary of how machine learning is a subset of artificial intelligence and plays a key role in artificial intelligence systems.

Thought Provokers

1. Corporate social responsibility is the idea that businesses have a responsibility to society, including the areas of environmental responsibility, ethical responsibility, philanthropic responsibility, and economic responsibility. Given the contentious history of computer science and computer technologies, what can businesses (or businesspeople) that wish to employ a "disruptive" computer technology do to ensure corporate social responsibility?
2. Computer technologies like the Internet have changed everyone's lives, regardless of whether they use the Internet directly or not. Yet, with computer technologies, the future is rarely certain. How can a business stay relevant and profitable in the face of new technologies while ensuring corporate social responsibility? In what ways does ensuring corporate social responsibility create economic value and more diverse kinds of value?
3. Give a real-life public policy problem involving a computer technology or dataset and use it to illustrate differences between the fields of data science, information science, and computer science.
4. Mathematics is one of three perspectives that computer scientists use to design, analyze, and evaluate computational structures, systems, and processes. However, mathematics is often regarded as an abstract or "pure" field of study that is not involved in social or political concerns. How might computer science's ability to automate and represent problems in mathematical terms have social or political consequences?
5. The future of society will be shaped by the *philosophy* of computer science and people's purposes, motivations, and political values. Give another philosophy that might influence or affect how computer scientists go about creating solutions.
6. If your organization is interested in artificial intelligence technology to enhance operations, what could you do to ensure the system is designed and implemented in a safe, socially responsible, and just manner?

Labs

1. Explore the Parable of the Polygons (https://openstax.org/r/76polygons). How does computer science

contribute to the simulation? What does the simulation suggest is needed in the world? What are the limitations of the simulation as a model of the much more complicated real world?

2. Explore the Anatomy of an AI System (https://openstax.org/r/76AIanatomy) that breaks down the "human, labor, data and planetary resources" behind an Amazon Echo device. What parts surprised you? Based on your understanding of computer science, what parts are emphasized in the public conscience? What parts are downplayed or minimized? Then, select one surprising aspect and investigate it further.
3. Explore DALL-E (https://openstax.org/r/76DALL-E), a very large neural network created by OpenAI "that creates images from text captions for a wide range of concepts expressible in natural language." If the neural network is learning from English language images and captions on the Internet, what are some of the social risks of this system? How might it encode problematic ideas about marginalized people in society?



2 Computational Thinking and Design Reusability

Figure 2.1 A work environment, such as this modifiable "sp.ace" at Johnson Space Center where furniture can move and any surfaces can be written on, can encourage computational thinking and lead to innovation. (credit: modification of "The sp.ace in Building 29 at Johnson Space Center" by Christopher Gerty, NASA JSC/APPEL Knowledge Services, Public Domain)

Chapter Outline

2.1 Computational Thinking
2.2 Architecting Solutions with Adaptive Design Reuse in Mind
2.3 Evolving Architectures into Useable Products

Introduction

In the rapidly evolving landscape of technology and innovation in various domains, computational thinking promotes design reusability and is a fundamental skill set essential for problem-solving. This chapter illustrates how computational thinking, through practical insights and theoretical frameworks, facilitates the creation of reusable designs that improve the qualities (e.g., scalability, efficiency) of solutions.

Developing innovative solutions to business problems today involves reinventing, rethinking, and rewiring existing business solutions and leveraging the latest technologies to assemble competitive products that solve business problems. It is key to apply computational thinking throughout this process to tackle new problems in specific areas. Computational thinking is a problem-solving and cognitive process rooted in principles derived from computer science. It involves breaking down complex problems into smaller, more manageable parts and devising systematic approaches to solve them.

Adaptive design reuse also makes it possible to quickly assemble business solutions by assembling existing design building blocks that require minimal customizations to be a good match for the problem at hand. TechWorks is a company focused on developing innovative products and services. TechWorks is looking to take advantage of computational thinking and adaptive design reuse to enable the creation of next-generation, secure, super society, intelligent, autonomous business solutions. At TechWorks, a skilled team of engineers, data scientists, and designers is on a mission to revolutionize society. Their goal is to create advanced and secure autonomous business solutions. The team believes in the power of computational thinking and adaptive design reuse for success.

Led by the CIO, the team gathered in a cutting-edge laboratory to tackle challenges in transportation, security,

and automation. They embraced computational thinking by applying algorithms and machine learning to analyze data. Recognizing the efficiency of adaptive design reuse, the team explored successful projects like robotics and self-driving cars for inspiration. These projects have become the foundation for their own innovation. With minimal adjustments, they seamlessly integrate these building blocks into comprehensive solutions such as self-driven cars that can smoothly navigate the city, drones that can monitor public spaces, and robotics that automate tasks. The company plans to bring their vision of the future to life by transforming cities into hubs of interconnected, intelligent systems. Knowing that innovation is a continuous process that requires rapidly evolving solutions, the team faced challenges while implementing their initial prototype. However, they are able to adapt their super society solutions using computational thinking and adaptive design reuse to ensure that they stay ahead of technological advancements. TechWorks is a symbol of the successful integration of forward-thinking strategies to create secure and technologically advanced super society solutions.

2.1 Computational Thinking

Learning Objectives

By the end of this section, you will be able to:

- Define computational thinking
- Discuss computational thinking examples

This chapter presents key aspects of computational thinking, including logical thinking, assessment, decomposition, pattern recognition, abstraction, generalization, componentization, and automation. These elements guide how computer scientists approach problems and create well-designed solution building blocks at both the business and technical levels. Computational thinking often involves a bottom-up approach, focusing on computing in smaller contexts, and seeks to generate innovative solutions by utilizing data structures and algorithms. Additionally, it may make use of existing design building blocks like design patterns and abstract data types to expedite the development of optimal combinations of data structures and algorithms.

What Is Computational Thinking?

The problem-solving and cognitive process, known as **computational thinking**, is rooted in principles derived from computer science. It involves breaking down complex problems into smaller, more manageable parts and devising systematic approaches to solve them. Complex problems are situations that are difficult because they involve many different interrelated parts or factors. These problems can be hard to understand and often don't have simple solutions.

While "computational thinking" is still perceived by some as an abstract concept without a universally accepted definition, its core value is to facilitate the application of separate strategies and tools to address complex problems. In problem-solving, computers play a central role, but their effectiveness centers on a prior comprehension of the problem and its potential solutions. Computational thinking serves as the bridge between the problem and its resolution. It empowers solution designers to navigate the complexity of a given problem, separate its components, and formulate possible solutions. These solutions, once developed, can be communicated in a manner that is comprehensible to both computers and humans, adopting effective problem-solving.

LINK TO LEARNING

Computational thinking serves as the intermediary that helps us read complex problems, formulate solutions, and then express those solutions in a manner that computers, humans, or a collaboration of both can implement. Read this article for a good introduction to computational thinking (https://openstax.org/r/

76CompThinking) from the BBC.

Vision

To further qualify computational thinking, Al Aho of the Columbia University Computer Science Department describes computational thinking as "the thought processes involved in formulating problems so their solutions can be represented as computational steps and algorithms." Jeannette Wing, also of Columbia University, brought the idea of computational thinking to prominence in a paper she wrote in 2006 while at Carnegie Mellon University. She believes that computational thinking details the mental acts needed to compute a solution to a problem either by human actions or machine. Computational thinking encompasses a collection of methods and approaches for resolving (and acquiring the skills to resolve) complex challenges, closely aligned with mathematical thinking through its utilization of abstraction, generalization, modeling, and measurement (Figure 2.2). However, it differentiates itself by being more definitely aware than mathematics alone in its capacity for computation and the potential advantages it offers.

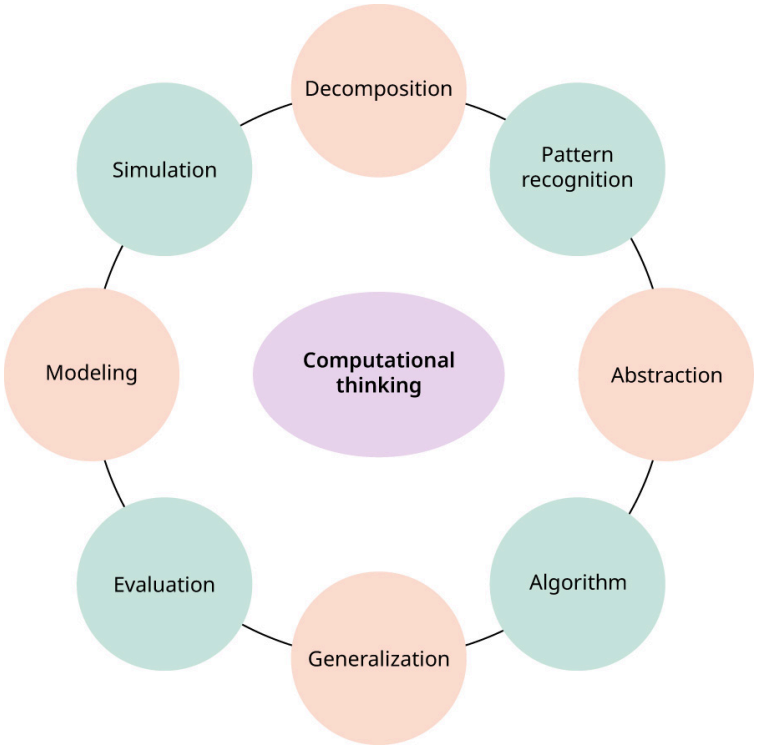


Figure 2.2 This diagram illustrates the main components of computational thinking. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Critical thinking is an important skill that can help with computational thinking. It boils down to understanding concepts rather than just mastering technical details for using software, prioritizing comprehension over rote learning. It's a core skill, not an extra burden on a curriculum checklist, and it uniquely involves humans, not computers, blending problem-solving and critical thinking. Critical thinking focuses on ideas, not tangible items, applying advanced thinking to devise solutions. Critical thinking is essential for everyone and is, comparable to foundational abilities like reading, writing, and arithmetic.

Computational Thinking Concepts

The description provided by the International Society for Technology in Education (ISTE) outlines the key components and dispositions of computational thinking. Let's explore each characteristic in more detail:

- Decomposition: The analytical process of breaking down complex concepts or problems into smaller parts

is called **decomposition**. This approach helps analyze and solve problems more effectively.

- Pattern recognition (logically organizing and analyzing data): Computational thinking emphasizes the logical organization and analysis of data. This includes the ability to structure information in a way that facilitates effective problem-solving.
- Representing data through abstractions: An **abstraction** is a simplified representation of complex systems or phenomena. Computational thinking involves representing data through an abstraction, such as a simulation, which uses models as surrogates for real systems.
- Automation through algorithmic thinking: Using a program or computer application to perform repetitive tasks or calculations is considered **automation**.
- Identification, analysis, and implementation of solutions: Computational thinking emphasizes identification, analysis, and implementation of potential solutions to achieve optimal efficiency and effectiveness through a combination of steps and resources.
- Generalization and transferability: Generalizing and transferring this problem-solving process across a wide variety of problems showcases the adaptability and applicability of computational thinking.

These abilities are supported and enriched by fundamental abilities integral to computational thinking. These abilities involve the following characteristics: confidence in navigating complexity, resilience in tackling challenging problems, an acceptance of ambiguity, adeptness in addressing open-ended issues, and proficiency in collaborating with others to attain shared objectives or solutions. Another illustration of computational thinking is the three As, which is organized into three phases, as visualized in Figure 2.3:

1. Abstraction: The initial step involves problem formulation.
2. Automation: Next, the focus shifts to expressing the solution.
3. Analysis: Finally, the process encompasses solution execution and evaluation.

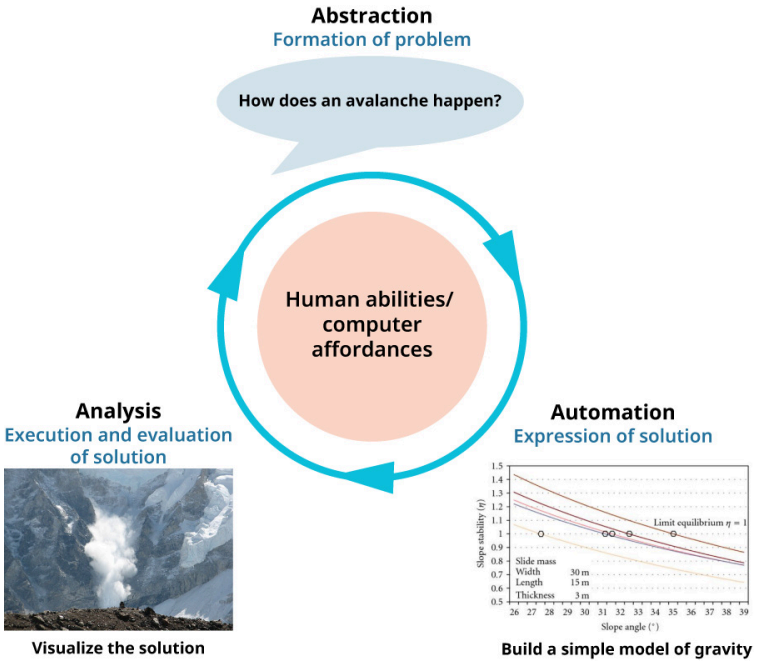


Figure 2.3 The three As—abstraction, automation, analysis—illustrate the power of computational thinking. (credit photo: modification of "Avalanche on Everest" by Chagai/Wikimedia Commons, Public Domain; credit graph: modification of "Slope stability calculation for a model landslide" by B. Terhost and Bodo Damm/Journal of Geological Research, CC BY)

Computational Thinking Techniques

In today's technology world, mastering computational thinking techniques is important. These techniques offer a systematic way to solve problems using tools like data structures, which are like containers used to organize and store data efficiently in a computer. They define how data is logically arranged and manipulated, making it easier to access and work with information in algorithms and programs. There are four key

techniques (cornerstones) to computational thinking, as illustrated in Figure 2.4:

- Decomposition is a fundamental concept in computational thinking, representing the process of systematically breaking down a complex problem or system into smaller, more manageable parts or subproblems. By breaking down complexity into simpler elements, decomposition promotes a more organized approach to problem-solving.
- Logical thinking and pattern recognition is a computational thinking technique that involves the process of identifying similarities among and within problems. This computational thinking technique emphasizes the ability to recognize recurring structures, relationships, or sequences in various problem-solving situations.
- Abstraction is a computational thinking technique that centers on focusing on important information while ignoring irrelevant details. This technique enables a clearer understanding of the core issues.
- Algorithms are like detailed sets of instructions for solving a problem step-by-step. They help break down complex tasks into manageable actions, ensuring a clear path to problem-solving.

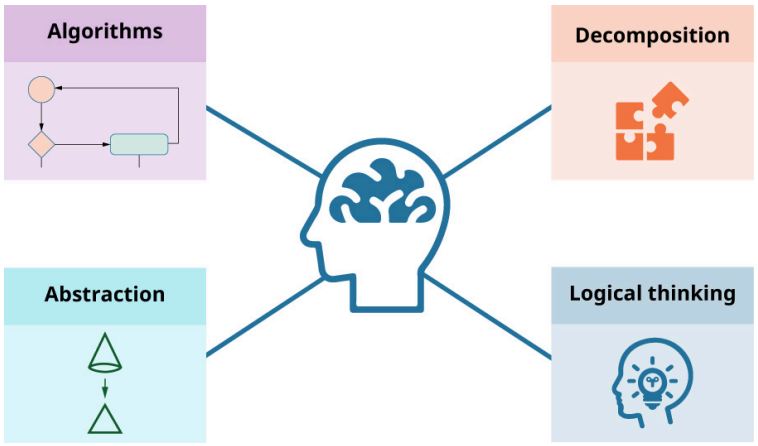


Figure 2.4 Users can explore the essence of computational thinking through decomposition, logical thinking, abstraction, and algorithms. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

In addition to the four techniques, computational thinking involves essential steps such as testing and debugging. Testing is crucial for uncovering errors within the step-by-step instructions or algorithms employed to tackle a problem. On the other hand, **debugging** entails identifying and rectifying issues within the code.

A programmer is someone who writes instructions for a computer to follow. A typical example is that of a programmer who gives instructions to a robot and tells it to make a jam sandwich. In this case, applying computational techniques to give instructions to the robot entails the following techniques: decomposition, logical thinking and pattern recognition, abstraction, and algorithms. These techniques are explained in the following subsections as they apply to the jam sandwich example.

TECHNOLOGY IN EVERYDAY LIFE

Traffic Accident Data

Analyzing data involves collecting and cleaning information, exploring patterns through visual and statistical methods, and forming hypotheses. Statistical analysis and visualization are used to draw conclusions, and findings are interpreted and communicated in reports or presentations to help in the process of decision-making. Analyze the patterns and trends in traffic accident data to understand the prevalence of road injuries and fatalities, and examine the progression of traffic incidents over time. To enhance road safety measures and policies, you should apply computational thinking skills to identify recurring patterns and abstract the most crucial information from the data. By extracting valuable insights,

you can contribute to the development and refinement of strategies that effectively improve road safety.

Decomposition

Decomposition involves solving a complex problem by breaking it up into smaller, more manageable tasks. Decomposition enables the consideration of various components essential for solving a seemingly complex task, allowing it to be redefined into a more manageable problem. In the case of the jam sandwich example, decomposition involves identifying all the required ingredients and the steps the robot must take to successfully create a jam sandwich.

Logical Thinking and Pattern Recognition

Pattern recognition makes it possible to group all the different features considered in decomposition into categories. In the jam sandwich example, pattern recognition leads to grouping the various things identified via decomposition into categories, in this case, ingredients, equipment, and actions. Therefore, applying decomposition and pattern recognition will lead to thinking of as many things as possible that are required to make a jam sandwich. The more things that can be thought of (i.e., ingredients, equipment, and actions), the clearer the instructions will be. A first attempt at decomposition and pattern recognition is summarized in Table 2.1.

Ingredients	Equipment	Actions
Bread	Plate	Repeat *x* times
Jam	Knife	Left hand (LH)
Butter		Right hand (RH)
		Pick up
		Unscrew

Table 2.1 Logical Thinking and Pattern Recognition Example The jam sandwich pattern recognition defines the ingredients, equipment, and actions needed for completion.

The process of identifying patterns typically requires logical thinking such as inductive or deductive reasoning. Inductive reasoning makes it possible to go from specific examples to general principles. For example, recognizing that dividing any number by 1 results in the original number leads to the broader conclusion that holds true for any number. Similarly, understanding that the sum of two odd numbers yields an even number leads to the generalization that adding two odd numbers always results in an even number. Inductive reasoning turns an observation into a pattern, which allows making a tentative hypothesis that can be turned into a theory. Deductive reasoning is the process of drawing valid conclusions from premises given the fact that it is not possible for the premises to be true and the conclusion to be false. A traditional example illustrates how the premises “all men are mortal” and “Socrates is a man” lead to the deductively correct conclusion that “Socrates is mortal.”

TECHNOLOGY IN EVERYDAY LIFE

Computational Thinking in Our Life

Computational thinking is a method of problem-solving that is extremely useful in everyday life. It involves breaking down complex issues into manageable parts, identifying patterns, extracting essential information, and devising systematic solutions. This process not only applies to technical fields, but also to everyday situations.

For example, imagine someone trying to manage their monthly expenses within a tight budget. Here's how you might apply computational thinking to this common problem of managing a monthly budget:

1. Decomposition: Break down the financial challenge into different categories such as rent, groceries, utilities, and entertainment.
2. Pattern recognition: Analyze past spending to identify patterns.
3. Abstraction: Focus on key areas where costs can be reduced.
4. Algorithmic thinking: Develop a systematic approach to allocate monthly income.

By using computational thinking, you can manage your finances more effectively, ensuring they cover essential costs while maximizing their savings.

Abstraction

Abstraction makes it possible to pull out the important details and identify principles that apply to other problems or situations. When applying abstraction, it may be useful to write down some notes or draw diagrams to help understand how to resolve the problem. In the jam sandwich example, abstraction means forming an idea of what the sandwich should look like. To apply abstraction here, you would create a model or draw a picture representing the final appearance of the jam sandwich once it is made. This simplifies the details, providing a clearer image of the desired outcome. Simple tools like the Windows Paint program can be used to do this, as shown in Figure 2.5.

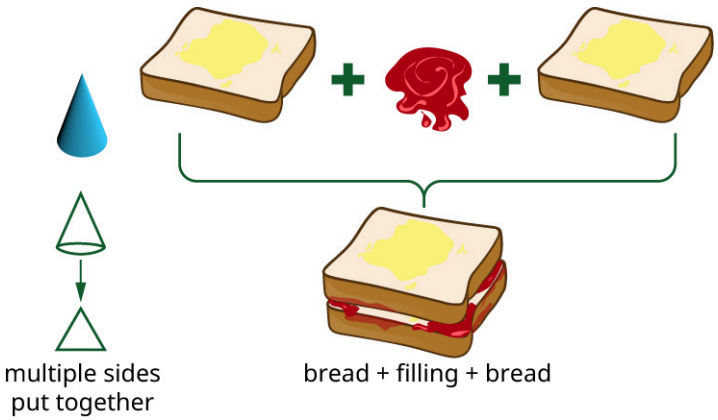


Figure 2.5 This jam sandwich abstraction example illustrates what the final product should look like. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

In technology, data are represented at different levels of abstraction to simplify user interaction and manage complex operations efficiently. Users interact with a web application through a straightforward interface, like requesting help from a GenAI tool, without seeing the underlying complexity. This GenAI prompt is then processed by the application's logic, which validates and directs it appropriately, often invisibly to the user. Finally, at the back end, the prompt is processed and a GenAI-generated response is provided. Each layer of abstraction serves a separate role, making the entire process efficient for both the user and the system (Figure 2.6).

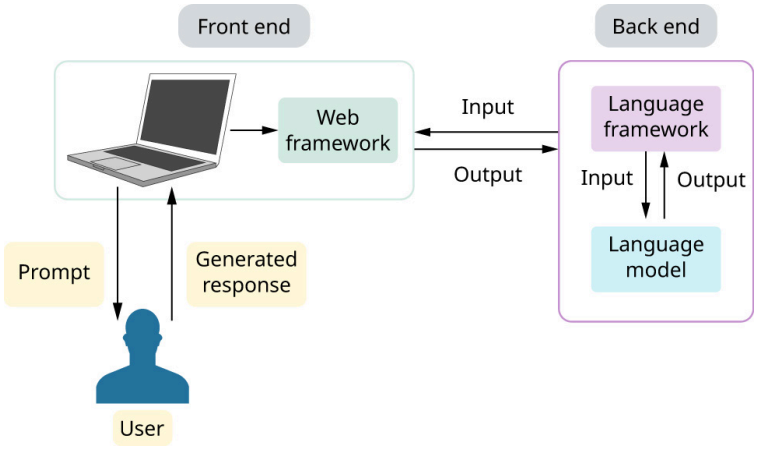


Figure 2.6 When using GenAI, a user interacts with the interface while the application processes the prompt with layers of abstraction on the back end. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Algorithm

An algorithm is a sequence of steps/instructions that must be followed in a specific order to solve a problem. Algorithms make it possible to describe a solution to a problem by writing down the instructions that are required to solve the problem. Computer programs typically execute algorithms to perform certain tasks. In the jam sandwich example, the algorithm technique is about writing instructions that the robot can follow to make the jam sandwich. As you will learn in Chapter 3 Data Structures and Algorithms, algorithms are most commonly written as either pseudocode or a flowchart. An outline of the logic of algorithms using a combination of language and high-level programming concepts is called **pseudocode**. Each step is shown in a clearly ordered, written structure. A **flowchart** clearly shows the flow and direction of decisions in a visual way using a diagram. Either way is fine, and it is a matter of personal preference. Basic templates for the flowchart and pseudocode are in Figure 2.7.

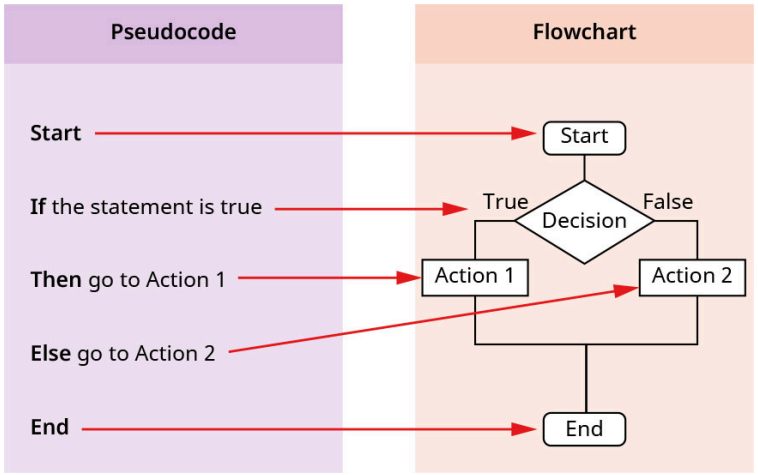


Figure 2.7 Pseudocode lists each step, while a flowchart visually outlines the process of decision-making. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Writing algorithms requires practice. Not everyone likes butter in their jam sandwich. The robot needs a method of making sure it adds or does not add butter, depending on preferences. It is therefore necessary to account for the following steps in the pseudocode and flowchart:

1. Ask whether there should be butter on the bread.
2. Either spread butter on the bread,
3. Or, do not use butter.

These steps can be added as actions in the table previously shown and expressed as steps in the pseudocode

using programming keywords such as INPUT, OUTPUT, IF, THEN, ELSE, and START. The corresponding instructions can then be converted into a flowchart using the symbols in Figure 2.8.

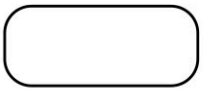
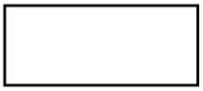
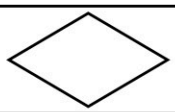
Symbol	Instruction
	Start/end
	Task (e.g., spread jam)
	Decision (e.g., do you want butter?)
	Direction of flow

Figure 2.8 The symbols used in a flowchart are associated with their instructions. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Algorithm Execution Model Patterns

Various patterns of execution models may be used to step through the instructions provided in an algorithm. So far, we have only considered the traditional sequential (i.e., step-by-step) execution model for algorithm instructions. However, it is also possible to leverage parallelism/concurrency and recursion as alternative models to drive the execution of algorithms' instructions.

Parallel/concurrent execution models are typically used to optimize algorithm execution efficiency. As an example, if you and a friend are buying tickets for a movie and there are three independent lines, you may opt for a parallel processing model of execution by having you and your friend join two separate lines to buy the tickets. In that case, you are guaranteed to be able to obtain the tickets quicker assuming one of the lines operating in parallel with the other ends up serving customers faster, which is most often the case. Note that executing the same algorithm simultaneously on a computer may not be possible if you only have one central processing unit (CPU) in your machine. In that case, you can simulate parallelism by having the operating system running on the machine execute the two algorithms concurrently as separate tasks while sharing the single processor resources. This approach is less efficient than true parallelism. More detail on the differences between concurrency and parallelism will be provided in Chapter 4 Linguistic Realization of Algorithms: Low-Level Programming Languages.

Recursive models of execution provide another elegant and effective alternative to the traditional sequential model of execution. The problem-solving technique where a process calls itself in order to solve smaller instances of the same problem is called **recursion**. It can be a powerful tool in programming because it allows for elegant solutions to complex problems by breaking them down into smaller, more manageable parts. By leveraging recursion, programmers can write concise and efficient code to solve a wide range of problems.

One of the key advantages of recursion is its ability to handle complex tasks with minimal code. Instead of writing lengthy iterative loops to solve repetitive tasks, recursion allows programmers to define a process that calls itself with modified input parameters, effectively reducing the amount of code needed. However, it's essential to be cautious when using recursion, as improper implementation can lead to stack overflow errors due to excessive process calls. Programmers should ensure that recursive processes have proper base cases to terminate the recursion and avoid infinite loops. Example:

```
#include <iostream>
using namespace std;
```

```
int recursiveSum (int x) {
  // Base case
  if (x == 0) {
    return 0;
  } else {
    // Recursive step
    return x + recursiveSum (x - 1);
  }
}
int main() {
  cout << recursiveSum (10);
  // Answer is 55
  return 0;
}
```

In this scenario, the process involves gradually adding values to the total variable as you iterate through a loop. However, a different approach involves leveraging computational thinking to deconstruct the problem, breaking it down into smaller subcomponents. This method tackles these subcomponents individually to address the overarching issue. When these smaller parts represent scaled-down versions of the original problem, recursion becomes a valuable tool.

In practical scenarios, recursion often manifests as a **function**, which is a set of commands that can be repeatedly executed. It may accept an input and may return an output. The base case represents the function's most straightforward operation for a given input. To effectively implement recursion, two primary steps must be followed: (a) identify the base case, and (b) outline the recursive steps. In the context of a recursive function, when *n* is 0, the cumulative sum from 0 to 0 is intuitively 0, representing the most fundamental subproblem of the main issue. Armed with this base case, you can commence crafting the initial part of the function.

```
int recursiveSum (int x) {
  // Base case
  if (x == 0)
    return 0;
}
```

Recursion operates through a process of simplification, progressively reducing the value of *x* until it meets the base condition, where *x* equals 0. This technique presents an alternative method, offering a refined and effective algorithmic solution for the current problem:

```
#include <iostream>
using namespace std;


int recursiveSum (int x) {
  // Base case
  if (x == 0) {
    return 0;
  }
  else {
```

```
    // Recursive step
    return x + recursiveSum (x - 1);
  }
}
int main() {
cout << recursiveSum(10); // Output will be the sum = 55.
  return 0;
}
```

While it looks like recursion amounts to calling the same function repeatedly, it is only partially true, and you should not think about it that way. What happens is much more than repeating the call of a function. It is more useful to think of it as a chain of deferred operations. These deferred operations are not visible in your code or your output—they are in memory. The program needs to hold them somehow, to be able to execute them at the end. In fact, if you had not specified the base case, the recursive process would never end. Figure 2.9 illustrates a flowchart for an iterative solution that adds *N* numbers.

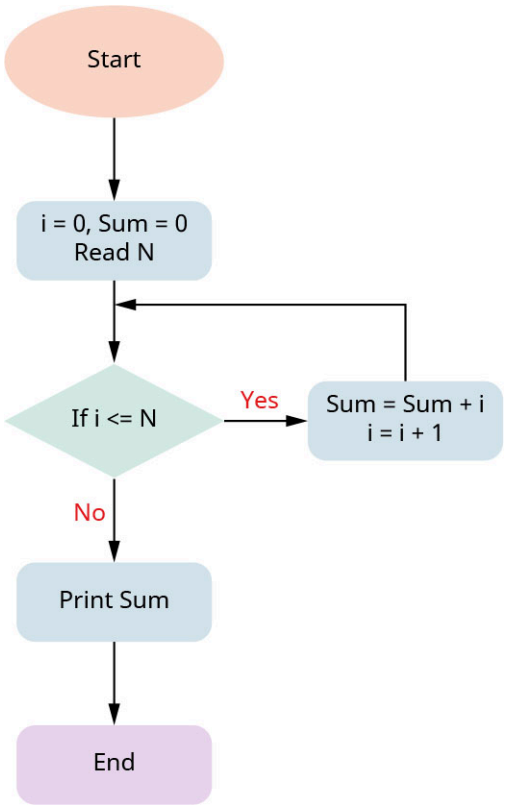


Figure 2.9 A flowchart represents an iterative solution for adding *N* numbers. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

CONCEPTS IN PRACTICE

Computational Thinking for Chess Problem-Solving

Computers can be used to help us solve problems. However, before a problem can be tackled, the problem itself and how it could be solved need to be understood. Computational thinking transcends mere programming; it doesn't equate to thinking in the binary fashion of computers, as they fundamentally lack the capacity for thought. Rather, while programming is the craft of instructing a computer on the actions to execute, computational thinking empowers you to meticulously determine what those instructions should

be. Take, for instance, the strategic gameplay involved in chess. To excel in a chess match, a player must:

- Understand the unique movements and strategic values of each piece, recognizing how each can be maneuvered to control the board.
- Visualize the board's layout, identifying potential threats and opportunities, and planning moves several steps ahead to secure an advantageous position.
- Recognize patterns from previous games, understanding common tactics and counters, to formulate a robust, adaptable strategy.

In devising a winning strategy, computational thinking is the underpinning framework:

- The complex game is dissected into smaller, more manageable components (e.g., the function of each chess piece, the state of the board)—this is decomposition.
- Attention is concentrated on pivotal elements that influence the game's outcome, such as the positioning of key pieces and the opponent's tendencies, sidelining less critical factors—this is an abstraction.
- Drawing from prior knowledge and experiences in similar scenarios, a step-by-step approach is developed to navigate through the game—this is algorithmic thinking.

Should you venture into developing your own chess program or strategy, these are precisely the types of considerations you would deliberate on and resolve before actual programming.

Testing and Debugging

Testing and debugging are techniques used to identify flaws in algorithms and defects in code to be able to correct them. Test cases rely on providing specific input data to check whether a software program functions correctly and meets its designed requirements. Test cases need to be identified to drive tests. If a test associated with a test case fails, debugging needs to be conducted to identify the source of the problem and correct it. In other words, debugging is about locating and fixing defects (i.e., bugs) in algorithms and processes to make them behave as expected. In programming, everyone makes mistakes, they are part of the learning process. The important thing is to identify the mistake and work out how to overcome it. There are those who feel that the deepest learning takes place when mistakes are made.

In the jam sandwich algorithm, testing can be facilitated by taking turns to play the role of the programmer who gives instructions as well as the robot. If you are a programmer, your job is to read out the instructions and follow each step. You can choose to follow your pseudocode or your flowchart. Each instruction becomes a test case, and the test succeeds if the robot can follow every instruction exactly and successfully. In the alternative, you will need to debug the instruction to identify the source of the problem and correct it. Table 2.2 can be used to record problems encountered and improvements that need to be made.

Test Case	Input	Problem	Expected Outcomes	Observed Outcomes	Improvement	Responsible User
1.						
2.						

Table 2.2 Sample Table for Recording Test Problems and Improvements

INDUSTRY SPOTLIGHT

DNA Sequencing

Computational thinking is important in every industry today. In DNA sequencing, computational thinking helps manage the massive and complex data involved. It starts by breaking the large DNA sequence into smaller pieces. Then, it involves identifying patterns or sequences within these pieces, which might indicate genetic information like the presence of certain genes. The focus is on the most relevant parts of the sequence, discarding unnecessary data to concentrate on potentially significant genetic regions. Finally, refined algorithms process and reconstruct the original sequence to identify genetic variations. This approach is used for efficiently handling massive datasets in DNA sequencing and extracting meaningful insights. The parts of computational thinking (CT) can be identified and highlighted in the process of DNA sequencing, a complex task within the field of genomics:

- Decomposition: Break down the DNA sequencing process into specific steps such as sample collection and DNA extraction.
- Pattern recognition: Identify similarities in DNA sequences that could indicate genetic traits or anomalies.
- Abstraction: Focus on the essential parts of the genetic information that are relevant for the study at hand.
- Algorithms: Create step-by-step protocols for each part of the sequencing process.
- Logical thinking: Determine the most accurate methods for sequencing based on the type of sample and the required depth of sequence analysis.
- Evaluation: Assess the quality and accuracy of the sequencing data obtained.
- Debugging: Identify issues that may arise during the sequencing process.

Practical Computational Thinking Examples

Here are different real-life scenarios of practical applications of computational thinking with suggested solution approaches to provide problem-solving and decision-making:

- Organizing a city's recycling program to maximize efficiency. How can you ensure the most effective collection routes and times?
- Solution: Use a route optimization algorithm to analyze and plan the most efficient paths for collection trucks, considering factors like distance and traffic patterns.

- Planning the layout of a community garden to optimize space and sunlight exposure for different plant types. How do you decide where to plant each type of vegetable or flower?
- Solution: Employ a simulation algorithm that models sunlight patterns, plant growth rates, and space requirements to design a garden layout that maximizes space and plant health.

- Creating a schedule for a multistage music festival to minimize overlaps and ensure a smooth flow of audiences. How do you schedule the performances across different stages?
- Solution: Implement a scheduling algorithm that considers audience preferences, artist availability, and stage logistics to create a timetable that maximizes attendee satisfaction and minimizes conflicts.

- Determining the most efficient way to allocate computer resources in a cloud computing environment to handle varying user demands. How do you manage the computational load?
- Solution: Use load balancing algorithms to distribute tasks across servers dynamically, ensuring optimal resource utilization and maintaining system performance.

2.2 Architecting Solutions with Adaptive Design Reuse in Mind

Learning Objectives

By the end of this section, you will be able to:

- Describe how business solutions design heuristics and how patterns are used
- Discuss the role of enterprise architecture (EA) and related architecture domains
- Differentiate between enterprise and solution architecture

While computational thinking commonly employs a bottom-up strategy for crafting well-structured components, adaptive design reuse adopts a top-down methodology, emphasizing the creation and assembly of business solutions by combining existing design components. A **design component** is a reusable element within a larger system that serves a specific purpose. These components, akin to low-level solution building blocks, necessitate minimal modifications. This approach, often termed *computing in the large*, diverges from individual algorithmic instructions. Instead of composing instructions for the execution of specific actions through computational thinking's decomposition, adaptive design reuse identifies fitting components based on the articulated requirements of the business solution's potential users, commonly referred to as stakeholders. The method(s) used to identify these needs will be discussed in detail in Chapter 9 Software Engineering. While adaptive design reuse typically considers algorithmic designs as building blocks, it uses similar techniques (i.e., decomposition, logical thinking and pattern recognition, abstraction/generalization, componentization, testing, and debugging) to identify and reuse building blocks.

The structural designs that stem from the top-down adaptive design reuse approach to business solution design are referred to as *business solutions architecture models*. A business **solution architecture** is a structural design that is meant to address the needs of prospective solution users. When a structural design meets these needs, it is considered "architecturally sound" for the problem at hand. Furthermore, to accelerate the putting together of complete solutions, the adaptive design reuse approach relies on architectural models or component assemblies that were developed from previous designs. When the granularity of these architectural models is at the level of subsystems, they are referred to as system family architectures or architectural patterns. An **architectural pattern** is a reusable solution to a recurring problem in software architecture design. Chapter 10 Enterprise and Solution Architectures Management provides more detail about the various levels of patterns and how to organize them and bookkeep them into pattern catalogs to facilitate reuse.

Business Solutions Design Patterns

Business solutions are strategies/systems created to solve specific challenges in a business. They aim to make operations more efficient, improve decision-making, and contribute to overall success. Creating business solutions is a multifaceted and intricate process, demanding knowledge in different technological domains and the relevant business sector. A crucial step in this procedure is outlining the solution's architecture, serving as a master blueprint, and guiding the entire design and implementation process. A **blueprint** is a detailed plan or design that outlines the structure, components, and specifications of a building, product, system, or process.

Computational thinking and adaptive design reuse are simply methods used to bootstrap and/or accelerate the design of business solutions by providing a comprehensive set of methods for developing software solutions to business problems—for example, gathering solutions needs, building, and deploying turnkey solutions. These solutions will be discussed in detail in Chapter 9 Software Engineering.

As explained earlier, the computational thinking and adaptive design reuse approach only provide high-level techniques to help create or reuse components. As a result, one of the typical criticisms is that these approaches are too vague, as it is sometimes not clear how they differ from other forms of thought. This is why experts at applying these methods are usually seasoned architects who can instinctively recognize when

various types of components may be reused. When you ask Thoughtworks' enterprise architecture expert Martin Fowler why a particular business solution architecture is sound in a given context, he will often reply that it simply "smells good" to him.

Rather than looking at computational thinking and adaptive design reuse as a best practice set of techniques, it is best to consider them as process patterns that may be applied and adapted to help create a business solution **architecture model**, which represents the content of the application architecture. Process patterns may, in turn, leverage other process patterns that are more focused. When process patterns that are an inherent part of the process of creating a work product become rules of thumb, they are referred to as heuristics.

INDUSTRY SPOTLIGHT

Case Study: Making Online Shopping Easier with Smart Design

Imagine an online store called SwiftShop. They had a problem. Even though they had lots of great products to buy, people were leaving their website without buying anything. It was like having a store full of customers who walked in and out without buying anything! The business challenge is SwiftShop wanted to make shopping on their website easier and more fun. They wanted people to stay longer, buy more, and come back again and again. SwiftShop decided to use smart design tricks to fix their website and make it super easy to shop. Here's how they did it:

- User interface (UI) design patterns: Breadcrumb navigation: SwiftShop added breadcrumb trails so shoppers could easily see where they were on the website and find their way back if they got lost.
- Progress indicators: During checkout, SwiftShop put in progress bars so shoppers knew how far along they were in the buying process.
- Information architecture (IA) design patterns: Card sorting: SwiftShop asked real shoppers to help organize their products. By sorting cards and asking people how they'd group things, SwiftShop made it easier to find what customers were looking for.
- Faceted search: SwiftShop added filters so shoppers could narrow down their searches by attributes like price, size, and brand.
- Interaction design (IxD) patterns: One-click purchase: To speed up the buying process, SwiftShop let registered users buy with just one click.
- Personalized recommendations: SwiftShop used machine learning algorithms to suggest products that customers might like based on what they've bought before.
- Results: After making these changes, SwiftShop saw great results:
  - More people buying: With the new features, more people ended up buying products from SwiftShop.
  - Happier shoppers: Customers spent more time on the website, which meant they liked shopping there more.
  - More repeat customers: Because it was easier to shop, people kept coming back to SwiftShop again and again.

So, by using smart design tricks like breadcrumb navigation, progress indicators, card sorting, faceted search, one-click purchase, and personalized recommendations, SwiftShop made their online store a better place to shop.

Layering and Componentization

Two heuristics are inherent to the design of business solutions and the creation of business solution architectures. These heuristics are known as layering and componentization and can be thought of as design approaches followed with an intent of architectural concerns.

Componentization has already been introduced as a computational thinking and adaptive design reuse technique.[1] Layering in business solution architecture involves creating distinct layers that abstract specific aspects of the overall architecture. This **heuristic** enables the separation of concerns between layers and improves modularity. Each layer comprises components, and a layer may consist of multiple components. This hierarchical structure helps organize and separate different functionalities or responsibilities within the architecture, making it easier to manage and understand.

The layering strategy is based on the concept of separating different concerns. This method structures software design into distinct, stacked layers, with each layer tasked with specific functions. Commonly, business solution architectures are organized into three main layers: the presentation, business logic, and data management layers. The **presentation layer** is the user's touchpoint, handling the user interface (UI) and delivering the **user experience (UX)**, which is the overall experience that a person has when interacting with a product, service, or system. The **business logic layer** holds the business logic for the business solution or application, separating the UI/UX from the business-centric computations. This separation affords the flexibility to adjust business operations as needs evolve without disrupting other system components. Although not tied to a specific domain, the **data management layer** is responsible for interacting with persistent storage systems like databases and various data processing mechanisms. In a layered architecture, information and commands flow through each layer, enhancing the design's abstraction level, which denotes the granularity or detail at which a system is represented. Despite its structured approach and abstraction benefits, software designed in this layered manner might lean toward a monolithic build, potentially making modifications challenging. Layered architecture can lead to tightly connected layers in software, making it difficult to change one part without affecting others. This tight connection can also make it harder to update or expand the software.

A **monolithic structure** is a type of system or application where all the parts are closely combined into one single unit. This setup means that everything from the user interface to data handling and processing is interconnected within one big software application. While this can make the system easier to set up and manage initially, it also means that changing one part can require changes throughout the whole system, making it less flexible. As illustrated in Figure 2.10, there are many more recent variations of layered architectures that also provide complementary capabilities, such as tiered architectures, service-oriented architectures (SOA), and microservices architectures.[2]

1 Componentization was used initially in component-based business solution architectures that were derived from the Object Management Group's Object Management Architecture (OMA), including CORBA 3, JEE, DCOM, and COM+.

2 Per an article written by Thoughtworks Martin Fowler in 2014, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.

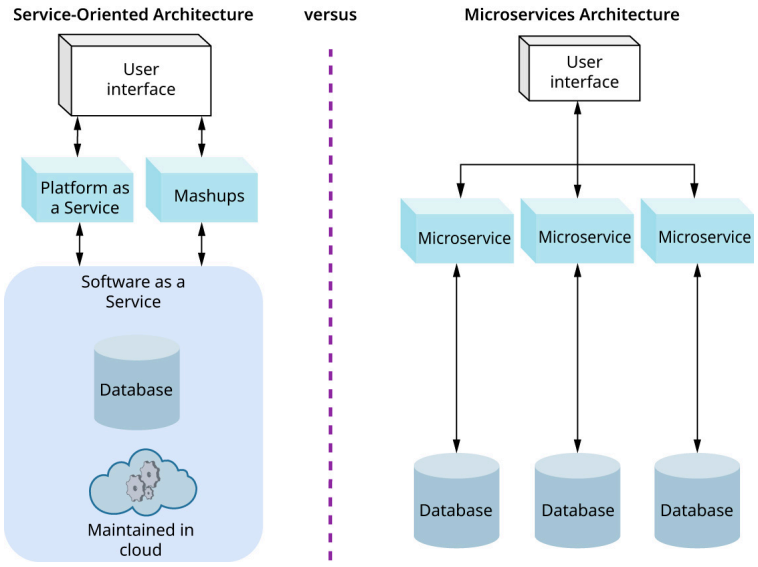


Figure 2.10 The different layered architectures include tiered architectures, service-oriented architectures (SOA), and microservices architectures. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Detailed coverage of these specific architectures is not part of the scope of this chapter. They will be discussed in more detail in multiple later chapters. It's fundamental to understand that each software architecture model is crafted to address the key challenges found in its preceding models. Being well-versed in various architectural approaches equips you to devise a robust and effective architecture for your specific project. While no software architecture can claim absolute perfection, an approach can be deemed highly suitable or "relatively perfect" if it aligns well with the specific requirements and goals of your current project.

Enterprise-Level Architecture Domains

Enterprise-level architecture encompasses various domains that define the structure, components, and operations of an entire organization. These domains provide a comprehensive framework for managing an enterprise.

Up to this point, our efforts have been centered on employing the adaptive design reuse methodology to construct business solution architectures tailored to individual projects. Within this scope, we aim to facilitate the conversion of solution requirements into a cohesive solution concept, comprehensive business and IT system outlines, and a collection of implementation activities, essentially forming the project plan. However, because the adaptive design reuse approach is top-down, it is possible to start applying it at a higher level. The enterprise level is typically the highest level of an organization, and it covers all strategic and tactical functions. An enterprise often spans multiple organizations.

Enterprise architecture (EA) emerged at the beginning of the information age in the 1970s, and various enterprise architecture frameworks (EAFs) were developed and used over time. The Open Group Architecture Framework (TOGAF) is one such framework. According to TOGAF, an **EA domain** represents business, data, application, and technology architectures. While specific frameworks may vary, Table 2.3 illustrates the common enterprise-level architecture domains.

Architecture	Purpose	Components
Business architecture	Defines the organization's business strategy, goals, processes, and functions	Business models, processes, capabilities, and organizational structure
Information architecture	Manages data and information assets	Data models, information flow diagrams, data governance, data standards, and metadata
Application architecture	Designs and organizes software applications	Application portfolio, application integration, and interface design
Technology architecture	Specifies the hardware, software, and technology infrastructure	Servers, networks, databases, cloud services, security protocols
Security architecture	Ensures the protection of information assets, systems, and networks	Security policies, access controls, and encryption mechanisms
Integration architecture	Facilitates seamless communication and data exchange	Middleware, messaging systems, and integration patterns
Process architecture	Defines and optimizes business processes	Process models, workflow diagrams, and performance metrics

Table 2.3 Common Enterprise-Level Architecture Domains

EA adopts a holistic perspective, treating the enterprise as a unified system or a system of systems. Its primary aim is to integrate disparate processes, both manual and automated, into a cohesive environment that is responsive to change and aligned with the enterprise's business strategy. EA involves designing comprehensive systems to support business processes, going beyond individual software or technology systems. By standardizing and integrating fundamental processes across various business divisions, EA facilitates enterprise-wide alignment and optimization of operations.

Solution architecture complements EA by tailoring specific system solutions to meet defined business and technological needs. While EA focuses on defining the present situation and future goals of the organization, solution architecture ensures the timely availability of suitable systems to fulfill business requirements.

TOGAF architectures promote the reuse of building blocks but lack a prescriptive method for managing them. Adaptive design reuse becomes valuable in this context, involving understanding enterprise architectures and adapting preexisting models and implementations. Detailed management of EA, solution architectures, and leveraging EAFs is explored further in Chapter 10 Enterprise and Solution Architectures Management.

Enterprise Business Architecture and Model

The enterprise business architecture (EBA) is a comprehensive framework that defines the structure and operation of an entire organization. It is related to corporate business and the documents and diagrams that describe the architectural structure of that business. It characterizes the processes, organization, and location aspects of the EBA at hand. EBAs usually consist of different business capabilities grouped into categories, as illustrated in Figure 2.11. In this example, sample categories include product, sales, and service. The business capabilities shown under the various categories typically have business models of their own that are part of the overall organization's business model. Note that it is clear from this diagram that the layering and

componentization heuristics play an important role in structuring capability models.



Figure 2.11 The EBA framework categories, including product, sales, and service, support the business organizations. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Business architecture modeling helps extract the overall business model of an organization's capability, which includes the set of underlying processes necessary to operate the various capabilities of the organization.

A **business model** is a framework that outlines how a business creates, delivers, and captures value. It covers the way a company generates revenue, identifies its target audience, and defines the products/services it offers. The business model includes the organization, location, and process model, as described in the following sections. It may include additional models, such as the financial model, which will not be covered here. To illustrate what a business model entails practically, we will focus on a typical financial instruments trading capability, which could be one of the capabilities in the overall business model of an organization operating in the global markets industry. The organization, location, and process models are typically represented at a logical level of abstraction, which is the most detailed representation for these types of models. Note again that layering and componentization heuristics play an important role in structuring these models.

Organizational Model

The organizational model is the structure and design of an organization, outlining how roles, responsibilities, and relationships are defined. It provides a framework for understanding how various components within the organization interact and collaborate to support its functions. In addition, it offers clear definitions of roles and a matrix mapping of processes to roles.

Role definitions encapsulate a set of functional capabilities essential for an individual to manage one or several distinct business processes, as shown in Table 2.4. It is common for an individual to fulfill multiple roles. The process/role matrix delineates which business roles are accountable for particular business processes. Typically structured at the elementary business process level, this matrix can also be extended to more overarching levels when beneficial.

Role Title	Responsibilities	Reporting
DBA (database administrator)	Managing and monitoring the database activities	IT manager or VP of IT
CIO (chief information officer)	Overall IT leadership	CEO (chief executive officer)
HR employee	Human resources operations	HR manager

Table 2.4 Organizational Roles

In general, the role matrix is a visual representation or chart that outlines the various roles within an organization and their respective responsibilities. It helps in clarifying and communicating the distribution of

tasks and authorities among different positions or departments.

In a financial instruments trading business model, various roles contribute to the overall functioning of the organization, as shown in Table 2.5.

Title	Roles and Responsibilities
Traders	• Manage trading portfolios • Develop strategies to maximize profits and manage risk
Sales team	• Build and maintain client relationships • Understand client needs and offer suitable products and services
Back-office operations[3]	• Process and settle trades • Handle trade confirmation and reconciliation
Risk managers	• Monitor and manage risk exposure • Ensure compliance with risk management policies
Legal advisors	• Provide legal advice and services • Ensure compliance with laws and regulations
Technology professionals	• Develop and maintain trading systems and IT infrastructure • Ensure the security and efficiency of technology resources
Human resources	• Manage recruitment, training, and employee relations • Ensure the organization is staffed with skilled and motivated employees

Table 2.5 Roles and Responsibilities within a Financial Organization

In addition to these titles, there are two primary management layers: upper management and systems management. Upper management is tasked with the supervision of various sectors within the organization, encompassing numerous business units, operations departments, and other key areas. This tier includes roles such as division managers, group managers, and risk managers, who collectively ensure the smooth operation and strategic direction of the business. On the other hand, systems management is dedicated to the routine functioning of the trading system. This includes positions like site manager, systems maintenance manager, content manager, and help desk personnel, all of which collaborate daily to ensure the trading system is operational, well-maintained, and user-friendly.

Process Model

The business process is a series of interrelated tasks, activities, or steps performed in a coordinated manner within an organization to achieve a specific business goal. The business process model can be encapsulated within a framework of business process hierarchies. These hierarchies visually illustrate the outcome of breaking down complex processes, a method known as process decomposition. This decomposition is carried out until it reaches the **elementary business process (EBP)**, which is a fundamental and indivisible activity

3 Often referred to as "back-office personnel." A distinction may be necessary between back-office and front-office functionality. Support—Front-office operations: Typically work alongside traders and salespeople. They facilitate telephone calls, process trades, and resolve trading and sales/customer issues. Front-office personnel would consist of assistant traders and salespeople.

within a business that is not further subdivided into smaller processes (i.e., the smallest unit of business activity). Figure 2.12 illustrates an example of business process modeling for creating an order.

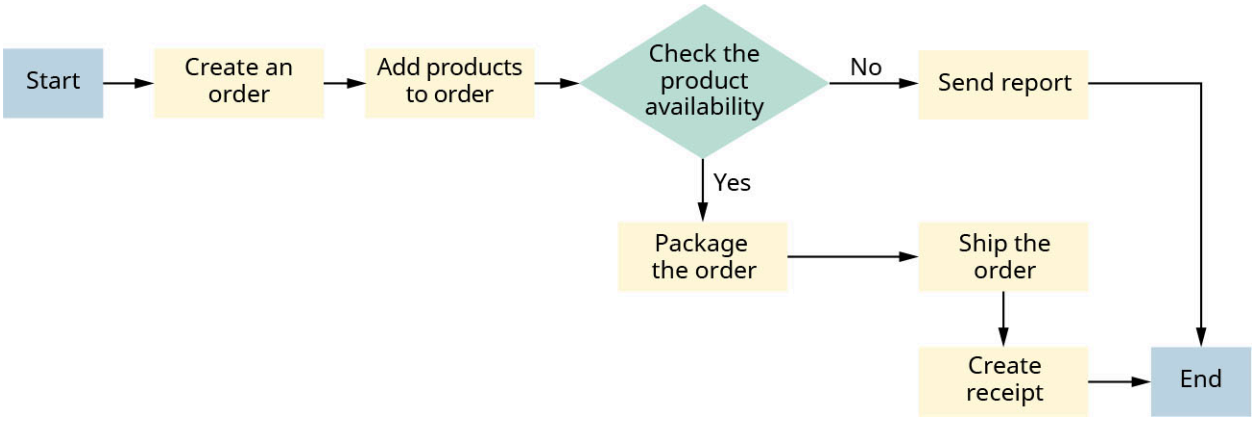


Figure 2.12 This flowchart outlines the business process model for creating an order, checking availability, shipping, and payments. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

EBPs possess distinct characteristics that make them noteworthy in the context of business operations. They are considered significant and candidates for in-depth analysis. EBPs are typically associated with the actions of a single user, emphasizing a focused responsibility within the organization. Furthermore, these processes may encompass both manual and automated activities, reflecting the diverse nature of contemporary business workflows. User involvement in EBPs is concentrated at a specific point in time, streamlining the temporal aspect of these activities. The preservation of crucial business distinctions between processes ensures clarity and coherence. Finally, EBPs aim to leave the conceptual entity model in a consistent state, contributing to data integrity and maintaining a reliable representation of the organization's entities and relationships where the **entity model** represents the various objects or concepts and their relationships within a system. A **process map** displays the order of chosen processes from the process hierarchies, highlighting their connection to the roles responsible for executing them. A **business process hierarchy** organizes a company's activities from broad, general processes down to specific tasks, making it easier to manage and improve how the business operates. Figure 2.13 illustrates a high-level business process hierarchy that could be used for any manufacturing business process.

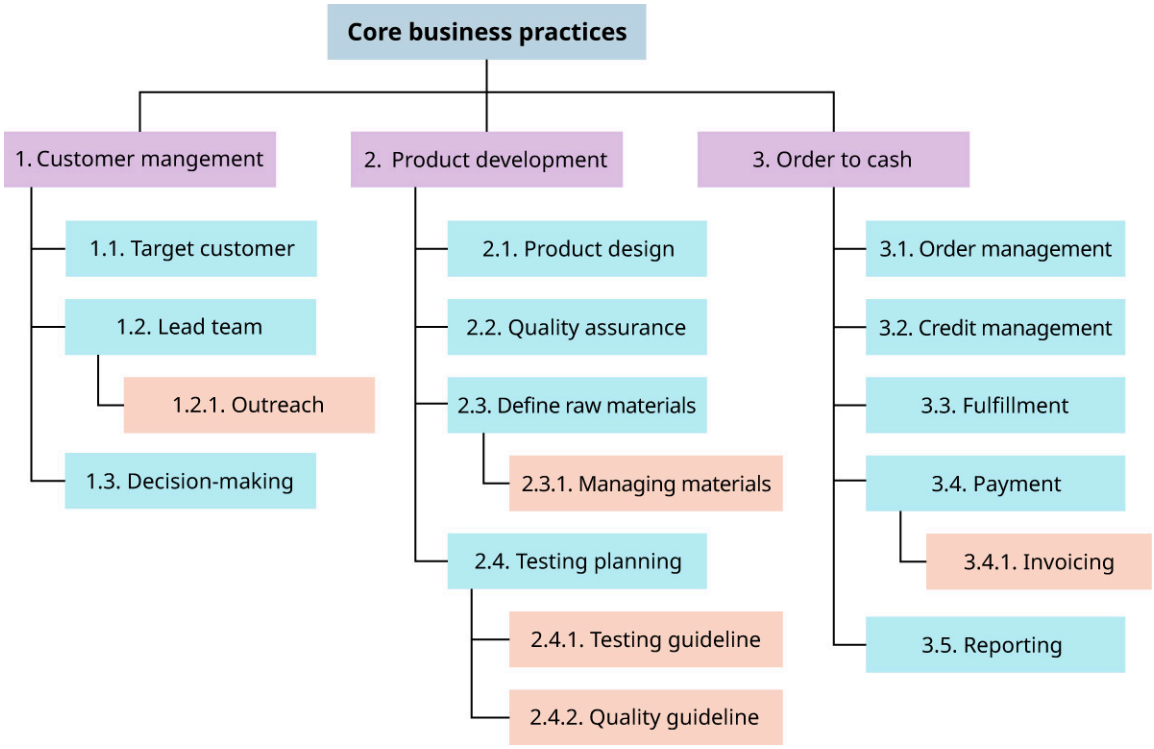


Figure 2.13 The trading business process hierarchy includes activities such as customer management, product development, quality control, order fulfillment, and customer payments. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

At the top of the hierarchy, there are three core business processes:

1. Customer management: This process plays a central role, interacting with various activities designed to acquire, retain, and develop customer relationships. It focuses on all aspects of customer interaction. There are three subsections as follows:
    - 1.1. Target customer: Customers first discover a company and its offerings through various marketing efforts. They then evaluate these offerings to decide if they meet their needs, leading to a purchase decision. After buying, customers may require support, which is provided to ensure satisfaction. A positive experience can turn these customers into loyal buyers and advocates for the company and maintain strong customer relationships.
    - 1.2. Lead team: This involves identifying potential leads and assessing their likelihood to purchase. It includes:
        - 1.2.1. Outreach: This includes reaching out to customers (current and new) with promotions and advertisements.
    - 1.3. Decision-making: This step focuses on the creation and evaluation of product prototypes to ensure they meet the required standards before launch.
2. Product development: This encompasses the entire life cycle of a product from initial idea generation through design and development to launch. There are four subsections:
    - 2.1. Product design: This focuses on the creation and evaluation of product designs to ensure they meet the required standards before testing.
    - 2.2. Quality assurance: QA involves checking the quality of the design and matching it with the listed requirements.
    - 2.3. Define raw materials: Raw materials are the basic substances needed to make products. These can be things taken from nature, like wood, oil, or metals, which are used in making everything from buildings to clothes and food. The type of raw material used affects how products are made, their quality, and how much they cost. It includes:
        - 2.3.1. Managing materials: Managing materials well is important for businesses to make sure they

have enough to meet demand, keep production running smoothly, and reduce waste, making the whole process more efficient and sustainable.

- 2.4. Testing planning: Writing detailed instructions for setting up and conducting usability tests, including how to select participants and record feedback, is important and includes:
  - 2.4.1. Testing guidelines: Rules that help ensure a product or service works well and is safe before it's made available or used. These rules guide how to test the item, what tools to use, and what problems to look for, aiming to fix any issues found. The main goal is to make sure the product does what it's supposed to do and meets quality standards.
  - 2.4.2. Quality guidelines: These are rules that help ensure products or services are consistently good and meet customers' needs. By following these guidelines, companies aim to make their customers happy, reduce mistakes, and work more efficiently. This involves regularly checking and improving how things are done to make sure they meet high standards.

3. Order to cash: This is a comprehensive business flow that starts when a customer places an order and ends when the payment is received and recorded:
   - 3.1. Order management: The process begins when a customer places an order through a chosen method, such as an online platform, phone call, or sales representative.
   - 3.2. Credit management: A credit check is performed to ensure the customer has the necessary credit to make the purchase. If the customer passes the credit check, the order is approved for processing. Otherwise, it may be held until payment is secured or canceled.
   - 3.3. Fulfillment: The required products are allocated from inventory. If products are not available, this may trigger a back-order or manufacturing process.
   - 3.4. Payment: The customer makes a payment using one of the accepted payment methods.
     - 3.4.1. Invoicing: This step will start if the customer pays with a purchase order (i.e., the customer may use different payment methods such as credit card or money transfer). Once the order is shipped, an invoice is generated and sent to the customer, detailing the amount paid and/or due for the products or services provided.
   - 3.5. Reporting: Reports are generated to analyze the sales volume, payment collection efficiency, and customer buying patterns.

EBP definitions include brief explanations of the activities within each process. These descriptions, when paired with the appropriate process flow, serve as a foundation for progressing to intricate design stages and aid in the ongoing creation of functional specifications. Furthermore, they highlight any presumptions established during the architectural development, delineate rules for decisions that require manual intervention, and, where relevant, provide cross-referencing details to ensure compatibility with the functional demands of the application being developed.

The assignment of numbers to EBPs mirrors their placement within hierarchical structures, enabling their linkage to various work products throughout the business, organizational, and location-specific domains, along with pertinent models.

Process map diagrams illustrate the order of chosen processes from the process hierarchies, focusing on their connection to the roles responsible for executing them. The process maps specifically highlight key processes that hold particular business importance. Although every process featured in a process map is also included in the business process hierarchy, not all processes in the hierarchy are depicted in the process maps. Process flow diagrams can depict various elements, such as the business events triggering a process, outcomes of completing a process, the processes themselves, the sequence of process steps, interruptions in the process flow, possibilities for repetition, choices, exclusivity among processes, and any additional notes. Figure 2.14 illustrates a process map diagram for an order to cash business process.

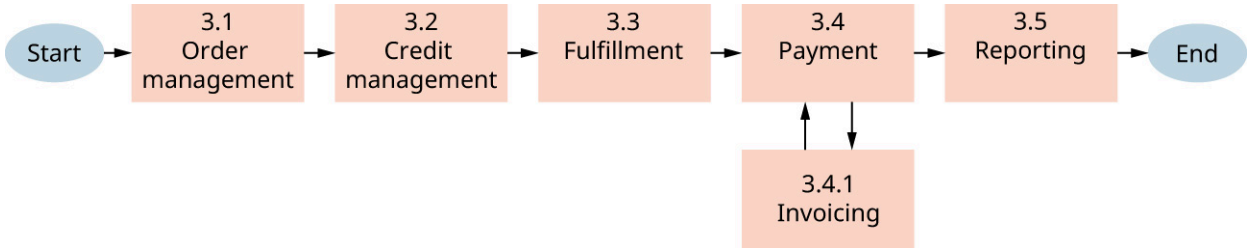


Figure 2.14 The order to cash business process flows from order management through product fulfillment and payment. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Figure 2.15 illustrates the process of managing orders within a business. It begins with order management, where orders are received and verified for accuracy. Next is credit management, which assesses the customer's credit to ensure they can fulfill payment requirements. The process then moves to fulfillment, where the order is prepared and shipped. Then, during payment, the customer is invoiced and payment is processed. Invoicing is a detailed part of this step, where an invoice is generated and sent to the customer. The final stage is reporting, where the business generates reports on sales and financial transactions. This process ends once all steps are completed, ensuring each order is handled efficiently from start to finish.

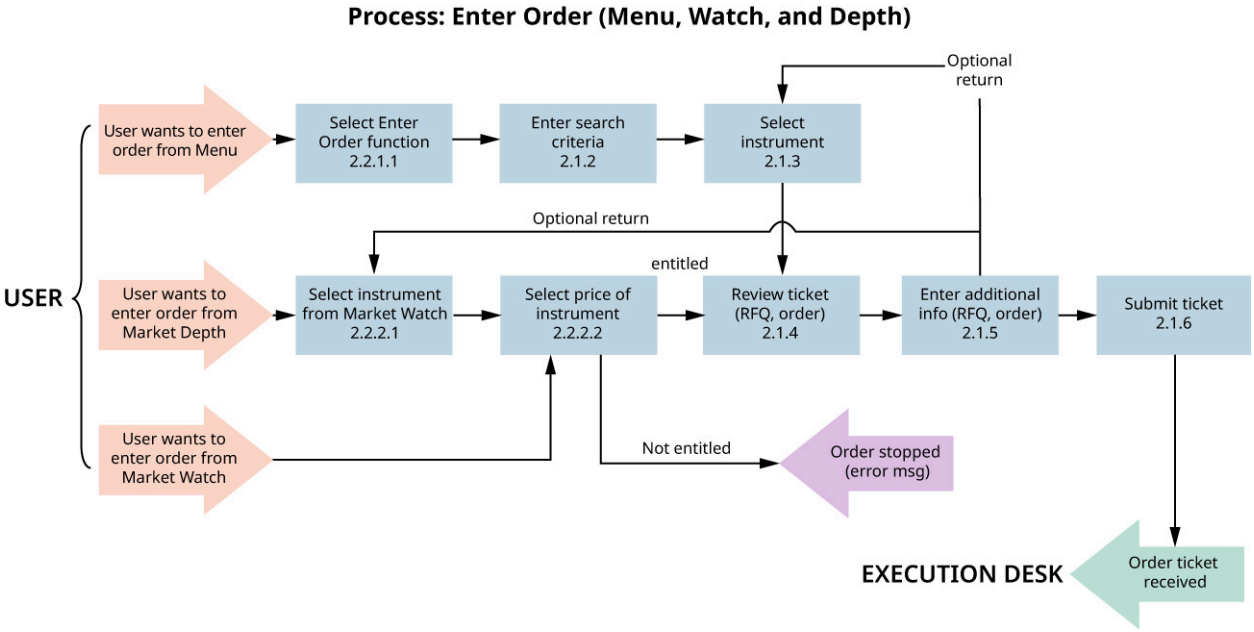


Figure 2.15 This sample trading business model represents a process map diagram for the enter order process. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Location Model

A location model refers to a set of rules used to analyze and make decisions related to the positioning of entities, activities, or resources. The conceptual location model shows how business processes will be distributed geographically. Within the location model, definitions of location types specify the identification of locations based on both their type and general geographic area. Additionally, the location model contains a matrix illustrating the relationship between processes and locations, indicating the specific location types where each process takes place.

Enterprise Technical Architecture

The enterprise technology architecture (ETA) is a comprehensive framework that defines the structure, components, and interrelationships of an organization's technology systems to support its business processes and objectives. In general, the ETA guides the development and support of an organization's information systems[4] and technology infrastructure. Therefore, it characterizes the organization's application, data, and technology infrastructure architectures and describes their models. The technology (virtual infrastructure) supports the execution of information systems services, which in turn support business functions and related services. Application, data, and technology infrastructure architecture models may be described via blueprints at different levels of abstraction, including presentation, conceptual, logical, and physical. Layering and componentization heuristics play an important role in structuring these models.

The abstraction level indicates the extent to which a design is distanced from tangible technological details. This level of abstraction can differ across various architectural fields, but four main levels are widely recognized:

1. Presentation: At this level, architecture is simplified into a form to streamline the communication of essential ideas, particularly to business executives. Distilling complex architectural diagrams into more straightforward, high-level visuals ensures that the core messages are conveyed effectively.
2. Conceptual: This level offers a more structured view of architecture, deliberately omitting many specifics. The rationale for not including certain details is that they may not be relevant to the diagram's intended purpose or are yet to be decided.
3. Logical: The architecture is depicted in detail but remains uncertain of specific technologies. It aims to give a full outline without committing to the particular technologies that will be employed for the final build.
4. Physical: This level focuses on the actual technological specifics used or to be used in the architecture's realization, making it the most concrete representation of the four.

CONCEPTS IN PRACTICE

Manufacturing Industry

Business operations are commonly articulated through a *value chain* framework, a concept originating from manufacturing practices. Analogous to the manufacturing industry where raw materials like steel are transformed into various components, each step in the value chain plays a role in creating a finished product, such as a car. In our approach, we've utilized the componentization heuristic to break down business operations into distinct steps constituting the elements of a value chain. Additionally, the layering heuristic has been applied to structure the value chain as layers of elements. Each layer in this model relies on the output of the preceding layer to perform its specific function. This methodology enhances the understanding of how value is sequentially added throughout the business process.

Application Architecture Model

The **application architecture** is a subset of the enterprise solution architecture that includes a process to architect and design the application architecture as well as the actual application architecture model, which represents the content of the application architecture. Application architecture helps organizations decide how to invest in new software and systems. It makes sure that any new software being looked at, designed, or put into use can work well with the systems the organization already has. This includes brand-new software, updates to old software, making old software better, and buying and updating software packages.

4 Information systems architectures combine the use of application and data architecture details and specify control and data flow necessary to operate the systems.

Figure 2.16 illustrates a conceptual application architecture model of the sample trading business model that was introduced earlier in this section. The goal of a conceptual application architecture is to demonstrate how members of the organization interact with the application architecture from different locations when invoking business processes. This example illustrates three distinct offices accommodating a diverse range of users, including management, trading, IT, and others.

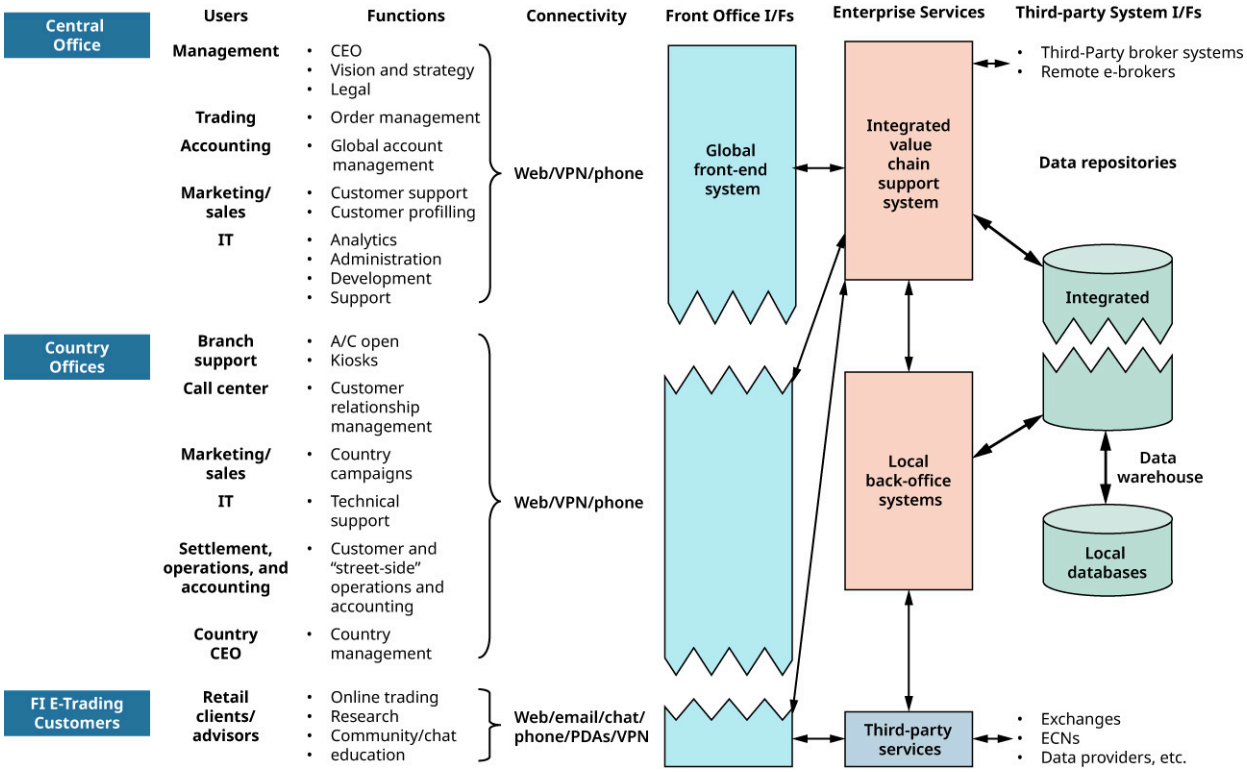


Figure 2.16 A conceptual trading application architecture moves through various levels, including users, functions, enterprise services, and third-party systems. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Figure 2.17 takes the conceptual trading application architecture a bit further and describes what one of the alternative application architectures may look like at the logical level. The goal of the logical application architecture is to identify the various functional blocks within the architecture without getting into the details of how they connect with and/or use each other.

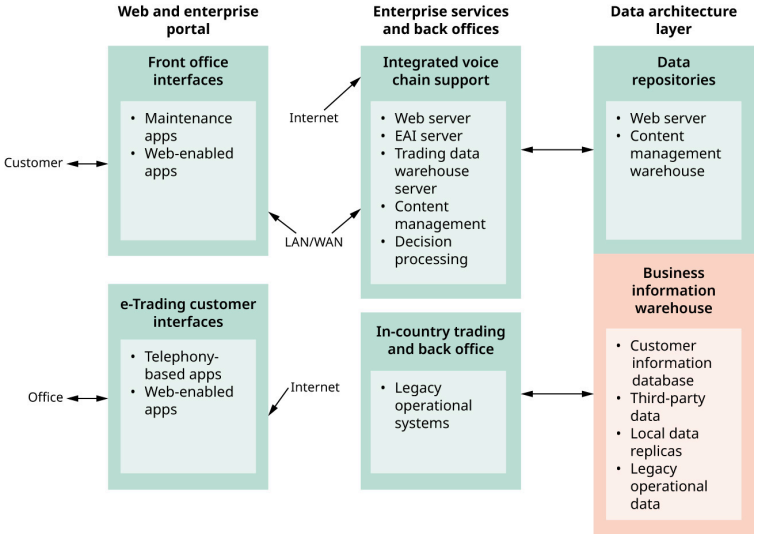


Figure 2.17 The logical trading application architecture illustrates each function block for the processing of customer orders. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

THINK IT THROUGH

Enterprise Business Architecture

What is the mechanism that prompts the creation of organization, location, and process domains to describe an (enterprise) business architecture? How does decomposition help describe (enterprise) technology architectures?

Hint: The organization domain focuses on the structure, roles, responsibilities, and relationships within the enterprise. The location domain deals with the physical or virtual places where business activities take place. The process domain delves into the business processes that drive the value chain and operational activities.

The process involves dividing a system or architecture into smaller, more discrete elements, which aids in analysis, understanding, and effective communication. Technology architectures in enterprises can be highly complex, involving numerous components and interdependencies. Decomposition simplifies this complexity by breaking down the architecture into smaller, comprehensible pieces. Each component can then be analyzed and understood independently.

Figure 2.18 is a simplified view of the logical trading application architecture model that provides callouts to explain what the various functions are meant to accomplish. It is good practice to document the functions in that way. It is also good practice to provide an additional diagram (not included here) that includes callouts identifying the actual third-party technologies that are used to implement the various functions.

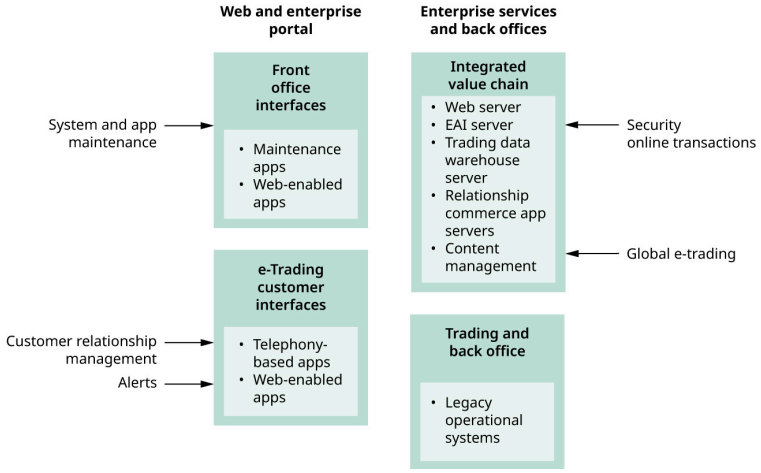


Figure 2.18 This logical trading application architecture includes function callouts that identify the technologies used for this process. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Data Architecture Model

A **data architecture model** is a conceptual framework that outlines how an organization structures, organizes, and manages its data assets. Data architecture forms a component of the broader enterprise solution architecture. It encompasses the design process for both information and actual data architecture models, representing the content within the data architecture. This framework aids organizations in strategically planning investments in data management solutions and associated systems. The evaluated, designed, and delivered data management solutions must seamlessly coexist with established ones, managing newly developed databases as well as legacy database extensions.

In general, information can be extracted from raw data, knowledge can be gleaned from information, and wisdom can be obtained from knowledge. Most enterprises refer to the relationship between data, information, knowledge, and wisdom as the *pyramid of knowledge*. A wealth of additional information related

to data management and the pyramid of knowledge is provided in Chapter 8 Data Management. Information modeling is the process used to describe the metadata necessary to understand the data, processes, and rules that are relevant to the enterprise, as illustrated in Figure 2.19.

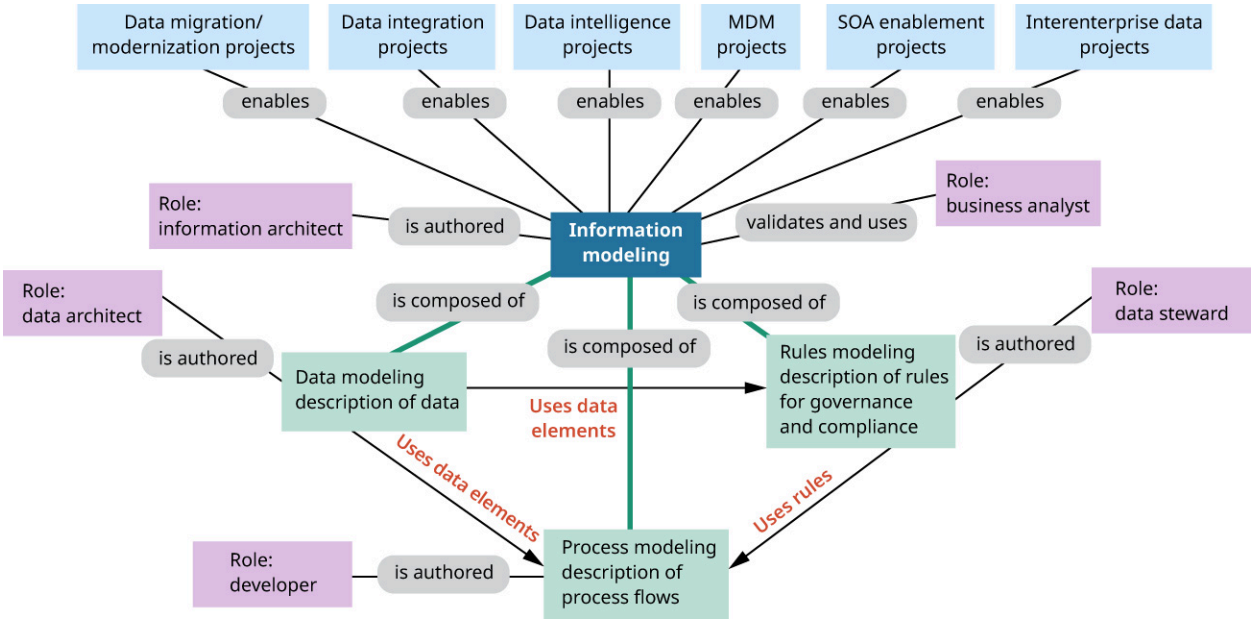


Figure 2.19 Examining and learning from data is integral to an organization's success, as outlined in this role of information modeling figure. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

In the collaborative process of **data modeling**, IT and business stakeholders establish a shared understanding of essential business terms, known as entities, which typically end up being represented as tables that contain data in a relational database management system. This involves defining the attributes that characterize these terms and establishing the relationships between them. The capability to uphold and document the data model is integral to an organization's capacity to address varied data acquisition needs across critical business projects. In essence, a well-maintained data model serves as a foundational element for ensuring coherence and effectiveness in managing diverse data requirements.

Figure 2.20 is an example of an enterprise-level conceptual data architecture for an insurance company. The goal of the enterprise conceptual data architecture is to illustrate the various types of data repositories and the way data is collected and managed within the enterprise.

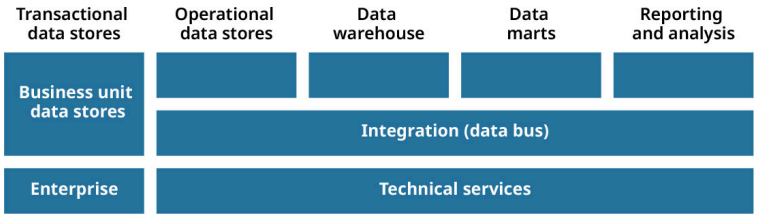


Figure 2.20 The enterprise conceptual data architecture for a fictitious insurance company highlights the different ways in which data is handled and managed. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Concerning the data that flows through the application architecture, there may be many types of data (e.g., relational and NoSQL) and various ways to represent the corresponding data architecture models. A relational database organizes data into tables where each row represents a unique record and each column represents a property of that record. It uses a language called SQL to manage and query the data. NoSQL databases are designed to store and manage data that doesn't fit neatly into tables and can handle various types of data models like documents or graphs.

INDUSTRY SPOTLIGHT

Design Reuse

Adaptive design reuse plays a crucial role across various industries today, including the health-care sector, where its impact is profoundly significant. Understanding and leveraging adaptive design reuse in health care can lead to innovative solutions for complex medical problems, enhancing patient care and treatment outcomes. One prime example of its application is in the design of artificial valves that can replace natural heart valves in people.

Heart valve disease is a condition where one or more valves in the heart do not function properly, leading to disrupted blood flow. The traditional solution involves surgical replacement with prosthetic valves, which can be derived from biological sources or made from synthetic materials. Adaptive design reuse in this context refers to the innovative process of designing these prosthetic heart valves by repurposing existing materials, technologies, and design principles from within or outside the medical field. This approach can accelerate the development of more effective, durable, and safer heart valve replacements.

Infrastructure Architecture (Infrastructure Pillars)

Technology architecture is a fundamental component of enterprise architecture, supported by four main pillars: compute, memory, storage, and network. It outlines the organization and functionality of an enterprise's solutions or system's technology framework. This encompasses the configuration of client and server hardware, the applications operating on this hardware, the services these applications provide, and the protocols and networks facilitating communication between applications and hardware components. It's important to distinguish technology architecture from system architecture. The **system architecture** deals with applications and data, how they are related to each other, and what business process they support together. The **technical architecture** includes the software and hardware capabilities to fully enable application and data services.

Refer to Figure 10.36 for an example of an enterprise-level conceptual technology architecture for a fictitious company. The goal of the enterprise conceptual technology architecture is to illustrate the various types of hardware components that are part of the enterprise infrastructure and the way they are laid out at a high level.

GLOBAL ISSUES IN TECHNOLOGY

Design Reuse Broader Impacts

Focusing on reusing existing designs and technologies can save time and money, but it might also limit new ideas and innovations because designers could stick too closely to what's already been done. This approach can have several effects:

- Socially, it might not meet the needs of all users, especially if the technology doesn't consider different cultures or lifestyles, leading to some people being left out.
- Ethically, there's a question about fairness and whether technology serves everyone equally, as relying on old designs may not address current or future challenges well.
- Environmentally, while using existing designs could reduce waste and save resources, it might also keep using outdated, less eco-friendly technologies instead of developing cleaner, more efficient options.
- Economically, countries that already have a lot of designs and technologies could get ahead because they have more to reuse, making it harder for countries with fewer resources to catch up or compete.

While reusing designs has its benefits, it's important to also think about these broader impacts and strive for a balance between recycling old ideas and creating new ones to make sure technology keeps improving in a way that's good for everyone.

Figure 2.21 illustrates a possible physical architecture for the sample trading business model that was introduced earlier in this section. This diagram depicts the layout of the actual hardware components that make up the infrastructure of the trading solution. It also delineates where the functional blocks of the application architecture are physically deployed. Note that this physical technology architecture leverages the layout and components of the enterprise application architecture illustrated previously at the conceptual level.

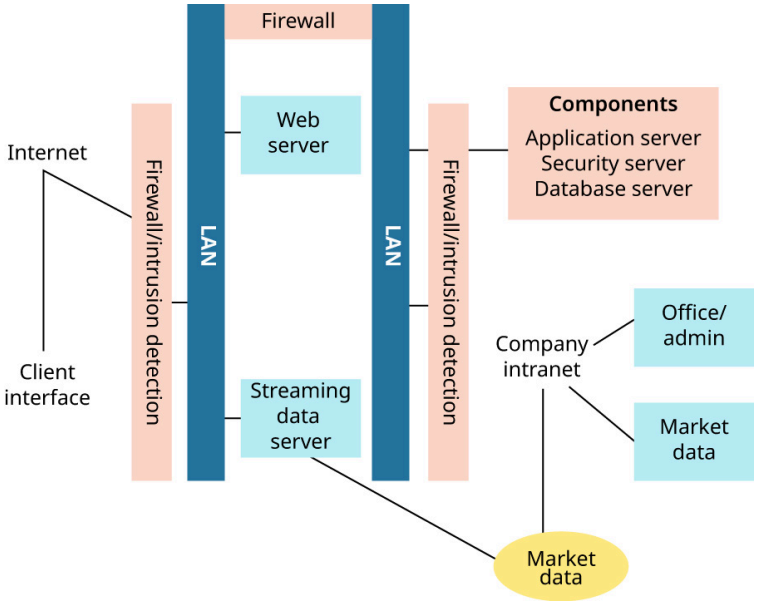


Figure 2.21 The physical trading application architecture highlights the use of hardware to meet the organization's goals. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

An alternative physical application architecture for the trading solution is shown in Figure 2.22. The diagram does not delineate where the functional blocks of the application architecture are physically deployed.

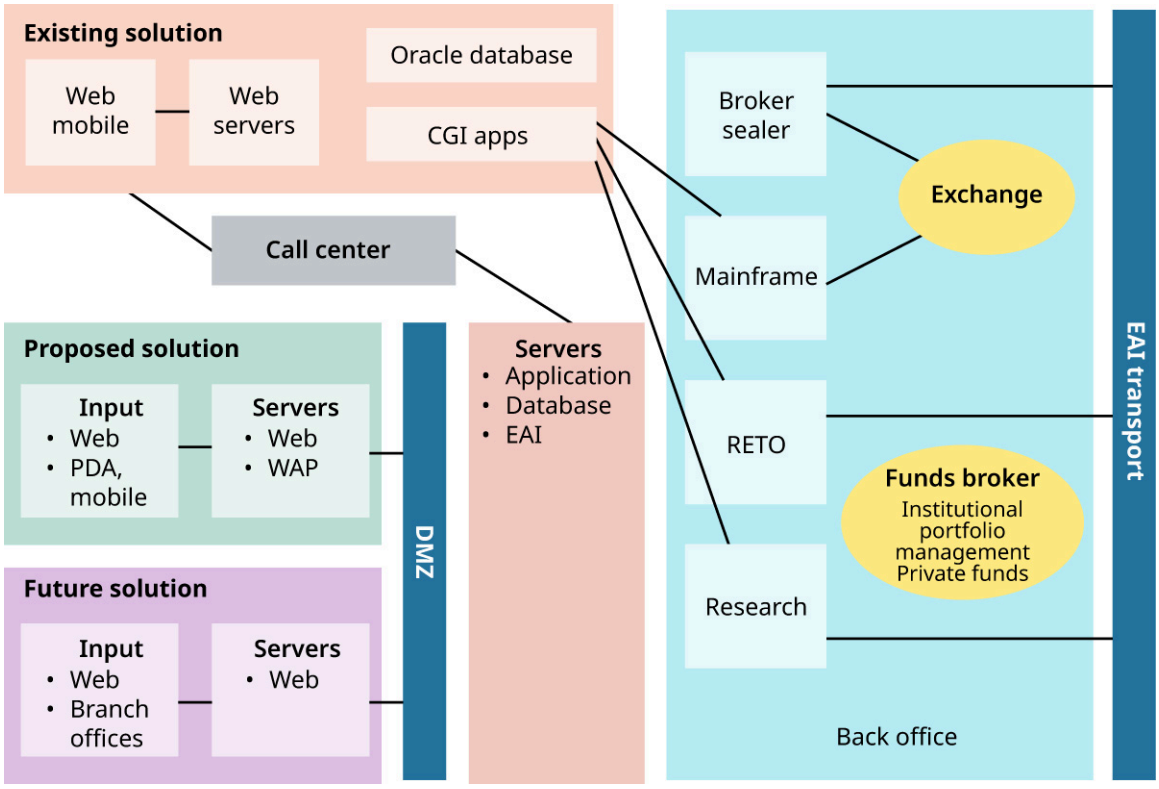


Figure 2.22 This alternative physical trading application architecture outlines possible changes from the existing web solution. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Information Systems Architecture View

Architecture views are representations of the overall system design that matter to different stakeholders. In addition to the application, data, and technology architecture models, IT architects create specific views to communicate and ensure that the system meets the needs of various stakeholders. An architecture is typically conveyed through one or more architecture models that collectively offer a clear description of the system's structure. A singular, all-encompassing model is often too intricate to be easily grasped, displaying every intricate relationship among diverse business and technical elements.

Just as an architect designs different aspects of a building, such as wiring diagrams for electricians, floor plans for owners, and elevations for planners to address their unique needs, the architecture of an information system is similarly broken down into various views for effective communication among its stakeholders. In the IT world, for example, an architect might create specific views of the system's physical layout and security measures. These tailored views ensure that stakeholders, each with their own concerns and areas of focus, have a clear understanding of the system's components relevant to their interests.

From Enterprise to Solution Architecture

After identifying business and technical characteristics through the diagrams discussed in the previous section, solution models can be developed, and implementations can be created. This involves constructing new components as necessary and combining them with reusable design components obtained from a pattern catalog. If implementations of these reusable components already exist and can be customized, the implementation of the solution becomes much faster. This approach helps avoid reinventing the wheel and developing software components or systems that already exist, focusing instead on assembling existing components for efficiency.

Breadth of Applicability of Models

Figure 2.23 illustrates the key dimensions for representing and categorizing architecture models,

accompanying diagrams, and related patterns. These architecture domains align with the TOGAF standard, as discussed earlier. The diagram also illustrates the levels of abstraction to characterize various architectural models. Additionally, it introduces the architecture scope as another dimension, classifying the models' breadth of applicability at the enterprise, portfolio, or project level. The **architecture scope** is the extent and boundaries within which architectural considerations, decisions, and solutions apply.

<table>
<tr><td colspan="4">To address the concerns of the following stakeholders...</td></tr>
<tr><td>Users, planners, business management</td><td>Database designers and administrators, system engineers</td><td>System and software engineers</td><td>Acquirers, operators, administrators, and managers</td></tr>
<tr><td colspan="4">... the following views may be developed</td></tr>
<tr><td>Business architecture views</td><td>Data architecture views</td><td>Applications architecture views</td><td>Technology architecture views</td></tr>
<tr><td>Business function view</td><td rowspan="5">Data entity view</td><td rowspan="5">Software engineering view</td><td rowspan="3">Networked computing/ hardware view</td></tr>
<tr><td>Business services view</td></tr>
<tr><td>Business process view</td></tr>
<tr><td>Business information view</td><td rowspan="3">Communications engineering view</td></tr>
<tr><td>Business locations view</td></tr>
<tr><td>Business logistics view</td><td rowspan="4">Data flow view (organization data use)</td><td rowspan="4">Applications interoperability view</td></tr>
<tr><td>People view (organization chart)</td><td rowspan="3">Processing view</td></tr>
<tr><td>Workflow view</td></tr>
<tr><td>Usability view</td></tr>
<tr><td>Business strategy and goals view</td><td rowspan="5">Logical data view</td><td rowspan="5">Software distribution view</td><td>Cost view</td></tr>
<tr><td>Business objectives view</td></tr>
<tr><td>Business rules view</td><td rowspan="3">Standards view</td></tr>
<tr><td>Business events view</td></tr>
<tr><td>Business performance view</td></tr>
<tr><td></td><td colspan="3">System engineering view</td></tr>
<tr><td colspan="4">Enterprise security view</td></tr>
<tr><td colspan="4">Enterprise manageability view</td></tr>
<tr><td colspan="4">Enterprise quality of service view</td></tr>
<tr><td colspan="4">Enterprise mobility view</td></tr>
</table>

Figure 2.23 TOGAF architectural dimensions include various levels of abstraction. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Portfolio- or domain-level architectures usually concentrate on collections of solutions and projects associated with a specific business unit, like marketing or sales in a large organization. In contrast, project- or system-level architecture is geared toward individual solutions and projects within those business units. Defining the

scope of a model is crucial because there exists a direct relationship between the scope and the level of detail achievable in a blueprint. This is due to the necessity for increased generalization, such as simplification, feature selection, and grouping, as the blueprint's scope expands.

TECHNOLOGY IN EVERYDAY LIFE

Adaptive Design Reuse: Eco-Friendly Homes from Recycled Materials

Adaptive design reuse can significantly benefit people in everyday life by promoting efficiency, sustainability, and improved user experiences. Imagine a neighborhood called EcoHomes, where all the houses are built using old materials from buildings that were taken down or left unused. It is all about making new homes without needing to produce or buy more materials, which helps the environment. In EcoHomes, architects and builders take things like bricks, glass, and wood from old sites and use them to build new, modern houses. For example, wooden beams from an old barn become part of the living room in a new house, adding a cool, old-time feel to a modern design. Windows from an old office building let in lots of sunlight, cutting down on the need for electric lights. EcoHomes is a hit because it shows how reusing building materials can save money and help the planet. The people living there have lower energy bills and are proud of their unique, eco-friendly homes. This story shows how using what we already have in new ways can make a big difference for our wallets and the world.

2.3 Evolving Architectures into Useable Products

Learning Objectives

By the end of this section, you will be able to:

- Analyze similarities between architectures and apply patterns
- Discuss how to accelerate the creation of applications

The combination of top-down, adaptive design reuse, and bottom-up, computational thinking, optimizes modern software development. This blend allows software developers to find a middle ground by adapting and assembling existing components, minimizing the need for developing entirely new software. A clear example of this cooperation is evident in modern websites, where the Model-View-Controller architectural pattern is widely employed. The **Model-View-Controller (MVC)** is a software architectural pattern commonly used in the design of interactive applications, providing a systematic way to organize and structure code. The pattern separates an application into three interconnected components: model, view, and controller. The *model* represents the application's data structure and business logic, managing data and rules. The *view* is responsible for displaying the user interface; it shows data to the user and sends user commands to the controller. The *controller* serves as an intermediary between the model and the view. It processes user input received from the view, interacts with the model to retrieve or update data, and determines the appropriate view for presenting the response. Many practical web application frameworks, such as Django, have already implemented the MVC pattern. In this setup, the application is divided into three parts: the model handles the data structure, the view displays the data on web pages, and the controller manages the business logic, facilitating interaction between the model and the view. Adding a broker pattern to MVC architectures can improve the system's scalability and flexibility when applicable and/or necessary. The broker acts as a middleman that manages communication between different parts of the application, helping to handle more data and complex operations efficiently.

Leveraging these existing frameworks enables developers to concentrate on crafting the specific logic relevant to the website rather than reinventing the wheel. The beauty of this approach lies in the ability to swiftly piece together solutions by extending and adapting the available frameworks. By doing so, developers streamline the development process, enhance efficiency, and capitalize on the collective wisdom embedded in proven

frameworks, thereby fostering innovation in a more focused and resource-efficient manner.

Leveraging Architectural Similarities and Applying Patterns

The adaptive design reuse approach is a strategy in software development that emphasizes the efficient reuse of existing design solutions to create new systems or applications. The beauty of the adaptive design reuse approach is that the business solution architecture model helps create abstract representations of real systems. Therefore, if there exist tangible realizations of the various components that are part of these abstract representations, it is possible to implement the model and create specialized running business solutions from it.

A **solutions continuum** is a strategy where existing solutions, components, or patterns are leveraged and adapted for use in different contexts. Figure 2.24 illustrates the TOGAF model of reuse that is referred to as the solutions continuum. As mentioned earlier, TOGAF does not provide a prescriptive approach to creating and/or managing a catalog of patterns. However, various pattern repositories are available on the Internet and the adaptive design technique can be used to avoid having to reinvent the wheel when architectural patterns and related component implementations exist and can be customized and assembled with new components. More information on this topic is provided in Chapter 10 Enterprise and Solution Architectures Management.

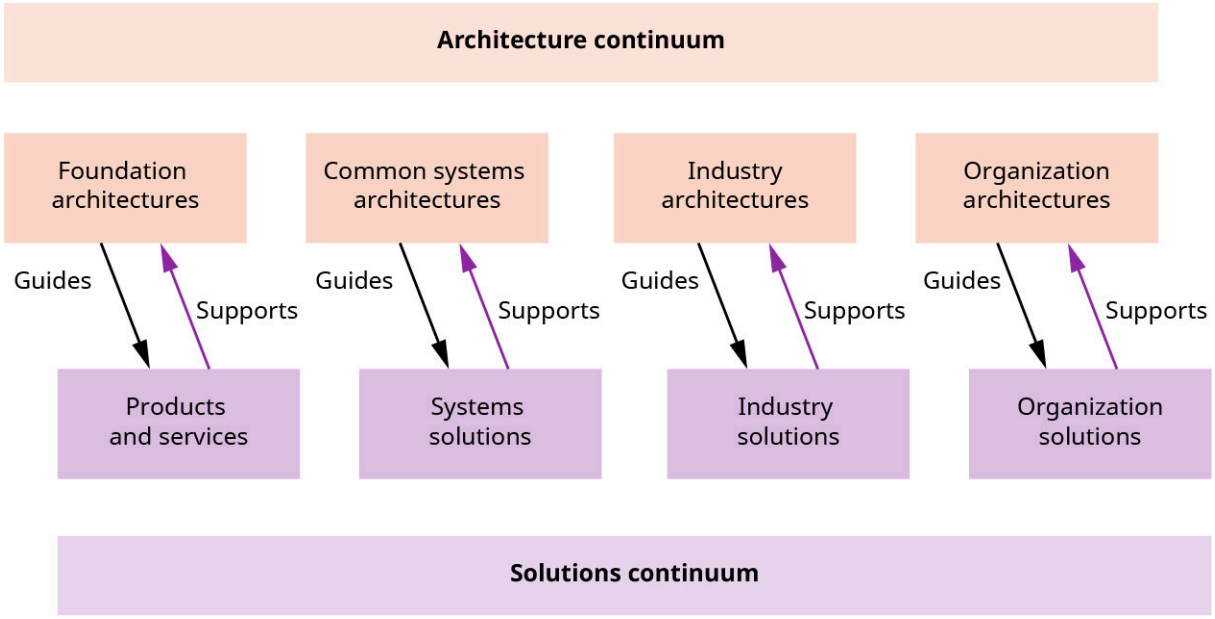


Figure 2.24 This TOGAF solutions continuum illustrates how each architecture guides and supports the others. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

As illustrated in Figure 2.25, the TOGAF solutions continuum offers a limited set of dimensions. It serves as a guideline, and The Open Group allows interested parties to enhance the model by incorporating additional dimensions that are relevant to their specific needs.

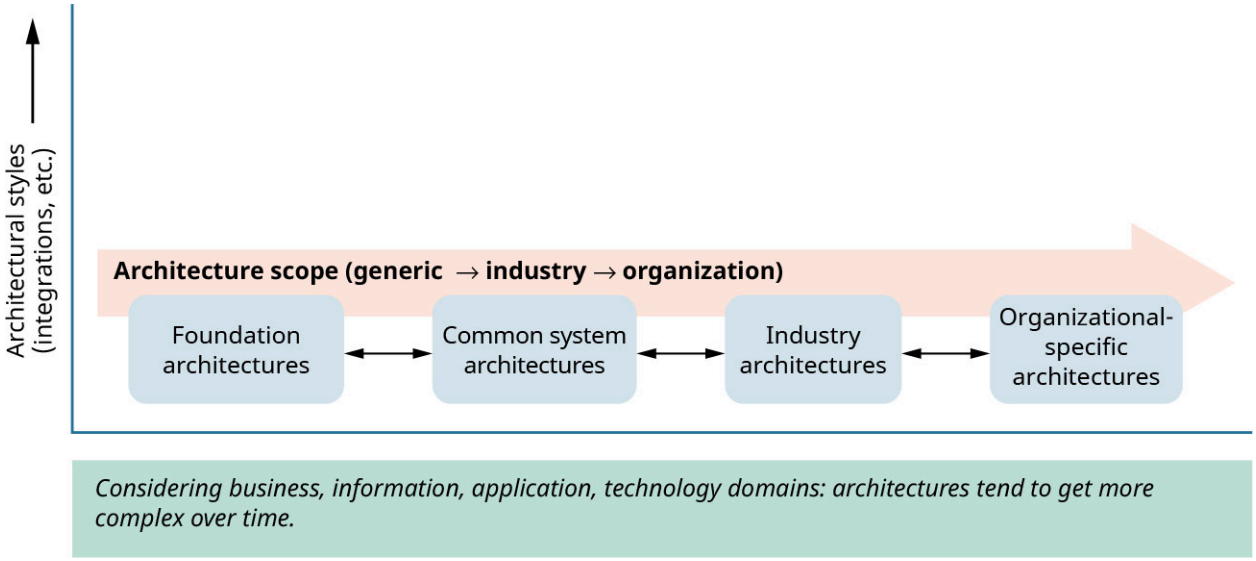


Figure 2.25 The TOGAF architecture continuum can suggest extensions but may only be able to focus on one aspect at a time. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Accelerating the Creation of Mainstream Business Solutions

To illustrate the power of architectural design and adaptive design reuse, various designs used in mainstream business solutions are surveyed, followed by explanations as to how corresponding turnkey solutions can be derived from these models. Several subsequent chapters of the book elaborate on building-related solutions.

CONCEPTS IN PRACTICE

Object Management Architecture (OMA)

Organizations like the Object Management Group (OMG) create foundational and common system architectures that may be used across industries. An example is the Object Management Architecture (OMA), which is a foundation for developing architectures as building blocks. It then elaborates in providing Object Services, Horizontal Facilities, and Vertical Facilities as subcomponents to help classify common system architectures that may be used to assemble a complete OMA-centric architecture. It is then the responsibility of the various industries to establish standard architectures that may be leveraged by organizations that operate in these industries. Finally, organizations benefit from being able to leverage foundational, common systems and industry architectures to develop their own proprietary architectures. Based on the models of the various architectures that organizations may use and assuming there exists solutions for them, organizations can develop their own solutions faster by reusing and customizing existing solution components instead of reinventing the wheel. This is actually how the TOGAF solution continuum applies adaptive design reuse.

Responsive Web 2.0 Business Solutions

World Wide Web Consortium (W3C) is an international community that develops guidelines to ensure the long-term growth and accessibility of the World Wide Web. **Web 2.0** is the second generation of the World Wide Web when we shift from static web pages to dynamic content. **Web 3.0** is the third generation of the World Wide Web and represents a vision for the future of the Internet characterized by advanced technologies. Most modern websites rely on the Web 2.0 architectural model set forth by W3C. A sample logical application architecture model is illustrated in Figure 2.26.

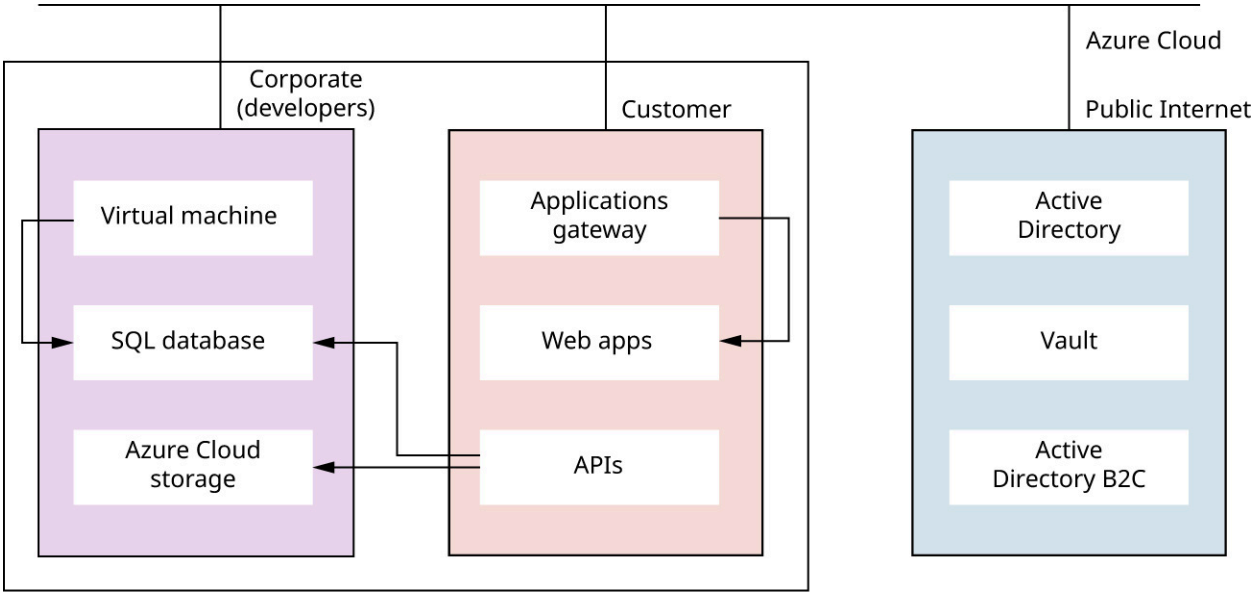


Figure 2.26 The logical application architecture of Microsoft Azure-hosted web applications allows for responsive web and mobile solutions for users. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

In this case, the model leverages the various components available on the Microsoft Azure Cloud. **Microsoft Azure** is a comprehensive cloud computing platform provided by Microsoft. Azure is designed to help organizations build, deploy, and manage applications and services through a global network of data centers. Azure provides streamlined development capabilities under its DevOps offering to make it very easy to develop and quickly deploy websites on the Azure platform using mainstream web application frameworks (e.g., ASP.Net, PhP, Java). DevOps is an Agile Software Engineering tools-driven approach that focuses on developing software and deploying it into operation.

Many of the support components required to support website implementations are readily available on Microsoft Azure and other systems that provide reusable components for responsible web design. It is easy to evolve the model shown below into a running website. A **web application framework** has built-in support for architectural patterns that make it easy to extend the framework and use plug-ins to implement commercial-grade websites in a reasonable amount of time. They also support the use of web frameworks that make it possible to build a responsive web application that makes the functionality available on the desktop version of the application seamlessly available on a mobile device. In addition to these capabilities, the adaptive design reuse approach may be used to create the custom part of the web application. More information related to the development of web solutions is provided in Chapter 9 Software Engineering, Chapter 10 Enterprise and Solution Architectures Management, and Chapter 11 Web Applications Development.

THINK IT THROUGH

Architectural Similarities

What is one of the mechanisms that makes it possible to compare architectural similarities between two solutions at different levels?

Native Mobile Business Solutions

A **web application (web app)** is a software application that is accessed and interacted with through a web browser over the Internet. Many web-based solutions leverage the inherent capabilities of mobile devices, offering web apps tailored for various types of phones in addition to responsive websites. Numerous frameworks exist to facilitate the development of native web apps, streamlining the process of creating

applications that can run seamlessly on different mobile platforms. These frameworks often provide a unified and efficient approach to building cross-platform mobile applications, ensuring a consistent user experience across various devices.

In certain frameworks and development environments, React Native UI component libraries can be leveraged to, port web apps to mobile devices. Examples include React Native support for Android apps using the Android Studio (Android Studio provides a comprehensive environment for developing, testing, and debugging Android apps) or iPhone web app using XCode IDEs (Xcode is an integrated development environment [IDE] developed by Apple for macOS that offers a suite of tools for building software for Apple platforms, including macOS, iOS, watchOS, and tvOS). Figure 2.27 illustrates the logical application architecture of mobile web apps that use React Native. In addition to these capabilities, the adaptive design reuse approach may be used to create the custom part of the native web app. More information related to the development of native web app solutions is provided in Chapter 9 Software Engineering, Chapter 10 Enterprise and Solution Architectures Management, and Chapter 11 Web Applications Development.

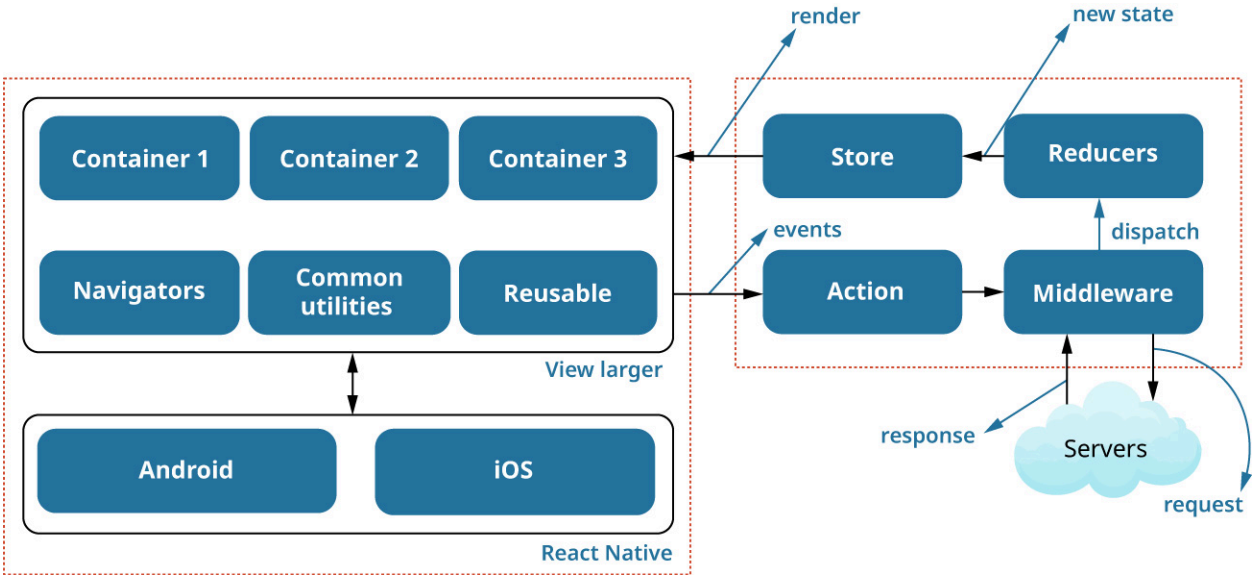


Figure 2.27 The logical application architecture of React Native mobile web apps shows the back-end processes that allow both Android and IOS customers to use the same application. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Native Mobile Business Examples

Native mobile apps are designed specifically for mobile operating systems, providing optimal performance and a seamless user experience.

- WhatsApp: WhatsApp is a native mobile app designed specifically for iOS and Android platforms. It directly accesses the hardware of the device, such as the GPS, camera, and microphone, which allows for features like real-time location sharing, voice and video calls, and media sharing.
- Instagram: Instagram is a photo- and video-sharing app. Native development helps Instagram manage high-quality media content efficiently, apply real-time filters, and smoothly handle in-app animations.
- Uber Eats: Uber Eats is a food-delivery service that operates as a native app on mobile devices. Being native allows the app to use device-specific features, such as GPS for tracking the delivery person's location in real time.
- Spotify: Spotify uses its native app to deliver personalized music and podcast streaming services. The app's native nature allows it to integrate closely with the device's hardware, offering features like offline downloading, low-latency streaming, and background play.

Web 3.0 Business Solutions

The secure and transparent way of recording transactions that uses a chain of blocks, each storing a list of encrypted transactions is called **blockchain**. Once a block is full, it is linked to the previous one, forming a chain. Blockchain technology decentralizes processing to ensure the integrity of transactions across multiple computer nodes. This ensures that no single computer node gets assigned to processing transactions repeatedly, thereby preventing possible fraudulent modifications of transactions. A **smart contract** is an automated agreement written in code that runs on blockchain technology. They enforce contract terms automatically when specific conditions are met, removing the need for intermediaries and ensuring security. The use of blockchain smart contracts within web applications is becoming more popular. The logical application architecture model in Figure 2.28 illustrates how this is made possible by creating hybrid Web 2.0 websites that interact with Web 3.0 smart contracts.

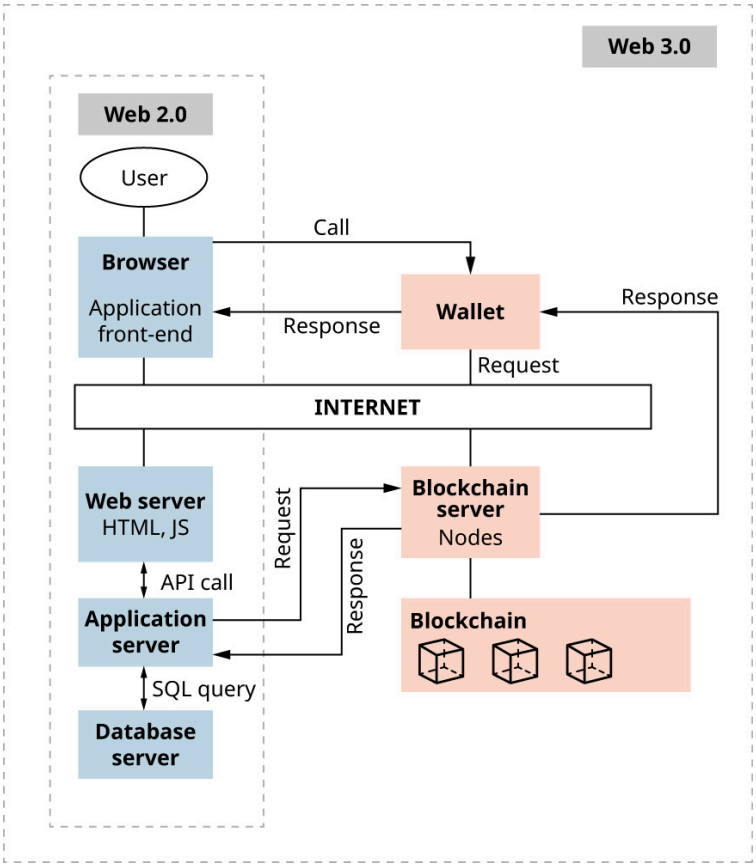


Figure 2.28 The flowchart show the logical application architecture of a Web 2.0 and Web 3.0 Website. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Building these types of business solutions is greatly facilitated by the use of the **Ethereum platform**, an open-source blockchain platform that enables the creation and execution of smart contracts and decentralized applications, or Cloud blockchain platforms provided by one of the Cloud service providers such as Amazon AWS, Google GCP, IBM Cloud, Consensys, Oracle Cloud, and others. These platforms provide frameworks and APIs that make it easy to develop and deploy smart contracts. The Web 2.0 portion of the website can leverage the frameworks mentioned earlier. In addition to these capabilities, the adaptive design reuse approach may be used to create the custom part of the Web 3.0 application. More information related to the development of Web 3.0 solutions is provided in Chapter 9 Software Engineering, Chapter 10 Enterprise and Solution Architectures Management, and Chapter 13 Hybrid Multicloud Digital Solutions Development.

Cloud-Native Business Solutions

A way of building software by breaking it into small, independent pieces where each piece, or service, does a

specific job and works on its own is called **microservices**. A large number of businesses have been migrating their legacy business solutions to the cloud to take advantage of microservices that are designed around specific business functions and can be deployed independently using automated deployment systems. Figure 2.29 illustrates how secure, managed, and monetized APIs that are critical for a digital enterprise can be created by leveraging a combination of API-led integration frameworks and cloud-native technologies. The use of such frameworks and technologies helps streamline the migration of legacy business solutions. The process of **migrating legacy business solutions** means upgrading or replacing old systems with newer, more efficient ones. In addition to these capabilities, the adaptive design reuse approach may be used to create the custom part of the cloud-native applications. More information related to the development of cloud-native solutions is provided in Chapter 9 Software Engineering, Chapter 10 Enterprise and Solution Architectures Management, and Chapter 12 Cloud-Native Applications Development.

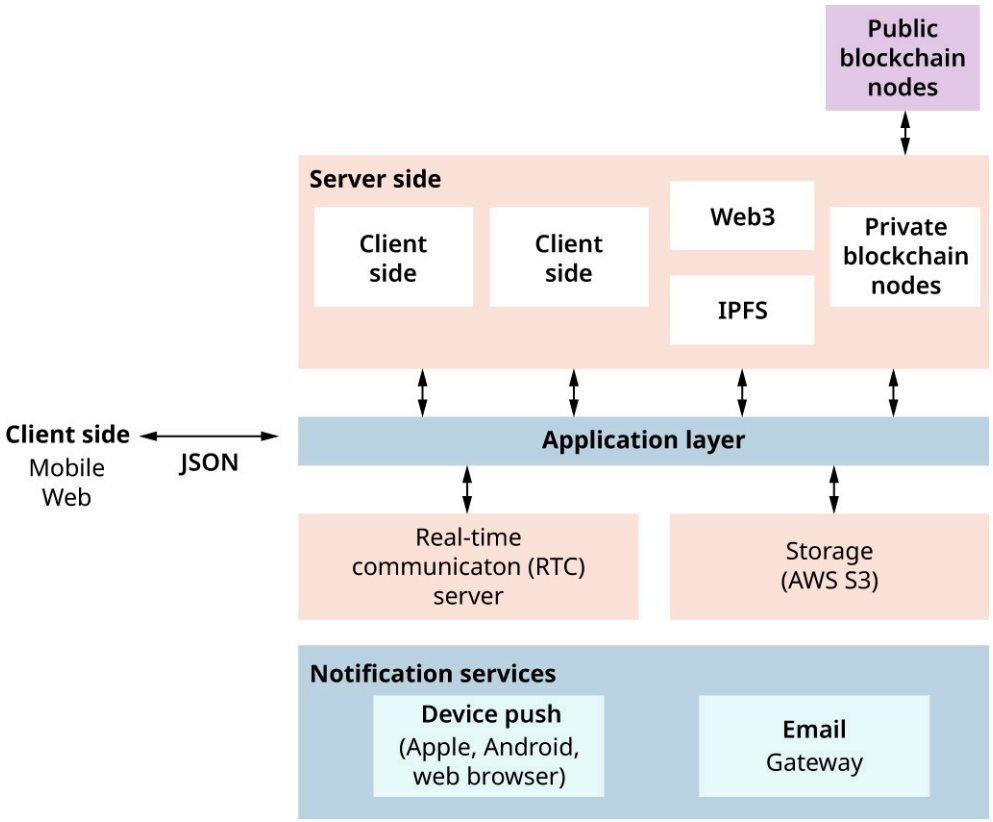


Figure 2.29 The cloud-native application architecture view of a digital enterprise shows both client-side and server-side processes through each layer. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Figure 2.29 illustrates the architecture of a digital platform that combines web and mobile applications with blockchain technology. On the client side, users interact with the platform through web dashboards and mobile apps, which communicate with the server using JSON and handle notifications via services like Firebase, Huawei, and Apple. The server side includes an API layer that processes these requests, a caching layer to improve performance, and a back-end logic layer responsible for application logic, backups, and analytics. The architecture also features integration with public blockchain networks for enhanced security and transparency, and it supports various notification services to keep users informed.

GLOBAL ISSUES IN TECHNOLOGY

Scale Transformations

The approach that consists of reinventing, rethinking, and rewiring solutions in various industries seems to

favor countries that have the means to perform broadscale transformations. This may have ethical, social, and economic implications in other parts of the world.

Consider how advanced countries are rapidly adopting electric cars. They have the resources to reinvent transportation by investing in electric vehicle (EV) technology, rethinking their energy use to reduce pollution, and rewiring their infrastructure to support EV charging stations. This shift toward electric cars is more challenging in less wealthy countries due to the high costs of EVs and the lack of charging infrastructure. As a result, these countries may continue to depend on older, more polluting vehicles, facing both environmental and economic disadvantages.

Innovative Cloud Mashups

Innovative cloud mashups refer to creative combinations of different innovative business solutions that leverage disruptive technologies such as IoT, big data analytics, machine learning, blockchain, and others that can be quickly assembled today as hybrid cloud applications. A **hybrid cloud application** combines the benefits of both private and public clouds, allowing organizations to optimize their infrastructure based on specific requirements.

Internet of Things (IoT) refers to the network of physical devices embedded with sensors, software, and connectivity, enabling them to collect and exchange data. The process of examining, processing, and extracting valuable insights from large datasets is called **big data analytics**. Developing algorithms that enable computers to learn from data and make decisions without explicit programming is called **machine learning**. This is made possible by creating mashups of platform services provided by various public cloud vendors to gain access to these disruptive technologies.

Figure 2.30 and Figure 2.31 illustrate models of solutions that are used today to support a variety of mobile health (MHealth), body area networks (BANs), emotions monitoring, and social media applications. In addition to the capabilities provided by the Big Clouds, the adaptive design reuse approach may be used to create the custom part of these hybrid solutions. Google Maps and Zillow are prime examples of applications that utilize location-based data to deliver valuable services. A GPS device identifies a user's location, and that information flows through the central network. Apps then display this data in a user-friendly manner, connecting users with real-time geographic information in Google Maps or housing market details in Zillow. The integration of GPS with other IoT systems allows for the seamless presentation of customized, location-specific content to enhance the user experience. More information related to the development of web solutions is provided in Chapter 9 Software Engineering, Chapter 10 Enterprise and Solution Architectures Management, and Chapter 13 Hybrid Multicloud Digital Solutions Development.

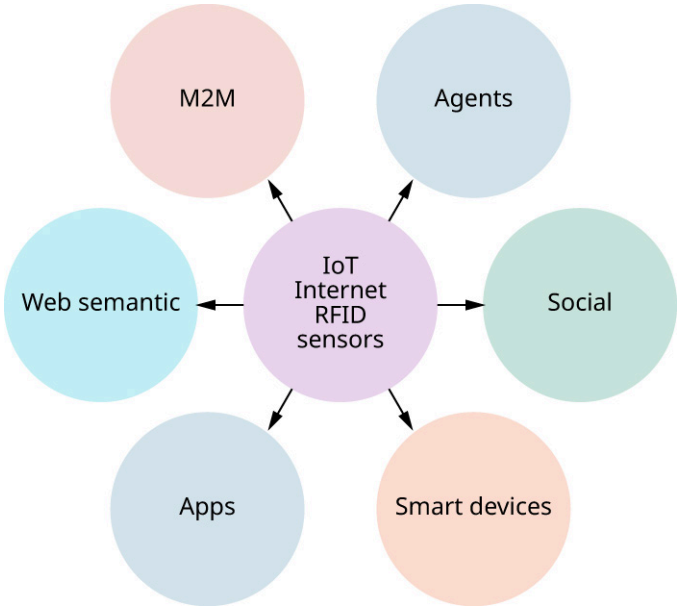


Figure 2.30 IoT devices use sensors, applications, and connectivity to interact, collect, and exchange data. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

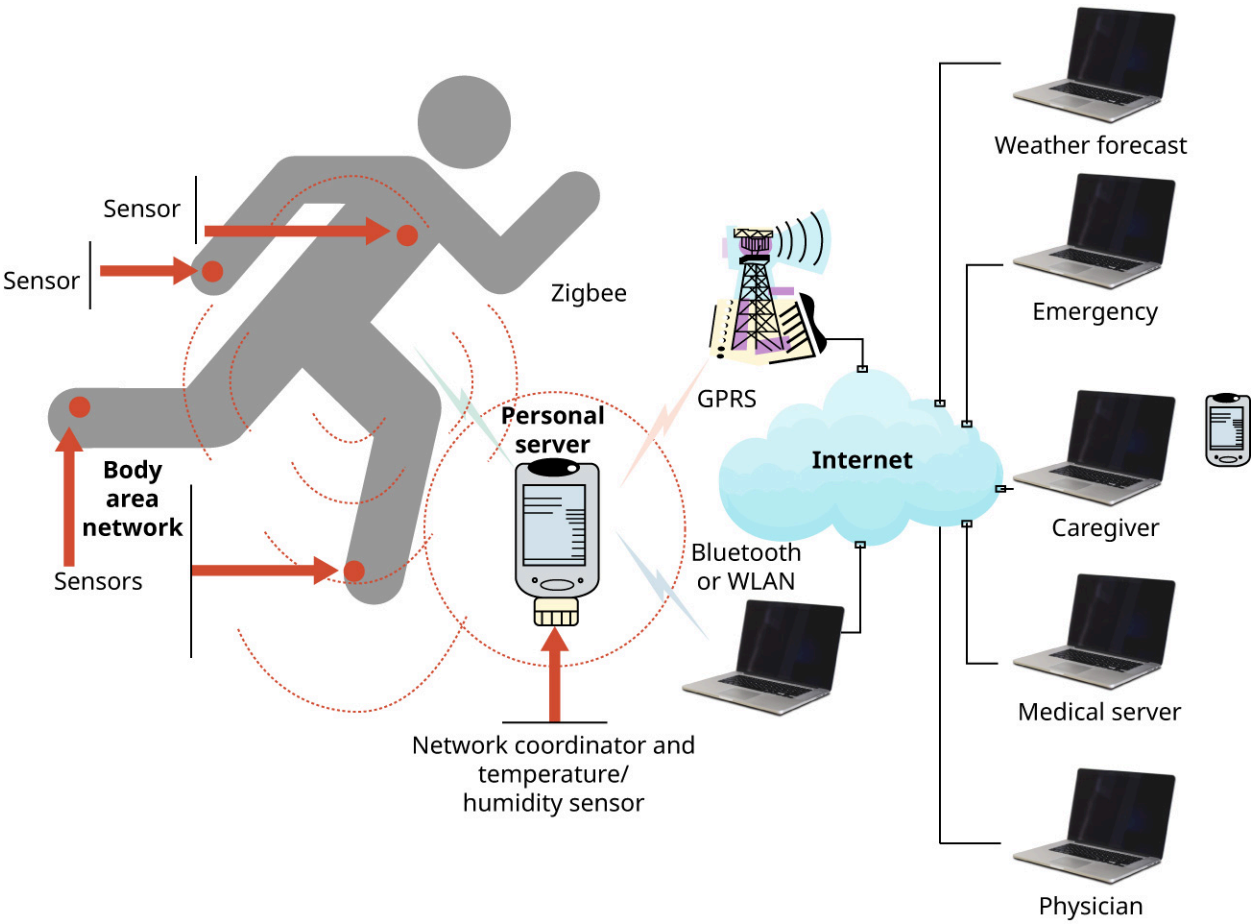


Figure 2.31 This diagram depicts an architecture of a body area network. (credit: modification of "Body Area Network" by "Jtel"/Wikimedia Commons, Public Domain)

Web 3.0 enables businesses to create more personalized and predictive services for users, fostering greater trust and engagement by giving users control over their own data. For companies, this translates to new opportunities for collaboration, innovation, and reaching consumers directly without intermediaries, ultimately

driving more efficient business models and creating value in ways that were not possible with earlier web technologies.

Chapter Review

Key Terms

abstraction simplified representation of complex systems or phenomena
application architecture subset of the enterprise solution architecture; includes a process to architect and design the application architecture as well as the actual application architecture model, which represents the content of the application architecture
architectural pattern reusable solution to a recurring problem in software architecture design
architecture model represents the content of the application architecture
architecture scope extent and boundaries within which architectural considerations, decisions, and solutions apply
automation using a program or computer application to perform repetitive tasks or calculations
big data analytics process of examining, processing, and extracting valuable insights from large datasets
blockchain secure and transparent way of recording transactions; uses a chain of blocks, each storing a list of transactions
blueprint detailed plan or design that outlines the structure, components, and specifications of a building, product, system, or process
business logic layer holds the business logic for the business solution or application
business model framework that outlines how a business creates, delivers, and captures value
business process hierarchy organizes a company's activities from broad, general processes down to specific tasks, making it easier to manage and improve how the business operates
computational thinking problem-solving and cognitive process rooted in principles derived from computer science that involves breaking down complex problems into smaller, more manageable parts and devising systematic approaches to solve them
data architecture model conceptual framework that outlines how an organization structures, organizes, and manages its data assets
data management layer responsible for interacting with persistent storage systems like databases and various data processing mechanisms
data modeling collaborative process wherein IT and business stakeholders establish a shared understanding of essential business terms, known as entities, which typically end up being represented as tables that contain data in a relational database management system
debugging finding and fixing of issues in code
decomposition solving of a complex problem by breaking it up into smaller, more manageable tasks
design component reusable element within a larger system that serves a specific purpose
EA domain represents business, data, application, and technology architectures
elementary business process (EBP) fundamental and indivisible activity within a business that is not further subdivided into smaller processes
entity model represents the various objects or concepts and their relationships within a system
Ethereum platform open-source blockchain platform that enables the creation and execution of smart contracts and decentralized applications
flowchart method for showing the flow and direction of decisions in a visual way using a diagram
function set of commands that can be repeatedly executed
heuristic form of a pattern that is well-known and considered a rule of thumb
hybrid cloud application combines the benefits of both private and public clouds, allowing organizations to optimize their infrastructure based on specific requirements
Internet of Things (IoT) network of physical devices embedded with sensors, software, and connectivity, enabling them to collect and exchange data
machine learning developing algorithms that enable computers to learn from data and make decisions without programming

microservices way of building software by breaking it into small, independent pieces; each piece, or service, does a specific job and works on its own
Microsoft Azure comprehensive cloud computing platform provided by Microsoft
migrating legacy business solutions upgrading or replacing old systems with newer, more efficient ones
Model-View-Controller (MVC) software architectural pattern commonly used in the design of interactive applications, providing a systematic way to organize and structure code
monolithic structure system or application architecture where all the components are tightly integrated into a single unit
presentation layer user's touchpoint, handling the user interface (UI) and delivering the user experience (UX), which encapsulates the overall feel and interaction a person has with a system or service
process map displays the order of chosen processes from the process hierarchies, highlighting their connection to the roles responsible for executing them
pseudocode outline of the logic of algorithms using a combination of language and high-level programming concepts
recursion programming and mathematical concept where a function calls itself during its execution
smart contract automated agreement written in code that runs on blockchain technology
solution architecture structural design that is meant to address the needs of prospective solution users
solutions continuum strategy where existing solutions, components, or patterns are leveraged and adapted for use in different contexts
system architecture deals with application and data, and how they are related to each other and what business process they support together
technical architecture includes the software and hardware capabilities to fully enable application and data services
user experience (UX) overall experience that a person has when interacting with a product, service, or system
Web 2.0 second generation of the World Wide Web when we shift from static web pages to dynamic content
Web 3.0 third generation of the World Wide Web and represents a vision for the future of the Internet characterized by advanced technologies
web application (web app) software application that is accessed and interacted with through a web browser over the Internet
web application framework built-in support for architectural patterns that make it easy to extend the framework and use plug-ins to implement commercial-grade websites in a reasonable amount of time
World Wide Web Consortium (W3C) international community that develops guidelines to ensure the long-term growth and accessibility of the World Wide Web

Summary

2.1 Computational Thinking

- Complex problems are situations that are difficult because they involve many different parts or factors.
- Computational thinking means breaking these problems into smaller parts, understanding how these parts relate to each other, and then coming up with effective strategies or steps to solve each part.
- Computational thinking is a set of tools or strategies for solving (and learning how to solve) complex problems that relate to mathematical thinking in its use of abstraction, decomposition, measurement, and modeling.
- Characterization of computational thinking is the three As: abstraction, automation, and analysis.
- Decomposition is a fundamental concept in computational thinking, representing the process of systematically breaking down a complex problem or system into smaller, more manageable parts or subproblems.
- Logical thinking and pattern recognition are computational thinking techniques that involve the process of identifying similarities among and within problems.
- Abstraction is a computational thinking technique that centers on focusing on important information

while ignoring irrelevant details.

- Algorithms are detailed sets of instructions to solve a problem step-by-step.
- Testing and debugging is about finding and fixing mistakes in the step-by-step instructions or algorithms used to solve a problem.

2.2 Architecting Solutions with Adaptive Design Reuse in Mind

- Computational thinking commonly employs a bottom-up strategy for crafting well-structured components.
- A business solution architecture is a structural design that is meant to address the needs of prospective solution users.
- Business solutions are strategies/systems created to solve specific challenges in a business. Designing business solutions can be described as a complex systemic process that requires expertise in various spheres of technology as well as the concerned business. A blueprint is a detailed plan or design that outlines the structure, components, and specifications of a building, product, system, or process.
- Two heuristics are inherent to the design of business solutions and the creation of business solution architectures. Layering in business solution architecture involves creating distinct layers that abstract specific aspects of the overall architecture. The layering approach relies on the principle of separation of concerns. The presentation layer holds the user interface (UI) that interacts with the outside world.
- User experience (UX) refers to the overall experience that a person has when interacting with a product, service, or system.
- A monolithic structure is a system or application architecture where all the components are tightly integrated into a single unit.
- Enterprise-level architecture encompasses various domains that define the structure, components, and operations of an entire organization. Enterprise architecture (EA) views the enterprise as a system or a system of systems.
- The enterprise business architecture (EBA) is a comprehensive framework that defines the structure and operation of an entire organization. A business model is a framework that outlines how a business creates, delivers, and captures value. The organizational model is the structure and design of an organization, outlining how roles, responsibilities, and relationships are defined.
- The business process is a series of interrelated tasks, activities, or steps performed in a coordinated manner within an organization to achieve a specific business goal.
- Location model refers to a set of rules used to analyze and make decisions related to the positioning of entities, activities, or resources.
- The enterprise technology architecture (ETA) is a comprehensive framework that defines the structure, components, and interrelationships of an organization's technology systems to support its business processes and objectives.
- The application architecture is a subset of the enterprise solution architecture.
- A data architecture model is a conceptual framework that outlines how an organization structures, organizes, and manages its data assets.
- Data modeling is the collaborative process wherein IT and business stakeholders establish a shared understanding of essential business terms, known as entities.
- Architecture views are representations of the overall system design that matter to different stakeholders.

2.3 Evolving Architectures into Useable Products

- The combination of top-down, adaptive design reuse and bottom-up, computational thinking optimizes modern software development.
- Model-View-Controller (MVC) is a software architectural pattern commonly used in the design of interactive applications, providing a systematic way to organize and structure code.
- The adaptive design reuse approach is a strategy in software development that emphasizes the efficient reuse of existing design solutions to create new systems or applications.
- World Wide Web Consortium (W3C) is an international community that develops guidelines to ensure the

long-term growth and accessibility of the World Wide Web.

- Web 2.0 is the second generation of the World Wide Web when we shift from static web pages to dynamic content.
- Web 3.0 is the third generation of the World Wide Web and represents a vision for the future of the Internet characterized by advanced technologies.
- A web application (web app) refers to a software application that is accessed and interacted through a web browser over the Internet.
- Blockchain is a secure and transparent way of recording transactions. It uses a chain of blocks, each storing a list of transactions.
- Microservices is a way of building software by breaking it into small, independent pieces. Each piece, or service, does a specific job and works on its own.
- Migrating legacy business solutions means upgrading or replacing old systems with newer, more efficient ones.
- Innovative cloud mashups refer to creative combinations of different innovative business solutions that leverage disruptive technologies.

Review Questions

1. What term is a problem-solving and cognitive process rooted in principles derived from computer science that involves breaking down complex problems into smaller, more manageable parts and devising systematic approaches to solve them?
   a. abstraction
   b. decomposition
   c. computational thinking
   d. recursion

2. What shape in a flowchart represents a decision point?
   a. oval
   b. parallelogram
   c. rectangle
   d. diamond

3. What does pseudocode spell out in natural language?
   a. an algorithm
   b. a test case to debug
   c. a flowchart
   d. the programming language of choice

4. After a test case fails, what is the next step to determine the cause of the failure?

5. What are the key elements of CT that distinguish CT from other types of problem-solving strategies?

6. What is the primary difference between a heuristic and a pattern?
   a. A heuristic is a general rule used for quick problem-solving when an exact solution is not possible, while a pattern is a repeatable solution to a commonly occurring problem.
   b. A heuristic and a pattern are both specific guidelines used to achieve exact solutions in complex problems.
   c. A heuristic is used for creating new designs, whereas a pattern refers to repeating decorative motifs.
   d. There is no difference; both terms refer to specific scientific methods used in research.

7. What level of architecture is described as having a narrower scope, a detailed blueprint, and a lower level of abstraction?
   a. system architecture
   b. technical architecture

c. enterprise architecture
d. solution architecture

8. What level of architecture is described as having a wider scope, a vague plan for the entire organization, and a higher level of abstraction?
a. system architecture
b. technical architecture
c. enterprise architecture
d. solution architecture

9. What component holds the business logic for the business solution or application?
a. presentation layer
b. data management layer
c. business logic layer
d. business process hierarchy

10. What is the difference between data, information, knowledge, and wisdom?

11. Explain why an information system architecture is considered an architecture view in TOGAF.

12. Once architectural similarities have been identified between the architecture of a new problem and existing architectural solutions, what is required to apply these patterns effectively?
a. a comprehensive understanding of the new problem's requirements and constraints
b. the ability to modify existing patterns to fit the new problem's unique context
c. both a comprehensive understanding of the new problem's requirements and the ability to modify existing patterns
d. approval from a higher authority to use the identified patterns

13. In the Model-View-Controller, what layer is responsible for acting as an intermediary between two layers?
a. view
b. model
c. business logic
d. controller

14. What does Web 3.0 provide that Web 2.0 did not?
a. dynamic web pages as opposed to only static web pages
b. represents a vision for the future of the Internet characterized by advanced technologies
c. shifted from HTTP to HTTPS
d. based on JSON as opposed to HTML

15. A smart home with a thermostat, a refrigerator, and lights that all can be controlled remotely is an example of devices that can be described with what terminology?
a. Internet of Things (IoT)
b. machine learning
c. hybrid cloud application
d. solutions continuum

16. Why doesn't TOGAF provide prescriptive methods to create and manage repositories of architectural patterns?

17. What is a responsive web application?

18. What is a cloud mashup?

Conceptual Questions

1. Suppose you plan to meet with your friends at a location you are unfamiliar with. In what ways could you employ computational thinking to efficiently navigate and locate the meeting spot?
2. Explain how the pyramid of knowledge concept helps describe the learning progress you make when reading a textbook.
3. What are specific examples of business architecture similarities between two banks?
4. What are specific examples of technology architecture similarities between two banks?

Practice Exercises

1. Think of a complex problem—one that can be broken into many layers of smaller problems. Explain how computational thinking could help you develop a solution to your complex problem.
2. Look at the following pseudocode that describes an algorithm to make a peanut butter and jelly sandwich:
    a. Get the peanut butter.
    b. Get the jelly.
    c. Get the bread.
    d. Open the peanut butter jar.
    e. Open the jelly jar.
    f. Open the bread.
    g. Take out slice of bread.
    h. Take out another slice of bread.
    i. Dip the knife into the peanut butter.
    j. Spread the peanut butter on one slice of bread.
    k. Dip the knife into the jelly.
    l. Spread the jelly on the other slice of bread.
    m. Put the two slices of bread together.

    Write a new algorithm that utilizes abstraction to simplify the number of steps of the original algorithm and can be used as a pattern to make any sandwich.
3. Research what the Fibonacci number sequence is. Write the pseudocode to compute the *n*th number in the Fibonacci number sequence. Utilize recursion to model a pattern of computation.
4. Create a model that describes the business of running your daily life. Please note that this is not suggesting that you should run your life as a business. *Hint*: To answer this question, think about the various players, locations, and processes involved in your daily activities and create simple models that mimic the structure provided for the trading business model in the current chapter section.
5. Draw an application architecture diagram for a business solution that uses smart contracts for payment and transactions logging purposes. Feel free to leverage some of the figures from this chapter, rather than create something new.

Problem Set A

1. Create an algorithm to explain to a robot how to cross a street. Use computational thinking to break down the problem into smaller parts. Use the following information to guide your thinking.

Task	Decomposition	Pattern Recognition	Abstraction	Algorithm
Crossing the road	Vehicles, actions, decision	Identify the different considerations you can group together to form a pattern of what needs to be done.	Act out crossing the road. Do you do something differently from someone else?	Write your instructions in either pseudocode or as a flowchart.

2. Create an algorithm to explain how to bake a four-tiered wedding cake.
3. Reflect on what happens when you try to figure out driving directions from point A to point B.
4. Create an enterprise architecture business model for an insurance company that specializes in insuring home and car owners.
5. Create two alternative enterprise technology architecture models for the insurance company business model created in the previous question.
6. Draw an application architecture diagram for the Web 2.0 responsive website of a fictitious insurance company that focuses on home and car insurance and assume that the company also provides native apps to its customers in addition to the website.

Problem Set B

1. Create an algorithm to explain to a robot how to play a game of rock paper scissors. Use computational thinking to break down the problem into smaller parts. Use the following information to guide your thinking.

Task	Decomposition	Pattern Recognition	Abstraction	Algorithm
Rock, paper, scissors	Actions, choices, timings, winning conditions	Identify the different considerations you can group together to form a pattern of what needs to be done.	Play the game. Think of the actions you perform.	Write your instructions in either pseudocode or a flowchart.

2. Create an algorithm to show a robot how to play a game of tic-tac-toe. Use computational thinking to break down the problem into smaller parts. Use the following information to guide your thinking.

Task	Decomposition	Pattern Recognition	Abstraction	Algorithm
Tic-tac-toe	Moves that can be made, winning conditions	Identify the different considerations you can group together to form a pattern of what needs to be done.	Play against someone. What strategies do you use in order to win?	Write your instructions in either pseudocode or a flowchart.

3. Perform some research on the Internet to piece together enterprise architectures for as many industries as you can think of.

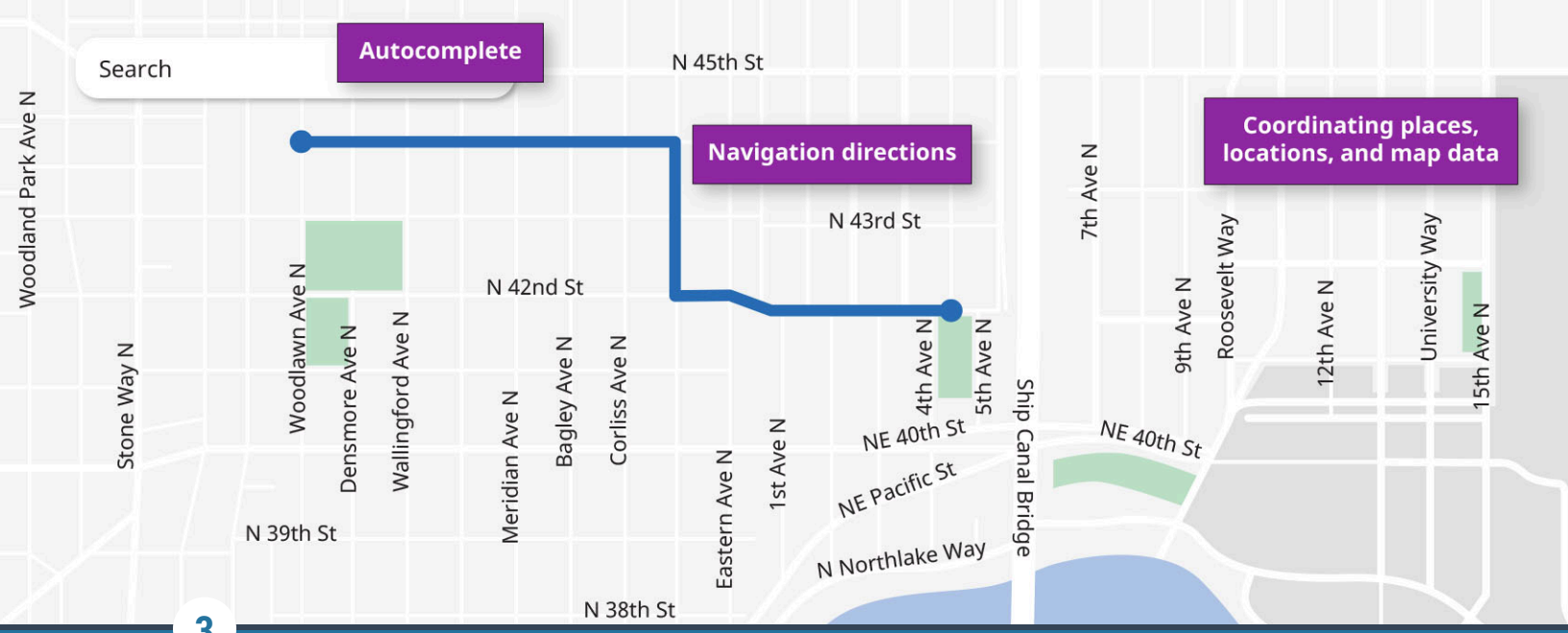
4. Draw a cloud-native application architecture diagram for the trading business and technical model documented in the previous section of this chapter.
5. A company wants to develop a business solution that takes pictures of the license plates of cars that drive too fast through intersections in a given city, sends tickets to the drivers, and manages ticket payments. Draw an innovative cloud mashup application architecture diagram for such a solution. Please note that IoT, machine learning, and blockchain PaaS services should be used as part of your design.
6. Document the architecture of a pattern catalog that could be used to provide access to solution architecture diagrams that would help accelerate the creation of mainstream business solutions.

Thought Provokers

1. Consider TechWorks, which is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe how it can best use computational thinking to create products or services that can generate business. Give precise examples and explain how the start-up would be able to scale the resulting business (i.e., keep sustaining the cost of doing business while increasing its number of customers). *Hint*: Some companies leverage an incubation arm to come up with innovative ideas and then accelerate the process of developing these ideas into practical solutions via a solution accelerator.
2. Consider our start-up company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe how it can best use adaptive design reuse to create products or services that can generate business. Give precise examples and explain how the start-up would be able to scale the resulting business (i.e., keep sustaining the cost of doing business while increasing its number of customers)? *Hint*: The company may decide to sell reusable design models and their implementation from a proprietary catalog; it may also focus on providing consulting services to derive complete solutions from its proprietary models.
3. Consider our start-up company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe how it can best leverage evolving architectures into usable products to create products or services that can generate business. Give precise examples and explain how the start-up would be able to scale the resulting business (i.e., keep sustaining the cost of doing business while increasing its number of customers).

Labs

1. Perform some research on the Internet to find examples of problem scenarios that computational thinking may help solve and create a catalog of problem scenarios. Then, elaborate and show practically how this catalog may be used to compare the scenarios and classify them so they may be used as part of your pattern discovery as you apply computational thinking to new problem scenarios.
2. Create an enterprise architecture capability model for a company of your choice using your research from problem set B. Then, expand one of the capabilities and provide business and technology architecture models for it; identify a project within the capability you expanded upon and provide a complete solution architecture for it.
3. Perform some research on the Internet to piece together additional solutions architecture diagrams for the various categories of mainstream solutions covered in this chapter section. This should include application, data, and technology diagrams.
4. Catalog additional types of solution architectures that may be used to accelerate the creation of mainstream business solutions.
5. Apply critical thinking strategies to develop a study plan for your current semester's courses, aiming to achieve an A or pass each course.



3

Data Structures and Algorithms

Figure 3.1 Online mapping applications represent places, locations, and map data while providing functionality to look around, search for places, and get navigation directions. The right combination of data structures to manage collections of places, locations, and map data along with efficient search and navigation algorithms will help optimize the experience of users trying to find their way through the map and will also make optimal use of computing resources. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license; data source: OpenStreetMap under Open Database License; attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Chapter Outline

3.1 Introduction to Data Structures and Algorithms
3.2 Algorithm Design and Discovery
3.3 Formal Properties of Algorithms
3.4 Algorithmic Paradigms
3.5 Sample Algorithms by Problem
3.6 Computer Science Theory

Introduction

Online maps help people navigate a rapidly changing world. It was not long ago that maps were on paper and that knowledge came from non-digital, trusted sources. In this chapter, we will study how computer scientists design and analyze the foundational structures behind many of today's technologies. Data structures and algorithms are not only foundational to map apps, but also enable an amazing variety of other technologies too. From self-driving cars to inventory management to simulating the movement of galaxies to transferring data between computers—all these applications use data structures and algorithms to efficiently organize and process large amounts of information.

3.1 Introduction to Data Structures and Algorithms

Learning Objectives

By the end of this section, you will be able to:

- Understand the difference between algorithms and programs
- Relate data structures and abstract data types
- Select the data structure that is appropriate to solve a given problem practically

Computer science is the study of computers and computational systems that involve data representation and process automation. Owing to their historical roots as calculators, computers can easily represent numerical data. Calculators rely on algorithms to add, subtract, multiply, and divide numbers. But what about more complex data? How do computers represent complex objects like graphs, images, videos, or sentences? What complications arise when we represent or process data in certain ways? These are some of the foundational questions that computer scientists and programmers focus on when designing software and applications that we use to solve problems.

A **data type** determines how computers process data by defining the possible values for data and the possible functionality or operations on that data. For example, the integer data type is defined as values from a certain range of positive or negative whole numbers with functionality including addition, subtraction, multiplication, and division. The string data type is defined as a sequence of characters where each character can be a letter, digit, punctuation, or space, with functionalities that include adding or deleting a character from a string, concatenating strings, and comparing two strings based, for example, on their alphabetical order.

Data types like strings are an example of **abstraction**, the process of simplifying a concept in order to represent it in a computer. The string data type takes a complex concept like a sentence and represents it in terms of more basic data that a computer can work with. When a computer compares two strings, it is really comparing the individual numerical character codes (see Chapter 5 Hardware Realizations of Algorithms: Computer Systems Design) corresponding to each pair of characters within the two strings.

In this section, we will learn how to solve problems by choosing abstractions for complex data. We will see that just as our data grows more complex, so do our algorithms.

Introduction to Algorithms

An algorithm is a sequence of precise instructions that operate on data. We can think of recipes, plans, or instructions from our daily lives as examples of algorithms. Computers can only execute a finite pre-defined set of instructions exactly as instructed, which is why programming can feel like such a different way of communicating as compared to our natural human languages. A **program** is an implementation (realization) of an algorithm written in a formal programming language.

Although each programming language is different from all the others, there are still common ideas across all of them. Knowing just a few of these common ideas enables computer scientists to address a wide variety of problems without having to start from scratch every single time. For example, the abstraction of string data enables programmers to write programs that operate on human-readable letters, digits, punctuation, or spaces without having to determine how to delve into each of these concepts. Programming languages allow us to define abstractions for representing ideas in a computer (see Chapter 4 Linguistic Realization of Algorithms: Low-Level Programming Languages for more).

The study of data structures and algorithms focuses on identifying what is known as a **canonical algorithm**: a well-known algorithm that showcases design principles helpful across a wide variety of problems. In this chapter, rather than focusing on the programming details, we will instead focus on algorithms and the ideas behind them.

Understanding Data Structures

For many real-world problems, the ability to design an algorithm depends on how the data is represented. A **data structure** is a complex data type with two equally important parts:

1. a specific **representation** or way of organizing a collection of more than one **element**, which is an individual value or data point, and
2. a specific **functionality** or operations such as adding, retrieving, and removing elements.

In our previous example, a string is a data structure for representing sentences as a sequence of characters. It has specific functionality such as character insertion or deletion, string concatenation, and string comparison.

Although the values for complex data are often diverse, computer scientists have designed data structures so that they can be reused for other problems. For example, rather than designing a specialized data structure for sentences in every human language, we often use a single, universal string data structure to represent sentences, including characters from different languages in the same sentence. (We will later see some drawbacks of generalizing assumptions in the design of data structures and algorithms.) In addition, computers take time to execute algorithms, so computer scientists are concerned about efficiency in terms of how long an algorithm takes to compute a result.

Among the different types of universal data structures, computer scientists have found it helpful to categorize data structures according to their functionality without considering their specific representation. An **abstract data type (ADT)** consists of all data structures that share common functionality but differ in specific representation.

Common abstract data types for complex data follow, and list and set types are shown in Figure 3.2. We will discuss each abstract data type in more detail together with their data structure implementations.

- A **list** represents an ordered sequence of elements and allows adding, retrieving, and removing elements from any position in the list. Lists are indexed because they allow access to elements by referring to the element's **index**, which is the position or address for an element in the list. For example, a list can be used to represent a to-do list, where each item in the list is the next task to be completed in chronological order.
- A **set** represents an unordered collection of unique elements and allows adding, retrieving, and removing elements from the set. Sets typically offer less functionality than lists, but this reduction in functionality allows for more efficient data structure representations. For example, a set can be used to represent the names of all the places that you want to visit in the future.
- A **map** represents unordered associations between key-value pairs of elements, where each key can only appear once in the map. A map is also known as a dictionary since each term (key) has an associated definition (value). Maps are often used in combination with other data structures. For example, a map can be used to represent a travel wish list: each place that you want to visit in the future can be associated with the list of things that you want to do when you arrive at a given place.
- A **priority queue** represents a collection of elements where each element has an associated priority value. In addition to adding elements, priority queues focus on retrieving and removing the element with the highest priority. For example, a priority queue can be used to represent inpatient processing at a hospital emergency room: the patients with more urgent need for care may be prioritized and dealt with first.
- A **graph** represents binary relations among a collection of entities. More specifically, the entities are represented as vertices in the graph, and a directed or undirected edge is added between two vertices to represent the presence or absence of a certain relation. For example, a friendship graph can be used to represent the friendship relations between people, in which case an undirected edge is added between two persons if they are friends. Graphs allow operations such as adding vertices and edges, removing vertices and edges, and retrieving all edges adjacent to a given vertex.

List

Index	0	1	2	3	4	5	6	7
Data	22	39	45	62	69	79	90	98

Set

Index	0	1	2	3	4	5	6	7
Data	22	39	98	45	69	79	65	90

Index	0	1	2	3	4	5	6	7
Data	DC	Alabama	California	Wyoming	New York	Florida	Texas	Arizona

Figure 3.2 Lists and sets are common abstract data types used to represent complex data and can be in the form of integers or string data. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Selecting a Data Structure

Since data representation is a fundamental task in designing algorithms that solve problems, how do we select

data structures for a particular problem? Computer scientists apply a top-down approach.

1. Select an appropriate abstract data type by analyzing the problem to determine the necessary functionality and operations.
2. Select an appropriate data structure by quantifying the resource constraints (usually program running time) for each operation.

The primary concern is the data and the operations to be performed on them. Thinking back to simple data types like numbers, we focused on addition, subtraction, multiplication, and division as the basic operations. Likewise, to represent complex data types, we also focus on the operations that will most directly support our algorithms. After deciding on an abstract data type, we then choose a particular data structure that implements the abstract data type.

Linear Data Structures

If a problem can be solved with an ordered sequence of elements (e.g., numbers, payroll records, or text messages), the simplest approach might be to store them in a list. Some problems require that actions be performed in a strict chronological order, such as processing items in the order that they arrive or in the reverse order. In these situations, a **linear data structure**, which is a category of data structures where elements are ordered in a line, is appropriate. There are two possible implementations for the list abstract data type. The first, an **array list** (Figure 3.3), is a data structure that stores list elements next to each other in memory. The other is a **linked list** (Figure 3.4), which is a list data structure that does not necessarily store list elements next to each other, but instead works by maintaining, for each element, a link to the next element in the list. Both array lists and linked lists are linear data structures because their elements are organized in a line, one after the other. An advantage of array lists is that they allow (random) access to every element in the list in a single step. This is in sharp contrast with linked lists, which only supports "sequential access." On the other hand, linked lists support fast insertion and deletion operations, which array lists do not.

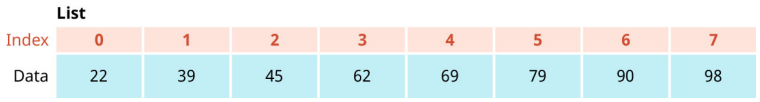


Figure 3.3 An array list stores elements next to each other in memory in the exact list order. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

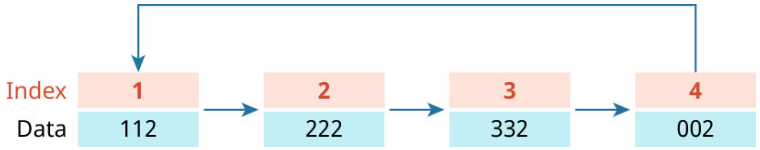


Figure 3.4 A linked list maintains a link for each element to the next element in the list. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Earlier, we introduced sets as offering less functionality than lists. Both array lists and linked lists can also implement the set abstract data type. Sets differ from lists in two ways: sets are unordered—so elements are not assigned specific positions—and sets only consist of unique elements. In addition to implementing sets, linear data structures can also implement the map, priority queue, and graph abstract data types. If linear data structures can cover such a wide range of abstract data types, why learn other data structures? In theory, any complex data can be represented with an array list or a linked list, although it may not be optimal, as we will explain further.

One drawback of relying only on linear data structures is related to the concept of efficiency. Even if linear data structures can solve any problem, we might prefer more specialized data structures that can solve fewer problems more efficiently, and help represent real world data arrangements more closely. This is particularly useful when we have large amounts of data like places or roads in an online map of the entire world. Linear data structures ultimately organize elements in a line, which is necessary for implementing lists but not necessary for other abstract data types. Other data structures specialize in implementing sets, maps, and

priority queues by organizing elements in a hierarchy rather than in a line.

Tree Data Structures

A **tree** is a hierarchical data structure. While there are many kinds of tree data structures, all of them share the same basic organizing structure: a **node** represents an element in a tree or graph. A node may or may not have a descendant. A **child node** is a descendant of another node. Often, the primary node is referred to as the "parent node." Trees maintain a hierarchy through parent-child relationships, which repeat from the **root node** at the top of the tree down to each **leaf node**, which is at the bottom of the tree and has no children. The height of a tree corresponds to the depth of the hierarchy of descendants. Figure 3.5 illustrates the structure and elements of a tree.

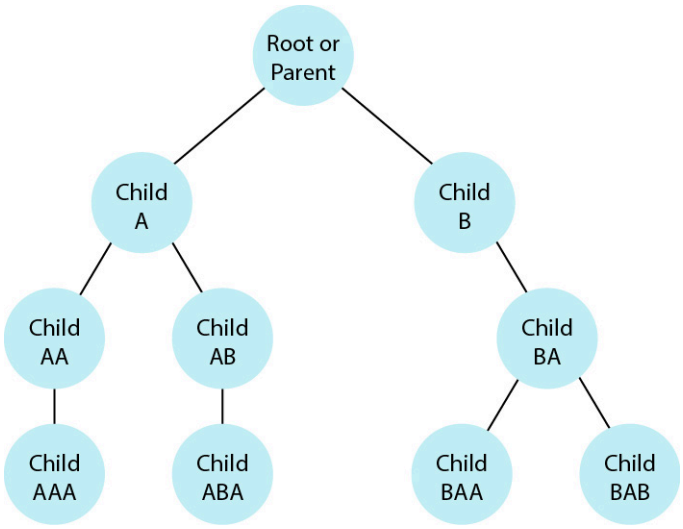


Figure 3.5 A tree is a hierarchical data structure with nodes where each node can have zero or more descendant child nodes. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Binary Search Trees

A **binary search tree** is a kind of tree data structure often used to implement sets and maps with the **binary tree property**, which requires that each node can have either zero, one, or two children, and the **search tree property**, which requires that elements in the tree are organized least-to-greatest from left-to-right. In other words, the values of all the elements of the left subtree of a node have a lesser value than that of the node. Similarly, the values of all the elements of the right subtree of a node have a greater value than that of the node. The search tree property suggests that when elements are read left-to-right in a search tree, we will get the elements in sorted order. For numbers, we can compare and sort numbers by their numeric values. For more complex data like words or sentences, we can compare and sort them in dictionary order. Binary search trees use these intrinsic properties of data to organize elements in a searchable hierarchy (Figure 3.6).

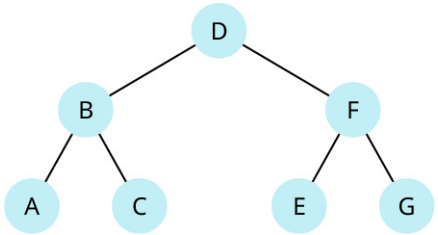


Figure 3.6 A binary search tree organizes elements least to greatest from left to right. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The tree illustrated satisfies the binary tree property based on the natural alphabetical order between letters since the elements in the tree are organized least to greatest from left to right. In other words, for a given list of letters (A, B, C, D, E, F, G), start at the middle of the list of letters with D (the root node) then pick B as the left

sub-node of D which is at the middle of the list of letters (A, B, C) that is on the right of D and pick F as the right sub-node of D, which is at the middle of the list letters (E, F, G) that is on the left of D. Finally, organize the remaining letters under sub-nodes B and F to ensure that they are least-to-greatest from the left to right.

The search tree property is responsible for efficiency improvements over linear data structures. By storing elements in a sorted order in the search tree rather than in an indexed order in a list, binary search trees can more efficiently find a given element. Consider how we might look up words in a dictionary. A binary search tree dictionary storing all the terms and their associated definitions can enable efficient search by starting at the middle of the dictionary (the root node) before determining whether to go left or right based on whether we expect our word to appear earlier or later in the dictionary order. If we repeat this process, we can repeatedly rule out half of the remaining elements each time. Searching for a term in a list-based dictionary that is not sorted, on the other hand, would require us to start from the beginning of the list and consider every word until the end of the list since there is no underlying ordering structure to the elements.

Balanced Binary Search Trees

Binary search trees are not as effective as we have described. The dictionary example represents a best-case scenario for binary search trees. We can only rule out half of the remaining elements each time if the binary search tree is **perfectly balanced**, which means that for every node in the binary search tree, its left and right subtrees contain the same number of elements. This is a strong requirement, since the order in which elements are added to a binary search tree determines the shape of the tree. In other words, binary search trees can easily become unbalanced. It is possible for a binary search tree to look exactly like a linked list, in which each node contains either zero children or one child, which is no more efficient than a linear data structure.

An **AVL tree** (named after its inventors, Adelson-Velsky and Landis) is a balanced binary search tree data structure often used to implement sets or maps with one additional tree property: the **AVL tree property**, which requires the left and right subtrees to be balanced at every node of the tree. AVL trees are just one among many "self-balancing" binary search trees. A **balanced binary search tree** introduces additional properties that ensure that the tree reorganizes elements to maintain balance (Figure 3.7).

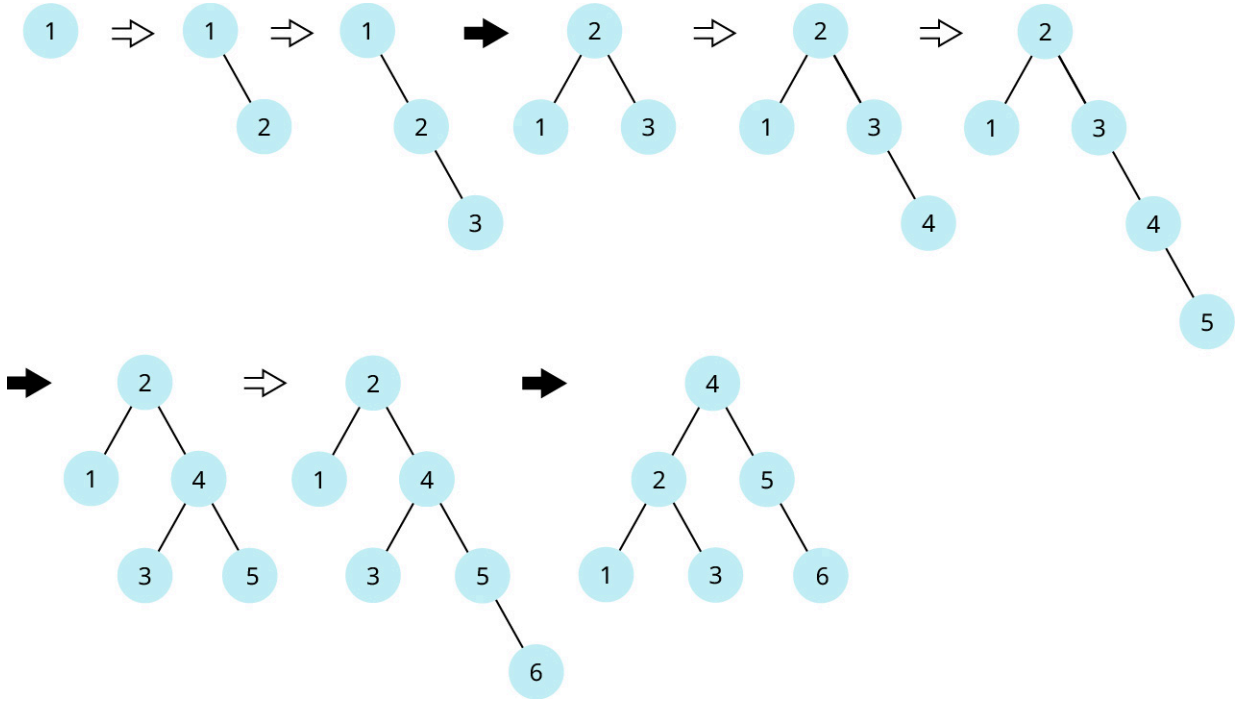
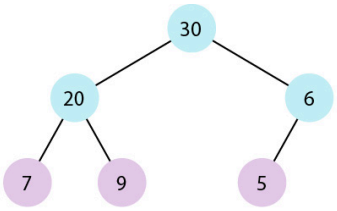


Figure 3.7 An AVL tree rotates nodes in a binary search tree to maintain balance. This sequence of steps illustrates the insertion of numbers 1, 2, 3, 4, 5, 6 into an initially empty AVL tree. (The steps in which rotation occurs are represented by the solid black arrows.) (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Balanced binary search trees such as AVL trees represent just one approach for ensuring that the tree never enters a worst-case situation. There are many other balanced binary search tree data structures in addition to AVL trees. Balanced binary search trees can also be used to implement the priority queue abstract data type if the elements are ordered according to their priority value. But balanced search trees are not the only way to implement priority queues.

Binary Heaps

Priority queues focus on retrieving and removing the highest-priority elements first, adding an element to a priority queue also involves specifying an associated priority value that is used to determine which elements are served next. For example, patients in an emergency room might be served according to the severity of their health concerns rather than according to arrival time. A **binary heap** is a type of binary tree data structure that is also the most common implementation for the priority queue abstract data type (Figure 3.8). A binary heap is not a search tree, but rather a hybrid data structure between a binary tree and an array list. Data is stored as an array list in memory, but the binary heap helps visualize data in the same way that a binary tree does, which makes it easier to understand how data are stored and manipulated. Binary heaps organize elements according to the **heap property**, which requires that the priority value of each node in the heap is greater than or equal to the priority values of its children. The heap property suggests that the highest-priority element will always be the root node where it is efficient to access.



Index	0	1	2	3	4	5
Data	30	20	6	7	9	5

Figure 3.8 A binary heap is the most common implementation of the priority queue abstract data type. The priority value of each node in the binary heap is greater than or equal to the priority values of the children. Note that the value stored in the root node of the right subtree can be smaller than the value stored in any node in the left subtree, while not violating the heap property. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

CONCEPTS IN PRACTICE

Tracking Earthquakes

Earthquakes, hurricanes, tsunamis, and other natural disasters occur regularly, and often demand an international response. How do we track natural disasters and identify the most affected areas in order to coordinate relief and support efforts? In the United States, the U.S. Geological Survey (USGS) is responsible for reporting earthquakes using thousands of earthquake sensors. However, that still leaves many places without earthquake sensors. Outside the United States, sensor technology may be less robust or inaccessible.

Social network data can be used to enhance this information and more quickly alert governments about natural disasters in real-time. By monitoring public social network platforms for occurrences of short posts such as “earthquake?,” we can quickly localize earthquakes based on the user’s geolocation data. However, aggregating and understanding this data—often thousands of data points arriving in minutes—requires efficient data structures and algorithms. We can use a binary heap that implements the priority queue abstract data type for an earthquake-tracking program. For each “earthquake?” post received for a given geolocation, we can increase the priority of the earthquake locations, which helps identify the likely-

earthquake location that is closest to the user's real location. At any time, we can efficiently retrieve the highest-priority element from the priority queue. By choosing to use a binary heap rather than a linear data structure for implementing the priority queue, we can ensure that the earthquake-tracking program is able to keep up with the thousands of posts made every minute during an earthquake.

Graph Data Structures

Both binary search trees and binary heap data structures represent more efficient ways to implement sets, maps, and priority queues by organizing data according to their intrinsic properties. In both cases, the properties of data enable efficient addition, retrieval, and removal of elements.

Graphs are a different kind of abstract data type. Rather than focusing on addition, retrieval, and removal, graphs focus on explicitly modeling the relationships between elements. Graphs afford access not only to elements, but also to relationships between elements.

- A **vertex** represents an element in a graph or a special type of it, such as a tree.
- An **edge** is the relationship between vertices or nodes. Optionally, edges can have associated weights. In a graph abstract data type, the relationships between two vertices connected by an edge are considered **adjacent**.

LINK TO LEARNING

Visualgo is a website that provides animations to help users learn more about algorithms and data structures. They have exercises to help understand several concepts presented in this chapter. You can access animations on various data structures and algorithms (https://openstax.org/r/76Visualgo) such as a linked list, a binary search tree, and graph structures.

In computer networks such as the Internet, graphs can represent individual network routers as nodes with data packets flowing between directly connected routers along edges. Even though not every router is directly connected to every other router, the router at the current node can analyze an incoming data packet to determine which edge it should travel through next. By repeating this process, a data packet can travel from a router on the Internet to another router on the Internet even though the two routers are not directly adjacent to each other.

Graphs are unique in that they can directly represent a wide variety of real-world problems, such as the following:

- A social network, where each vertex is a person, and each edge is a friendship.
- The Web, where each vertex is a webpage, and each edge is a link.
- A campus map, where each vertex is a building, and each edge is a footpath.
- A course prerequisite diagram, where each vertex is a course, and each edge is a prerequisite.

Unlike the list, set, map, and priority queue abstract data types, which have relatively standardized functionality focusing on the addition, retrieval, and removal of elements, the graph abstract data type is much less standardized. Typically, graph algorithm designers will create their own graph data type to represent a problem. The corresponding graph problem can then be represented using or adapting a standard graph algorithm. Unlike programming with other abstract data types, much of the hard work of solving a problem with a graph occurs when programmers decide what the vertices and edges represent, and which graph algorithm would be appropriate to solve the problem. They also must consider the consequences of how they represent the problem.

GLOBAL ISSUES IN TECHNOLOGY

Contact Tracing

Epidemiology is the study of how infectious diseases spread across the world. Within epidemiology, contact tracing attempts to identify confirmed cases of disease and limit its spread by tracing contacted people and isolating them from further spreading the disease.

Graph data structures can help epidemiologists manage the data and people involved in contact tracing. Imagine a graph where each vertex in a tracing graph represents a person, and each edge between two people represents a possible contact. When a person receives a positive test result for contracting the disease, healthcare professionals can identify all the people that they've been in contact with by tracing through the graph.

In addition to improving public health through contact tracing, our imaginary graph can also represent a history of the spread of the disease for epidemiologists to understand how the disease moves through communities. For example, if each vertex includes identity characteristic data such as age, race, or gender, epidemiologists can study which groups of people are most affected by the disease. This can then inform the distribution of vaccines to assist the most impacted groups first.

Complex Data

Now that we have seen several data structure implementations for abstract data, let us consider how these data structures are used in practice. Recall that we compared calculators whose algorithms operated on numbers with the idea of a computer whose algorithms operated on complex data. Data structures can be used to represent complex data by modeling hierarchy and relationships.

We might represent an online retail store as a map associating each item with details such as an image, a brief description, price, and the number of items in stock. This map makes some online storefront features easier to implement than others. Given an item, this map makes it easy to retrieve the details associated with that item. On the other hand, it is not so easy to sort items by price, sort items by popularity, or search for an item by keywords in its description. All these features could be implemented with additional data structures. We can combine multiple data structures together to implement these features. In computer science, we use database systems (see Chapter 8 Data Management) that work behind the scenes in many applications to manage these data structures and facilitate long-term storage and access to large amounts of data.

Just as calculators have algorithms for calculating numbers, computers have algorithms for computing complex data. Data structures represent these complex data, and algorithms act on these data structures.

3.2 Algorithm Design and Discovery

Learning Objectives

By the end of this section, you will be able to:

- Understand the approach to solving algorithmic problems
- Explain how algorithm design patterns are used to solve new problems
- Describe how algorithms are analyzed

Our introduction to data structures focused primarily on representing complex data. But computer scientists are also interested in designing algorithms for solving a wider variety of problems beyond storing and retrieving data. For example, they may want to plan a route between a start location and an end location on a map. Although every real-world problem is unique, computer scientists can use a general set of principles to design solutions without needing to develop new algorithms from scratch. Just like how many data structures

can represent the same abstract data type, many different solutions exist to solve the same problem.

Algorithmic Problem Solving

An algorithm is a sequence of precise instructions that takes any input and computes the corresponding output, while **algorithmic problem-solving** refers to a particular set of approaches and methods for designing algorithms that draws on computing's historical connections to the study of mathematical problem solving. Early computer scientists were influenced by mathematical formalism and mathematical problem solving. George Pólya's 1945 book, *How to Solve It*, outlines a process for solving problems that begins with a formal understanding of the problem and ends with a solution to the problem. As an algorithm's input size is always finite, finding a solution to an algorithmic problem can always be accomplished by exhaustive search. Therefore, the goal of algorithmic problem-solving, as opposed to mathematical problem solving, is to find an "efficient" solution, either in terms of execution time (e.g., number of computer instructions) or space used (e.g., computer memory size). Consequently, the study of algorithmic problem-solving emphasizes the formal **problem** or task, with specific input data and output data corresponding to each input. There are many other ways to solve problems with computers, but this mathematical approach remains the dominant approach in the field. Here are a few well-known problems in computer science that we will explore later in this chapter.

A **data structure problem** is a computational problem involving the storage and retrieval of elements for implementing abstract data types such as lists, sets, maps, and priority queues. These include:

- **searching**, or the problem of retrieving a target element from a collection of elements
- **sorting**, or the problem of rearranging elements into a logical order
- **hashing**, or the problem of assigning a meaningful integer index for each object

A **graph problem** is a computational problem involving graphs that represent relationships between data. These include:

- **traversal**, or the problem of exploring all the vertices in a graph
- **minimum spanning tree** is the problem of finding a lowest-cost way to connect all the vertices to each other
- **shortest path** is the problem of finding the lowest-cost way to get from one vertex to another

A **string problem** is a computational problem involving text or information represented as a sequence of characters. Examples include:

- **matching**, or the problem of searching for a text pattern within a document
- **compression**, or the problem of representing information using less data storage
- **cryptography**, or the problem of masking or obfuscating text to make it unintelligible

Modeling

Computer scientists focus on defining a **problem model**, often simply called a model, which is a simplified, abstract representation of more complex real-world problems. They apply the algorithmic problem-solving process mentioned previously to design algorithms when defining models. Algorithms model phenomena in the same way that data structures implement abstract data types such as lists, sets, maps, priority queues, and graphs. But unlike abstract data types, models are not necessarily purely abstract or mathematical concepts. Models are often linked to humans and social phenomena. A medical system might want to decide which drugs to administer to which patients, so the algorithm designer might decide to *model* patients as a complex data type consisting of age, sex, weight, or other physical characteristics. Because models represent *abstractions*, or simplifications of real phenomena, a model must emphasize some details over others. In the case of the medical system, the algorithm designer emphasized physical characteristics of people that were deemed important and chose to ignore other characteristics, such as political views, which were deemed less important for the model.

If an algorithm is a solution to a problem, then the model is the frame through which the algorithm designer

defines the rules and potential outcomes. Without models, algorithm designers would struggle with the infinite complexity and richness of the world. Imagine, for example, designing a medical system that models patients at the level of individual atoms. This model offers a detailed representation of each patient in the most physical or literal sense. But this model is impractical because we do not know how particular configurations and collections of atoms contribute to a person's overall health. Compared to this atomic-scale model, our former model consisting of age, sex, weight, and other physical characteristics is more practical for designing algorithms, but necessarily involves erasing our individual humanity to draw certain conclusions.

In order to design algorithms, we need to be able to focus on relevant information rather than detailed representations of the real world. Further, computer science requires a philosophical mind to aid in problem solving. According to Brian Cantwell Smith, philosopher and cognitive and computer scientist, "Though this is not the place for metaphysics, it would not be too much to say that every act of conceptualization, analysis, or categorization, does a certain amount of violence to its subject matter, in order to get at the underlying regularities that group things together."[1] Without performing this "violence," there would be too many details to wade through to create a useful algorithm.

The relationship between algorithms, the software they empower, and the social outcomes they produce is currently the center of contested social and political debate. For example, all media platforms (e.g., Netflix, Hulu, and others) use some level of targeted advertising based on user preferences in order to recommend specific movies or shows to their users. Users may not want their information to be used in this way, but there must be some degree of compromise to make these platforms attractive and useful to people.

On the one hand, the technical definition of an algorithm is that it represents complex processes as a sequence of precise instructions operating on data. This definition does not overtly suggest how algorithms encode social outcomes. On the other hand, computer programs are human-designed and socially engineered. Algorithm designers simplify complex real-world problems by removing details so that they can be modeled as computational problems. Because software encodes and automates human ideas with computers, software engineers wield immense power through their algorithms.

To further complicate the matter, software engineering is often a restrictive and formal discipline. Problem modeling is constrained by the **model of computation**, or the rules of the underlying computer that is ultimately responsible for executing the algorithm. Historically, computer science grew from its foundations in mathematics and formal logics, so algorithms were specialized to solve specific problems with a modest model of the underlying phenomena. This approach to algorithm design solves certain types of problems so long as they can be reasonably reduced to models that operate on a modest number of variables—however many variables the algorithm designer can keep in mind. In the case of the medical system, the algorithm designer identified certain characteristics as particularly useful for computing a result.

But there are many other problems that defy this approach, particularly tasks that involve subtle and often unconscious use of human sensory and cognitive faculties. An example of this is facial recognition. If asked to describe how we recognize a particular person's face, an algorithm designer would be challenged to identify specific variables or combinations of variables that correspond to only a single person. The formal logic required to define an algorithm is strict and absolute, whereas our understanding human faces is defined by many subtle factors that are difficult for anyone to express using formal logic.

INDUSTRY SPOTLIGHT

Machine Learning Algorithms

A machine learning algorithm addresses these kinds of problems by using an alternative model of

1 B. C. Smith, "The limits of correctness." *ACM SIGCAS Comput. Soc.*, vol. 14, 15, no. 1, 2, 3, 4, pp. 18-26, Jan. 1985. https://doi.org/10.1145/379486.379512.

computation, one that focuses on generalized algorithms designed to solve problems with a massive model of the underlying phenomena. Instead of attempting to identify a few key variables for facial recognition, for instance, machine learning algorithms can take as input a digital image represented as a rectangular grid of colored pixels. While each pixel in the image offers very little information about the person in mind, the facial features unique to each human arise from the arrangements and patterns of pixels that result from seeing many images of the same person.

Think about the way your Apple iPhone or Google Pixel phone may look at you when you try to access it and have facial recognition enabled. The algorithm is not going to try to match your face to a saved picture of you because it would not work all the time if you do not look exactly like you did in the picture. Rather, it uses machine learning to extract patterns out of a person's face and match them, making it possible to recognize people all the time even if they are wearing glasses but was not wearing them when they set up facial recognition on their phone. This method does seem to mimic the way humans recognize people, even if they have not seen them for decades.

Machine learning algorithms offer a more robust approach to modeling these kinds of problems that are not easily expressed in formal logic. But in this chapter, we focus on the earlier, classical perspective on algorithmic problem-solving with the end goal of designing specialized algorithms to solve problems with modest models of the underlying phenomena.

Search Algorithms

In computer science, searching is the problem of retrieving a target element from a collection that contains many elements. There are many ways to understand search algorithms; depending on the exact context of the problem and the input data, the expected output might differ. For example, suppose we want to find the target term in a dictionary that contains thousands or millions of terms and their associated definitions. If we represent this dictionary as a list, the search algorithm would return the index of the term in the dictionary. If we represent this dictionary as a set, the search algorithm would return whether the target is in the dictionary. If we represent this dictionary as a map, the search algorithm would return the definition associated with the term. The dictionary data structure has implications on the output of the search algorithm. Algorithmic problem-solving tends to be iterative because we might sometime later realize that our data structures need to change. Changing the data structures, in turn, often also requires changing the algorithm design.

Despite these differences in output, the underlying canonical searching algorithm can still follow the same general procedure. The two most well-known canonical searching algorithms are known as sequential search and binary search, which are conducted on linear data structures, such as array lists.

- Sequential search (Figure 3.9). Open the dictionary to the first term. If that term happens to be the target, then great—we have found the target. If not, then repeat the process by reading the next term in the dictionary until we have checked all the terms in the dictionary.
- Binary search (Figure 3.10). Open the dictionary to a term in the middle of the dictionary. If that term happens to be the target, then great—we have found the target. If not, then determine whether the target comes before or after the term we just checked. If it comes before, then repeat the process except on the first half of the dictionary. Otherwise, repeat the process on the second half of the dictionary. Each time, we can ignore half of the remaining terms based on the place where we would *expect* to find the target in the dictionary.

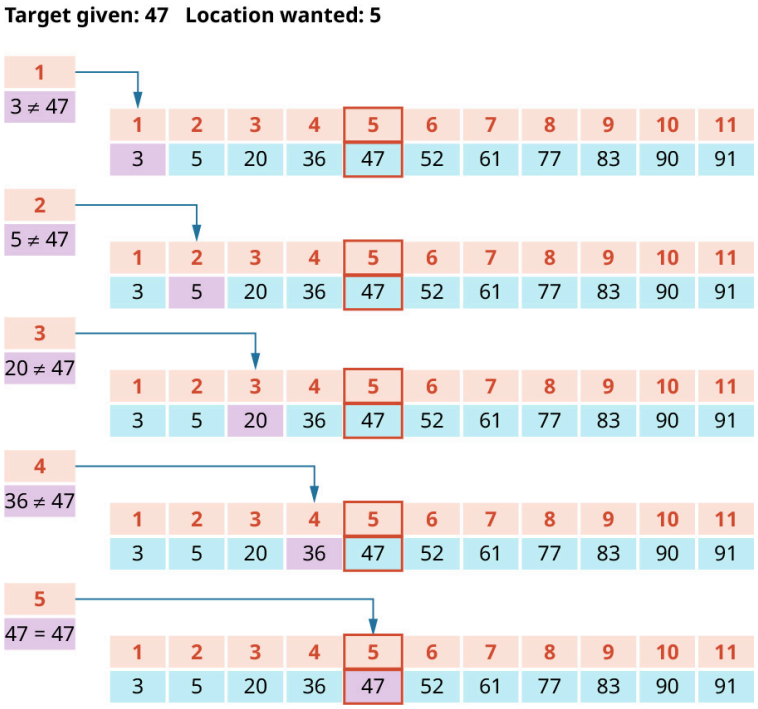


Figure 3.9 A sequential search can find the number 47 in an array by checking each number in order. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

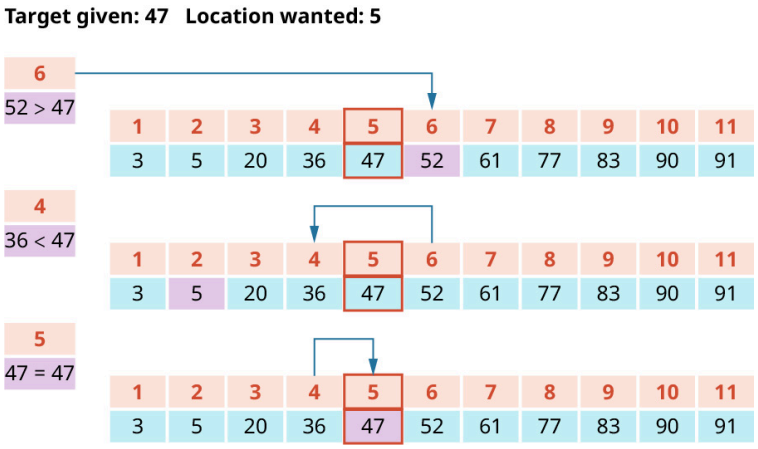


Figure 3.10 A binary search can find the number 47 in an array by determining whether the desired number comes before or after a chosen number. It eliminates half of existing data points and then searches in the remaining half, repeating the pattern, until the number is found. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Algorithm Design Patterns

This case study of canonical search algorithms demonstrates some ideas about algorithmic problem-solving, such as how algorithm design involves iterative improvement (from sequential search to binary search). But this case study does not demonstrate how algorithms are designed in practice. Algorithm designers are occasionally inspired by real-world analogies and metaphors, such as relying on sorted order to divide a dictionary into two equal halves. More often, they depend on knowledge of an existing **algorithm design pattern**, or a solution to a well-known computing problem, such as sorting and searching. Rather than develop wholly new ideas each time they face a new problem, algorithm designers instead apply one or more algorithm design patterns to solve new problems. By focusing on algorithm design patterns, programmers can solve a wide variety of problems without having to invent a new algorithm every time.

For example, suppose we want to design an autocomplete feature, which helps users as they type text into a program by offering word completions for a given prefix query. Algorithm designers begin by modeling the

problem in terms of more familiar data types.

- The input is a prefix query, such as a string of letters that might represent the start of a word (e.g., “Sea”).
- The output is a list of matching terms (completion suggestions) for the prefix query.

In addition to the input and output data, we assume that there is a list of potential terms that the algorithm will use to select the matching terms.

There are a few different ways we could go about solving this problem. One approach is to apply the sequential search design pattern to the list of possible words (Figure 3.11). For each term in the list, we add it to the result if it matches the prefix query. Another approach is to first *sort* the list of potential terms and then apply two binary searches: the first binary search to find the first matching term and the second binary search to find the last matching term. The output list is all the terms between the first match and the last match.

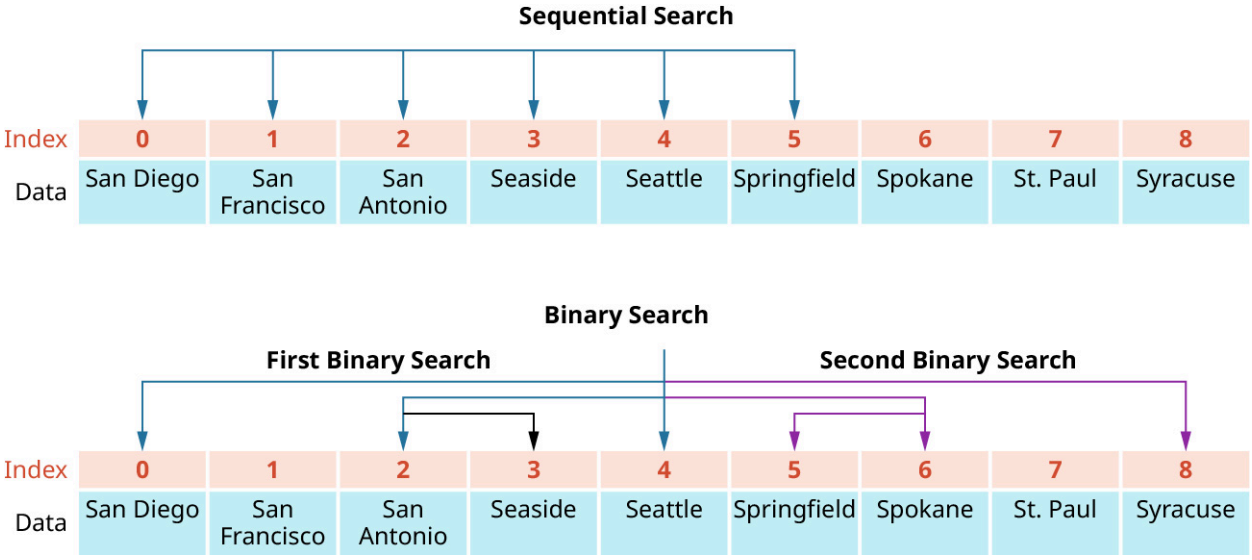


Figure 3.11 Sequential search needs to check every term to see if it matches the prefix “Sea,” whereas two binary searches can be used to find the start and end points of the matching terms in the list. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

TECHNOLOGY IN EVERYDAY LIFE

Online Autocomplete Algorithms

In online mapping, autocomplete might take the prefix *Sea* and automatically suggest the city *Seattle*. We know that search algorithms solve this problem by maintaining a sorted collection of place suggestions. But many online mapping applications allow users to change maps as the places change in the real world. How can we design the autocomplete feature to support real-world user changes? To apply the binary search algorithm, all place names must be stored in a sorted array-based list. So, every change will also need to maintain the sorted order.

If we instead add all the place names to a binary search tree, what are the steps for the autocomplete algorithm? How does this choice affect additions, changes, and removals?

Algorithm Analysis

Rather than rely on a direct analogy to our human experiences, these two algorithms for the autocomplete feature compose one or more algorithm design patterns to solve the problem. How do we know which approach might be more appropriate to use? One type of analysis is known as **algorithm analysis**, which studies the outputs produced by an algorithm as well as how the algorithm produces those outputs. Those

outputs are then evaluated for **correctness**, which considers whether the outputs produced by an algorithm match the expected or desired results across the range of possible inputs. An algorithm is considered correct only if its computed outputs are consistent with all the expected outputs; otherwise, the algorithm is considered incorrect.

Although this definition might sound simple, verifying an algorithm for correctness is often quite difficult in practice, because algorithms are designed to generalize and automate complex processes. The most direct way to verify correctness is to check that the algorithm computes the correct output for *every* possible input. This is not only computationally difficult, but even potentially impossible to achieve, since some algorithms can accept an infinite range of inputs.

Verifying the correctness of an algorithm is difficult not only due to generality, but also due to ambiguity. Earlier, we saw how canonical searching algorithms may have different outputs according to the input collection type. What happens if the target term contains special characters or misspellings? Should the algorithm attempt to find the closest match? Some ambiguities can be resolved by being explicit about the expected output, but there are also cases where ambiguity simply cannot be resolved in a satisfactory way. If we decide to find the closest match to the target term, how does the algorithm handle cultural differences in interpretation? If humans do not agree on the expected output, but the algorithm *must* compute some output, what output does it then compute? Or, if we do not want our algorithms to compute a certain output, how does it recognize those situations?

Correctness primarily considers consistency between the algorithm and the model, rather than the algorithm and the real world. Our autocomplete model from earlier returned all word completions that matched the incomplete string of letters. But in practice, this output would likely be unusable: a user typing "a" would see a list of all words starting with the letter "a." Since our model did not specify how to order the results, the user might get frustrated by the irrelevancy of many of the word completions. Suppose we remedy this issue by defining a relevancy metric: every time a user completes typing a word, increase that word's relevancy for future autocompletion requests. But, as Safiya Noble showed in *Algorithms of Oppression*, determining relevance in this universalizing way can have undesirable social impacts. Perhaps due to relevancy metrics determined by popular vote, at one point, Google search autosuggestions included ideas like:

- Women cannot: drive, be bishops, be trusted, speak in church
- Women should not: have rights, vote, work, box
- Women should: stay at home, be in the kitchen
- Women need to: be put in their places, know their place, be controlled, be disciplined[2]

Noble's critique extends further to consider the intersection of social identities such as race and gender as they relate to the outputs of algorithms that support our daily life.

GLOBAL ISSUES IN TECHNOLOGY

Searching for Identity

In *Algorithms of Oppression*, Safiya Noble describes how search engines can amplify sexist, racist, and misogynistic ideas. While searching for "Black girls," "Latina girls," and "Asian girls" circa 2013, Safiya was startled by how many of the top search results and advertisements that appeared on the first page of Google Search led to pornographic results when her input query did not at all suggest anything pornographic. In contrast, searching for "White girls" did not include pornographic results. As algorithms become more commonplace in our daily lives, they also become a more potent force for determining certain social futures. Algorithms are immensely powerful in their ability to affect not only how we act, but also what we see, what we hear, what we believe about the world, and even what we believe about

2 S.U. Noble, *Algorithms of Oppression: How Search Engines Reinforce Racism*. NYU Press, 2018.

ourselves.

As the amount of input data increases, computers often need more time or storage to execute algorithms. This condition is known as **complexity**, which is based on the degree computational resources that an algorithm consumes during its execution in relation to the size of the input. More computational time also often means consuming more energy. Given the exponential explosion in demand for data and computation, designing efficient algorithms is not only of practical value but also existential value as computing contributes directly to global warming and resultant climate crises.

THINK IT THROUGH

Content Moderation

Online social media platforms facilitate social relationships between users by allowing users to create and share content with each other. This user-generated content requires moderation, or methods for managing content shared between the platform users. Some researchers argue that content moderation defines a social network platform; in other words, content moderation policies determine exactly what content can be shared on the platform, which in turn defines the value of information. As social media platforms become increasingly prevalent, the information on these platforms plays an important role in influencing their users.

One approach for content moderation is to recruit human moderators to review toxic content, or content that is profane, abusive, or otherwise likely to make someone disengage. An algorithm could be developed to determine the toxicity of a piece of content, and the most toxic content could be added to a priority queue for priority moderation.

What are the consequences of this approach? How does the definition of toxicity prioritize (or de-prioritize) certain content? Who does it benefit? Consider the positionality of the users that interact with the platform:

- marginalized users of the platform, who may be further marginalized by this definition of toxicity.
- content moderators, who are each reviewing several hundred pieces of the most toxic content for hours every day.
- legal teams, who want to mitigate government regulations and legislation that are not aligned with corporate interests.
- social media hackers, or users who want to leverage the way certain content is prioritized in order to deliberately shape public opinion.

3.3 Formal Properties of Algorithms

Learning Objectives

By the end of this section, you will be able to:

- Understand time and space complexity
- Compare and contrast asymptotic analysis with experimental analysis
- Explain the Big O notation for orders of growth

Beyond analyzing an algorithm by examining its outputs, computer scientists are also interested in examining its efficiency by performing an algorithmic **runtime analysis**, a study of how much time it takes to run an algorithm.

If you have access to a runnable program, perhaps the most practical way to perform a runtime analysis is to

time exactly how long it takes to run the program with a stopwatch. This approach, known as **experimental analysis**, evaluates an algorithm's runtime by recording how long it takes to run a program implementation of it. Experimental analysis is particularly effective for identifying performance bugs or code that consumes unusually large amounts of computation time or system resources, even though it produces the correct output. In e-commerce, for example, performance bugs that result in slow website responsiveness can lead to millions of dollars in lost revenue. In the worst-case scenario, performance bugs can even bring down entire websites and networks when systems are overloaded and cannot handle incoming requests. As the Internet becomes more heavily used for information and services, performance bugs can have direct impacts on health and safety if the computer infrastructure cannot keep up with demand.

While experimental analysis is useful for improving the efficiency of a program, it is hard to use if we do not already have a working program. Programming large systems can be expensive and time-consuming, so many organizations want to compare multiple algorithm designs and approaches to identify the most suitable design *before* implementing the system. Even with sample programs to represent each algorithm design, we can get different results depending on the processing power, amount of memory available, and other features of the computer that is running the program.

Designing more efficient algorithms is not just about solving problems more quickly, but about building a more sustainable future. In this section, we will take a closer look at how to formally describe the efficiency of an algorithm without directly executing a working program.

CONCEPTS IN PRACTICE

Performance Profiling

Modern computer systems are complicated. Algorithms are just one component in a much larger ecosystem that involves communication between many other subsystems, other computers in a data center, and other systems on the Internet. Algorithmic runtime analysis focuses on the properties of the algorithm rather than all the different ways the algorithm interacts with the rest of the world. But once an algorithm is implemented as a computer program, these interactions with the computing ecosystem play an important role in determining program performance.

A profiler is a tool that measures the performance (runtime and memory usage) of a program. Profilers are commonly used to diagnose real-world performance issues by producing graphs of how computational resources are used in a program. A common graph is a flame graph (Figure 3.12) that visualizes resource utilization by each part of a program to help identify the most resource-intensive parts of a program. Saving even a few percentage points of resources can lead to significantly reduced time, money, and energy expenditure. As the global demand for computation continues to increase, performance engineers who know how to leverage profilers to analyze systems and implement resource-saving changes will be key to a green energy future.

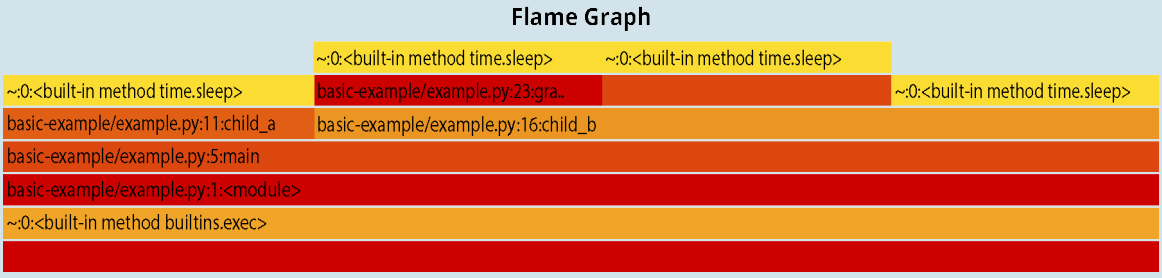


Figure 3.12 A flame graph shows which parts of a program require the most resources. The x-axis shows relative duration and the width indicates the percentage of total duration spent in a function. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Time and Space Complexity

One way to measure the efficiency of an algorithm is through **time complexity**, a formal measure of how much time an algorithm requires during execution as it relates to the size of the problem. In addition to time complexity, computer scientists are also interested in **space complexity**, the formal measure of how much memory an algorithm requires during its execution as it relates to the size of the problem.

Both time and space complexity are formal measures of the efficiency of an algorithm as it relates to the size of the problem, particularly when we are working with large amounts of complex data. For example, gravity, the universal phenomenon by which things attract and move toward each other, can be modeled as forces that act on every pair of objects in the universe. Simulating a subset of the universe that contains only 100 astronomical bodies will take a lot less time than a much larger universe with billions, trillions, or even more bodies, all of which gravitate toward each other. The size of the problem plays a large role in determining how much time an algorithm requires to execute. We will often express the size of the problem as a positive integer number corresponding to the size of the dataset such as the number of astronomical bodies in our simulation.

The goal of time and space complexity analysis is to produce a simple and easy-to-compare characterization of the efficiency of an algorithm as it relates to the size of the problem. Consider the following description of an algorithm that searches for a target word in a list. Start from the very beginning of the list and check if the first word is the target word. If it is, we have found the word. If it is not, then continue to the next word in the list and repeat the process.

The first task is to identify a metric for representing the size of the problem. Typically, time complexity analysis assumes **asymptotic analysis**, focusing on evaluating the time that an algorithm takes to produce a result as the size of the input increases. There are two inputs to this algorithm: (1) the list of words, and (2) the target word. The average length of an English word is about five characters, so the size of the problem is primarily determined by the number of words in the list rather than the length of any word. (This assumption might not be right if our dataset was instead a DNA sequence consisting of millions of nucleotides—the time it takes to compare a pair of long DNA sequences might be more important than the number of DNA sequences being compared.) Identifying the size of the problem is an important first task because it determines the other factors we can consider in the following tasks.

The next task is to model the number of steps needed to execute the algorithm while considering its potential behavior on all possible inputs. A **step** represents a basic operation in the computer, such as looking up a single value, adding two values, or comparing two values. How does the runtime change as the size of the problem increases? We can see that the “repeat” part of our description is affected by the number of words in the list; more words can potentially lead to more repetitions.

In this case we are choosing a **cost model**, which is a characterization of runtime in terms of more abstract operations, such as the number of repetitions. Rather than count single steps, we instead count repetitions. Each repetition can involve several lookups and comparisons. By choosing each repetition as the cost model, we declare that the few steps needed to look up and compare elements can be effectively treated as a single operation to simplify our analysis.

However, this analysis is not quite complete. We might find the target word early in the list even if the list is very large. Although we defined the size of the problem as the number of words in the list, the size of the problem does not account for the exact words and word ordering in the list. Computer scientists say that this algorithm has a best-case situation when the word can be found at the beginning of the list, and a worst-case situation when the word can only be found at the end of the list (or, perhaps, not even in the list at all). One way to account for the variation in runtime is via **case analysis**, which is based on factors other than the size of the problem.

Finally, we can formalize our description using either precise English or a special mathematical notation called **Big O notation**, which is the most common type of asymptotic notation in computer science used to measure

worst-case complexity. In precise English, we might say that the time complexity for this sequential search algorithm has two cases (Figure 3.13):

- In the best-case situation (when the target word is at the start of the list), sequential search takes just one repetition to find the word in the list.
- In the worst-case situation (when the target word is either at the end of the list or not in the list at all), sequential search takes *N* repetitions where *N* is the number of words in the list.

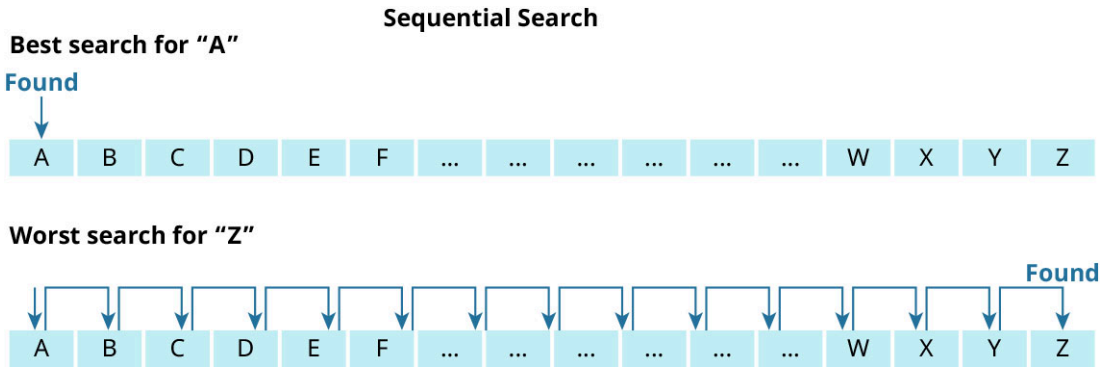


Figure 3.13 The best case for sequential search in a sorted list is to find the word at the top of the list, whereas the worst case is to find the word at the bottom of the list (or not in the list at all). (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

While this description captures all the ideas necessary to communicate the time complexity, computer scientists will typically enhance this description with mathematics to convey a geometric idea of the runtime. The **order of growth** is a geometric prediction of an algorithm's time or space complexity as a function of the size of the problem (Figure 3.14).

- In the best case, the sequential search has a **constant** order of growth that does not take more resources as the size of the problem increases.
- In the worst case, the sequential search has a **linear** order of growth where the resources required to run the algorithm increase at about the same rate as the size of the problem increases. This is with respect to *N*, the number of words in the list.

Constant and linear are two examples of orders of growth. An algorithm with a constant order of growth takes the same amount of time to execute even as the size of the problem grows larger and larger—no matter how large the dictionary is, it is possible to find the target word at the very beginning. In contrast, an algorithm with a linear time complexity will take more time to execute as the size of the problem grows larger, and we can predict that an increase in the size of the problem corresponds to roughly the same increase in the runtime.

This prediction is a useful outcome of time complexity analysis. It allows us to estimate the runtime of the sequential search algorithm on a problem of any size, before writing the program or obtaining a dictionary of words that large. Moreover, it helps us decide if we want to use this algorithm or explore other algorithm designs and approaches. We might compare this sequential search algorithm to the binary search algorithm and adjust our algorithm design accordingly.

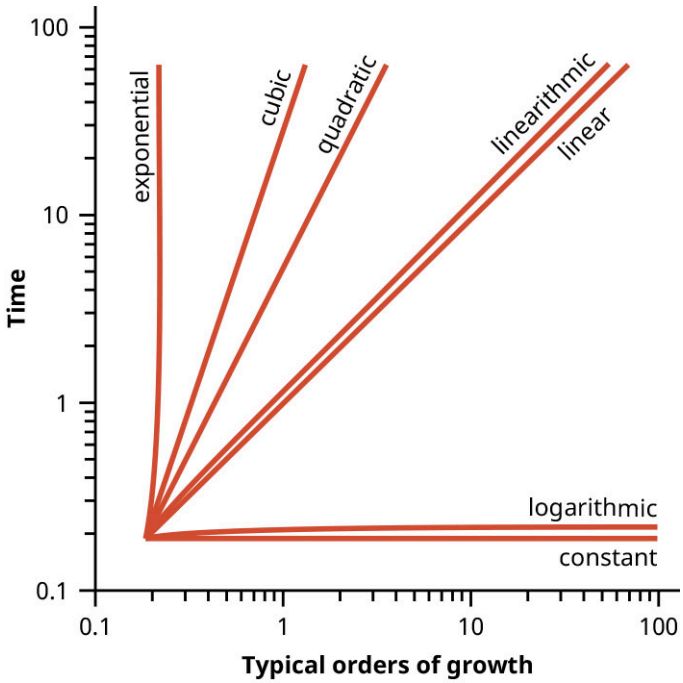


Figure 3.14 The order of growth of an algorithm is useful to estimate its runtime efficiency as the input size increases (e.g., constant, logarithmic, and other orders of growth) to help determine which algorithmic approach to take. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Big O Notation for Orders of Growth

In the 1970s, computer scientists applied **asymptotic notation**, a mathematical notation that formally defines the order of growth. We can use Big O notation to describe the time complexity of the sequential search algorithm. In general, we say a function f(N) is in the class of O(g(N)), denoted by f(N) = O(g(N)) or f(N) in O(g(N)), if when N tends to infinity, the ratio f(N)/g(N) is upper bounded by some constant. The function g(N) is usually some simple function that defines the order of growth such as g(N) = 1 (constant function), g(N) = N (linear function), g(N) = log N (logarithmic function), or other functions as follows:

- In the best case, the order of growth of sequential search is in O(1).
- In the worst case, the order of growth of sequential search is in O(N) with respect to N, the number of words in the list.

The constant order of growth is described in Big O notation as "O(1)" while the linear order of growth is described in Big O notation as "O(N) with respect to N, the size of the problem." Big O notation formalizes the concept of a prediction. Given the size of the problem, N, calculate how long it takes to run the algorithm on a problem of that size. For large lists, in order to double the worst-case runtime of sequential search, we would need to double the size of the list.

O(1) and O(N) are not the only orders of growth.

- O(1), or constant.
- O(log N), or logarithmic.
- O(N), or linear.
- O(N log N), or linearithmic.
- O(N^2), or quadratic.
- O(N^3), or cubic.
- O(2^N), O(3^N), . . . , or exponential.
- O(N!), or factorial.

The O(log N), or logarithmic, order of growth appears quite often in algorithm analysis. The **logarithm** of a

large number tells how many times it needs to be divided by a small number until it reaches 1. The **binary logarithm**, or $\log_2$, tells how many times a large number needs to be divided by 2 until it reaches 1. In the worst case, the time complexity of sequential search is in O(N) with respect to N, the number of words in the list, since each repetition of sequential search rules out one remaining element. How about binary search? In the worst case, the time complexity of binary search is in O(log N) with respect to N, the number of words in the list, since each repetition of binary search rules out half the remaining elements.

Another way to understand orders of growth is to consider how a change in the size of the problem results in a change to the resource usage. When we double the size of the input problem, algorithms in each order of growth respond differently (Figure 3.15).

- O(1) algorithms will not require any more resources.
- O(log N) algorithms will require 1 additional resource unit.
- O(N) algorithms will require 2 times the number of resources.
- O(N log N) algorithms will require a little more than 2 times the number of resources.
- O(N^2) algorithms will require 4 times the number of resources.
- O(N^3) algorithms will require 8 times the number of resources.
- O(2^N), O(3^N), . . . algorithms will require the squared or cubed number of resources.
- O(N!) algorithms will require even more.

This growth compounds, so quadrupling the size of the problem for an O(N^2) algorithm will require 16 times the number of resources. Algorithm design and discovery is often motivated by these massive differences between orders of growth. Note that this explanation of how each order of growth responds differently oversimplifies the problem. Rigorously speaking, a function $f(N)$ expressed in the big-O notation as being is in the class of O($g(N)$) can be much more complex than the simple function $g(N)$. For example, $f(N) = 4\log N + 100 \log(\log N)$ is in O(log N), but when N doubles, $f(2N)$ is definitely not just one unit more than the original function $f(N)$. A similar argument applies for all other functions other than the constant O(1) function.

Order of growth	Algorithm	Execution time for N = 10	Execution time for N = 1,000,000
Constant	O(1)	1 µsec	1 µsec
Logarithmic	O(log N)	3 µsec	18 µsec
Linear	O(N)	10 µsec	1 sec
Log-linear	O(N log N)	33 µsec	19.8 sec
Quadratic	O(N^2)	100 µsec	11.6 days
Cubic	O(N^3)	1 msec	31.7 years
Exponential	O(2^N)	10 msec	Billion of years
Factorial	O(N!)	3.6 sec	Practically infinite

Figure 3.15 This chart shows the time it would take for an algorithm with each of the given orders of growth to finish running on a problem of the given size, N. When an algorithm takes longer than 1025 years to compute, that means it takes longer than the current age of the universe. (data source: Geeks for Geeks, "Big O Notation Tutorial—A Guide to Big O Analysis," last updated March 29, 2024; attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

As the size of the problem increases, algorithmic complexity becomes a larger and larger factor. For problems dealing with just 1,000 elements, the time it would take to run an exponential-time algorithm on that problem exceeds the current age of the universe. In practice, across applications working with large amounts of data, O(N log N) is often considered the limit for real-world algorithms. Even then, O(N log N) algorithms cannot be run too frequently. For algorithms that need to run frequently on large amounts of data, algorithm designers target O(N), O(log N), or O(1).

TECHNOLOGY IN EVERYDAY LIFE

Arranging Invisible Icons in Quadratic Time

Have you ever been annoyed by computer slowness? For some users, opening the start menu can take 20 seconds because of an $O(N^2)$ algorithm, where N is the number of *desktop files*. The Microsoft Windows computer operating system allows users to organize files directly on top of their desktop wallpaper. A quadratic-time algorithm arranges these *desktop icons* in a grid layout so that they fill column-by-column starting from the left side of the screen. The algorithm is executed whenever the user opens the start menu or launches the file explorer.

Most users only keep a couple dozen desktop icons, so the $O(N^2)$ algorithm takes microseconds—practically unnoticeable. But for users with hundreds of desktop icons, the impact of each additional icon adds up. With 1,000 desktop files, launching the start menu takes 20 *seconds*. With 10,000 desktop icons, the runtime grows to 30 *minutes*!

To avoid the clutter of thousands of desktop icons, users can hide desktop icons. But this does not prevent the quadratic-time algorithm from running. Users with too many desktop icons beware: your computer slowness may be due to arranging invisible icons in quadratic time.[3]

3.4 Algorithmic Paradigms

Learning Objectives

By the end of this section, you will be able to:

- Apply the divide and conquer technique
- Explain the brute-force method
- Interpret and apply the greedy method
- Understand how to apply reductions to solve problems

Algorithm design patterns are solutions to well-known computing problems. In 3.5 Sample Algorithms by Problem, we will survey algorithm design patterns by problem. As it turns out, many of these algorithm design patterns share similarities in their approaches to solving problems. Here, we will introduce the **algorithmic paradigm**, which is the common concepts and ideas behind algorithm design patterns.

Divide and Conquer Algorithms

A **divide and conquer algorithm** is an algorithmic paradigm that breaks down a problem into smaller subproblems (divide), recursively solves each subproblem (conquer), and then combines the result of each subproblem to form the overall solution. The algorithm idea of recursion is fundamental to divide and conquer algorithms because it solves complex problems by dividing input data into smaller instances of the same problem known as subproblems. Such recursion calls terminate when the inputs become so small or so simple that other non-recursive procedures can provide the answers.

A **subproblem** is a smaller instance of a problem that can be solved independently, and each subproblem can be solved independently of other subproblems by reapplying the same recursive algorithm. To design recursive subproblems, algorithm designers often focus on identifying structural self-similarity in the input data. This process repeats until the input data is small enough to solve directly. Once all the subproblems have been solved, the recursive algorithm reassembles each of these independent solutions to compute the result for the original problem.

3 B. Dawson, "Arranging invisible icons in quadratic time," 2021. https://randomascii.wordpress.com/2021/02/16/arranging-invisible-icons-in-quadratic-time/

Earlier, we introduced binary search to find a target within a sorted list as an analogy for finding a term in a dictionary sorted alphabetically. Instead of starting from the beginning of the dictionary and checking each term, as in a sequential search, we could instead start from the middle and look left or right based on where we would expect to find the term in the dictionary. But binary search can also be understood as an example of a divide and conquer algorithm (Figure 3.16).

1. The problem of finding a target within the entire sorted list is broken down (divided) into the subproblem of finding a target within half of the list after comparing the middle element to the target. Half of the list can be ruled out based on this comparison, leaving binary search to find the target within the remaining half.
2. Binary search is repeated on the remaining half of the sorted list (conquer). This process continues recursively until the target is found in the sorted list (or reported as not in the list at all).
3. To solve the original problem of finding a target within the entire sorted list, the result of the subproblem must inform the overall solution. The original call to binary search reports the same result as its subproblem.

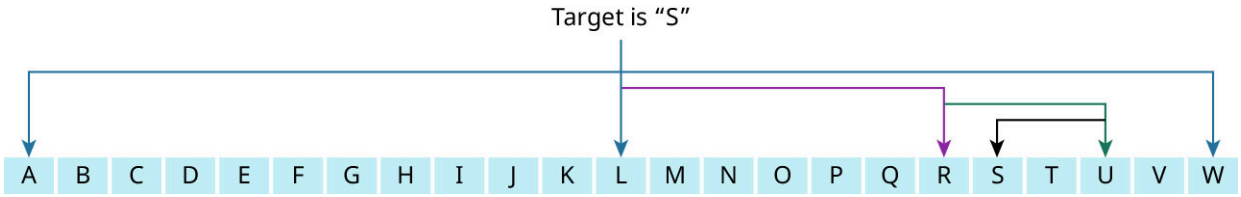


Figure 3.16 Binary search is a divide and conquer algorithm that repeatedly makes one recursive call on half of remaining elements. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Binary search makes a single recursive call on the remaining half of the list as part of the conquer step, but other divide and conquer algorithms make multiple recursive calls to solve their problems. We will also see algorithms that do a lot of work in the final step of combining results more than just reporting the same result as a subproblem.

Given a list of elements in an unknown order, a sorting algorithm should return a new list containing the same elements rearranged into a logical order, such as least to greatest. One canonical divide and conquer algorithm for comparison sorting is called **merge sort**. The problem of comparison sorting is grounded in the comparison operation. The comparison operation is like a traditional weighing scale that tells whether one element is heavier, lighter, or the same weight as another element, but provides no information about the exact weight or value of the element. Though this might seem like a serious restriction, comparison sorting is actually a very rich problem in computer science—it is perhaps the most deeply studied problem in computer science. Choosing comparison as the fundamental operation is also practical for complex data, where it might be hard (or even impossible) to assign an exact numeric ranking (Figure 3.17).

1. The problem of sorting the list is broken down (divided) into two subproblems: the subproblem of sorting the left half and the subproblem of sorting the right half.
2. Merge sort is repeated to sort each half (conquer). This process continues recursively until the sublists are one element long. To sort a one-element list, the algorithm does not need to do anything, since the elements in the list are already arranged in a logical order.
3. To solve the original problem of sorting the entire list, combine adjacent sorted sublists by merging them while maintaining sorted order. Merging each pair of adjacent sorted sublists repeats to form larger and larger sorted sublists until the entire list is fully sorted.

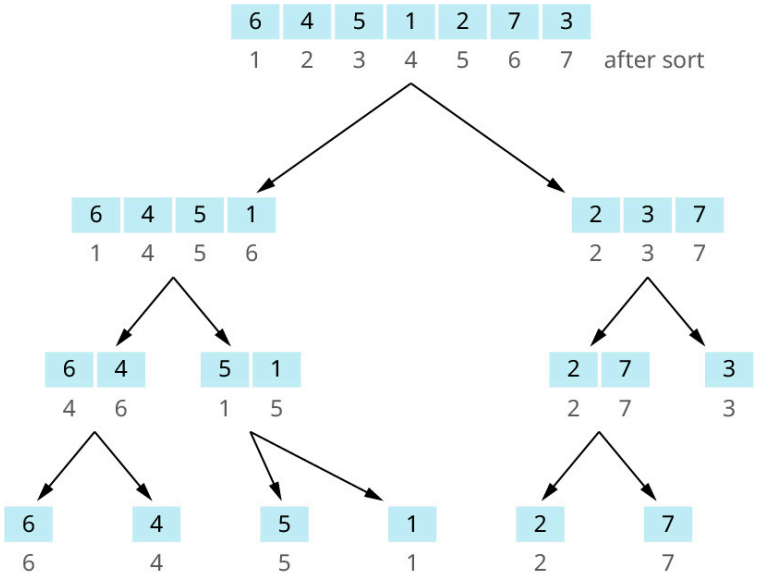


Figure 3.17 Merge sort is a divide and conquer algorithm that repeatedly makes two recursive calls on both halves of the sublist. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Brute-Force Algorithms

Solving a **combinatorial problem** involves identifying the best candidate solution out of a space of many potential solutions. Each solution to a combinatorial problem is represented as a complex data type. Many applications that involve simulating and comparing different options can be posed as combinatorial problems.

- Route planning in online mapping asks, "Out of all the possible routes from point A to point B, which route is the shortest?" (Shortest paths problem)
- Municipal broadband planning asks, "Out of all the possible ways to connect every real-world address to the Internet, which network of connections is the cheapest to build?" (Minimum spanning trees problem)
- The **interval scheduling problem** is a combinatorial problem involving a list of scheduled tasks with the goal of finding the largest non-overlapping set of tasks that can be completed.

Rather than searching for a single element, combinatorial problems focus on finding candidate solutions that might be represented as a list of navigation directions, network connections, or task schedules. And unlike sorting, where the output should be a sorted list, combinatorial problems often attempt to quantify or compare the relative quality of solutions in order to determine the best candidate solution out of a space of many potential solutions. Even if our route plan is not the "best" or shortest route, it can still be a valid solution to the problem.

A **brute-force algorithm** solves combinatorial problems by systematically enumerating all potential solutions in order to identify the best candidate solution. For example, a brute-force algorithm for generating valid credit card numbers might start by considering a credit card number consisting of all zeros, then all zeros except for a one in the last digit place, and so forth, to gradually explore all the possible values for each digit place.

Brute-force algorithms exist for every combinatorial problem, but they are not typically used because of runtime issues. To systematically enumerate all potential solutions, a brute-force algorithm must generate every possible combination of the input data. For example, if a credit card number has sixteen digits and each digit can have any value between zero and nine, then there are 10^{16} potential credit card numbers to enumerate. The **combinatorial explosion** is the exponential number of solutions to a combinatorial problem that makes brute-force algorithms unusable in practice. Despite continual improvements to how quickly computers can execute programs, exponential time brute-force algorithms are impractical except for very small problem input data.

INDUSTRY SPOTLIGHT

Protein Folding

Proteins are one of the fundamental building blocks of biological life. The 3-D shape of a protein defines what it does and how it works. Given the string of a protein's amino acids, a protein-folding problem asks us to compute the 3-D shape of the resulting protein.

Protein folding has been studied since the first images of their structures were created in 1960. In 1972, Christian Anfinsen won the Nobel Prize in Chemistry for his research into the "protein-folding problem," which involved algorithms to predict the structure of proteins. Because of the huge number of possible formations of proteins, computational studies and algorithms are better able to predict the structures. Given that a brute-force algorithm cannot solve this problem in a reasonable amount of time, computational biologists have developed algorithms that generate high-quality approximations or potential solutions that typically are not quite correct, but run in a more reasonable amount of time.

Modern protein-folding algorithms, such as Google's DeepMind AlphaFold machine-learning algorithm,[4] use machine learning to identify protein-folding patterns from millions of input amino acid sequences and corresponding output 3-D conformations. Rather than utilizing a simple rule for selecting the next element to include in the solution, these machine learning algorithms learn highly complicated rulesets from subtle patterns present in the data.

Improving our understanding of protein folding can lead to massive improvements only in medical health contexts such as drug and vaccine development, but also enable us to design biotechnologies such as enzymes for composting plastic waste, and even limit the impact of global warming by sequestering greenhouse gases from the atmosphere.[5] In 2024, the Nobel Prize Committee recognized the impact of this work by granting the Chemistry prize to Demis Hassabis and John M. Jumper for their work on DeepMind and Alphafold[6], as well as David Baker for using a similar tool, Rosetta, to create entirely new proteins.

Greedy Algorithms

A **greedy algorithm** solves combinatorial problems by repeatedly applying a simple rule to select the next element to include in the solution. Unlike brute-force algorithms that solve combinatorial problems by generating all potential solutions, greedy algorithms instead focus on generating just one solution. These algorithms are *greedy* because they select the next element to include based on the immediate benefit.

For example, a greedy interval scheduling algorithm might choose to work on the task that takes the least amount of time to complete; in other words, the cheapest way to mark one task as completed (Figure 3.18). If the tasks are scheduled in advance and we can only work on one task at a time, choosing the task that takes the least amount of time to complete might prevent us from completing multiple other (longer) tasks that just so happen to overlap in time. This greedy algorithm does not compute the right output—it finds a solution but not the optimal solution.

4 W. D. Haven, "DeepMind's protein-folding AI has solved a 50-year-old grand challenge of biology," 2020. https://www.technologyreview.com/2020/11/30/1012712/deepmind-protein-folding-ai-solved-biology-science-drugs-disease/
5 Google DeepMind, "AlphaFold: A solution to a 50-year-old grand challenge in biology," 2020." https://deepmind.com/blog/article/alphafold-a-solution-to-a-50-year-old-grand-challenge-in-biology
6 Google DeepMind, "AlphaFold: Demis Hassabis & John Jumper awarded Nobel Prize in Chemistry," 2024." https://deepmind.google/discover/blog/demis-hassabis-john-jumper-awarded-nobel-prize-in-chemistry/

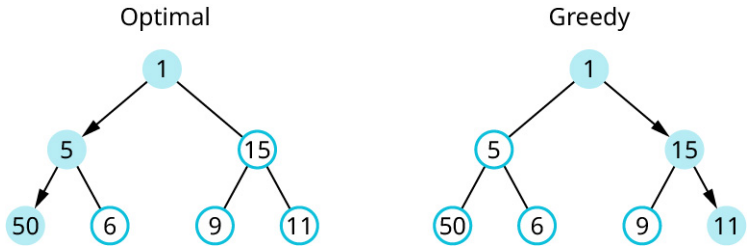


Figure 3.18 Greedy interval scheduling will not work if the simple rule repeatedly selects the shortest interval. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The majority of greedy algorithms do not always compute the best solution. But there are also certain scenarios in which cleverly crafted greedy algorithms guarantee finding optimal solutions. The problems they solve are formulated to ensure that the greedy algorithm never makes a mistake when repeatedly applying the simple rule to select the next element.

Consider the municipal broadband planning problem or, more formally, the minimum spanning tree problem, of finding a lowest-cost way to connect all the vertices in a connected graph to each other. If we want to minimize the sum of the selected edge weights, one idea is to repeatedly select the next edge (connections between vertices) with the lowest weight so long as it extends the reach of the network. Or, in the context of the municipal broadband planning problem, we want to ensure that the next-cheapest connection that we choose to build reaches someone who needs access to the Internet (Figure 3.19).

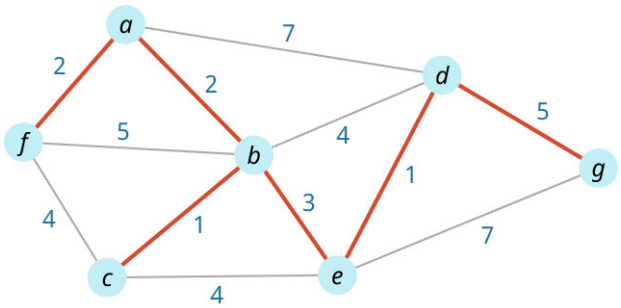


Figure 3.19 Municipal broadband planning can be represented as a minimum spanning trees graph problem where the weight of each edge represents the cost of building a connection between two vertices or places. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

GLOBAL ISSUES IN TECHNOLOGY

Municipal Broadband as a Public Utility

The municipal broadband planning problem is just one component of the larger public policy issue of Internet access. As the Internet and the use of digital platforms becomes the standard mode for communication, many people are viewing municipal broadband as a fundamental public utility and civil right. "Municipal broadband can solve access and affordability problems in areas where private ISPs [Internet service providers] have not upgraded networks to modern speeds, fail to provide service to all residents, and/or charge outrageous rates."[7]

While a minimum spanning tree algorithm can solve the municipal broadband planning problem, the challenges of deploying municipal broadband for everyone is more political rather than algorithmic. But other algorithms can also directly contribute to the way in which society understands the problem. For example, we might use algorithms to better visualize and understand who currently has access to affordable and reliable high-speed Internet. We can design algorithms to ensure equitable distribution,

7 J. Broken, "Victory for municipal broadband as Wash. state lawmakers end restrictions," 2021. Ars Technica, https://arstechnica.com/tech-policy/2021/04/victory-for-municipal-broadband-as-wash-state-lawmakers-end-restrictions

deployment, and integration of new technologies so that marginalized communities can realize the positive economic benefits first. Or we can reconfigure the minimum spanning trees problem model to take specifically account for expanding network access in an equitable fashion.

Computer scientists have designed algorithms that repeatedly apply this simple rule to find an optimal minimum spanning tree:

- **Kruskal's algorithm**, a greedy algorithm which sorts the list of edges in the graph by weight.
- **Prim's algorithm**, a greedy algorithm that maintains a priority queue of vertices in the graph ordered by connecting edge weight.

For most problems, greedy algorithms will not produce the best solution. Instead, algorithm designers typically turn to another algorithmic paradigm called dynamic programming. Still, greedy algorithms provide a useful baseline starting point for understanding problems and designing baseline algorithms for generating potential solutions.

Reduction Algorithms

Algorithm designers spend much of their time modeling problems by selecting and adapting relevant data structures and algorithms to represent the problem and a solution in a computer program. This process of modeling often involves modifying an algorithm design pattern so that it can be applied to the problem. But there is also a different approach to algorithm design that solves problems by changing the input data and output data to fit a preexisting problem. Rather than solve the problem directly, a **reduction algorithm** solves problems by transforming them into other problems (Figure 3.20).

1. Preprocess: Transform the input data so that it is acceptable to an algorithm meant for the other problem.
2. Apply the algorithm meant for the other problem on the preprocessed data.
3. Postprocess: Transform the output of the algorithm meant for the other problem so that it matches the expected output for the original problem.

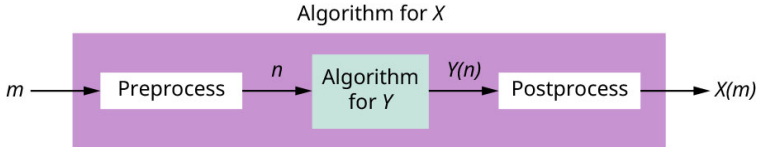


Figure 3.20 A reduction algorithm preprocesses the input data, passes it to another algorithm, and then postprocesses the algorithm's output to solve the original problem. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Consider a slightly different version of the municipal broadband planning problem, where instead of only considering connections (edges), we expand the problem to consider the possibility of installing broadband nodes directly to each address without relying on potentially expensive neighboring connections. That is, all vertices installed with broadband nodes are inter-connected with each other through another broadband network. If we were to run an algorithm for solving the minimum spanning tree problem on this graph directly, then our result would never consider installing broadband nodes directly, since minimum spanning tree algorithms do not consider vertex weights (Figure 3.21).

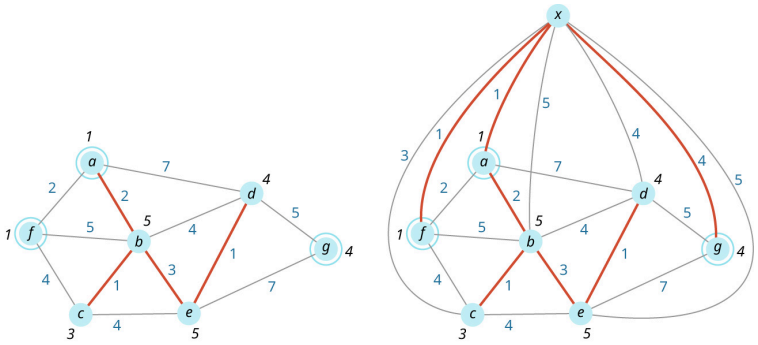


Figure 3.21 The problem of finding a minimum spanning tree in a graph with vertex weights can be reduced to the problem of finding a minimum spanning tree in a graph *without* vertex weights. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

We say that this more complicated municipal broadband planning problem reduces to the minimum spanning tree problem because we can design a reduction algorithm consisting of preprocessing and postprocessing procedures.

- Preprocess: Introduce an extra vertex that does not represent a real location but connects to every address (vertex) in the city. The edge weight of each connection is the cost of installing the broadband node directly at that location.
- Postprocess: After computing a minimum spanning tree for the preprocessed graph, remove the extra vertex that we added during the processing step. Any edges connected to the extra vertex represent a direct broadband node installation, while other edges between real locations are just the same network connections that we saw earlier.

Reduction algorithms enable algorithm designers to solve problems without having to modify existing algorithms or algorithm design patterns. Reduction algorithms allow algorithm designers to rely on optimized canonical algorithms rather than designing a solution by composing algorithm design patterns, which can lead to performance or correctness bugs. Reduction algorithms also enable computer scientists to make claims about the relative difficulty of a problem. If we know that a problem *A* reduces to another problem *B*, *B* is as difficult to solve as *A*.

3.5 Sample Algorithms by Problem

Learning Objectives

By the end of this section, you will be able to:

- Discover algorithms that solve data structure problems
- Understand graph problems and related algorithms

Earlier, we introduced several computing problems, like searching for a target value in a list or implementing an abstract data type (lists, sets, maps, priority queues, graphs). Although every computational problem is unique, these types of problems often share significant similarities with other problems. Computer scientists have identified many canonical problems that represent these common problem templates. Although each of these canonical problems may have many algorithmic solutions, computer scientists have also identified canonical algorithms for solving these problems. In this section, we will introduce canonical problems and survey canonical algorithms for each problem.

Data Structure Problems

Data structure problems are not only useful for implementing data structures, but also as fundamental algorithm design patterns for organizing data to enable efficient solutions to almost every other computing problem.

Searching

Searching is the problem of retrieving a target element from a collection of elements. Searching in a linear data structure, such as an array list, can be done using either sequential search or binary search.

Sequential Search Algorithm

A **sequential search algorithm** is a searching algorithm that sequentially checks the collection element-by-element for the target. The runtime of sequential search is in O(*N*) with respect to *N*, the number of elements in the list (Figure 3.22).

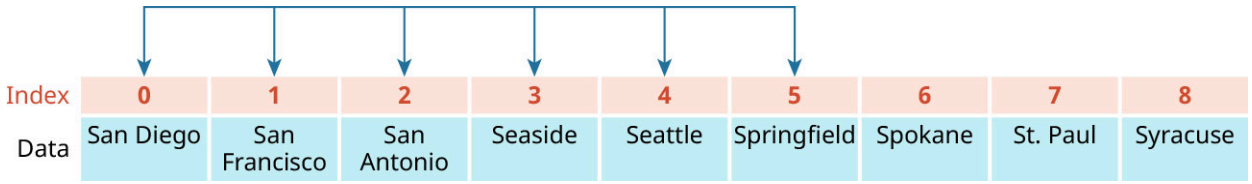


Figure 3.22 Sequential search is a search algorithm that checks the collection element by element to find a target element. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Binary Search Algorithm

A **binary search algorithm** recursively narrows down the possible locations for the target in the sorted list. Initially, the algorithm has no information—the target can be anywhere in the entire range of the sorted list. By comparing the target to the middle element of the range, we can rule-out half of the elements in the range. This process can be repeated until we have found the expected location of the target in the sorted list. The runtime of binary search is in O(log *N*) with respect to *N*, the number of elements in the sorted list, so long as indexing is a constant-time operation (see Figure 3.11).

Although the sequential search algorithm works with any collection type such as lists, sets, dictionaries, and priority queues, the binary search algorithm relies on a sorted list with access to elements by index. Consequently, binary search is efficient on array lists that provide constant-time access to the element at any index and inefficient on linked lists, which do not enable constant-time access to elements by index. Binary search relies on the structure of the sorted list to repeatedly rule-out half of the remaining elements.

Binary search trees represent the concept of binary search in a tree data structure by arranging elements in the tree in sorted order from left to right. Ideally, the root node represents the middle element in the sorted tree and each child roughly divides each subtree in half. But, in the worst case, a binary search tree can look exactly like a linked list where each node contains either zero children or one child. Although such a tree still arranges its elements in sorted order from left to right, comparing the target to each node only reduces the size of the problem by one element rather than ruling out half of the remaining elements—the worst-case binary search tree degrades to sequential search. Balanced binary search trees, such as AVL trees, addressed this worst-case scenario by maintaining the AVL property of balance between left and right subtrees.

Sorting

Sorting is the problem of rearranging elements into a logical order, typically from least-valued (smallest) to greatest-valued (largest). Sorting is a fundamental problem not only because of the tasks that it directly solves, but also because it is a foundation for many other algorithms such as the binary search algorithm or Kruskal's algorithm for the minimum spanning tree problem.

The most common type of sorting algorithm solves the problem of **comparison sorting**, or sorting a list of elements where elements are not assigned numeric values but rather defined in relation to other elements. For simplicity, the data are typically assumed to be stored in an array list for indexed access, and the sorting algorithm can either return a new sorted list or rearrange the elements in the array list so that they appear in sorted order.

Merge Sort Algorithm

A merge sort algorithm recursively divides the data into sublists until sublists are one element long—which we know are sorted—and then merges adjacent sorted sublists to eventually return the sorted list. The merge operation combines two sorted sublists to produce a new, larger sorted sublist containing all the elements in sorted order. The actual rearranging of elements occurs by repeatedly applying the merge operation on pairs of adjacent sorted sublists, starting with the smallest single-element sublists, to larger two-element sublists, and eventually reaching the two halves of the entire list of elements. The runtime of merge sort is in $O(N \log N)$ with respect to N, the number of elements (see Figure 3.17).

Quicksort Algorithm

A **quicksort algorithm** recursively sorts data by applying the binary search tree algorithm design pattern to partition data around pivot elements. Whereas the merge sort algorithm rearranges elements by repeatedly merging sorted sublists after each recursive subproblem, the quicksort algorithm rearranges elements by partitioning data before each recursive subproblem. The partition operation takes a pivot and rearranges the elements into three sections, from left to right: the sublist of all elements less than the pivot, the pivot element, and the sublist of all elements greater than (or equal to) the pivot. Each of the two sublists resulting from the partition operation is a recursive quicksort subproblem; when both of the sublists are sorted, the entire list will be sorted. The runtime of quicksort depends on the choice of each pivot element during the execution of the recursive algorithm, but in practice, for most of the inputs, the runtime is in $O(N \log N)$ with respect to N, the number of elements (Figure 3.23).

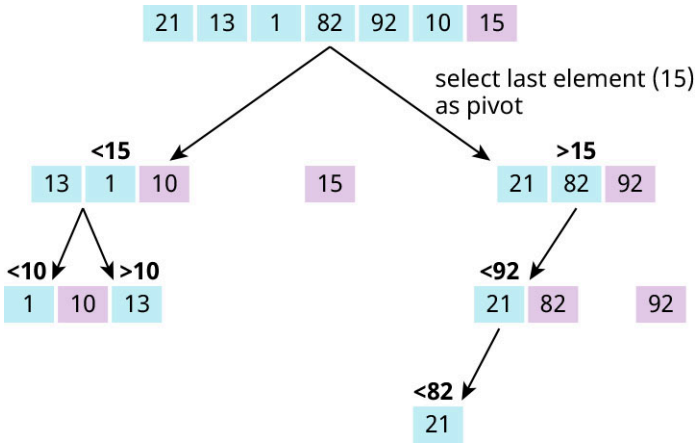


Figure 3.23 Quicksort is a divide and conquer sorting algorithm that sorts elements by recursively partitioning elements around a pivot. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Heapsort Algorithm

A **heapsort algorithm** adds all elements to a binary heap priority queue data structure using the comparison operation to determine priority value, and returns the sorted list by repeatedly removing from the priority queue element by element. The runtime of heapsort is in $O(N \log N)$ with respect to N, the number of elements. The logarithmic time factor is due to the time it takes to add or remove each element from the binary heap (Figure 3.24).

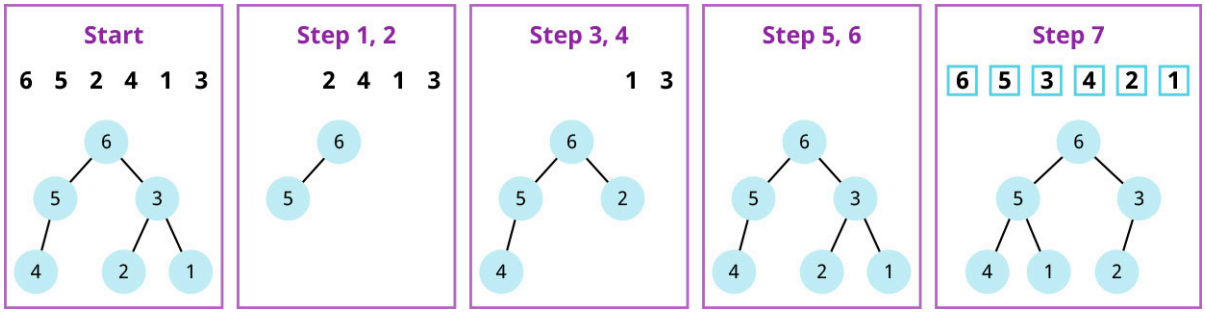


Figure 3.24 Heapsort uses the binary heap data structure to sort elements by adding and then removing all elements from the heap. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Many comparison sorting algorithms share the same O(N log N) runtime bound with respect to N, the number of elements. Computer scientists have shown with a combinatorial proof that, in the worst case, a comparison sorting algorithm cannot do better than O(N log N) comparisons. It is impossible to design a worst-case O(N) runtime comparison sorting algorithm. In fact, a commonly used version of heapsort (which is also asymptotically faster) is to first build a binary heap (i.e., first arrange the input numbers in an array, then repeatedly call a function to turn the original array into a binary heap; one can show that the running time of this part is linear in N, which is why this is faster than constructing the heap by adding numbers one by one), then remove elements one by one from the end of the heap.

But not all sorting problems are comparison sorting problems. In fact, a commonly used version of heapsort (which is also asymptotically faster) is to first build a binary heap (i.e., first arrange the input numbers in an array, then repeatedly call a function to turn the original array into a binary heap. One can show that the running time of this part is linear in N, which is why this is faster than constructing the heap by adding numbers one by one), then remove elements one by one from the end of the heap as explained previously.

Another type of sorting problem that is not restricted to pairwise comparisons is known as **count sorting**, or sorting a list of elements by organizing elements into categories and rearranging the categories into a logical order. For example, the problem of sorting a deck of cards can be seen as a count sorting problem if we put the cards into numeric stacks and then rearrange the stacks into a logical order. By changing the assumptions of the problem, count sorting algorithms can run in O(N) time by first assigning each element to its respective category and then unpacking each category so that elements appear in sorted order.

Hashing

Hashing is the problem of designing efficient algorithms which map each object to an integer so that most (if not all) objects will be assigned distinct integers. Although hashing algorithms are often specific to each data type, there exist some general approaches for designing hashing algorithms. The hash value of a simple data type such as an integer can just be the value of the integer itself. The hash value of a string of text can be some mathematical combination of the numeric value of each character in the string. Likewise, the hash value of a collection such as a list can be some combination of the underlying numeric data within each element in the collection.

Hashing has a variety of applications spanning computer systems, database systems, computer security, and searching algorithms. For example, hashing algorithms are often used designing secure systems for protecting stored passwords even after a security breach occurs. In the context of data structure problems, hashing offers a different approach than binary search. Instead of relying on pairwise comparisons to narrow down the expected location of an element in a sorted list in logarithmic time, hashing search algorithms can instead directly index an element by hash value in constant time. If binary search trees implement sets and maps by applying the concept of binary search, a **hash table** implements sets and maps by applying the concept of hashing (Figure 3.25). Rather than organize elements in sorted order from left to right as in a binary search tree, hash tables store and retrieve elements in an array indexed by hash value.

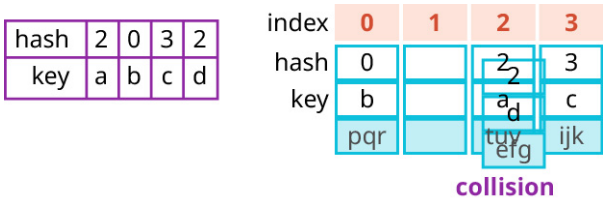


Figure 3.25 Hash tables data structures apply hashing to implement abstract data types such as sets and maps, but must handle collisions between elements that share the same hash value. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Hashing search algorithms are often preferred over binary search algorithms for their runtime benefits, but they come with unique drawbacks. Ideally, different objects would hash to different hash values. But the combinatorial explosion of possibilities for unique strings or collections of complex data means that a **collision**, or a situation in which multiple objects hash to the same integer index value, is inevitable since integers in a computer can typically only represent a certain, fixed range of integer numbers. Combinatorial explosion is not only a problem for the design of efficient algorithms, but also the design of efficient data structures too.

Graph Problems

While data structure problems focus primarily on storage and retrieval of elements in a collection, graph problems include a wide variety of computing problems involving the graph data structure. Unlike other data structures, graphs include not only elements (vertices) but also relationships between elements (edges). Graph problems often ask algorithm designers to explore the graph in order to answer questions about elements and the relationships between elements.

Traversal

Traversal is the problem of exploring all the vertices in a graph. Graph data structures differ from tree data structures in that there is no explicit root node to begin the traversal and edges can connect back to other parts of the graph. In general, there is no guarantee of hierarchy in a graph. Graph traversal algorithms such as depth-first search and breadth-first search begin at an arbitrary start vertex and explore outwards from the start vertex while keeping track of a set of explored vertices.

Depth-First Search

A **depth-first search** is a graph traversal algorithm that recursively explores each neighbor, continuing as far possible along each subproblem depth-first (Figure 3.26). Explored vertices are added to a global set to ensure that the algorithm only explores each vertex once. The runtime of depth-first search is in $O(|V| + |E|)$ with respect to $|V|$, the number of vertices, and $|E|$, the number of edges.

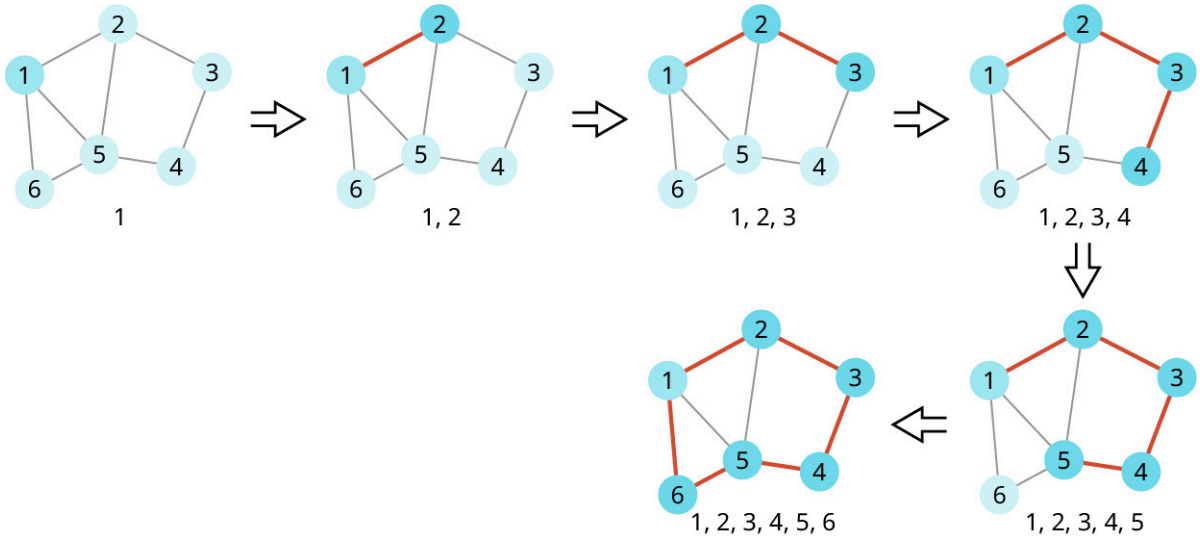


Figure 3.26 Depth-first search is a graph traversal algorithm that continues as far down a path as possible from a start vertex before backtracking. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Breadth-First Search

A **breadth-first search** iteratively explores each neighbor, expanding the search level-by-level breadth-first (Figure 3.27). Explored vertices are also added to a global set to ensure that the algorithm explores each vertex once and in the correct level-order. The runtime of breadth-first search is also in $O(|V| + |E|)$ with respect to $|V|$, the number of vertices, and $|E|$, the number of edges.

Graph traversal algorithms are the foundational algorithm design patterns for most graph processing algorithms. Many algorithms require some amount of exploration, and that exploration typically starts at some vertex and continues processing each reachable vertex at most once. A **reachable vertex** can be reached if a path or sequence of edges from the start vertex exists. As opposed to depth-first search, breadth-first search has the benefit of exploring vertices closer to the start before exploring vertices further from the start, which can be useful for solving problems such as unweighted shortest paths.

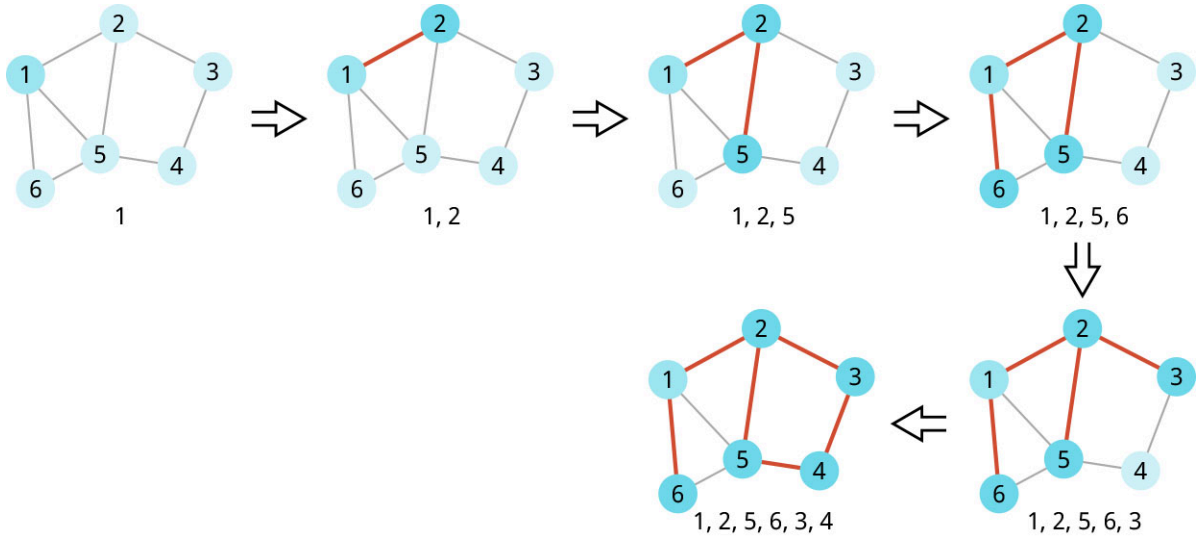


Figure 3.27 Depth-first search is a graph traversal algorithm that continues as far down a path as possible from a start vertex before backtracking. Breadth-first search is a graph traversal algorithm that explores level by level expanding outward from the start vertex. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Minimum Spanning Trees

Minimum spanning trees is the problem of finding a lowest-cost way to connect all the vertices to each other,

where *cost* is the sum of the selected edge weights. The two canonical greedy algorithms for finding a minimum spanning tree in a graph are Kruskal's algorithm and Prim's algorithm. Both algorithms repeatedly apply the rule of selecting the next lowest-weight edge to an unconnected part of the graph. The output of a minimum spanning tree algorithm is a set of $|V| - 1$ edges connecting all the vertices in the graph with the least total sum of edge weights, where $|V|$ is the number of vertices.

Kruskal's Algorithm

Kruskal's algorithm begins by considering each vertex as an independent "island," and the goal of the algorithm is to repeatedly connect islands by selecting the lowest-cost edges. A specialized data structure (called *disjoint sets*) is typically used to keep track of the independent islands. The runtime of Kruskal's algorithm is in $O(|E| \log |E|)$ with respect to $|E|$, the number of edges (Figure 3.28).

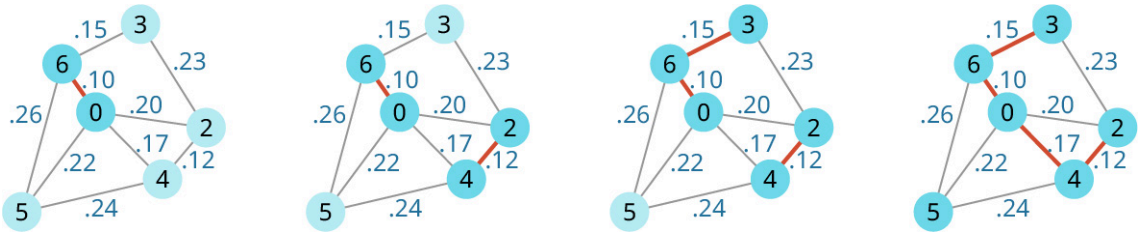


Figure 3.28 Kruskal's algorithm repeatedly selects the next lightest edge that connects two independent "islands." (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Prim's Algorithm

Prim's algorithm grows a minimum spanning tree one edge at a time by selecting the lowest-weight edge to an unexplored vertex. The runtime of Prim's algorithm is in $O(|E| \log |V| + |V| \log |V|)$ with respect to $|V|$, the number of vertices, and $|E|$, the number of edges (Figure 3.29).

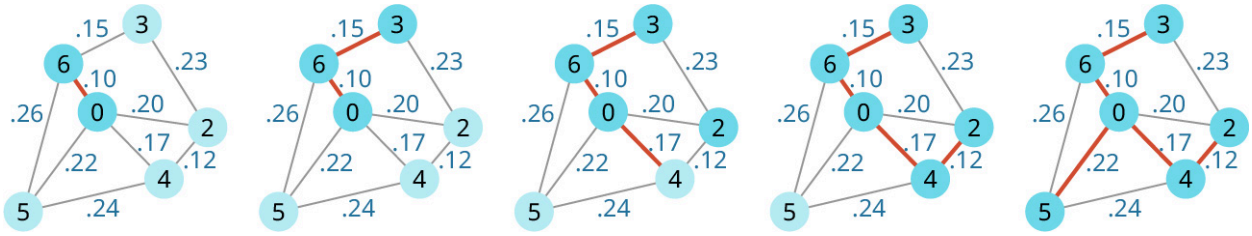


Figure 3.29 Prim's algorithm expands outward from the start vertex by repeatedly selecting the next lightest edge to an unreached vertex. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Shortest Paths

The output of a shortest paths algorithm is a **shortest paths tree**, the lowest-cost way to get from one vertex to every other vertex in a graph (Figure 3.30). The **unweighted shortest path** is the problem of finding the shortest paths in terms of the number of edges. Given a start vertex, breadth-first search can compute the unweighted shortest paths tree from the start vertex to every other reachable vertex in the graph by maintaining a map data structure of the path used to reach each vertex.

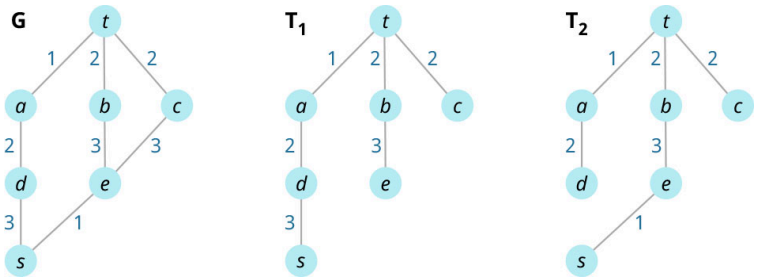


Figure 3.30 Three shortest paths trees of the lowest-cost way to get from the start vertex to every other vertex in the graph are shown. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Weighted Shortest Path

A **weighted shortest path** is the problem of finding the shortest paths in terms of the sum of the edge weights. Earlier, we introduced Prim's algorithm, a minimum spanning tree algorithm that maintains a priority queue of edges in the graph ordered by weight and repeatedly selects the next lowest-cost edge to an unconnected part of the graph.

THINK IT THROUGH

Weighted Shortest Paths for Navigation Directions

One of the most direct applications of the shortest paths problem is to provide recommended routes for navigation directions in real-world mapping. Many applications use distance as the metric for edge weight, so the shortest path between two points represents the real-world route with the smallest distance between the two places.

What does a distance metric not consider when providing a recommended route? What values are centered and emphasized by even using a shortest paths algorithm for recommending routes?

Dijkstra's Algorithm

Dijkstra's algorithm maintains a priority queue of vertices in the graph ordered by distance from the start and repeatedly selects the next shortest path to an unconnected part of the graph. Dijkstra's algorithm is almost identical to Prim's algorithm except processing shortest paths (sequences of edges) rather than individual edges. Dijkstra's algorithm grows a shortest paths tree one shortest path at a time by selecting the next shortest path to an unexplored vertex. The runtime of Dijkstra's algorithm is in $O(|E| \log |V| + |V| \log |V|)$ with respect to $|V|$, the number of vertices, and $|E|$, the number of edges (Figure 3.31).

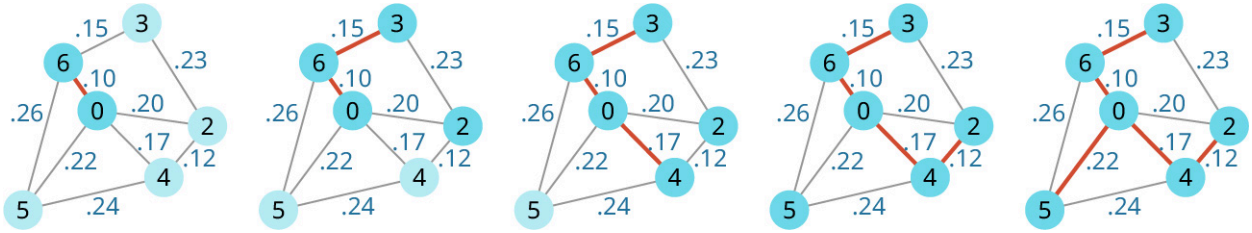


Figure 3.31 Dijkstra's algorithm expands outward from the start vertex by repeatedly selecting the next lowest-cost path to an unreached vertex. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

3.6 Computer Science Theory

Learning Objectives

By the end of this section, you will be able to:

- Understand the models and limits of computing
- Relate Turing machines to algorithms
- Describe complexity classes
- Interpret NP-completeness
- Differentiate between P and NP

Throughout this chapter, we have introduced several techniques and canonical case studies for the design and analysis of algorithms—oftentimes focusing on the ideas and details behind individual algorithms. But the study of algorithms is more than just the study of individual algorithms, algorithm design, or even algorithm analysis.

Models of Computation

Computers include basic algorithms for solving problems like adding, subtracting, or comparing two numbers. Computers, owing to their roots in calculators, are optimized to solve these problems; these basic algorithms are constant-time operations. What programming offers is the ability to define our own algorithms that can be used to solve more complex problems, such as searching, sorting, and hashing. Unfortunately, these programmed algorithms are not as fast as basic operations. We have even seen certain problems deal with a combinatorial explosion in the number of potential solutions. For many of these problems, the best-known algorithms do not do much better than brute-force, which takes exponential time.

In the bigger picture, computer science engages the central question of how humans can encode intelligence. Our discussion of algorithm design grounded the activity in problem modeling, the process of encoding a complex real-world phenomenon or problem in a more abstract or simple form. How is problem modeling constrained by the model of computation, or the rules of the underlying computer that executes an algorithm? Why are certain problems challenging for computers to execute?

Combinatorial explosion poses a problem for computer algorithms because our model of computation assumes computers only have a single thread of execution and only execute one basic operation on each step. If we overturn some part of this assumption, either by creating computers with multiple processors or by creating more sophisticated operations, then it might be possible to deal with combinatorial explosion. Almost all of today's computer hardware, ranging from massive supercomputers to handheld smartphones, rely at least to some degree on expanding the model of computation to compute solutions to problems more efficiently. Even so, much of today's computer hardware still relies on the same fundamental programming assumptions: that there are variables to represent data and arithmetic or logical operations.

Turing Machines

In the 1800s, Charles Babbage imagined a mechanical machine—the Analytical Engine—that could automatically calculate mathematical formulas. Ada Lovelace then extrapolated that the Analytical Engine could solve more general algorithms by using loops to repeat processes and variables to represent data. Lovelace's vision of algorithms represented a synthesis between human intuition and mathematical reasoning. In the mid-1900s, Lovelace's ideas inspired Alan Turing to imagine a more general notion of algorithms and machines that could run those algorithms. A **Turing machine** is an abstract model of computation for executing any computer algorithm. A Turing machine describes computers in terms of three key ideas:

1. a memory bank for storing data.
2. an instruction table, where each instruction can either:
    a. store a value to the memory bank.
    b. retrieve a value from the memory bank.
    c. perform a basic operation on a value.
    d. set which instruction will be executed next by modifying the program counter.
3. a program counter that keeps track of the current instruction in the instruction table.

A Turing machine executes a computer algorithm by following each instruction specified by the program counter. An algorithm can use these basic operations to compute the sum of 1 and 1.

1. Store the value 1 to address A in the memory bank.
2. Store the value 1 to address B in the memory bank.
3. Add the values at addresses A and B and then store the result at address A.

What makes computers useful is not just the fact that they can calculate numbers, but that they can encode logic in the instructions. Instead of just computing the sum of 1 and 1, this program continues adding 1 to a growing sum stored at address A.

1. Store the value 1 to address A in the memory bank.

2. Store the value 1 to address B in the memory bank.
3. Add the values at addresses A and B and then store the result at address A.
4. Set the program counter to execute step 3 next.

The Turing machine abstract model of computation assumes a single thread of execution following each instruction in an algorithm. Although today's computers are much more efficient than the first computers that realized the Turing machine, most computers still rely on the same fundamental assumptions about how to execute algorithms. The O(N)-time sequential search algorithm, though it might execute 1,000 times faster on today's computers, still grows linearly with respect to the size of the input. An O(2^N)-time brute-force algorithm, though it might execute 1,000 times faster on today's computers, still grows exponentially with respect to the size of the input. Even as computers become faster over time, inefficient algorithms still cannot be used to solve any problems larger than a few thousand elements.

Complexity Classes

One subfield of computer science is theoretical computer science, which studies models of computation, their application to algorithms, and the complexity of problems. The complexity of a problem is the complexity (in terms of time or memory resources required) of the most efficient algorithms for solving the problem. Theoretical computer scientists are interested in understanding the difficulty of a problem in terms of time complexity, space complexity, and some other complexity measures.

In this chapter, we have focused on solving problems known to have polynomial time algorithms. Searching, sorting, hashing, traversal, minimum spanning trees, or shortest paths are all examples of problems in the **polynomial (P) time complexity class** because they are all problems that have runtimes with a polynomial expression such as O(1), O(log N), O(N), O(N log N), O(N^2), O(N^3). In general, these problems are considered **tractable** because computers can solve them in a reasonable amount of time. But there are many problems that are considered **intractable** because they do not have efficient, polynomial-time algorithms.

The **nondeterministic polynomial (NP) time complexity class** refers to all problems that can be solved in polynomial time by a nondeterministic algorithm. A **nondeterministic algorithm** is a special kind of Turing machine, which at each step can nondeterministically choose which instruction to execute, and is considered to successfully find a solution if any combination of these nondeterministic choices eventually lead to a correct solution. In other words, in contrast to a deterministic algorithm, such as a greedy algorithm, which must repeatedly apply a simple rule to deterministically select the next element in a solution, a nondeterministic algorithm is able to simultaneously explore all the possible choices. We do not yet have computers that can execute nondeterministic algorithms, but if we did, then we would be able to efficiently solve any combinatorial problem by relying on the special power of nondeterminism.

Technically, all P problems are also NP problems because we already have deterministic algorithms for solving them and therefore do not need to rely on the special power of nondeterminism. For example, Dijkstra's algorithm provides a deterministic polynomial-time solution to the shortest paths problem by building up a shortest paths tree from the start vertex outward. This application of the greedy algorithmic paradigm relies on the structure of the shortest paths tree, since the shortest path to a point further away from the start must build on the shortest path to a point closer to the start.

NP-complete Problems

NP-complete refers to all the hardest NP problems—the combinatorial problems for which we do not have deterministic polynomial-time algorithms. More precisely, a problem PI is said to be NP-complete if PI is in NP and for every problem in NP, there is a reduction that reduces the problem to PI. For example, a **longest path**, or the problem of finding the highest-cost way to get from one vertex to another without repeating vertices, is an NP-complete problem opposite to shortest paths (Figure 3.32). What makes longest paths so much harder to solve than shortest paths? For one, there is no underlying structure to the solution that we can use to repeatedly apply a simple rule as in a greedy algorithm. With the shortest paths problem, we could rely on the

shortest paths tree to inform the solution. But in longest paths, the goal is to *wander* around the graph. The longest path between any two vertices will probably involve traversing as many edges as possible to maximize distance, visiting many vertices along the way. For some graphs, the longest paths might even visit all the vertices in the graph. In this situation, the longest paths do not form a tree and instead involve ordering all the vertices in the graph for each longest path. Identifying the correct longest path then requires listing out all the possible paths in the graph—a combinatorial explosion in the combinations of edges and vertices that can be selected to form a solution.

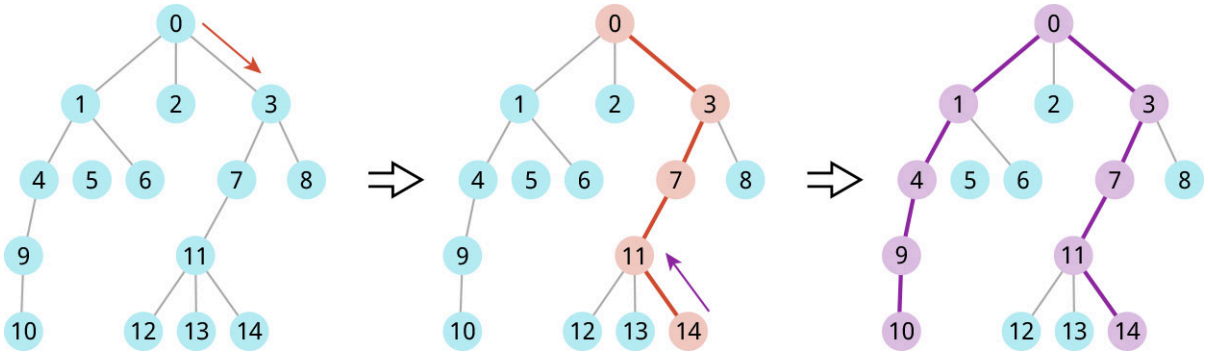


Figure 3.32 The longest path in a graph maximizes the distance, which often (but not always) involves visiting many vertices along the way. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Related to the problem of longest paths is the **traveling salesperson problem (TSP)**, which is the problem of, given the path cost of every pair of vertices, finding the lowest-cost tour, or path from a start vertex visiting every vertex in the graph once including a return to the start vertex. Compared to the TSP, which is finding the lowest-cost tour, the longest paths problem is like finding the highest-cost tour. What makes both these problems difficult is that we do not have a simple rule for selecting the next element to include in the solution. Unless we have the special power of nondeterminism, it is hard to tell from the beginning which edge is the right one to include in the final solution. Applying a simple rule like, "Select the edge with the lowest cost," might not necessarily lead to the overall lowest-cost tour. Even though this simple rule worked for the problem of minimum spanning trees, the additional restriction of a tour rather than a tree makes the TSP a much harder problem to solve efficiently. Although we have efficient algorithms for shortest paths, we do not have efficient algorithms for the shortest tour (TSP).

INDUSTRY SPOTLIGHT

Delivery Logistics

Companies such as Amazon, FedEx, UPS, and others that rely on logistics to deliver goods to various locations seek to optimize the sequence of stops to save costs. The only way to achieve this would be to rely on an optimal algorithm for the traveling salesperson problem. The TSP aims to find the minimum distance tour that visits all the vertices such that no vertex is visited more than once. But this is not a perfect match for real-world delivery logistics. Fuel or battery efficiency, for example, is not just about distance traveled, but also the speed of travel, the time spent idling, and even the way that the route is organized. In the United States, where vehicles drive on the right side of the road, safety can be improved by reducing the number of left-hand turns across the divider. Drivers might also want to take breaks during a trip. Modeling all these factors requires carefully formulating the problem and considering the limits of TSP.

How do we know if a problem is NP-complete? Earlier, we introduced reduction as an algorithm paradigm that is not only useful for solving problems, but also for relating the difficulty of problems. A reduction from a difficult problem A to another problem B proves that B is as difficult as A. It turns out that all NP-complete problems can be reduced to all the others, so an algorithm for solving *any* NP-complete problem solves *every*

NP-complete problem.

LINK TO LEARNING

The longest paths problem and the traveling salesperson problem are just two examples of NP-complete problems. Visit A Graph of NP-Complete Reductions (https://openstax.org/r/76NPCompReuct) to visualize many NP-complete problems and their relationships to each other. For example, the longest paths problem (LPT) and the traveling salesperson problem (TS) both reduce to Hamiltonian paths (HP). In turn, Hamiltonian paths (HP) reduce to vertex cover (VC) which reduces to 3-satisfiability (3-SAT).

P versus NP

Longest paths and TSP are just two among thousands of NP-complete problems for which we do not have efficient algorithms. The question of P versus NP asks whether it is possible to design a deterministic polynomial-time algorithm for solving any—and therefore all—of NP-complete problems. There are two possible answers to the question of P versus NP:

- If P = NP, then there is a deterministic polynomial-time algorithm for solving all NP-complete problems.
- If P $\neq$ NP, then there are NP-complete problems that cannot be solved with a deterministic polynomial-time algorithm.

Most theoretical computer scientists believe that P $\neq$ NP; in other words, it is *impossible* to design a deterministic polynomial-time algorithm for longest paths, TSP, or any other NP-complete problem. However, they do not have any definite proof that P = NP or P $\neq$ NP. An efficient algorithm for any one NP-complete problem would not only directly solve routing and logistics problems but would also enable massive advancements in drug discovery through scientific simulation, for instance. It would also break essentially all modern Internet security and password systems—among thousands of other problems.

Chapter Review

Key Terms

abstract data type (ADT) consists of all data structures that share common functionality
abstraction process of simplifying a concept in order to represent it in a computer
adjacent in a graph abstract data type, the relationship between two vertices connected by an edge
algorithm analysis study of the results produced by the outputs as well as how the algorithm produces those outputs
algorithm design pattern solution to well-known computing problems
algorithmic paradigm common concept and ideas behind algorithm design patterns
algorithmic problem solving refers to a particular set of approaches and methods for designing algorithms that draws on computing's historical connections to the study of mathematical problem solving
array list data structure that stores elements next to each other in memory
asymptotic analysis evaluates the time that an algorithm takes to produce a result as the size of the input increases
asymptotic notation mathematical notation that formally defines the order of growth
AVL tree balanced binary search tree data structure often used to implement sets or maps that organizes elements according to the AVL tree property
AVL tree property requires the left and right subtrees to be balanced at every node in the tree
balanced binary search tree introduces additional properties that ensure that the tree will never enter a worst-case situation by reorganizing elements to maintain balance
Big O notation most common type of asymptotic notation in computer science used to measure worst case complexity
binary heap binary tree data structure used to implement priority queues that organizes elements according to the heap property
binary logarithm tells how many times a large number needs to be divided by 2 until it reaches 1
binary search algorithm recursively narrows down the possible locations for the target in the sorted list
binary search tree tree data structure often used to implement sets and maps that organizes elements according to the binary tree property and the search tree property
binary tree property requires that each node can have either zero, one, or two children
breadth-first search iteratively explores each neighbor, expanding the search level-by-level breadth-first
brute-force algorithm solves combinatorial problems by systematically enumerating all potential solutions in order to identify the best candidate solution
canonical algorithm well-known algorithm
case analysis way to account for variation in runtime based on factors other than the size of the problem
child node descendant of another node
collision situation where multiple objects hash to the same integer index value
combinatorial explosion exponential number of solutions to a combinatorial problem that makes brute-force algorithms unusable in practice
combinatorial problem involves identifying the best candidate solution out of a space of many potential solutions
comparison sorting sorting of a list of elements where elements are not assigned numeric values but rather defined in relation to other elements
complexity condition based on the degree of computational resources that an algorithm consumes during its execution in relation to the size of the input
compression problem of representing information using less data storage
constant type of order of growth that does not take more resources as the size of the problem increases
correctness whether the outputs produced by an algorithm match the expected or desired results across the range of possible inputs

cost model characterization of runtime in terms of more abstract operations such as the number of repetitions

count sorting sorting a list of elements by organizing elements into categories and rearranging the categories into a logical order

cryptography problem of masking or obfuscating text to make it unintelligible

data structure complex data type with specific representation and specific functionality

data structure problem computational problem involving the storage and retrieval of elements for implementing abstract data types such as lists, sets, maps, and priority queues

data type determines how computers process data by defining the possible values for data and the possible functionality or operations on that data

depth-first search graph traversal algorithm that recursively explores each neighbor, continuing as far possible along each subproblem depth-first

Dijkstra's algorithm maintains a priority queue of vertices in the graph ordered by distance from the start and repeatedly selects the next shortest path to an unconnected part of the graph

divide and conquer algorithm algorithmic paradigm that breaks down a problem into smaller subproblems (divide), recursively solves each subproblem (conquer), and then combines the result of each subproblem in order to inform the overall solution

edge relationship between vertices or nodes

element individual data point

experimental analysis evaluates an algorithm's runtime by recording how long it takes to run a program implementation of it

functionality operations such as adding, retrieving, and removing elements

graph represents binary relations among collection of entities, specifically vertices and edges

graph problem computational problem involving graphs that represent relationships between data

greedy algorithm solves combinatorial problems by repeatedly applying a simple rule to select the next element to include in the solution

hash table implements sets and maps by applying the concept of hashing

hashing problem of assigning a meaningful integer index for each object

heap property requires that the priority value of each node in the heap is greater than or equal to the priority values of its children

heapsort algorithm adds all elements to a binary heap priority queue data structure using the comparison operation to determine priority value and returns the sorted list by repeatedly removing from the priority queue element-by-element

index position or address for an element in a list

interval scheduling problem combinatorial problem involving a list of scheduled tasks with the goal of finding the largest non-overlapping set of tasks that can be completed

intractable problems that do not have efficient, polynomial-time algorithms

Kruskal's algorithm greedy algorithm that sorts the list of edges in the graph by weight

leaf node node at the bottom of a tree that has no children

linear type of order of growth where the resources required to run the algorithm increases at about the same rate as the size of the problem increases

linear data structure category of data structures where elements are ordered in a line

linked list data structure that does not necessarily store elements next to each other and instead works by maintaining, for each element, a link to the next element in the list

list ordered sequence of elements and allows adding, retrieving, and removing elements from any position in the list

logarithm tells how many times a large number needs to be divided by a small number until it reaches 1

longest path problem of finding the highest-cost way to get from one vertex to another without repeating vertices

map represents unordered associations between key-value pairs of elements, where each key can only

appear once in the map

matching problem of searching for a text pattern within a document

merge sort canonical divide and conquer algorithm for comparison sorting

minimum spanning tree problem of finding a lowest-cost way to connect all the vertices to each other

model of computation rules of the underlying computer that is ultimately responsible for executing the algorithm

node represents an element in a tree or graph

nondeterministic algorithm special kind of Turing machine, which at each step can non-deterministically choose which instruction to execute and is considered to successfully find a solution if any combination of these nondeterministic choices leads to a correct solution

nondeterministic polynomial (NP) time complexity class all problems that can be solved in polynomial time by a nondeterministic algorithm

NP-complete all the hardest NP problems—the combinatorial problems for which we *do not* have deterministic polynomial-time algorithms

order of growth geometric prediction of an algorithm's time or space complexity as a function of the size of the problem

perfectly balanced for every node in the binary search tree, its left and right subtrees contain the same number of elements

polynomial (P) time complexity class all problems that have runtimes described with a polynomial expression such as $O(1)$, $O(\log N)$, $O(N)$, $O(N \log N)$, $O(N^2)$, $O(N^3)$

Prim's algorithm greedy algorithm that maintains a priority queue of vertices in the graph ordered by connecting edge weight

priority queue represents a collection of elements where each element has an associated priority value

problem task with specific input data and output data corresponding to each input

problem model simplified, abstract representation of a more complex real-world problem

program realization or implementation of an algorithm written in a formal programming language

quicksort algorithm recursively sorts data by applying the binary search tree algorithm design pattern to partition data around pivot elements

reachable vertex vertex that can be reached if a path or sequence of edges from the start vertex exists

reduction algorithm solves problems by transforming them into other problems

representation particular way of organizing a collection of elements

root node node at the top of the tree

runtime analysis study of how much time it takes to run an algorithm

search tree property requires that elements in the tree are organized least-to-greatest from left-to-right

searching problem of retrieving a target element from a collection of elements

sequential search algorithm searching algorithm that sequentially checks the collection element-by-element for the target

set represents an unordered collection of unique elements and allows adding, retrieving, and removing elements from the set

shortest path problem of finding a lowest-cost way to get from one vertex to another

shortest paths tree output of the shortest paths problem, the lowest-cost way to get from one vertex to every other reachable vertex in a graph

sorting problem of rearranging elements into a logical order

space complexity formal measure of how much memory an algorithm requires during its execution as it relates to the size of the problem

step basic operation in the computer, such as looking up a single value, adding two values, or comparing two values

string problem computational problem involving text or information represented as a sequence of characters

subproblem smaller instance of a problem that can be solved independently

time complexity formal measure of how much time an algorithm requires during execution as it relates to the size of the problem

tractable problems that computers can solve in a reasonable amount of time

traveling salesperson problem (TSP) problem of, given the path cost of every pair of vertices, finding the lowest-cost tour, or path from a start vertex visiting every vertex in the graph once including a return to the start vertex

traversal problem of exploring all the vertices in a graph

tree hierarchical data structure

Turing machine abstract model of computation for executing any computer algorithm

unweighted shortest path problem of finding the shortest paths in terms of the number of edges

vertex represents an element in a graph or special type of it such as a tree

weighted shortest path problem of finding the shortest paths in terms of the sum of the edge weights

Summary

3.1 Introduction to Data Structures and Algorithms

- Data structures represent complex data types for solving real-world problems. Data structures combine specific data representations with specific functionality.
- Abstract data types categorize data structures according to their functionality and ignore differences in data representation. Abstract data types include lists, sets, maps, priority queues, and graphs.
- To select an appropriate data structure, first select an abstract data type according to the problem requirements. Then, select an appropriate data structure implementation for the abstract data type.
- Linear data structures organize elements in a line, ideal for implementing the list abstract data type. Linear data structures include array lists and linked lists.
- Linear data structures can implement any abstract data type. The study of data structures in general focuses on opportunities to improve efficiency (in terms of execution time or memory usage) over linear data structures.
- Tree data structures organize elements in a hierarchy of levels defined by parent-child relationships. Trees are defined with a root node at the top of the tree, parent-child relationships between each level, and leaf nodes at the bottom of the tree.
- Binary search trees require that elements in the tree are organized least-to-greatest from left-to-right. Binary search trees are often used to implement the set and map abstract data types.
- Balanced binary search trees and binary heaps represent two approaches for avoiding the worst-case situation with binary search trees. Binary heaps are often used to implement the priority queue abstract data type.
- Graph data structures focus on explicitly modeling the relationships between elements. Graphs afford access not only to elements, but also to the relationships between elements.

3.2 Algorithm Design and Discovery

- Just like how many data structures can represent the same abstract data type, many algorithms exist to solve the same problem. In algorithmic problem-solving, computer scientists solve formal problems with specific input data and output data that correspond to each input.
- Modeling is the process of representing a complex phenomenon such as a real-world problem as a formal problem. Modeling is about abstraction: the simplification or erasure of details so that the problem can be solved by a computer.
- Historically, the model of computation emphasized specialized algorithms operating on a modest model of the underlying phenomenon. Modeling is a violent but also necessary act in order to simplify the problem so that it can be solved by a computer.
- Searching is the problem of retrieving a target element from a collection of many elements. Sequential search and binary search are two algorithms for solving the search problem.
- To solve real-world problems, computer scientists compose, modify, and apply algorithm design patterns,

such as search algorithms.

- Algorithm analysis is the study of the outputs produced by an algorithm as well as how the algorithm produces those outputs.
- Correctness considers whether the outputs produced by an algorithm match the expected or desired results across the range of possible inputs. Correctness is defined as a match between the algorithm and the model of the problem, not between the algorithm and the real-world.
- Correctness is complicated by the complexity of social relationships, power, and inequity in the real-world. Since algorithms automate processes and operate in existing power structures, they are likely to reproduce and amplify social injustice.
- In addition to correctness, computer scientists are also interested in complexity, or measuring the computational resources that an algorithm consumes during its execution in relation to the size of the input.

3.3 Formal Properties of Algorithms

- Runtime analysis is a study of how much time it takes to run an algorithm. Experimental analysis is a runtime analysis technique that involves evaluating an algorithm's runtime by recording how long it takes to run a program implementation of it.
- Time complexity is the formal measure of how much time an algorithm requires during execution as it relates to the size of the problem. The goal of time complexity analysis is to produce a simple and easy-to-compare characterization of the runtime of an algorithm as it relates to the size of the problem.
- Space complexity is the formal measure of how much memory an algorithm requires during execution as it relates to the size of the problem.
- Steps in time complexity analysis are to identify a metric for representing the size of the problem; to model the number of steps needed to execute the algorithm; and to formalize the model using either precise English or asymptotic notation to define the order of growth. Big O notation is the most common type of asymptotic notation in computer science.
- Differences in orders of growth are massive: as the input size grows, the difference between orders of growth becomes more and more vast. For problems dealing with just 1,000 elements, the time it would take to run an exponential-time algorithm on that problem exceeds the current age of the universe—whereas that same-size problem running on the same computer would take just 1 second on a quadratic-time algorithm.
- In practice, across applications working with large amounts of data, $O(N^2)$ is often considered the limit for real-world algorithms. For algorithms that need to run frequently on large amounts of data, algorithm designers target $O(N)$, $O(\log N)$, or $O(1)$.

3.4 Algorithmic Paradigms

- Algorithmic paradigms are the common concepts and ideas behind algorithm design patterns, such as divide and conquer algorithms, brute-force algorithms, greedy algorithms, and reduction algorithms.
- Divide and conquer algorithms break down a problem into smaller subproblems (divide), recursively solve each subproblem (conquer), and then combine the result of each subproblem to inform the overall solution. Recursion is an algorithm idea fundamental to divide and conquer algorithms that solves complex problems by dividing input data into smaller, independent instances of the same problem known as subproblems.
- Binary search is an example of divide and conquer algorithm with a single recursive subproblem. Merge sort is an example of a divide and conquer algorithm with two recursive subproblems.
- Brute-force algorithms solve combinatorial problems by systematically enumerating all potential solutions in order to identify the best candidate solution. Combinatorial problems identify the best candidate solution out of a space of many potential solutions.
- Brute-force algorithms exist for every combinatorial problem, but they are not typically used in practice because of long run time issues. To enumerate all potential solutions, a brute-force algorithm must generate every possible combination of the input data.

- Greedy algorithms solve combinatorial problems by repeatedly applying a simple rule to select the next element to include in the solution. Unlike brute-force algorithms that solve combinatorial problems by generating all potential solutions, greedy algorithms instead focus on generating just one solution.
- Greedy algorithms are not always guaranteed to compute the best solution depending on the assumptions and goals of the problem. A greedy algorithm for the interval scheduling problem will not compute the correct result if we choose to complete the shortest tasks.
- Kruskal's algorithm and Prim's algorithm are two examples of greedy algorithms for the minimum spanning trees problem. These algorithms are a rare example of a greedy algorithm that is guaranteed to compute the correct result.
- Reduction algorithms solve problems by transforming them into other problems. In other words, reduction algorithms delegate most of the work of solving the problem to another algorithm meant for a different problem.
- Reduction algorithms allow algorithm designers to rely on optimized canonical algorithms rather than designing a solution by composing algorithm design patterns, which can lead to performance or correctness bugs. Reduction algorithms also enable computer scientists to make claims about the relative difficulty of a problem.

3.5 Sample Algorithms by Problem

- Data structure problems focus on the storage and retrieval of elements for implementing abstract data types such as lists, sets, maps, and priority queues. Data structure problems include sorting, searching, and hashing.
- Searching is the problem of retrieving a target element from a collection of elements. Searching in a linear data structure such as an array list can be done using either sequential search or binary search.
- Sorting is the problem of rearranging elements into a logical order, typically from least-valued (smallest) to greatest-valued (largest). Sorting is a fundamental problem not only because of the tasks that it directly solves, but also because it is a foundation for many other algorithms such as the binary search algorithm or Kruskal's algorithm for the minimum spanning tree problem.
- Merge sort and quicksort are two examples of divide and conquer algorithms for sorting. Heapsort is a sorting algorithm that relies on adding to a heap and then repeatedly removing each element in sorted order.
- Hashing is the problem of assigning a meaningful integer index (hash value) for each object. Hash tables are a data structure for implementing sets and maps by applying the concept of hashing.
- Graph problems include a wide variety of problems involving the graph data type. Graph problems include traversal, minimum spanning trees, and shortest paths.
- Traversal is the problem of exploring all the vertices in a graph. Depth-first search and breadth-first search are both graph traversal algorithms that expand outward from a start vertex, ultimately visiting every reachable vertex.
- Minimum spanning trees is the problem of finding a lowest-cost way to connect all the vertices to each other, where cost is the sum of the selected edge weights. The two canonical greedy algorithms for finding a minimum spanning tree in a graph are Kruskal's algorithm and Prim's algorithm.
- Shortest paths is the problem of finding a lowest-cost way to get from one vertex to another. The output of a shortest paths algorithm is a shortest paths tree from the start vertex to every other vertex in the graph.
- Breadth-first search computes the unweighted shortest paths tree, the shortest paths in terms of the number of edges. Dijkstra's algorithm computes the weighted shortest paths tree, the shortest paths in terms of the sum of the edge weights.

3.6 Computer Science Theory

- Problem modeling is constrained by the model of computation, or the rules of the underlying computer that is ultimately responsible for executing the algorithm. Combinatorial explosion poses a problem for computer algorithms because our model of computation assumes computers only have a single thread of execution and only execute one basic operation on each step.

- A Turing machine is an abstract model of computation for executing any computer algorithm. A Turing machine describes computers in terms of three key ideas: a memory bank, an instruction table, and a program counter.
- Although today's computers are much more efficient than the first computers that realized the Turing machine, most computers still rely on the same fundamental assumptions about how to execute algorithms. Even as computers become faster over time inefficient algorithms still cannot be used to solve any problems larger than a few thousand elements.
- The complexity of a problem is the complexity (i.e., the time or memory resources required) of the most efficient algorithms for solving the problem. In this chapter, we have focused on solving problems known to have polynomial time algorithms that can be described with a polynomial expression such as O(1), O(log N), O(N), O(N log N), O(N2), O(N3).
- Nondeterministic polynomial (NP) time complexity class refers to all problems that can be solved in polynomial time by a nondeterministic algorithm. A nondeterministic algorithm is a kind of algorithm that can rely on the special power of exploring infinitely many possible "alternate universes" in order to complete a computation.
- Technically, all P problems are also NP problems because we already have deterministic algorithms for solving them and therefore do not need to rely on the special power of nondeterminism. NP-complete refers to all the hardest NP problems—the combinatorial problems for which we do not have deterministic polynomial-time algorithms.
- Longest paths and the traveling salesperson problem (TSP) are two well-known examples of NP-complete problems. What makes both these problems difficult is that we do not have a simple rule for selecting the next element to include in the solution.
- All NP-complete problems can be reduced to all the others, so an algorithm for solving any NP-complete problem solves every NP-complete problem. The question of P versus NP asks whether it is possible to design a deterministic polynomial-time algorithm for solving any—and therefore all—of these NP-complete problems.
- Most theoretical computer scientists believe that it is impossible to design an efficient algorithm for longest paths, TSP, or any other NP-complete problems. An efficient algorithm for any one NP-complete problems would not only directly solve routing and logistics problems but would also enable massive advancements in drug discovery through scientific simulation, for instance. It would also break essentially all modern Internet security and password systems—among thousands of other problems.

Review Questions

1. Why did we introduce tree data structures as an alternative to linear data structures?
   a. Some complex data can only be represented with a tree data structure.
   b. Some simple data can only be represented with a tree data structure.
   c. Linear data structures cannot implement sets and maps.
   d. Tree data structures are typically more effective at implementing sets and maps.

2. How does the graph abstract data type differ from other abstract data types?
   a. It can model the relationships between elements.
   b. It is more efficient than other abstract data types.
   c. It can solve problems that other abstract data types cannot solve.
   d. It does not specify a particular data structure implementation.

3. What abstract data type do binary heaps most commonly implement?
   a. lists
   b. sets
   c. maps
   d. priority queues

4. What is one way to describe the relationship between algorithms, problems, and modeling?
- **a.** Algorithms are the foundation for problem models.
- **b.** Algorithms solve a model of a problem.
- **c.** Each algorithm can only be used to solve a single problem.
- **d.** Each problem can only have a single model.

5. At what point do computer scientists apply algorithm design patterns?
- **a.** when learning canonical algorithms
- **b.** when modeling a problem
- **c.** when solving new problems
- **d.** when analyzing an algorithm

6. How does the model of computation relate to the problem model?
- **a.** The model of computation is synonymous with problem model.
- **b.** Problem models constrain the model of computation.
- **c.** The model of computation constrains the problem modeling process.
- **d.** The model of computation describes a single algorithm for each problem model.

7. Why is case analysis important?
- **a.** Case analysis provides an alternative to asymptotic analysis.
- **b.** Case analysis focuses on small inputs.
- **c.** Case analysis simplifies the step-counting by introducing a cost model.
- **d.** Case analysis considers factors other than the size of the problem.

8. What is true about the order of growth of binary search with respect to the size of the sorted list?
- **a.** In the best case, the order of growth of binary search is constant.
- **b.** In the best case, the order of growth of binary search is logarithmic.
- **c.** In the worst case, the order of growth of binary search is constant.
- **d.** In the worst case, the order of growth of binary search is linear.

9. How does time complexity relate to space complexity?
- **a.** Time complexity measures efficiency according to the size of the problem, while space complexity does not.
- **b.** Both time and space complexity measure the efficiency of algorithms as they relate to the nature of the problem.
- **c.** Time complexity focuses on asymptotic analysis while space complexity focuses on experimental analysis.
- **d.** Both time and space complexity can apply methods from asymptotic analysis and experimental analysis.

10. What are the three steps in divide and conquer algorithms?

11. Why do many greedy algorithms fail to compute the best solution to a combinatorial problem?

12. What are the three steps in reduction algorithms?

13. What is quicksort's algorithm design pattern and algorithmic paradigm?
- **a.** Quicksort is an application of the binary search tree algorithm design pattern and an example of the divide and conquer algorithmic paradigm.
- **b.** Quicksort is an application of the binary search tree algorithm design pattern and an example of the brute-force algorithmic paradigm.
- **c.** Quicksort is an application of the binary search tree algorithm design pattern and an example of the greedy algorithmic paradigm.
- **d.** Quicksort is an application of the binary search tree algorithm design pattern and an example of the

randomized incremental construction algorithmic paradigm.

14. What graph problems can breadth-first search solve?
- **a.** exponential node tree
- **b.** minimum spanning trees
- **c.** unweighted shortest paths
- **d.** weighted shortest paths

15. What is a primary drawback of hashing?
- **a.** There can be collisions as the same element can hash to multiple values.
- **b.** There can be collisions between multiple elements that hash to the same value.
- **c.** Hashing is slower than binary search for search problems.
- **d.** Hashing is faster than binary search for search problems.

16. What is the relationship between Turing machines and models of computation?

17. What is one of the three key ideas of the Turing machine?
- **a.** infinite memory by virtualization
- **b.** a memory bank for storing data
- **c.** using divide and conquer to always reduce an algorithm to O(n) runtime
- **d.** using divide and conquer to always reduce an algorithm to O(1) runtime

18. What is P versus NP?
- **a.** P refers to the polynomial time complexity class, whereas NP refers to the nondeterministic polynomial time complexity class.
- **b.** P refers to any Big O notation past O(N^3), whereas NP refers to any Big O notation less than O(N^3).
- **c.** NP is a constant runtime, whereas P is polynomial runtime.
- **d.** P refers to constant runtime, whereas NP is linear runtime.

Conceptual Questions

1. Explain the difference between algorithms and programs.

2. Explain the difference between data structures and abstract data types.

3. Explain the relationship between data representation, data structures, and algorithms.

4. What is the relationship between search algorithms and the searching problem?

5. What is the relationship between search algorithms and the autocomplete feature?

6. Why is algorithmic correctness difficult to determine?

7. What are some limitations of experimental analysis?

8. What are some benefits of experimental analysis over asymptotic analysis?

9. Why is a 1-element list the best-case situation for sequential search?

10. If phone numbers are ten digits long and can contain digits from zero through nine, what is the total number of potential phone numbers?

11. Why might we prefer a sub-optimal greedy algorithm over a correct brute-force algorithm?

12. What's problematic about the statement, "municipal broadband planning reduces to Kruskal's algorithm"?

13. Describe the relationship between the pivot element and the left and right sublists after the first partition in quicksort.

14. The runtime of Kruskal's algorithm is in O($|E|$ log $|E|$) with respect to $|E|$, the number of edges. What

primarily contributes to this linearithmic order of growth?

15. Both Prim's algorithm and Dijkstra's algorithm are greedy algorithms that organize vertices in a priority queue data structure. What is the difference between the ordering of vertices in the priority queue for Prim's algorithm and Dijkstra's algorithm?

16. What is the relationship between models of computation and algorithms?

17. What is the common difficulty preventing us from designing an efficient algorithm for solving NP-complete problems?

18. What are the consequences of P = NP?

Practice Exercises

1. Binary search trees organize elements in ascending sorted order within the tree. However, binary search trees can become unbalanced. In the worst-case, a binary search tree can look exactly like a linked list. Give an order for inserting the following numbers into a binary search tree such that the resulting tree appears like a linked list: 7, 3, 8, 1, 2, 5, 6, 4.

2. Consider these two different approaches for implementing the priority queue abstract data type using a linked list data structure: (1) organize the elements by decreasing priority value, and (2) organize the elements arbitrarily. Describe algorithms for inserting an element as well as retrieving and removing the highest-priority element from these two data structures.

3. In our definition of a priority queue, we emphasized retrieval and removal of the highest-priority elements—a maximum priority queue. What if we wanted to instead prioritize retrieval and removal of the lowest-priority elements—a minimum priority queue? Describe a simple change that we could make to make any maximum priority queue function as a minimum priority queue.

4. Formally describe the problem model for a drug administration medical system in terms of input data and output data represented as lists, sets, maps, priority queues, and graphs.

5. Formally describe the problem model for a music recommendation system in terms of input data and output data represented as lists, sets, maps, priority queues, and graphs.

6. There can sometimes be thousands, if not millions, of results that match a Web search query. To make this information more helpful to humans, we might want to order the results according to a relevance score such that more-relevant results appear before less-relevant results. Describe how we can solve this problem of retrieving the *N*-largest elements using the following algorithm design patterns: (1) a sorting algorithm, and (2) a priority queue abstract data type.

7. What is the best-case and worst-case Big O order of growth of sequential search with respect to *N*, the size of the list?

8. What is the best-case and worst-case Big O order of growth of binary search with respect to *N*, the size of the sorted list?

9. What is the worst-case Big O order of growth of sequential search followed by binary search with respect to *N*, the size of the sorted list?

10. What two sublists are combined in the final step of merge sort on the list [9, 8, 2, 5, 4, 1, 3, 6]?

11. If a connected graph has unique edge weights, will Kruskal's algorithm find the same minimum spanning tree as Prim's algorithm? How about a connected graph with duplicate edge weights?

12. What is a reduction algorithm for the problem of finding the median element in a list?

13. The heapsort algorithm applies a binary heap priority queue ordered by the comparison operation to sort

elements. What can we say about the first element removed from the binary heap if it implements a minimum priority queue? What about the last element removed? Is the binary heap data structure sorted?

14. Hashing algorithms can provide a constant-time solution to the search problem under certain conditions. What are the conditions necessary to ensure the runtime of a hashing search algorithm is constant?

15. Why is it the case that depth-first search cannot be directly applied to compute an unweighted shortest paths tree?

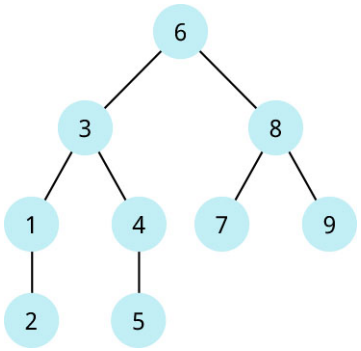
Problem Set A

1. Linked lists and binary search trees are two examples of linked data structures, in which each node in the data structure is connected to other nodes. Given a linked list of the numbers one through seven, organized in ascending sorted order, draw two different examples of binary search trees representing the same numbers.



(attribution: Copyright Rice University, OpenStax, under CC BY 4.0 license)

2. Given the following binary search tree, draw the corresponding ascending-sorted linked list containing the same numbers.



(attribution: Copyright Rice University, OpenStax, under CC BY 4.0 license)

3. We defined autocomplete as a problem that takes as an input a string of letters that might represent the start of a word, and outputs a list of words that match the input. Describe two other problem models for autocomplete and define the ramifications of the models.

4. Compare and contrast the autocomplete problem models. What are the trade-offs of each problem model?

5. Consider the problem of arranging desktop icons in a grid layout so that they fill column-by-column, starting from the left side of the screen. Suppose we are given a list of desktop icons in alphabetical order and that placing an icon in a grid position is a constant-time operation. What is the Big O order of growth of an algorithm that takes each icon in the given order and places each icon in the next open grid position with respect to *N*, the number of desktop icons?

6. Suppose we're given a list of desktop icons in alphabetical order, but that placing an icon in a grid position is a linear-time operation with respect to the number of icons. (Perhaps a sequential search is needed to check that the icon has not already been placed on the desktop.) What is the Big O order of growth of this icon arrangement algorithm with respect to *N*, the number of desktop icons?

7. Why does the greedy interval scheduling, which selects the cheapest, least time-consuming task, fail to maximize the number of completed tasks?

8. What is a simple rule for greedy interval scheduling that will maximize the number of completed tasks?

9. The runtime of most binary heap priority queue operations is in O(log N) with respect to N, the size of the priority queue. The logarithmic factor is due to the height of the binary heap data structure. But finding an element in a binary heap typically requires sequential search. How can we apply the hashing algorithm design pattern to find an element in a heap in constant time?

10. The runtime of breadth-first search is in O($|V| + |E|$) because each reachable vertex and edge is processed one-by-one during the level-order graph traversal. Why is the runtime of Prim's algorithm in O($|E| \log |V| + |V| \log |V|$)? Explain in terms of the time it takes to process each vertex or edge in the graph.

Problem Set B

1. Each unique key in a map is paired with one (possibly not-unique) value. Sometimes, we want to associate more than one value with a given key. Describe how we can use a list or set abstract data type to associate more than one value with a unique key.

2. Each vertex in a graph can be labeled with a unique identifier, such as a unique number. Describe the relationship between adjacent vertices. How could we use abstract data types such as lists, sets, and maps to represent these relationships?

3. Describe how we might implement the graph abstract data type using other abstract data types such as lists, sets, and/or maps. Explain for graphs whose edges have associated weights as well as graphs whose edges do not have associated weights.

4. Describe two or three different problem models for a medical system designed to help doctors recommend preventive care for patients.

5. Compare and contrast the medical system problem models. What are the trade-offs of each problem model?

6. How does the choice of problem model affect potential algorithms? Describe an algorithm for each problem model.

7. Consider an autocomplete implementation that relies on a sequential search to find all matching terms. What is the worst-case Big O order of growth for computing a single autocomplete query with respect to N, the number of potential terms?

8. Consider an autocomplete implementation that sorts the list of potential terms and then performs binary search to find the matching terms. If the order of growth of the sorting algorithm is in O($N \log N$), what is the worst-case Big O order of growth for computing a single autocomplete query with respect to N, the number of potential terms?

9. Why might algorithm designers prefer to use binary search instead of sequential search for autocomplete?

10. In a 1-D space where each point is defined with x coordinates, a common problem is to find the closest pair of points in the space: the pair of points that has the least distance among all potential pairs of points. What is a brute-force algorithm for solving this 1-D closest pair problem? What is the Big O notation order of growth of this algorithm?

11. In a 2-D space, each point is defined with (x, y) coordinates. What is a brute-force algorithm for solving the 2-D closest pair problem?

12. What are the recursive subproblems in a divide and conquer algorithm for solving for the closest pair problem?

13. Digital images are represented in computers as a 2-D grid of colored pixels. In image editing, the flood fill problem takes a given starting pixel and replaces all the pixels in a contiguous region that share the same color with a different color. How can we represent the colored pixels in a digital image as a graph with

vertices and edges for the flood fill problem?

14. How should we modify a graph traversal algorithm to solve the flood fill problem using your graph representation?

15. What is a reduction algorithm for reducing from the flood fill problem to the graph traversal problem? In this case, the graph traversal algorithm cannot be modified. Instead, define a preprocessing step to create a graph representation that encodes the flood fill same-color rule.

Thought Provokers

1. Maps can be defined in terms of sets: every map is a set whose elements represent key-value pairs, where the key must be unique, but the value might not be unique. Consider other relationships between abstract data types. Can sets and maps be defined in terms of graphs? Can lists be defined in terms of maps? Can priority queues be defined in terms of maps? Why might it be useful to define abstract data types in terms of other data types?

2. Graph theory refers to the mathematical study of graphs. How might a graph theorist describe linked lists and tree data structures? How does this differ from our use of abstract data types?

3. Sorting and searching are two examples of data structure problems related to the storage and retrieval of elements. Where do sorting and searching appear in linear data structures, tree data structures, and/or graph data structures?

4. What are some benefits and drawbacks of simpler problem models, as they compare to more complicated problem models?

5. The formal definition of Big O notation does not exactly match our working definition for orders of growth. Do some additional research to explain why binary search is also in O(N).

6. Since binary search is in O(N), it is also true that binary search is in O(N^2). Explain why computer scientists might find O(N^2) to be a less useful description of the runtime of binary search compared to O(log N).

7. We can show that the worst-case order of growth for any comparison sorting algorithm must be at least linearithmic using an argument from combinatorial explosion in the number of unique permutations of elements in a list. What are the number of unique permutations of a list with N elements? How many comparison questions need to be asked to identify a particular permutation from among all the permutations? How do these questions relate to comparison sorting?

8. Breadth-first search is a fundamental algorithm design pattern for graph problems. How is breadth-first search applied as a foundation for designing greedy algorithms such as Prim's algorithm and Dijkstra's algorithm? How does Kruskal's algorithm fit into these algorithm design patterns and paradigms? If Prim's algorithm is analogous to sorting in Kruskal's algorithm, why is there no analogue to the Dijkstra's algorithm in sorting as well?

9. Suppose we want to find the longest path from a starting vertex to an ending vertex in a graph. How might a nondeterministic algorithm solve this problem in polynomial time?

10. Suppose we want to find the longest path from a starting vertex to an ending vertex in a graph (solving the *function problem*) without using a nondeterministic algorithm. Let's say that P = NP and we have a deterministic polynomial-time algorithm that returns whether there is a path with exactly cost k (solving the *decision problem*). We also know the cost of the actual longest path. How can we repeatedly apply this *decision algorithm* to design a polynomial-time longest paths *function algorithm*?

Labs

1. Simulate patients entering and exiting a hospital emergency room with a priority queue using patients'

time of arrival, basic assessment of severity, and availability of doctors specializing in the appropriate type of care. Decide how to prioritize patients based on a property of each patient, such as their arrival time, numeric severity rating, numeric urgency rating, and availability of care providers. Then, consider how your decision might result in unfair allocation of medical care to patients.

2. Choose a lab from the Ethical Reflections Modules (https://openstax.org/r/76Ethics) for CS1. Follow the instructions to complete the assignment. Once you have finished, answer this additional reflection question connecting back to algorithm design: How did you utilize algorithm design patterns? How did your choice of algorithm designs affect the end outcomes in your program?

3. Experimental analysis: Use a software-based "stopwatch" to compare the time (in microseconds) it takes to run a sequential search versus a binary search for successively larger and larger inputs. Relate experimental analysis to asymptotic analysis. What happens to small arrays? What happens to large-size array inputs? What happens when the target is near the front of the array? What happens when the target is near the end of the array?



4

Linguistic Realization of Algorithms: Low-Level Programming Languages

Figure 4.1 Low-level programming languages support little or no abstraction; they allow programmers to write software in languages that are closer to English and are suitable for system software that powers mobile devices with limited energy and computing resources such as prosthetics. (credit: modification of "Tilly Lockey at the SingularityU The Netherlands Summit 2016" by Sebastiaan ter Burg/Flicker, CC BY 2.0)

Chapter Outline

4.1 Models of Computation
4.2 Building C Programs
4.3 Parallel Programming Models
4.4 Applications of Programming Models

Introduction

The machines we call "computers," including modern desktop computers, laptops, and web servers, are remarkably fast and capacious. However, computer hardware is also embedded in devices that do not fit the traditional definition of computers: home appliances, automobiles, smart thermostats, tools, and televisions. Along with being energy-efficient and affordable, these devices may also need to be lightweight, portable, or even wearable. For these reasons, embedded systems have meager processing speed and memory capacity. This chapter focuses on low-level programming languages, which are used in practice to create software for resource-constrained devices. Efficiency is critical to making these devices useful and economically viable. The efficiency of embedded software is make-or-break; in other words, if we can write efficient code that runs fast and uses little memory, we enable technologies that can help people with their daily lives. Therefore, computer scientists place considerable emphasis on the efficiency and speed of low-level languages, which is why low-level languages are important to society. For efficiency reasons, the syntax of low-level programming languages relies on instructions that are computer-centric and challenging for humans to work with. This has led computer scientists to create "middle-level" languages that emphasize human-readability without compromising efficiency.

Consider our fictional company, TechWorks, which is bringing a line of next-generation prosthetics to the market. While legacy prosthetics are sometimes awkward to use and limited in function, "smart" prosthetics can be revolutionary. These prosthetics are computer-controlled, Internet-connected, and make use of artificial

intelligence, allow those who need prosthetics to enjoy a quality of life that meets or exceeds expectations.

Computer control refers to the ability to manage, organize, or run something on a computer, whereas intelligent control is a class of control techniques that use various artificial intelligence computing approaches. For example, artificial intelligence algorithms can accurately determine the intentions of the wearer and control a prosthetic's motion in an accurate and natural way. Internet connectivity means devices can be conveniently controlled by applications, relay telemetry to healthcare providers, and automatically apply over-the-air updates. TechWorks has fitted a small, inexpensive, energy-efficient system-on-chip computer to their devices, and are using the middle-level language C to implement these features efficiently.

4.1 Models of Computation

Learning Objectives

By the end of this section, you will be able to:

- Define low-level programming languages, including assembly language
- Define middle-level and high-level programming languages, such as C and JavaScript
- Compare and contrast the various programming paradigms

Algorithms are used to solve computational problems and create computational models. A **computational model** is a system that defines what an algorithm does and how to run it. Examples of such computational models include physical devices that can run software, programming languages, or a design specification of such. A programming language is a linguistic application of an algorithm, which uses computational models focused on defining notations for writing code.

Many computational models have been devised for a host of other applications. There are many different roles and perspectives within the worlds of computer science and software development. The end goal of software development is to create working software that can run on a **hardware model**, which itself uses a (hardware) realization of an algorithm that enables specific physical computers to execute software programs. A hardware model is designed for the convenience of a machine, not a human software author, so hardware models are poorly suited to writing code. Computer scientists have created programming languages which are designed specifically for programmers to develop practical applications. These languages are usually classified into high-level (Java, Python) and low-level languages (assembly language). A **high-level programming language** operates at a high level of abstraction, meaning that low-level details such as the management of memory are automated. In contrast, a **low-level programming language** operates at a low level of abstraction. Languages like C and C++ can perform both high-level and low-level tasks.

Most software is designed, written, and discussed in terms of how a program should work. It is basically a series of steps that provide a direction of how the program must be executed. An example of this would be the "Map Reduce model" which is used in distributed systems like the Google search engine to produce search results for large data sets using a complex algorithm. Moving even further away from hardware models, computer scientists have also defined an **abstract model**, which is a technique that derives simpler high-level conceptual models for a computer while exploring the science of what new algorithms can or cannot do.

Modern computers are equipped with a central processor, referred to as a **central processing unit (CPU)**, which is a computer chip capable of executing programs. A CPU's hardware model relies on a specific CPU **instruction set architecture (ISA)** that defines a list of operations that the CPU can execute, such as storing the results of calculations (Figure 4.2). With the advancements of technology, computer engineers have designed computer architectures with increasing sophistication and power. Examples of hardware models include the MOS Technology 6502 architecture used by the Nintendo Entertainment System, the ARM architecture used by mobile phones, and the x86-64 and AMD64 architectures used by modern personal computers. Computer engineers design architectures with hardware specifications, such as execution speed or energy use, in mind. Therefore, hardware models are not suitable for humans to use for communicating

algorithms.

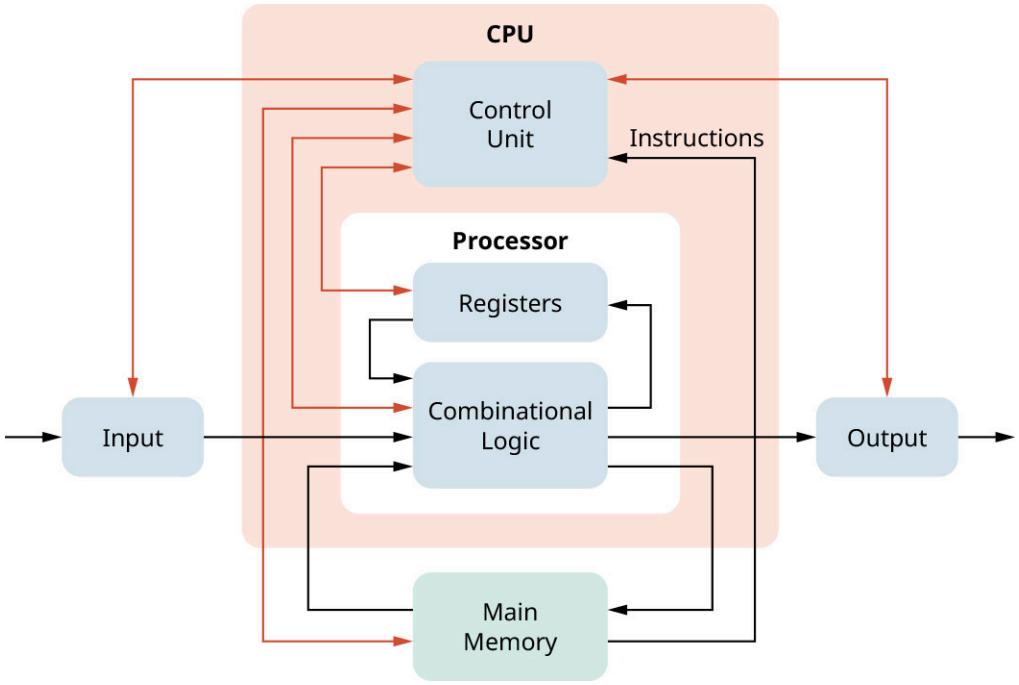


Figure 4.2 A standard CPU model shows how a program logic applies low-level instructions to an input to get an output; the program leverages registers and memory (black arrows) and the CPU orchestrates the overall execution of the program (red arrows). (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

A **programming model** is designed for humans to read and write. A programming model focused on defining a notation for writing code can also be called a programming language. A programming model can be used to implement a software algorithm using a strict set of syntactical rules. A language's syntax can define keywords such as "if." The syntax may include a mathematical **operator**, a fundamental programming operation that combines values, such as "+." The syntax can define punctuation such as ";". Essentially, the syntax gives the precise meaning for what each of these elements directs a computer to do. The text of a program written in a programming language is called **source code**. Software engineers have created practically all software by writing source code in various programming languages. Since a programming model cannot directly execute a program, a **compiler** or **interpreter** must translate source code from a middle-level or high-level language into something a computer can execute.

As mentioned, abstract models are computational models used to think about algorithms in the abstract, rather than being used to create and run software. The goal of an abstract model is to make it easy for people to devise and convey algorithms. Computer scientists use abstract models to create new algorithms, analyze the efficiency of algorithms, and prove facts about what algorithms can and cannot do. An abstract model is not concerned with the details of computer architectures, which makes it easier to focus on these sorts of deep questions. Examples of abstract models include the Random Access Machine, the Turing machine, and the Lambda calculus. The **Random Access Machine** (Figure 4.3) is a CPU that consists of unlimited memory cells that can store any arbitrary value. Just like any other CPU, the PC determines the statement to be executed next. A Random Access Machine can be used to analyze the efficiency of algorithms. A Turing machine (Figure 4.4) is a mathematical model that can implement any algorithm. The **Lambda calculus** is a theoretical computation concept using lambda functions. It was defined by Alonzo Church and inspired the functional programming paradigm, which you will learn more about in Programming Language Paradigms.

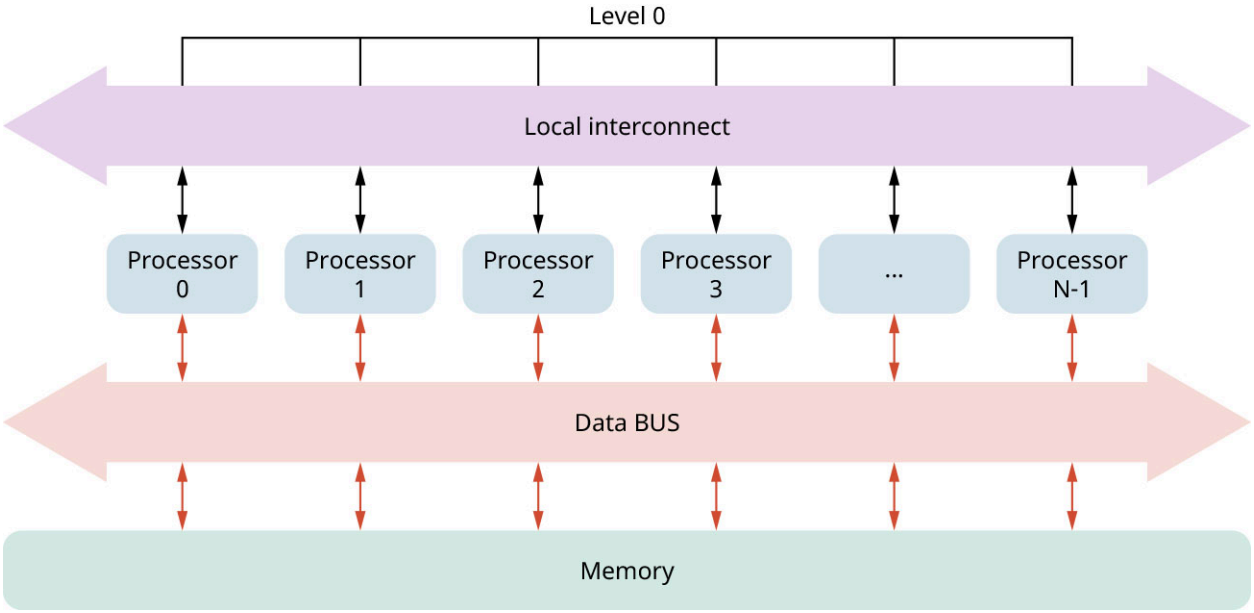


Figure 4.3 A Random Access Machine has unlimited memory cells that can store any arbitrary value. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

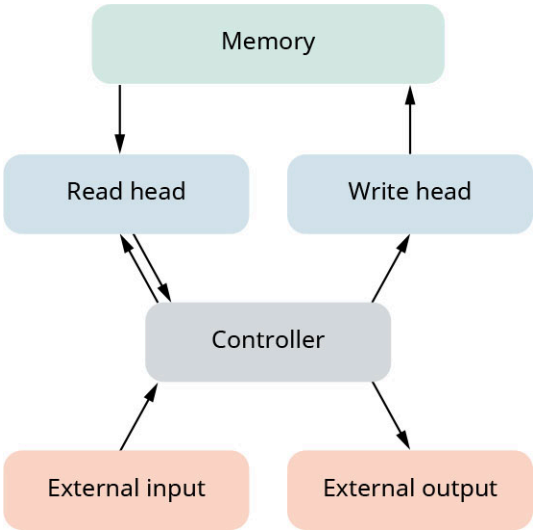


Figure 4.4 A neural Turing machine (NTM) leverages the pattern matching capabilities of neural networks in addition to more traditional computational models. It uses a controller that interacts with external memory resources through attention mechanisms that mimic human attention to improve performance. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

A function defines how to convert an input into an output, and **functional programming** is a paradigm in which algorithms are written as mathematical functions. An example of a functional programming paradigm is a recursion (refer to the following code snippet), where a function is used to call itself. The factorial of an integer can be computed using a recursive function.

```
import java.util.*;
public class Recursion {
  public static int Factorial_Recursion(int Val){
    if(Val==0){
      return 1;
    }
    else
      return Val*Factorial_Recursion(Val-1);
```

```
  }
  public static void main(String[] args){
    Scanner s =new Scanner(System.in);
    System.out.println("Enter an input value: ");
    int Val=s.nextInt();
    System.out.println("The factorial of " + Val + " is: " +
Factorial_Recursion(Val));
  }
}
```

An algorithm described in an abstract model cannot be run directly. First, a software developer must implement the algorithm, which means translating the abstract algorithm into source code in a programming language.

It may be surprising that so many different programming models can exist, and that algorithms can be translated from one model to another. However, this translation is by design; computer science established the **Church-Turing Thesis**, which is a scientific theory stating that an algorithm can be converted from any reasonable computational model to another. The Church-Turing Thesis provides a lens through which computer scientists can invent many computer architectures and programming languages, all of which can run algorithms.

Computer scientists have created terminology to make sense of the similarities and differences among all these programming languages. For example, any programming language can be low level or high level, or it can fall anywhere on the spectrum.

Low-Level Programming

A programming language's **level of abstraction** is the degree to which a computational model, programming language, or piece of software relates to computer hardware. A low-level language has a low level of abstraction, while a high-level language has a high level of abstraction. In a low-level language, the programmer must describe an algorithm in terms that the computer hardware can understand directly. Source code must describe details such as the location of data in memory and the particulars of how the computer calculates arithmetic.

Generally, low-level programs execute faster but are more labor-intensive to create and maintain. In a low-level language, the programmer is forced to think deliberately about how the computer hardware executes, so the finished program usually executes efficiently. However, that deliberate thought takes time and effort. In a high-level language, the programmer is not burdened with thinking about so many details and can finish their work faster while preventing certain types of programming errors from occurring. A compiler automates converting high-level code to low-level code, but that automated process can introduce some inefficiency. In some settings code performance is more important, and in other settings programmer productivity is more important, which is why we have both kinds of languages.

We can think of low-level programming languages in terms of cooking: when you cook a meal from scratch, you control every ingredient and every detail of preparation, so the finished meal has precisely the taste and nutrition that you desire. An alternative is to prepare a meal that uses some prepared ingredients, and when you do that, you lose a lot of control over details, but the process is significantly faster and easier.

There are many examples of low-level programming languages, but the most fundamental language understood by computers is made up of a sequence of digits.

Machine Code

The sequence of binary digits (bits) that can be understood and executed directly by a computer is called **machine code** (Figure 4.5). Machine code is the most low-level language. It is also known as **binary code**. It is

a program in the native format that can be understood by a CPU, in the form of a long series of 0s and 1s. Machine code, or binary code, is the only computational model that a computer can execute; a program written in any other language must be compiled or interpreted into machine code before the program can run. The CPU of a computer is a computer chip capable of executing machine code programs (Figure 4.6). It is impractical for humans to work with machine code directly because a machine code program is not designed to be human-readable. The patterns of 0s and 1s are designed to be convenient for a CPU to decode, not for humans to manipulate; and such programs are long, typically millions or billions of bits long. Another obstacle is that machine code is hardware dependent. As discussed in 5.3 Machine-Level Information Representation, every processor architecture has its own machine language, so machine language written for one architecture (for example, INTEL X86) cannot work on any other architecture (such as ARM). When the very first digital computers were built, and programming languages had yet to be invented, programmers had no choice but to write machine code by hand. However, this is extremely time-consuming and prone to errors, so is almost never done today.

Machine Code

```
11010001100101110110 0
11011001101111000011 0
11000001111000011110 1
01010100001111110010 1
01000000111011101111 1
10000100111000110111 1
10010011000111000000 0
```

Figure 4.5 Machine code, with its 0s and 1s, is the only computational model a computer can execute. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

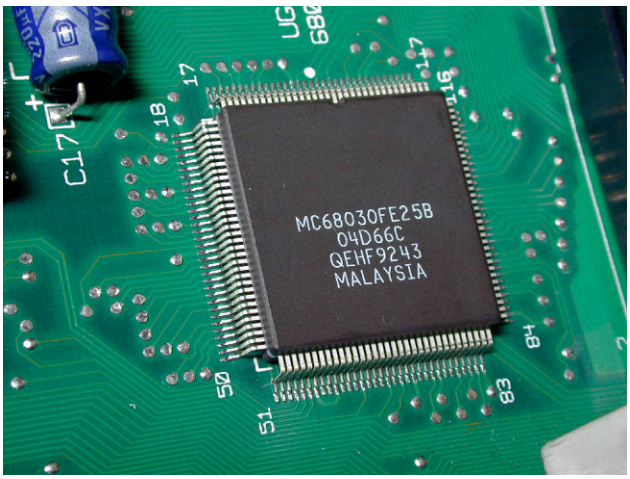


Figure 4.6 A computer chip is a computer's central processing unit (CPU). (credit: modification of "IIci" by raneko/Flickr, CC BY 2.0)

Assembly Language

The low-level language in which every statement corresponds directly to a machine instruction is called **assembly language**. Assembly language is a small step above machine code but is still a very low-level language. Assembly language is a textual representation of machine code. Just like a machine code program, an assembly language program is a series of instructions that a CPU will execute. However, rather than writing the instructions in a binary format of 0s and 1s, each instruction has a textual name such as "ADD" or "MOVE." An **assembler** is a program that translates assembly language source code into machine code. As shown in Figure 4.7, an assembler translates each textual instruction into the corresponding list of 0 and 1 bits.

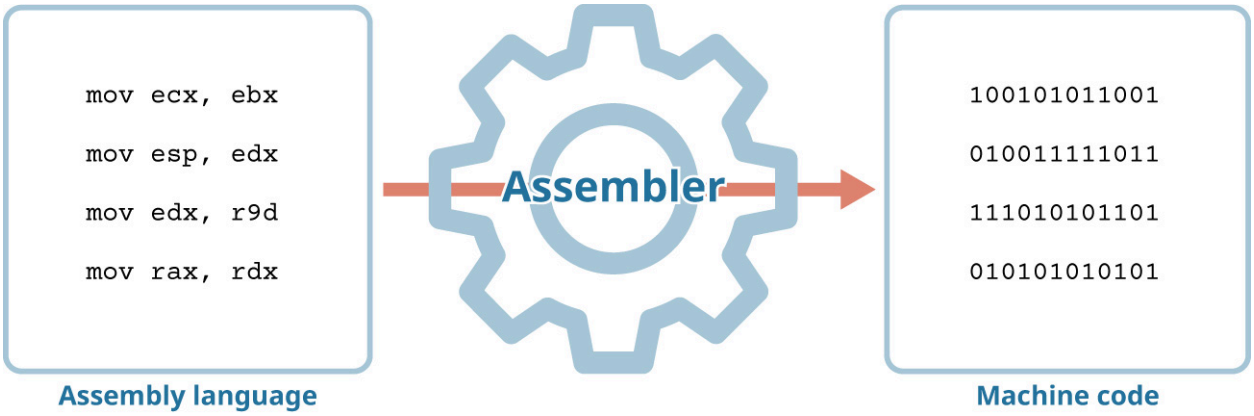


Figure 4.7 An assembler is a program that translates assembly language source code into machine object code. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

While it is practically impossible for a human to write a complete program in machine code, writing programs in assembly language is viable. Because assembly language is extremely low-level, these programs tend to run quickly, but they are labor-intensive to write, and are machine-dependent. This type of programming was sufficient in the 1960s, 1970s, and 1980s when software was written for one-off capital-intensive machines, such as multimillion-dollar mainframes or space vehicles. Programmer labor was comparatively cheap then, and there was no need to move programs to different hardware. But today, we expect applications to be compatible with multiple kinds of platforms, including phone, computer, and gaming systems. Programmer labor is more expensive than computer hardware, so writing entire programs in assembly language is uneconomical. Consequently, programs are often written predominantly in a high-level language, with assembly language used to write short excerpts on an as-needed basis. Writing code in a higher level language makes it easier to write correct code that does not have defects.

LINK TO LEARNING

Assembly language has been used in high-profile, high-budget projects, such as Apollo 11, the NASA spaceflight that first landed humans on the moon. You can examine the assembly code for the embedded computers (https://openstax.org/r/76AssemblyCode) in the space vehicles, which has been released publicly. Notice how it is quite low-level, perhaps difficult to follow, and reflects an immense amount of fastidious work.

Middle-Level and High-Level Programming

As the name implies, a **middle-level programming language** is at a level of abstraction in between low-level and high-level language, and allows for direct hardware access. The **C** programming language is a middle-level language that has been in wide use since the 1970s. The **C++** programming language is a middle-level object-oriented language based on C. In general, the trade-off between low-level and high-level languages is that writing low-level code is laborious and error-prone, but the finished code executes very quickly; high-level code is faster, easier, and safer to write but does not run quite as quickly. Middle-level code is a compromise that executes nearly as fast as low-level code yet has some of the productivity benefits of high-level code. Like low-level languages, middle-level languages allow direct access to computer hardware, making it possible to write hardware-specific programs such as operating systems and device drivers. An **operating system** is the software that provides a platform for applications and manages hardware components. A **device driver** is a piece of code responsible for connecting to a hardware component, such as a video card or keyboard. Figure 4.8 summarizes the trade-offs between low-level, middle-level, and high-level programming languages.

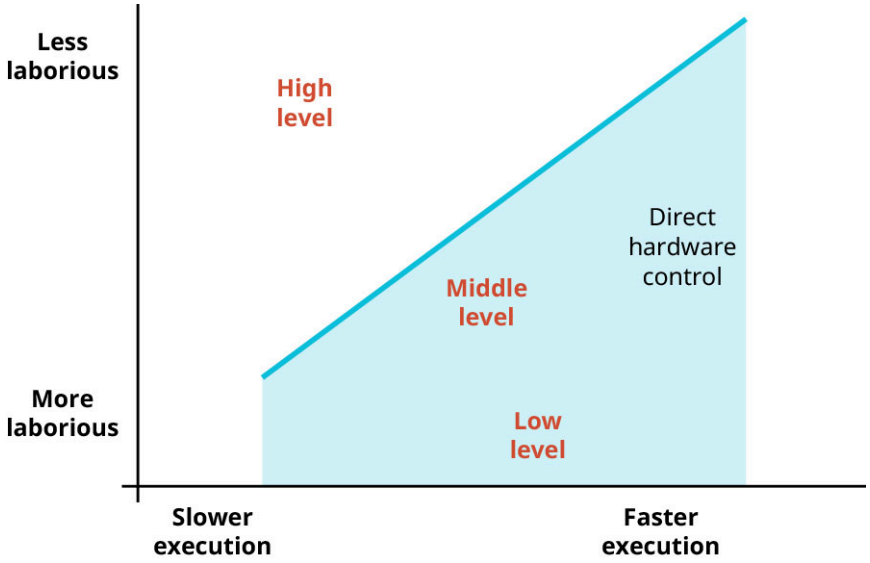


Figure 4.8 As a rule, high-level languages are less laborious to write, and slower to execute, than low-level languages. High-level languages typically do not support direct hardware control. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Middle-level languages are ideally suited to writing **systems software**, programs that provide infrastructure and platforms that other programs rely upon. The core part of an operating system that is responsible for managing and interfacing with hardware components is called the **kernel**. Kernels need direct hardware access, so high-level languages are inadequate. Practically all widespread kernels are written in C and/or C++ (such as Windows, MacOS, Linux, iOS, Android, Xbox, and PlayStation). Compilers for high-level languages, such as Python, Java, and C#, are themselves implemented in middle-level languages such as C.

Figure 4.9 summarizes the various types of programming languages and how middle-level languages overlap with low-level and high-level programming languages.

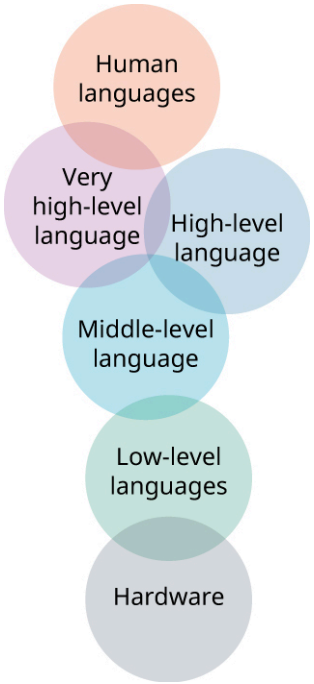


Figure 4.9 Computation models fit onto a spectrum from low to high level. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

A high-level language is farther from a hardware model, and closer to an abstract model. Source code in a

high-level language does not address low-level details, and instead focuses on how an algorithm proceeds, such as "visit every item in a list." A high-level language is like viewing Earth at a high altitude, revealing large features such as the contours of rivers and highways, whereas a low-level language is like viewing Earth at ground level, which allows for focusing on fine details such as the activity of individual people and animals.

Web application frameworks (e.g., React, Node) are written in high-level languages, principally JavaScript. A web framework is a special tool for building and managing web applications. Some common ones used in web clients are HTML5, CSS, and JavaScript. Native Android apps are primarily written in the high-level language Java, and iOS apps are primarily written in the high-level language Swift.

Programming Language Paradigms

So far, we have categorized programming languages according to their level of abstraction into low-level, middle-level, and high-level languages. A different approach is to categorize languages into paradigms. A **programming language paradigm** is a philosophy and approach for organizing code, the ideas in a program, and the layout of its source code. Real-world programs involve many thousands of lines of source code, which is too much for a human to digest without some kind of organizational structure. Computer scientists have developed several different paradigms for creating this structure.

Unlike level of abstraction, paradigms do not fall on a spectrum. Instead, a particular programming language either adheres to the philosophy of a paradigm, or it does not. For example, C is a structured procedural language and not an object-oriented language. Without getting into too many details, C is procedural because it allows programmers to place code in functions that can be called from various places in a program. However, C is not object-oriented because C does not allow, like Java does, the creation of objects that are instances of classes. We will broadly explore these different types of paradigms later in this section, but the chapter on Chapter 7 High-Level Programming Languages elaborates on these and other paradigms in more detail.

The Imperative Programming Paradigm

In **imperative programming**, the programmer writes a series of steps that must be followed in order. Source code spells out a precise series of operations that the computer must execute in order. Since the computer is told to take specific actions and execute these statements, the language is referred to as "imperative." An imperative is an order or command. Low-level languages are imperative languages, and middle-level languages, such as C, are imperative and include another paradigm. While low-level languages can mimic the style of a structured language, these properties are not inherent in the language itself and must be imposed by the programmer as a practice. Assembly code can easily be written in a non-structured way.

Declarative and Functional Programming

Another type of programming, **declarative programming**, is a paradigm in which code dictates a desired outcome without specifying how that outcome is achieved. Declarative languages are an alternative to imperative languages. In a declarative language, the programmer declares the desired outcome, and it is the compiler's job to create a series of imperative steps that obtains that outcome. For example, the Structured Query Language (SQL) used to query database systems makes it possible to specify what data should be retrieved from a database, without specifying how the database system should retrieve that data.

Functional languages are another alternative to imperative languages. Recall that a function is a mathematical object that defines how to convert an input into an output. For example, given $x = 4$, the function $f(x) = x + 3$ converts the input 4 into the output 7. Functions can be defined in most programming languages and correspond to small sections of code that perform a specific task such as a calculation. Functions can be defined in most programming languages.

Functional programming is a programming paradigm in which algorithms are written as mathematical functions (Figure 4.10). In functional programming, practically every part of the program is written as a function. The programmer writes functions that convert inputs to outputs, and it is the compiler's job to create

imperative steps to evaluate the functions.

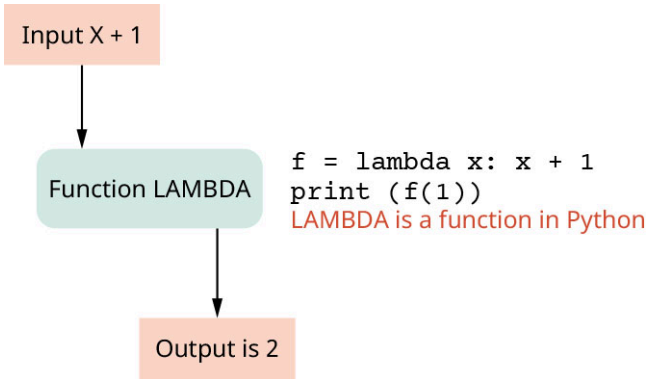


Figure 4.10 This diagram shows a functional programming example using Python. Here the function "LAMBDA" is used to increment the value of *x* by 1 whenever it is called. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Declarative and functional languages are considered high-level languages because the compiler is creating these steps on behalf of the programmer. Functional languages are discussed in more detail in Chapter 7 High-Level Programming Languages.

Structured Programming

In low-level languages and early high-level languages such as **BASIC**, some special statements called conditional statements (using "if/then") and iteration (called "loops") are programmed using an operation called **GOTO**, a non-structured operation that instructs a computer to jump to an entirely different part of the program. In large programs, these jumps from one spot to another interact in complex ways, so the flow of execution is difficult to understand when attempting to read the code. These sorts of programs are criticized for being messy "spaghetti code" (Figure 4.11).

Structured:

```
IF x<=y THEN
   BEGIN
      z := y-x;
      q := SQRT(z);
   END
ELSE
   BEGIN
      z := y-x;
      q := -SQRT(z);
   END;
WRITELN(z,q);
```

Unstructured:

```
IF x<=y THEN GOTO 2;
z := y-x;
q := SQRT(z);
GOTO 1;
2: z:= y-x;
q:=-SQRT(z);
1: writeln(z,q);
```

Figure 4.11 The same program that prints the square root of an integer (using SQRT(*z*)) is shown in a structured format and an unstructured format. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Newer languages were developed to help avoid spaghetti code. In a **structured programming** language, control flow leverages conditionals (e.g., "if-then") and iteration statements (e.g., "while" or "do while") and never uses GOTO statements. For example, C is a structured language that includes the conditional statements "if" and "switch," and the iteration statements "for," "while," and "do." In proper C, all the code sections that involve conditionals and iteration are written with these statements, and GOTO should not be used. Note that the fact that GOTO is provided as a keyword in the C language relates to the fact that C is a low-level language and programmers at that level are given the choice of using unstructured programming if necessary, although it is not recommended.

LINK TO LEARNING

Read this seminal article about how GOTO statements can be considered harmful (https://openstax.org/r/76GOTOStatemnts) written by Edgar Dijkstra.

The benefit of using these statements is that they make the flow of execution clear in the source code. When writing an "if," for instance, it is clear which code is inside the "if" and which is outside. And when mixing an "if" with a "for" loop, it is clear whether the "if" is inside the "while" or vice-versa. These sorts of inside/outside relationships are difficult to perceive in unstructured code. In a conditional statement like "if", the compiler executes a line if the condition has been met or is true. Otherwise, it moves to the next statement:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Enter an input value: ");
        int Val = s.nextInt();
        int Curr_Val = 10;

        if (Val > Curr_Val) {
            System.out.println("The Value that you entered is greater than the
current.");
        } else {
            System.out.println("The Current value is greater than the value that you
entered.");
        }

    }
}
```

Inside a loop, like "while", the statements are executed only if the condition in the loop is true. Otherwise, the loop execution terminates, and the compiler moves to the statements after the loop:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Enter an input value: ");
        int Val = s.nextInt();
        int Prod = 1;

        while (Val != 0) {
            Prod = Prod * Val;
            Val--;
        }

        System.out.println("The factorial is " + Prod);
```

```
    }
}
```

There is a substantial upside to making a language structured, and the only significant downside is that it makes the language a bit more high-level. Therefore, among programming languages that are currently in widespread use, all the middle-level and high-level languages are structured.

Procedural Programming

In a procedural language, each part of the program is a **procedure**, which is a function in the context of programming. Known as **procedural programming**, this is a paradigm in which code is organized into procedures (Figure 4.12). It is a sub-type of imperative programming. All procedural languages, then, are imperative, but not all imperative languages are procedural. A programmer designs each procedure to accomplish a specific task and gives it a descriptive name. This allows the programmer to break a large and complicated program into smaller, more manageable pieces, which are easier to write and easier for other programmers to understand. This property of code being divided into small, reusable piece is called **modularity**, and it is considered a virtue.

Procedural Programming

1. Take 2 pieces of bread.
2. Separate the pieces of bread.
3. Get a knife.
4. Get some peanut butter and jelly.
5. Take the knife and put peanut butter on it.
6. Spread the peanut butter on the bread with the knife.
7. Take the knife and put some jelly on it.
8. Spread the jelly on the other slice of the bread.
9. Take the slice of bread with the peanut butter and the slice of bread with the jelly and put the two together.

Procedural Programming in C++

```
cout << “Enter the Length:”,
cin >> Length;
cout << “Enter the Width:”,
cin >> Width;
Area = Length * Width;
Cout << “The area is:
<<Area << endl;
```

Figure 4.12 This procedural programming comparison relates a real-life scenario, such as making a sandwich, to the corresponding C++ version of the same scenario. In both cases, each step has its own procedure. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

For example, in C, a procedure that opens a **socket**, or an Internet connection between two computers, is called `g_socket_connect()`(Figure 4.13). This procedure involves executing a series of imperative commands that use operating system features (such as the transport layer), system calls, and networking hardware (such as network interface cards, or NICs), to set up a socket connection. To close that connection and end communication, C uses the `g_socket_close()` procedure, which executes a series of commands that shut down the connection. These procedures use the imperative approach to accomplish their respective tasks, so each function contains a series of imperative statements.

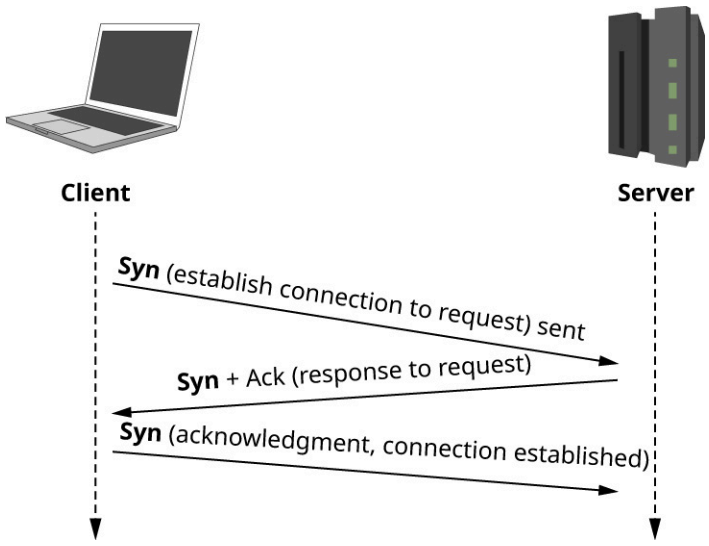


Figure 4.13 This diagram demonstrates various functions used by the socket on both the client and the server side to establish and close a connection. (credit: modification of "Tcp connect" by Sébastien Koechlin/Wikimedia Commons, CC BY 3.0)

A procedural programming language provides syntax for defining procedures but cannot force individual programmers to follow through with breaking their code up into small procedures and giving the procedures descriptive names. So, even when a program is written in a procedural language, the source code may not necessarily be written in a procedural style.

Object-Oriented Programming

Object-oriented code is organized around objects. An **object** has both data, or variables, and procedures that work together to represent a specific human concept. For example, in a desktop or mobile application, every button on the screen is an object. Each button has variables to represent information, such as the location and color of the button, and procedures that perform tasks, such as clicking, hiding, or displaying the button. This programming paradigm is known as **object-oriented programming**. It is a programming paradigm in which code is organized into objects, where each object has both data and procedures. It is a sub-type of procedural programming. All object-oriented languages, then, are procedural (and by extension, imperative), but not all procedural languages are object-oriented. A simple example of an object can be a rectangle used to represent meaningful concepts in real life, such as the rooms in a house or a person or robot and what it can do. Different rooms may have different attributes, representing features that are specific to a kitchen, a living room, or a bedroom. A robot can have a name and age and can receive input commands and respond or print a greeting, as illustrated in Figure 4.14.

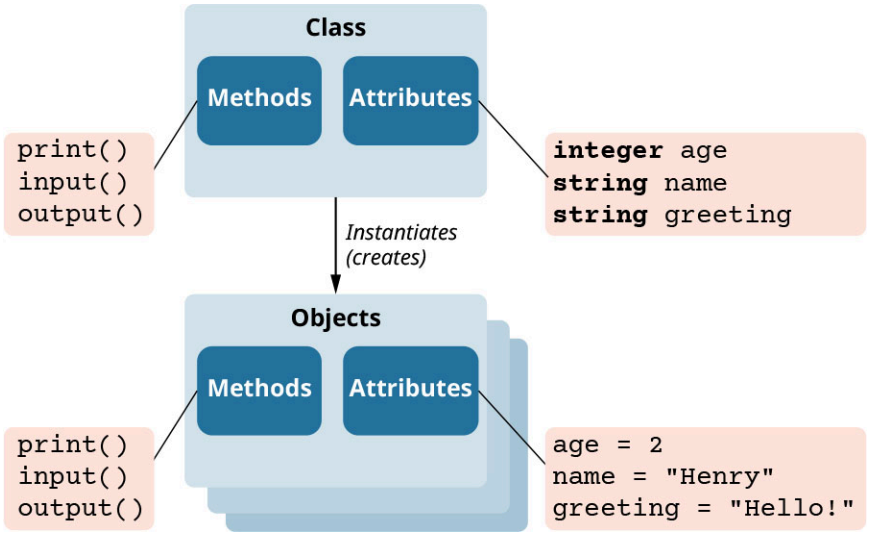


Figure 4.14 The object-oriented use of a robot class instantiates robot objects that have attributes of age, name, and greeting and methods of `input()`, `output()`, and `print()`. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Object-oriented programming was invented to help programmers organize their code, and it has been very successful. The most widely used high-level languages, including C++, C#, Java, JavaScript, and Swift, are all object-oriented. Object-oriented programming is discussed further in Chapter 7 High-Level Programming Languages.

4.2 Building C Programs

Learning Objectives

By the end of this section, you will be able to:

- Write C code using fundamental elements of the language
- Summarize the steps to develop a C program
- Understand the process to compile and run a C program
- Describe how linking is used in a C program
- Understand how to apply version control management

As discussed, a programming language is a kind of computational model that is used to write programs. C is a popular middle-level language that is widely used to create systems software. This section is a crash course in the essentials of C.

Introduction to C

The C programming language was invented in 1972 by Dennis Ritchie of Bell Labs (Figure 4.15) and popularized by the book *The C Programming Language* by Brian Kernighan and Dennis Ritchie. C's peculiar name—a single letter—was a pragmatic choice, since C replaced an earlier language named B. C is a procedural, middle-level language that gives programs low-level access to memory. It is a relatively simple language, which makes learning C, and creating a C compiler, easier than for more complex languages. This combination of features made C an instant hit, and it has maintained great popularity and import to this day. C has influenced other programming languages, too. C++ is a newer language that adds the object-oriented paradigm to C.

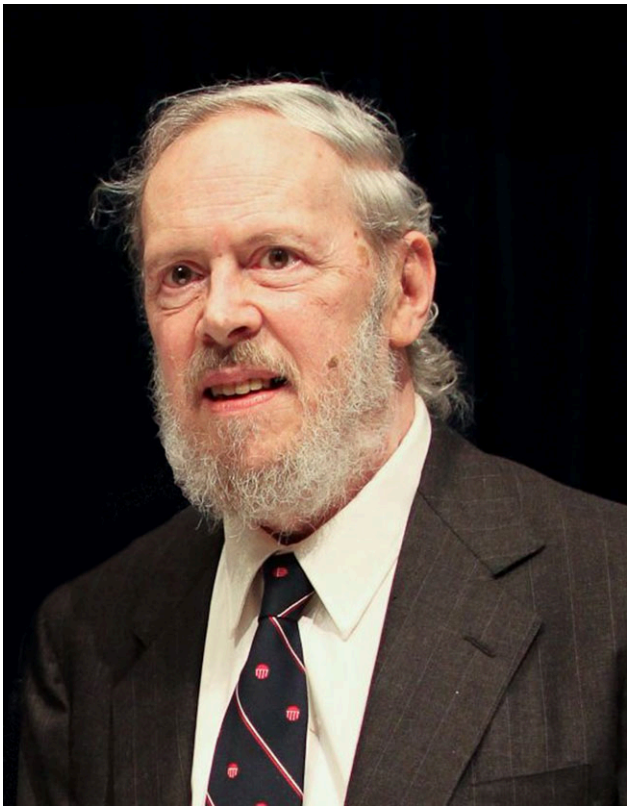


Figure 4.15 Dennis Ritchie of Bell Labs invented the C programming language. (credit: "Dennis Ritchie 2011" by Denise Panyik-Dale/ Wikimedia Commons, CC BY 2.0)

Why is C so popular? Mainly because its designers managed to strike a balance between low-level and middle-level features that allows C code to execute at practically the same speed as assembly language, while allowing programmers to be productive enough to create large, dependable, programs. C is the programming language behind much of the lower-level software that we depend on, including operating systems, language compilers, assemblers, text editors, print servers, network drivers, language interpreters, and command-line utilities. Here are some specific software products that are written in C:

- The Java virtual machine (ANSI C)
- **Linux**, an open-source operating system (C, and some assembly)
- Python (C)
- macOS X kernel (C)
- Windows (C, C++)
- The Oracle database (C, C++)
- Cisco routers (C)

INDUSTRY SPOTLIGHT

Applications of C

C is used in a variety of industries. One example is astrophysics, where scientists write programs that simulate the motion of stellar bodies, and control instruments such as telescopes. Owing to the large size of the universe, these simulations involve performing calculations on very large arrays of numbers. C's ability to execute fast, and control the layout of large arrays in memory, is advantageous for this application. As a relatively simple language, C is approachable to physicists who are not necessarily expert in computer science. Scientific experiments need to be reproducible, which means that code involved in science needs to work even decades in the future. The fact that C has been a stable, popular language for

so long means that it is very likely to endure, which cannot be said of newer niche languages.

One notable feature of C is the way it handle memory. In a program, we have variables and values. For example, in $x = 10$, x is a variable and 10 is the value. Every value in a program is stored in memory. Memory regions are divided into four blocks: stack, heap, static, and code blocks. These regions store various parts of a running program. Running programs create and destroy values extremely rapidly (perhaps millions or billions per second), and memory is finite, so memory locations must be reused, or else would run out quickly. When a value is created, memory is set aside as **allocated memory** to hold that value. Eventually, when the value is no longer needed, that memory becomes **freed memory**, meaning it is given back so that it can be reused. The process of allocating and freeing memory is called **memory management**. A **memory leak** happens when some memory is allocated but never freed. A memory leak is a bug that causes a program to waste resources; severe leaks can waste all the memory on the computer, causing it to become unresponsive or crash. As a middle-level programming language, C requires programmers to handle memory management manually. This type of flexibility must be used with caution as it may result in creating programs that are not reliable and secure. In high-level languages, memory management is automated.

Here are some other notable features of C:

1. Efficient execution: C is lower in expressive power than some other middle-level languages like C++ and yet simple enough that compilers can generate machine code that is comparable in speed to hand-written assembly code. A lot of research and development have focused on creating performance-oriented C compilers.
2. Portability: C can run in multiple computing environments, also known as having the property of portability. Unix was designed to work on various hardware architectures, so the C language is not hardware-dependent. The same C code can be compiled and executed on different hardware architectures and operating systems.
3. Modularity: Modular programming refers to the process of dividing computer programs into separate sub-programs. A module is a separate software component, such as an error handler, that may be used by a variety of applications and functions within a system. C has language support for modularity.
4. Procedural and structured programming support: C adheres to the procedural and structured paradigms.
5. Data types and operators: Every variable in a C program has a data type. Data types dictate how much memory is used to store the variable, and which kinds of operators can be used with the variable.
6. Recursion support: Recursion is the phenomenon of a system being defined in terms of itself. In code, this means a function may call itself again and again. C supports recursion. However, it does not provide a feature called “tail-calling” that makes recursion efficient, so recursion is not used in C as much as in languages that provide tail-calling. A tail call is a function call performed as the final action of a function. If the target function of a tail is the same function, the function is said to be tail recursive, which is a special case of recursion. Tail recursion (also called “tail-end recursion”) is useful and helps with code optimizations.
7. Pointers: A **pointer** is a variable that holds the memory address of another variable and points to that variable (Figure 4.16). Pointers play a crucial role in the C language. They are used to store and manage addresses of dynamically allocated blocks in memory in the underlying computer system. Managing hardware devices involves manipulating certain memory locations, and C’s support for pointers is one of the reasons that it is used to implement kernels and device drivers.

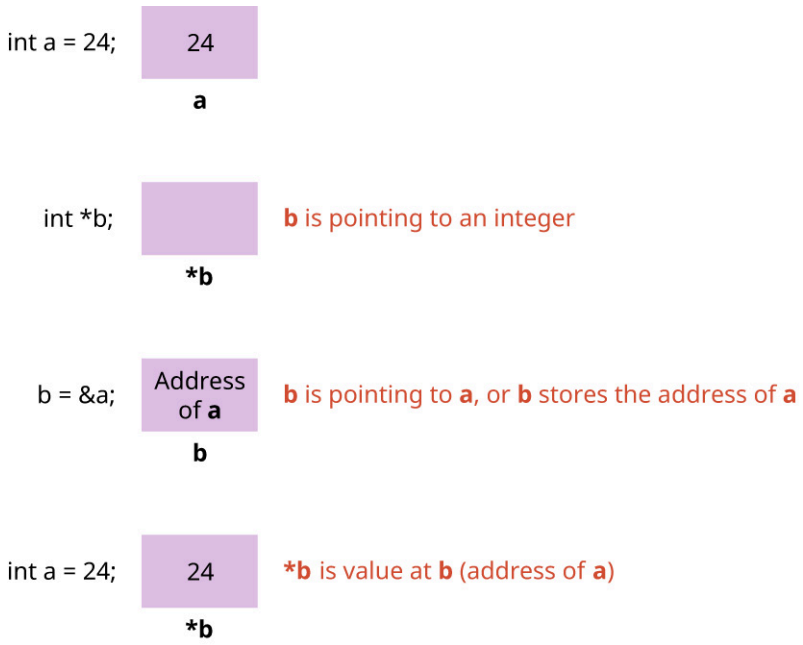


Figure 4.16 A pointer is a variable whose value is another variable's address in memory. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

TECHNOLOGY IN EVERYDAY LIFE

C's Application in the Early Stages of YouTube

C has a variety of integer types, such as `short`, `int`, and `long`. A C programmer needs to decide on the most appropriate type for each piece of information in their program. Smaller types use less memory but can only store a narrower range of values. On a typical computer, the maximum `short` value is about 32 thousand and the maximum `int` value is about 2 billion. A good practice is to think critically about how large a particular value might become and pick the smallest data type that accommodates that range.

The YouTube programmers faced this issue when they implemented the view counter for YouTube videos. They had to think through: what is the maximum number of views that a video is likely to garner? Two billion seemed like a safe choice, so they chose `int`.

This decision turned out to be misguided. In 2014, the viral hit music video "Gangnam Style" by the Korean artist Psy accumulated more than two billion views, and the view counter broke. The `int` variable storing the number of views of "Gangnam Style" overflowed and wrapped around to a negative number. This proved to be an embarrassment for YouTube, who had to quickly change their code to use `long` instead.

What is the most appropriate integer data type (`short`, `int`, or `long`) for the following quantities?

1. The number of people on an airplane
2. The number of people on Earth
3. The number of people in a household
4. The number of dollars in a bank account

High-level languages usually check array indices at runtime, which makes out-of-range bugs easy to identify and fix, but slows down array subscripts slightly. As a middle-level language, C does not check array indices. An array is a storage space where the elements are stored in contiguous memory cells. They are indexed from 0 (the first cell) to n−1 (last cell).

In C, an invalid array subscript will access memory outside of the array variable. If the subscript is only out of range by a little bit, this will access nearby variables, which is a subtle bug that may go unnoticed. A **segmentation fault** ("segfault" for short) occurs if the subscript is very far out of range. When this occurs, it will access a memory address that is off-limits to the program, and your operating system will forcibly shut down the program in response. This kind of runtime error can be notoriously difficult to remedy. Out-of-bounds array subscripts are a common source of segmentation fault errors.

Every value in a program is stored at a specific memory address. A pointer is a value that contains a memory address. Technically, a pointer should contain the location of a valid data value. However, many memory locations do not contain valid data values, so it is possible to have an **invalid pointer** that does not hold a valid location. The pointee is the value that a pointer points at. A pointer is analogous to a street address such as "123 Main Street," because it refers to a specific location. In that analogy, each building is a pointee. Usually, an address is valid and refers to a place you can visit. However, it is possible to have an invalid address that is not a place that can be visited; for example, if the building at that location was demolished.

One of the differences between middle-level and high-level languages is that high-level languages either prohibit invalid pointers entirely, or provide mechanisms to handle them safely. As a middle-level language, C gives programmers the freedom to create null/invalid pointers, which can be helpful when writing code that interfaces with hardware devices. Since all hardware devices do not support the same functionality, the support of individual features by a given device may be indicated as a null/uninitialized pointer, which is fine as long as the program checks for un-initialized pointers to determine if a given functionality is available. However, in general, the freedom of using null/invalid pointers comes with a responsibility to ensure that pointers are always used properly. This has proven to be difficult; invalid pointers are a common source of bugs in C programs.

In C, the programmer is responsible for making sure that character arrays are actually big enough to fit strings, and that strings include the null terminator character. A character array is a string of characters sometimes terminated using a null. An example might be something like: `char *arr= "string\0"`. Overlooking either of these results in bugs. This is a prime example of how middle-level languages such as C expect programmers to deal with more details than do high-level languages.

LINK TO LEARNING

The C standard library has dozens of header files and hundreds of functions. It is impractical to memorize all this information. Programmers do not memorize the prototypes (i.e., name and parameters) of library functions. Instead, they refer to reference documents, and develop the skill of finding information in these documents quickly. These C library reference documents (https://openstax.org/r/76CLibraryDocs) are available in many places.

Developing C Programs

A programmer spends significant time working in their development environment; indeed, a professional developer might spend most of their workday using it. It pays to invest some up-front time and attention toward learning your environment and customizing it to your needs so that your ongoing experience will be frictionless and ergonomic. Chefs, mechanics, and other tradespeople focus much attention on cultivating safe and productive workspaces, and in the same way, experienced programmers attend to their development environment.

Programmers working with compiled languages, including C, generally work using the cycle shown in Figure 4.17.

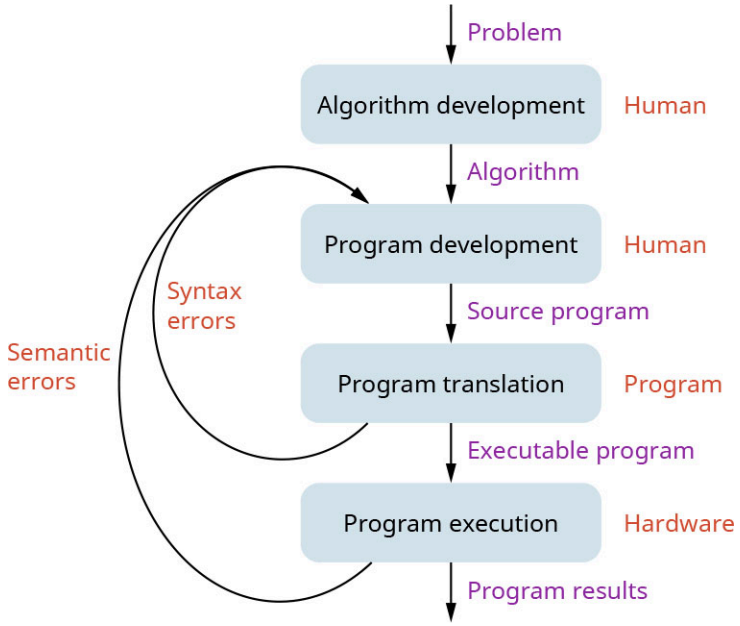


Figure 4.17 A typical work cycle includes multiple compilation steps after a program has been written and is ready for compilation. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Specifically, these steps are:

1. Algorithm Development: The developer designs a high-level understanding of what the code will do and how it will do it. They may document the algorithm with pseudocode, a block diagram, or a sketch. In the case of extremely simple programs, the algorithm may be trivial enough that the programmer can keep it in their head. In some cases, a programmer is implementing an algorithm that someone else created and described in a reference work or research paper.
2. Program Development: The programmer writes code that implements the steps of the algorithm.
3. Program Translation: The programmer runs the compiler on the code. Often, the code has syntax errors, and the compiler provides error messages describing the errors. A syntax error is a violation of the rules for constructing valid statements in the language. For example, the user may have introduced a typo of some sort, like a missing semicolon, or using a keyword as a variable name. In this case, the programmer goes back to Step 2 (Program Development) to resolve the errors one by one.
4. Program Execution: At this step, the code has no syntax errors, so it successfully compiled into a runnable program. The developer runs the program, and tests that it operates properly. An initial draft of code often has a **semantic error**, which is when code compiles and runs, but does not behave as it should. When a programmer finds a semantic error, they go back to Step 2 to debug the code and fix the semantic error. Eventually, after thorough testing, which requires a specific approach not described here, no more semantic errors can be found, and the code is considered finished.

Some C compilers include:

- **GCC**, an open-source C compiler developed by the GNU Project
- **Clang**, an open-source C compiler developed by the LLVM project
- **Visual C++**, a C and C++ compiler developed by Microsoft

Depending on which operating system you are using, there will be many viable alternative C development environments. An operating system is a complex software program that helps the user control the hardware and help with several other applications. Examples include Windows 10 and 11, and Linux versions such as Ubuntu, Fedora, CentOS. You may choose to use an **integrated development environment (IDE)**, which is a program with a graphical user interface that includes a text editor, compiler, and other tools, all in one application. For example, you can install and use Eclipse for C/C++, an open-source multi-language IDE

originally created for Java programming. Eclipse is portable as it is built in Java and can be installed on any operating system.

Compiling and Running C Programs

The compilation process involves several steps:

- compiler: high-level language converts to assembly
- assembler: assembly converts to machine code
- **linker**: a program that performs **linking**, a process of collecting and combining various pieces of object code into a single program file that can be loaded into memory and executed

In practice, compilers such as GCC bundle all these steps into one command. Usually, when you run the GCC command, GCC compiles, assembles, and links a program.

To write, compile, and run a simple C program:

1. Write text of program (i.e., source code) using a text editor, and save it as a text file (e.g., "my_program.c")
2. Run the compiler, assembler, and linker to convert your program from source to an "executable" or "binary." Compilation is necessary for every program to run and perform the desired operation.
   `$ gcc –Wall –g –o my_program my_program.c`
   GCC compiler options:
   - `-Wall` tells the compiler to generate all "warnings." These warnings will often identify mistakes.
   - `-g` tells the compiler to generate debugging information.
   - If you don't supply a –o option to set an output filename, it will create an executable called `a.out`.
   - A `.c` file is called a "module." Often programs are composed of multiple `.c` files and libraries that are linked together during the compilation process.
3. If the compiler gives errors and warnings, edit the source file, fix it, and recompile. It is a good practice to work on just one error/warning at a time, namely the first one. This is because a syntax error can cause false-alarm errors later in the source code, so warnings/errors after the first one could be false alarms. We recommend that, when you get compile errors or warnings, you edit to fix just the first one, and recompile; do not try to fix warnings/errors after the first one.

Consider the following "Hello World" C program[1]:

```
#include <stdio.h>      /* include printf prototype */
/* The simplest C Program */
int main(int argc, char **argv)      /* main program entry point */ {
    printf("Hello World\n");
    return 0;      /* return without error */
}
```

To run a program in the current directory (on Linux) use `./program .` ("." means the current directory). In the world of operating systems, everything is defined in terms of directories and files. Even the desktop is a directory, which is a collection of files. A directory can sometimes be empty too, and some directories have hidden files for security reasons. A subdirectory is a directory within a directory.

```
> ./my_program
Hello World
>
```

1 **argv means that the program is accepting a multidimensional array of input arguments. It is a pointer to the pointer of array of arguments.

Linking Programs

Figure 4.18 illustrates the processing steps of C programs from source code to execution.

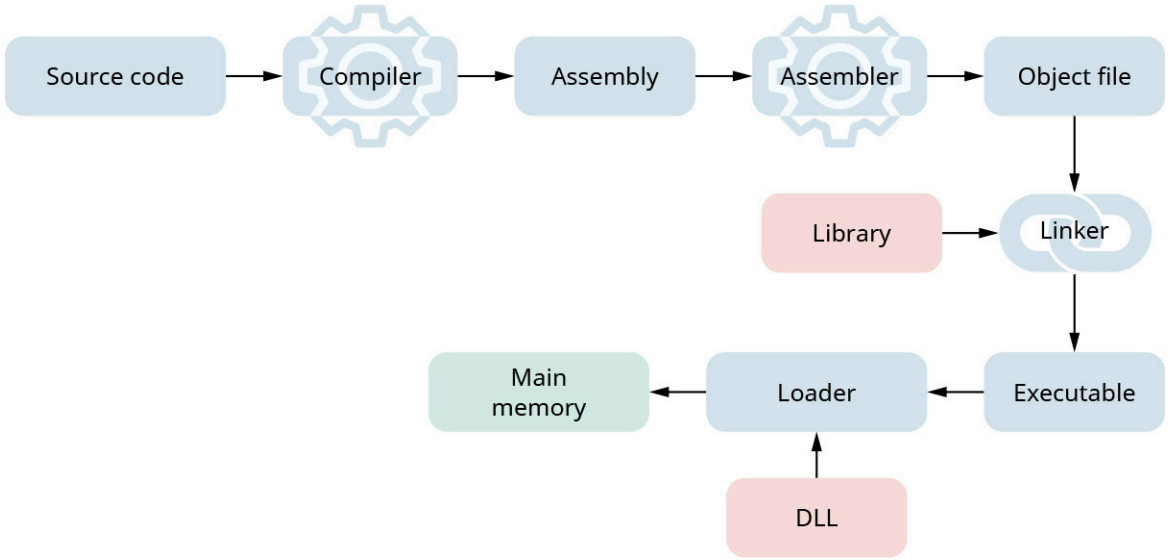


Figure 4.18 The linking process that is used by languages to make them portable requires a number of different steps. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Linking refers to the process of collecting and combining various pieces of object code into a single program file that can be loaded into memory and executed. A linker is a program that performs linking. Understanding linkers will help you build large programs, avoid dangerous programming errors, understand how language scoping rules are implemented, understand other important systems concepts (such as virtual memory and paging), and use shared libraries (a file that is to be shared by an executable file). Virtual memory is an operating system concept where the secondary memory acts as main memory to compensate for memory shortage. Paging is a technique where the secondary memory is used to store and retrieve the data into the main memory. The memory is divided into small regions called pages which enables for the quick access of the data. If a page is found, it is called a “Page hit;” otherwise, it is a “Page miss.”

Programs are translated and linked using a compiler driver, a program that invokes other components that helps in translating the high-level program to a machine code, as in Figure 4.19 and using the following code:

```
linux> gcc -Og -o prog main.c sum.c
linux> ./prog
```

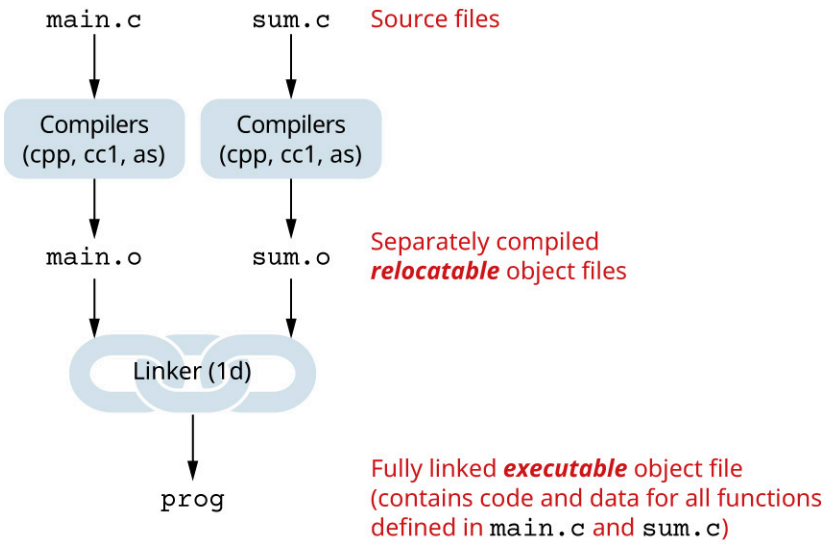


Figure 4.19 Source files and separately compiled relocatable object files can be linked into an executable object file. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Linkers are used to ensure:

- Modularity: Program can be written as a collection of smaller source files, rather than one monolithic mass. Using a linker facilitates building libraries of common functions (e.g., Math library, standard C library). A **library** is a file that contains object code (a code from the object file that is generated after compilation) for functions and global variables (variables that have global scope and can be used anywhere in the program) that are intended to be reused.
- Efficiency: It saves time to run separate compilations and change one source file, compile, and then relink since there is no need to recompile other source files. Also, libraries save memory space because common functions can be aggregated into a single file and yet executable files (the end product after compiling and linking) and running memory images (current memory) contain only code for the functions they actually use.

Linking Steps

Programs define and reference symbol. A **symbol** is an identifier for a function or a global variable. The first linking step performs **symbol resolution**. During the symbol resolution step, the linker associates each symbol reference with exactly one symbol definition (Figure 4.20).

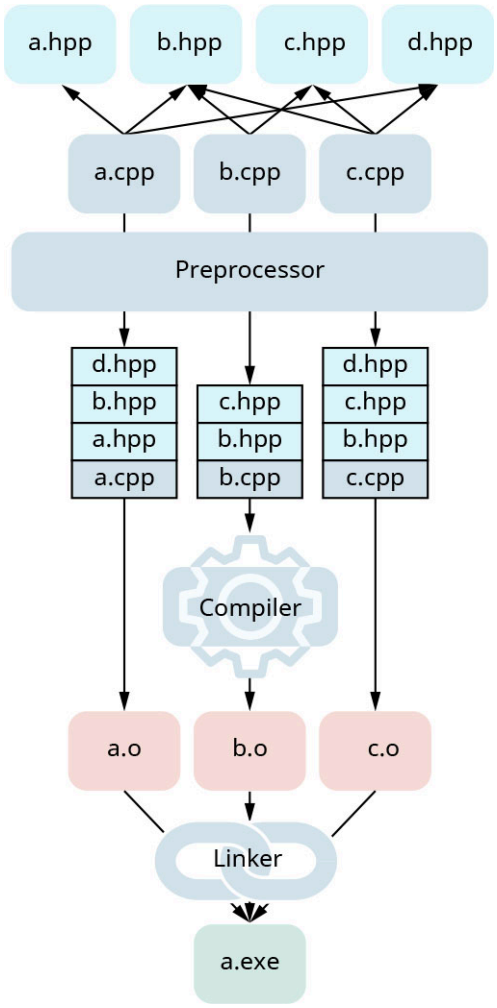


Figure 4.20 In the process of compilation and linking in C++, ".hpp" are the header files, ".cpp" are the actual C++ programs, ".o" are the object files, and ".exe" is the executable. (credit: modification of "C++ compilation process" by "Prog"/Wikimedia Commons, CC0 1.0)

Symbol definitions are stored in an object file (by the assembler) called a **symbol table**. A symbol table is an array of structures in which each entry includes name, size, and location of symbol.

The second linking step performs **code relocation** (Figure 4.21). This step merges separate code and data sections into single sections (one for code and one for data). It relocates symbols from their relative locations in the `.o` files (the object files) to their final absolute memory locations in the executable. It updates all references to these symbols to reflect their new positions.

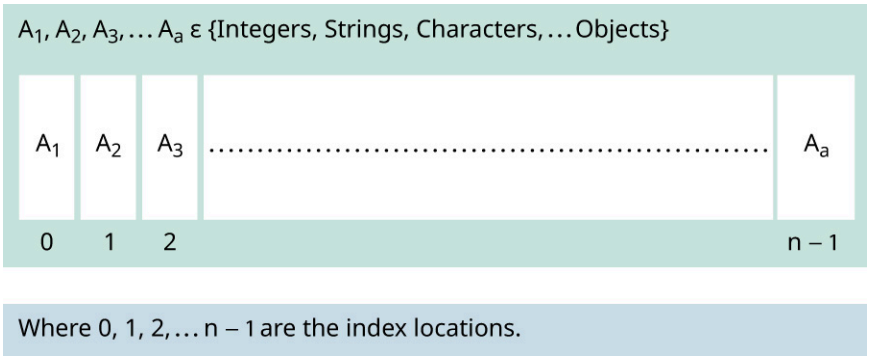


Figure 4.21 This diagram shows the input types in an array and the index locations. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Executable and Linkable Module Format

There are three kinds of object files (modules) that relate to the linking process (Figure 4.22):

- Relocatable Object File (`.o` file): Contains code and data in a form that can be combined with other relocatable object files to form executable object file. Each `.o` file is produced from exactly one source (`.c`) file.
- Executable Object File (`a.out` file): Contains code and data in a form that can be copied directly into memory and then executed.
- Shared Object File (`.so` file): Special type of relocatable object file that can be loaded into memory and linked dynamically, at either load time or runtime. These object files are called Dynamic Link Libraries (DLLs) on Windows.

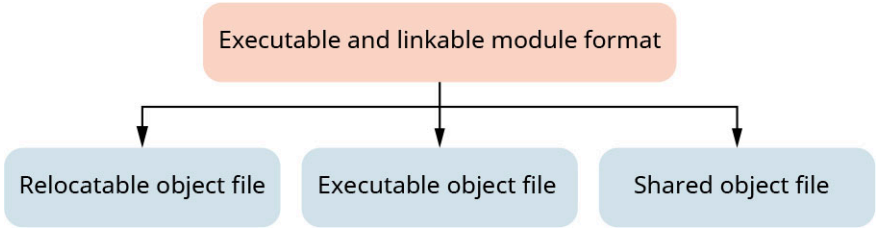


Figure 4.22 The subcategories of Executable and Linkable modules are arranged in a hierarchy. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

All three object files follow the **executable and linkable format (ELF)** which is a standard binary format for object files originally proposed by AT&T System V Unix, and later adopted by BSD Unix variants and Linux. **Unix** is an operating system that has been used widely, primarily in servers and software development since the 1970s, and Linux is Unix-compatible.

Symbol Types and Resolution

A linker classifies symbols in three categories as illustrated in Figure 4.23 and Figure 4.24. A symbol can be the name of a variable or a string. In other cases, it can be the function names or procedure, such as

- Global symbols: Symbols defined by module *m* that can be referenced by other modules (e.g., non-static C functions and non-static global variables)
- External symbols: Global symbols that are referenced by module *m* but defined by some other module.
- Local symbols*:* Symbols that are defined and referenced exclusively by module m (e.g., C functions and global variables defined with the static attribute); local linker symbols are *not* local program variables (linker does not deal with the local variables of a function). Also note that local non-static C variables are stored on the stack while local static C variables are stored in either .bss, or .data.

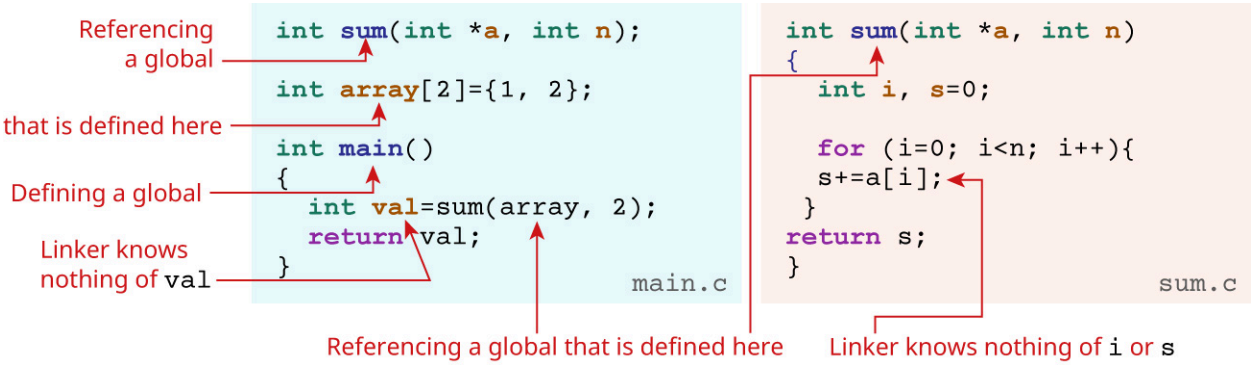


Figure 4.23 The code shown illustrates how the linker identifies local and global symbols. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

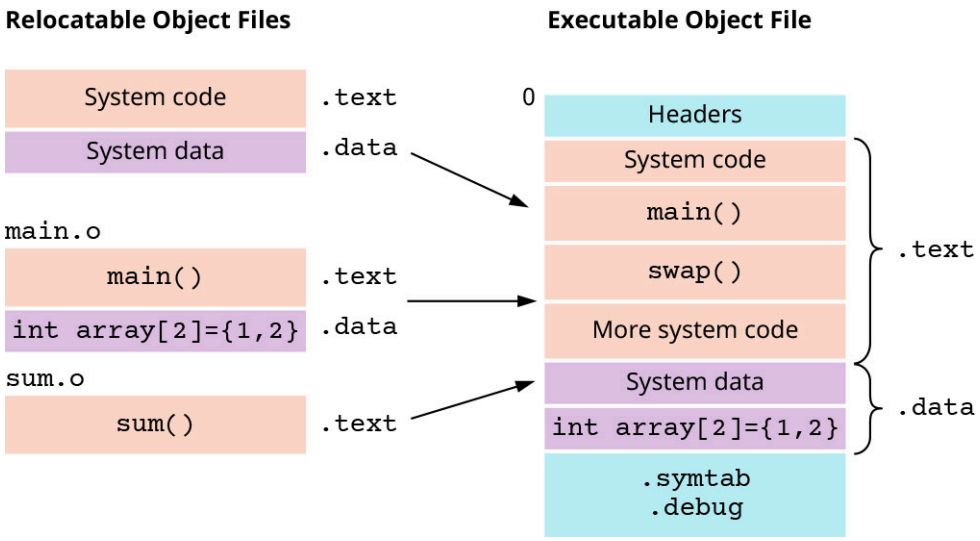


Figure 4.24 This diagram illustrates how various symbols are organized in .text and .data segments within relocatable object files and are mapped into the .text and .data segment by the linker to create executable object files. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Program symbols are either strong (e.g., procedures and initialized globals) or weak (e.g., uninitialized globals). A strong symbol has a unique memory location. Let's take the example of: `int array[2] = {1 ,2};`. This creates an ambiguity during the linking process when there is another file that tries to access the same symbol again due to strong definition. On the other hand, a weak symbol allows multiple definitions of the same symbol without creating an ambiguity. This helps during the linking process when another file creates a strong definition of the same name. In languages like C and C++, the weak symbol is defined using the `_attribute_((weak))` keyword. Global variables should be avoided (i.e., use static whenever you can, initialize the global variable, or use extern if you reference an external global variable).

The linker applies the following rules:

- Rule 1: Multiple strong symbols are not allowed. Each item can be defined only once, otherwise the linker issues an error.
- Rule 2: Given a strong symbol and multiple weak symbols, choose the strong symbol (references to the weak symbol resolve to the strong symbol).
- Rule 3: If there are multiple weak symbols, pick an arbitrary one (can override this with `gcc –fno-common`). "-fno-common" helps in catching accidental common name collisions.

Static Libraries

Functions commonly used by programmers (e.g., math, I/O, memory management, string manipulation) can be packaged into a file called a library. A **static library** (or `.a`, an **archive file**) is a simple kind of library that that copies the contents of object files into a single file called an *archive*. The linker tries to resolve unresolved **external references** by looking for the symbols in one or more archives. An external reference is a symbol that is used in a module, but not defined in that module, so it is expected to be defined in some other module. If an archive member file resolves a reference, the linker links it into the executable. The archiver allows incremental updates; it also recompiles functions that changed and replaces the corresponding `.o` file in the archive (Figure 4.25).

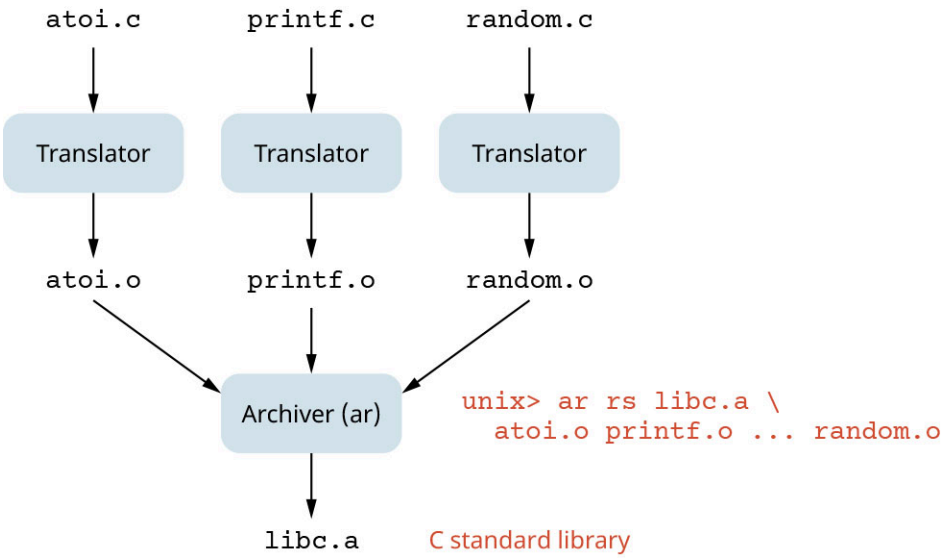


Figure 4.25 The diagram illustrates how the ar archiver utility is used to create a sample version of the `libc.a` static library that only includes the `atoi.o`, `printf.o`, and `random.o` object files. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Commonly used libraries include `libc.a` (the C standard library), which handles: I/O, memory allocation, signal handling, string handling, data and time, random numbers, and integer math. Another common library is `libm.a` (the C math library) that handles floating point math (e.g., `sin`, `cos`, `tan`, `log`, `exp`, `sqrt`).

Figure 4.26 illustrates how to link programs with static libraries.

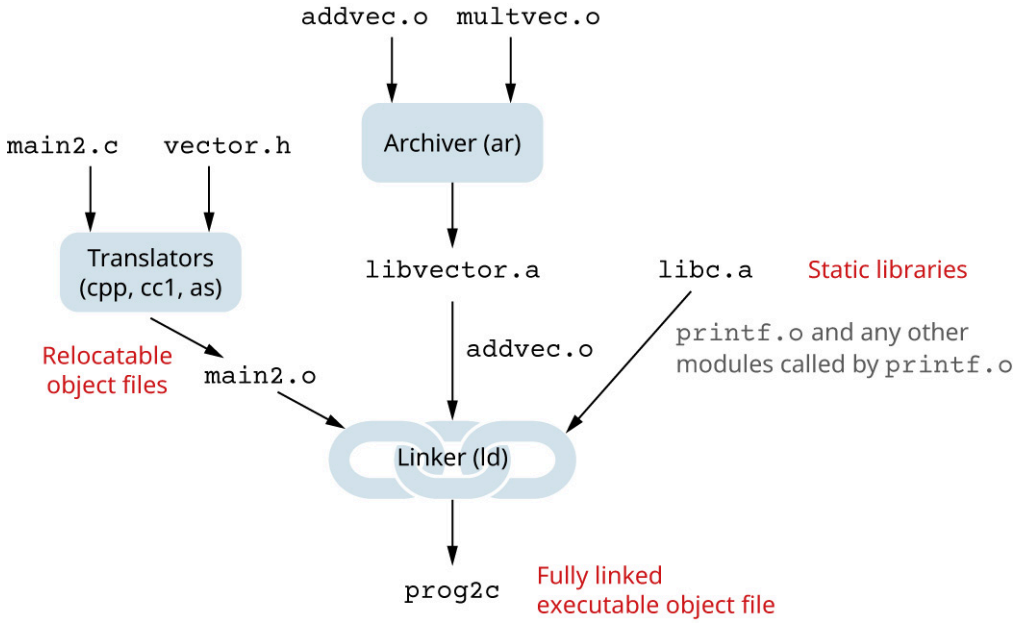


Figure 4.26 This diagram demonstrates the creation of an executable file using various static libraries. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The linker uses the following algorithm to resolve external references:

1. Scan `.o` files and `.a` files in the command line order.
2. During the scan, keep a list of the current unresolved references.
3. As each new `.o` or `.a` file, *obj*, is encountered, try to resolve each unresolved reference in the list against the symbols defined in *obj.*
4. If any entries in the unresolved list at end of scan, then issue error.

Therefore, the command line order matters and libraries should be placed at the end of the command line to avoid linker errors as illustrated in Figure 4.27.

```
unix> gcc -L. libtest.o -lmine
unix> gcc -L. lmine libtest.o
libtest.o: In function `main':
libtest.o(.text+0x4): undefined reference to 'fun'
```

Figure 4.27 Notice the link errors caused due to the incorrect order of the files. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

CONCEPTS IN PRACTICE

APIs in C

The C language makes it possible to create a modular API (Application Programming Interface) as a library with publicly visible function prototypes but secret function definitions. This is accomplished by distributing the `.h` files with function declarations freely, while keeping the `.c` files secret and instead distributing only `.a` or `.so` compiled object code. A `.h` file is used in C, C++, where the libraries can be used in the current program instead of writing the code completely.

An example is `math.h`. This strategy is used in many industries, such as video games. DirectX is an API created by Microsoft for the platforms that are used on Windows PCs and Xbox. Microsoft provides a C library with many function calls for game-related operations such as drawing graphics, playing sounds, and reading inputs from the keyboard, mouse, or joystick. A game programmer writes their game as a C program that calls those functions. This arrangement is a good compromise—the convenience of the DirectX API makes game programmers' work easier, and entices them to create games for Windows and Xbox. But keeping the `.c` files proprietary means that Microsoft does not have to give away the hard work that went into creating DirectX, Windows, or Xbox.

The same arrangement works on other platforms, too. OpenGL is a cross-platform API that works on almost every modern platform, and Sony PlayStation has a similar API. Both of these are distributed as C libraries with public .h files and proprietary implementations.

Loading Executable Object Files

An object file is a file that is a combination of metadata from the source or object code along with a combination of bytecode

Dynamic Load-Time Linking

Static libraries have the following disadvantages: duplication in the stored executables (every function needs `libc`), duplication in the running executables, and minor bug fixes of system libraries require each application to explicitly relink. A modern solution to this problem is to use shared libraries (also called dynamic link libraries, DLLs, or `.so` files). A **shared library** is a library file that can be shared by multiple programs at the same time (Figure 4.28).

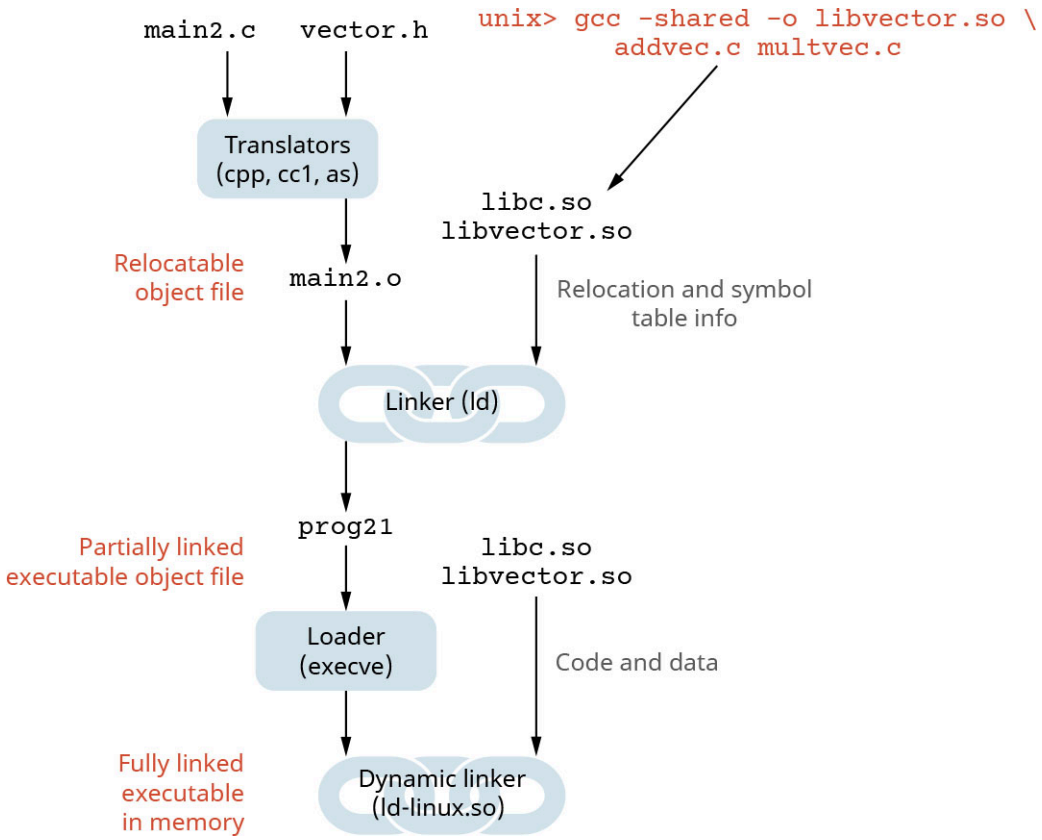


Figure 4.28 A shared library can be used by multiple programs simultaneously. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

When using shared libraries, object files that contain code and data may be loaded and linked into an application dynamically at load time, as illustrated in Figure 4.29. This **load time linking** occurs when dynamic linking happens at the same time that a program executable is first run. This is a common case for Linux, which is handled automatically by the dynamic linker (`ld-linux.so`). The standard C library (`libc.so`) is usually dynamically linked. The ldd tool may be used to identify dependencies/libraries needed at load time. In static linking the routines code becomes a part of the executable. In dynamic linking, the routines can be updated during the code execution. To dynamically link a library at load time on Linux, place it in the `/lib/x86_64-linux-gnu/` directory and compile the source files with the `-l` flag (e.g., `gcc main.c -lcso`).

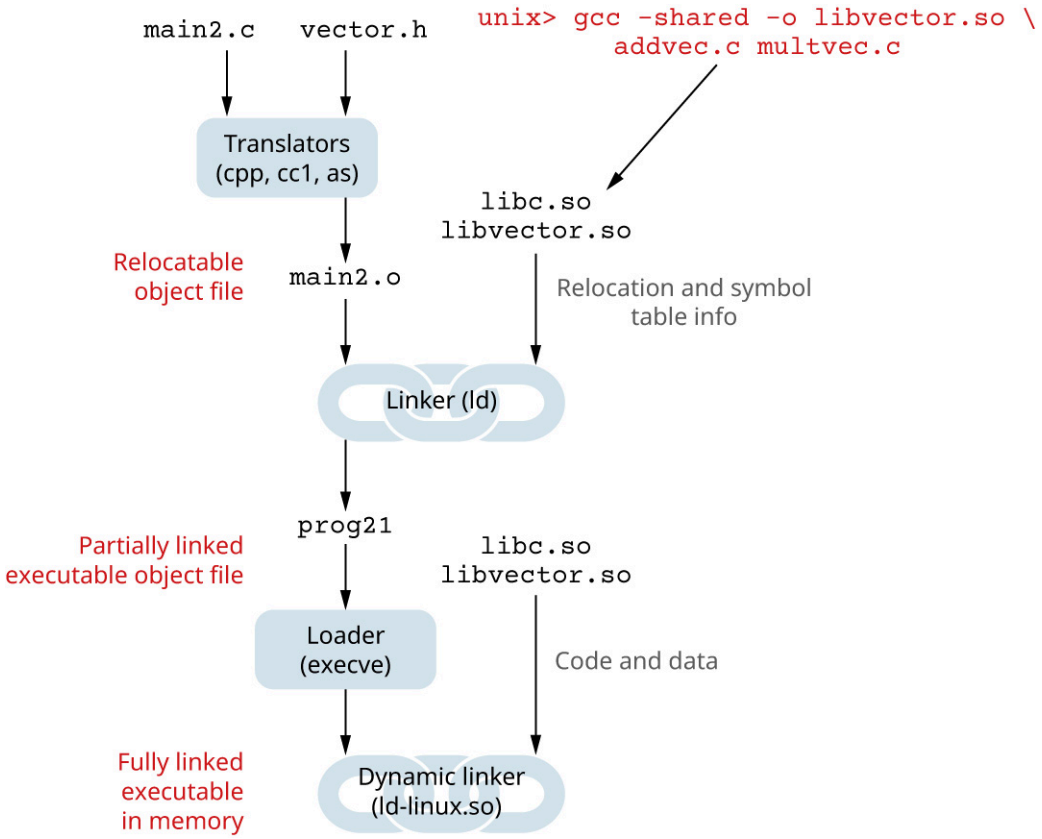


Figure 4.29 The diagram illustrates how object files may be loaded and linked into an application *dynamically*, at *load-time*; in that case, dynamic linking can occur when the program executable is first loaded and run (i.e., load-time linking), which is a common case for Linux that is handled automatically by the dynamic linker (`ld-linux.so`). (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Dynamic Runtime Linking

An alternative to load-time linking is **runtime linking**, which means that linking occurs after a program has already started running. As illustrated in the sample code, the program source code needs to explicitly call functions to link additional libraries. In Linux, this is done by calls to the `dlopen()` interface and compiling the source with the `-l` flag (e.g., `gcc main.c -ldl`). This is a better approach to help distribute software, support high-performance Web servers, or perform runtime library interpositioning.

```
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>
int x[2] = {1, 2};
int y[2] = {3, 4};
int z[2];
int main() {
    void *handle;
    void (*addvec)(int *, int *, int *, int);
    char *error;
    /* Dynamically load the shared library that contains addvec() */
    handle = dlopen("./libvector.so", RTLD_LAZY);
    if (!handle) {
      fprintf(stderr, "%s\n", dlerror());
      exit(1);
    }
```

```
    ...
    /* Get a pointer to the addvec() function we just loaded */
    addvec = dlsym(handle, "addvec");
    if ((error = dlerror()) != NULL) {
      fprintf(stderr, "%s\n", error);
      exit(1);
    }
    /* Now we can call addvec() just like any other function */
    addvec(x, y, z, 2);
    printf("z = [%d %d]\n", z[0], z[1]);
    /* Unload the shared library */
    if (dlclose(handle) < 0) {
      fprintf(stderr, "%s\n", dlerror());
      exit(1);
    }
    return 0;
}
```

Tools to Manipulate Object Files

An object file contains a lot of information such as metadata, machine code, and other information from symbols. To manipulate such files, Unix provides certain tools to use them effectively, such as:

- ar: Creates static libraries, and inserts, deletes, lists and extracts members.
- strings: Lists all the printable strings contained in an object file.
- strip: Deletes symbol information from an object file.
- nm: Lists the symbols defined in the symbol table od an object file.
- size: Lists the names and sizes of the sections in an object file.
- readelf: Displays the complete structure of an object file, including all of the information encoded in the ELF header; subsumes the functionality of “size” and “nm.”
- objdump: Displays all of the information in an object file; useful for disassembling binary instructions in the .text section.
- ldd (linux): Lists the shared libraries that an executable needs at runtime.

Version Control Management

The process and tools used to store and improve multiple versions of project files is called **version control**. Version control also helps support team collaboration, and allows for the ability to revert to an earlier versions. **Git** is a widely-used version control system. Creating and updating project files using Git requires the creation of a Git **repository**, also known as “repo” for short. A repository is a container for files and related information stored in a version control tool. **GitHub** is a website that allows free storage of public git repositories.

LINK TO LEARNING

Learn more by installing Git on your local machine (https://openstax.org/r/76InstallGit) on any platform. You may run “`brew install git`” on MacOS to install Git or “`sudo apt install git`” on Linux.

Useful Git commands are as follows:

- `git config --global user.email "you@example.com"` and `git config --global user.name "Your Name"`
- Clone: to download contents

- Pull: `git pull origin master` to pull latest changes
- Status: `git status` to see staged (shown in green) and un-staged (shown in red) files
- Staging: `git add <filename'` to add files to staged area (wildcards accepted)
- Commit: `git commit –m "<your message here>"` to commit the staged files
- Push: `git push origin master` to push all changes made locally to the origin

LINK TO LEARNING

Explore the Git/GitHub tutorial (https://openstax.org/r/76GitHubTutor) for more details on how to use Git.

4.3 Parallel Programming Models

Learning Objectives

By the end of this section, you will be able to:

- Define parallel computing and related terminology
- Discuss parallel programming approaches

So far, our programs have run on a single at a time and the assumption is that the underlying machine only supported a single GPU core. A CPU **core** is a chip consisting of billions of transistors that function according to an instruction or opcode. It is like a single processor. While there is a lot that we can do with these single-core programs, there is also a need for programs to run in parallel, meaning that they execute code on multiple CPUs, cores, or computers at the same time. In parallel programming, bigger tasks are split into smaller ones, and they are processed in parallel, sharing the same memory. Parallel programming is trending toward being increasingly needed and widespread as time goes on. Many computers now come equipped with a **graphics processing unit (GPU)**, which is a massively parallel processor that supplements a CPU. GPUs were originally designed for rendering real-time graphics in video games and are sometimes called "video cards." A typical GPU has thousands of cores, although each is weaker than a CPU core. Parallel techniques are essential for making use of GPUs.

Parallel Computing Overview

In the 20th century, a computer typically had only one processor. Now, a CPU chip typically holds not just one processor, but multiple processors built into a single computer chip. Each individual processor built into a CPU is a core. A **multicore** processor is a CPU chip that has multiple cores. Multicore CPUs are prevalent; smartphones and budget PCs typically have two to four cores, and high-end PCs have eight or more cores. The trend is for these core counts to increase over time.

By default, a program runs on one core at a time. That means that a four-core computer can run up to four programs at full speed at the same time. That capability is occasionally useful, but more often a user wants a single high-demand program to make full use of their computer. This is the case with productivity software, games, embedded systems, and Web server software. For this to work, the program needs to be coded in a way that explicitly divides work among multiple cores.

Fundamentally, in order to use multiple cores, a program needs to work in "parallel." That means that multiple cores are working together at the same time (Figure 4.30). A real-world example of parallel work is a factory assembly line. If an assembly line has twenty workers, then at any given moment twenty people are working in parallel. This concept of parallel work also applies to software. The parallelism concepts discussed here are:

- **parallel computer**: a multiple-processor system that supports parallel programming.
- **parallel computing**: the practice of making productive use of parallel computers.
- **parallel programming**: a computer programming technique that provides for executing code in parallel

on multiple processors.

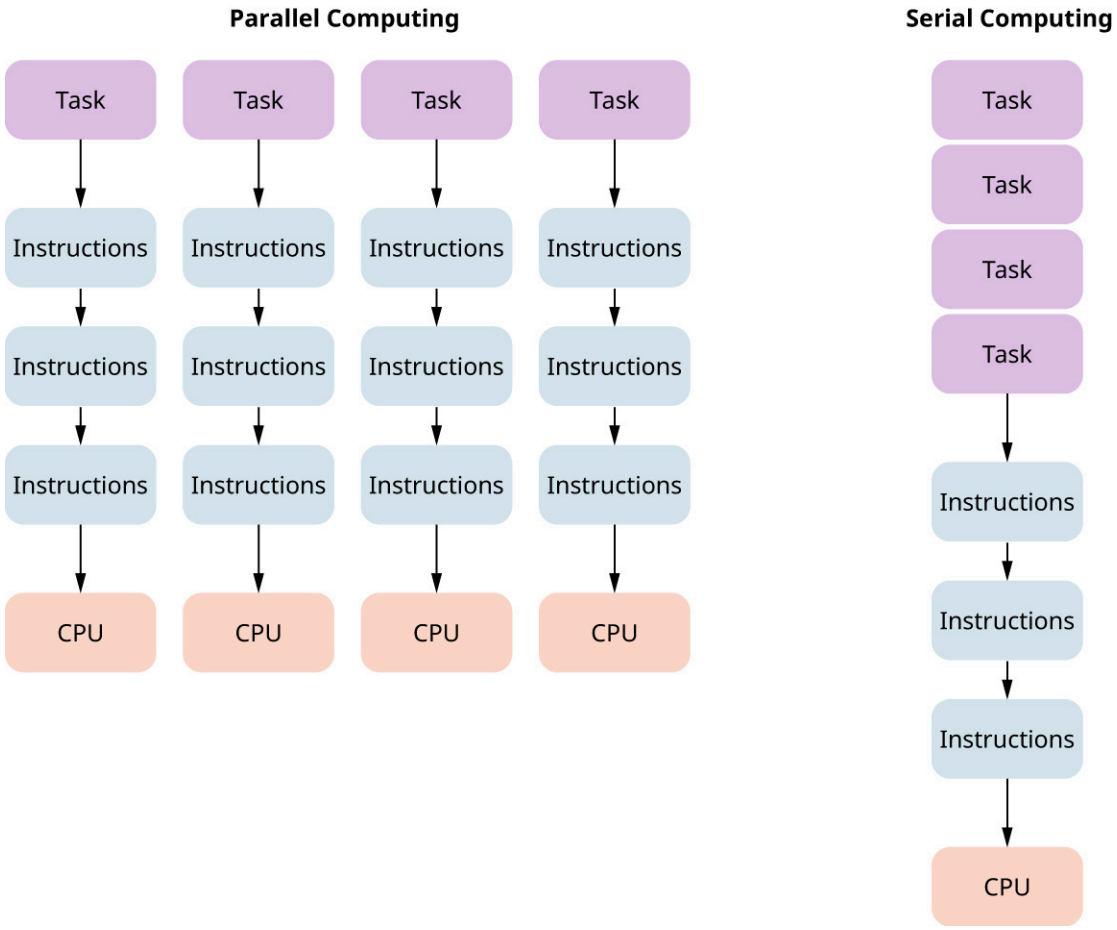


Figure 4.30 This diagram illustrates how multiple computer programs can be executed as tasks on a multi-core machine either in parallel on separate individual cores using parallel computing or in sequence (on a single or on multiple cores) using serial computing. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

There are two related but distinct terms that we should define at this point: **concurrent programming**, which refers to any situation where multiple programs or tasks are running simultaneously, regardless of whether they are using multiple processors or sharing one processor; and **distributed computing**, which is a more specific form of parallel programming where processors are working together in parallel, but the processors are in multiple connected computers, not a single computer. Concurrent programming is a broader term than parallel programming, while distributed computing usually refers to massively parallel programs that run on hundreds or thousands of servers, usually at large companies such as Amazon, Google, the NSA, and the NIH.

THINK IT THROUGH

GPU Applications

GPUs are high-performance parallel processors. Some major applications of GPUs include cryptocurrency mining, video games, and the dashboard computers embedded in automobiles. In 2021, there was a shortage of GPUs due to a “perfect storm” of world events. The COVID-19 pandemic complicated manufacturing, limiting the rate at which GPUs could be built. In response to the pandemic, demand for computers increased, as many workers were forced to work from home. Demand for video games also increased as people sought indoor entertainment. At the same time, cryptocurrency prices went up, which stimulated interest in mining cryptocurrency, so even more people tried to buy GPUs at the same time.

All these events caused a severe shortage. Customers encountered long waiting lists for GPUs, or found that they were unavailable entirely. Scalpers sold GPUs at a substantial upcharge. Some people were unable to buy video games, or computers they needed to complete work. The shortages affected heavy industry; automobile manufacturers had to idle their factories, which impacted the factory workers' livelihoods, and triggered a shortage in automobiles.

This situation pitted knowledge workers, gamers, market speculators, and manufacturers against each other in a struggle for scarce resources.

To what degree is this a problem? Do computing professionals have a responsibility to offer a technical solution, such as a technological alternative to GPUs? Do they have a responsibility to anticipate these kinds of unintended consequences? How should policy makers handle a shortage for a critical resource?

Parallel Programming

Parallel programming involves writing code that divides a program's task into parts, works in parallel on different processors, has the processors report back when they are done, and stops in an orderly fashion. C was not designed with parallel programming in mind, so we need to use third-party libraries for parallel programming in C. Some newer languages were designed with parallel programming facilities from the start.

Parallel Programming Models and Languages

A parallel programming model is a high-level conception of how the programmer can control processors and the data that moves between them.

- Shared Memory: In the **shared memory** programming model, processes/tasks share a common address space, which they read and write to asynchronously. Various mechanisms such as locks/semaphores are used to control access to the shared memory, resolve contentions and to prevent race conditions and deadlocks. One example is SHMEM.
- Threads: This programming model is a type of shared memory programming. A **thread** is a single "heavyweight" process can have multiple "lightweight", concurrent execution paths. A simple example of a thread includes a chat feature, video, or audio in an application like Microsoft Teams. Examples include Pthreads, OpenMP, Microsoft Threads, Java and Python threads, and CUDA threads for GPUs.
- Message Passing: A parallel programming approach where separate processes communicate only by sending messages, not sharing memory. Each set of tasks use their own local memory during computation. Multiple tasks can reside on the same physical machine and/or across an arbitrary number of machines. One example is the **Message Passing Interface (MPI)** that was first developed in the 1990s.
- Hybrid Model: A hybrid model combines more than one of the previously described programming models; currently, a common example of a hybrid model is the combination of the MPI with the threads model. Other examples of hybrid models include MPI with CPU-GPU using CUDA, MPI with Pthreads, and MPI with non-GPU.

To program in parallel, you can extend compilers (i.e., translate sequential programs into parallel programs), extend languages (i.e., add parallel operations on top of sequential language), add a parallel language layer on top of sequential language, and define a totally new parallel language and compiler system. The extend language strategy (2) is the most popular, and MPI/OpenMP are examples.

THINK IT THROUGH

Multi-Threading Parallel Programming

Why is it important at this time for application developers to turn to the multi-threading parallel

programming paradigm and new emerging computing technologies for their application needs?

Designing Parallel Programs

Designing and developing parallel programs has historically been a very manual process. The programmer is typically responsible for both identifying and actually implementing parallelism. Developing parallel code is often a time-consuming, complex, error-prone, and iterative process. For a number of years now, various tools have been available to assist the programmer with converting serial programs into parallel programs. The most common type of tool used to automatically parallelize a serial program is a parallelizing compiler or pre-processor. A parallelizing compiler generally works in two different ways: fully automatic or programmer directed.

In the fully automatic method, the compiler analyzes the source code and identifies opportunities for parallelism. The analysis includes identifying inhibitors to parallelism, and it may determine whether the parallelism would actually improve performance. Loops (do, for) are the most frequent target for automatic parallelization.

In the programmer-directed method, the programmer explicitly tells the compiler how to parallelize the code using "compiler directives" or possibly compiler flags. This approach may be used in conjunction with some degree of automatic parallelization. The most common compiler-generated parallelization is done using on-node shared memory and threads.

If you are beginning with an existing serial code and have time or budget constraints, then automatic parallelization may be the answer. However, there are several important caveats that apply to automatic parallelization: wrong results may be produced, performance may actually degrade, it can be much less flexible than manual parallelization, is limited to a subset (mostly loops) of code, and it may actually not parallelize code if the compiler analysis suggests there are inhibitors or the code is too complex.

The first step in developing parallel software is to (1) understand the problem that you wish to solve in parallel. Next steps include (2) partitioning, or breaking the problem into discrete "chunks" of work; (3) identifying the need for communications between tasks; (4) synchronizing the sequence of work and the tasks being performed; (5) identifying data dependencies between program statements; (6) performing load balancing to distributing approximately equal amounts of work among tasks so that all tasks are kept busy all of the time; (7) establishing granularity as the qualitative measure of the ratio of computation to communication; (8) managing I/O operations that are generally regarded as inhibitors to parallelism; (9) debugging (a technique where the program is read through line-by-line to check for any bugs) parallel code; and (10) analyzing and tuning parallel program performance. Figure 4.31 shows these steps.

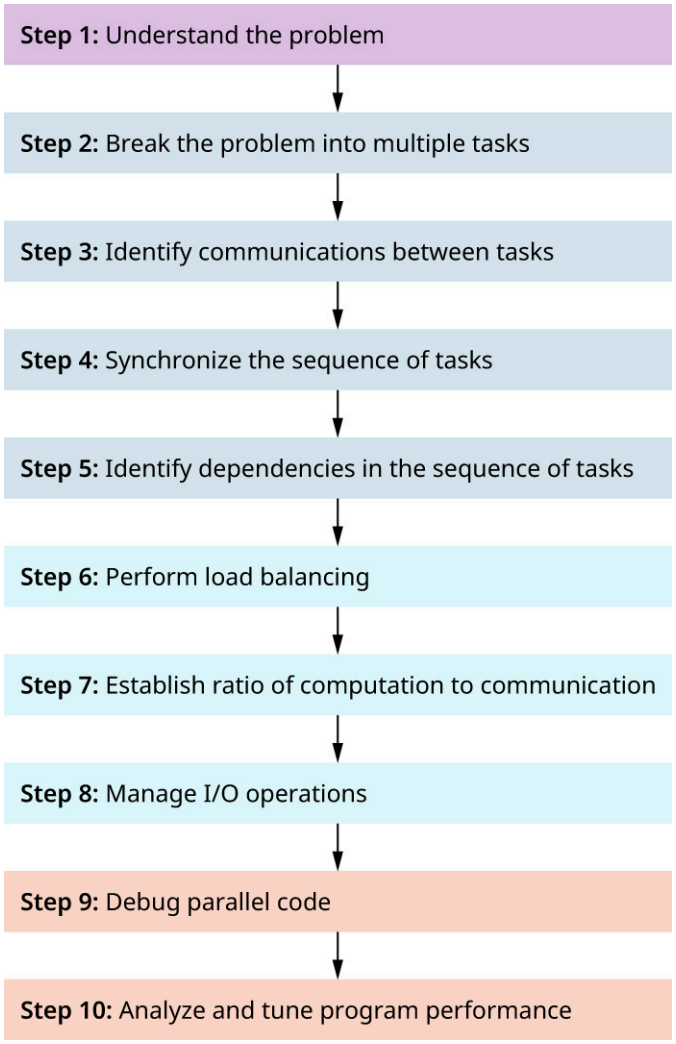


Figure 4.31 Developing parallel software follows ten important steps. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

Parallel programming is a deep subject with many avenues for further study, from the low-level details of hardware and programming to high-level parallel algorithm design. Learn more about the Introduction to Parallel Computing Tutorial (https://openstax.org/r/76ParallelComp) at Lawrence Livermore National Laboratory.

Using C with MPI and OpenMP Parallel Libraries

We focus here on how parallel programs can be written in the C language using an API, which is the most popular method. Some programming languages support parallel programming and may also be used to program parallel applications using message passing features that are built into the language itself. A **message passing** feature is a parallel programming approach where separate processes communicate only by sending messages, not sharing memory. The **symmetric multiprocessor (SMP)** model applies when programming multiple processors that are practically identical. **OpenMP** is a library for parallel programming in the SMP model. When programming with OpenMP, all threads share memory and data. OpenMP supports C, C++ and Fortran. The OpenMP functions are included in a header file called omp.h. An OpenMP program has sections that are sequential and sections that are parallel. In general, an OpenMP program starts with a

sequential section in which it sets up the environment, initializes the variables, and so on. When run, an OpenMP program will use one thread in the sequential sections, and several threads in the parallel sections. The **parent thread** is the thread that runs from the program beginning through end, and starts and manages child threads. A **child thread** is started by the parent thread and only runs for a limited period in a parallel section. A section of code that is to be executed in parallel is marked by a special directive that will cause child threads to form. Each thread executes the parallel section of the code independently. When a thread finishes, it joins the parent. When all threads finish, the parent continues with code following the parallel section.

INDUSTRY SPOTLIGHT

Artificial Neural Networks

The field of artificial intelligence makes heavy use of parallel computing. Artificial neural networks (ANNs) are a widely-used technology that simulates the flow of impulses through nerve cells in a brain. An ANN needs to be "trained" by feeding it many examples of the kinds of inputs and outputs that it will deal with. This training process benefits greatly from parallel programming. A typical ANN has thousands of simulated cells, and is trained on thousands of examples. This makes for millions, or even billions, of computations; parallel computing is a great benefit because this training process can be performed in parallel. Hardware manufacturers, including NVIDIA, Intel, and Tesla, have even created GPU-based computers specifically for the task of training ANNs. Figure 4.32 illustrates the model of a neural network.

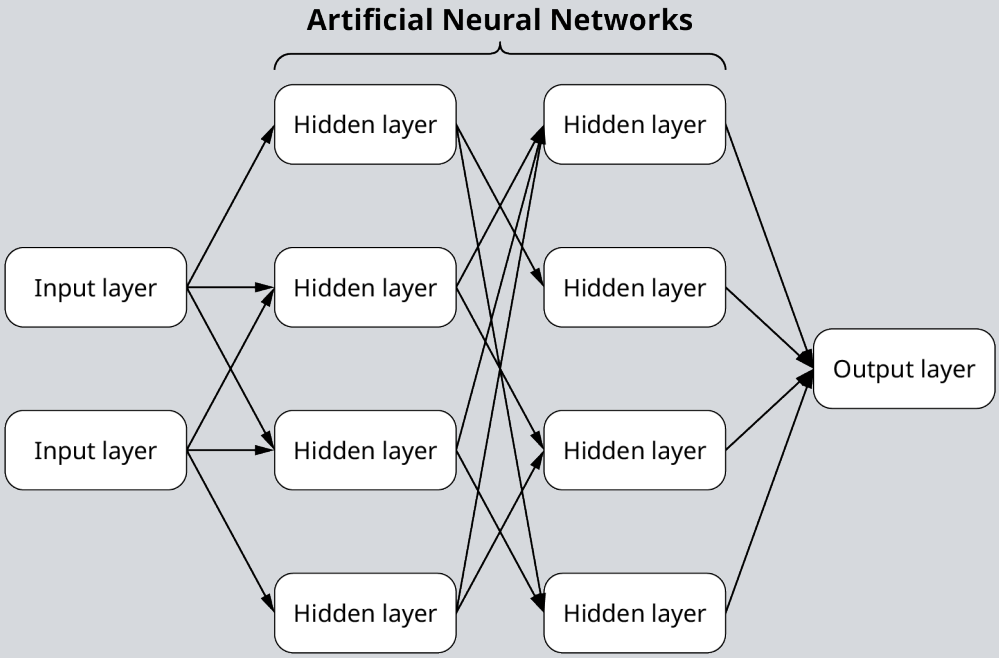


Figure 4.32 The figure illustrates a fully connected set of layers in an ANN. In this case, computing the value maintained by each node requires combining values provided by all the node's input nodes, which explains why training ANNs requires so many computations if all nodes are fully connected to other nodes. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

4.4 Applications of Programming Models

Learning Objectives

By the end of this section, you will be able to:

- Discuss the future of low-level programming
- Understand how the C language is used to develop firmware for embedded systems
- Develop kernel code using the C programming language

High-level languages are popular, and have their place, but there are certain applications where only middle-level languages such as C will do. This section showcases two such applications: firmware and kernel development.

The Future of Low-Level Programming

The economic trends that diminish interest in low-level programming are expected to continue and even accelerate. We are using wider varieties of computer hardware—not just personal computers, mobile devices, and servers, but also narrower segments such as tablets, set-top boxes, video streamers, and system-on-chip (SoC) computers such as the Raspberry Pi (Figure 4.33).



Figure 4.33 A Raspberry PI demonstrates the concept of a system-on-chip (SOC). (credit: "Raspberry-Pi-2-Bare-BR" by "Evan-Amos"/Wikipedia, Public Domain)

The IoT is the growing network of products that are not used as a computer, but nevertheless contain an Internet-connected computer. IoT devices include but are not limited to smart voice assistant speakers, thermostats, home appliances, speakers, and tap payment systems. The computer embedded inside an IoT device is limited in terms of size, energy use, and cost, so it typically has a slow CPU and small memory. This makes middle-level languages well suited to writing IoT software; writing Internet-connected applications in low-level languages is impractical, and high-level languages may not be efficient enough.

Rust

Rust is a relatively new middle-level language created by the Mozilla Foundation in the 2010s. Many of C's positive features are also found in Rust: efficient execution, portability, modularity, procedural and structured programming, and recursion. Rust has the capability to manipulate pointers but adds native safety features so that the compiler can help the programmer prevent bugs related to pointers. The language also includes some features that are more common to high-level languages and are unavailable in C, including higher-level data types (lists, maps, and sets), macros, templates, and parallel programming. These features do make Rust more complicated than C, though. Since Rust has the same positive attributes as C, with some additional desirable features, we expect to see increasing use of Rust as a middle-level language.

GLOBAL ISSUES IN TECHNOLOGY

Naming of C Procedures

It is a best practice to give procedures descriptive names that help a reader understand what the procedure does. English has been globally accepted as the language of programming, but this can cause some problems in non–English-speaking countries. For example, “open_file” is more descriptive than “fopen”; it is also difficult to tell what “fopen” might mean out of context and especially if you are not a fluent English speaker. Unfortunately, procedure names in C are almost always written in English. How can this practice of using English names affect aspiring programmers whose first language is not English? How will it make it more difficult for programmers around the world to collaborate on source code?

Firmware

Hardware is purely physical machinery, and software is purely digital code. To bridge this gap, we have **firmware**, which is very low-level code that communicates directly with hardware, and provides a convenient interface for other software. (“Firm” is the halfway point between “hard” and “soft.”)

If you want to learn to be an embedded systems engineer, it would be best to start from a simple hardware kit, rather than starting with the latest Intel or ARM chipset. **Arduino** is a hardware platform intended for creating simple, low-cost hardware for educational or hobbyist purposes. There are many series of Arduinos, but their "Arduino PLC Starter Kit" has a simple processor and comes with a guide book. Atmega328P has an 8-bit core, which is a good place to start digital circuit design and firmware development. You do not need to know how to draw schematics and layouts and assemble the chips. But you do need to know how to read schematics and understand how the chips are connected. Firmware developers should be able to read the schematics and figure out how to send data to the target device.

TECHNOLOGY IN EVERYDAY LIFE

You and the Internet of Things

As IoT technology advances and computer parts get less expensive, more and more categories of IoT device are coming to market. Fitness monitors have helped people stay in shape. Smart thermostats have conserved energy and made homes more comfortable. Other types of home automation have improved quality of life for older adults and people living with disabilities. These technologies are enabled by middle-level languages that make efficient use of computer hardware, such as C.

What is a new category of IoT device that does not exist yet, but would make your life, or the life of your loved ones, better? What kind of software would this device need? Could you write it in C?

OS Kernels and Device Drivers

The Raspberry Pi board has a Cortex-A53 Processor that supports a 64-bit instruction set. This allows you to experience a modern processor architecture with rPi. Information relating to Raspberry Pi is constantly changing, and the best way to fully understand it is to tackle making your own kernel. There are several websites where you can do this:

- OSDev Wiki (https://openstax.org/r/76OSDevWiki)
- Older toy kernel (https://openstax.org/r/76ToyKernel) that supports 64-bit long mode, paging, and very simple context switching
- The Little Book about OS Development (https://openstax.org/r/76OSDevBook)

- Operating Systems: From 0 to 1 (https://openstax.org/r/76OperSystems)

Making a toy kernel is good way to understand modern computer architecture and hardware control. In fact, you already have a powerful processor and modern hardware devices on your laptop or desktop. This may be all you need to get started.

The Qemu emulator (https://www.qemu.org/) can emulate the latest ARM processors and Intel processors, so everything you need is already on hand. There are many toy kernels and documents you can refer to. You can install Qemu emulator and make a tiny kernel that just boots, turns on paging, and prints some messages. You do not need to make a complete operating system. Join the Linux community and participate in development.

LINK TO LEARNING

This step-by-step guide teaches how to create a simple operating system (OS) kernel (https://openstax.org/r/76OSkernel) from scratch. Each lesson is designed in such a way that it first explains how some kernel feature is implemented in the rPi OS, and then it tries to demonstrate how the same functionality works in the Linux kernel.

Chapter Review

Key Terms

abstract model technique that derives simpler high-level conceptual models for a computer while exploring the science of what new algorithms can or cannot do

allocated memory memory region set aside to hold a value

archive file **.a** file that contains a static library

Arduino hardware platform intended for creating simple, low-cost hardware for educational or hobbyist purposes

assembler program that translates assembly language source code into machine code

assembly language low-level language in which every statement corresponds directly to a machine instruction

BASIC early high-level programming language

binary code program in the native format that is understood by a CPU, which is a long series of 0s and 1s

C middle-level language that has been in wide use since the 1970s

C++ middle-level object-oriented language based upon C

central processing unit (CPU) computer chip capable of executing machine code programs

child thread thread that is started by the parent thread, and only runs for a limited period in a parallel section

Church-Turing Thesis scientific theory stating that an algorithm can be converted from any reasonable computational model to another

Clang open-source C compiler developed by the LLVM project

code relocation merges separate code and data sections into single sections (one for code and one for data)

compiler (also: **interpreter**) program that translates source code from a middle-level or high-level language into something a computer can read

computational model system for defining what an algorithm does and how to run it

concurrent programming situation where multiple programs or tasks are running at the same time, regardless of whether they are using multiple processors or sharing one processor

core individual processor built into a CPU chip

declarative programming paradigm in which code dictates a desired outcome without specifying how that outcome is achieved

device driver piece of code that is responsible for connecting to a hardware component such as a video card or keyboard

distributed computing specific form of parallel programming where processors are working together in parallel, but the processors are in multiple connected computers, not a single computer

executable and linkable format (ELF) standard binary format for object code

external reference symbol that is used in a module, but not defined in that module, so is expected to be defined in some other module

firmware very low-level code that communicates directly with hardware, providing a convenient interface for other software

freed memory memory that is given back to be reused when a value is no longer needed

functional programming paradigm in which algorithms are written as mathematical functions

GCC open-source C compiler developed by the GNU Project

Git widely-used version control system

GitHub website that allows free storage of public git repositories

GOTO non-structured operation that instructs a computer to jump to an entirely different part of the program

graphics processing unit (GPU) massively-parallel processor that supplements a CPU; GPUs were originally designed for rendering real-time graphics in video games

hardware model design for a how a specific physical computer executes algorithms
high-level programming language programming language that operates at a high level of abstraction, meaning that low-level details such as the management of memory are automated
imperative programming paradigm in which the programmer writes a series of steps that must be followed in order
instruction set architecture (ISA) type of hardware model that defines a list of operations that a CPU can execute
integrated development environment (IDE) program with a graphical user interface that includes a text editor, compiler, and other tools, all in one application
interpreter (also: **compiler**) program that translates source code from a middle-level or high-level language into something a computer can read
invalid pointer pointer that does not hold a valid location
kernel core part of an operating system that is responsible for managing and interfacing with hardware components
Lambda calculus abstract computational model defined by Alonzo Church that inspired the functional programming paradigm
level of abstraction degree to which a computational model, programming language, or piece of software relates to computer hardware
library file that contains object code for functions and global variables that are intended to be reused
linker program that performs linking
linking process of collecting and combining various pieces of object code into a single program file that can be loaded into memory and executed
Linux open-source operating system kernel that is Unix-compatible
load time linking when dynamic linking happens at the same time a program executable is first run
low-level programming language programming language that operates at a low level of abstraction, meaning that code is similar to machine code
machine code sequence of binary digits (bits) that can be understood and executed directly by a computer
memory leak occurs when some memory is allocated but never freed
memory management process of allocating and freeing memory
message passing parallel programming approach where separate processes communicate only by sending messages, not sharing memory
Message Passing Interface (MPI) message-passing interface that was first developed in the 1990s
middle-level programming language programming language that is somewhat abstracted above low-level, but not as much as a high-level programming language; allows direct hardware access
modularity property of code that allows it to be divided into a small, reusable piece
multicore CPU chip that contains more than one core
object a program value that has both data, or variables, and procedures that work together to represent a specific human concept
object-oriented programming paradigm in which code is organized into objects, where each object has both data and procedures
OpenMP library for parallel programming in the SMP model
operating system software that provides a platform for applications and manages hardware components
operator fundamental programming operation that combines values
parallel computer multiple-processor system that supports parallel programming
parallel computing practice of making productive use of parallel computers
parallel programming computer programming technique that provides for executing code in parallel on multiple processors
parent thread thread that runs from the program beginning through the end, and starts and manages child threads
pointer variable that holds the memory address of another variable and points to that variable

procedural programming paradigm in which code is organized into procedures
procedure function in the context of programming
programming language paradigm philosophy and approach for organizing code
programming model design for humans to read and write
Random Access Machine abstract computational model used to analyze the efficiency of algorithms
repository container for files and related information stored in a version control tool
runtime linking when linking occurs after a program has already started running
Rust a relatively new middle-level programming language created by the Mozilla Foundation in the 2010s
segmentation fault occurs if the subscript is very far out of range
semantic error when code compiles and runs, but does not behave as it should
shared library library file that can be shared by multiple programs at the same time
shared memory programming model in which processes/tasks share a common address space, which they read and write to asynchronously
socket Internet connection between two computers
source code text of a program written in a programming language
static library simple kind of library that that copies the contents of object files into a single file called an "archive"
structured programming paradigm in which control flow is always controlled with conditionals ("if") or loops ("while") and never GOTO
symbol identifier for a function or global variable
symbol resolution during the symbol resolution step, the linker associates each symbol reference with exactly one symbol definition
symbol table array of structures in which each entry includes name, size, and location of symbol
symmetric multiprocessor (SMP) model in which there are multiple parallel processors that are practically identical
systems software programs that provide infrastructure and platforms that other programs rely upon
thread light-weight parallel execution path that shares memory with other threads
Unix operating system that has been used widely, primarily in servers and software development since the 1970s
version control tools that are used to store and improve multiple versions of project files and support team collaboration, and the ability to revert to an earlier versions
Visual C++ proprietary-license C and C++ compiler developed by Microsoft

Summary

4.1 Models of Computation

- A computational model defines what an algorithm or program does. There are hardware models, programming language models, and abstract models.
- Low-level programming means writing machine code that can be understood by a CPU directly, or something very near to that. It is laborious but yields very fast code.
- Middle-level programming is a compromise that is reasonably efficient and more convenient than low-level programming.
- High-level programming is more abstract and intuitive for humans. It is less labor-intensive, but high-level code can be slower than low-level code.
- Programming language paradigms are approaches for organizing source code.
- In the imperative paradigm, code orders the CPU to execute specific actions.
- In the declarative paradigm, the programmer declares the outcome that they need.
- In the functional paradigm, the programmer defines mathematical functions to evaluate.
- In structured programming, the flow of execution is specified with explicit syntax elements ("if-then-else," "for" loop, "while" loop) and never GOTO.
- In procedural programming, a program is divided into procedures. Each procedure performs one specific

task and has a descriptive name.
- In object-oriented programming, the basic building block is an object. An object combines data and procedures that together represent a human concept.

4.2 Building C Programs

- The C programming language is the most prominent example of a low-level language. Programs written in C typically execute as fast as assembly language and allow programmers to directly manipulate machine features such as memory via the use of pointers.
- The C programming language is used by most of the system software we depend on today (e.g., operating systems, compilers, interpreters, and device drivers) because of its efficient execution, portability, and modularity.
- The C data model supports the creation of a variety of basic types including integers and floating point numbers, as well as pointers. C also provides type constructors used to create collections using the `array`, `struct`, and `union` keywords.
- C supports the imperative and structured/procedural programming paradigms and allows for conditional and iterative statements as well as functions, which can leverage recursion.
- Program development steps in C require designing algorithms, developing programs that implement algorithms, and compiling, linking, and executing programs. All of these steps may be performed within a C development environment, which is a suite of tools that a programmer uses to create software. It must include a text editor, compiler, and linker, and may include other tools such as a version control manager, and others.
- A development environment may be an assemblage of separate command-line programs, or an IDE, which is a development environment bundled into a single app. There are a multitude of C development environments, and many of them are free to use.
- Compiling C programs involves converting C into assembly language, which can itself be translated into machine code. This process is performed by using a combination tools known respectively as a C compiler, assembler, and linker. The GCC compiler is an open-source C compiler developed by the GNU project, it include gcc, ar, and ld to implement compiling, assembly, and linking.
- Linking is the process of combining the .o files that result from separate C modules into one deliverable library or executable program.
- A library is a file that contains the object code of compiled functions. There are several variations of libraries (static, dynamic, shared) and ways of linking them (load-time, run-time).
- Version control tools manage the files created in programming, facilitating collaboration, backups, and undoing errors.

4.3 Parallel Programming Models

- Multicore computers are commonplace. Most consumer mobile devices, computers, and video game consoles have between two and eight cores. As time goes on, the number of cores in computers tends to increase.
- Parallel computing does not happen automatically. Rather, a programmer must deliberately write a program in a parallel manner in order for it to use multiple cores.
- A variety of models of parallel programming exist, including shared memory, threads, and message passing.
- OpenMP is a library for writing parallel code using the message-passing model.
- One strategy for parallel computing is to have a parent thread, which creates and controls child threads. The child threads work in parallel.

4.4 Applications of Programming Models

- Low-level and middle-level programming will continue to be important as society increasingly relies on low-powered computing devices and IoT.
- Middle-level languages including C are ideal for developing firmware and kernels.

- Rust is a relatively new middle-level language that is gaining traction.
- Arduino is an embedded computer platform. Arduino firmware can be written in C.
- Raspberry Pi kernels can be developed in C.

Review Questions

1. What is the difference between machine code and assembly language?
   a. Machine code is written in a textual format, while assembly language is written in hexadecimal.
   b. Machine code is executed directly by the CPU, while assembly language must be interpreted.
   c. Machine code is written in binary, while assembly language is written in a textual format.
   d. Machine code and assembly language both require a compiler to be executed.

2. Why are middle-level programming languages like C important?
   a. They are used exclusively for Web development.
   b. They are used to create systems software, such as operating system kernels.
   c. They do not allow access to hardware features.
   d. They are used only for academic purposes.

3. What is an advantage of high-level programming over low-level programming?
   a. High-level programming languages are less time-consuming for the programmer.
   b. High-level programming languages are slower.
   c. High-level programming languages have a lower level of abstraction.
   d. High-level languages offer less security and reliability.

4. What defines the order of code execution in the imperative programming paradigm?
   a. the compiler
   b. data flow and transformations
   c. the steps in the code
   d. There is no specific order of execution.

5. What is the difference between an IDE and a development environment that is not an IDE?
   a. An IDE is a single tool for coding, while a non-IDE environment requires no tools.
   b. An IDE bundles all of the tools into one app with a graphical interface, while a non-IDE environment is a collection of several tools.
   c. An IDE uses only command-line programs, while a non-IDE environment uses graphical tools.
   d. A non-IDE environment uses graphical tools only.

6. What is a critical step that a programmer must focus on when participating in the development of a C program?
   a. planning the development
   b. documenting the code
   c. fully testing the code
   d. generating code

7. What is “ELF” in C programming?
   a. a function library for C programs
   b. an error logging framework in C
   c. a coding standard for writing C programs
   d. a file format for object code

8. Why is linking necessary in C programming?
   a. to combine separate object files into a single executable or library
   b. to compile the source code into object code
   c. to debug the program before it is executed

d. to run the compiled program on the operating system

9. What is a core?
a. a type of memory used for storing data in a computer
b. an individual processor in a CPU chip that can execute instructions
c. a program that runs on a computer
d. a software module that manages system resources

10. What is a thread in the context of computing?
a. a single task that executes on a core
b. a component that stores data in a database
c. a network connection between two computers
d. a type of memory used in parallel computing

11. What are the roles of a parent thread and child thread in parallel computing?
a. The child thread starts other threads and manages the program's execution.
b. The parent thread performs tasks assigned by the child thread.
c. The parent thread starts child threads, monitors them, and cleans the program.
d. Both parent and child threads perform the same tasks simultaneously.

12. What are examples of parallel programming models?
a. single-threaded and multi-threaded
b. interpreter and compiler
c. input/output and file systems
d. shared memory and message passing

13. What is firmware?
a. a code that interacts with hardware devices
b. a type of high-level software with high abstraction
c. an operating system that runs on embedded devices
d. a hardware component that updates software

14. Why is it recommended to start with simple hardware kits when learning embedded systems engineering?
a. Simple kits provide advanced processors.
b. Simple kits are intended for industrial use and can be used for complex processes.
c. Simple kits allow users to build digital circuit design and explore firmware development.
d. Simple kits require extensive prior knowledge of x86-64 architecture.

15. What features does the Rust programming language offer that makes Rust more desirable then C?
a. Since the language is object oriented, there is higher level data types.
b. Rust is a high-level language, so it tends to be more readable.
c. Rust provides a standard library and C does not.
d. Has the same positive features of C and includes new features supporting higher level data types and parallel programming.

Conceptual Questions

1. Why are low-level languages like assembly language not portable?

2. Why do multiple computational models exist? Why don't we just use one model for everything?

3. What are some disadvantages with structured programming languages like C?

4. Why does the execution speed of software matter? Describe a scenario where slow execution speed would impact a computer user negatively.

5. What is an advantage of low-level programming over high-level programming?
6. What is the best way to use GOTO in the structured paradigm?
7. For each of the following programming languages: investigate the language. Is the language low-level, middle-level, or high-level? Which of the paradigms covered in this section apply to the language (imperative, declarative, functional, structured, procedural, object-oriented)? Cite your sources.
    - Fortran
    - Haskell
    - Smalltalk
8. What is the output of the following C program, and what is the program's purpose here?

```c
#include <stdio.h>
int main() {
  int a=10;
  int *b=&a;
  printf("%d", b);
  printf("\n");
  printf("%d", &a);
  printf("\n");
  printf("%d",*b);
  return 0;
}
```

9. What is the output of the following C program, and what is the program's purpose here?

```c
#include <stdio.h>
int main() {
  int arr[2][3] = { 10, 20, 30, 40, 50, 60 };
  printf("Array:\n");
  for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
      printf("%d ",arr[i][j]);
    }
    printf("\n");
  }
  return 0;
}
```

10. Compare and contrast static linking of a program with dynamic load-time linking. What are the advantages and challenges of each?
11. Compare and contrast dynamic load-time linking of a program with dynamic run-time linking. What are the advantages and challenges of each?
12. Non-parallel programs work, and parallel programming can be difficult. Why is it important for programmers to make the effort to make their programs parallel? Why not stick with non-parallel programming?
13. Why is it difficult for developers to use the multi-threaded programming paradigm in order to fully utilize the capabilities of today's available multicore processors?

14. Do you need to know how to assemble hardware and draw schematics to implement firmware for embedded systems?

Practice Exercises

1. List three pros or cons about each level of language in terms of execution time, complexity, readability, abstraction, and speed of development.
2. Write 8086 assembly code to add the values 1 and 3 together. The registers abx and cdx are available to use for this operation and the result should be stored in abx.
3. Write a main function in C that calls another function to add the numbers 1 and 3 together and return the sum as an output parameter. Finally, print out the answer to the console.
4. Write a main method in Java that calls another method to add the numbers 1 and 3 together and then print out the answer to the console.
5. Write the GCC commands to compile file1.c, file2.c and file3.c and then link the object files to create a static library titled myLib.
6. Write a main function in C that calls a public function titled `PrintList()` in a module titled listOperations.
7. Write a C module that includes 2 global integer variables declared in the file titled `variables.h` and then print the global variables in a main function that exists in `main.c`.
8. Write a Git command to pull changes from a repository on your local host machine with the URL "https://localhost/MyRepository" into your working repository checkout.
9. Write a GCC command to compile a C file titled `main.c` that includes a static library titled `myStaticLib`.
10. Trace the following C code and list the contents of the array after the iteration.

```
int main() {
  int List[5];
  int a = 10;
  for (int i = 0; i < 5; i++)
  {
    List[i] = a + i;
  }
  return 0;
}
```

11. The following is a C "hello world" program that uses OpenMP. How many lines of messages will this program generate at runtime?

```
#include
int main() {
int x = 1;
int y = x + 2;
#pragma omp parallel num_threads(y * 3)
{
printf("https://helloacm.com\n");
}
return 0;
}
```

12. Write a C module that creates four threads to call a function that prints the thread number out.

Problem Set A

1. For each of the following programming languages: investigate the language. Is the language low-level, middle-level, or high-level? Which of the paradigms covered in this section apply to the language (imperative, declarative, functional, structured, procedural, object-oriented)? Cite your sources.

    A. Kotlin
    B. Lisp
    C. PASCAL

2. Explain how levels of abstraction affect speed of development and speed of execution.
3. Explain how a compiler assists in providing abstractions for high-level languages.
4. Write a module titled “triangle operations” that has two functions: one to compute the area of a triangle given a base and height and one to compute the perimeter of the triangle given three sides. Then, write a main function that iterates three times, increasing each variable by one, and then calls each function.
5. OpenMP provides the `omp_get_thread_num()` function in the header file `omp.h`. To get the number of total running threads in the parallel block, you can use function `omp_get_num_threads`. How can you modify this program to ensure that only one thread executes the “Greetings from process” `printf` statement?

```
#include <stdio.h>
#include <omp.h>

int main() {
  #pragma omp parallel num_threads(3)
  {
  int id = omp_get_thread_num();
  int data = id;
  int total = omp_get_num_threads();
  printf("Greetings from process %d out of %d with Data %d\n", id, total, data);
  }
  printf("parallel for ends.\n");
  return 0;
}
```

6. Read the documentation on device drivers (https://openstax.org/r/76PSA1) and implement the various examples provided.

Problem Set B

1. Rosetta Code (https://openstax.org/r/76RosettaCode) is an archive of computing tasks and source code written in many different languages that accomplish the same task. Explore the page about converting numbers into Roman numerals (https://openstax.org/r//76Rosetta) and study the source code written in 8080 assembly (low-level), C (middle-level), and JavaScript (high-level). Compare and contrast the following aspects of the code:

    - Length
    - Readability: how easy is it to understand how the code works?
    - Level of abstraction

- Structured or unstructured

2. Research x86, ARM, and PowerPC architectures—specifically, how each of them has different assembly language features and syntax. Then research and explain how high-level languages can be compiled on different computer architectures.
3. Provide a real-life example of abstraction and explain how it is similar to abstraction in computing.
4. Research Java Abstract classes. After researching, provide a detailed usage of abstract classes and explain why abstraction is useful in software development.
5. Let a and b denote object modules or static libraries in the current directory, and let a->b denote that a depends on b, in the sense that b defines a symbol that is referenced by a. For each of the following scenarios, show the minimal command line (i.e., one with the least number of object file and library arguments) that will allow the static linker to resolve all symbol references:

   a. `p.o -> libx.a -> p.o`
   b. `p.o -> libx.a -> liby.a and liby.a -> libx.a`
   c. `p.o -> libx.a -> liby.a -> libz.a and liby.a -> libx.a -> libz.a`

6. Write a program that utilizes parallel computing, then has a safety-critical section of code that only allows one thread, and then the remainder of the program can use the same number of threads the first section used.
7. Use QEMU (https://openstax.org/r/76QEMU) and gdb (https://openstax.org/r/76gdb) to run the kernel source line-by-line.

Thought Provokers

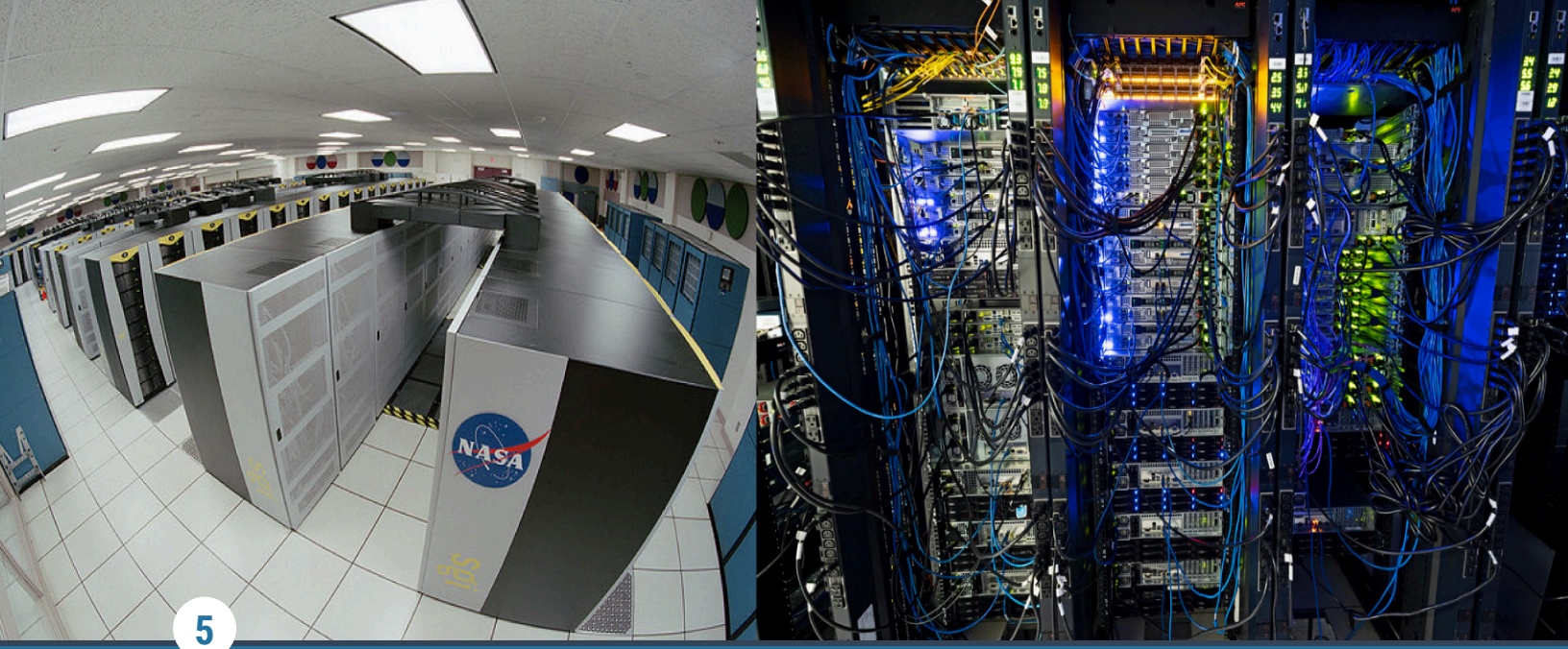
1. You are working on a project as a lead software engineer. Your team is tasked with writing a web application and a sensor that will collect and report the temperature of a room over the course of a week on a single battery charge. Your team is well versed in JavaScript and C. What language would you select to write the web application GUI and the code to operate the sensor? Explain the choice of language while connecting the level of each language.
2. TechWorks decided to implement their prosthetic control software with a low-specification (e.g., reduced instruction set) CPU and the C programming language. Alternatively, they could have used the high-level language Java, and a more powerful CPU that consumes more energy. How does this design decision impact the user's experience?
3. The TIOBE Index (https://openstax.org/r/76TIOBEIndex) is a ranking of the popularity of programming languages. How does the popularity of C, Java, and Python compare? Why do you think that is?
4. TechWorks is a small, growing startup and has four intern programmers working on their prosthetic product. Suppose that you are their manager. So far, they have made do without using any version control system. One of the intern programmers, Alice, suggests that they should set up and use Git as other programmers do in the company. She estimates that she would need to spend one day setting up the server, and all four intern programmers would need to spend one day to learn how to use Git. Is this a justified use of time? Why or why not?
5. The TechWorks prosthetic CPU has two cores, and the control software is written in C. Currently the code does not use any parallel programming. One of the programmers on the team, Bob, suggests that the software should use threads so that the function that moves the prosthetic, and the function that applies software updates, can run at the same time. What are the advantages and disadvantages of this approach?

6. When the Mozilla Foundation created Rust, C was already an established middle-level language. Why was it worth the effort for them to create an entirely new language? You may wish to consult online sources; if you do, cite them.

Labs

1. Work with a partner to collaborate using GitHub. Both students should create GitHub accounts, which are free. Student A creates a repository and adds Student B as a collaborator. B pulls the repo, makes changes, and pushes them. A pulls B's changes, makes some additional changes, and pushes them. B pulls again and sees A's changes reflected.
2. Set up a Git server and client. Install and configure the Git server on your computer; you may need to consult Internet resources. Then, use the command line `git` client tool to create a repository, add some files to it, and push the files. Confirm that you can pull the repo and view your changes from a different computer or in a different directory.
3. Write two versions of a program that takes as input a 2-D array of integers and increment each element by 1. The first version accesses the array row-wise. The second version accesses the array column-wise. Which version is faster? Why?
4. Experiment with the QEMU emulator (https://openstax.org/r/76QEMUEmulat) to emulate the latest Intel processor and run a toy kernel using one of the following:

- The Little Book about OS Development (https://openstax.org/r/76LittleBook)
- Operating Systems: From 0 to 1 (https://openstax.org/r/76OS0to1)



5

Hardware Realizations of Algorithms: Computer Systems Design

Figure 5.1 What does the word *computer* really mean? For example, supercomputers (left) and data centers (right) are types of computers. "(credit left: modification of “Columbia Supercomputer—NASA Advanced Supercomputing Facility” by Trower, NASA/ Goddard Space Flight Center, Public Domain; credit right: modification of "2020 Data Center" by Jefferson Lab/Flickr, Public Domain)"

Chapter Outline

5.1 Computer Systems Organization
5.2 Computer Levels of Abstraction
5.3 Machine-Level Information Representation
5.4 Machine-Level Program Representation
5.5 Memory Hierarchy
5.6 Processor Architectures

Introduction

We use the word *computer* a lot, but we may not know a precise definition of it. More often than not, we use it to mean our desktops and laptops. But computers exist in many different forms, like your laptop, smartphone, or tablet. Embedded processors are used to power smart home security systems. When you access your social media accounts on Facebook, Twitter, Instagram, or any website, you are using very big computers hosted by these companies. These huge and powerful computers are clusters of computers hosted in data centers and supercomputers for some applications.

These computers have many things in common, but they also differ in many aspects. In general, computers all have processors, memory, storage, and input/output devices such as keyboards, screens, and speakers. What are these components? And how do they interact with each other to form what we know as a computer? This is what we will explore in this chapter. The difference between the computer inside your watch and the one running the big sites is the number and strength of processors (computer brain), the size of memory and disks (main means of storage), and how these many pieces are connected to each other. But the main concepts are the same.

A computer’s main job, as you may have guessed, is to do computations—a lot of computations. The faster a computer, the more computations it can do per second. All computer programs you use are based on computations, whether a modern immersive graphics intensive game that leverages artificial intelligence (AI),

a text editor, or a web browser. In this chapter, you will learn how computers can do computations in a fast and correct manner using processors, memory, disks, and other related hardware.

A company called TechWorks is taking advantage of leverages such as the latest nanotechnology, processor models known as neuromorphic processors, to enable the creation of the next generation of super society intelligent autonomous solutions (e.g., advanced robotics, autonomous cars and drones, or other autonomous systems). The use of Intel's Kapoho Point 8-chip Loihi 2 board technology allows TechWorks' developers to solve larger problems by stacking large-scale workloads and enabling AI models with up to one billion parameters and solving optimization problems with up to eight million variables.

5.1 Computer Systems Organization

Learning Objectives

By the end of this section, you will be able to:

- Define a computer system
- Explain how information is stored and transferred in a computer system
- Differentiate between high-level and machine-level programs
- Identify the elements of a typical computer system

At its core, a computer system is an electronic device that does computations. These computations appear to the outside world as executing programs. When you play a game, listen to a song, or browse the web, you are instructing your computer to do computations. You may wonder how does the computer function and how do the computations performed such as browsing the web or listening to a song relate to one another?

Let us start with the second part of that question: the relationship between computation and executing a program. A song that has been digitized for storage is actually a bunch of numbers that the computer reads; it produces sound based on those numbers. The computer must calculate (i.e., compute) the frequency and volume of the sound based on the numbers read. Another example is when you open a browser and type the address of a website. The computer reads what you typed on the keyboard and leverages network capabilities to translate it to a long number called an Internet Protocol (IP) address (Figure 5.2). The IP address consists of several digits, similar to your phone number, that allow the data to reach your computer over the Internet and for other computers over the Internet to recognize your computer. Your computer passes that IP address over the Internet to another big computer asking for the content of the required website, receives the content from that big computer, and executes browser software that translates it to content such as pictures, sounds, and animations on the screen. All these steps involve computations. But how does the computer do all this? To answer this question, we first need to know the components of a computer system.

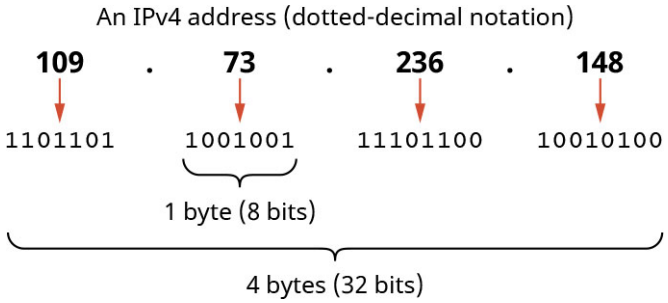


Figure 5.2 IP addresses use a standardized dotted-decimal notation that corresponds to a string of bits. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Computer Systems

A computer system consists of two major parts: hardware and software. The hardware includes the physical components of the computer system, such as hard drives, motherboards, and processor chips. The software consists of the programs you execute, such as media players or web browsers, and the data needed for these

programs to function. For example, the media player is a program, and the songs that you listen to using the media player are data. So, the hardware executes programs that are fed with data. Computers come in different shapes, from the small ones in your smartwatch, for example, to big supercomputers. Different computers may have different components besides the components that exist in all computer shapes. Bigger computers, datacenters, and supercomputers need extra components for getting data from multiple sources, more sophisticated parts to connect to the Internet, and cooling equipment. Mainframes that were created a few decades ago are big computers used for a different purpose. You don't need those in your smartphone.

Data Storage Inside a Computer

All programs and their corresponding data are stored inside a computer or downloaded from the Internet. So how are these programs and data stored? And how does a computer understand and execute them? Computers do not understand English or other natural languages; they can only relate to strings of 1s and 0s and therefore all the programs and data must be stored inside the computer as a sequence of 1s and 0s. A 1 or a 0 is called a binary digit or **bit**. Every 8 bits unit is called a **byte**. This is pretty much how computers that depend on electricity work. Other types, still not in the mainstream market but in design and testing stages, such as quantum computers, do not use bits but use something else called quantum bits or qubits.

If we say that a text editor program takes 1 megabyte of storage, it means that the editor is stored inside the computer, more specifically in the **disk** of your computer, which is a storage mechanism for data, as a sequence of almost 1 million bytes (1 megabyte is a bit larger than 1 million bytes). We call a program stored inside a computer an **executable**. Since programs and data are stored as a sequence of bits or bytes, what is the difference between a program and the data?

THINK IT THROUGH

Why Learn About Computer Systems?

Whether you want to be a software developer, a programmer, or a hardware designer, or even to efficiently use any computer system, you need a minimum knowledge of computer organization. You need to know what the different pieces in the computer systems are, how they interact, and how fast each piece is. Knowing the internal workings of computers helps you write more efficient software, design computers of different sizes (like the one in your watch, the one in your phone, or big supercomputers), and make the best decisions when buying a computer.

Application Programs and Executables

The programs we use in our computer systems, called applications, have been designed and written by software developers using computer languages, also called high-level languages (HLLs). HLLs have been designed to make the interaction between software developers and computers easier. However, computers do not understand HLLs. Computers understand only 1s and 0s. So, there is a set of programs designed to translate the programs written in an HLL into the 1s and 0s that computers can relate to. The programs written by software developers are translated into a set of instructions such as, "Add number 1 to number 2 and save the result as number 3." These instructions are stored as a series of 1s and 0s. Each group of those 1s and 0s represents a single instruction. All the instructions, in their representation as 1s and 0s, are stored in a file called an executable file on the disk.

When you click an icon or type a command, the operating system, software whose job is to manage the interaction between the user of the computer, the hardware, and the programs, loads the executable from storage into the computer memory. At this point, the computer is ready to start executing the program. Figure 5.3 shows the steps for writing programs, in various HLLs by software developers, and then translating these programs to an executable (the 1s and 0s representation of the program) using a special software toolchain

(Figure 5.3). This special software that translates HLLs to executables includes tools called compilers, assemblers, and linkers. We will not discuss these tools here, (refer to 5.2 Computer Levels of Abstraction and 5.4 Machine-Level Program Representation), but we now know what they do. Here, we will look at how the hardware executes instructions.

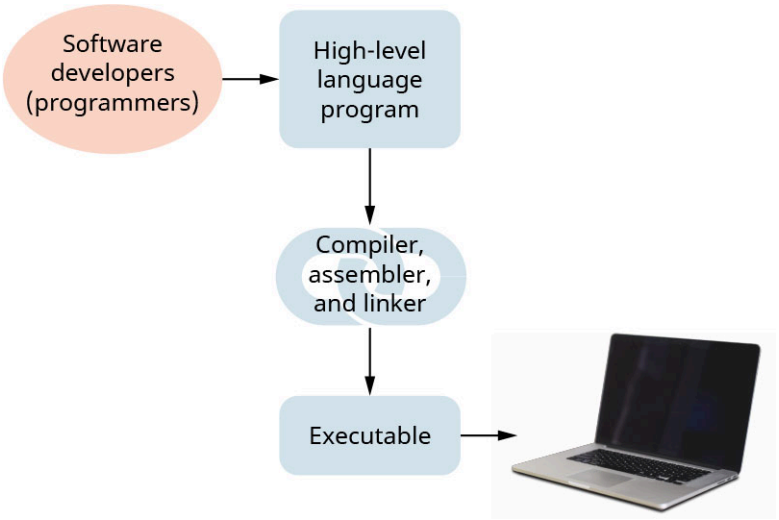


Figure 5.3 Programs developed in high-level languages (HLLs) such as C++, Java, or Python are then translated into executables that a computer can run. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Hardware Organization of Systems

Once the executable is in the memory, the hardware needs to do several things in order to execute it. Recall that an executable is a group of instructions in the form of 1s and 0s. The hardware first needs to fetch an instruction from memory by bringing it from the memory to inside the CPU, which is the brain of the computer that performs and executes the instructions of a program. Once the instruction is fetched from memory and stored in temporary storage inside the CPU, called a register, the CPU decodes this instruction; that is, it deciphers or understands the meaning of the several 1s and 0s that constitute the instruction. For example, the software program may contain an instruction such as "add this number x to the number y and put the result in z." Since computers do not understand English, this instruction is stored inside the computer in the form of 1s and 0s. The CPU has to read those 1s and 0s, understand what they mean (i.e., decode it), and then execute it.

After the instruction has been decoded, the CPU instructs the **arithmetic logic unit (ALU)** to execute it. The ALU is the piece of hardware inside the CPU that performs computations and logical operations such as comparisons. Once the instruction is executed, the result is saved into a register or sent back to the memory. The CPU is now ready to fetch the next instruction. Figure 5.4 shows a computer system's hardware. Inside the CPU we can see the ALU, a group of registers (called a **register file**), and a piece of hardware, called the memory controller, that helps the CPU talk to the memory. Inside the CPU we also find fast storage, which is faster than the memory but slower than the register, called cache memory. We will discuss cache memory in 5.5 Memory Hierarchy.

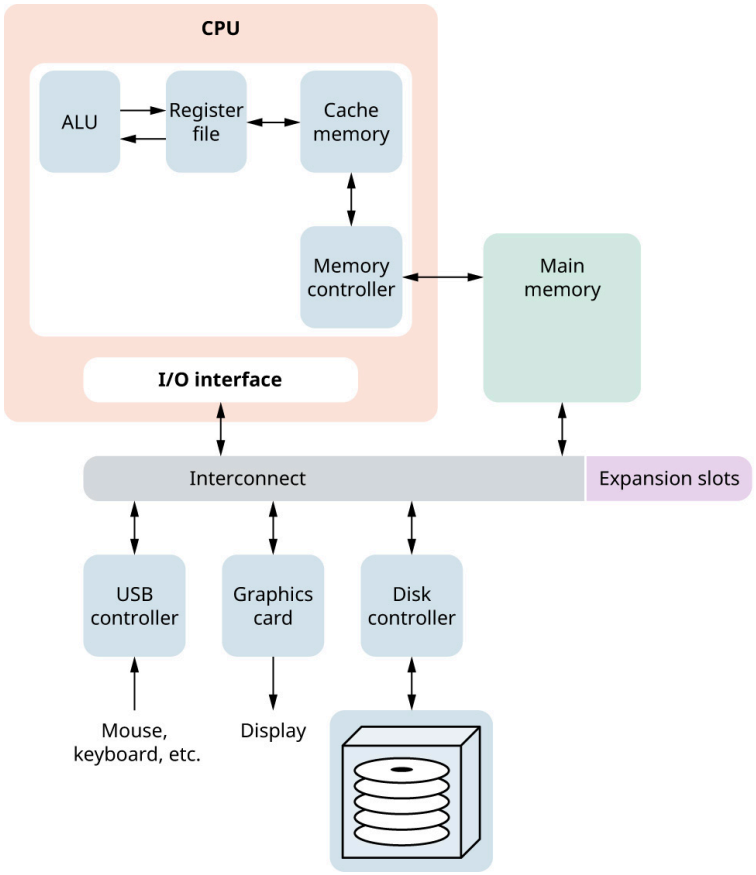


Figure 5.4 Any computer system consists of several components that, together, help the computer do its job of executing programs. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

One final piece of hardware inside the CPU is the I/O interface. The **input/output (I/O)** interface helps the CPU talk to the other I/O devices such as a keyboard or mouse. All the pieces—CPU, memory, graphics card—are connected via a collection of wires referred to as a bus. The purpose of a bus is to transfer data between the various pieces in the computer. Buses can differ in speed, which affects how fast the data are transferred. Figure 5.4 uses the generic word *interconnect* to designate buses. We can add more devices to the computer system such as a different display through expansion slots.

LINK TO LEARNING

Read this concise history of computers starting from the nineteenth century (https://openstax.org/r/76CompHistory19) to see how computers have changed and been developed over time.

Input/Output Devices

The job of input/output (I/O) devices is to take input from a user (i.e., typing on a keyboard or speaking into a microphone), transform it into 1s and 0s, and store this information in memory. It also takes some 1s and 0s, generated by some type of software program, from memory and translates them to an output format such as a picture on the screen or a sound from a speaker. Distinct devices have different speeds and varying ways of transforming the input and output to/from 1s and 0s.

Any I/O device, your keyboard for example, connects to the computer system using an interface such as a USB port that we see in all computers. The USB controller shown in Figure 5.4 is the piece of hardware that manages the USB port and allows it to detect that a device has been connected. Another important piece of hardware is the main memory.

Main Memory

For the CPU to execute programs and process data stored locally, it needs to obtain them from the computer's disk. But a disk is very slow. So, the CPU brings what it needs from the disk and temporarily stores it in faster storage referred to as main memory or random access memory (RAM). When you buy a laptop, one of the specifications is the amount of memory it has, such as 16 GB of RAM or 32 GB of RAM. The main memory is much faster than the disk and connected to the CPU with a faster interconnect, as shown in Figure 5.4. When you click an icon to start a program (e.g., your web browser), the program and its needed data are copied from the disk to the main memory. Then, the CPU reads the data and instructions from the memory into the CPU's registers and the ALU starts executing it.

Processor

The **processor**, also called the microprocessor, is another name for the CPU. It is the brain of the computer system, and its main job is to execute programs. As we discussed earlier, the processor fetches instructions from the memory, understands what each instruction wants to do, gets the data needed to execute the instruction, executes the instruction, and then stores the result in a register or in the main memory. It keeps doing so until the program ends. There have been huge advances in processor design leading to faster and more powerful computers. The processor in your smartphone today is more powerful than a big supercomputer was a few decades ago.

5.2 Computer Levels of Abstraction

Learning Objectives

By the end of this section, you will be able to:

- Describe abstraction levels from the highest to the lowest
- Explain application programs abstractions in relation to HLLs and instruction set architectures
- Discuss processor abstractions and how microarchitecture supports them
- Identify the role of the operating system within abstraction
- Discuss examples of new disruptive computer systems

When you look around, you see that complex systems can be viewed as layers of abstractions. The removal of unimportant elements of a program or computer code that distract from its process is called **abstraction**. This way of looking at complex systems makes it easier to understand them. For example, cars are very complicated inventions. At the highest level of abstraction, we look at a car as a set of devices used to operate it such as a steering wheel, brake and accelerator pedals, and so on. If we go to a lower level, we see devices that power the car such as its engine, gears, and spark plugs. If we take these parts and look at how they are designed, then we are at an even lower level where we see metals, plastics, and other materials. The same approach can be applied to computers to understand how they work. We can see computers as several layers of abstractions, as shown in Figure 5.5. For the remaining part of this section, we start from the highest level and then work through each abstraction layer to illustrate how it is used as a building block by the layer before it.

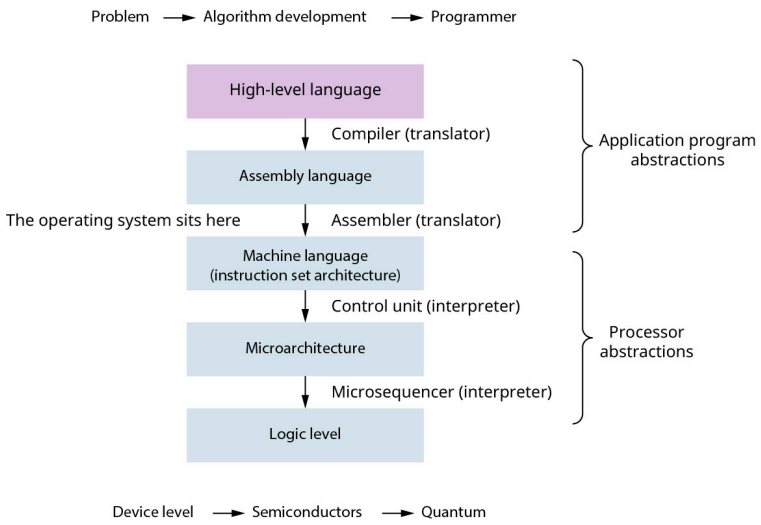


Figure 5.5 A computer system can be viewed at several levels of abstraction, each one layered on top of the other. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Computers are just a tool used to solve a problem. You may use computers to play games or listen to music, and in these cases, the problems that the computer is trying to solve are associated with programs that you use for entertainment purposes.

The top line in Figure 5.5 starts with the problem; we must have a very precise definition of the problem we are trying to solve with a computer. So, the first step in solving a problem with a computer is to know exactly, and with no vagueness, what we are trying to solve. You cannot make a computer solve a problem unless there is a defined and repeatable set of instructions to solve the problem. You may wonder then why it is necessary to use a computer in the first place if you can solve the problem yourself. Well, computers do not get bored, are precise, and can deal with very large problems. This is why, the next step after problem definition is to lay out the steps for solving the problem. This solution layout is called an algorithm. The algorithm is written in free format; that is, it can be steps written as a bulleted list, it can be a flowchart, or it can be a series of mathematical equations. Regardless of the format you choose for writing the algorithm, the algorithm needs to have a key set of characteristics.

The first characteristic is that an algorithm must be unambiguous. Each step of the algorithm must be very well defined and precise. The second characteristic is that the algorithm must be deterministic to be reproducible and repeatable so that the same set of inputs produce the same output. The third characteristic is that the algorithm, when implemented on a computer, must consume a reasonable amount of time and storage based on the problem needs. For example, an algorithm can be finite and precise, but if it requires 100 years to generate a result, it is clearly useless. For instance, an algorithm that counts the number of even numbers is not finite because we have an infinite number of even numbers.

Assuming we have an algorithm, we then need to prepare it for execution on the computer. First, we must prepare the input before it is ready to be consumed by the computer, so we give the algorithm to a programmer whose job it is to read the algorithm, understand it, and then write a program in a known computer language such as C/C++ or Python. The program tells the computer what to do, but in a formal way rather than a freeform way as an algorithm. At that point, we move to another level of abstraction.

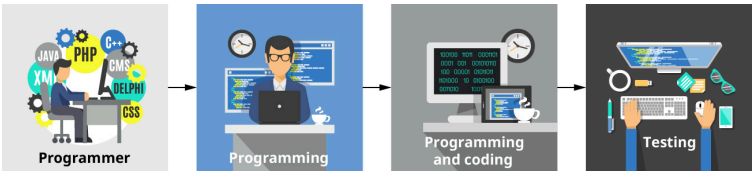


Figure 5.6 The resulting program is consumable by a computer during and after testing. (credit: “Programmer Flat Set” by Macrovector_official/FreePik, CC BY 2.0)

Application Programs Abstractions

Next, the programmer writes a program. The main difference between an algorithm and a program is that the former is written by an algorithm designer and the latter is written by a programmer so that the program can be executed by a machine. These programs are called application programs, or simply, programs, and there are billions of them in existence today. Once the programmer finishes writing the program, there are two more steps before it can be executed by a computer.

High-Level Programming Language

The program generated in the previous step is written in a programming language. There are many programming languages in the world currently. A **high-level language (HLL)** is the most evolved method by which a human can direct a computer on how to perform tasks and applications. The phrase *high-level* means that it is closer to a natural language rather than a machine level language such as strings of 1s and 0s. That is, HLL is more user-friendly, and it is made to make the life of the programmer easier regardless of the hardware or the machine. If you look at the code of a program, you find that it is still in English, yet a restricted version of English with very specific keywords and formats to remove the ambiguity that usually exists in natural human language.

Even though HLLs rely on restrictive versions of English, they still use English at a high level. The machine does not understand English and needs a low-level language; therefore, we need yet another next step: assembly language.

THINK IT THROUGH

One Hundred or One?

Since HLLs aim to make the life of the programmers easy, why do we have many HLLs? Why not one language that all programmers use?

Programming languages are typically designed to help create readable programs. However, some languages are designed with specific applications in mind. That is, some programming languages are easier to use for designing games, while other languages are meant to address mathematical problems or artificial intelligence. However, we can write any program in any language. But our task will be easier if we use the language that is designed with specific applications in mind.

Assembly Language

When you look at most mainstream programming languages, you find constructs such as functions, methods, subroutines, and objects. These concepts were invented to help human beings (the programmers) write their programs after understanding the algorithm. High-level languages make life easier for programmers. By using functions, objects, and other constructs, programmers can write programs faster and make them understandable and reusable for others. Computers, on the other hand, need specific instructions to perform tasks (for example, add this number to that other number and store the result in that place). Writing programs in this way is not easy for programmers because it is error prone, takes a lot of time to write correctly, and is not portable from one computer to the next. So, how do we deal with these conflicting requirements for programmers and computers? The answer is compilers.

A compiler is a piece of software that takes a program written in a given HLL and generates another program that does the same task as the initial program but is written in a language that is much friendlier to the computer called assembly language. Assembly language is then translated into machine language code so that the program can be executed (Figure 5.7).

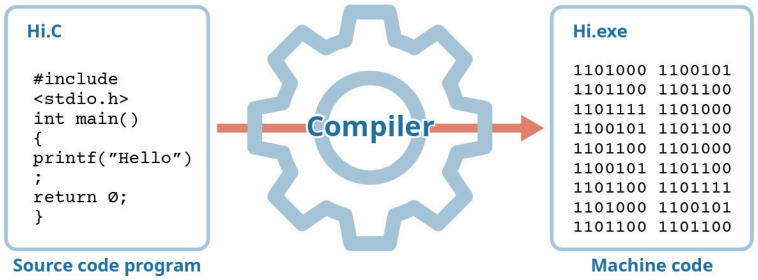


Figure 5.7 A compiler takes code written in an HLL and converts it to "computer-friendly" and simple code called assembly language. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

One low-level language designed to be computer friendly rather than human friendly is called **assembly language**. It does not use all the constructs found in HLLs such as objects or sophisticated data structure, which leads to two challenges.

The first challenge is to manage the output of the compiler, which is an assembly language program that you can open and read. It is basically a text file, so it is written in English. Remember, computers do not understand English; they only relate to 1s and 0s. To deal with this challenge we use yet another program, shown in Figure 5.5, called an assembler (note that compilers may invoke assemblers directly). The **assembler** takes, as input, the assembly program generated by the compiler, and as output, a file that contains the equivalent of that assembly program in terms of 1s and 0s. This generated file is no longer in English, and you cannot open it with your favorite text editor. It is called a machine language file. This is the file that a computer understands. Figure 5.8 shows an example of a program written in an HLL (actually, a middle-level language, which is the C programming language), which is then translated to x86 assembly language and then to binary (refer to Chapter 4 Linguistic Realization of Algorithms: Low-Level Programming Languages).

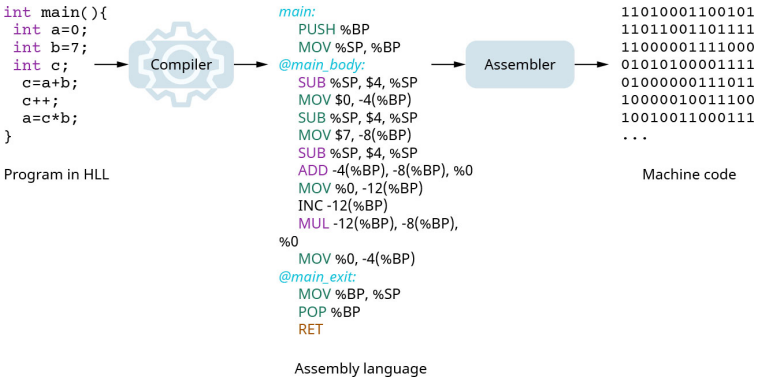


Figure 5.8 Programmers write in programs using HLLs but computers execute binary code, so we need to perform a translation. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The second challenge relates to managing the assembly language itself. If you look at the assembly language program, you find that it consists of instructions such as add, divide, and jump. But are these instructions recognized by all the processors (i.e., the main part of the computer that executes the program) in the world? If you give these instructions to a processor such as Intel, AMD, ARM, Qualcomm, or IBM Power, will they all recognize these instructions? The answer depends on a new concept referred to as the instruction set architecture.

Instruction Set Architecture

The instruction set architecture (ISA) is the set of instructions recognized by each processor family. For example, both Intel and AMD processors use the same ISA, called x86-64, which is different from the ISA recognized by ARM or IBM. This makes us revisit the concept of compilers. A compiler takes as input a program written in an HLL, and we have a compiler for each HLL. The output of the compiler is an assembly program in a specific ISA, and we have a compiler for each different ISA (e.g., ISA 1 and ISA 2). So, if we have programs written in three HLLs and we need to generate assembly for processors of two different families,

then we need six compilers as shown in Table 5.1.

Compiler#	Input to the Compiler	Output from the Compiler
1	HLL 1	ISA 1
2	HLL 2	ISA 2
3	HLL 3	ISA 1
4	HLL 1	ISA 2
5	HLL 2	ISA 1
6	HLL 3	ISA 2

Table 5.1 Compilers and Their I/O

As we saw earlier, the output of the compiler is the input to the assembler so we need assemblers for each ISA in existence to generate a machine language file (e.g., .exe file) that the processor can execute.

Processor Abstractions

As we cross the layer of ISA in Figure 5.5, we cross the boundary between software and hardware. Before we discuss hardware, we need to understand two words: translator and interpreter. Both words mean "translating from language 1 to language 2" regardless of what those languages are. The main difference is the process by which translation is done. A translator takes a whole program in language 1 and generates another program in language 2. For example, the compiler takes an HLL as input (language 1) and generates the corresponding assembly language program (language 2). The interpreter takes one line (or command) in the program in language 1 and generates one (or more) instructions in language 2. Python is a popular example of an interpreted language.

Understanding the hardware level allows us to see how computers execute programs. The main part of the computer hardware that does the execution is called the processor. The processor takes one instruction from the machine language file, executes it, and writes the results back in a designated place. Then, it fetches the following instruction and does the same. It keeps doing this until the program ends or an error occurs. This is an oversimplification, but it conveys the general idea. As you can see, it takes one instruction at a time, which is why the vertical arrow coming out of the machine language box in Figure 5.5 shows the word *interpreter*. But how does the processor do its job of fetching an instruction and executing it? To answer this question, we need to look at the main components of a processor.

LINK TO LEARNING

The transistor is the building block of the hardware of any computer. A transistor is merely an on/off switch; when it is on, it lets the electrical current pass. When it is off, it blocks the electrical current. This is very similar to the light switch you find in your room. But with these transistors, we can do more: with transistors, we build computers. Read this article to learn more about transistors (https://openstax.org/r/76Transistors) and how they work.

Microarchitecture

The architecture (i.e., design) of the microprocessor (i.e., the processor) is called **microarchitecture** (Figure 5.9). Its main job is to design the different components of the processor and decide how to connect them so that the processor can do its job. For example, one important piece inside the processor is the part that fetches the next instruction of the program. Another crucial part, called the control unit, is responsible for decoding, or understanding, what this instruction wants to do and then tells the other components of the processor to execute this instruction. So, the control unit's job is to take an instruction as input and generate signals to control the rest of the processor to make it execute the instructions. Other parts of the processor include the execution units that do the actual computations such as divide, multiply, add, and subtract.

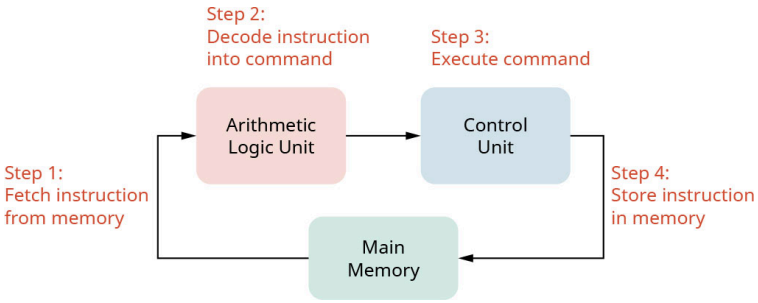


Figure 5.9 Although it may just take seconds to execute, there are several steps included in a microarchitecture process to deliver results for the user. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

To summarize, inside the processor you find those different pieces that fetch instructions, decode them, and execute them. But what is inside each one of these black boxes?

Digital Logic Abstraction

The main building block that forms the processor is called a **logic gate**. There are very few logic gate types: AND, OR, NOT, NAND, NOR, XOR, and XNOR. Using these gates, and most of the time only a subset of them, you can design all the pieces that form the processor discussed earlier. But what is inside these gates? How are they built?

The Lowest Level of Abstraction

The main building block of all logic gates, and hence of all processors, is the transistor. This is shown at the bottom of Figure 5.5 as device level. Though you may have heard the word *transistor* before, you may not know exactly what it does. Simply speaking, a transistor is an on and off switch. This is very similar to the light switch in your house that can turn a light on or off. A **transistor** lets an electrical current pass, the ON state, or can block the electrical current, the OFF state. Transistors are turned ON/OFF based on the voltage input to the transistor. If the voltage is higher than a threshold, the transistor is in the ON state. Otherwise, it is in the OFF state. There are only two states: ON/OFF, which correspond to 1 and 0. This is why computers understand only 1s and 0s. By interconnecting several transistors in some way, we build an AND gate. If we connect them in a different way, we build an OR gate, and so on.

If we try to see how transistors are built and work, then we move to the semiconductor level. At this level we use a special material such as silicon and a special, and very expensive, process to turn it into a working transistor. A single processor contains billions of transistors. How can a material like silicon make a transistor? This takes us to the level of atoms and quantum physics.

The Role of the Operating System

Now that we have rundown the problem definition to quantum physics, you may wonder where the operating system (OS) (i.e., OS X, Windows, Linux) fits in this bigger scheme. The OS is similar to any application program in the sense that it has to be written in an HLL, typically C/C++, and passed through the compiler and assembler to generate machine code. However, the OS differs from traditional application programs in that it

has more privileges in the computer system.

The **operating system (OS)**, shown in Figure 5.10 is the only piece of software that can directly access the hardware. Any other program that needs to access the hardware, such as printing something, must talk to the OS, and the OS achieves the requested task on behalf of the program. The reason all computers are designed this way is to increase security (only one program deals with the hardware so other programs cannot affect the machine) and reliability (a program cannot affect a piece of hardware, which would then affect other programs). In order for the OS to do its job efficiently, it stores the data and programs on disk in an organized way using a file system. The file system helps organize files so that it is easier to find them when needed.

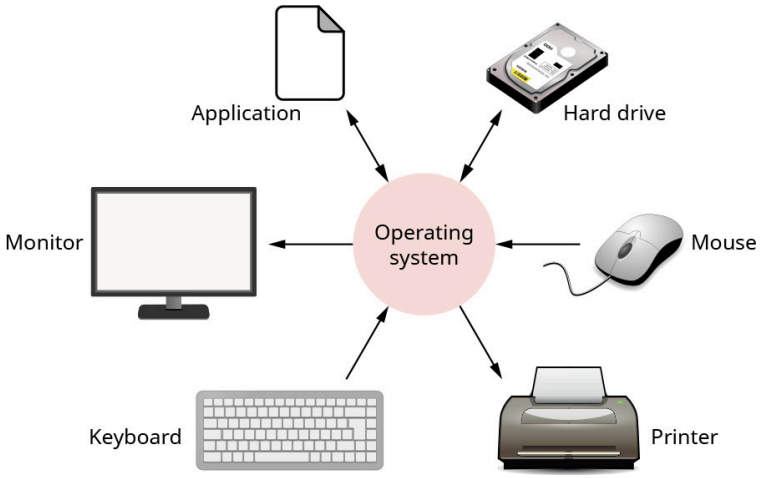


Figure 5.10 The operating system functions as a manager that connects the hardware in the computer to the software. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

A file is a generic name for any entity we want to store in the computer. For example, any program that you use consists of one or more files. Each song you listen to is a file. Any image or video you watch is a file, and so on. The process in which the OS manages which program to use with what part of the hardware at any given time is called **scheduling**. That is, the OS decides when your web browser must use the processor and when your media player uses the screen or speaker. There is a generic name for all programs running on your computer: process. So, if you are listening to music while browsing the web then you are using two processes: your media player and your browser. Therefore, part of the job of the OS is process scheduling.

The technique that the OS uses to isolate different programs from each other so that they do not overwrite each other's data or corrupt each other's files is called **virtual memory**. Additionally, the OS can leverage several computers referred to as virtual machines together to act as one big computer and to serve several uses at the same time. Users may think they have control of the whole machine, even though the reality is not so. This is why this technology used by the OS is called virtual machine.

There are many OSs in the world but the most famous are Windows from Microsoft, macOS from Apple, and Linux.

New Disruptive Computer System Abstractions

Almost all computers in the world are designed in the way we have learned so far and involve very similar levels of abstraction. However, there are very futuristic designs that scientists are tinkering with today that differ from the traditional transistors. Scientists are trying, for example, to build computers using DNA. We have DNA computing, and we have a prototype for DNA storage, too.

We have several prototypes from various companies of quantum computing where instead of using bits, 1 and 0, the machines use quantum bits (qubits) that take a value between 1 and 0. We must not forget that traditional computers in general operate in binary state (i.e., using 0 and 1). To use these computers, we need to build different types of compilers, operating systems, programming languages, and so on. Another form of

a non-traditional computer is a **neuromorphic computer**, which is built to act like a simplified version of the brain. So, it consists of hardware neurons connected together. These computers are not programmed but trained. What will computers look like 100 years from now? We do not know, yet.

5.3 Machine-Level Information Representation

Learning Objectives

By the end of this section, you will be able to:

- Interpret binary numbers
- Explain the use of standard character codes to represent letters and symbols
- Define fractional binary numbers and explain how they are used

In this section we look at two very important and widely used types of information: numbers and text. The reason we concentrate on these two types is because they are standardized, and almost all computers store them in the same format, unlike other types of information that have many different formats, some of which are proprietary.

It is important to know that a series of bits does not have a meaning by itself. For example, in order to interpret or translate the binary number 0011 as a decimal number, you need to know whether it is an unsigned or a signed integer. Therefore, we need to understand how the different types of information are presented in binary.

Integer Numbers Representation

As human beings, we always think in numbers, specifically decimals. Whenever you mention a number, it is usually in decimal form; that is, base-10. But what does base-10 mean? It means the value of the number is calculated by scanning the number from right to left. And as you pass by a digit, multiply it by 10 to the power of the position of that digit. The number at the far right, called the least significant digit, has position 0. The one after is at position 1, and so on. Figure 5.11(a) shows this process.

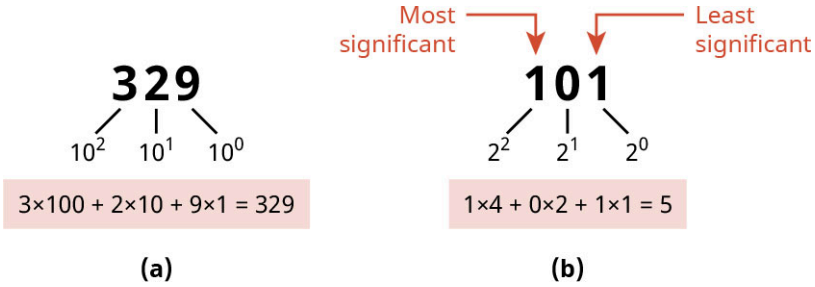


Figure 5.11 The only difference between (a) decimal and (b) binary is the base. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

What about binary numbers? Binary numbers use base-2 and are presented as a series of 1s and 0s. In order to go from binary to the equivalent decimal, we need to differentiate between the ways unsigned integers and signed integers are represented in binary.

LINK TO LEARNING

The best way to get a clear view of how numbers are translated to binary so that computers can deal with them is to see the process in action. Visit this site to convert any number from decimal to binary and vice versa (https://openstax.org/r/76DecimalBinary) and to read more about how to do it manually.

Unsigned Integer Numbers Representation

An **unsigned integer** is a non-negative integer that starts from 0. If you see a number in binary and you are told that this number represents an unsigned number, you can get its decimal equivalent in the same way as base-10 integers but using base-2 instead. Figure 5.11(b) shows the operation of getting the decimal equivalent of the binary number 101 by starting from the right and moving left. The least significant bit has a position 0, the next one has position 1, and so on. As you pass by each element, add $X \times 2^p$, where p is the position of the digit and X is the digit itself (0 or 1), to the total sum.

One important thing to keep in mind is the range of numbers that can be presented by an unsigned number. One bit can present two values only: 1 and 0. Two bits can present four values: 00, 01, 10, and 11 which correspond to 0, 1, 2, and 3. Three bits can present eight values. In general n bits can present 2^n values, which is the range from 0 to $2^n - 1$. As you can see, there are no negative numbers. To be able to present negative numbers, we need to use signed numbers.

Signed Integer Numbers Representation

A **signed integer** is an integer that can be negative or positive. The word *signed* means that they have a sign of + or –. To present negative numbers, designers experimented with several options before picking the de facto choice. The obvious option is to use the most significant bit (i.e., the leftmost bit) as a sign bit where 0 means positive and 1 means negative. The rest of the number is treated with a method called sign-magnitude. Is it a good option? Let us look at the first column of Figure 5.12. It represents the sign-magnitude for a 3-bit number. We see two things.

Sign-Magnitude:	One's Complement	Two's Complement
000 = +0	000 = +0	000 = +0
001 = +1	001 = +1	001 = +1
010 = +2	010 = +2	010 = +2
011 = +3	011 = +3	011 = +3
100 = –0	100 = –3	100 = –4
101 = –1	101 = –2	101 = –3
110 = –2	110 = –1	110 = –2
111 = –3	111 = –0	111 = –1

Figure 5.12 Two's complement is the representation of choice for signed numbers. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

First, there are two presentations of 0 (+0 and –0), which is a waste of data. Second, the numbers are not mirrored around 0. That is, usually, the number after 0 is +1 and the one before 0 is –1. This is not realized here. The importance of this mirror is to make the hardware a bit simpler.

The second method to try is a bit less intuitive. It is called the **one's complement** method of a binary number, which is obtained by flipping each 1 in the original number to 0 and each 0 to 1. For example, 101 has its one's complement as 010. The one's complement method is explained in more detail:

- Check the most significant bit and if it is 1, the number is negative and is represented in its one's complement form. To get the original number, you need to get its one's complement.
- The one's complement of the one's complement is the original number. For example, the one's complement of 101 is 010. The one's complement of 010 is 101.
- Looking at the second column of Figure 5.12, let us take a number such as 110. Its most significant bit is 1 so the number is negative and in its one's complement form. To know its original value, we need to get its one's complement again. The one's complement of 110 is 001 which corresponds to decimal number 1, so the result is –1.
- If the most significant bit is 0, then the number is positive, and to get its decimal equivalent, we treat it as if it is unsigned. With the number 010, the most significant bit is 0, so the number is positive and the decimal equivalent of 010 is 2. Therefore, the result is +2.

Is the one's complement a good method? There is a mirroring effect around the +3/–3. But we still have the two presentations of 0. The third method is called the two's complement, which is the least intuitive method. It is important to note that what is good for machines is not intuitive or easy for human beings!

The **two's complement** of a binary number is simply the one's complement with 1 added to the result. For example, to get the one's complement of 011, we do it in two steps. First, we get the one's complement: 011 is 100. Second, we add 1 to the result: 100 + 1 = 101, and 101 is the two's complement of 011. Now that we know the definition of two's complement, let us see how we can get the decimal equivalent of a binary number. We perform a very similar technique as the one's complement but use the two's complement.

Look at the most significant bit. If it is 0, the number is positive, and the decimal equivalent can be calculated as if the number is unsigned. For example, 011 is positive and the decimal equivalent is +3. If the most significant bit is 1, then the number is negative, and it is written in its two's complement form. Also, the two's complement of the two's complement brings the original number so 101 has the most significant bit of 1. The number is negative and written in its two's complement form.

The two's complement of 101 is 011, which corresponds to 3. The number 101 represents –3, as you can see in the third column of Figure 5.12. A close look at the figure shows that even though the two's complement is not the most intuitive method, it is the most efficient. Since we have only one presentation of 0, we can see from the figure that with three bits, the two's complement method can present up to –4, which we did not find in the other two methods.

Another important issue with the two's complement is that the hardware implementation of it is very efficient because we can use the same piece of hardware for both addition and subtraction and for both signed and unsigned integers. Therefore, a two's complement presentation of a signed number is the standard presentation in all computer systems.

With n bits, the range of numbers that can be presented is $[-2^{n-1}, +2^{n-1} - 1]$. You can see that this range contains 2^n different numbers, exactly like n-bit unsigned integer numbers. The only difference is that in the signed range, half the range is negative, while in an unsigned integer, the whole range is positive.

Lastly, let us look at how additions and subtractions are done. The process is similar to a decimal by starting from the right and a possible carry propagates to the left. Additionally, the subtraction is nothing but an addition; that is, A – B is the same as A + two's complement of B. When we add two numbers in decimal, it is straightforward. With binary, it is also straightforward. Just remember the information in Table 5.2; op1 and op2 are the two inputs to the addition operation.

Op1	Op2	Carry	Sum
0	0	**0**	**0**
0	1	**0**	**1**
1	0	**0**	**1**
1	1	**1**	**1**

Table 5.2 Op1 and Op2 Inputs to the Addition Operation

Let us apply this to numbers longer than one bit. In Figure 5.13, the left side of the example is a traditional decimal addition as done on paper by hand. The right side of the example shows what computers do. Computers, as we already know by now, use only 1s and 0s. So, if they add 1 to 1, for example, the result is not

2 because computers do not know 2. The result of 1 + 1 is 10, which is the binary representation of 2. From this 10, the 0 is the result and 1 is used as carry.



Figure 5.13 This illustrates the difference between traditional addition versus computer computation. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

THINK IT THROUGH

Why Integers?

A real number, represented on a computer as a single precision floating point, takes four bytes. An integer also takes four bytes. Yet, the range of numbers that a floating point can represent is much larger than the integer.

- Why do we use an integer in the first place?

Computers are much slower dealing with floating points because the hardware for a floating point is much more complicated than the one for an integer, and it is better to make things faster by performing computations with an integer. We also only use floating points if it is an absolute need.

Character Representation

Remember that what is stored inside the computer is not only integers. If you look at a keyboard you see a bunch of characters, digits, and symbols and when you press a key, that symbol appears on the screen. But how do computers recognize these characters and deal with them?

Each printable and non-printable character on the keyboard has a unit code or unique binary number. These codes are known as **American Standard Code for Information Interchange (ASCII)**. Capital letters have different codes from lowercase letters. For example, “A” has a different code than “a.” Decimal digits also have their own code. The code is 7-bit length, which covers 128 characters. This encoding has been extended to 8 bits to encompass non-printable characters as well (i.e., characters on the keyboard that cannot be printed on the screen: can you guess them?). The reason to have standardized codes across all machines is to allow computers of different brands, specifications, and sizes to work together. This code was approved in 1963, before personal computers existed, and then revised in 1965, 1967, 1968, 1977, and 1986. Now, all computers use this code to represent the characters.

You may realize that 128 characters, or even 256 ones, cannot include all written languages which is why there are new encoding standards such as Universal Coded Character Set (UCS) and Unicode that encompass all written languages and incorporate ASCII as the first 128 codes, which is backward compatible, allowing for the integration with older legacy systems.

Real Numbers (Floating Points)

There is no computer worth its salt that cannot present and manipulate real numbers. A real number, such as 3.14, is also called a **floating point number** because there is a decimal point somewhere in the middle. In most HLLs, a single precision floating point number requires four bytes of storage such as integers (signed and unsigned).

Let us start with an easy question: if given a binary floating-point number, for example 1010.010101, how do

we get the decimal equivalent? Figure 5.14 illustrates how fractional binary numbers are represented. We use the same technique as getting the decimal equivalent of an unsigned integer. The only difference is that the digits at the right, after the floating point, have negative ranks starting from –1. If we have a number such as 11.111, the rank of the first digit after the floating point is –1, the following one is –2, and the leftmost one is –3. The digits at the left of the floating point have the usual rank that starts from 0. Therefore, the decimal equivalent of $11.111 = 2^1 + 2^0 + 2^{-1} + 2^{-2} + 2^{-3} = 2 + 1 + 0.5 + 0.25 + 0.125 = 3.875$.

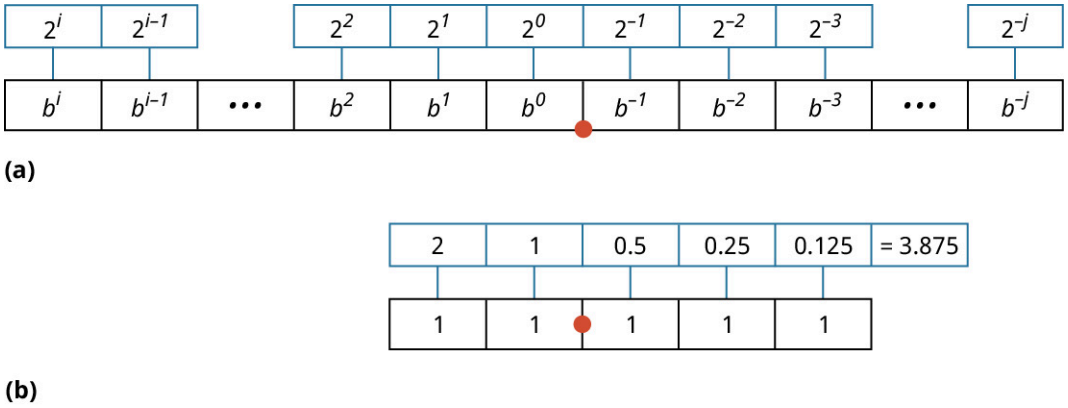


Figure 5.14 Floating point uses the same base 2, but after the decimal point the exponent is negative. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

The best way to get a clear view of how numbers are translated to binary so that computers can deal with them is to see the process in action. Visit this site to convert any number from floating point to binary and vice versa (https://openstax.org/r/76FloatPoBinary) and to read more about how to do it manually.

But how do we store numbers like 11.111 inside the computer memory or register? The hard part is the position of the floating point itself because it is not known beforehand and can change during computations, which makes it very hard to decide how many bits to reserve for the digits at the right of the floating point and the digits at the left of the floating points. If we try to make it a fixed number, say 16 bits and 16 bits, we do not get good precision. Going back to the mathematical representation of decimal floating point numbers, you may recall that we can move the floating point to the left or to the right and keep the final value unchanged by multiplying by 10 to some power. For example, 1.875 is the same as 18.75×10^{-1}, which is the same as 1875×10^{-3}, which is the same as 0.1875×10, and so on. In binary, we can do the same by multiplying by 2 to the power of something. The number 11.111 is the same as 111.11×2^{-1} and so on. In fact, we can express any floating point binary number in the form $1.xxxx \times 2^y$. Except for special cases, such as 0, expressing any binary number in this format requires storing three pieces of information:

- The sign bit specifying if the whole number is positive (sign bit of 0) or negative (sign bit of 1)
- The exponent (the *y* in $1.xxxx \times 2^y$)
- The fraction (the xxxx in $1.xxxx \times 2^y$)

Note we don't need to store the "1." because we know that it exists except for some special cases. And this is why one of the names of the floating point format is "the hidden 1 technique." If we are talking about single precision floating point, it takes 4 bytes (32 bits) and the format is shown in Figure 5.15. One bit is needed for the sign, 8 bits for the exponent, and the rest (32 bits) for the fraction. This format is called **IEEE 754** format, developed by the Institute of Electrical and Electronics Engineers (IEEE). It is the standard format used by almost all computers that support floating points with very few exceptions.

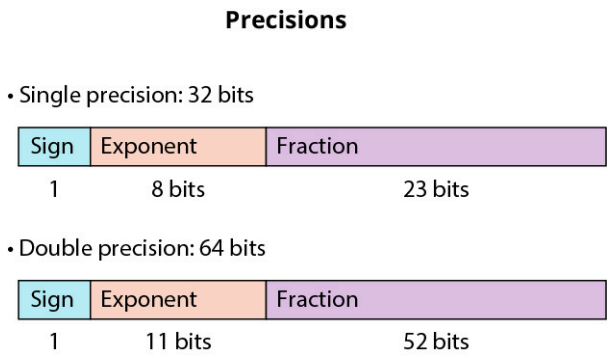


Figure 5.15 IEEE 754 is the standard of choice in most machines to present floating points. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

There is one piece of complexity regarding the exponent—the sign bit is responsible for the sign of the whole number. But, what about the exponent? We need to be able to have positive and negative exponents. If we use the two's complement format for the exponent, the whole floating point format is overly complicated. The hardware that deals with floating point operations is very complicated and much slower than the one dealing with integers, and we do not want to make it even more complicated. The solution is to shift the range of the numbers presented by the 8 bits of the exponent. What does that mean? With 8 bits, we can present numbers from 0 to 256. What if we subtract 127 from each number? That is, you read the 8 bits, get the decimal equivalent (same way as an unsigned integer), then subtract 127. The result will be the range that can be presented is now from –127 to +128. You get the idea.

Another name for the IEEE 754 format is "excess 127." Note that 127 is for the single precision. For double precision that takes 64 bits, we subtract 1023, which is roughly half the range. The double precision is shown at the bottom of Figure 5.15. Let us see an example. What is the decimal equivalent of 1011101101011000...0?

- First, let us divide it into its three main components: sign, exponent, and fraction. This makes it: **1 01110110 10110000...0**.
- Sign bit is 1, so the whole number is negative.
- Exponent = 01110110. Its decimal equivalent is 118. We subtract 127. This makes the exponent: 118 – 127 = –9.
- The rest is the fraction, but we need to add the hidden one, so 1011000...0 becomes 1.1011, which has a decimal equivalent of $1 + 2^{-1} + 2^{-3} + 2^{-4} = 1.6875$.
- This makes the whole number = -1.6875×2^{-9}.

Before we finish our discussion about floating points, there is the question of 0. How to present the 0? Even if we make all bits 0, the hidden 1 makes the final value a non-zero one. How can we deal with this problem? The representation approach that we have learned so far is called the normalized encoding of the IEEE 754 format. This is used if the exponent is non-zero and is not 11111111. If the exponent is 0 (i.e., 00000000) we are in denormalized encoding (also called subnormal). When we are in this special case, there are some differences in the translation to decimal:

- The exponent is 1-bias instead of 0-bias. The bias is 127 in single precision and 1023 in double precision.
- There is no hidden 1, so the fraction part is 0.xxxx (the 23 bits in the fraction in single precision) instead of 1.xxxx....

With these exceptions, we cannot present the 0 (but setting all 32 bits to 0) but can present very small numbers.

The case where the exponent is all 1s is called "special values encoding." If the exponent is all 1s and the fraction is all 0s, it represents infinity. If the exponent is all 1s and the fraction is non-zero, this is called NaN (Not a Number) and raises an exception. This happens when there is a bug in your program that does a division by 0 or the square root of –1, for example.

5.4 Machine-Level Program Representation

Learning Objectives

By the end of this section, you will be able to:

- Discuss x86-64 Intel processors and their architectures
- Differentiate between assembly and machine languages
- Explain basic concepts of assembly language and the types of operations

When you write a program in an HLL, there are several steps that need to be performed before the processor can start executing the code. Refer to Figure 4.18 for a high-level view of the process.

Let us assume you write your program in C and your program is spread over several source files for ease of management. As you now know, the first step is to go through the compiler. The compiler is totally oblivious to the fact that the multiple source code files belong to the same program; it just takes each file separately and generates the corresponding assembly language file for each one of them. If the input is three C files, the output of the compiler will be three assembly language files.

The next step is to take these language files and translate them to machine code files, also known as object files or binary files. Here too, the assembler is oblivious to the fact that the input assembly files do not belong to the same program, so it translates them separately. The first tool in this workflow that recognizes that all the files belong to the same program is the linker. The linker takes all the generated object files, looks for needed libraries, and links everything together into one executable file. Linked libraries are needed because it is very unlikely that programmers write self-contained code. You still use I/O, for example, for printing something on the screen, but you have not implemented those functions yourself—or you use mathematical functions someone else implemented. A library linked at this step is called a static library.

At this stage, you have an executable file residing on your disk until you decide to execute it by typing a command, clicking an icon, or even saying a command. At that moment, a part of the operating system, called the **loader**, loads the executable into the memory and arranges its content in a specific way to make it ready for execution by the processor. A **dynamic library** is when more libraries may be linked during execution or while the program is running.

This section takes a closer look at assembly language. As an example of a widely used assembly language ISA, we will look at x86 ISA used by Intel and AMD processors. But before we delve into this, we need to ask a simple question: Why learn assembly language? Will you ever need to write code in assembly language? Most likely not, except in rare cases where you are developing some part of an operating system, a device driver, or any other application that requires very low-level manipulation. However, by looking at the assembly language generated by the compiler for your code, you can find innovative ways to optimize your code, detect hidden bugs, and reason about the performance of your code once you execute it.

Intel Processors and Related Architectures

You may recall that each processor family understands a set of instructions, which is the ISA. The family of processors from Intel and AMD share the same ISA called x86 (x86-64 for the relatively newer version for later processors). This ISA has a long history that dates back to the 1970s. Figure 5.16 gives a quick glimpse at how things evolved. The figure does not show every single processor from Intel but instead focuses on some milestones.

Name	Transistors	MHz
• **8086** (1978) • First 16-bit processor. Basis for IBM PC & DOS • 1MB address space	29K	5–10
• **386** (1985) • First 16-bit processor, referred to as IA32 • Capable of running Unix	275K	16–33
• **Pentium 4** (1978) • First 64-bit processor, referred to as x86-64	125M	2800–3800
• **Core i7** (1978)	731M	2667–3333
• **Xeon E7** (1978)	2.2B	~2400
• **Core i9** (1978)	2.95B	~3900

Figure 5.16 x86 is a CISC ISA with backward compatibility dating back to the 1970s. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

First, as technology evolves and the need for faster processing power arises, we move from 16, to 32, to 64-bit processors. This number relates to the size of the registers (fast storage entities inside the processor), the width of the buses (parallel wires connecting the processor to memory), and the amount of memory the processor can access (for n-bit machines, the processor can access 2^n bytes of memory). The ISA also evolves in parallel to incorporate the larger registers (hence the move from IA32 to x86-64) and the computations with larger numbers. Second, we can see the tremendous increase in transistors in each generation. Having more transistors means more features implemented inside the processor and hence higher performance and potentially richer ISA.

The complex architecture structure that assists in executing operations such as mathematical computations and memory storage is called **complex instruction set computer (CISC)**. This is done by combining many simple instructions into a single complex one. This concept came from something called the semantic gap, which is the difference between the HLL program and its assembly equivalent. It is good for programmers to understand assembly language as this skill will help you code in any language. However, assembly programming is so much different from HLL programming that most programmers have difficulty understanding it. The wider the difference, the wider the semantic gap. To reduce this gap and make assembly language more accessible to programmers, x86 was designed to make its instructions a bit more complicated because statements in HLL are complicated. Complicated means a single assembly instruction can do several things. For example, an instruction like *addw %rax, (%rbx)* means access the memory at a specific address, get the data stored there, add that data to a number, and store that number in a specific place. So, it is accessing the memory, making an addition, and storing the result somewhere. Because the instructions are complicated, this set of these instructions is called CISC.

Complex instructions such as the ones corresponding to a for-loop in HLL were the norm until the 1980s when another viewpoint came into existence that said that complex instructions make the processor slow. Moving into the 1990s, and the appearance of portable devices with their sensitivity to power consumption and battery life, another disadvantage of CISC arose: complex instructions make the processor not only slow, but also power hungry. And, thus, the other viewpoint of simpler instructions called **reduced instruction set computer (RISC)** came to be the norm. Right now, all the processor families in the world are RISC except x86.

LINK TO LEARNING

There has been a debate among companies who are designing hardware as to whether CISC or RISC is better. Read this article chronicling this debate (https://openstax.org/r/76DebCISCvsRISC) from MicrocontrollerTips.

Assembly and Machine Code

In our discussion of Figure 4.18 we saw assembly (output of the compiler) and **object code**, binary code, and machine code, which all designate output of the assembler. Machine code is the binary presentation of the assembly code. In some cases, there are assembly instructions that do not have a counterpart in the machine code called **pseudo-assembly**. For example, there are instructions in assembly that execute a *go to* if a number is less than another number. The only conditions known in machine code are equal and not equal, but not less than, less or equal, and so on. We can see this in an instruction set like MIPS.

The assembler's job is to ensure that the machine code file only contains instructions that are native to the processor; that is, part of the ISA. So, we can think of the machine code as being a subset of the assembly code. You never find an instruction in the machine code file that is not part of the ISA of the processor for which you want to generate the binary. The reason there are pseudo-assembly instructions is to give the compiler a bit more freedom to generate efficient code. Let us assume that you write a program in C and you think about functions calling each other. If you write a program in C++ or Java, you think in terms of objects, methods, inheritance, and so on (refer to Chapter 4 Linguistic Realization of Algorithms: Low-Level Programming Languages). We call this the programmer view of the language. What if you write (or read) assembly code? What do you see? This view is summarized in Figure 5.17.

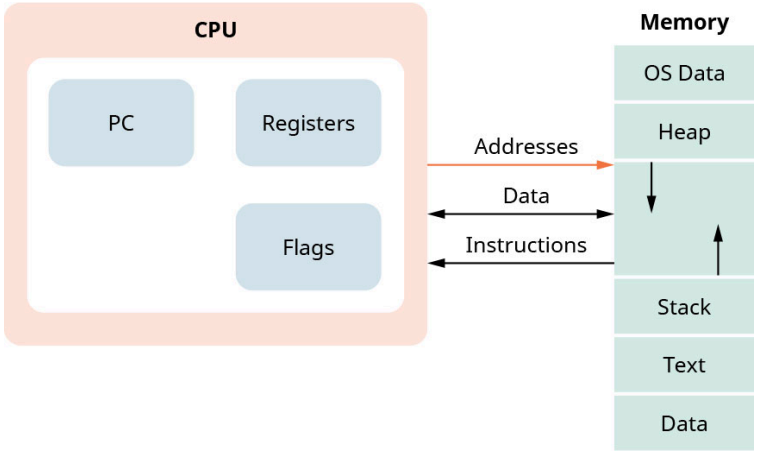


Figure 5.17 The assembly programmer sees a simpler, but more realistic view of the machine than the HLL programmer. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Figure 5.17 shows the following:

- There is a processor (CPU) and memory.
- The CPU and the memory are connected by a **bus**, which is a data pathway. When you access the memory, you may want to get data and must provide an address. Or you may want to write data to memory, so you must provide both the data and the address to which this data will be written. In both cases, the CPU must provide an address, therefore, the address bus is single directional. But the data bus is bidirectional because you can send data to memory or get data from memory.
- Data is not the only thing you need to bring from the memory to the CPU. The main job of the CPU is to execute instructions on data. For example, adding two numbers involves data (the two numbers) and the command for addition (instruction), which is why there is a single directional bus from memory to CPU for getting instructions from the memory.
- The memory holds several things: data, the instructions of the programs (shown as "Text" in Figure 5.17), some data needed by the OS to manage your program, and the resources it needs. The stack and heap are places in the memory to store data depending on the program at hand. The **stack** is used to store local variables (and some other stuff that we will discuss later), the **heap** stores dynamically allocated data, an Data in Figure 5.17 is another area in the memory to store global variables.
- Inside the CPU, there are registers which carry hardware parts that store data, instructions, addresses,

and so on. Each register stores one item. An x86 programmer has access to 16 registers, as we will see shortly. Because the CPU is executing a program, which is a series of instructions, the CPU must keep track of what the next instruction to be executed is.

- Keeping track is the job of a specific register, shown separately from the other registers, called the **program counter (PC)**. The PC is updated after executing each instruction to point to the next instruction to be executed. Also, it is useful to keep some information about the result of the instructions executed, such as whether the result generated by the current instruction is positive, negative, or zero, which is the job of the flags. A **flag** tells a program if a condition has been met.

Registers

A **register** is a memory unit that functions at very high speed. Figure 5.17 shows registers as one black box. If we open this box and see what is inside, we see 16 registers. Any instruction in x86 assembly uses only those 16 registers. The name of each register is shown in Figure 5.18 and starts with an "r." The naming convention of registers is a bit odd, but it is due to some historical naming (for the registers on the left). To keep backward compatibility, old register names cannot be changed.

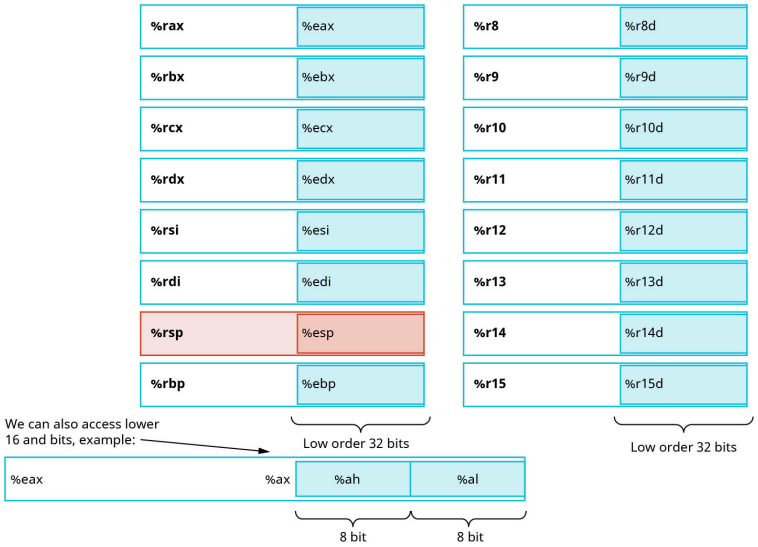


Figure 5.18 Registers in x86 are a bit complicated due to the need to keep backward compatibility and the fact that x86 is CISC. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Each one of these registers can hold 64 bits. When we had 32-bit machines, only eight registers existed, as shown on the left of Figure 5.18. In the 32-bit era, each register could hold 32-bits only and its name started with "e" (for extended). This is why we see that the lower 32 bits of the current registers hold the old names to keep backward compatibility. Not only that, in the 16-bit era, each register held 16 bits only. Those on the left had names: ax, bx, cx, and so on, which are the names of the lowest 16 bits of the current registers. Registers on the right side of the figure did not exist before the 64-bit era. If we go to the 8-bit era, we can even access the lowest 8-bits of the register. The register rax is shown as an example where parts of the register can be accessed using the naming convention: rax (16 bits), eax (32 bits), ax (16 bits), and ah and al (8 bits each). In the 8-bit era there were only four registers, which are the top four in the left column of the figure.

CONCEPTS IN PRACTICE

HLLs and Assembly

Most programmers use HLLs to write their programs. Why do you think world class programmers are very well versed in assembly? Professional programmers write in HLL but they often like to look at the assembly version of their code as well. This allows them to discover mistakes in their HLL code and also to find out

ways to enhance their HLL code.

However, there are some programs, or parts of programs, that need to be written in assembly and not in HLL for the sake of performance (assembly code written by a human in these cases is much faster than HLL translated to assembly by a compiler) and more control over the hardware. Assembly language common uses today include device drivers, low-level embedded systems, and real-time systems.

Operands

An assembly language program consists of a group of instructions. Each instruction does an operation and for this operation to be executed, it needs operands. An **operand** is a value used as input for an operator. Perhaps we want to add two numbers in an operation. To be executed it needs two numbers, which are the operands. For example: *add %RAX, %RBX* means add the content of the register RAX to the content of the register RBX and put the result in register RAX. Operands in assembly can be one of three things:

- A register
- An immediate operand (e.g., add 7 to rax; the operand here is mentioned explicitly in the instruction)
- Data from memory

Memory Addressing Modes

To get data from the memory, we need to specify the location, called the **addressing mode**, in the memory that contains the required data. Why are there several "modes" instead of just specifying an address directly? Well, the answer is related to the HLL.

In HLL programs, we use complicated data structures, such as arrays with one or more dimensions, structures, or linked lists. The compiler needs to translate this data structure to a much simpler assembly language. In assembly there are no complex data structures; there are only data items of 1, 2, 4, and 8 bytes. Then how can we map these complex data structures to the simple single dimension data items? One crucial way is to have rich addressing modes. In its most general form, an address in x86 is specified as D (Rb, Ri, S) where:

- D is a non-negative (but can be 0) integer whose range is from 0 to $2^{32} - 1$.
- Rb and Ri are registers, and they can be any one of the 16 registers.
- S is a scale that takes one of the following values: 1, 2, 4, or 8.

The address is calculated as: D + Rb + (S × Ri). While this is a general form, it can have more reduced forms such as (Rb, Ri). Here D's default is 0 and S's default is 1; D (Rb, Ri); or (Rb, Ri, S).

Now that we know about operands, let's look at the operations themselves that are implemented by the different assembly instructions.

LINK TO LEARNING

It is always good to see the concepts we learn in action. Visit this site to write an HLL program and see the corresponding assembly (https://openstax.org/r/76HLLProgram) at the same time. Any change you make to the HLL code will have an effect on the assembly.

Assembly Operations

In any ISA, all assembly instructions fall into one of three categories:

1. Data movement: from register to register, from memory to register, and from register to memory
2. Arithmetic and logic operations including addition, subtraction, AND, OR, and so on

3. Control instructions, which are the instructions that implement the “go to” operations, whether conditional or non-conditional (category also includes procedure calls)

Data Movement Operations

The data movement in assembly takes the form movx source or destination where:

- “x” specifies the number of bytes to be moved from source to destination. It can take one of the following values: “b” means 1 byte, “w” (word) means 2 bytes, “l” (long) means 4 bytes, and “q” (quad word) means 8 bytes. If you think about it, these are the sizes of all the data types we have in any HLL.
- the source and destination can be any of the operand types we mentioned earlier. There are only three combinations that are not allowed. The first is to move immediate to immediate as it does not make sense. The second is to move from memory to memory because the CPU must be involved. The last prohibited combination is when the destination is immediate as it also does not make any sense. Some examples:
  - “movq %rax, %rbx”
    Move 8 bytes from register rax and put them in register rbx, which is not really a move; it is a copy.
  - “movq (%rax, %rbx, 4), %rcx”
    This is a bit complicated. It involves three steps: first, calculate [rax + (4 × rbx)]; second, use this calculated value as an address and go to the memory at that address; and third, get 8 bytes, starting from the address you calculated (do not forget the “q” at the end of the mov instruction) and put them in register rcx. Now do you see why x86 is CISC where C means complex?

Arithmetic and Logic Operations

The arithmetic and logic operations involve very well-known operations such as add, subtract, multiply, and or, xor, shift left, and shift right. As you may have guessed, these are operations that require two operands. Look at the type of operands that we investigated earlier. For example:

```
addq %rax, %rbx
means rax = rax + rbx.
```

Additionally, there are some one operand operations such as increment and decrement. There are some complexities involved in multiplication and division where there are different instructions for signed and unsigned integers. And, for the division, yet another complexity is where to store the remainder.

Comparison and Test Operations

To be able to implement the complex data flow in HLLs (e.g., switch case or if-else), the assembly language must support conditional and unconditional *go to*. In x86 parlance, it is called a *jump* instruction. The generic form of jump is the jump instruction followed by a label. The **label** is a variable name that we give to an assembly instruction. This is needed because if you want to say (go to this instruction), how can you define “this” instruction? The label takes the following form: label: instruction. For example:

```
part: movq %rax, %rbx
. . . .
jmp part
```

In this example, “part” is the label for the move instruction. The “jmp” is a nonconditional jump; that is, it is always executed. There are some for conditional branches too. Let’s look at one of them.

For the jump if equal, or je, label, this instruction means jump to label if the zero flag is set to 1. In the programmer’s view of the assembly program, there are some flags that give information on the previous instruction (refer to Figure 5.18). One of the flags is called the zero flag, and it is set to 1 if the previous instruction has generated 0 as a result. For example, subtracting two registers and getting the result of 0.

Procedure Call Operations

One last item is related to procedure call—x86 has two instructions: CALL and RET to implement. However, the situation is more complicated than this. In HLLs, you have the concept of local variables and global variables. How is this enforced in assembly? Remember, the assembly program generated must behave in the same way you intended when you wrote the HLL program.

Assembly uses the concept of a stack, the very well-known data structure that works as last-in-first-out, to simulate the notion of local variables, passing arguments to procedure, and saving some of the registers in memory during a procedure call. Why do we need to save some registers? Because we have only 16 registers in x86, and programmers use way more variables than that in their HLL.

Vector Instructions

You may have realized that we have not mentioned floating points at all in the x86 operations. This is because there is another set of instructions and another set of registers for floating points—the **vector instructions**. These instructions operate not on individual registers, but on a vector (i.e., a group of numbers). So, an addition operation can add 32 numbers to another 32 numbers at once. That is, the first number is added to the corresponding first number in the second vector, the second number to second number, and so forth. These operations are usually very efficient in many applications.

5.5 Memory Hierarchy

Learning Objectives

By the end of this section, you will be able to:

- Discuss various memory and storage tools
- Differentiate between various types of storage technologies
- Explain how locality is used to optimize programs

For the processor to do its job, which is doing the calculations, it must be fed instructions and data. This means the overall performance depends on both the calculation's speed and the speed by which data and instructions are received. No matter how fast your processor is, you do not get good performance if the stream of instructions and data is not fast enough. Everything worked well in the early days of computing, from the 1940s until the early 1990s, and then computing hit a wall—a memory wall.

Researchers in industry and academia achieved good leaps in performance for processors by innovating ways to use transistors provided to them by Moore's law. They used those transistors, which were doubling per chip on average every 18 months, to add more features to the processor leading to better processor performance. However, the same was not done with memory, which resulted in a speed up gap between memory and processor. The gap started small and then got wider until it became a bottleneck of performance. Figure 5.19 shows the trend in the processor-memory performance gap.

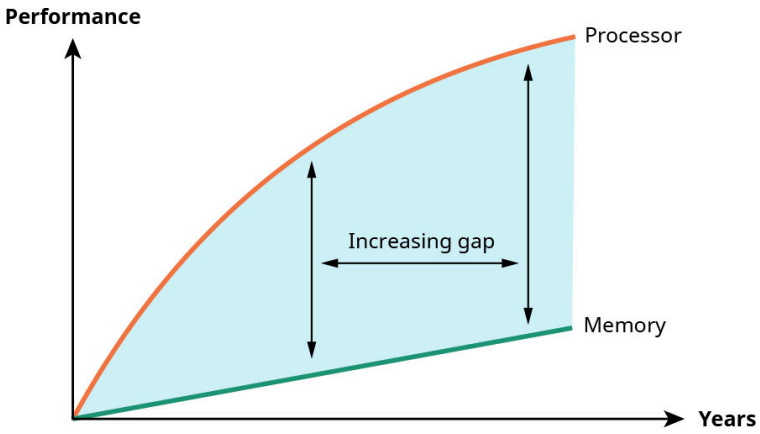


Figure 5.19 The gap between processor speed and memory speed is increasing. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

In this section, we explore memory technology. We will discuss the different technologies by which memory is built, explain what we mean by memory hierarchy, and see what researchers have done to deal with the processor-memory gap.

Memory and Storage

What is our wish list for the perfect memory? Probably speed—we want a fast memory. But be careful—memory speed is different from memory capacity. Memory capacity has increased throughout the years at a much faster rate than memory speed. We also want infinite capacity and persistence; that is, when the power is off, we want the memory to keep its content.

Next, we want to be able to pack a large amount of storage in as small an area as possible via density, which comes in handy especially for portable devices such as your smartwatch or smartphone. And what about the cost? If the memory is very expensive, the whole computer system is very expensive which means that nobody buys it; designers then have to put a smaller memory size into the computer system to keep the price low. But smaller memory means less functionality to the computer system and lower overall performance.

The reality is much less ideal. There is no single technology that excels in all these aspects. Some technologies are fast but more expensive, volatile, and less dense, while others are cheap and persistent but are relatively slower than other technologies. If we pick only one technology, we end up with a non-functional system. For example, if we pick the fast and volatile but expensive technology, the resulting computing system, which is probably very expensive, needs to be powered on indefinitely in order to retain the data. If we pick the persistent and cheap but slow technology, the system may be unusable due to its slow speed.

The word *storage* is usually used with persistence (long-term storage) while the word *memory* is used for volatility (short-term storage) even though this distinction may be blurred in future technologies. How can we get the best of both worlds?

CONCEPTS IN PRACTICE

The More You Know

Knowing about hardware is always beneficial to a software programmer. The cache, for example, is transparent to the programmer; however, if the programmer knows about the cache and how it works, they can write code that exhibits locality and gets good performance.

If you are writing a program that accesses a matrix, and you know how the matrix is stored in memory, you can adjust your code to access the matrix row by row (or column by column) to increase the locality which makes your program much faster.

The Memory Hierarchy

We have five items on our wish list for an ideal memory and, since there is no single technology that excels in all five, we must combine several technologies to come close to the ideal memory system. This ideal memory system must be fast, dense, persistent, large in capacity, and inexpensive. The technologies that we currently use have the following characteristics:

- Technology 1: very fast but expensive, less dense, and volatile
- Technology 2: faster and denser, but volatile and moderately expensive
- Technology 3: persistent and inexpensive but slow
- Technology 4: persistent and very inexpensive but very slow

We need to get the best of all of them, and the best way to combine them must ensure that we have higher capacity from technologies 3 and 4 but make technologies 1 and 2 closer to the processor so that they can respond faster to the processor. The obvious way to do this is to use an arrangement of storage available on a computer system in the form of a triangle as shown in Figure 5.20. We call this design **memory hierarchy**.

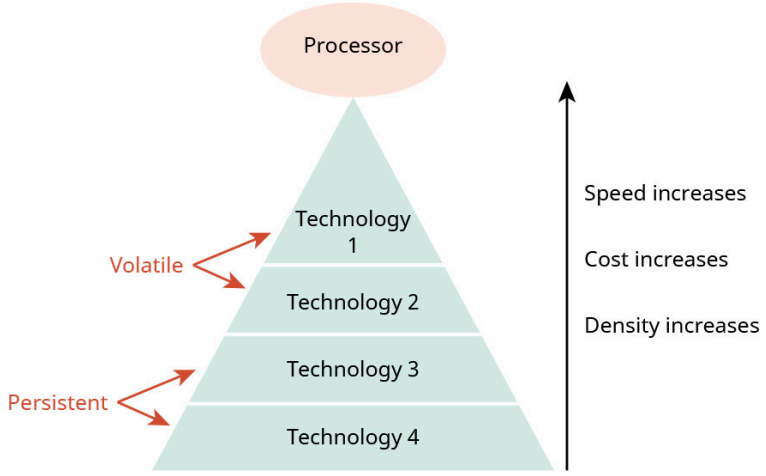


Figure 5.20 Memory hierarchy makes the best use of all technologies. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Table 5.3 gives the names of the technologies that we discuss later in this section. The processor is connected to the first cache using a set of wires called the bus. If the processor does not find what it wants, it goes to the next level, and so on.

	Technology Name	Use Case
Technology 1	Static random access memory (SRAM)	Cache memory • Volatile • Very fast • Expensive • Small in size (from few KB to few MB)
Technology 2	Dynamic random access memory (DRAM)	Main memory • Volatile • Fast • Less expensive • Average in size (1GB to 128GB)

Table 5.3 Technologies Used for Storage in a Typical Computer System

	Technology Name	Use Case
Technology 3	Solid-state drive (SSD)	Storage • Persistent • Slow • Cheap • Big in size (few GB to few TB)
Technology 4	Hard disk drive (HDD)	Storage • Persistent • Very slow • Very cheap • Several TB of storage

Table 5.3 Technologies Used for Storage in a Typical Computer System

Memory Technologies

Memory is volatile, at least for now, with research exploring other technologies for persistent memories, speed, storage, and expense. This covers technologies 1 and 2 and, therefore, they are closer to the processor. Technologies 1 and 2 cover two types of memories that both include **random access memory (RAM)**, which allows the processor to access any part of the memory in any order. Technology 2 is called DRAM, and technology 1 is called SRAM. Let us explore each one in turn.

Memory: DRAM

As you now know, the memory stores instructions and data, which are presented as 1s and 0s. One type of memory, **dynamic random access memory (DRAM)**, consists of a large number of capacitors. A **capacitor** is a very small electrical component that stores an electrical charge. A capacitor can be in one of two states: either it holds a charge, in which case we say that a 1 is stored in this capacitor, or it does not hold a charge, which means a 0. With millions of capacitors, we can store a large number of 1s and 0s. This is what you find in the specs of your laptop; when you say that you have 32GB of RAM, it means there are about 32 billion bytes in memory. Each byte consists of 8 bits. Each bit requires a capacitor.

Capacitors have a not-so-great characteristic though. When a charge is left on a capacitor for some time, the capacitor starts discharging and loses its charge. This means we lose the data stored in memory. Because of this, there is circuitry built inside the DRAM that, every few milliseconds, checks the capacitors and adds a charge to them. We call this the **refresh cycle**. It is done dynamically, hence the name dynamic RAM or DRAM.

The capacitors are not standalone by themselves. Transistors are used with them to help organize those capacitors into rows (also called word lines) and columns (also called bit lines) for addressing specific bits. Figure 5.21 shows a simplified view of DRAM. The cell, which stores 1 bit, is made up of the capacitor and some transistors.

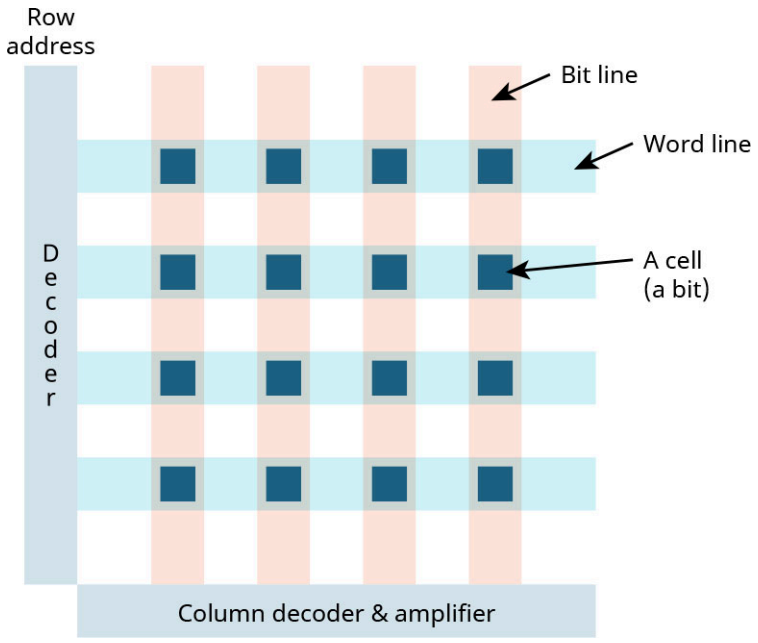


Figure 5.21 This simplified view of DRAM shows one bank. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

What is shown in Figure 5.21 is called a bank. Every few banks form a chip. Every few chips form a rank. A small memory board that contains several memory banks is a **dual in-line memory module (DIMM)**. Several DIMMs form a channel. This organization is shown in Figure 5.22. The reason for having this organization is twofold. First, if we have one huge 2-D array of memory cells, it is too complex, slow, and power hungry, so dividing it into parts makes those parts simpler and that means faster and less power hungry. Second, if there are several memory addresses that need to be accessed and they fall into different banks, for example, the memory can respond in parallel.

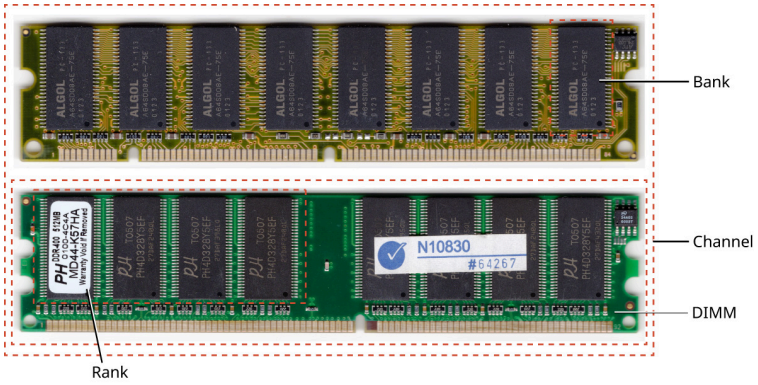


Figure 5.22 DRAM memory banks can be organized into chips, ranks, DIMMs, and channels. (credit: modification of "168 pin and 184 pin DIMM memory modules" by Veeblefetzer/Wikimedia Commons, CC BY 4.0)

You must have heard the term 64-bit machines, right? Most of our computer systems nowadays are 64 bit. One of the definitions of this term is that the connection between the processor and the memory has 64-bit width; its memory can send the data to, or receive data from, the processor in chunks of 64 bits.

The curve that we saw in Figure 5.19 shows the slow speed increase of the DRAM, which affects the performance of the overall system. If the processor must go to the memory for every instruction and every piece of data, the overall system performance is very low; therefore, computer designers speed things up by using a faster technology together with the DRAM. This faster technology, technology 1 in Figure 5.20 is called the SRAM.

Memory: SRAM

There are several reasons for the slow speed of getting the data from DRAM to processor. One is the much slower speed of DRAM technology relative to the processor. The second reason is that going off the chip that contains the processor and to the bus to reach the DRAM memory is a slow process. To overcome this, we need to have a faster memory technology inside the chip together with the processor.

The solution is to use **static random access memory (SRAM)** which keeps data in the computer's memory for as long as the machine is powered on. This means it does not need a refresh like the DRAM and is designed with a faster technology than DRAM. However, SRAMs are bigger in area; a single bit requires a large area in the chip as it needs four to six transistors, which is much larger than the capacitor in DRAM. Moreover, inside the chip we do not have a lot of space due to the existence of the processor itself. So, SRAM is a small, fast memory inside the chip that is connected to the processor from one side and to the DRAM off-chip from the other side, as shown in Figure 5.23. The DRAM is in the range of 8–64GB, while the SRAM starts from the KB range to a few MBs.

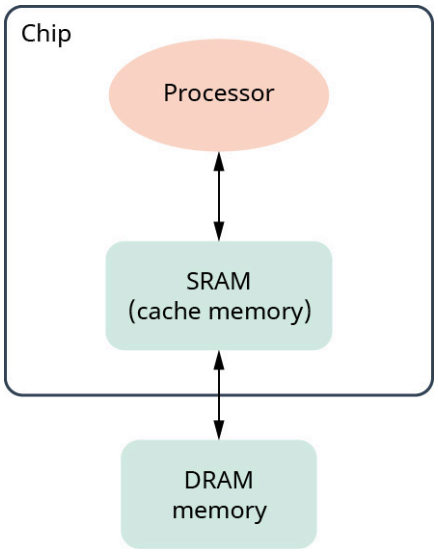


Figure 5.23 SRAM was introduced to overcome the slow speed of DRAM. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

One important distinction between the cache memory and the DRAM is that the former is programmer-transparent (i.e., its use cannot be managed by the programmer), but the latter is not. Your laptop has 32GB of RAM, which is the size of the DRAM, but you may not know how much cache your processor has. However, if you know how the cache works, you can write more efficient programs as we learn when we talk about locality.

In Figure 5.23, SRAM is more commonly referred to as **cache memory**, which is the memory that allows for high-speed retrieval of data. Let us see how the cache and the DRAM memory work together. From now on, whenever we say cache we mean the SRAM, and whenever we say memory, we mean the DRAM.

Suppose the processor executes a program that accesses array A consecutively. The processor starts with A[0] and asks the cache memory whether it has A[0]. Initially the cache is empty, so it does not have the needed data which is called a **cache miss**. The cache then gets the data from the memory. But instead of just getting A[0], it gets A[0], A[1], A[2], ..., A[x]. The number of extra elements the cache brings depends on the design of the cache. In most processors available now, the cache usually brings 64 bytes from the memory. So, if array A is an array of integers, that is, each element is 4 bytes in length, then the cache brings 16 elements from the memory, from A[0] to A[15]. The processor gets the A[0] it wants, and the extra elements brought from memory, which are in the cache. Now, if the processor wants A[1], it finds it right away in the cache, which is called a **cache hit**. However, if the processor instead needs A[17], then we have another cache miss and the cache gets to the memory again to bring several elements, including A[17].

Both the SRAM and DRAM, or cache and memory, are volatile—whenever there is a power perturbation or the machine runs out of battery, everything in the cache and the memory is gone. And we cannot build a full-fledged computer with volatile memory only; we need persistent storage too.

Storage Technologies

Storage exists in computer systems to ensure that data continues to exist even after the computer is powered off. Storage, presented as technologies 3 and 4 in Figure 5.20, has few characteristics that differ from DRAM and SRAM (technologies 1 and 2). The first, and most important one, is that they are persistent—they are non-volatile. The second characteristic is that they are slower in speed than DRAM and SRAM but have lower costs and higher capacities. It is to be noted that technologies 3 and 4 do not have to exist together in a computer system; you can have a computer with either or both.

Besides the storage that exists inside the computer system, there is a lot of storage in the cloud. That is, storage does not exist in your computer, but you can access it through the Internet. This storage is managed by big tech companies. For example, we have Azure from Microsoft, AWS from Amazon, Google Drive from Google, and so on. These companies are serving millions of users and are isolating users' data from each other. There are techniques to make each user access cloud storage and even software in the cloud.

Now, it is time to give them names. Technology 3 is called a solid state disk (SSD) and technology 4 is called hard disk drive (HDD). What are the differences and where does the commonly encountered term "flash drive" fit in? Let us start with the older technology first.

Hard Disk

Hard disks were the main storage solutions for all computers in the 1980s, 1990s, and until the mid-2000s. A **hard disk drive (HDD)** stores data on a rotating platter, has a very large capacity, and uses a small motor to rotate platters to get the data. You can easily buy an 8TB disk for a modest amount of money. The industry took about 25 years to move from 5MB disk to 1TB, and only two years to go from 1TB to 2TB, and, after that, the capacity increased by a whopping 60% per year. So, we have a very ample size with a low price but also a very slow disk. It is several orders of magnitude slower than the DRAM memory. The reason is shown in Figure 5.24. The main reason the disk is slow is due to the mechanical movement. Therefore, computer designers have been looking for storage that does not need mechanical movement drives and that does not have any moving parts.

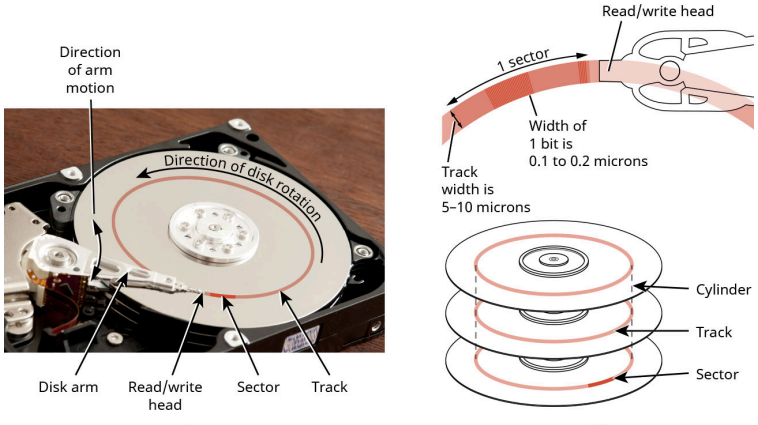


Figure 5.24 (a) The full hard disk drive (HDD) is (b) divided into platters, and each platter is divided into tracks and sectors. (credit a: modification of "Open HDD" by Gratuit/Freeimageslive, CC BY 3.0; credit b: attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Solid-State Drive

The term *solid state* means that a system does not have moving parts and is expected to be fast. A **solid-state drive (SSD)** stores data on a chip and is two to three orders of magnitude faster than HDD. SSD has two main

parts: the storage itself and the circuitry that accesses the storage. Nowadays SSDs use a storage technology called **flash memory**, which is a type of nonvolatile storage that can be electronically erased and reprogrammed. It is what you use in your thumb drives, just a bit faster. Flash memory in SSDs is based on NAND gates. A **NAND gate** is a type of logic gate used to store bits in flash memory. Flash memory is organized as pages, and a group of pages is called a **block**.

The circuitry that controls the flash memory, called the **translation layer**, has an important function. It maps the addresses to pages. One of the disadvantages of storage cells that make a page is that they wear out after 100 thousand to 1 million accesses. So, the translation layer tries to change the mapping to ensure that accesses are equally distributed among different cells. This is a complicated process and is one of the reasons SSDs are more expensive than HDDs. The sequential read from SSD reaches 7000 MB/s and the sequential write reaches 5000 MB/s. Random access to SSD has lower speed for reads and writes.

LINK TO LEARNING

Most laptops now have SSD storage, so it is good to know how SSD really works. Watch this video for a succinct explanation of how SSD works (https://openstax.org/r/76SSDHowItWorks) in the context of smartphones.

More About Cache Memory

As you have learned, cache memory is used as a fast, small memory inside the processor to close the gap between the processor speed and the memory speed. Given that latency is the rate of data transfer, assume DRAM memory's access latency is M cycles and the cache access latency is m. Also assume that for a specific program the probability of cache hits = p. Then the average latency of the combined cache + Memory = $mp + (1 - p)(M + m) = \boldsymbol{m + (1 - p)M}$.

Remember that whenever there is a cache miss, we have already spent m cycles searching the cache, then we go to the memory, which takes another M cycles. If we look at the equation $m + (1 - p)M$, we see that to get good performance, we need to do one or more of the following things:

- Have a faster cache and lower m.
- Increase cache hit rate and reduce p.
- Have a faster memory and lower M.

To have a faster cache, we must make it smaller in capacity, but smaller cache decreases **hit rate**, which is the number, usually a percentage, of times the cache was used to retrieve data. We cannot easily have faster memory to reduce M so the solution is to have more than one level of caches. Level 1 (L1), closest to the processor, is small in capacity and very fast but with a potentially low hit rate. When there is L1 cache miss, instead of going to the memory off-chip, we go to L2 cache, which is still on chip and bigger in capacity than L1. Most processors now have up to three levels of caches, as shown in Figure 5.25.

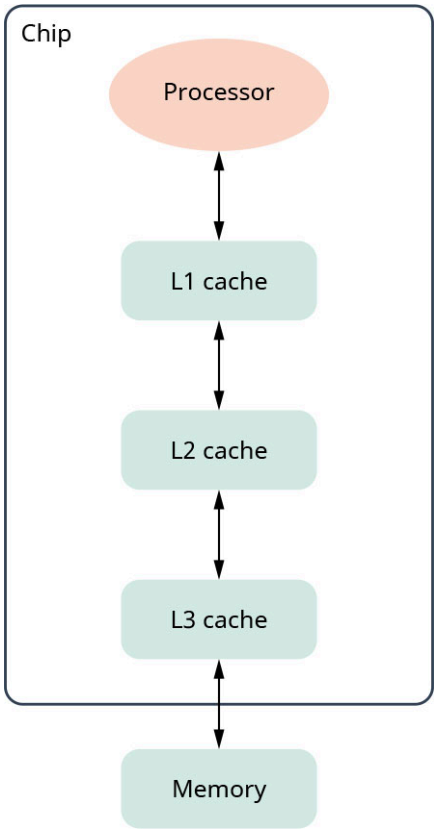


Figure 5.25 Most processors now have three levels of cache memory. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Locality

Throughout our discussion of the memory hierarchy, we have determined the following:

- Memory is read in a chunk of consecutive 64 bits which is 8 bytes.
- Whenever there is a cache miss, the cache works to bring a cache block, not the few bytes the processor needs. Most caches now have a cache block of 64 bytes. Those 64 bytes are consecutive bytes brought from the lower-level cache, that come from the even lower cache (i.e., from L3 to L2 to L1). The L3 cache brought the block from memory.

When a processor repeatedly visits the same memory locales, it has **locality**. We can surmise that if a program accesses the data in a consecutive manner, it gets better performance, called **spatial locality**. Also, if we reuse the data as much as possible, we get better performance because the data is available in the cache, and we can increase the cache hits. We call this second criteria **temporal locality**.

We can get even better performance if the programmer writes efficient code that makes the best use of the underlying hardware. An efficient program has a very important characteristic called locality. For example, if you want to add two arrays together (i.e., *A*[0] + *B*[0], *A*[1] + *B*[1]), then you get a good performance if you access these two arrays sequentially from element 0 until the end of the arrays which is an example of spatial locality. So, whenever you are writing a program, pay close attention to how you access the data.

Instructions also reside in memory. If we have a for-loop that is executed several thousand times, the instructions in the loop body are reused in every iteration which is an example of temporal locality.

5.6 Processor Architectures

Learning Objectives

By the end of this section, you will be able to:

- Discuss the history and advancements in traditional processor architectures and computation models
- Define heterogeneity and discuss its effect on computer systems

Processors have a relatively short history that starts in the late 1960s; however, there have been big jumps since then. In this section, we learn about the evolution of processors from the dawn of processor design until today. The main measures of success of processors involve correctness, speed, power, reliability, and security.

Speed has been the main measure of success for some time. But then computer designers began to worry about battery life (for portable devices) and the electricity bill (for big machines) so power became a pivotal issue. As computers have invaded almost all aspects of our lives, reliability has become a must because we do not want computers to fail. With the widespread use of the Internet and peoples' need to be connected all the time, security has also become an issue.

Homogeneous Processor Architectures

The current processors contain several CPUs inside the chip. If these CPUs are copies of each other, the design is **homogeneous**. If there are different types of CPUs, some are fast but power hungry while others are slow but power efficient, the design is called **heterogeneous**. An example of heterogeneous processors is the chip inside the latest MacBook Pro.

In its earliest version, a processor was just a big, bulky, black box that got its instruction from memory, executed it, got the next instruction, and so on. What was bad about this simple design? First, having one big bulky circuit made it slow and power hungry. Second, not all instructions took the same amount of time; a floating-point computation took as much as ten times longer than an integer computation. But since this design was one box working with one clock, the clock cycle had to be as big as the slowest instruction. The processor executed at the speed of the slowest instruction; therefore, even though the design was simple, it suffered from performance and power issues. However, physics came to the rescue.

CONCEPTS IN PRACTICE

Social Media and Supercomputers

Billions of people use social media sites such as Facebook, X (formerly Twitter), YouTube, and Instagram every day and at the same time. This means we need supercomputers that can serve all these people, store and manipulate huge amounts of data in a short time, and not go down. This cannot be done with a simple multicore; it requires millions of multicores and thousands, if not millions, of accelerators such as GPUs, TPUs, and FPGAs.

Moore's Law and Dennard Scaling

In 1965, Gordon Moore (cofounder of Intel) published a short paper that predicted that the number of devices inside the processor would double every 18 months, called Moore's law. Since transistors are the building blocks of logic gates, and logic gates are the building blocks of pieces such as adders, multipliers, and registers, then more transistors would mean more features implemented in the processor, hopefully leading to better performance. More transistors inside the processor's chip meant that transistors would get smaller in size, and smaller transistors would mean faster transistors. Not only that, but Robert Dennard from IBM found that as transistors got smaller, the power consumed and dissipated was also reduced in a phenomenon known

as Dennard scaling.

Traditional Processor Architectures

Figure 5.26 shows how the processor evolved from the single cycle implementation to more sophisticated and higher performance designs thanks to Moore's law and its enabling technology, Dennard scaling.

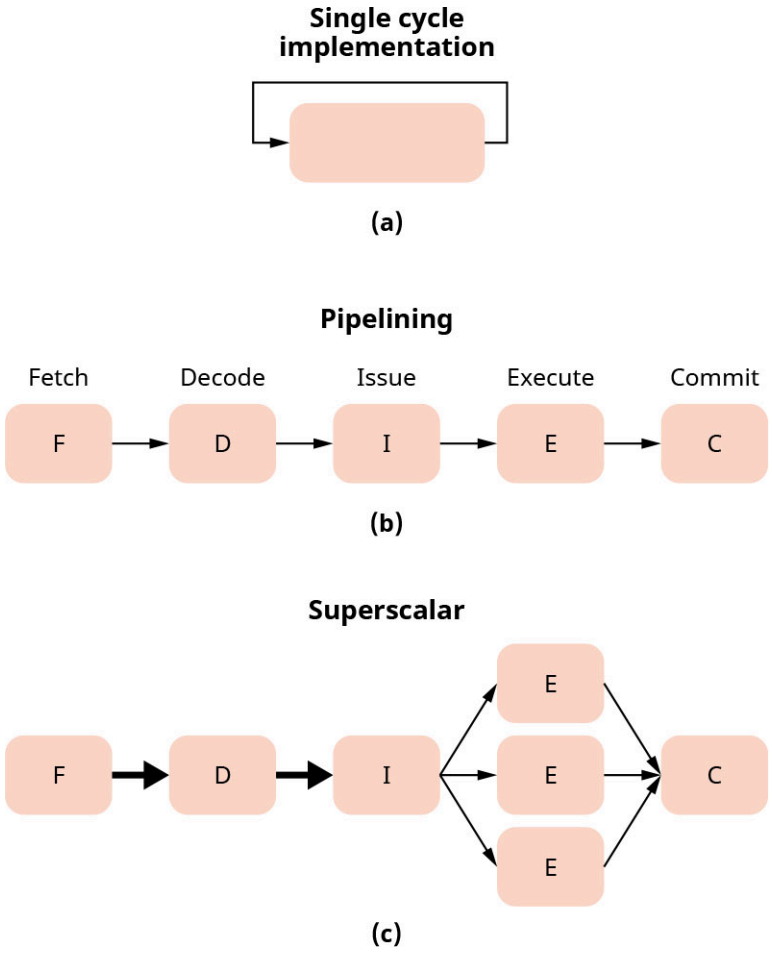


Figure 5.26 The processor has evolved from the (a) simple design single-cycle implementation to (b) pipelining to the very sophisticated (c) superscalar design in less than 70 years. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

If you think about what this bulky box in Figure 5.26(a) really does, you reach the following conclusion: it does few things repetitively with each instruction. It fetches an instruction from memory, decodes this instruction to know what needs to be done, and issues the instruction to the correct execution units (e.g., an integer operation to the integer execution unit and a floating-point operation to floating point execution unit).

After execution, it writes the result of the operation back to a destination specified by the instruction, which is called the commit. Given this description, why not take the bulky piece of hardware in Figure 5.26(a) and divide it into these pieces: fetch, decode, issue, execute, and commit? Each piece does the work needed by the following piece in a technique called **pipelining**, with each piece called a phase (Figure 5.26(b)). What do we gain from this?

First, each phase is now much less complicated, less power hungry, and faster. Second, once the fetch phase finishes fetching the first instruction and hands it to the decode phase, the fetch phase hardware is now free to fetch the second instruction. By the time the decode phase is done with instruction 1 and hands it to the issue phase, the fetch phase hands to it instruction 2 and starts fetching instruction 3, which is a form of parallelism.

If we take a snapshot of the pipeline during the execution of the program, we find different pipeline phases working on different instructions. We call this type of parallelism **temporal parallelism**. The benefit is that this parallelism—or better performance—is done with no involvement from the programmer. The hardware is doing it for you.

With more and more transistors available, designers move to executing several instructions at the same time, which means that several execution units are needed. The other phases must be modified to fetch/decode/issue several instructions at the same time, which is referred to as capability as shown in Figure 5.26(c). Another type of parallelism is **superscalar capability**, which is an execution unit that allows several instructions to be executed at the same time using another type of parallelism, **spatial parallelism**. If you look closely at Figure 5.26(c), you realize it combines both the temporal parallelism (from pipelining) and spatial parallelism (from superscalar capability).

The next step in enhancing a processor's performance is to fetch instructions from more than one thread, and you can technically execute two or more programs on the same processor at the same time. This is called **simultaneous multithreading (SMT)**, which Intel calls hyperthreading technology.

This last enhancement was introduced in the early 2000s, but something happened around 2004 that pushed both the hardware and software communities to change gears. Dennard scaling stopped and designers kept increasing the frequency of processors to make them faster. This led to a complete stall because increasing frequency resulted in drastic increases in power consumption.

Multiple Cores

With the stop of Dennard scaling and the inability to increase the frequency of the processor, it was time for a drastic change. On one chip, instead of putting one processor, designers decided to put multiple processors called cores inside the same chip which started the multicore era. All the processors we use today, from our watches to big supercomputers, are multicore processors.

The trend is to increase the number of cores per chip while decreasing their frequency. Each core in the multicore is an SMT to avoid an increase in power consumption as well as an increase in chip temperature. There is one catch though; all previous techniques (such as pipelining, superscalar, and SMT) were giving us performance without the programmers getting involved. To make use of all the cores in the chip with multicore, programmers must write parallel code that requires the use of a parallel programming language. This is bound to become the norm.

LINK TO LEARNING

Microprocessors (also known as processors) have evolved in the last half century in many ways. Visit this site to learn more about microprocessor trend data (https://openstax.org/r/76CMicroproData) from 1970 to the present.

Heterogeneous Processor Architectures

Multicore processors are designed to be good on average for most applications. However, they do not give the best performance for every single program. Designers have started introducing chips that have excellent performance, better than multicore, but for a small subset of program types. Examples of these chips are graphics processing units (GPUs), field programmable gate arrays (FPGAs), and tensors processing units (TPUs). The idea is to start the program on a multicore until there is a part where other chips excel. In that case, the multicore sends that piece of code to the other chips, and this gives rise to parallel programming for heterogeneous systems (i.e., computer systems that have chips with different capabilities). Laptops can now have a multicore plus GPU.

Multiple Nodes

Having multicore plus accelerator chips (e.g., GPUs) on the same board is now the norm and is called a node. But what about big machines that run in the cloud to give us services such as Amazon, Facebook, and X (formerly Twitter)? These big machines are built using thousands, if not millions, of nodes and they need an even more sophisticated way of programming.

Chapter Review

Key Terms

abstraction removal of unimportant elements of a program or computer code that distract from its process
addressing mode location in the memory that contains the required data
American Standard Code for Information Interchange (ASCII) unit code or unique binary number
arithmetic logic unit (ALU) piece of hardware inside the CPU that performs computations and logical operations such as comparisons
assembler takes the assembly program that takes I/O that contains the equivalent of an assembly program
assembly language low-level language that is designed to be computer friendly
bit binary digit made up of 1 or 0
block group of pages in flash memory
bus data pathway
byte 8 bits
cache hit data found in the cache needed by the processor
cache memory memory that allows for high-speed retrieval of data
cache miss when the cache is empty so it does not have the needed data
capacitor very small electrical component that stores an electric charge
complex instruction set computer (CISC) complex architecture structure that assists in executing such operations as mathematical computations and memory storage
disk storage mechanism for data
dual in-line memory module (DIMM) small memory board that contains several memory banks
dynamic library collection of libraries that may be linked during execution or while the program is running
dynamic random access memory (DRAM) consists of a large number of capacitors
executable program stored inside a computer
flag tells a program if a condition has been met
flash memory type of nonvolatile storage that can be electronically erased and reprogrammed is called
floating point number (also, real number) one with a decimal point in the middle
hard disk drive (HDD) stores data on rotating platter, has a very large capacity, and uses a small motor to rotate platters to get the data
heap stores dynamically allocated data
heterogeneous design feature where CPUs are different in speed, power, or efficiency
high-level language (HLL) most evolved method by which a human can direct a computer on how to perform tasks and applications
hit rate number, usually a percentage, of times the cache was used to retrieve data
homogeneous when CPUs are copies of each other in design
IEEE 754 standard format used by computers that support floating points with very few exceptions
input/output (I/O) interface that helps the CPU talk to other I/O devices such as a keyboard or mouse
instruction set architecture (ISA) set of instructions recognized by each processor family
label variable name that we give to an assembly instruction
library collection of files, functions, or scripts that are cited within a program's code
loader puts the executable into memory and arranges its content in a specific way to make it ready for execution by the processor
locality when a processor repeatedly visits the same memory locales
logic gate main building block that forms the processor
memory hierarchy arrangement of storage available on a computer system usually in the form of a triangle
microarchitecture architecture, or design, of the processor or microprocessor
NAND gate type of logic gate used to store bits in flash memory
neuromorphic computer nontraditional computer built to act like a simplified version of the brain

object code designates output of the assembler
one's complement obtained by flipping each 1 in the original binary number to 0 and each 0 to 1
operand value used as an input for an operator
operating system (OS) only piece of software that can directly access the hardware
pipelining technique where each piece of the process does the work needed by the following piece
processor another name for the CPU
program counter (PC) register that keeps track of instructions to be executed
pseudo-assembly assembly instruction that does not have a counterpart in the machine code
random access memory (RAM) allows the process to access any part of the memory in any order
reduced instruction set computer (RISC) simple instructions
refresh cycle regular operation that DRAM memory does by adding charges to memory cells in order to lose the data stored as charges
register memory unit that functions at a very high speed
register file group of registers
scheduling when the OS manages which program to use with what part of the hardware at any given time
signed integer integer that can be negative or positive (sign of + or –)
simultaneous multithreading (SMT) when two or more programs execute on the processor at the same time
solid-state drive (SSD) stores data on a chip and is two to three orders of magnitude faster than HDD
spatial locality idea that if a program accesses data in a consecutive manner, it gets better performance
spatial parallelism type of parallelism capability
stack used to store local variables
static random access memory (SRAM) keeps data in the computer's memory as long as the machine is powered on
superscalar capability execution unit that allows several instructions to be executed at the same time using spatial parallelism
temporal locality reuses cached data to get better performance
temporal parallelism allows for different pipeline phases to work on different instructions
transistor lets an electric current pass (on state) or blocks it (off state)
translation layer circuitry that controls the flash memory
two's complement one's complement with 1 added
unsigned integer non-negative integer that starts from 0
vector instructions another set of instructions and another set of registers for floating points
virtual memory technique the OS uses to isolate different programs from each other so they do not overwrite each other's data or corrupt each other's files

Summary

5.1 Computer Systems Organization

- The main components of a computer system include the processor, the main memory, the disk, and I/O devices.
- The components interact to execute computer programs efficiently.
- Applications are written in high-level languages that use a program to translate the language into the machine-level programs that computers understand.
- This computer system organization is mostly the same in your tablet or smartphone as it is in the huge systems running services like Facebook or Google. They just differ as it comes to how powerful each piece is, how many CPUs are there, the size of the memory and disk, and so on.

5.2 Computer Levels of Abstraction

- The levels from problem definition to assembly language are related to the computer science field. People studying computer science explore how to solve a problem using an algorithm, how to translate this

algorithm into a programming language, then how to translate this programming language into a language that computers can understand.

- Computer processors operate at various levels of abstraction going from the digital logic level up to the microarchitecture level and the machine language level. The highest level of abstraction for application programs is that obtained by writing programs in a high-level language. Using a compiler makes it possible to generate a representation of such programs at a lower level of abstraction known as assembly language. That representation uses instructions that are part of the instruction set architectures (ISA) specific to the processor family in use.
- The operating system (OS) is the only piece of software that can directly access the hardware of a computer. All other programs must interface with the OS to have it achieve a specific task in a secure fashion by scheduling the corresponding process. The OS stores data and programs on disk in an organized way, using a file system, and allocates memory to programs.
- Upcoming computer designs call for new computing abstractions that will deviate from the traditional binary logic. For example, quantum computing uses qubits and neuromorphic computing uses hardware neurons.

5.3 Machine-Level Information Representation

- The most frequently used data items are stored inside a computer.
- An integer is a number that does not have a floating point; 7 is an integer but 3.14 is not. Integers are divided into two categories: signed and unsigned. Signed integers can be positive, negative, or 0. Unsigned integers are 0 or positive; therefore, they do not need a sign because they will never be negative. This is why we call them unsigned.
- Real numbers, known as floating point numbers, are the numbers that represent fractions. Integers cannot represent numbers like 3.14 or –1.25. This is the role of floating point numbers.
- Characters and symbols on your keyboard are represented inside the computer as 1s and 0s. Every character has its own code.
- The range of numbers that *n*-bit binary number can present if interpreted as a signed or an unsigned integer is helpful to you when you write a program because if you know the values that a variable in your program may take, you can make a precise decision about the type of that variable when you declare it.

5.4 Machine-Level Program Representation

- An x86-64 Intel processor uses a complex instruction set computer architecture (CISC). This type of architecture combines many simple instructions into a simple complex one. Other processor architectures use a reduced instruction set architecture (RISC) based on simpler instructions. RISC V is a relatively new RISC ISA that is getting popular than CISC. All new high-tech companies use RISC instruction sets in their hardware. For example, ARM assembly, which is a RISC assembly, is used in most portable devices.
- Assembly language makes use of processor instructions that are part of the instruction set architecture (ISA) of the processor being used. These instructions can be converted to binary code, referred to as machine language code, using an assembler.
- Assembly language provides specific instruction to perform common operations such as addition and multiplication of signed and unsigned integers.

5.5 Memory Hierarchy

- We must use different technologies to build a near ideal memory because each technology has some good characteristics and a few shortcomings. Therefore, we use multiple technologies to get the best of all. Various types of memory, such as DRAM and SRAM, are available and differ in access speed and associated costs.
- Those technologies are organized as a hierarchy. Technologies that are fast but expensive are at the top of the hierarchy, but their storage capacity is not big. As we go down the hierarchy, the technology is slower, but cheaper; hence, we use a lot of storage from it.
- That is, characteristics of each technology determine its place in the hierarchy.

- By paying attention to locality in accessing the data, the programmer can get the best performance from the hierarchy.

5.6 Processor Architectures

- Processors evolved from a single cycle implementation to multicore.
- The norm now is to have a multicore processor working in tandem with one or more accelerator chips.
- The norm in software development is to write parallel code for such a heterogeneous system. Sequential programming will soon be dated.

Review Questions

1. What is the difference between CPU and ALU?
2. What is the difference between the processor used in your smartphone and the one used in your desktop, tablet, or laptop computer?
3. What is the definition of abstraction?
   a. a piece of software that takes a program written in a given HLL and generates another program
   b. the most evolved method by which a human can direct a computer on how to perform tasks and applications
   c. the set of instructions recognized by each processor family
   d. removal of unimportant elements of a program or computer code that distract from its process
4. What is the term for the technique the OS uses to isolate different programs from each other so they do not overwrite each other's data or corrupt each other's files?
   a. virtual memory
   b. scheduling
   c. neuromorphic computer
   d. assembler
5. Why do computers only work with 1s and 0s?
6. Why is the OS the only software allowed to access the hardware?
7. Do a big supercomputer and small budget laptop use the same levels of abstraction?
8. What type of number would we need to represent the number of marbles in a bag?
   a. signed integer
   b. unsigned integer
   c. floating point number
   d. Boolean number
9. What type of number would we need to represent the temperature outside to the nearest whole number?
   a. signed integer
   b. unsigned integer
   c. floating point number
   d. Boolean number
10. What type of number would we need to represent the amount of money in a bank account?
    a. signed integer
    b. unsigned integer
    c. floating point number
    d. Boolean number
11. What is the two's complement of the binary number 11100101?
    a. 01100101

b. 11100100
c. 00011010
d. 00011011

12. What is the one's complement of the binary number 10100010?
a. 00100010
b. 10100011
c. 01011101
d. 01011110

13. If you see a series of bits, can you know whether they present a signed or an unsigned integer, a floating point, or a character?

14. Why do we need unsigned integers?

15. What is the purpose of a linker?
a. A linker puts the executable into memory and arranges its content in a specific way to make it ready for execution by the processor.
b. A linker tells a program if a condition has been met.
c. A linker takes all the generated object files, looks for needed libraries, and then links everything together in one executable file.
d. A linker is a register that keeps track of instructions to be executed.

16. What is the output of the assembler called?
a. object code
b. source code
c. dynamic library
d. static library

17. What stores dynamically allocated data?
a. stack
b. register
c. bus
d. heap

18. Why do you need to learn assembly programming?

19. What is the main philosophy behind CISC?

20. What is the main philosophy behind RISC?

21. What technology do we use to build a cache?
a. static random access memory (SRAM)
b. NAND gate
c. dynamic random access memory (DRAM)
d. solid state drive (SSD)

22. What technology do we use to build memory?
a. static random-access memory (SRAM)
b. NAND bate
c. dynamic random-access memory (DRAM)
d. solid state drive (SSD)

23. What term means that, when the cache is empty, it does not have the needed data?
a. cache hit
b. cache memory

c. cache miss
d. cache reset

24. Why is it necessary to have multiple levels of cache memory?
a. to make it harder for the CPU to obtain instructions from RAM
b. to make it faster for the CPU to obtain data from RAM
c. just in case one level of cache fails
d. to be able to execute instructions in parallel

25. Is programming a single core or multicore easier? Why?

26. Why did the industry move to multicore?

27. What is the term used to describe when two or more programs execute on the processor at the same time?
a. temporal parallelism
b. spatial parallelism
c. simultaneous multithreading (SMT)
d. superscalar capability

Conceptual Questions

1. Why is the memory connected directly to the CPU?

2. Both the disk and memory store programs and data. Why do we need them both in a computer system?

3. Why do we need an assembler? Why don't we make compilers generate machine language directly?

4. If there are two processors that understand the same ISA, does this mean they have exactly the same microarchitecture?

5. Can the step from algorithms to HLL programs be automated instead of being done by a human being? Justify your answer.

6. The idea of compilers and assemblers made HLL more portable. Why is that?

7. Suppose we have a list of 1,000 numbers ordered in ascending order. We need to find whether a specific number is in the list or not. What is the best algorithm to accomplish this?
a. Scan the list from first number to last number until you find the number you want or reach the last number.
b. Scan the list from last number to first number until you find the number you want or reach the first number.
c. Go to the middle of the list and see whether the number you are looking for is bigger or smaller than the middle number. If it is bigger, discard the lower half. If it is smaller, discard the higher half. Then redo the same in the smaller list.
d. Take a quick look at the list and decide whether the number is present.

8. Why is the two's complement a good choice for presenting signed numbers?

9. As a programmer, do you think it is useful to know about data presentation? Why?

10. Why are computers slower in dealing with floating points than integers?

11. For portable devices, such as your smartphone, do you think processors supporting CISC ISA or RISC ISCA should be used? Why?

12. Is it possible to build a de-compiler? That is, if given an assembly code, can we bring the original HLL code? Explain.

13. Why are the different technologies organized as a hierarchy?
14. What is the relationship between SSD and flash memory?
15. Do you think it is better to have more cores in the chip or to use multiple chips with fewer cores each? Why?
16. Why is heterogeneous computing here to stay?

Practice Exercises

1. Perform a search on the Internet about the specifications of your own computer as well as other similar computers from companies like Lenovo or Dell in terms of memory size and disk size. What is the ratio disk size to memory size in each computer and why?
2. Document how you can obtain the amount of cache memory on your computer.
3. Search the web for five different ISAs. For each one, find which companies are building processors that use that ISA. Finally, for each processor from these companies, check whether this processor is used on your portable device (smartwatch, smartphone, tablet), your laptop or computer, or in big supercomputers and data centers.
4. You are using the Internet to access a website of your choice. Create a diagram showing the various levels of abstraction of a computer system to explain this particular scenario.
5. Research and explain a few key responsibilities of operating systems.

Suppose you wish to express –64 as a two's complement number.

6. What is the minimum number of bits we will need?
7. With this minimum number of bits, what is the largest positive number you can represent, assuming signed numbers of course? (Answer in both decimal and binary.)
8. With that same number of bits you used in the previous questions, what is the largest unsigned number you can represent? (Answer in both decimal and binary.)
9. Represent the decimal digit 90 in binary.
10. Add the following binary numbers: 0111000101 + 0000100101?
11. Find the one's complement of 1010001000011111.
12. Find the two's complement of 000010001111110.
13. Make a list of at least three ISAs in existence today. For each one, find out whether it is CISC or RISC and in which processors it is used. Finally, take the list of these processors and see whether they are used in high-performance computing machines or portable devices.
14. Make a list of three or four accelerators and, for each one, describe the type of applications they excel at and why.

Problem Set A

1. A cache has an access time of 1 cycle. The computer with that cache experienced an average memory access time of 4 cycles, and the hit rate is 70%. What is the access time of the main memory? Did we benefit from having a cache in this system?
2. Research and explain a few key responsibilities of operating systems.

Given $X = 01100110$,

3. What is the value of X once you logically shift X to the right by two digits?
4. What is the value of X once you arithmetically shift X to the right by two digits?

Given $X = 10100110$,

5. What is the value of X once you logically shift X to the right by two digits?
6. What is the value of X once you arithmetically shift X to the right by two digits?
7. What is the value of X if you arithmetically shift it to the right by eight digits?

Perform the following number conversions:

8. hexadecimal value 40A5F916 to binary
9. binary value 11011010011010111010 to hexadecimal
10. Considering the arguments passed in, what does the following assembly language program do?

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

movq	%rdi, %rax
subq	%rsi, %rax
movq	%rsi, %rdx
subq	%rdi, %rdx
cmpq	%rsi, %rdi
cmovle	%rdx, %rax
ret	

11. Suppose that we have a system with memory access time of 8 cycles. We need to add to that system a cache with 2 cycles access time. What is the smallest hit rate needed to make that cache beneficial? (Hint: avg access time must always be an integer number not floating point.)
12. A cache has an access time of 1 cycle. The computer with that cache experienced an average memory access time of 4 cycles, and the hit rate is 70%. What is the access time of the main memory? Did we benefit from having a cache in this system?

Problem Set B

1. Suppose that we have a system with memory access time of 8 cycles. We need to add to that system a cache with 2 cycles access time. What is the smallest hit rate needed to make that cache beneficial?
2. Explain how there are multiple levels of abstraction in a computer and explain how at each level, complex implementation knowledge of the lower levels is not needed.
3. Calculate 0 1000 0001 110...0 plus 0 1000 0010 00110...0. Both are single-precision IEEE 754 representations.

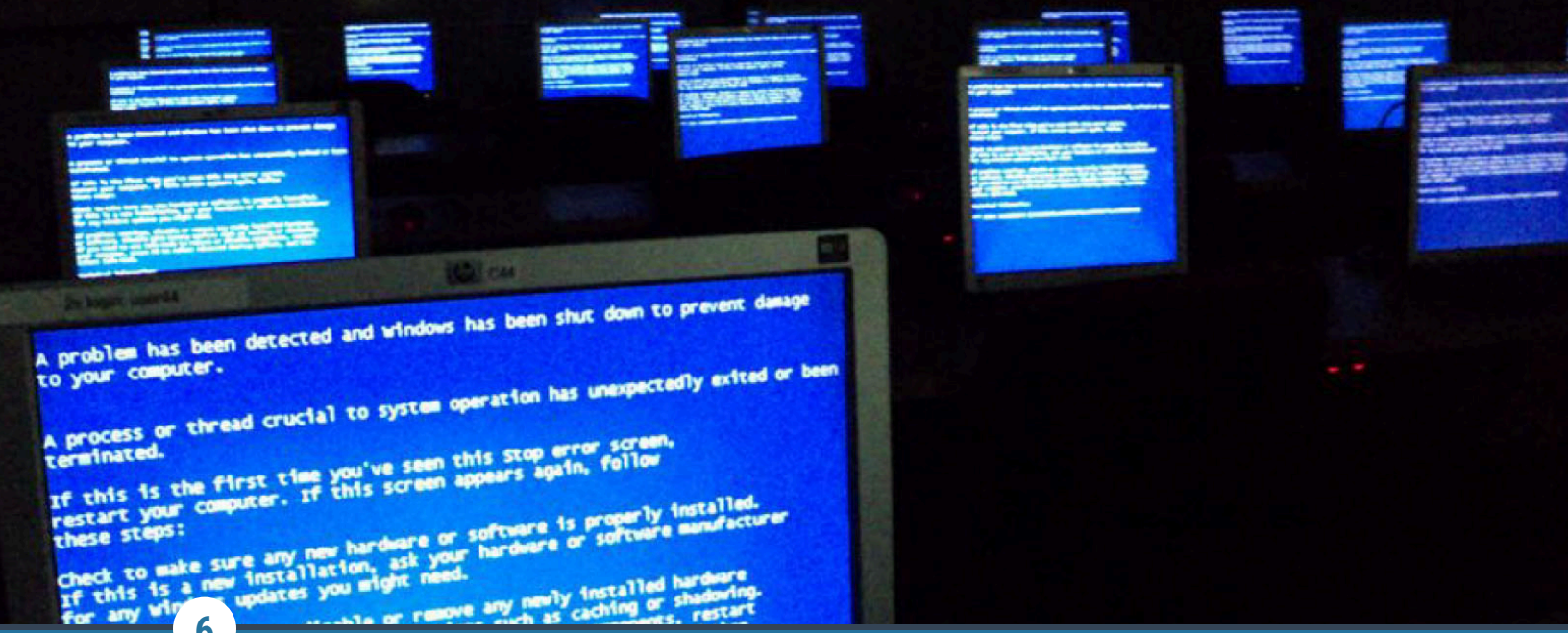
4. We derived the equation $m + (1 - p)M$ for average access latency when we have one cache and memory. Extend this equation to include two-level cache followed by memory.

Thought Provokers

1. What are the desired characteristics of a good computer system, aside from speed?
2. When a company decides to design a new processor to compete in the market, what are the criteria that this company needs to tackle to be competitive? Be careful—speed is not enough.
3. Consider our startup company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe at a high-level how it could combine the use of Unicode and Large Language Models to translate documents in any language. What would be some of the limitations of such a solution using existing technology?
4. With several levels of cache, the data in L1 is a subset of the data in L2 which is a subset of L3. This is called the inclusion property. This makes the data a bit redundant and loses chip area to store repetitive data. Do we gain anything from violating this inclusion property? If yes, what do we gain? What are the challenges? If no, why not?
5. Now that Moore's Law is coming to an end, how do you think the computer industry can get more performance from computer systems? Suggest several solutions.

Labs

1. Make a list of the I/O devices on your laptop, and another list of the software programs installed on your laptop. This second list is a bit tricky.
2. Find out the type of processor on your computer or laptop. Once you get the type, find out the following information about it:
   - Which company designed it?
   - How many transistors does it have?
   - What ISA does it use?
3. Using the link provided earlier in this section about translating from HLL to assembly, write a sample program and generate its assembly. Look at the assembly generate, understand it well, and then write yourself another version of this assembly that does the same thing.



6

Infrastructure Abstraction Layer: Operating Systems

Figure 6.1 Operating systems operate on many computers and devices, which means that when they need an update (due to an error or virus), all those systems need the same fix. (credit: modification of "Windows Blue Screen on room full of computers," by Grj23/Wikimedia.com, CC0)

Chapter Outline

6.1 What Is an Operating System?
6.2 Fundamental OS Concepts
6.3 Processes and Concurrency
6.4 Memory Management
6.5 File Systems
6.6 Reliability and Security

Introduction

TechWorks is a start-up company that is 100% committed to leveraging innovative technologies as part of its repeatable business model and as a business growth facilitator. TechWorks has been suffering for many months from the limitations of its operating system, especially the system's lack of security, which allows access to unauthorized and unauthenticated users. This problem reflects poorly on the company's reputation. To address the security limitations, one of the company's project managers decides to implement a new mode of protection that supports all of the operating systems the company's users use, including iOS, Windows, Linux, macOS, and Android. A data scientist working for the company proposes two solutions. The first solution involves integrating an authorization method called two-factor authentication, which you'll learn more about later in the chapter. This method adds another level of security by asking users for their cell phone number so that it can send a one-time code to verify their identity every time they try to log in using their username and password. The second solution involves adding a table that defines individual user privileges such as read (R), write (W), and execute (E). This solution regulates access by leveraging features that were already part of TechWorks's operating system. One of the core features of an operating system is to manage and regulate access to the program components that are running on various machines. This is particularly important today as the program components that power modern solutions (e.g., advanced robotics, autonomous cars, and drones) are typically distributed across many different machines that communicate with each other to perform various functions. To ensure the security of such systems, a single sign-on capability is

typically required to facilitate access to all the components involved.

6.1 What Is an Operating System?

Learning Objectives

By the end of this section, you will be able to:

- Describe an operating system and its role in computing
- Explain the architecture of operating systems

An **operating system (OS)** is the core piece of software that typically manages and controls the interconnection of hardware and software on a computer. The OS is loaded upon start-up and is the key piece of software needed to operate any computerized device.

Introduction

There are many operating systems (OSs), and anyone using a modern computer is using one of them. The typical OSs for computers are Windows, macOS, and Linux and for mobile devices, iOS and Android (Figure 6.2). Microsoft Windows is a popular operating system, celebrated for its ease of use and broad software compatibility. Apple Inc.'s macOS, which is the driving force behind Apple computers, ensures tight hardware-software cohesion. Linux, a freely available open-source OS, is acclaimed for its reliability, safety, and adaptability. Linux source code has over 27 million lines of C, while Windows has over 50 million lines. Linux offers a variety of versions or distributions, such as Ubuntu for ease of use, Fedora for the latest features, Debian for stability, and Kali Linux for security tasks. Each version is designed to meet specific needs, from general computing to specialized applications. All the commercial operating systems today manage security for users.

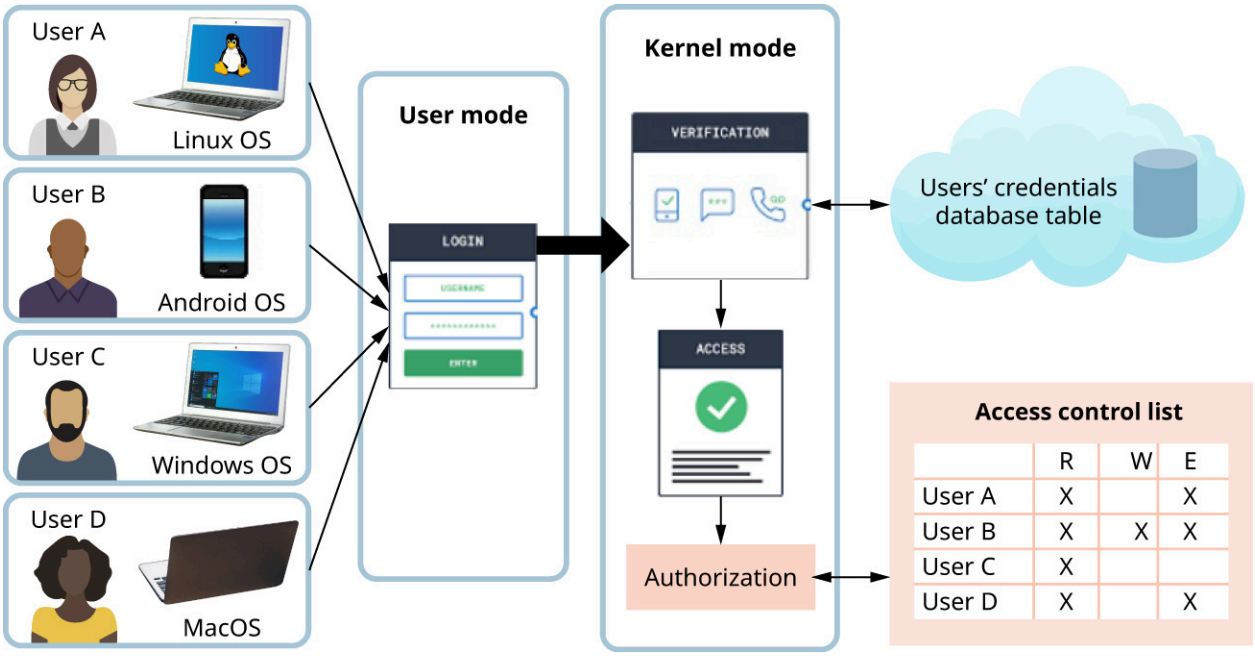


Figure 6.2 These days, organizations, such as companies and universities, must enable a variety of users using a variety of devices supported by a variety of operating systems to securely access their systems. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The goal of this section is to explain what an OS is. Traditionally, we think of the OS as the means by which communication between applications that the user wants to run and the computer hardware that is responsible for running them is facilitated. Applications typically do not directly manipulate the hardware; instead, they must ask the OS to do this on their behalf. One of the primary goals of the OS from an application software point of view is **isolation**. Isolation ensures that the multiple programs that are running

concurrently on the same CPU and memory operate independently without interfering with each other's execution or data. This maintains system stability and security. From a hardware point of view, the hardware is designed to enable programmers to write OSs and applications. In general, the OS mediates a program's access to hardware resources such as the CPU (for computation); memory (for volatile storage) and disk, flash memory, and so on (for persistent storage); TCP/IP stacks, Wi-Fi, and Ethernet network interface cards, and so forth (for network communications); and keyboard, display, touch screen, audio, game controllers, and so on (for input/output). Isolation is what allows device manufacturers to update their hardware without requiring software developers to rewrite their programs each time.

LINK TO LEARNING

To learn more about operating systems, read this informative article on understanding operating systems (https://openstax.org/r/76opsys) from How-To Geek.

Monitoring Access

Operating systems play a crucial role in managing and controlling access to hardware devices within computer systems, including peripherals such as keyboards, mice, hard drives, and other critical components. They act as gatekeepers, ensuring that interactions with these devices occur smoothly and securely. One of the key responsibilities of an OS is monitoring hardware access, which involves tracking and regulating which applications or users can communicate with the hardware and how they do so. This capability is vital for maintaining a system's overall integrity and performance.

In addition to hardware monitoring, the OS is instrumental in enforcing authentication mechanisms. This means the OS helps verify that only authorized users gain access to certain devices, especially those considered sensitive. By managing user permissions and access levels, the OS can prevent unauthorized access to critical hardware resources, safeguarding against potential security breaches or data theft. This aspect of the OS is particularly important in environments where access to information and resources needs to be tightly controlled, such as in corporate or government settings.

The role of an IT administrator is to define which hardware devices are available within a company's network and determine their sensitivity levels. This task requires a deep understanding of both the technical specifications of the hardware and the security implications of its use.

You may be wondering: How is the OS organized? How are resources shared across users? How is one user or process protected from another? Such questions relate to the characteristics or **properties** that are considered when designing an OS. Table 6.1 depicts various OS design questions and the properties they are associated with, such as structure, sharing, naming, protection, security, performance, availability, and reliability, among others.

Property	Question
Structure	How is the OS organized?
Sharing	How are resources shared across users?
Naming	How are resources named, and what is the scope?
Protection	How is one user or process protected from another?

Table 6.1 Operating System Design Considerations

Property	Question
Security	How is the integrity of the OS and its resources ensured?
Performance	How does an OS avoid making all the applications run slowly?
Availability	Can the applications always access the services they need?
Reliability	How often do things go wrong either with the hardware or with a program?
Extensibility	Can new features be added?
Communication	How do programs exchange information, including across a network?
Concurrency	How are simultaneous activities such as computation and I/O created and controlled?
Scale	What happens as demands or resources increase?
Persistence	How can data be made to last longer than program executions?
Distribution	How can a computation be allowed to span hardware, such as machine/network, boundaries?
Accounting	How can a user's resource usage of an OS be tracked, and how might the user be charged for it?
Auditing	Can actions and processes be reconstructed?

Table 6.1 Operating System Design Considerations

Efficiency Management

By managing system resources (e.g., CPU, memory, disk, and network) efficiently, the OS ensures that no resource is underutilized or overburdened. We start with this question: What level of ease or complexity is involved in developing applications with optimal efficiency on a computer? A well-performing operating system facilitates the development of efficient applications that enhance the user experience by providing faster response times. The OS also impacts user experience positively by managing resource allocation and multitasking efficiently.

At some point, you might work for an organization that is engaged in developing a new app, and the company may ask itself this question: If we were to take the time to develop all the additional software required for our application to boot and run on raw hardware, how much faster would it be? This is the same as asking what the penalty or cost is of developing what the OS provides (i.e., sharing of the hardware among apps, and limited damage when programs have defects). The answer to this question relates to runtime efficiency. With respect to coding time efficiency, the OS includes various abstractions, interface and libraries that application programmers can use to ease the burden of software development. Having to write these from scratch without the support of an operating system would greatly increase the required coding effort and extend the development time associated with application development.

Mechanisms Implementation

In operating systems, a **policy** is a way to choose which activities to perform (i.e., what needs to be done) and a **mechanism** is an activity that enforces policies (i.e., how to do it), which often depend on the hardware on which the operating system runs. If a process is granted resources using the first come, first served policy, then that policy may be implemented using a queue of requests. To understand how policies and mechanisms relate, consider this analogy. A car is a mechanism because it enables operations such as going, stopping, and turning that enable a driver to get from point A to point B. Notice that the mechanism (in this case, the car) does not say anything about how to use the mechanism to get anywhere in particular. The driver provides a policy by deciding on a route. Now compare a car with a public bus. In this case, the bus driver determines the policy. The same goes for the self-driving cars that are being developed these days. They serve as the mechanism as there are no decisions about routes being built into cars. Instead, a self-driving car uses machine learning to determine a policy, which includes deciding on a route, selecting the procedure on how to drive, and allowing the driver to override automated driving at any time.

OS-Level and Server Virtualization

The ability of a system or server to run different types of applications used by multiple users at a time on the same computer is called **virtualization**. Server virtualization places a software layer called a **hypervisor** (i.e., virtual machine monitor, or VMM) between a machine's (i.e., server's) hardware and the operating systems that run on it. The hypervisor creates and manages virtual machines. A **virtual machine (VM)** is software that is created to run like a physical computer and that operates its operating system and applications like separate physical servers. OS-level virtualization is a basic form of server virtualization. When using OS-level virtualization, there is no need for a hypervisor as the server's OS handles all resources.

Cloud computing leverages server virtualization and makes it possible for users to commission virtual machines. In this case, the servers reside in distributed data centers provided by cloud vendors. Commissioning a virtual machine is equivalent to gaining access to a virtual computer that runs a chosen operating system while leasing the underlying hardware and only paying per use (i.e., cost of CPU, storage, and network usage).

OS Architecture and Support Layers

As you've now learned, the OS is a system software program whose job it is to manage all the programs running on a computer. OSs enable users to run multiple applications at the same time and keep them from interfering with or crashing each other. OSs provide convenient abstractions to handle diverse hardware. They coordinate resources and protect users from each other using a few critical hardware mechanisms. OSs make it easier for developers to create applications by offering built-in features that help with managing errors. These features include **fault containment**, which prevents errors in one part of an application from affecting the whole system; **fault tolerance**, which allows the application to keep running even when errors occur; and **fault recovery**, which helps the system to fix itself or revert to a previous state after an error. The fact that these standard services are provided by the OS means that developers do not have to build these error management features from scratch, which simplifies the development process. Figure 6.3 shows the support layers within the UNIX/Linux system structure. The OS navigates between three layers: the user mode, where the application resides; the kernel mode, which is in effect sandwiched between the user and hardware; and the hardware, which refers to resources such as the CPU and memory. In the next sections, we will describe these modes and the features they support in detail.

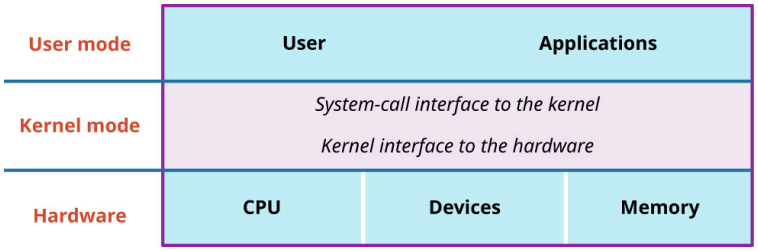


Figure 6.3 The UNIX/Linux system structure is made up of three main parts: (1) the user mode, (2) the kernel mode, and (3) the hardware. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

Now that you've been introduced to the basics of operating systems, it may come as no surprise to you that operating systems come in all shapes and sizes. There are many different types of OSs (from batch operating systems to network operating systems), and each type handles many different functions (from file management to device management). This OS tutorial (https://openstax.org/r/76OStutorial) explains the different types. As you read along, consider how much you interact with an OS over the course of a single day—or how many different OSs you interact with.

OS Kernel Features

The kernel is the program that is running at all times on the computer and provides basic services for all parts of the OS. Typically, the kernel is not the only program that is running; there is usually either a system program/operating system or an application program running as well. Any program running on top of the OS is considered to be a **process** from the OS standpoint. It includes the program code, its current activity represented by the program counter, and a set of resources like open files and allocated memory. It also consists of address space, one or more threads of control executing in that address space, and an additional system state associated with it. The operating system uses processes to manage the execution of programs, ensuring they have the necessary resources while keeping them separate from each other to prevent interference.

Software applications have to be compiled and linked with system libraries before they can run as executable programs on a machine. Each running program runs in its own process, and the OS can run, switch, and isolate processes from each other even though they are actually running on the same hardware. For a given application, the "machine" is the process abstraction provided by the OS. Processes provide user-friendly interfaces, rather than raw hardware, and an execution environment with restricted rights controlled by the OS.

An OS virtualizes the machine by providing easy-to-use abstractions of physical resources while masking limitations. In this context, a **thread** is a path of execution within a process, and a process may contain multiple threads. Thus, multithreading involves executing multiple threads (i.e., execution units that are part of a process and share the same resources) concurrently, which improves overall responsiveness and efficiency.

INDUSTRY SPOTLIGHT

Operating Systems and Health Care

In the fast-moving world of IT, it is crucial to think globally. This global perspective can be particularly useful when considering how IT and OSs affect health care. Namely, it can help us understand the role of OSs in patient care, medical research, and the efforts being made to make health care more widely accessible. The challenges associated with data sharing, privacy, and health-care disparities impact regions around the

world differently.

As the delivery of health care becomes more integrated with technology, the choices made in designing how health-care IT systems use OSs can significantly impact people's lives across the globe. These choices range from deciding how to take ethical considerations into account when using AI for diagnosis to creating IT solutions that respect cultural differences.

Check out these initiatives from the World Health Organization (https://openstax.org/r/76WHO) and Centers for Disease Control and Prevention (https://openstax.org/r/76CDC) for examples in the health-care industry.

Imagine you're considering entering the health-care IT field. Can you think of some ways that thinking globally and understanding the capabilities provided by OSs could improve your ability to solve problems and create solutions that work well for health-care systems in various countries?

Hardware Management

The ISA defines how the CPU is controlled by the software by abstracting the hardware details from the applications. The OS provides an abstract machine interface to the application programs and leverages the physical machine interface to do so. The OS communicates with input/output (I/O) hardware using device drivers, I/O ports, interrupts, direct memory access (DMA), and effective I/O scheduling. It provides abstractions to manipulate files (i.e., streams) and send messages to the network (i.e., sockets). Programming languages provide application programming interfaces (APIs) that leverage these abstractions (e.g., file I/O, socket libraries, and related APIs) so that the application program can access the underlying resources that are managed by the OS. One of the main responsibilities of the OS is to isolate hardware from programs by providing common services and background management functionality, for example, storage manager, network manager, and power manager.

The OS is the only system that should be able to directly access I/O devices (i.e., disks, network cards) and manipulate memory. Moreover, the CPU hardware provides a **privileged instruction** that can only be executed by the OS. The OS can use these instructions to establish an execution environment that limits access (to, for example, memory). The application cannot remove the restrictions because it must execute privileged instructions to do so.

Certain operations are prohibited when running in user mode, such as changing the page table pointer (i.e., the pointers to memory pages that are cached for faster access), disabling interrupts (i.e., the interrupts that the OS received from I/O devices), interacting directly with hardware, and writing to kernel memory. Carefully controlled transitions between user mode and kernel mode include system calls, interrupts, and exceptions. The **system call** appears when the program requests a service from the kernel. The **system interrupt** manages the communication between the computer hardware and the system. The system throws an **exception**, which is an error that occurs at runtime.

THINK IT THROUGH

The Ethics of Open-Source OSs

The idea of making all operating systems open-source (i.e., the copyright holder releases the content or product under a license that allows any user to access, modify, and distribute it freely) has both advantages and disadvantages. On the plus side, open-source operating systems can be more secure and innovative because anyone can inspect and improve the code. This openness also encourages a global community of developers to collaborate, potentially leading to a technology that is more user-friendly and accessible for

everyone. Additionally, openness aligns with ethical principles of transparency and freedom, as it allows users to understand and control their digital environments fully.

There are, however, downsides too. Open-source projects might struggle to secure consistent funding and professional support, which can lead to them having slower updates and fixes compared to commercial software. There's also the risk of fragmentation, where too many variations of the system can create compatibility issues and confuse users.

From an ethical standpoint, the use of open-source operating systems globally can democratize access to technology, ensuring that no single company has too much control over our digital lives. It also encourages a culture of sharing and collaboration, which is essential for addressing global challenges like digital divide and ensuring equitable access to technology. However, the success of such a model depends on balancing openness with the need for sustainable development and support systems to keep the technology reliable and up to date.

Should operating systems all be open source? Discuss pros and cons.

Protected Sharing

An OS's functions should guarantee protection, isolation, and the sharing of resources efficiently via resource allocation and communication. To implement protected sharing, the OS provides common services (e.g., sharing and authorization). There are many ways to leverage OS sharing. One involves the sharing of processors to perform computations concurrently. In this case, these computations will be completed as if only one processor had been allocated to them although in reality multiple processors are performing the computations/tasks in parallel. An OS also allows a computer's memory, input and output devices, and files to be shared within the tasks and the processes. It can also allow groups of computers to work together within the network and share resources. All of the sharing capabilities are controlled using secured channels.

6.2 Fundamental OS Concepts

Learning Objectives

By the end of this section, you will be able to:

- Explain various key concepts and components of operating systems
- Discuss the various designs of operating systems

An OS manages computer resources (hardware) and provides services for computer programs (software). The OS works as an interface between the computer user and the system. The OS manages the memory, the files, the hardware, and the software; it also handles the inputs and the outputs such as the keyboard and printer (Figure 6.4).

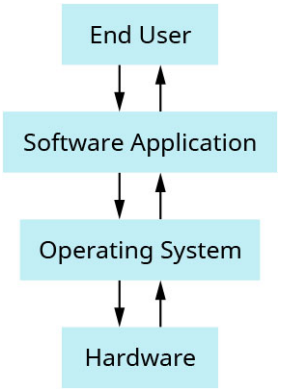


Figure 6.4 The end user initiates this process by using the software/applications, which are built on top of the operating system in

the computer. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

In this module, we study OS components such as process management and threads, memory and address space management, and device drivers and I/O devices. In addition, we cover OS structures such as monolithic OS design, layered OS design, hardware abstraction layer (HAL), and microkernels.

OS Components

OS is a complex system that is typically created using the divide and conquer mechanism. That is, the system is divided into small pieces, each of which defines a part of the system. The OS component's structure is static, and the OS and the hardware are tightly coupled together. The **application programming interface (API)** is a set of rules and tools that allows different software applications to communicate with each other. Applications make requests to the OS through API. The user can, using a keyboard and/or mouse, interact with the OS through the OS interface. The OS interface could be a **graphical user interface (GUI)**, which allows users to interact with electronic devices through graphical icons and visual indicators (e.g., Windows), or a command line (e.g., DOS). For example, in Figure 6.5, the OS interface would be the window into which the Chrome browser opens.

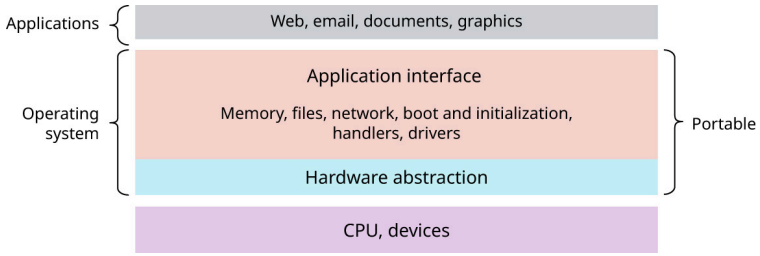


Figure 6.5 The operating system and hardware are tightly linked. The end user is using multiple applications, such as Chrome, Photoshop, Acrobat, and JVM, which they had previously downloaded in a particular OS. To use these software programs, the user will interact through the OS interface and/or API, which is connected with OS components such as file systems, memory managers, process managers, network supports, device drivers, and interrupt handlers. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Process Management and Threads

With a single click, end users seamlessly launch applications, enjoying the simplicity of the process. Have you ever wondered what manages the application once it is launched? The direct answer is the OS. An OS executes many kinds of activities starting from users' programs, background jobs or scripts, to system programs. There are many system programs such as print managers, name servers, and file servers. Each one of these programs or activities is encapsulated within a process. As you learned in the preceding section, a process is a running program that has an execution context plus the runtime instance of the program itself. Examples of an execution context are program counters (PCs), registers, VM maps, OS resources, and examples of runtime instance are code and data.

Here are some other process-related concepts that will be introduced here and developed as the chapter progresses:

- A register is a high-speed memory storing unit.
- A process can be in **running state** (when it is executed by the CPU), **ready state** (when it is waiting in the CPU), or **blocked state** (when it is waiting for an event to occur).
- The OS's process module manages the processes via creation, destruction, and scheduling. One way an OS controls access to a resource is through a data type called a **semaphore**.
- Managing the sharing of a system's resources to avoid interference and errors is called **process synchronization**.

THINK IT THROUGH

Failure Existence

Given the complexity of an OS's tasks (i.e., the management of a computer system's resources and the scheduling of tasks to make an application do what the user expects it to do) and the speed at which these tasks need to happen, it's amazing how often technology *doesn't* fail. But, of course, it does at times.

Let's suppose you ran into an OS-related failure this morning—namely, your laptop did not fully boot. What are some steps you would take to diagnose the problem on your own before seeking help or a repair?

Processes vs. Threads

As we have learned, a process is an active program. A thread is a smaller or lightweight portion of a process that is managed independently. Table 6.2 shows a comparison between the process and the thread.

Attribute	Process	Thread
Definition	An executed program	Part of the process
Weight	It could be heavy	Lightweight
Processing time	More time	Less time
Resources	Needs more resources	Needs fewer resources
Sharing	Mostly isolated	Shares memory and data

Table 6.2 Process vs Thread

Processing

A program is passive; it is just bytes on a disk that encode instructions to be run. A process is an instance of a program being executed—or processed—by a processor. Thus, processing involves a program, a process, and a processor. The processor can be a real processor or a virtual processor (i.e., CPU core assigned to a virtual machine). At any time, there may be many processes running copies of the same program (e.g., an editor); each process is separate and usually independent. An OS is responsible for managing these processes.

Different OSs approach process management in different ways. For example, the Windows operating system's approach consists of adding an operating system call to create a process and other system calls for process operation. The approach used in the UNIX OS is different from this as it consists of splitting the process into two steps using fork and exec functionalities. The fork functionality is used to set up privileges by creating a complete copy of the process, and the exec functionality brings the executable file into memory to start the execution.

Address Space and Memory Space Management

The **address space** is the set of addresses generated by programs as they reference instructions and data. The memory space holds the actual main memory locations that are directly addressable for processing. The OS is responsible for managing these memory and address spaces (Figure 6.6). To enhance performance, computers use virtual memory address space to create the illusion of a large and continuous block of memory for applications and the operating system to utilize. To do so, the computer's physical memory is used in

combination with a portion of a hard drive that contains the swap file or page file. Pages containing address space information for programs are moved in and out of physical memory as necessary to ensure that there is enough physical memory to hold the pages of programs that are running at a given time.

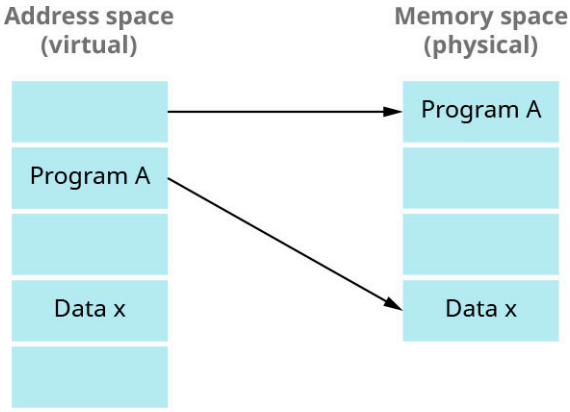


Figure 6.6 The addresses of the data will be in the address space waiting for the execution to be moved to the memory space. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Computer memory consists of two main types: primary and secondary memory. The initial point of access for a processor that also serves as direct storage for the CPU is called **primary memory**. To be executed, programs must reside in primary memory. In multiprocessor systems, primary memory can be classified into three architectures. The first, **uniform memory access (UMA)**, employs a single memory controller and, thus, access time to any memory location is the same across all processors. The second, **non-uniform memory access (NUMA)**, is a computer memory design that uses different memory controllers, thus its memory access time varies depending on the memory's location relative to a processor. The third, **cache-only memory architecture (COMA)**, uses multiple interconnected processing nodes, each equipped with a processor (i.e., a cache). This allows for the dynamic allocation of data for optimized access and performance in multiprocessor environments.

Memory that is used for long-term storage, housing the operating system, applications, and data that need to persist even when the power is off, is called **secondary memory**. Unlike primary memory, which is volatile and loses data during power interruptions, secondary memory is nonvolatile—meaning it retains data even during power failures—and thus it provides durability and data retention. Common examples of secondary memory include hard disk drives (HDDs) and solid-state drives (SSDs).

An OS must satisfy the policies of how much physical memory to allocate to each process and when to remove a process from memory. It must implement these policies using the following mechanisms: **memory allocation** and **memory deallocation**. Memory allocation is the process of setting aside sections of memory in a program to be used to store variables and instances of structures and classes. The memory allocation can be static memory allocation or dynamic memory allocation. Memory deallocation is the process of freeing the space corresponding to finished processes when that space is needed by the rest of the system. In addition, the OS must maintain mappings from virtual addresses to physical (i.e., page tables) and switch CPU context among address spaces.

INDUSTRY SPOTLIGHT

Windowing Systems

A windowing system is an OS software component that manages the display of graphical user interfaces (GUIs) on a computer screen. An investigation into the influence of windowing systems on various industries, particularly in sectors like retail marketing, has yielded insightful perspectives on how such systems impact business practices, productivity, customer engagement, and overall operational

effectiveness. Namely, it's been found that windowing systems significantly enhance user experience, which is critical to retail marketing. By making it easier to integrate marketing messages or interruptions with other media that customers are watching, a windowing system helps retailers conduct targeted marketing over the Internet. Windowing systems also improve the overall user experience for customers viewing the integrated content.

Device Drivers and I/O Devices

Computers have many input and output devices such as the keyboard, mouse, display, or USB port. Some examples of OS-specific devices include file system (disk), sockets (network), and frame buffer (video). A **frame buffer** is a portion of random access memory (RAM) containing a bitmap that drives a video display. A big chunk of the OS kernel deals with I/O (Input/Output). The OS provides a standard interface between programs/users and devices to communicate with them (Figure 6.7). A device driver's routines interact directly with specific device types and related hardware to initialize the device, request I/O, and respond to interrupts or errors. An **interrupt** is a signal to the processor from either software or hardware that indicate events that needs immediate attention. Examples of device drivers include Ethernet card drivers, video card drivers, sound card drivers, and PCIe (Peripheral Component Interconnect Express) device drivers, which are associated with graphics cards and other peripherals. Device drivers are implemented by device manufacturers or open-source contributors and support a standard, internal interface. They can execute in the OS address space and run at high privilege.

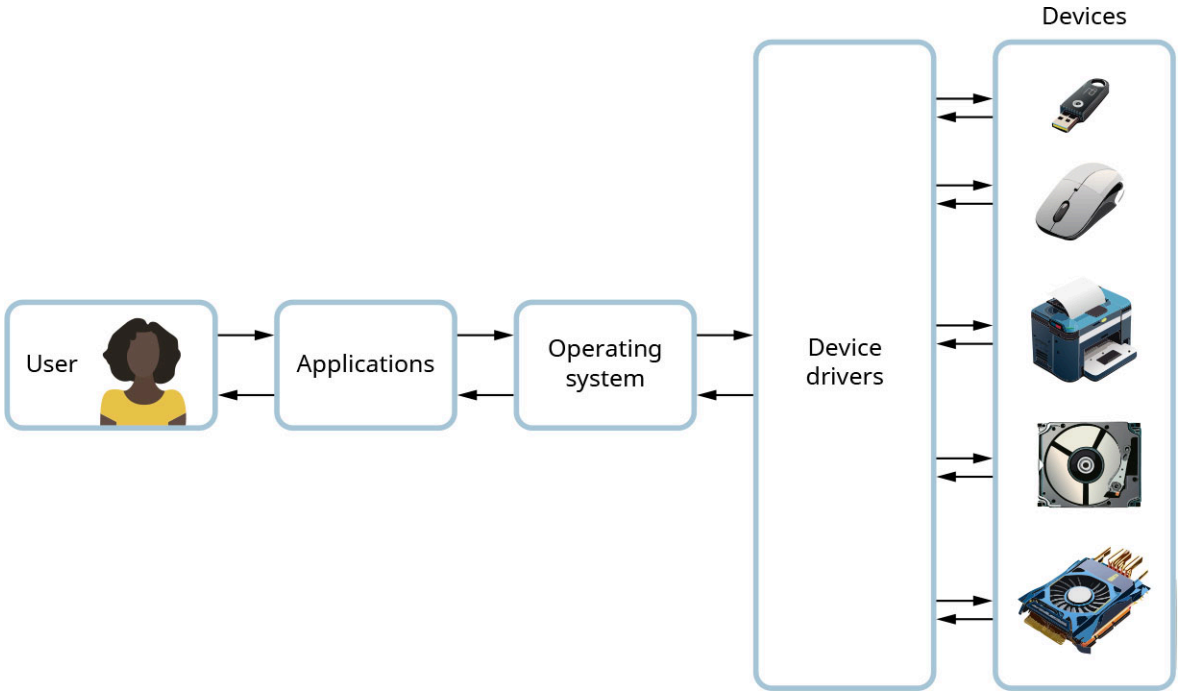


Figure 6.7 Multiple I/O devices are typically connected to a computer. When the user starts using an application, the operating system will define the devices used by the application by using each device's driver. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

Are your Windows drivers up to date? Check out to see if you have the latest drivers (https://openstax.org/r/76latestdriver) for your Microsoft devices.

Device drivers are specialized software components that enable higher-level computer programs to interact with hardware devices. These drivers provide a software interface to hardware devices, allowing operating systems and other computer applications to access hardware functions without needing to know precise details about the hardware being used.

- At the heart of any computer is the CPU, the primary component responsible for executing instructions. Interestingly, CPUs themselves do not typically require external device drivers for direct operation, as the core management of CPU resources is a fundamental role of the operating system.
- Memory operates under the direct management of the operating system, which allocates and manages the system's memory resources. RAM can be accessed randomly and used for storing data temporarily while a computer is running. While standard RAM modules—both dynamic and static RAM—do not necessitate distinct drivers, specialized memory hardware, such as flash memory devices, including solid-state drives (SSDs), USB flash drives, and memory cards, interact with the system through file system drivers that manage the organization and access of stored data.
- Storage devices, encompassing a broad range of hardware from traditional hard disk drives (HDDs) to modern SSDs and removable storage media, require device drivers to facilitate data read/write operations.
- Network connectivity relies on an array of device drivers designed to manage the protocols and hardware functions of network interfaces.

Device Registers

A **device register** is the interface a device presents to a programmer where each I/O device appears in the physical address space of the machine as a memory address. The operating system reads and writes device registers to control the device. There are three types of device registers: status, command, and data. The status register provides information about the current state of the device (e.g., read), the command register issues a command to the device (e.g., writing a value to the register), and the data register is used to transfer data between the computer and the device. Device registers use bits to service three purposes: parameters provided by the CPU to the device, status bits provided by the device, and control bits set by the CPU to initiate operations. Figure 6.8 provides an example of the content of each of a device register's bits.

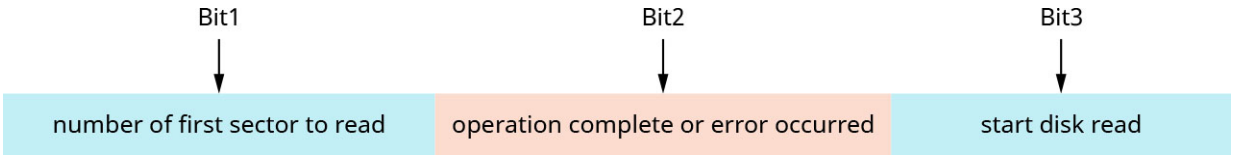


Figure 6.8 Device register bits define the first sector to read, the status of the operation, and the operation itself. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The behavior of device registers is different from that of ordinary memory locations. For example, the start operation bit may always read as 0, and the bits may change from one status to another (i.e., the operation complete bit may change without being written by the CPU).

The OS uses the device register to communicate with an I/O device as follows:

1. The CPU writes device registers to start the operation.
2. The CPU polls the ready bit in the device register.
3. The device sets the ready bit when the operation is finished.
4. The CPU loads the buffer address into a device register before starting the operation to define where to copy data read from disk.
5. Fast storage media devices move data directly to/from physical memory via direct memory access (DMA); other devices require the intervention of the CPU via programmed I/O or interrupt initiated I/O.
6. Interrupts allow the CPU to do other work while devices are operating, and the OS figures out which device interrupted.

CONCEPTS IN PRACTICE

Operating Systems, Printing, and Networking

Pretty much anyone needs to print a document these days. It is therefore important to understand how an operating system enables us to print documents so easily. When an application wants to print a document, it hands that task off to the operating system. The operating system sends instructions to the printer's drivers, which then send the correct signals to the printer.

Being able to access the network is another critical need in today's business world. It is therefore important to understand how OSs facilitate access to the network and how they control it. When a user uses an application that interacts with the Internet, the application sends messages from process to process using the OS's transport layer socket API. These messages are then split into packets within the operating system and eventually passed to a network interface card device, which transmits them to the Internet.

Figure 6.9 depicts the device register for a keyboard. Note that it has 2 bytes (16 bits). The data is in the first byte, and the second byte includes all zeros (i.e., from bit 8 to bit 15). When the user starts typing, the ready bit, which is bit number 15, sets to 1.



Figure 6.9 In this device register for a keyboard, the ready bit is set to 1 for when a user starts using the keyboard. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

OS Structure

In this section, we learn about the organization and structure of the OS. As you may recall, an OS consists of many core components but also includes other components such as privileged and non-privileged system programs. In an OS, it is important to have dual mode operations to ensure a high level of security and authority. The **dual mode** is responsible for separating the user mode from the kernel mode. A program that can run only in the kernel mode is known as a **privileged system program**. An example of a privileged system program is the bootstrap code, which is the first code that is executed when the OS starts. The OS depends on the bootstrap to be loaded and to work correctly. Figure 6.10 demonstrates the bootstrap use. If the user attempts to make any execution on privileged systems instructions, the execution will not be performed.

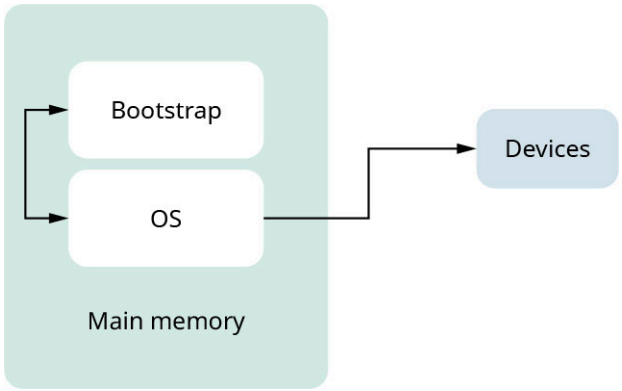


Figure 6.10 To start using any device, the OS will use the bootstrap program. (attribution: Copyright Rice University, OpenStax, under

CC BY NC-SA 4.0 license)

A program that can run only in the user mode is called a **non-privileged system program**. An example of a non-privileged system program is reading the processor status and the system time. In general, this is an instruction that any application or user can execute.

Designing a large, complex program is a major software engineering challenge because the program must not only perform well, but it must also be reliable, extensible, and backward compatible. OS design has been an evolutionary trial and error process. Successful OS designs have had a variety of architectures, such as monolithic, layered, cloud infrastructure and exokernels, microkernels, and virtual machine monitors. As the design of an OS—and even its role—are still evolving, it is simply impossible today to pick one correct way to structure an OS. The choice of OS architecture depends on various factors, including the specific requirements, trade-offs, and goals of the OS's intended use.

Monolithic OS Design

A **monolithic design** refers to a specific architecture for OSs where the entire OS operates within the kernel space, and all components and functionalities of the operating system are organized within the space. In a monolithic architecture, the entire operating system functions as a single, integrated unit, and all of its components, such as process management, file systems, device drivers, and memory management, reside and operate within the same address space, known as the kernel space. This means that the entire OS operates as a single, large program, and any module or component can directly call the functions of another without any restrictions.

While a monolithic design simplifies the communication and interaction between OS components, it has both advantages and disadvantages. One advantage is that it generally provides efficient and fast communication between different parts of the OS because they share the same address space. Another major advantage of the monolithic design is that it uses a familiar architectural style, and the cost of module interactions in terms of procedure calls is low. However, there are many disadvantages. Namely, this structure is hard to understand, modify, or maintain, and does not support fault containment. A failure in any one component can potentially crash the entire system, making it less fault-tolerant compared to more modular architectures. In this case, the alternative is to find an organizational way to simplify the OS design and implementation.

Traditional OSs such as UNIX (Figure 6.11) were built using the monolithic architecture. In contrast to monolithic designs, other OS architectures such as microkernel or hybrid designs distribute OS functionalities into separate modules or user-space processes, leading to better modularity and potentially improved system stability.

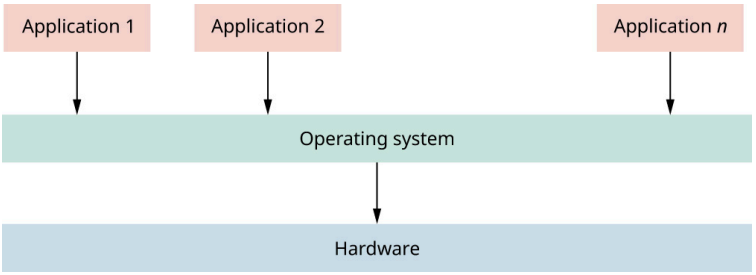


Figure 6.11 The entire OS works as one piece in the monolithic architecture. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Layered OS Design

One alternative way to achieve monolithic OS design is the **layered OS architecture**, which consists of implementing the OS as a set of layers where each layer exposes an enhanced "virtual machine" to the layer above, as illustrated in Figure 6.12.

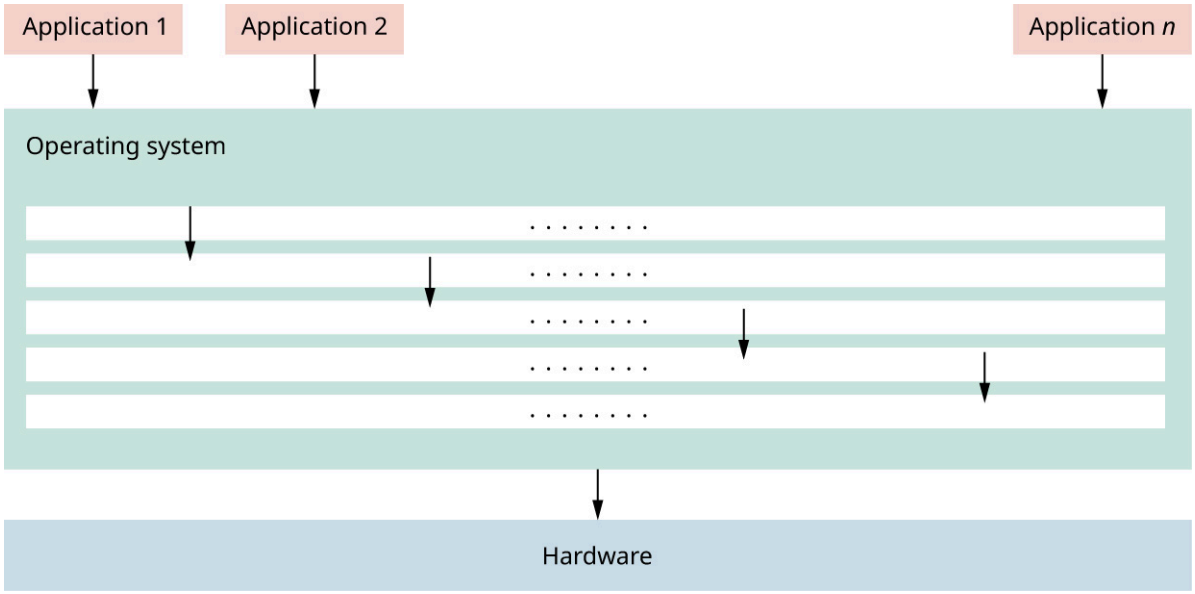


Figure 6.12 In a layered OS architecture, the OS is divided into layers, and each layer will be responsible for a specific task. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The first example of a layered approach was Dijkstra's THE system, which was designed in 1968. There were six layers, and they were organized as follows.

- Layer 5: A job manager executes users' programs.
- Layer 4: A **device manager** handles devices and provides buffering.
- Layer 3: A console manager implements virtual consoles.
- Layer 2: A **page manager** implements virtual memories for each process.
- Layer 1: A kernel implements a virtual processor for each process.
- Layer 0: Hardware is the physical components of the computer.

Each layer in this setup can be tested and verified independently. Layering also helped with implementation and supported attempts at formal verification of correctness (if you can call only layers below, it is not possible to run into a loop across the various layers).

There are, however, many disadvantages to using the layered OS architecture. Namely, it imposes a hierarchical structure, while real systems are more complex because a file system requires virtual memory (VM) services, and the VM likes to use files for its backing store. Layering also imposes a performance penalty as each layer crossing has overhead associated with static versus dynamic enforcement of invocation restrictions. There is also a disconnect between model and reality as systems modeled as layers may not really be built that way.

Hardware Abstraction

The **hardware abstraction layer (HAL)** is an example of layering in modern OSs, and it allows an OS to interact with a hardware device at a general or abstract level rather than going deep into a detailed hardware level, which improves readability and maintainability. In general, the goal of HAL is to separate hardware-specific routines from the core OS. HAL enables portability of core code across different pieces of hardware.

Microkernels

Another alternative OS design to monolithic OS design is a microkernels architecture. Within a **microkernel**, the functionality and capabilities are added to a minimal core OS as plug-ins. Microkernel architecture is also called a plug-in architecture because functionalities are added as plug-ins (Figure 6.13).

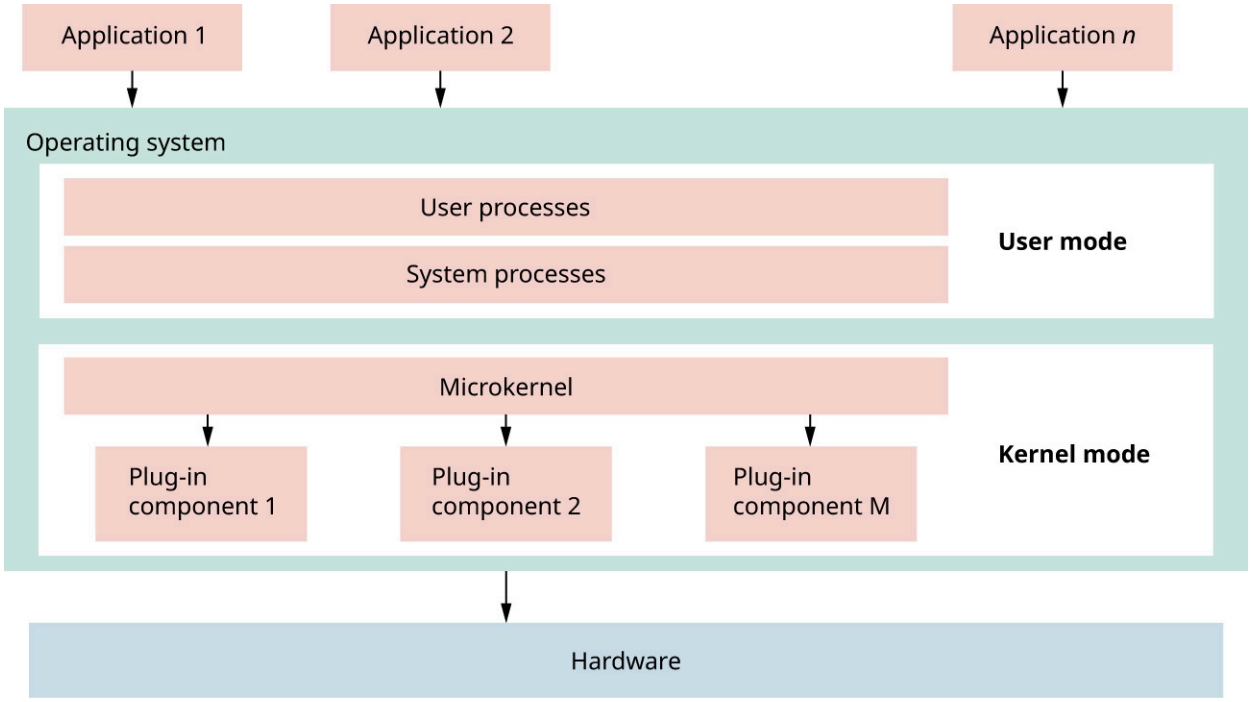


Figure 6.13 In the microkernel architecture, kernel mode is divided into multiple plug-ins to process the operations. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The goal of a microkernel architecture is to minimize what goes into the kernel and implement everything else that traditionally goes in an OS in terms of user-level processes. This results in improving reliability due to the isolation between components. Also, there is less code running at full privilege, and greater ease of extension and customization. However, performance is generally poor due to user and kernel boundary crossings, which represent a security risk when the kernel code operates on the data.

The first microkernel system was Hydra (CMU, 1970), followed by Mach (CMU), Chorus (French UNIX-like OS), and OS X (Apple). Windows OS used to use a microkernel, but now uses a hybrid kernel architecture that combines the benefits of monolithic, microkernel, and plug-in OS architectures (Figure 6.14).

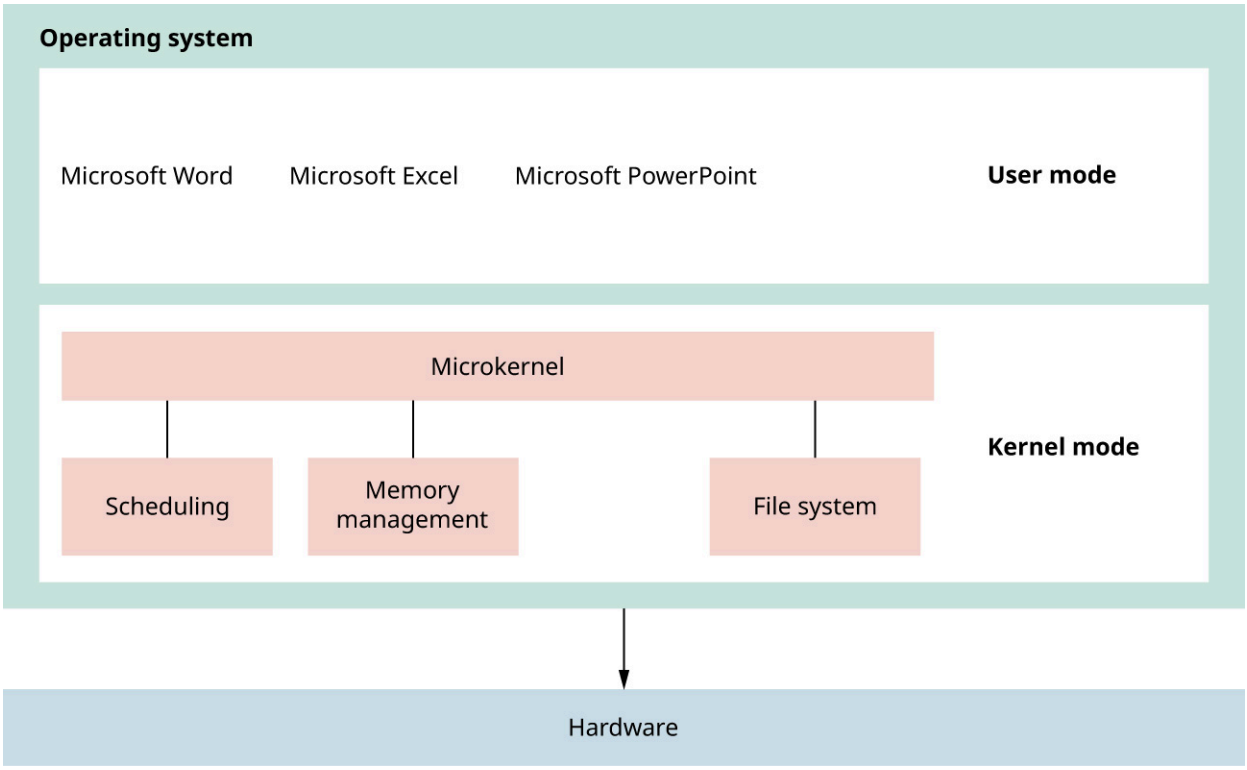


Figure 6.14 Windows OS as plug-in architecture—in Windows OS, all of the applications are in the user mode, and the OS operations are divided into plug-ins in kernel mode. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

Read this article about how Apple uses kernel architecture (https://openstax.org/r/76kernelarch) in its macOS.

Cloud Infrastructure and Exokernel

Two concepts that each represent critical advancements in the field of computing by offering different methodologies for resource abstraction, allocation, and management are cloud infrastructure and the exokernel.

The **exokernel** architecture simplifies the OS by making its core (kernel) really small and letting apps have more control over the computer's hardware. Unlike usual systems that hide hardware details, an exokernel shows these details to apps, letting them use the hardware more smartly. This way, it cuts down on unnecessary steps and lets each app manage resources its own way, which can make the computer run better and faster. This design is especially good for special computing tasks where apps can really benefit from managing hardware directly, and this can lead to better performance and more efficient use of the computer's parts. There are several limitations associated with exokernels. They lack abstractions for operating systems services, which makes it difficult to achieve consistency across applications. They also require an application developer to manage resource allocation and protection and to implement security mechanisms, which raises concerns.

The virtualized and scalable hardware resources that are delivered over the Internet as a service are called **cloud infrastructure**. This infrastructure encompasses a range of components including servers, storage devices, network equipment, and virtualization software, all hosted within data centers managed by cloud service providers. Unlike traditional physical infrastructure, cloud infrastructure offers flexibility, scalability, and accessibility, allowing users to access and manage computing resources remotely through the Internet. Cloud

infrastructure is categorized into three service models:

- Infrastructure as a Service (IaaS) provides virtualized computing resources over the Internet, such as AWS EC2, Google Compute Engine, and Microsoft Azure VMs.
- Platform as a Service (PaaS) offers hardware and software tools over the Internet, typically targeting developers such as Google App Engine and Microsoft Azure.
- Software as a Service (SaaS) delivers software applications over the Internet, accessible from a web browser without installation or running on the user's personal devices such as Google Workspace and Microsoft Office 365.

The main disadvantages of cloud infrastructure are potential downtime, security and privacy concerns, vulnerability to attacks, limited control and flexibility, vendor lock-in, cost concerns, latency issues, Internet dependency, technical issues, lack of support, bandwidth issues, and varied performance.

TECHNOLOGY IN EVERYDAY LIFE

Remote Emailing

Virtual machines have an impact on every industry today as they facilitate the running of numerous business applications. For example, most companies use Google Mail or Microsoft Outlook to supply their employees with email services. These are Software as a Service (SaaS) cloud infrastructures that operate the email software by making use of virtual machines that run in Google and Microsoft Cloud data centers.

A major advantage of SaaS email might be portability. Because the application can run on any local machine with an appropriate browser, this enables users to use the app on many different platforms, including mobile phones and tablets. In addition, there is an automatic backup of saved data, and no need for complicated installations and configuration. The disadvantages of SaaS might include the lack of security of the data stored in the app with respect to unauthorized access and the inability of users to use the application when the network connection is weak or nonexistent.

6.3 Processes and Concurrency

Learning Objectives

By the end of this section, you will be able to:

- Explain what processes are and how they interact with the operating system
- Discuss how operating systems support concurrency

As we mentioned before, the OS divides the tasks it needs to perform into processes. It would be a waste of time for every process to wait until the current process completes. Instead, the OS performs more than one task at the same time, or concurrently. The computing model that improves performance when multiple processors are executing instructions simultaneously is **concurrent processing**. In this module, we learn about processes and concurrency by digging down into process management and inter-process communication (IPC), threads, scheduling and dispatching, and synchronization.

Process

To review, a process is a fundamental concept in an OS that represents an instance of a program in execution. It's an abstraction used by the OS to provide the environment a program needs to run. When we talk about a program, we are typically referring to a set of instructions stored on disk; these instructions are passive and don't do anything by themselves. However, when the program is loaded into the memory of a computer and begins execution, it becomes an active entity known as a process. This transformation is crucial for any computational task, as it moves the program from a static state into an active one where it can perform

actions, manipulate data, and interact with other processes.

CONCEPTS IN PRACTICE

Under the Cover

Most of the applications we use today on our smartphones or laptops use IPC and a client-server architecture. It is therefore important to understand what is under the cover in case these applications suddenly stop working.

In an OS, client-server communication refers to the exchange of data and services among multiple machines or processes. One process or machine acts as a client requesting a service or data, and another machine or process acts like a server providing those services or data to the client machine. The communication between server and client uses various OS protocols and mechanisms for message passing, including sockets, remote procedure calls, and inter-process communication.

Process Management

A process consists of at least an address space, a CPU state, and a set of OS resources. A process's address space is illustrated in Figure 6.15. As you learned earlier in this chapter, the address space contains the instruction code for the corresponding running program and the data needed by the running program. The data can be **static data**, which does not change within the program, or **heap data**, which serves a collection of elements and uses a tree or stack data structure. A **CPU state** consists of the program counter, the stack pointer (SP), and general-purpose registers. The PC is a CPU register located in the processor that has the next instruction address. The **stack pointer (SP)** is a register that indicates the location of the last item that was added to the stack. A **general-purpose register (GPR)** is an extra register that is used for storing operands and pointers; GPRs are where the instructions can read and write the value of their parameters mostly when the program is interrupted. There are many OS resources such as the CPU, network connections, file storage, I/O, and sound channels. An address space, CPU state, and OS resources are everything you need to run the program or to resume it if it is interrupted at some point.

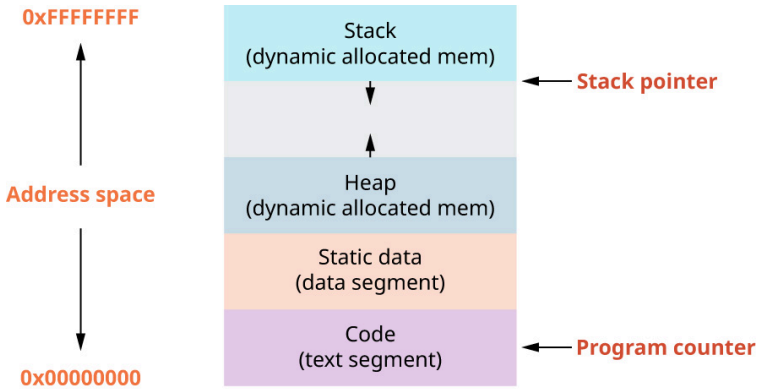


Figure 6.15 A process's address space includes the stack pointer (SP) and the program counter (PC). It has static data in the data segment and code in the text segment. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The OS's process namespace particulars depend on the specific OS, but in general, the name of a process is called a **process ID (PID)**, and it has an integer type. A PID namespace is a set of unique numbers that identify processes. The PID namespace is global to the system, and only one process at a time has a particular PID. It is possible to create a container to isolate a process, along with only the files and configurations necessary to run and operate it. This type of isolated process environment allows for greater security, consistency, and portability across different systems. PID namespaces allow containers to provide functionality such as suspending the set of processes and resuming different set of processes in the memory. The OS maintains a

data structure called the **process control block (PCB)** or process descriptor to keep track of a process state and to store all of the information identified by the PID about a process. As illustrated in Figure 6.16, the PCB contains information that serves as metadata for the process such as PID, process state, parent process ID (PPID), execution state, PC, SP, registers, address space info, and user id (uid).

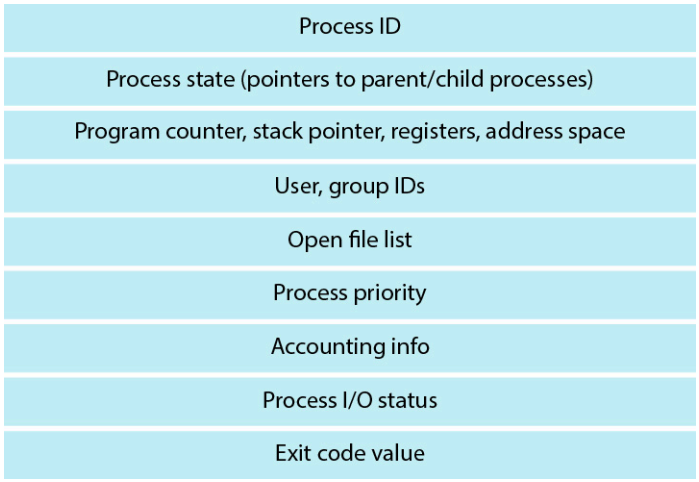


Figure 6.16 PCB are data structures that an OS uses to store detailed attributes for each process it is tracking. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Suppose that there's a process that needs input from a user. Because the OS does not use any CPU while waiting for this input, it marks the process in the PCB as suspended. When the user enters the input, the process's status in the PCB changes. The OS keeps the details relating to all of a process's execution state in the PCB when the process is not running. The CPU state (e.g., PC, SP, GPRs) is transferred out of the hardware registers into the PCB when execution is suspended. When a process is running, its state is spread between the PCB and the CPU.

Here is an example in Linux that illustrates how to use the PID when creating a child process from a parent main process, which uses the fork command in C:

```
#include "unistd.h"
#include <stdio.h>

int main() {
  // Forking to create a new process
  pid_t pid = fork();
  if (pid == 0) {
    // Child process
    printf("This is the child process with PID %d\n", getpid());
  } else if (pid > 0) {
    // Parent process
    printf("This is the parent process with PID %d\n", getpid());
  } else {
    // Fork failed
    printf("Fork failed!\n");
    return 1;
  }
  return 0;
}
```

Inter-Process Communication

Processes provide isolation to guarantee a high level of protection, but sometimes these processes need to communicate and collaborate. This is made possible via **inter-process communication (IPC)**, which is a mechanism that enables different processes running on an operating system to exchange data among themselves and thus allows these processes to communicate and collaborate. As Figure 6.17 shows, IPC allows one process, P_1, to provide input to another process, P_2, while yet another process, P_3, is also running.

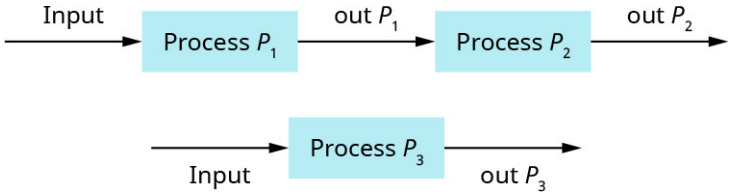


Figure 6.17 In this example of inter-process communication (IPC), the process P_1 has an output called "out P_1" that will be the input that process P_2 needs to start working. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

Read this article on inter-process communication (https://openstax.org/r/76interproc) to learn more about how the processes running in a computer system can be independent or noncooperating.

Streams, Pipes, and Sockets

The IPC has a range of mechanisms that enable processes to communicate with each other such as pipes, shared memory, and sockets. A **pipe** (sometimes called a named pipe) is a data communication method between two processes that uses a specific name and standard I/O operations, and thus allows for data transfer within a file system. Shared memory allows the processes to communicate with each other without a middleman. A socket is an end point for sending and receiving data between different machines in the same network using Internet protocols.

What this means is that, for example in Figure 6.17, where process P_1 provides input to process P_2, there are many ways for the IPC to deliver this input. It could send command-line arguments that are available only to the parent process (i.e., input data is passed to a program when the program is invoked from a shell); or it could communicate via files (e.g., one process writes, the other process reads). Alternatively, the IPC could optimize file communication via the use of pipes with memory buffers (effective when processes are related), or it could utilize environment variables (i.e., variables defined within a shell that can hold a dynamically allocated value).

Concurrency

Multiple activities and processes happening at the same time—in other words, the OS handling multiple tasks at once—is called **concurrency**. Concurrent processing can be achieved via a multiprogramming environment, a multiprocessing environment, or a distributed processing environment (Figure 6.18). In a multiprogramming environment, multiple tasks are shared by one processor. In a multiprocessing environment, two or more processors that have a shared memory are used. In a distributed processing environment, two or more computers are connected by a communication network, and there is no shared memory between the processors (i.e., each computer has its own memory). Multiprocessing is used to accelerate processing by running tasks in parallel, while distributed computing environments are typically used to implement client-server or peer-to-peer architectures. As noted earlier and as will be investigated further in this section, threads are another means of achieving concurrency. However, true parallelism can only be achieved by using multiple processors to execute multiple threads simultaneously.

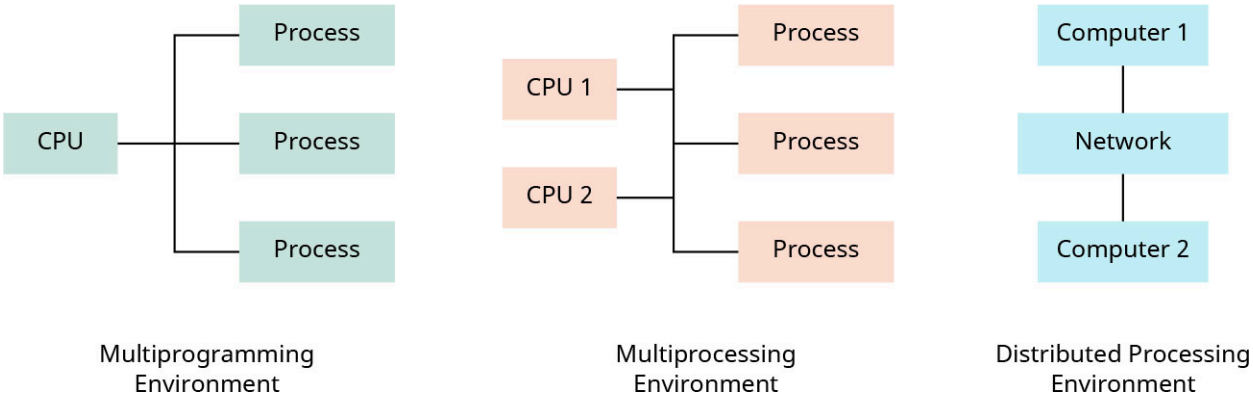


Figure 6.18 Concurrent processing, in which the OS handles multiple tasks at once, can be achieved through three different types of environments. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

Read this article on concurrency in operating systems (https://openstax.org/r/76concurr) to learn more about the principles and problems associated with the concept of concurrency.

Threads

As discussed earlier in this chapter, processes represent running programs, and threads enable programs to perform multiple things at once. A process is an instance of a program that is being executed by an OS. Each process has its own memory space and resources. The OS creates a new process for every executed program and allocates the required resources for the process. For example, it allocates a specific size of memory and CPU time. The process may have one or more threads, each thread has its own context, but all of the threads within a process share the same resources.

Threads are the OS's unit of concurrency and the abstraction of a virtual CPU core. Each thread is a basic unit of execution that executes a set of instructions in a specific order. A thread is a lightweight process that shares OS resources (e.g., memory and I/O) with other threads within the same process. Threads are created by the OS kernel, and each thread has its own register stack. All the threads in a given process are sharing the same memory space. Threads are essentially paths of execution that operate within the confines of a process.

For example, consider today's web browsers. Each open tab in a web browser is its own process with its own address space, but within a tab, there might be multiple things going on. A user can scroll around and interact with a web page while a video is playing in the background. In this case, one thread could be used to manage the user interactions, while another thread is used to manage video playback on the web page.

In the early versions of the operating system used on IBM and DEC mainframe computers, concurrency was achieved via time sharing. In other words, a single task was performed using a single process with a single thread. This kind of process allowed only one user at a time to process or run a job. This old way of processing required more resources such as memory and processors to finish a single task. By the late 1970s, the more prominent approach became **multitasking**, which makes it possible for the OS to run multiple processes at the same time using time slicing. A **time slice** is a short time frame that gets assigned to a process for CPU execution. It corresponds to the time frame during which a process is allotted to run in preemptive multitasking CPUs. In that case, a scheduler runs each process every single time slice. The preemptive multitasking approach was not sufficient to improve the OS performance, so twenty years later, OSs still support multitasking using multiple threads. Figure 6.19 illustrates single and multithreaded processes.

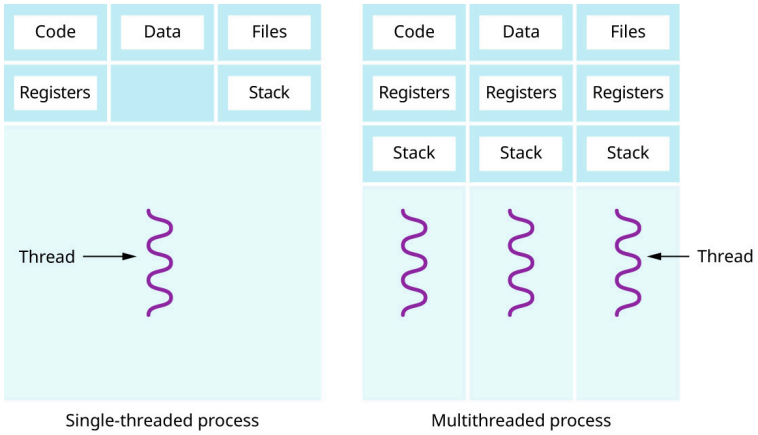


Figure 6.19 In the single-threaded process, a single thread will use its own code, data, and files along with its own registers and stack. In the multithreaded process, a multithread will have a set of registers and stacks. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

When a program is run, it operates as a process in the OS. This process can execute multiple threads simultaneously, allowing for parallel execution of tasks within the same application environment. The threads are managed and run as follows:

- Thread creation: Threads are created by the process using specific system calls to the operating system, such as pthread_create in UNIX/Linux or CreateThread in Windows. When created, each thread starts its execution at a designated start point in the program code. This is often a function passed to the thread creation call.
- Execution and scheduling: Once created, threads are scheduled by the operating system's scheduler, which allocates CPU time to them. The scheduling can be preemptive, where the OS decides when to switch between threads, or cooperative, where threads voluntarily yield control to allow other threads to run.
- Sharing and isolation: Threads share the same process resources, such as memory and file handles, making inter-process communication and data sharing more efficient than between separate processes. However, they run in their own thread of execution, meaning each has its own stack, program counter, and set of registers to keep track of its execution state.
- Synchronization: To safely manage the access to shared resources and ensure data consistency, threads often use synchronization mechanisms like mutexes, semaphores, and condition variables. These tools help prevent race conditions, where the outcome of operations depends on the sequence or timing of other uncontrollable events.
- Completion: A thread completes its execution when it exits its start function, either by returning normally or by being explicitly terminated by itself or another thread. Upon completion, any resources specifically allocated to the thread are cleaned up by the operating system.

Scheduling/Dispatching

Scheduling tasks and determining which resources should be used when are central responsibilities of the OS—they are also the means by which the OS achieves concurrent processing.

OSs may switch the CPU from process to process hundreds of thousands of times per second. On today's hardware, this takes a few microseconds. Choosing which process to run next is called scheduling. The activity of handling the removal of the running process from the CPU and the selection of another process based on a particular strategy is called scheduling. In OSs, a process can be in one of several states. These states are part of the process life cycle, and understanding them is essential for grasping how the OS manages processes. The specific names and number of states can vary between OSs, but the fundamental concepts remain the same—or at least quite similar. Each process has an execution state, which indicates what it is currently doing and can be as follows:

- ready: waiting to be assigned a core

- running: executing on a core
- blocked (also known as “waiting”): waiting for an event, so not eligible to be given a core

Figure 6.20 represents a process’s life cycle within an OS. It starts with the creation of a process, which brings a new process into existence and places it in the ready state. In this state, the process is loaded into memory and is prepared to run, but is waiting for the CPU to become available. When the scheduler dispatches the process, it transitions to the running state, where it is actively executing its instructions. If the process is interrupted, it reverts to the ready state, waiting once again for a chance to run. Certain events, such as I/O requests or page faults, can cause the running process to experience a trap or exception. When this happens, the process enters the blocked state because it can’t proceed until the event it’s waiting for, such as the completion of an I/O operation, occurs. Once the awaited event is finished, the process can leave the blocked state and reenter the ready state, once again waiting for CPU time.

When a process completes its execution or is terminated, it reaches the terminate state. However, termination doesn’t necessarily mean the process is immediately removed from the system; it may enter a zombie state. In this state, the process has finished its job but still occupies an entry in the process table, effectively being in a state of limbo until its parent process acknowledges its completion. This acknowledgment allows the OS to fully clean up any remaining information, officially ending the process’s life cycle.

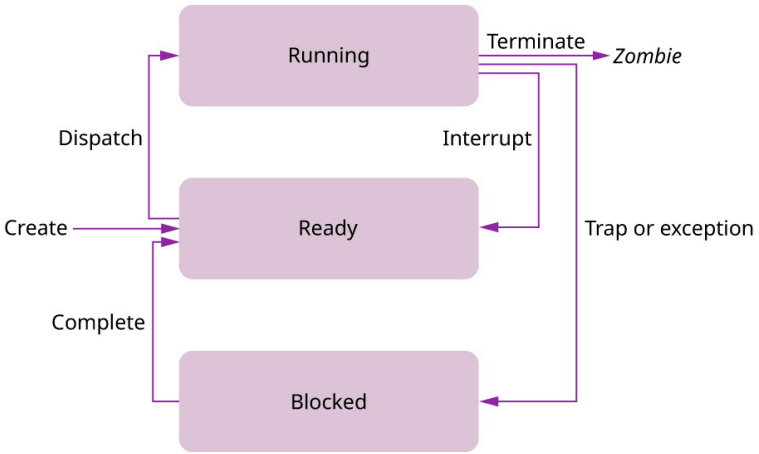


Figure 6.20 The life cycle of a process within an OS starts with the creation of the process and ends with an acknowledgment of completion. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

As noted earlier, each process is represented by a unique identifier called a process identifier (PID). An OS encapsulates the process in a data structure type called a process control block (PCB) that contains information about the process, such as current status, which could be running status, ready status, or blocked status. A PCB defines the register status, the process ID, the execution time, the memory space, as well as other information. The OS kernel scheduling function is responsible for maintaining the content of PCB and scheduling processes for the CPU to execute based on assigned priorities.

Another policy decision the OS makes is to decide whether to give out non-CPU resources such as memory and I/O. As they are data structures, PCBs are located in OS memory. When a process is created, the OS allocates a PCB for it. After initializing the PCB, the OS then does other things not related to the PCB such as allocating the PCB to the correct queue. As a process executes, the OS moves the process’s PCB from queue to queue. When a process is terminated, the PCB may be retained for a while. Eventually, the OS deallocates the PCB.

The act of switching the CPU from one process to another is called a **context switch**. Context switching is a procedure that a computer’s CPU follows to change from one task to another while ensuring that the tasks do not conflict. In Figure 6.21, the process P_0 is in the running state, and the process P_1 is in the ready state. When an interruption occurs, the process P_0 must be switched from the running to the ready state, and the process P_1 must be switched from the ready to the running state. To accomplish this, the OS performs these steps: it saves the context of the process P_0 in PCB_0, switches P_0 from the ready state, selects P_1 to be

executed, and, finally, updates PCB_1 of process P_1.

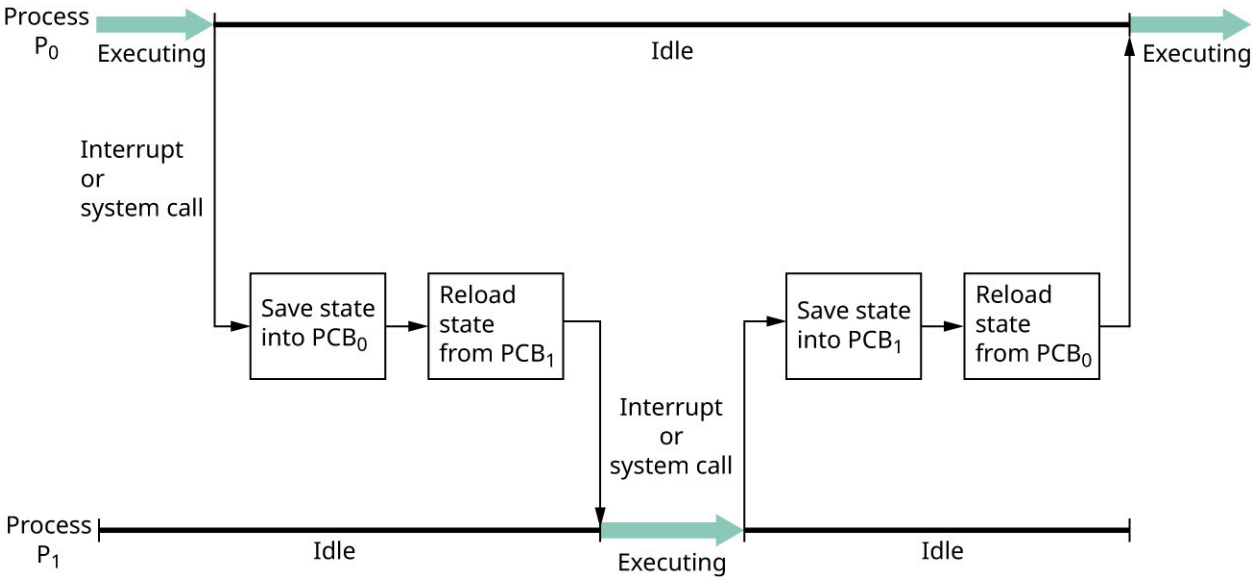


Figure 6.21 This example of context switching shows the steps involved when a CPU switches from executing process P_0 to running process P_1 and then back to P_0. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Transactions and Scheduling

As you've learned, scheduling is the act of determining which process is in the ready state and should be moved to the running state when more resources are requested than can be granted immediately and in which order such requests should be serviced. Examples are processor scheduling (i.e., one processor, many threads) and memory scheduling in virtual memory systems.

A good scheduling algorithm minimizes response time, efficiently utilizes resources (that is, it ensures full utilization by keeping cores and disks busy, with minimal context switches), and implements fairness by distributing CPU cycles equitably. Ideally, scheduling algorithms should not affect the results produced by the system. Optimal scheduling schemes would require the ability to predict the future, making adaptive algorithms the preferred choice.

Examples of simple scheduling algorithms include:

- **first come, first served (FCFS)** scheduling, also called first in, first out (FIFO): In this algorithm, the first job that comes to the processor is executed first. For example, suppose we have two processes—process P_1 with execution time 3 and process P_2 with execution time 2. The arrival time for both process P_1 and process P_2 is t_0. The system will start with process P_1 at t_0 and finish at t_3 and process P_2 will start at time t_3 to time t_5, as shown in Figure 6.22. The wait time for process P_1 is 0 and the wait time for process P_2 is 3.

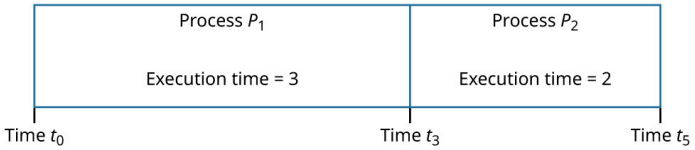


Figure 6.22 Two processes, P_1 and P_2, are being processed using the FCFS algorithm. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

- **round-robin scheduling (RR)**: This algorithm, which is widely used in time-sharing systems, is designed to ensure fairness among processes by giving each process an equal share of the CPU. Its operation is relatively straightforward but effective in environments where a large number of processes need to be handled efficiently. The core idea of RR scheduling is to assign a fixed time slice, often referred to as a

quantum, to each process in the ready queue. The CPU scheduler cycles through the queue, allocating the CPU to each process for a duration equal to 1 quantum. For the previous example, let us set the quantum to 2. Then, the processor will execute part of process P_1 and move to process P_2 then go back to process P_1 (Figure 6.23).

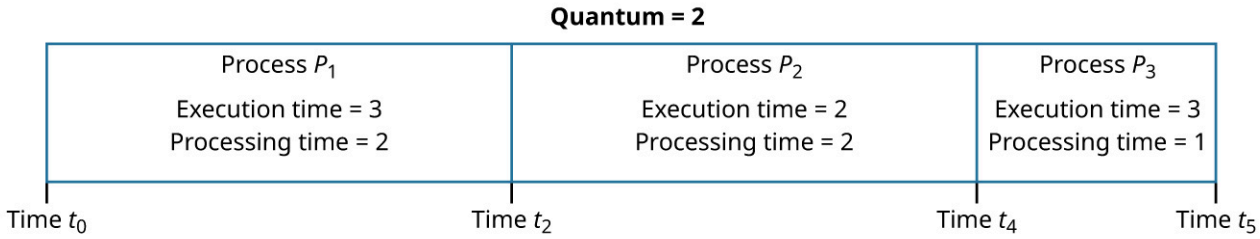


Figure 6.23 Two processes, P_1 and P_2, are being processed using the RR algorithm. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

- **shortest time to completion first (STCF)**, also called shortest job first (SJF): This algorithm takes the best approach to minimize the waiting time, but it requires that the processor knows the processing time in advance. In the previous example, the processor will run process P_2 before process P_1 because the execution time is less.
- **shortest remaining processing time (SRPT)**: In SRPT, the processor is reallocated to a newer ready job with a shorter remaining completion time whenever such a job arrives.

Synchronization

Synchronization in concurrent programming is crucial for ensuring that multiple threads or processes can work together without interfering with each other's operations on shared resources. Through mechanisms like locks, condition variables, and semaphores, developers can design systems that are both efficient and safe, avoiding issues such as data races and deadlocks. One way of coordinating multiple concurrent activities that are using shared state is **synchronization**, which groups operations together automatically to ensure cooperation between threads. To ensure that only one thread does a particular task at a time, we can use a program called **mutual exclusion** that prevents simultaneous access to a shared resource. A **critical section** is a piece of code that only one thread can execute at once. Also, only one thread at a time will get into this section of code. A critical section is the result of mutual exclusion. Critical sections and mutual exclusion are in fact two ways of describing the same thing. Critical sections are sequences of instructions that may get incorrect results if executed simultaneously. Mutual exclusion is required to ensure that a process cannot enter its critical section while another concurrent process is currently present in its critical section. Figure 6.24 illustrates a representation of two processes, process P_1 and process P_2. At Time 1, process P_1 entered the critical section by printing on a shared printer. Process P_1 will finish printing at Time 2. While process P_1 is printing, process P_2 is attempting to print, but the OS will block process P_2 from printing until process P_1 reaches Time 2.

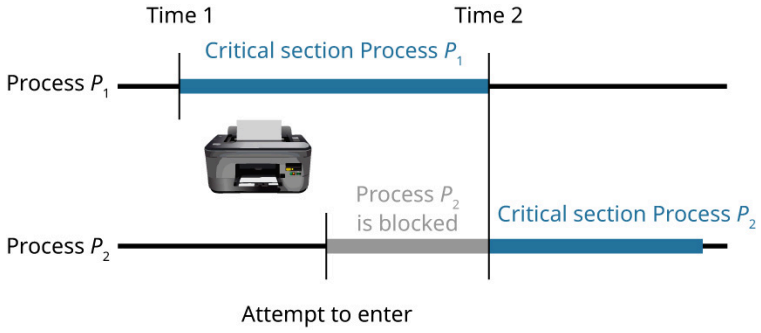


Figure 6.24 Process P_1 is using the printer starting from Time 1 and will finish printing at Time 2. Process P_2 is trying to print while process P_1 is printing. The OS will use mutual exclusion to prevent process P_2 from starting until process P_1 has ended. (attribution:

Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

A **lock** is a synchronization mechanism that is used to enforce mutual exclusion. A thread must acquire a lock before entering a critical section; if the lock is already held by another thread, the attempting thread will block until the lock becomes available. Another synchronization mechanism called a **condition variable** is used in conjunction with locks to allow threads to wait for certain conditions to become true. While a lock facilitates exclusive access to resources, a condition variable helps threads wait for specific states of the world.

A classic example of a synchronization challenge is the producers/consumers problem, where producers generate data and place it into a buffer, and consumers remove data from the buffer for processing. In a producer/consumer scenario, a consumer might wait on a condition variable if the buffer is empty, and a producer might signal this variable once it adds an item to the buffer. The key concerns associated with this scenario include ensuring that producers don't add data to a full buffer and that consumers don't try to remove data from an empty buffer. Synchronization tools like locks and condition variables are used to solve these issues. A **deadlock** occurs when a set of threads are blocked forever, waiting for each other to release resources. This can happen, for example, if thread A holds lock 1 and waits to acquire lock 2, while thread B holds lock 2 and waits to acquire lock 1. Avoiding deadlocks requires careful design, such as acquiring locks in a consistent order or using timeout mechanisms.

Some other tools used in synchronization are:

- Semaphores: Similar to locks, but allow multiple threads to access a finite number of resources
- Barriers: Enable multiple threads to wait until all have reached a certain point in their execution before any are allowed to proceed
- Read-Write Locks: Allow multiple readers to access a resource concurrently but require exclusive access for writers
- Mailboxes: Dedicated channels that connect two processes directly, allowing data to be exchanged between them; mailboxes behave like queues but use semaphores for controlled automatic access, and operate in first in, first out (FIFO) order only.

TECHNOLOGY IN EVERYDAY LIFE

Multitasking

The ability to run multiple programs at once on computers today has a huge productivity impact in all industries. In concurrent programming, managing shared resources among multiple threads or processes is crucial for maintaining data integrity and preventing race conditions. One common scenario is the producer-consumer problem, where one or more threads (producers) generate data, and others (consumers) consume it. To avoid conflicts, synchronization mechanisms like locks and condition variables are employed. Locks help ensure that only one thread accesses a critical section of code at a time. In the context of the producer-consumer problem, locks can be utilized to safeguard shared data structures, preventing simultaneous access by multiple threads and ensuring data consistency.

Condition variables are another synchronization tool that allows threads to coordinate their activities. In the context of producers and consumers, a condition variable could signal when data is available for consumption or when space is available for production. Threads can use these signals to efficiently wait for or notify others about the state of shared resources. The combination of locks and condition variables provides a powerful means to synchronize complex interactions between producers and consumers, ensuring orderly access to shared resources.

Despite the benefits of synchronization mechanisms, the improper use of locks can lead to issues like deadlocks, where two or more threads or processes are stuck in a circular wait, unable to proceed because each is waiting for the other to release a resource. This situation can bring a system to a standstill, and

careful design and coding practices are necessary to prevent or detect and recover from deadlocks. To handle deadlocks, techniques such as deadlock detection algorithms and prevention strategies are employed, contributing to the robustness of concurrent systems.

Allocation

The method that defines how data is stored in the memory by providing a set of requests for resources and identifying which processes should be given which resources to make the most efficient use of the resources is called **allocation**. Like scheduling, allocation is another kind of decision-making that an OS performs about how to use resources to support concurrency. There are three main forms of allocation: contiguous allocation, linked allocation, and indexed allocation.

In contiguous allocation, each file is assigned to a contiguous (i.e., neighboring) set of blocks in the memory. For example, if a file requires three blocks and is given a starting location x, the file will be allocated in x, $x + 1$, $x + 2$. The directory entry with contiguous allocation contains the starting block address and the length of the file. In linked allocation, each file is a linked list of memory blocks. Using the same example, the first block will be allocated in location x and it will include the address of the second block. The directory entry with linked allocation contains a pointer to the starting block and a pointer to the last block. In indexed allocation, each file has an index block containing the pointers to all blocks for that file.

6.4 Memory Management

Learning Objectives

By the end of this section, you will be able to:

- Discuss key concepts related to memory
- Evaluate dynamic storage management solutions
- Discuss the differences between virtual and physical memory

As you have learned, memory plays a huge role in OSs. Here, we discuss the memory multiplexing, linkers and dynamic linking, dynamic storage management, virtual memory, and demand paging.

Memory

Different processes and threads share the same hardware. It is therefore necessary to multiplex the CPU (i.e., temporal execution), memory (spatial access), and disks and devices. As discussed earlier, the complete working state of processes and/or kernels is defined by its data (i.e., memory, registers, and disk). For the sake of safety, security, and reliability, processes should be barred from having access to each other's memory. Dividing the capacity of the communication channel into multiple logical channels is considered **memory multiplexing**. There are several concepts that are critical to memory multiplexing, namely, isolation, sharing, virtualization, and utilization.

As you learned earlier in this chapter, isolation is important because it ensures that the multiple programs that are running concurrently on the same CPU and memory operate independently without interfering with each other's execution or data. In memory multiplexing, isolation is achieved through a set of technologies that prevent distinct process states from colliding in physical memory due to unintended overlap (i.e., overlap control). These technologies aim to, for example, prevent process P_1 from spying on process P_2. Or, if process P_1 has a bug, they ensure that this bug does not impact process P_2. There are many isolation mechanisms, including:

- User/kernel mode flag is a register that represents the CPU mode as user mode or kernel mode. As we have learned, the CPU boots in kernel mode, then it marks the flag as kernel mode. When the user starts any application, the CPU marks the flag as user mode.

- Address space boundaries protect the kernel and address space programs from each other.
- System call interface is the programming interface for application users to process a system call. As shown in Figure 6.25, a system call is executed by the user mode to request the kernel mode to perform a specific action (e.g., `syscall ()` function).

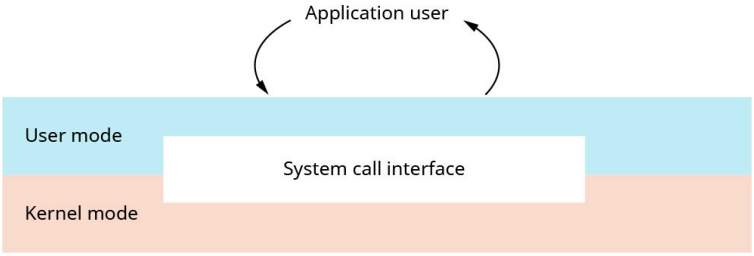


Figure 6.25 The system call interface uses an isolation mechanism to address a system call, which is a request that arises from the user mode and requires the kernel mode to perform an action. The system call interface prevents these processes from overlapping or colliding. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Time slicing provides a time frame for each process to run in a preemptive multitasking CPU such that each process runs every single time slice. If the process finishes the job before the time slice, it releases the CPU and does not need to be swapped out. If the time slice ends and the process did not finish the job, the CPU shifts it out to the end of the processes queue. For example, assume we have three processes P_1 with execution time 3 ms, P_2 with execution time 4 ms, and P_3 with execution time 2 ms, and a time slice of 2 ms. Figure 6.26 illustrates how the CPU manages the processing using time slice and indicates in which time slice each process completes execution.

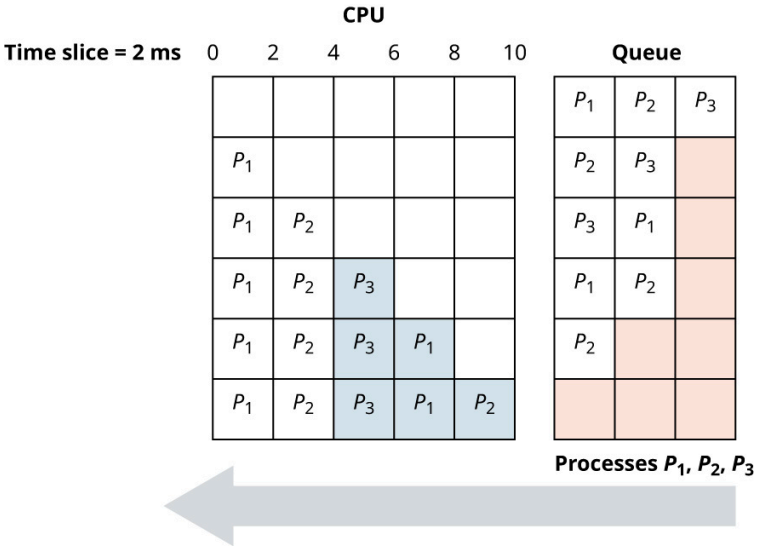


Figure 6.26 The three processes P_1, P_2, and P_3 (blue) will work as scheduled based on the time slice. The processes are scheduled in the queue and executed, so they come out of the queue (arrow). Empty spaces in the queue are shown in pink. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

When multiple processes can use the same piece of data concurrently, it is called **sharing**. The option for overlapping processes should be available when desired for efficiency and communication reasons. Memory sharing improves the performance of the system because the data is not copied from one address space to another, so memory allocation is done only once.

With respect to memory, virtualization is a technique that gives an application the impression that it has its own logical memory and that it is independent from the available physical memory. Thus, in virtualization, there is a need to create the illusion that there are more resources than those that actually exist in the underlying physical system. There are two approaches of memory virtualization: full virtualization and guest modification. When multiple operating systems run concurrently on a single physical machine, fully isolated from each other, by emulating hardware resources through a hypervisor, it is called **full virtualization**. For full

virtualizations, all OSs expect contiguous physical memory that starts from physical address 0. In the context of virtualization, **guest modification** refers to altering the guest operating system or its configuration to improve compatibility, performance, or integration with the virtualization environment or hypervisor. Guest modification modifies the OS to avoid using instructions that virtualize inefficiently. An optimal use of limited resources is warranted to guarantee a high level of utilization.

Processes use different amounts of memory, and their memory needs change over time. Whenever a new process cannot fit into contiguous space in physical memory, it results in fragmentation (specifically, external fragmentation). When the memory blocks cannot be allocated to the processes due to their small size and the blocks remain unused, this problem is called **fragmentation**. There are two types of fragmentation: internal fragmentation and external fragmentation. When the process is allocated a block and its size exceeds the process size, it leaves part of the memory allocated unused and results in internal fragmentation. In the external fragmentation, the total space that is needed for the process is available, but we can't use it because the space is not contiguous.

Linkers and Dynamic Linking

Linkers are software tools that an OS uses to combine object files into an executable file. A linker performs name resolution, matching the name of a variable or function in an application to a virtual memory address it will have when loaded and run. A linker combines many separate pieces of a program, reorganizes storage allocation so that all the pieces can fit together, and touches up addresses so that the program can run under the new memory organization. After a linker completes the task of combining multiple object files generated by a compiler into a single executable file, the executable file can be loaded and executed by the OS (Figure 6.27).

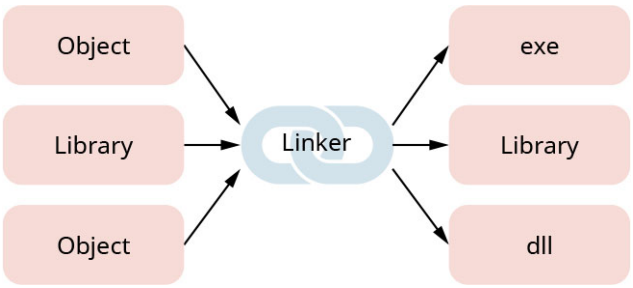


Figure 6.27 A linker process includes object files and libraries. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The mechanism that allows a program to associate external code symbols to addresses at runtime is **dynamic linking**. The allocation process starts when the process is running by dividing the memory into smaller parts called segments. For example, Linux's memory layout is divided such that the code starts at location 0, the data starts immediately above the code, and the stack starts at the highest address, as illustrated in Figure 6.28. When a process is started, the OS will load the file to the memory with the added option of sharing the memory with others. The OS facilitates the memory size at runtime by adding more assigned memory when needed.

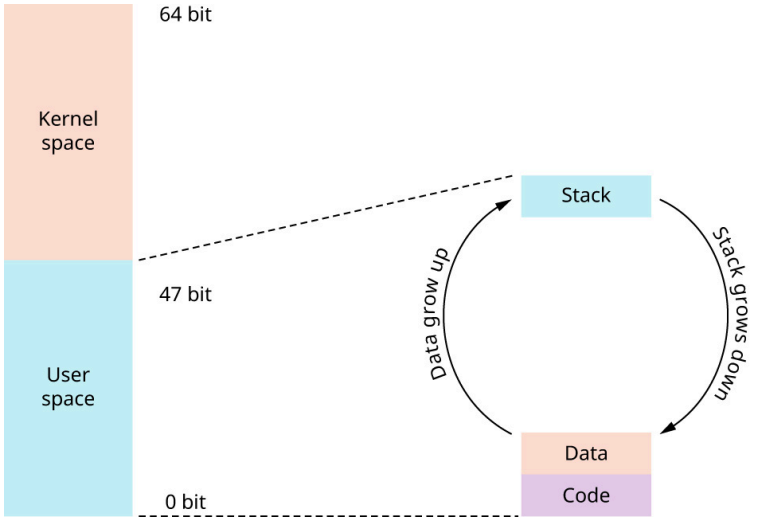


Figure 6.28 In Linux's memory layout, the code starts at location 0, the data starts immediately above the code and grows upward, and the stack starts at the highest address and grows downward. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

In dynamic linking, the code is located and loaded when the program is first run. Since the late 1980s, most systems started supporting shared libraries and dynamic linking by only keeping a single copy of common library packages in memory that is shared by all processes. This means that the system does not know where the library is loaded until runtime and must resolve references dynamically when the program runs.

Dynamic Storage Management

There are two basic operations used in dynamic storage management to manage a memory or storage to satisfy various needs: allocate a block with a given number of bytes or free a previously allocated block. There are two general approaches to applying these dynamic storage allocation operations: (1) stack allocation, which is hierarchical and restricted, but simple and efficient; and (2) heap allocation, which is more general but more difficult to implement and less efficient.

The linear data structure that follows a LIFO order (last in, first out), as in the stack data figure configuration in Figure 6.29, is called **stack allocation**. In the stack approach, the memory is freed in opposite order from allocation. For example, if procedure *X* calls *Y*, then *Y* will certainly return before returning from *X*. Stacks take advantage of this programming practice to store the state of the current procedure call. When memory allocation and freeing are partially predictable, then a stack approach can be used.

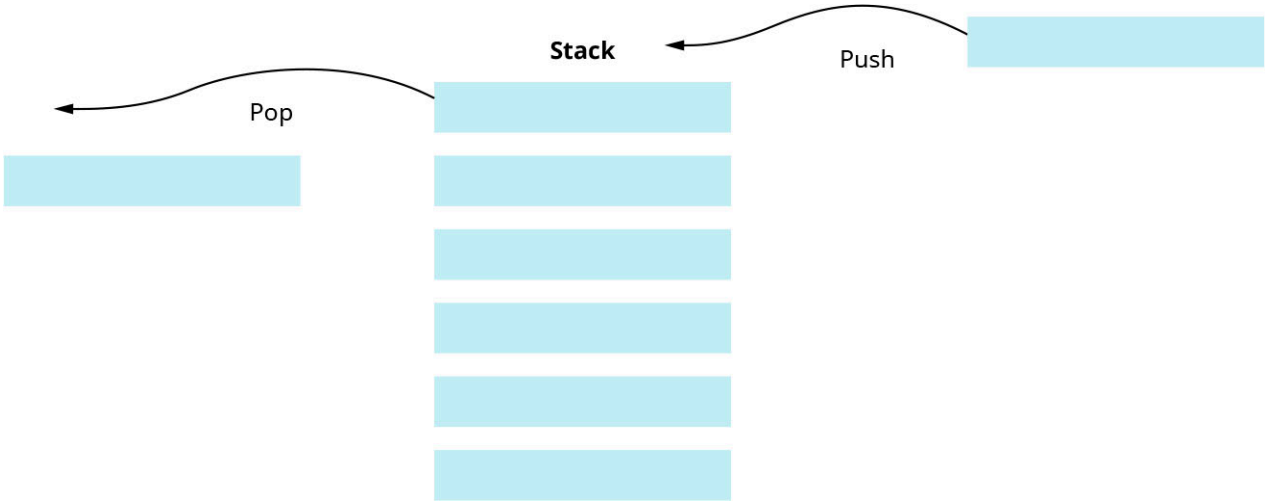


Figure 6.29 This stack data structure representation shows a last in, first out approach. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Allocating the data in a tree-based data structure called a heap is **heap allocation**. A heap is represented by a complete binary tree. As shown in Figure 6.30, a heap data structure can be of two types: max heap and min heap. Max heap presets the root node with the greatest value and the same for the sub trees. It is the opposite for the min heap, where the root will have the minimum value and the same for the sub trees.

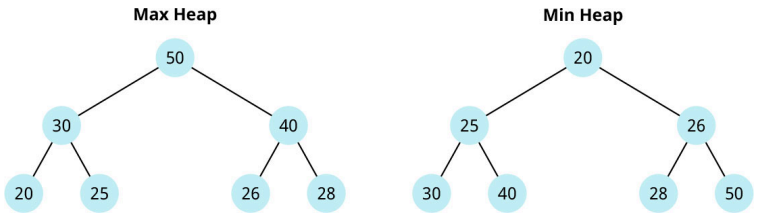


Figure 6.30 Heap allocation uses a data structure called a heap to manage memory and storage. There are two types of heap structures or trees: one for the max heap and one for the min heap. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Memory managers, such as the ones used in C and C++, do not actually store available memory in a heap data structure. Instead, they manipulate a doubly linked list of blocks, which they confusingly refer to as a "heap," and attempt to optimize memory using a "best fit" method.

Memory managers use the heap approach when the allocation and release of memory are not predictable (i.e., when it is not clear how much memory is needed until we run the program). Typically, the heap stores data structures that can change in size over time based on how many elements are added or removed from the data structure. The corresponding heap memory consists of allocated areas and free areas (or holes).

Virtual Memory

The key component of the operating system that ensures process isolation by guaranteeing that each process gets its own view of the memory is virtual memory. A running program (process) has a seemingly infinite view of memory and can access any region without worrying about other programs that might also be running on the computer. The OS seamlessly translates each process memory request into a separate region of the physical hardware memory through address translation. When the system needs to find a physical address in the memory that matches the virtual address, **address translation** occurs. The running process only deals with virtual addresses and never sees the physical address. Virtual memory is mapped to physical memory in units called "pages."

There is a time cost associated with performing virtual-to-physical memory address translation, however, and this can add up given that most programs need access to the memory to store data. To speed up address translation, the CPU has dedicated hardware for caching (storing) recent address translations called a **translation lookaside buffer (TLB)**. A TLB is a memory cache that stores the virtual memory recent transaction to physical memory. TLBs help the CPU avoid making multiple round trips to main memory just to resolve a single virtual memory access by only requiring one round trip (Figure 6.31).

A TLB contains page table entries that have been most recently used. Given a virtual address, the processor examines the TLB table. If a page table entry is present, it's a "hit." This means the frame number is retrieved, and the real address is formed. If a page table entry is not found in the TLB, then it's a "miss." In this case, the page number is used as an index while processing the page table. TLB checks if the page is already in the memory; if it's not, then a page fault is issued and the TLB is updated to include the new page entry.

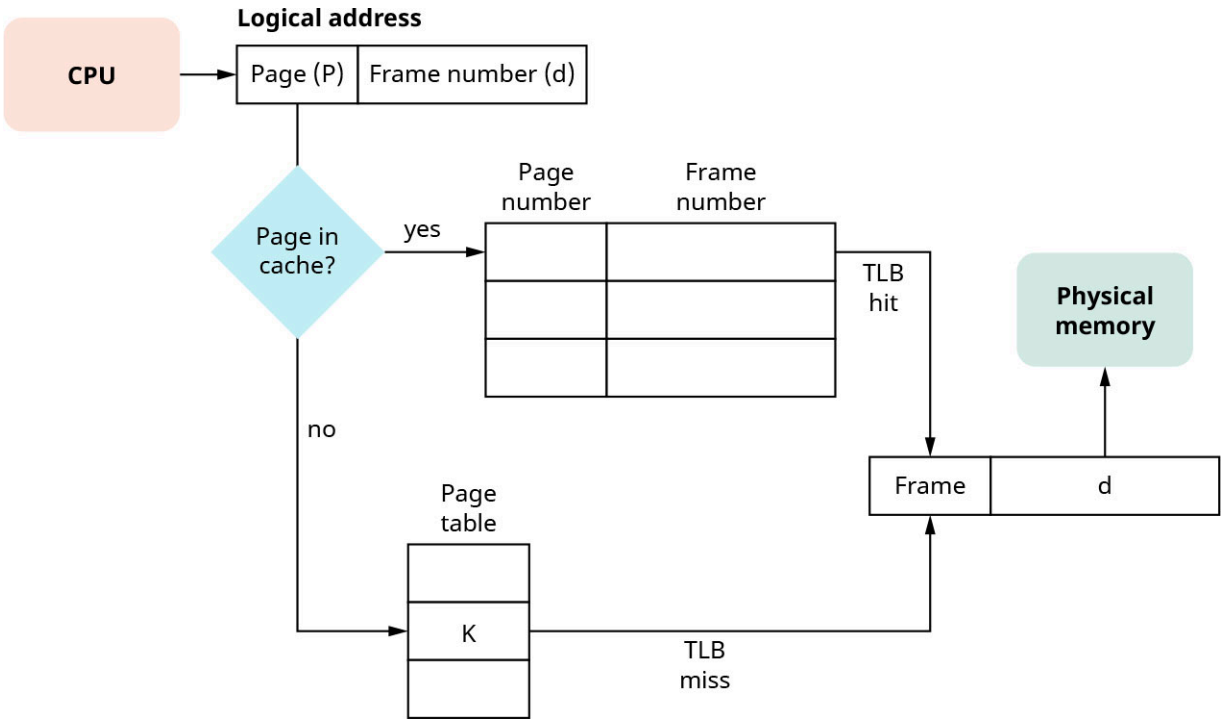


Figure 6.31 Translation lookaside buffers (TLBs) speed up address translation by using an approach that involves detecting "hits" (which means a page table entry is present in the TLB) and "misses" (a page table entry is not found in the TLB). (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

What is the fundamental concept that makes it possible to implement virtual memory? Check out an explanation of how to implement virtual memory (https://openstax.org/r/76virtmem) to investigate this question.

Demand Paging

The storage mechanism that uses a page form in retrieving a process from secondary or virtual memory to main memory is called **paging**. Virtual memory presented a seemingly infinite amount of memory to the running process, but what happens when the operating system runs out of free physical memory? Modern operating systems also have a backup when DRAM runs out, which means virtual memory can be mapped to disk to meet demand. The storage mechanism in which pages should only allocate in the memory if it is required from the execution process is called **demand paging**. Figure 6.32 shows a CPU that is demanding pages from the virtual memory to the main memory (i.e., swap in) and releasing pages from the main memory to the virtual memory (i.e., swap out). The **working set size (WSS)** refers to the total amount of memory a process requires during a specific period of activity, measured as the set of pages or data blocks the process accesses. WSS is measured by tracking the unique pages a process references over a fixed interval of time. This provides an estimate of the process's active memory footprint and helps in memory management decisions like paging and swapping to optimize performance and resource allocation.

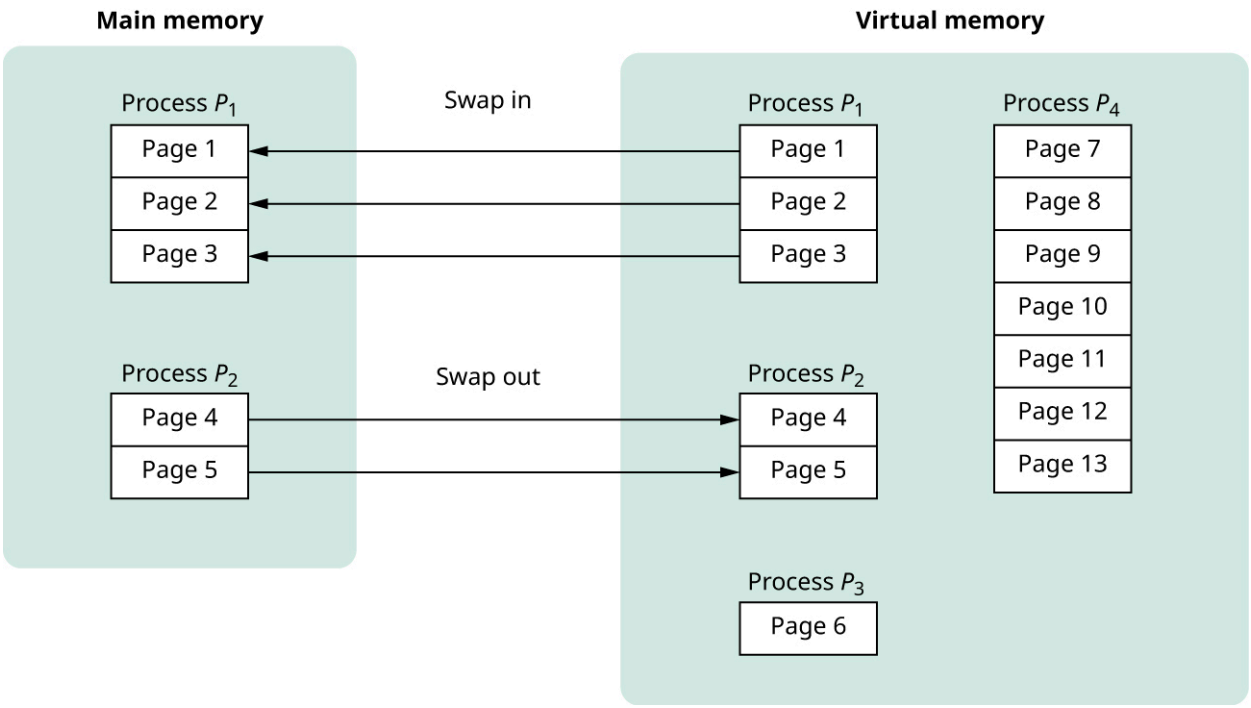


Figure 6.32 In demand paging, a CPU retrieves pages from the virtual memory to the main memory and releases pages from the main memory to the virtual memory. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

When the CPU demands a page and this page is not present in the main memory, we call this situation a **page fault**. A page fault occurs when a process references a page that is in the backing store. To handle a page fault, the CPU transfers control from the program to the OS to demand the requested page to the main memory. OS finds a free page frame in memory, loads the page from the backing store to the main memory, and resumes execution of the thread. The CPU has special hardware to assist in resuming execution after a page fault.

Given that access to the disk is much slower than DRAM, operating systems are often designed to predictively swap in-use pages into DRAM and out-of-use pages to disk. The process of bringing pages into memory (i.e., demand paging) is called **page fetching**. Most modern OSs use page fetching by starting the process with no pages loaded and do not load a page into memory until it is referenced. If a requested page is stored on the disk, prefetching, which is the act of trying to predict when pages will be needed and loading them ahead of time to avoid page faults, is performed.

If all memory is in use, it is necessary to throw out one page each time there is a page fault. This process is called **page replacement**. In page replacement, one page in the (full) DRAM is swapped to disk while the requested page is brought into DRAM. However, if too many processes need access to a lot of memory back and forth between DRAM and disk, this causes problems. For example, each page fault causes one of the active pages to be moved to disk, so another page fault soon occurs, and this leads to **thrashing**. Thrashing is when a computer's operating system becomes overwhelmed by the number of processes requesting memory. This situation leads to a cycle where the system spends more time moving data between the physical memory and disk storage (paging or swapping) than executing actual processes. It's like a busy restaurant where the staff spends more time rearranging tables than serving food. The main cause of thrashing is often that too many programs are running at the same time. These activities exceed the available memory, causing the system to constantly try to make space for new requests by moving data to and from the disk.

Strategies to prevent thrashing include limiting the number of simultaneously running programs to avoid memory overcommitment, optimizing how memory is allocated to processes, and possibly increasing the system's physical memory. By managing memory more efficiently and ensuring that the system is not overloaded with too many tasks, the system can not only avoid a slowdown, but significantly improve its

performance.

In extreme cases of thrashing, the OS can spend all its time fetching and replacing pages and will not get much work done. This is one reason why our devices can slow to a halt when they run out of memory and each thread must wait on requested pages.

CONCEPTS IN PRACTICE

Impatience with Computers

We've all been there: At our laptops, putting the final touches on a slide presentation, while checking email, while uploading a photo to our social media feed, while listening to a new playlist our friend just shared with us, while watching a video, while inputting data into a spreadsheet, and everything freezes—the screen, the keyboard, the trackpad. Not only does nothing do what we ask it to do, it all just goes still. Or worse, the little pinwheel starts spinning and never stops. Now having read about memory, dynamic storage management, resource allocation, and thrashing, think about what's happening inside your CPU.

If you're using a computer that runs Windows, check out this resource to see how you could help your operating system (https://openstax.org/r/76WindowsOS) operate better.

6.5 File Systems

Learning Objectives

By the end of this section, you will be able to:

- Explain features of various file systems
- Discuss file system structures and layers

In this module, we learn about files, file management, disk devices, file systems, file system interface, and distributed file systems.

Files, File Systems, Directories, and File Management

A **file** is a collection of related information that is stored on a storage instrument such as a disk or secondary/virtual storage. It is the smallest storage unit from the user's perspective. The file name includes two parts: name and extension (e.g., filename.txt). Each extension is for a specific purpose such as .exe (in Windows OS) to run a program and .txt for text files.

A **file system** is responsible for defining file names, storing files on a storage device, and retrieving files from a storage device. When designing a file system for managing many files, some issues to consider are as follows: most files are small so per-file overheads must be low; most of the disk space is in large files; many of the I/O operations are for large files so performance must be good for large files; files may grow unpredictably over time; users want to use text names to refer to files.

Special disk structures called directories are used to map names to support hierarchical directory structures. A **directory** is a set of files that is managed by the OS, and it also contains all the required information about the files, such as attributes, location, and ownership. The UNIX/Linux approach is as follows: directories are stored on disk just like regular files except with extra information to indicate that it is a directory. Each directory contains <name, address> pairs. The file referred to by the address may be another directory; hence, we can have nested and hierarchical directory structures.

The problems facing modern file systems include disk management, naming, and protection. File systems are often trying to improve access to files by minimizing seeks, sharing space between users, and making efficient use of disk space. A system's ability to reduce faults and ensure that the information in the system survives OS

crashes and hardware failures is called its **reliability**. In addition to improving reliability, a file system should guarantee a high level of protection by maintaining isolation between users and controlling the sharing of resources.

Disk Devices

While file systems are a layer of abstraction that provides structured storage and defines logical objects such as files and directories, disk devices are considered raw storage. Data that can be directly accessed by the CPU with minimum or no delay and does not survive a power failure is held in **primary storage**. Persistent memory that survives power failures most of the time, such as spinning disks, SSDs, and USB drives, is considered **secondary storage**. Routines that interact with disks are typically at a very low level in the OS and are used by many components such as file systems and virtual machines. These secondary storage devices may handle the scheduling of disk operations, error handling, and often the management of space on disks. A trend is for disks to do more of this themselves.

File System Architectures

Operating systems use various methods to locate files by their names, and the methodology often depends on their underlying file system architecture. To illustrate these concepts, here are some examples from UNIX-like systems and Windows:

- UNIX/Linux (Inodes): In UNIX-like systems, the file system uses a structure called an **inode** to represent files and directories. An inode contains metadata about a file or directory but not its name. The name-to-inode mapping is stored in directories, which are special files that list names of files and their corresponding inodes. When searching for a file by name, the OS starts at the root directory and follows the path specified in the file name. Each part of the path is looked up in the current directory's list of names and their associated inodes. The OS reads the directory file, finds the name, and retrieves the inode number, which then leads to the inode itself. The inode provides the location of the data blocks, allowing the OS to access the file's data. This process may involve multiple steps if the file is in a nested directory structure.
- Windows (File Allocation Table and NTFS): In File Allocation Table (FAT) format, files are located using a table that maps file names to the clusters (blocks) on the disk where their data is stored. The FAT is essentially a list, with each entry containing the location of the next part of the file. This creates a chain that the OS follows to read the entire file. In New Technology File System (NTFS), files are located using a Master File Table (MFT), and each file and directory on an NTFS volume has an entry in the MFT containing data, including the file name, size, time stamps, permissions, and the locations of the file's data on disk. When searching for a file, the OS consults the MFT to find the entry corresponding to the file name, which then provides the information necessary to access the file's data.

As you may recall, files represent values stored on disk and directories represent file metadata. File systems define operations on objects such as create, read, and write, and they may also provide higher-level services such as accounting and quotas, incremental backup indexing or search, file versioning, and encryption. A **quota** is the amount of space to store files based on the available memory space. Quotas are used to protect the system from unnecessary load and help in organizing the data in the storage. An **incremental backup** is a backup image containing the pages that have been updated from the time of the previous backup. The method that converts the data into secret code that hides the data's true meaning is called **encryption**. The system that allows a file to exist in several versions at the same time, which gives the user complete control over file creation as in the file versioning example, is called **file versioning** (Figure 6.33).

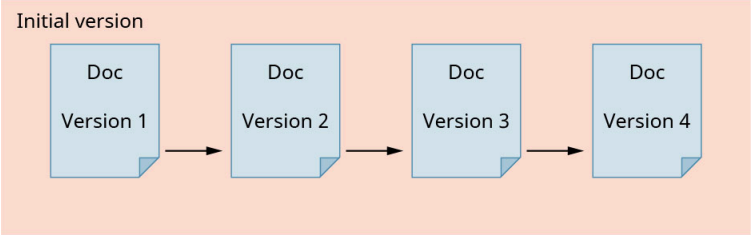


Figure 6.33 In file versioning, the OS saves all copies of a file (in this case, a doc or document file) . (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

File systems are concerned with lower-level characteristics such as performance and failure resilience. Both performance and failure resilience may be strongly affected by hardware characteristics.

File System Interface

In general, the file system interface defines standard operations such as file (or directory) creation and deletion, manipulation of files and directories, copy, and lock. Remember the various file attributes are name, type, size, and protection. The file system uses these attributes to provide system calls for the following operations:

- Create: Find a space for the file in the disk and enter the new file information in the directory.
- Write: Search the directory for a specific file and start writing from where the writing pointer is pointing to.
- Read: Specify the name of the file and start reading from where the reading pointer is pointing to.
- Seek: Search for the specific byte position in the file.
- Delete: Search for the file in the directory and erase the file from the directory.
- Truncate: Reset the file length to zero and release the allocated space for the file.
- Append: Add new information to the end of the file.
- Copy: Create a new file and read the data from an old file, then write it to the new one.

If multiple processes are trying to open a file at the same time, then there is a role for file management that should be applied—namely, lock.

LINK TO LEARNING

As people and businesses use various types of computers today, the ability to interchange files between various file systems is critical. In fact, insurance companies often ask their clients to sign insurance contracts over the Internet and rely on their digital signature in online documents. Teachers use Google Classroom to enable their students to collaborate on assignments or class presentations. Read this overview of common file systems (https://openstax.org/r/76comfile) to see how file systems enable us to seamlessly share documents and files.

Inodes

As mentioned earlier, inodes are OS data structures used to represent information about files and folders stored on disk along with file data and kept in memory when the file is open. An inode contains information including file size, sectors occupied by file, access times (e.g., last read and last write), and access information (e.g., owner id and group id). In Linux, whenever the system creates a new file, it gives it an inode unique number called **i-number**. Internally, the OS uses the i-number as an identifier for the file—in effect, as its name. When a file is open, its inode is kept in main memory. When the file is closed, the inode is stored back to disk. If you are using Linux, you can check the total number of inodes on disk using the df command and –i option, as shown in Figure 6.34.

Input:

```
$df –i /dev/sda
```

Output:

Filesystem	Inodes	IUsed	IFree	IUse%
/dev/sda	1624000	128000	1496000	9%

Figure 6.34 In the Linux OS, the total number of inodes on the directory /dev/sda can be viewed using the command df and the option –i. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

File systems are responsible for managing parts of the disk that are used (inodes) and parts of the disk that are not used (free blocks). Each file system has different strategies and approaches for managing this information, with different trade-offs. Additional features of file systems include file system–level encryption, compression, and data integrity assurances.

Distributed File Systems

A **distributed file system (DFS)** is a file system that is distributed on multiple file servers or multiple locations that support network-wide sharing of files and devices. The presentation of a DFS is similar to the traditional view (i.e., client is using a file system). The main idea of a DFS is that it uses a namespace, which means all clients see a single namespace where files and directories are shared across the network. In a DFS, clients can read and write files on a remote machine as if they were accessing their local disks. A DFS provides an abstraction over physical disks that is akin to the abstraction virtual memory provides over physical memory (Figure 6.35).

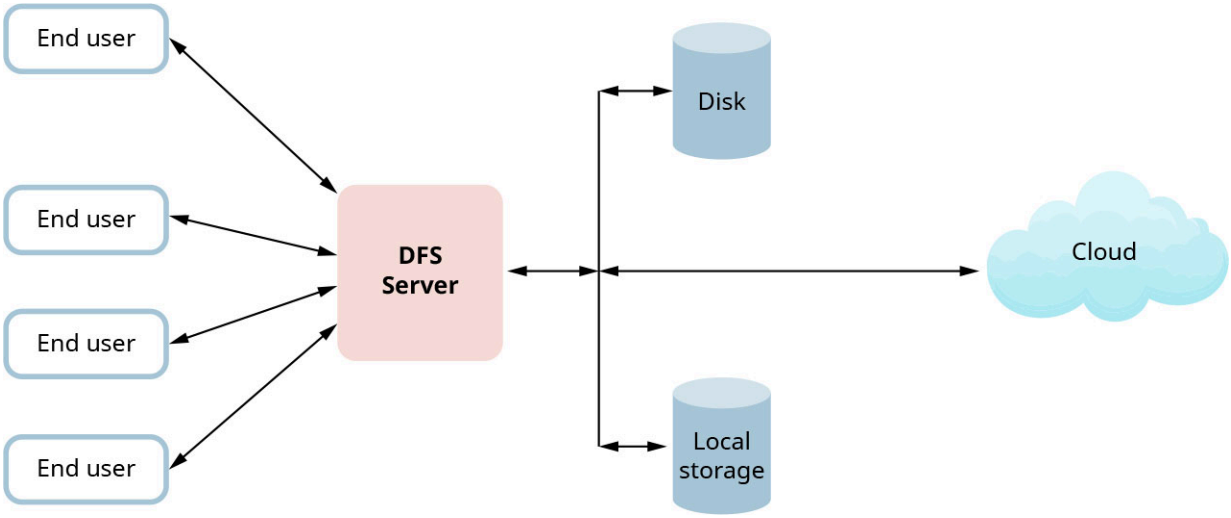


Figure 6.35 In a distributed file system architecture, the DFS server works like a middleman between the end user and the data, which can be in any storage format. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

DFS technologies like Google's GFS (Google File System), Apache Hadoop's HDFS (Hadoop Distributed File System), and Apache Spark's RDDs (Resilient Distributed Datasets) have revolutionized the way we handle and process large volumes of data. These systems are designed to accommodate High Throughput Computing (HTC), complementing the capabilities of High-Performance Computing (HPC) by focusing on the efficient processing of vast datasets across clusters of computers.

- Google File System (GFS) is a prime example of a DFS that is highly optimized for large-scale data processing. It is designed to provide high fault tolerance while running on low-cost commodity hardware.
- Hadoop Distributed File System (HDFS) follows a similar principle but is open-source and commonly associated with the Hadoop ecosystem. It's designed to store very large files across machines in a large cluster and to stream those files at high bandwidth to user applications. By breaking down files into blocks

and distributing them across a network of computers, HDFS can process data in parallel, significantly speeding up computations and data analysis tasks.
- Resilient Distributed Datasets (RDDs) in Apache Spark are a further step in distributed computing, offering an abstraction that represents read-only collection of objects partitioned across a set of machines that can be rebuilt if a partition is lost. Spark's use of RDDs allows it to process data in-memory, which is much faster than the disk-based processing used by Hadoop, making Spark an excellent choice for applications requiring quick iterations over large datasets.

To facilitate the communication necessary in these distributed environments, protocols such as Remote Procedure Call (RPC) and Distributed Hash Tables (DHTs) are employed. RPC is a protocol that a program can use to request a service from another program located in another computer in another network without having to understand the network's details. DHTs are a class of decentralized distributed systems that provide a lookup service similar to a hash table; keys are mapped to nodes, and a node can retrieve the content associated with a given key.

Beyond these, the concept of N-Tier distributed file systems, such as the Network File System (NFS), plays a foundational role. NFS allows a system to share directories and files with others over a network. By using NFS, users and programs can access files on remote systems almost as if they were local files.

The basic abstraction of a remote file system is via open, close, read, and write. As it comes to naming, the names are location transparent. Location transparency hides the location where in the network the file is stored. The procedure that allows multiple copies of a file to exist in the network is called **replication**. This improves performance and availability. DFS handles the updates, checks if clients are working on separate copies, and performs reconciliation.

LINK TO LEARNING

Distributed file systems are used worldwide in a range of industries, from banking to health care. Read this brief tutorial on distributed file systems (https://openstax.org/r/76distrfile) and name two advantages as well as two disadvantages of using DFSs.

Flash Memory

Flash memory is used for general storage and the transfer of data between computers and other digital products. Many of today's storage devices, such as SSDs, utilize flash memory, which offers considerable performance improvements over traditional mechanical hard disk drives (HDDs). The performance improvements of flash-based storage devices like SSDs come from their ability to access data much faster than mechanical drives. Here's why:

- No moving parts: Unlike HDDs that use rotating disks and read/write heads, SSDs have no mechanical parts. This not only increases durability, but also means that data can be read from and written to the drive much faster.
- Random access: Flash memory allows random access to any location on the storage, making it much quicker at reading data that is scattered across the drive. HDDs need to physically move the read/write head to the data location, which takes more time.
- Faster read and write speeds: SSDs can handle rapid read and write operations. This is especially beneficial for applications that require quick access to large amounts of data, such as video editing, gaming, and high-speed databases.
- Lower latency: Because they lack a physical read/write head that needs to be positioned, SSDs significantly reduce the time it takes for a storage device to begin transferring data following an I/O request.
- Improved durability and reliability: With no moving parts to wear out or fail, SSDs are generally more reliable and can better withstand being dropped or subjected to sudden impacts.

- Lower power consumption: SSDs consume less power, which can contribute to longer battery life in laptops and less energy use in data centers.

GLOBAL ISSUES IN TECHNOLOGY

Global Distributed File Systems

Distributed file systems enable companies that operate globally and handle vast amounts of data from many different sources and in many different ways, such as the following:

- To store and manage that data in a cloud
- To scale up their operations as needed
- To enable users across the world to access the data seamlessly
- To use encryption and other protection mechanisms to secure sensitive data
- To ensure that data is regularly backed up and can be recovered if there's a disaster

6.6 Reliability and Security

Learning Objectives

By the end of this section, you will be able to:

- Explain how OSs protect computer systems
- Discuss key security-related functions of the OS
- Explain how the OS helps the computer system recover from failures
- Discuss how advances in technology affect the longevity of an OS

Remember that we consider an OS to be reliable if it delivers service without errors or interruptions. In addition to reliability, an OS should provide a high level of protection, security, and stability. Here, we learn about OS protection, security, recovery, and longevity.

Protection

The general mechanism that is used throughout the OS for all resources that need to be protected, such as memory, processes, files, devices, CPU time, and network bandwidth is called **protection**. The objectives of the protection mechanism are to allow sharing (which in this context means using the hardware to do more than one thing at a time), help detect and contain accidental or unintentional errors, and prevent intentional/malicious abuses. The main challenge when it comes to protection is that intentional abuse is much more difficult to eliminate than accidents.

There are three aspects to a protection mechanism: authentication, authorization, and access enforcement. Authentication identifies a responsible party or principal behind each action, authorization determines which principals are allowed to perform which actions, and access enforcement controls access using authentication and authorization information. A tiny flaw in any of these areas can compromise the entire protection mechanism. It is extremely difficult to make all these protection mechanism techniques operate in such a way that there are no loopholes that can be exploited by adversaries. Figure 6.36 illustrates the relationship between authentication, authorization, and access enforcement.

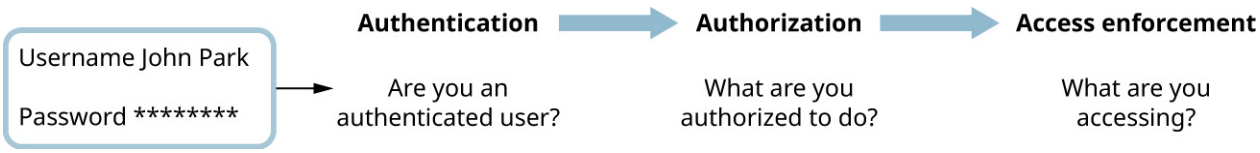


Figure 6.36 The first step of the protection mechanism is authentication, which checks the username and password; then comes authorization, which checks the privileges; and, finally, there is access enforcement, which controls access. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Security

The process of checking to see if a user's credentials match the credentials in a database of authorized users or in a data authentication server is called **authentication**. The traditional means of authentication involves the user providing a password, which is a secret piece of information that is used to establish the identity of a user and should be relatively long and hard to guess. Most systems store the passwords in a password database. A password database must be protected because it is vulnerable most of the time. For example, both organizations and users should avoid storing passwords in a directly readable form.

An alternate form of authentication involves using a badge or key. The **badge** is a logical access system. The badge does not have to be kept secret. It can be counterfeit, but if it is, the owner will know. The badge must be cheap to make but hard to duplicate.

Another form of authentication is **two-factor authentication**, which involves two factors: the system calls or texts a user's phone for the traditional password during login, employing the cell phone as a key. For example, a site sends a text message to a user's phone with a one-time passcode. The user must read the passcode from the phone and type it into the login page.

In two-factor authentication, an attacker must have both your password and cell phone to hijack your account. This approach is particularly effective for authenticating to websites, as the requiring of both the password and the physical cell phone is a sufficient deterrent. To enhance efficiency, the two-factor authentication process can be optimized for websites. Once the authentication is completed, a cookie is loaded into your browser. This cookie then transforms your browser into a type of key, granting you the ability to log in with the password as long as the cookie persists.

Whenever a user needs to log in from a different browser or different machine, two-factor authentication is used again. After logging in, the user id is associated with every process executed under that login because the user id is stored in the process control block and children inherit the user id from their parents. Once authentication is complete, the next step of protection is authorization.

The process of determining the relationship between principals, operations, and objects by defining the kind of principals that are allowed to carry out a specific activity with a defined set of objects is called **authorization**. These principals are represented using a matrix that includes an entry for each principal and a column for each object as a representation of authorization information on given operations. For example, defining who has the authorization to read/view, edit, or delete the file. Each entry in the access matrix describes the capability of each principal over each one of the objects. When the matrix includes all of the principals and all of the assigned objects, it can become complex and hard to manage. The ideal way to solve this problem is to use a guideline such as an access control list. An **access control list (ACL)** is a set of guidelines that outline the authority of each user (i.e., which user is permitted access to given resources). An ACL controls the access and privileges using a matrix design.

The ACL from Oracle features the assignment of users to roles such as basic users, advanced users, customer administrator, among others. A role is configured to confer privileges on objects rather than attaching privileges to individual users, as this would be much more difficult to set up and maintain. The most general form of setting privileges is creating a list of user and privilege pairs, which is called a capability list. A **capability list** is a list of objects and operations for each user that defines the user rights and capabilities. Typically, capabilities also act as names for objects, which means the list cannot even name objects not referred to in your capability list. For simplicity, users can be organized into groups with a single ACL entry for an entire group, and each group can be made to share the same privileges. While in Windows OS, ACLs are very general, they are relatively simple in UNIX/Linux. For example, in UNIX/Linux, access can be read, write, and execute, and it can be granted to the file owner, the file owner's group, or "the world" of all users. In many cases, the user root has full privilege for all of the operations and has access to all of the permissions. For example, the user root can view, edit, and delete a file. ACLs are straightforward and can be utilized by any file

systems in Windows. The utilization involves sharing a namespace at a high level of visibility by making it public, while defining another namespace as private—akin to the encapsulation of objects in object-oriented programming.

One component of an OS must be in charge of enforcing access rules and safeguarding authentication and authorization to provide a high level of security. The system's **access enforcement** mechanism has complete authority; therefore, it must be simple in programming and small in size. The security kernel is a substitute approach that is composed of hardware and software and serves as the OS's inner protection layer. Generally, any kind of management such as memory and interrupt management are provided by a security kernel.

LINK TO LEARNING

Every once in a while, you may get a notification on your computer asking you to update your OS, and the update may include a "patch" to address a security issue. Often, this security issue is related to a cyberattack that is exploiting some vulnerability in the OS. Check out this tutorial on OS vulnerabilities (https://openstax.org/r/76OSvulnera) to gain a deeper sense of the kinds of OS vulnerabilities that these attacks target.

Recovery

Like any other system, an OS can crash in the middle of critical sections or while the system is running. These crashes may result in lost data, unexpected results, and inconsistency. For example, if the crash happened before the system had stored a user's information in the main memory, the system will have lost this information. Unexpected results provide the wrong output and may affect other calculations.

An **inconsistency** is a situation that causes the system to produce errors or hardware failure. Inconsistencies may occur when a modification affects multiple blocks; a crash may occur when some of the blocks have been written to disk but not the others. For example, when the system adds a block to a file, it updates the free list to indicate that the block is in use, but if the inode is not yet written to point to the block, this will result in an inconsistency. Another inconsistency can occur when the system while creating the link to a file to make a new directory entry refers to an inode, but the reference count has not yet been updated in the inode.

The process of resolving OS faults or errors is called **recovery**. Three approaches that can address inconsistency issues include:

- Check consistency during reboot, and repair problem. A good example of checking for inconsistency is the file system check (fsck) command implementation for UNIX and UNIX-like file systems. The system executes fsck as part of every system boot sequence so it can check whether the system was shut down correctly or not. If it was properly shut down, it proceeds normally. In the alternative (e.g., crash, power failure, or any other reason), the recovery process will start. The recovery process will scan disk contents, identify inconsistencies, and repair them. The limitations of fsck are as follows: it will restore disk to consistency, but does not prevent information loss. This loss of information can lead to instability. Also, the fsck has security issues because a block could migrate from the password file to some other random file, which could make it visible to unauthorized users. In addition, running fsck may take a long time, and the user will not be able to restart the system until fsck completes. The recovery process with fsck will take more time if the disk size is big. Figure 6.37 illustrates an example of the code errors produced from fsck and the meaning of each code in the Linux OS.

```
# man fsck

0       No errors
1       Filesystem errors corrected
2       System should be rebooted
4       Filesystem errors left uncorrected
8       Operational error
16      Usage or syntax error
32      Checking canceled by user request
128     Shared-library error
```

Figure 6.37 Running a recovery process with fsck resulted in these code errors. The meaning of each code in Linux OS is also given. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

- Check the order of the writes. This approach avoids some discrepancies by applying changes in a specific write sequence. For instance, to ensure the free list doesn't still contain the file's new block, write the content of the list before adding that block to the file. After ensuring that the list is not including that block, create a reference for the new block in the inode. Using this approach, you'll guarantee that you'll never write a pointer prior to initializing the block to which it points without validation. The validation will force the system to never clear the last pointer prior to setting a new pointer. The advantage of this approach is that it reduces the time spent waiting, as there is no need to wait for fsck while rebooting. However, there are several drawbacks, such as the potential for resource leaks (e.g., when the system runs fsck to recover some lost resources). Another drawback is that this approach slows file operation because writing while running the system requires considerable metadata.
- Perform write-ahead logging. This term is known as journaling file system and refers to the practice of recording the changes in the information in a separate log file sometimes called a journal file. These changes will be recorded prior to any new change or update on the system. Windows NTFS and Linux ext3 implement this kind of log file. The log procedure is analogous to the way log files are used in a database system to enable the correction of updated inconsistencies, which enables the healing quickly in case of any error. Prior to performing any operation, the recovery process will initially store information regarding the operation in a special log file. The next step is to flush the information to the disk before updating any other blocks. For example, a log entry such as "I'll add block 100101 to inode 313 at index 90" will be added to the system's log in case the operation involves adding a block to a file. This will guarantee that the actual block modifications can be performed. The system will restore the log in case of any crash to ensure that all of the updates have been saved in the disk. There are many benefits to employing logging such as reducing the time needed to recover from any failure. Also, improving the ability of localizing logs in the disk will result in improving the system's performance. However, this approach has some drawbacks, namely, the size of the log file will grow over time, and this will affect the system's processing time. This problem can, however, be resolved by performing periodic checkpoints.

Longevity

How long does an OS last? Did companies stop developing new OSs? How can the current OS survive? To answer these questions, we need to discuss concepts such as paging, TLBs, disks, storage latency, and multicores as well as virtual machines (VM).

Technology and OSs

Many of the basic ideas in OSs were developed 30–50 years ago, when technology was very different. The question is not only whether these ideas will still be relevant in the future, but whether they are relevant even today. After all, technology has changed considerably over the last thirty or so years. For example, CPU speeds went from 15 MHz in 1980 to 2.5+ GHz in 2024, a 167-fold increase. Memory size went from 8 MB to 16+ GB, a 2,000-fold increase. Disk capacity went from 30 MB to 2+ TB, a 6,667-fold increase. Disk transfer rate went from

2 MB/sec to 200+ MB/sec, a 100-fold increase. Network speeds went from 10 Mb/sec to 10+ Gb/sec, a 1,000-fold increase. As you can see, there were huge increases in size, speed, and other capabilities.

As you may recall, paging is a storage mechanism that allows processes to be retrieved from secondary memory and moved to main memory using pages. In the 1960s, paging originally touted disk speed latency of 80 ms, a data transfer rate of 250 KBs/sec, memory size of 256 Kbytes. Thus, for 64 pages, it took 6.4 sec to replace all of the memory to address individual page faults and 1 sec to address sequential page faults. Today, we have disk speed latency of 10 ms, a data transfer rate of 150+ MB/sec, and memory size of 64+ GB. For 16,000,000+ pages, it takes 44+ hours to replace all of memory to address individual page faults, and 320+ sec to address sequential page faults. Therefore, we cannot afford to page something out unless the system is going to be idle for a long time. But the real question is: does paging make sense anymore as a mechanism for the incremental loading of processes? The answer is yes, but by reading the entire binary at once because 15 MB of binary takes 0.1 sec to read.

TLBs have not kept up with memory sizes; 64 entries provide 256 KB coverage. In the mid-1980s, this was a substantial fraction of memory (i.e., 8 Mbytes). Today, TLBs can only cover a tiny fraction of memory. Some TLBs support larger page sizes of 1 Mbyte or 1 GB, but this complicates kernel memory management.

Disk capacity has increased faster than access time; storage access latency for disks is around 10 ms, and it is around 100 µs for flash memory. There are new nonvolatile memories, such as Intel's 3D XPoint, that improve the latency to 100ns–300ns.

Chip technology improvements allowed processor clock rates to improve rapidly. Unfortunately, however, faster clock rates mean more power dissipation, and now power limitations limit improvements in clock rate. Chip designers are now using technology to put more processors (cores) on a chip. In general, all OSs must now be multiprocessor OSs. However, it is not clear how to utilize these cores, and application developers must write parallel programs, which is very hard.

Lastly, the current/hot trend for OS development is the data center, which coordinates thousands of machines working together trying to achieve very low-latency communication.

LINK TO LEARNING

As nearly every person and business on the planet uses computers today, their reliability and security are increasingly essential. At the same time, there is growing concern about whether the underlying technologies we are relying on to power OSs will become obsolete soon. And there are also questions about what will replace OSs. Check out the debate on what will replace OS (https://openstax.org/r/76replaceOS) and see whether you share any of the concerns.

Virtual Machines

As you learned earlier in this chapter, a virtual machine is a software emulation of a physical computer that creates an environment that can execute programs and manage operations as if it were a separate physical entity. This emulation allows multiple operating systems that are isolated from each other to run concurrently on a single physical machine. In essence, a VM provides the functionality of a physical computer, including a virtual CPU, memory, hard disk, network interface, and other devices.

Recall that the underlying technology enabling VMs is called a hypervisor or virtual machine monitor (VMM). This technology resides either directly on the hardware (Type 1 or bare-metal hypervisor) or on top of an operating system (Type 2 or hosted hypervisor). The hypervisor is responsible for allocating physical resources to each VM and ensuring that they remain isolated from each other. This isolation ensures that processes running in one VM do not interfere with those running in another and thereby enhances security and stability. VMs are widely used for a variety of purposes, including server virtualization, software testing and

development, and desktop virtualization. Virtual machines have become a fundamental component of cloud computing, as they allow cloud providers to offer scalable and flexible computing resources to users on a pay-as-you-go basis.

Figure 6.38 illustrates the difference between a Type 1 virtual machine monitor and container environment such as via Docker. A **container** is a standardized unit of software that logically isolates an application, enabling it to run independently of physical resources.

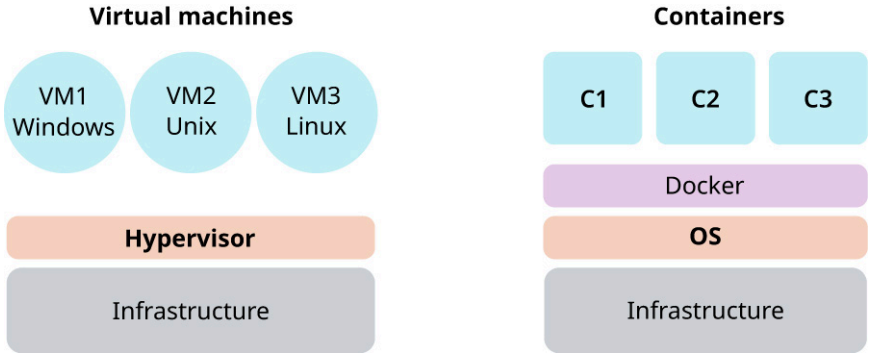


Figure 6.38 One notable difference between the virtual machine and containers is that VMs allow for the use of multiple operating systems, whereas containers share a single OS. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

When the complete OS is running within a VM, then the system will be called a **guest operating system**. VMs are heavily used in cloud computing such as Microsoft Azure, Amazon Web Services, Google Cloud Platform, and IBM Cloud.

THINK IT THROUGH

VMs vs. On-Premise Computing

VMs on the cloud represent a paradigm shift in how we utilize computing resources, offering compelling advantages over traditional on-premises computing. Cloud-based VMs provide scalability, flexibility, and cost-efficiency, making them a promising technology for businesses and individuals alike. In a traditional on-premises setup, a company or user must invest in physical hardware, maintain that hardware, and often overprovision resources to handle peak demand periods. This approach ties up capital and resources in equipment that may quickly become outdated or underutilized.

Why are virtual machines on the cloud a promising technology as compared to on-premises use of a computer?

Chapter Review

Key Terms

access control list (ACL) list of rules that specifies which users are granted the access to a specific object or resource

access enforcement part of the OS that is responsible for enforcing access controls and protecting authentication and authorization information

address space set of addresses generated by programs as they reference instructions and data

address translation stage of virtualization that occurs when the system needs to find a physical address in the memory that matches the virtual address

allocation method that defines how data is stored in the memory by providing a set of requests for resources and identifying which processes should be given which resources to make the most efficient use of the resources

application programming interface (API) set of rules and tools that allows different software applications to communicate with each other

authentication process of checking to see if a user's credentials match the credentials in a database of authorized users or in a data authentication server

authorization process of determining the relationship between principals, operations, and objects by defining which principals can perform which operations on which objects

badge logical access system that serves as a form of authentication

blocked state when a process is waiting for an event to occur

cache-only memory architecture (COMA) computer memory architecture where all memory is treated as cache, which allows for the dynamic allocation of data for optimized access and performance in multiprocessor environments

capability list list of objects and operations for each user that defines the user rights and capabilities

cloud infrastructure virtualized and scalable hardware resources delivered over the Internet as a service

concurrency multiple activities and processes happening at the same time—in other words, the OS handling multiple tasks at once

concurrent processing computing model that improves the performance when multiple processors execute instructions simultaneously

condition variable synchronization mechanism that is used in conjunction with locks to allow threads to wait for certain conditions to become true

container standardized unit of software that logically isolates an application enabling it to run independently of physical resources

context switch procedure that a computer's CPU follows to change from one task to another while ensuring that the tasks do not conflict

CPU state consists of the program counter (PC), the stack pointer (SP), and general-purpose registers

critical section piece of code that only one thread can execute at once; also, only one thread at a time will get into this section of code

deadlock synchronization challenge that occurs when a set of threads are blocked forever, waiting for each other to release resources

demand paging storage mechanism in which pages should only allocate in the memory if it is required from the execution process

device manager layer in the layered OS architecture that handles devices and provides buffering

device register interface view any device presents to a programmer where each I/O device appears in the physical address space of the machine as a few words

directory set of files that contains all the required information about the files such as attributes, location, and ownership, which is managed by the OS

distributed file system (DFS) file system that is distributed on multiple file servers or multiple locations that

support network-wide sharing of files and devices

dual mode OS structure that is responsible for separating the user mode from the system mode

dynamic linking code is located and loaded when the program is first run

encryption method that converts the data into secret code that hides the data's true meaning

exception error that occurs at runtime

exokernel OS architecture that simplifies the operating system by making its core (kernel) really small and letting apps have more control over the computer's hardware

fault containment feature of an OS that prevents errors in one part of an application from affecting the whole system

fault recovery feature of an OS that helps the system to fix itself or revert to a previous state after an error

fault tolerance feature of an OS that allows the application to keep running even when errors occur

file collection of related information that is stored on secondary/virtual storage; it's the smallest storage unit from the user's perspective

file system responsible for defining file names, storing files to a storage device, and retrieving files from storage devices

file versioning system that allows a file to exist in several versions at the same time, which gives the user complete control over file creation

first come, first served (FCFS) scheduling algorithm that operates on a simple queue mechanism where the first process to request the CPU is the first to receive it (or the first element added to the queue is the first one to be removed); commonly used in resource scheduling and data buffering and also known as FIFO (first in, first out)

fragmentation problem where the memory blocks cannot be allocated to the processes due to their small size and the blocks remain unused

frame buffer portion of random access memory (RAM) containing a bitmap that drives a video display

full virtualization memory virtualization approach that allows multiple operating systems to run concurrently on a single physical machine, fully isolated from each other, by emulating hardware resources through a hypervisor

general-purpose register (GPR) extra register that is used for storing operands and pointers

graphical user interface (GUI) visual interface that allows users to interact with electronic devices through graphical icons and visual indicators

guest modification in the context of virtualization, altering the guest operating system or its configuration to improve compatibility, performance, or integration with the virtualization environment or hypervisor

guest operating system complete operating system inside a virtual machine

hardware abstraction layer (HAL) example of layering in modern OS that allows an OS to interact with a hardware device at a general or abstract level rather than going deep into a detailed hardware level; this improves readability

heap allocation dynamic storage management approach that allocates the data in a tree-based data structure

heap data tree-based data in process management

hypervisor software layer between machine hardware and the operating systems that run on it

i-number unique number given to an inode whenever an OS that uses inodes creates a new file, which, in effect, functions as the file's name

inconsistency situation that causes the system to produce errors or hardware failure

incremental backup backup image containing the pages that have been updated from the time of the previous backup

inode structure used by file system that contains metadata about a file or directory but not its name

inter-process communication (IPC) mechanism that enables processes to exchange data among different processes running on an operating system

interrupt signal to the processor from either software or hardware that indicate events that need immediate attention

isolation ensures that the multiple programs that are running concurrently on the same CPU and memory operate independently without interfering with each other's execution or data

layered OS architecture OS architecture where the OS is implemented as a set of layers where each layer exposes an enhanced virtual machine to the layer above

lock synchronization mechanism that is used to enforce mutual exclusion

mechanism activities that enforce policies and often depend on the hardware on which the operating system runs

memory allocation process of setting aside sections of memory in a program to be used to store variables and instances of structures and classes

memory deallocation process of freeing the space corresponding to finished processes when that space is needed by the rest of the system

memory multiplexing dividing the capacity of the communication channel into multiple logical channels

microkernel OS architecture where the functionality and capabilities are added to a minimal core OS as plug-ins

monolithic design OS architecture where the entire OS is working in kernel space

multitasking approach toward achieving concurrency that makes it possible for the OS to run multiple processes at the same time using time slicing

mutual exclusion program that prevents simultaneous access to a shared resource

non-privileged system program program that can run only in the user mode

non-uniform memory access (NUMA) computer memory architecture where memory access time varies depending on the memory's location relative to a processor

operating system (OS) core piece of software that typically manages the interconnection of hardware and software on a given computer

page fault when the CPU demands a page, and this page is not present in the main memory

page fetching process of bringing pages into memory

page manager layer in the layered OS architecture that implements virtual memories for each process

page replacement when one page in the DRAM is swapped to disk while the requested page is brought into DRAM

paging storage mechanism that uses a page form in retrieving process from secondary or virtual memory to main memory

pipe data communication method between two processes that uses a specific name and standard I/O operations, and thus allows for data transfer within a file system; sometimes called named pipe

policy controls how to use a mechanism in specific situations, that is, choose what activities need to be done

primary memory type of computer memory that is the initial point of access for a processor and serves as direct storage for the CPU

primary storage holds data that can be directly accessed by the CPU with minimum or no delay and does not survive a power failure

privileged instruction instruction provided by the CPU that can be executed only by the OS

privileged system program program that can run only in the system mode

process any program that is running on top of the OS

process control block (PCB) data structure used by the operating system to store information about a process, including its state, process ID, registers, scheduling information, memory management details, and I/O status

process ID (PID) unique identifier assigned by the operating system to each process running on a computer, used to track and manage process activities

process synchronization when an OS manages the sharing of a system's resources to avoid interference and errors

properties characteristics that are considered when designing an OS

protection general mechanism that is used throughout the OS for all resources that need to be protected, such as memory, processes, files, devices, CPU time, and network bandwidth

quota amount of space to store files based on the available memory space
ready state when a process is waiting for the CPU
recovery process of resolving or receiving treatment to solve OS faults or errors
reliability system's ability to reduce faults and ensure that the information in the system survives OS crashes and hardware failures
replication procedure that allows multiple copies of a file to exist in the network; this improves performance and availability
round-robin scheduling (RR) scheduling algorithm that is widely used in time-sharing systems and is designed to ensure fairness among processes by giving each process an equal share of the CPU
running state when a process is being executed by the CPU
secondary memory type of computer memory that is nonvolatile and thus used for long-term storage, housing the operating system, applications, and data that need to persist even when the power is off
secondary storage persistent memory that survives power failures most of the time such as spinning disks, SSDs, and USB drives
semaphore data type that an OS uses to control access to a resource
sharing multiple processes can use the same piece of data concurrently
shortest remaining processing time (SRPT) scheduling algorithm that prioritizes processes based on the shortest amount of time left to complete their execution
shortest time to completion first (STCF) scheduling algorithm that takes the best approach to minimize the waiting time, but it requires that the processor knows the processing time in advance; also called shortest job first (SJF)
stack allocation dynamic storage management approach that uses linear data structure that follows last in, first out (LIFO)
stack pointer (SP) register that indicates the location of the last item that was added to the stack
static data data that does not change within the program
synchronization way of coordinating multiple concurrent activities that are using shared state
system call appears when the program requests a service from the kernel
system interrupt manages the communication between the computer hardware and the system
thrashing when a computer's operating system becomes overwhelmed by the number of processes requesting memory
thread smallest unit of execution within a process, allowing parallel tasks to run in the same memory space; it enables efficient and independent execution of sequences of instructions
time slice short time frame that gets assigned to a process for CPU execution and facilitates multitasking
translation lookaside buffer (TLB) small memory cache that speeds up the computer's memory access by storing recent virtual-to-physical address translations; if the TLB has the address translation, it quickly retrieves data; if not, the computer must search more slowly through its memory
two-factor authentication form of authentication that involves two factors: the system calls or texts a user's phone for the traditional password during login, employing the cell phone as a key
uniform memory access (UMA) computer memory architecture where access time to any memory location is the same across all processors
virtual machine (VM) software that is created to run like a physical computer and that operates its own operating system and applications like a separate physical server
virtualization allows a system to run different types of applications used by multiple users at a time on the same computer
working set size (WSS) total amount of memory a process requires during a specific period of activity; it is measured as the set of pages or data blocks the process accesses

Summary

6.1 What Is an Operating System?

- An operating system (OS) is at the core of all of the connected hardware and software.

- Improving efficiency results in speeding up the implementation of applications from coding time and runtime standpoints. OSs have a large influence because of the abstractions/interfaces they implement.
- Operating systems provide both mechanism and policy. Mechanism refers to a set of activities that you can do. Policy is how to use the mechanism in specific situations.
- Virtualization in an operating system allows the system to run different applications that are handled by multiple users at a time on the same computer.
- Server virtualization places a software layer called a hypervisor (e.g., virtual machine monitor or VMM) between a machine (e.g., server) hardware and the operating systems that run on it.
- Using OS-level or server virtualization allows a server to run different types of operating systems at the same time on the same computer.
- The OS translates from the hardware interface to the application interface and provides each running program with its own process.
- A process consists of address space, one or more threads of control executing in that address space, and additional system state associated with it. The thread is a path of execution within a process and a process may contain multiple threads.
- The instruction set architecture (ISA) defines a set of instructions that can be used to write assembly language programs that use the CPU while abstracting the hardware details from the program.
- OS functions guarantee protection, isolation, and sharing of resources efficiently via resource allocation and communication.

6.2 Fundamental OS Concepts

- An OS manages computer resources (hardware) and provides services for computer programs (software).
- An OS is a complex system and executes many kinds of activities ranging from executing users' programs, to running background jobs or scripts, to completing system programs.
- Processing involves a program, a process, and a processor. An OS is responsible for managing processes, and different OSs approach process management in different ways.
- The address space is the set of addresses generated by programs as they reference instructions and data. The memory space holds the actual main memory locations that are directly addressable for processing.
- Computer memory consists of two main types: primary and secondary memory. An OS manages memory space through memory allocation and memory deallocation as well as by maintaining mappings from virtual addresses to physical and switching CPU context among addresses spaces.
- Device drivers are the routines that interact directly with specific device types and related hardware to indicate how to initialize the device, request I/O, and handle interrupts or errors.
- A device register is the interface a device presents to a programmer, whereas each I/O device appears in the physical address space of the machine as a few words.
- In an OS, it is important to have dual mode operations to ensure a high level of security and authority. The dual mode is responsible for separating the user from the kernel mode.
- Successful OS designs have had a variety of architectures, such as monolithic, layered, microkernels, and virtual machine monitors. As the design of an OS—and even its role—are still evolving, it is simply impossible today to pick one "correct" way to structure an OS.
- A monolithic OS design is an OS architecture where the entire OS is working in kernel space.
- A layered OS architecture consists of implementing the OS as a set of layers where each layer exposes an enhanced virtual machine to the layer above.
- Hardware abstraction layer (HAL) is an example of layering in modern OSs. It allows an OS to interact with a hardware device at a general or abstract level rather than going deep into a detailed hardware level, which improves readability.
- In a microkernel OS architecture, the functionality and capabilities are added to a minimal core OS.

6.3 Processes and Concurrency

- Concurrent processing is a computing model that improves performance when multiple processors are executing instructions simultaneously.

- A process consists of at least an address space, a CPU state, and a set of OS resources.
- The OS's process namespace particulars depend on the specific OS, but in general, the name of a process is called a PID (process ID), which is a set of unique numbers that identify processes.
- The OS maintains a data structure to keep track of a process state, which is called the process control block (PCB) or process descriptor.
- Concurrency refers to multiple activities and processes happening at the same time. An OS can achieve concurrent processing via the use of threads or one of three different processing environments: multiprogramming, multiprocessing, or distributed processing.
- Scheduling is the act of determining which process is in the ready state and should be moved to the running state when more resources are requested than can be granted immediately, and in which order the requests should be serviced.
- A good scheduling algorithm minimizes response time, efficiently utilizes resources, and implements fairness by distributing CPU cycles equitably. Four simple scheduling algorithms are FCFS, RR, STCF, and SRPT.
- Synchronization is a way of coordinating multiple concurrent activities that use a shared state.
- Allocation is a method that defines how data is stored in the memory by providing a set of requests for resources and identifying which processes should be given which resources to make the most efficient use of the resources. There are three main forms of allocation: contiguous allocation, linked allocation, and indexed allocation.

6.4 Memory Management

- The OS loads executable files into memory, allows several different processes to share memory, and provides facilities for processes to exceed the memory size after they have started running.
- Memory multiplexing is dividing the capacity of the communication channel into multiple logical channels.
- There are several concepts that are critical to memory multiplexing, namely, isolation, sharing, virtualization, and utilization.
- Time slicing is a time frame for each process to run in a preemptive multitasking CPU such that each process will be run every single time slice.
- Sharing means that multiple processes can share the same piece of data concurrently.
- Memory sharing improves the performance of the system because the data is not copied from one address space to another, so memory allocation is done only once.
- Virtualization is a technique that gives an application the impression that it has its own logical memory and that it is independent from the available physical memory.
- Fragmentation is a problem where the memory blocks cannot be allocated to the processes due to their small individual size and the distribution of sizes in the pool; there might be enough total free memory to satisfy the demand, but the available chunks cannot be allocated contiguously.
- Linkers combine many separate pieces of a program, reorganize storage allocation so that all the pieces can fit together, and touch up addresses so that the program can run under the new memory organization.
- There are two basic operations used in dynamic storage management to manage a memory or storage to satisfy various needs: allocate a block with a given number of bytes or free a previously allocated block.
- Virtual memory is a key component of the operating system for ensuring process isolation by guaranteeing that each process gets its own view of the memory.

6.5 File Systems

- A file system is responsible for defining file names, storing files on a storage device, and retrieving files from a storage device.
- File systems define operations on objects such as create, read, and write, and they may also provide higher-level services, such as accounting and quotas, incremental backup indexing or search, file versioning, and encryption.
- File systems are concerned with lower-level characteristics such as performance and failure resilience.

- The file system interface defines standard operations such the creation and deletion of files (or directories), manipulation of files and directories, copy, and lock.
- File systems are responsible for managing parts of the disk that are used (inodes) and parts of the disk that are not used (free blocks).
- A distributed file system (DFS) is a file system that is distributed on multiple file servers or multiple locations that support network-wide sharing of files and devices.
- A DFS provides an abstraction over physical disks that is akin to the abstraction that virtual memory provides over physical memory.

6.6 Reliability and Security

- We consider an OS to be reliable if it delivers service without errors or interruptions.
- Protection is a general mechanism used throughout the OS and for all resources needed to be protected such as memory, processes, files, devices, CPU time, and network bandwidth.
- There are three aspects to a protection mechanism: authentication, authorization, and access enforcement.
- The traditional way of authentication involves a password, which is a secret piece of information used to establish the identity of a user and should be relatively long and hard to guess. Another form of authentication is two-factor authentication, which involves two factors: the system calls or texts a user's phone for the traditional password during login, employing the cell phone as a key.
- The authorization determines the relationship between principals, operations, and objects by defining which principals can perform which operations on which objects.
- An access control list (ACL) is a list of rules that specifies which users are granted access to a specific object or resource.
- A capability list is a list of objects and operations for each user that defines the user rights and capabilities.
- To support access enforcement, one part of the OS must be responsible for enforcing access controls and protecting authentication and authorization information.
- There are many advantages to using logging: recovery is much faster; it eliminates inconsistencies; a log can be localized in one area of disk, which makes log writes faster; and it results in better performance. One of the disadvantages of logging is that synchronous disk write happens before every metadata operation.
- Virtual machines have become a fundamental component of cloud computing, as they allow cloud providers to offer scalable and flexible computing resources to users on a pay-as-you-go basis.



Review Questions

1. What is a privileged instruction that can only be executed by the kernel in Windows 10 or macOS operating systems?
   a. opening a text file
   b. modifying system clock settings
   c. printing a document
   d. creating a new user directory

2. You are building your own computer and have finished installing all hardware components. What should you install first?
   a. Microsoft Office
   b. Microsoft Windows OS
   c. external I/O device drivers
   d. antivirus software

3. What process or component allows a system to run different types of applications used by multiple users at a time on the same computer?
   a. virtualization

b. kernel
c. operating system
d. thread

4. How is efficiency defined with regard to operating systems?

5. What is virtualization as it relates to OSs?

6. Who sets policies in OSs?

7. What is the difference between user mode and kernel mode?

8. What component handles devices and provides buffering?
a. device driver
b. device register
c. device manager
d. I/O devices

9. How can a monolithic OS design be described?
a. an OS architecture where the entire OS is working in kernel space
b. OS architecture where the functionality and capabilities are added to a minimal core OS as plug-ins
c. an example of layering in modern operating systems
d. a computer memory design where memory access time varies depending on the memory's location relative to a processor

10. What type of memory access is described as computer memory architecture where access time to any memory location is the same across all processors?
a. cache-only memory architecture (COMA)
b. non-uniform memory access (NUMA)
c. uniform memory access (UMA)
d. random access memory (RAM)

11. What are the main components of any operating system?

12. What are the differences between thread and process?

13. What does the hardware abstraction layer (HAL) refer to?

14. What scheduling algorithm prioritizes processes based on the shortest amount of remaining execution time?
a. first come, first served (FCFS)
b. round-robin (RR)
c. shortest remaining processing time (SRPT)
d. priority scheduling

15. What is synchronization?
a. the way of coordinating multiple concurrent activities that are using a shared state
b. computing model that improves the performance when multiple processors execute instructions simultaneously
c. the memory that can be accessed by multiple processes and the processes that can communicate with each other without the middleman
d. the data communication method between two processes, using a specific name and standard I/O operations, allowing for data transfer within a file system

16. What is an example of static data?
a. a variable to keep track of the number of iterations in a loop in a program
b. the date and time in the operating system

c. the name of a file in a directory
d. a hardcoded country code in a program that is created with the final keyword

17. How are processes managed by the OS conceptually?

18. How are I/O devices managed by the OS conceptually?

19. Why is scheduling counted as an important operation in OSs?

20. What is the term for a technique where a process's memory is divided into various segments or sections, each representing different types of data or code?
a. time slicing
b. paging
c. isolation
d. segmentation

21. Stack allocation uses what data processing technique?
a. last in/last out
b. first in/first out
c. first in/last out
d. last in/first out

22. How does a linker work?

23. What is the difference between static and dynamic linking?

24. How does caching relate to virtual memory?

25. What component is responsible for defining file names, storing files to a storage device, and retrieving files from a storage device?
a. file system
b. file versioning
c. file
d. file path

26. What is a directory?
a. persistent memory that survives power failures most of the time, such as spinning disks, SSDs, and USB drives
b. a collection of related information that is stored on secondary/virtual storage and is the smallest storage unit from the user’s perspective
c. a system that allows a file to exist in several versions at the same time, which gives the user complete control over file creation
d. a set of files that contains all the required information about the files, such as attributes, location, and ownership, which is managed by the OS

27. What is a distributed file system?

28. What is an inode?

29. Define the file system interface.

30. What is the term for the operating system that is virtualized?
a. guest operating system
b. host operating system
c. default operating system
d. dual boot operating system

31. What is the term for checking to see if a user's credentials match the credentials in a database of

authorized users or in a data authentication server?

a. authorization
b. access enforcement
c. authentication
d. badge

Conceptual Questions

1. What is the difference between a policy and a mechanism? Please give examples to illustrate your explanation.
2. How are the compiler, OS, and CPU ISA coordinated? What are all the code modules, where do they exist, and how do they cooperate?
3. Give an example of an OS that uses a layered design.
4. What alternatives to monolithic OS design have been tried?
5. Explain in detail how caching relates to the use of virtual memory.
6. Explain how virtual memory became a key component of the operating system.
7. Explain in detail the file system's higher-level services.
8. Explain how the file versioning will help the user and the system.
9. Explain the difference between authentication, authorization, and access control. Are there any other types of security protections you would want an OS to provide when using software applications?

Practice Exercises

1. Search on the Web for "Windows system structure" and compare it with "UNIX/Linux system structure."
2. Draw a high-level diagram that illustrates the flow of control for an application of your choice that leverages an OS. Make sure that you identify the various components and layers as well as the users involved, if any.
3. Based on the operating system you are using, search on the Web for your operating system architecture.
4. Draw the architecture of your operating system.
5. Give an example of a scenario that requires synchronization.
6. Search on the Web for the most used allocation mechanism.
7. Explain how segments and pages are used to support virtual memory.
8. Search the Web for how to find the total number of inodes using your operating system.
9. Explain the relative merits of various recovery approaches. Start with the ones mentioned in the book and explore more on the Internet as needed.
10. Research various encryption algorithms and provide a summary of the results found.

Problem Set A

1. You have a Windows computer and need to test software that you developed in a Linux environment. How can you test your software with one machine?
2. Explain why we need to study OS architecture.
3. Explain how an OS decides how much physical memory to allocate to each process and decides when to remove a process from memory.

4. Explain why we need to study OS allocation methods.
5. Explain how an OS decides how much physical memory to allocate to each process and decides when to remove a process from memory.
6. Explain how the memory is divided.
7. Explain fragmentation.
8. Imagine you're tasked with creating a new file system that will only be utilized to store videos on YouTube. Describe the kind of access patterns you anticipate occurring most frequently in that specific file system.
9. Suppose you are asked by a company to select a new authentication method. If the company is not using multi-factor authentication, how could you argue the need for this method?
10. How can you use badging in the authentication process?

Problem Set B

1. Write a simple piece of code on an OS of your choice that calls a function in a programming language of your choice and explain how your program uses the stack.
2. Write a simple piece of code on an OS of your choice using a programming language of your choice that makes use of the heap.
3. Outline the fundamental differences between two of the most popular mainstream operating systems (e.g., Max OS X, Windows 10, Linux) from an OS architecture and OS components standpoint. Do some research on the Internet to obtain architectural diagram and component descriptions from a trustworthy source and show all your work.
4. Most of the OSs now are moving to multiprocessing. Explain how multiprocessing reduces the latency and increases the overall performance.
5. Discuss the benefits of memory sharing for the user and OS perspective.
6. How and why might a file system created specifically for storing movies on YouTube's website differ from the businesses outlined in this book?
7. What are the advantages and disadvantages of using logging in your OS?

Thought Provokers

1. Consider our start-up company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe how it can best use an operating system to create products or services that can generate business (e.g., mobile health application that detects elevated levels of stress and suggests playing games, listening to songs, or watching videos to reduce stress). Give precise examples and explain how the start-up would be able to scale the resulting business (i.e., keep sustaining the cost of doing business while increasing its number of customers).
2. Consider our start-up company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe how it could leverage a new operating system design to support an innovative application that leverages the use of various sensors located at the edge of the network. Give some precise examples and explain how the start-up would be able to scale this approach.
3. Consider our start-up company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe how it could leverage IPC and concurrency control to support an application that makes it possible to collect data at the edge of the network from a large variety of sensors and enable processing of that data in real time. Give some precise examples and explain how the start-up would be able to scale this approach in the context of an epidemic such as COVID-19.

4. Consider our start-up company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe how it could leverage memory management to support very memory-demanding applications, making it possible to perform all computations on data in memory. Are there some examples of similar technologies that already exist today? Give some precise examples and explain how the start-up would be able to scale this approach.
5. Consider our start-up company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe how it could leverage a distributed file system to make it possible to gather mission-critical data in real time from various users located at the edge of the network. Give some precise examples and explain how the start-up would be able to scale this approach.
6. Consider our startup company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe how it could leverage OS technology obsolescence. Give some precise examples and explain how the startup would be able to scale this approach.

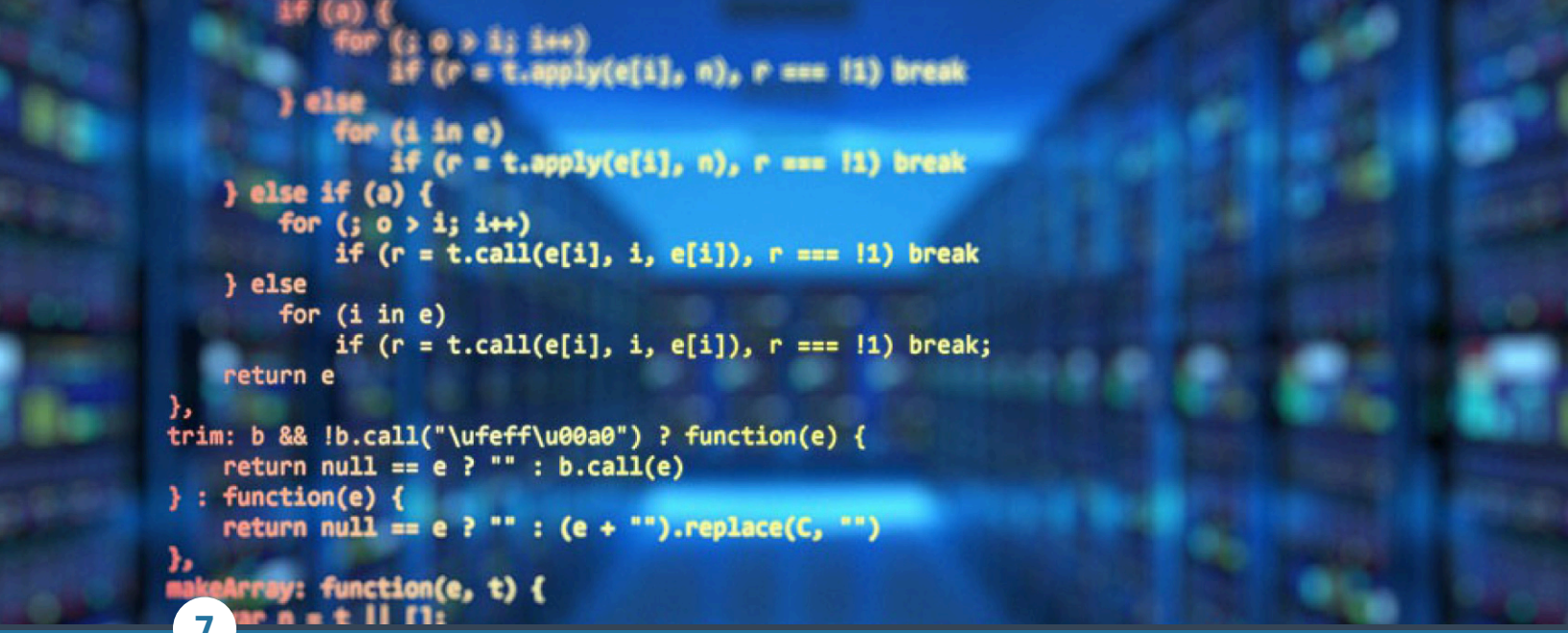
Labs

1. Install Oracle Virtual Box on your laptop and deploy an image of an operating system in Virtual Box that makes it possible to use a different operating system on your laptop.
2. Create a Linux virtual machine on a cloud of your choice and install X2Go to access the virtual machine from your laptop. Explain how the OS and windowing system make all of this possible.
3. Write a program in a programming language of your choice and deploy it using the OS of your choice. Use a GNU compiler tool to compile and link your code and demonstrate how your program makes used of memory management (e.g., dynamic memory allocation).
4. Create a file system on a cloud of your choice and mount it as a drive on your computer. Perform some experiments with various applications of your choice to determine if the performance is acceptable. Experiment with the SaaS functionality provided on various big clouds as you work on this lab.

   Examples with SaaS:

   - Office suites: Use SaaS offerings like Google Workspace or Microsoft 365 to create and edit documents. Observe the responsiveness of these services.
   - Development tools: Experiment with cloud-based IDEs like AWS Cloud9 or GitHub Codespaces to develop and run code. Pay attention to the execution speed and any latency in the development process.
   - Database management: Work with a cloud-based database service like Amazon RDS. Perform queries and updates to test performance.
   - Analytics: Utilize services like AWS QuickSight or Looker Studio to perform data analysis tasks. Evaluate the speed of data processing and visualization rendering.

5. Research the recovery features that are available on your computer's OS and document what you would need to do in case of a system crash. Create a recovery disk as needed so that you are prepared for the worst.



7

High-Level Programming Languages

Figure 7.1 High-level languages make it easier for programmers to solve problems and design software at a level above the computer's architecture. (credit: modification of "Computer science and engineering" by "BVECJordan"/Wikimedia Commons, CC0)

Chapter Outline

7.1 Programming Language Foundations
7.2 Programming Language Constructs
7.3 Alternative Programming Models
7.4 Programming Language Implementation

Introduction

Programming is the science behind writing programs, which makes it possible to implement algorithms that leverage mathematical and/or scientific knowledge. Programming is also an art that requires creativity and employs imagination. High-level languages (HLLs) give programmers the ability to produce linguistic realizations of algorithms using a notational system that facilitates human-computer interaction.

TechWorks is an example of a company focused on new technology; for it to leverage technology and fulfill its stated mission, it regularly makes decisions on which HLLs to use, what exactly to use them for, and many other HLL suitability factors such as the following:

- Types of application
- Target platforms
- Maintainability
- Scalability
- Performance
- Security

TechWorks will need to choose from a pool of programming languages that excel in different areas. For example, JavaScript is a versatile language that applies to the interactive elements that users will see and interact with when using TechWorks's web interfaces. JavaScript is a natural choice for this task due to its ability to create dynamic and engaging user experiences. For server-side operations, the choice of programming languages must strike a balance between latest technology needs and experienced

programmers' preferences. JavaScript with frameworks like Node offers a cutting-edge approach, while established languages like PHP or ASP.NET boast a larger pool of seasoned programmers. TechWorks will need to use the Structured Query Language (SQL) to communicate with database systems used to support its applications. Python is a powerful tool language for data analysis and manipulation. Its extensive libraries and clear syntax make it well-suited to extract insights from TechWorks's collected data. In a nutshell, TechWorks will need to strategically combine the use of various programming languages to create robust and user-friendly applications.

7.1 Programming Language Foundations

Learning Objectives

By the end of this section, you will be able to:

- Describe what HLLs are
- Summarize choosing appropriate HLLs
- Outline the history of HLLs
- Describe the implementation of HLLs

A high-level programming language is designed to be easy for humans to read, write, and understand. It abstracts away most of the complexities of the underlying hardware and machine code, allowing programmers to focus on solving problems and designing software without needing to manage the low-level details of the computer's architecture.

What Are HLLs?

High-level programming languages give humans the ability to direct computers to perform tasks and applications. There are many HLLs to choose from. Java is a popular choice for its ability to run on various operating systems (i.e., Windows, macOS, Linux) and mobile platforms (Android). This is called cross-platform compatibility. For development specifically targeting Windows systems, C# is another strong option. Additionally, to create the visual elements of a website, programmers can utilize HTML and CSS. HTML provides the structure and content of the web page, while CSS controls how web pages are styled and presented. Over time, many HLLs have evolved into a mature set of tools that are used to create modern applications (Figure 7.2).

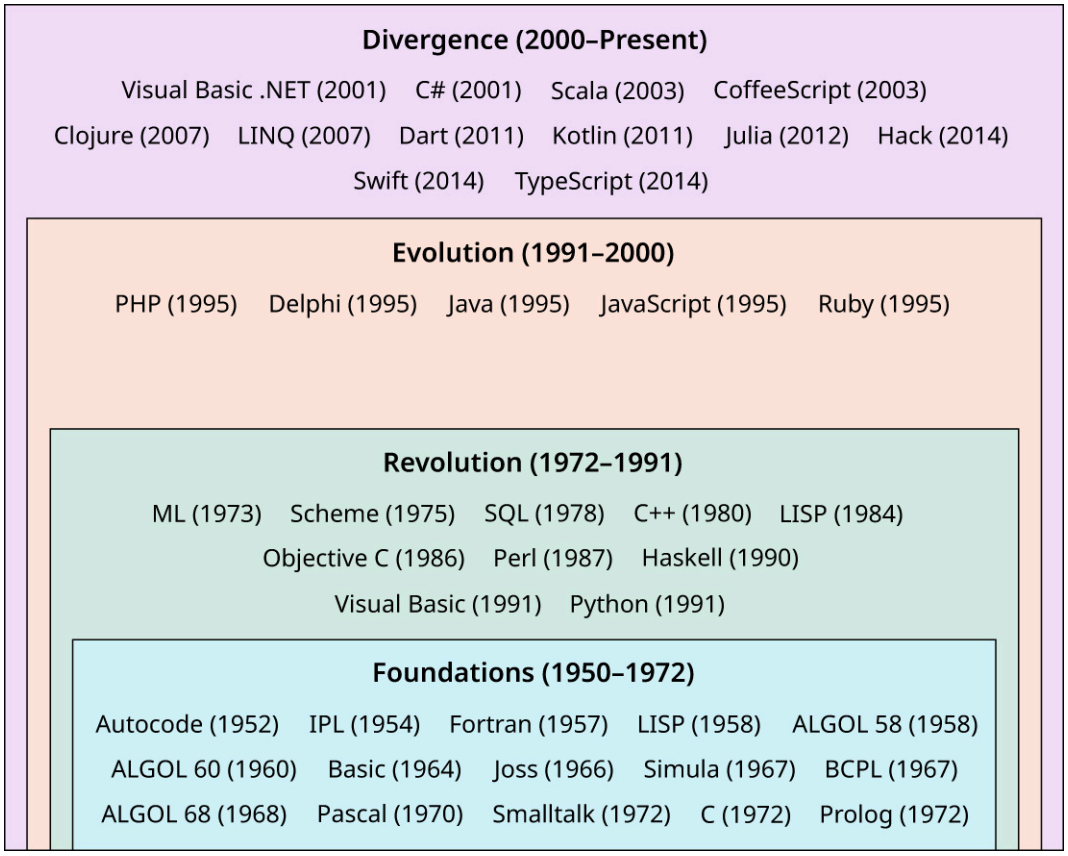


Figure 7.2 High-level programming languages have advanced from the foundational languages in the middle of the 20th century to more than 2500 HLLs that exist today. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

There is a possibility that the widespread use of a variety of HLLs to develop networked and mobile applications at a global scale creates potential cybersecurity issues. Some HLLs are considered more secure than others. Read this article on the most secure programming languages (https://openstax.org/r/76ProgLanguages) for further information about the security—or lack thereof—of these languages.

Learning Motivations

Studying the fundamental concepts provided by various HLLs is necessary to choose them correctly, employ them effectively, and program efficiently. From a user point of view, examining HLL concepts helps the user get better at thinking and expressing algorithms. From an implementor's point of view, understanding HLL concepts helps programmers abstract away from (virtual) machines and become better at specifying what they want the hardware to do without getting down into the bits. In the end, studying HLL concepts helps programmers make better use of whatever HLL they use.

Implementing Abstraction

One way to relate to **abstraction** is as a way of thinking and expressing algorithms to indicate what the programmer wants the hardware to do. For example, the following statement represents one form of abstraction in the Java programming language:

```
System.out.println("Hello world!");
```

It tells the computer's operating system at a high-level of abstraction to output a string of characters, which

practically consists of moving the pixels that form characters one by one to a hardware device.

Implementing a high level of tasks would be impossible without abstraction. For example, you would not want to program an invoicing application in 1s and 0s (machine language); abstraction allows a programmer to build it in an English-like syntax.

Abstraction may be taken to much higher levels. It is one of three central principles (along with encapsulation and inheritance) in such object-oriented HLLs as C++, Java, C#, and Python. Various programming paradigms were introduced in Chapter 4 Linguistic Realization of Algorithms: Low-Level Programming Languages, particularly the mechanisms of object-oriented programming (OOP) and its standards, perspectives, or sets of ideas that may be used to describe the structure and methodologies of an HLL. Object-oriented HLLs help organize software design around data, or objects, rather than functions and logic, as we will discuss in Alternative Programming Models.

Choosing Appropriate HLLs

Studying and understanding HLL concepts allows us to make most efficient use of them by becoming familiar with various criteria that may be used to evaluate them, which helps us choose the most appropriate language for a project. Some of these criteria are listed in Table 7.1, which also shows how they are related to the characteristics of an HLL. These criteria are as follows:

- **readability**: measures how easily an HLL can be read and understood
- **writability**: measures how easily an HLL can be used to create and modify programs
- reliability: measures conformance to specifications

There are many other criteria including scalability, cost, flexibility, efficiency, portability, and maintainability. These can be used to identify which HLL is best suited for a given task.

Characteristic	Readability	Writability	Reliability
Simplicity: a manageable set of features and constructs	•	•	•
Orthogonality: a relatively small set of primitive constructs can be combined in a relatively small number of ways	•	•	•
Data types: adequate predefined constructs to hold data	•	•	•
Syntax design: form and meaning via self-descriptive constructs and meaningful keywords	•	•	•
Supports abstraction: hides all but the relevant data about an object in order to reduce complexity and increase efficiency		•	•
Expressivity: relatively convenient ways of specifying operations		•	•
Type checking: built-in testing for type mismatches			•

Table 7.1 Criteria for Measuring Characteristics of HLLs

Characteristic	Readability	Writability	Reliability
Exception handling: support for catching run-time errors and specifying corrective measures			•
Restricted aliasing: presence of two or more distinct referencing methods for the same memory location			•

Table 7.1 Criteria for Measuring Characteristics of HLLs

Learning New HLLs

Studying the concepts of HLLs makes it easier to learn new HLLs since most have similarities in syntax, structure, and semantics. There are also several best practices that apply to different HLLs. A **best practice** is the most accepted style and structure of code that can be used to ensure proper software development, which makes it possible to learn new languages easily once a programmer has mastered a given one. The HLLs that are most used as teaching languages today are Java, C++, and Python. Java and C++ are languages that take a significant amount of study to master, while Python is considered a much simpler language to learn.

LINK TO LEARNING

HTML and CSS are markup languages and not exactly programming languages like Java or Python. The official HTML and CSS standards (https://openstax.org/r/76HTMLCSSStds) are available at World Wide Web Consortium (W3C).

Best Use of HLLs

Programmers have to figure out how HLLs support certain features. For example, a **variable** gives a name to a memory location that is used in any HLL to hold a value. However, different languages use variables differently. Java is a **strongly typed** language, meaning that a variable may only contain a value of one of the language's defined data types for its entire existence. Therefore, a variable that is a number cannot become a string of text. JavaScript is **weakly typed** so a variable may at different times hold values of any of the language data types. It may be storing a number, then later, the same variable may store a string of characters.

Another example is the use of pointers in C and C++. As visible in Figure 7.3, the pointer is the variable that holds actual computer memory addresses, but they do not exist in Java. However, understanding how C handles memory makes it easier to understand how data is passed from one place to another in Java or C#.

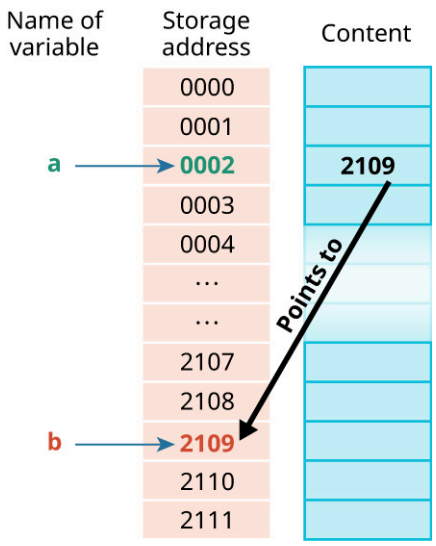


Figure 7.3 A C pointer variable "a" holds the memory address of the "b" variable. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Languages Are Purpose Driven

Let's again contrast C/C++ and JavaScript. Much of the syntax and grammar are the same, as are the flow of control structures of the language.

LINK TO LEARNING

You can review the current C/C++ standard (https://openstax.org/r/76C++Standard) to dig deeper into a syntax comparison.

So why pick one over the other? The answer is that languages are designed to fulfill certain purposes. C/C++ is a general-purpose programming language. As such, it is powerful for applications that include both systems programming and object-oriented graphical user interface (GUI) programming, JavaScript is intended for web programming.

Although programming languages differ in syntax, they all have libraries or packages that are installed as part of the language development environment. These libraries expose various functions via an application programming interface (API). These functions support the tasks for which the language is purposed while not requiring additional coding. The following illustrates the use of a C++ library function that prints a string to the screen:

```
cout << "Hello world!";
```

JavaScript's API contains a comprehensive set of features that enable the manipulation and dynamic behavior of web pages. A JavaScript API function that prints to the web page:

```
document.write("Hello world!");
```

TECHNOLOGY IN EVERYDAY LIFE

Using Map APIs to Navigate Your World

APIs are toolkits for programmers. They provide building blocks that make it easier to create software

applications. API functionality can also help people with everyday life situations. For example, Google provides a JavaScript Maps API for customizing map content to display on web pages. Imagine you are planning a road trip. You can then use the Maps API to create a customer map with your planned route, stops, and estimated travel times. Now imagine you own a coffee shop. The Maps API can then help you display your location and operating hours on a map, making it easier for customers to find you. Want to learn more about building apps with APIs? Check out the Maps JavaScript API (https://openstax.org/r/76MapJavaAPI) resource.

Google the APIs for an HLL that we have mentioned and find some functionality that applies to everyday life. Think of an app you use every day. How do you think it might use APIs? Provide a couple of scenarios to explain your choice.

History of HLLs

The evolution of HLLs began so that programmers could write programs in a familiar notation rather than using numbers (machine languages) or mnemonics (assembly languages). While there may be similarities in syntax among them, there are distinct purposes for which their development occurred. There is a much larger variety of HLLs than the ones mentioned in this section, but we will be looking at a few up close.

Fortran

In the early 1950s, IBM created one of the first HLL compilers for the Fortran language, which is one of the single biggest advances in computing. While Fortran was mostly used in mathematics and science, it could be easily read. Fortran makes it possible for programmers to comment their code by starting program lines with "!". It uses conditional statements with goto statements to branch out to different parts of the code. It also uses "do...end do" iterative statements. It was also the first HLL to use a compiler, computer software that converts source code from one language to another.

```
! Compute the average
Average = Sum / List_Len
! Count the values that are greater than the average
Do Counter = 1, List_Len
   If (Int_List(Counter) > Average) Then
      Result = Result + 1
   End If
End Do
```

COBOL

Common Business-Oriented Language (COBOL) was developed to be a high-level language for business that was standardized by the American National Standards Institute (ANSI) group in 1968. COBOL represents a distinct milestone in the evolution of computer science because of the ways in which it differed from Fortran.

The following code snippet illustrates reading an inventory record and computing the available stock:

```
100-PRODUCE-REORDER-LINE.
   PERFORM 110-READ-INVENTORY-RECORD.
   IF CARD-EOF-SWITCH IS NOT EQUAL TO "Y"
      PERFORM 120-CALCULATE AVAILABLE STOCK
      IF AVAILABLE STOCK IS LESS THAN BAL-REORDER-POINT
         PERFORM 130-PRINT-REORDER-LINE
```

```
110- READ-INVENTORY-RECORD.
   READ BAL-FWD-FILE RECORD
      AT END
         MOVE "Y" TO CARD-EOF-SWITCH.

. . . .
```

We can see that COBOL has a very different type of syntax than Fortran. It is purposed differently as it is very aligned to business applications and the programming of specific business activities that make up business processes.

BASIC

Beginner's All-Purpose Symbolic Instruction Code (BASIC) was developed in 1971. It is a programming language that has enjoyed widespread use. A variation of BASIC referred to as Visual Basic (VB) was the language responsible for much of the development work performed on the new generations of personal computers as it was easy to learn and read. Today it has evolved into Visual Basic .NET.

The following code snippet illustrates the same computation and comparison of an average we did in Fortran but this time in BASIC:

```
REM Compute the average
average = sum / listlen
REM Count the values that are greater than the average
FOR counter = 1 to listlen
   IF intlist(Counter) > average
      THEN result = result + 1
   End If
NEXT
```

Note that the BASIC syntax has its roots in Fortran but is more efficient.

Pascal and C

The programming languages introduced so far follow the **imperative language** paradigm that emphasize a "tell the computer what to do" approach. Pascal and C distinguish themselves by being both procedural and imperative languages, and they were invented at approximately the same time. A **procedural language** allows programmers to group statements into blocks of code within the scope of which variables may be defined and manipulated independently from the rest of a program. These blocks can be named, in which case it allows programmers to create functions or procedures that can be called from other parts of a program. Similar to other imperative languages, both Pascal and C also focus on evaluating expressions and storing results in variables (e.g., $a = 10$; $b = 5$; $c = a + b$).

The following Pascal code snippet illustrates the same computation and comparison of the average computed previously in BASIC:

```
{ Compute the average }
average := sum / listlen;
{ Count the values that are greater than the average }
for counter := 1 to listlen do
   if (intlist[counter] > average) then
      result := result + 1;
```

The introduction of procedures in Pascal improved programs' readability by allowing programmers to write

more modular code. Pascal became the preferred teaching language during the 1970s and early 1980s.

Pascal was overshadowed in commercial applications by C, which came into existence after the initial work on the UNIX operating system was completed in the late 1960s. That first OS version was written in assembly language, yet in the early 1970s, C became a better alternative. At the time, it was the perfect language for creating operating systems and was a huge commercial success.

The following C code snippet illustrates the same computation and comparison of the same computed average:

```
/* Compute the average */
average = sum / listlen;
/* Count the values that are greater than the average */
for (counter = 0; counter < listlen; counter++)
    if (intlist[counter] > average) result++;
```

The introduction of functions in the C language improved programs' readability and writability by allowing programmers to write more modular code. The C language runtime was also more efficient. C became the preferred language for commercial applications during the 1970s and early 1980s.

C++ and Objective C

By the mid-1980s, businesses started focusing on user experience (UX), the overall experience of a person using a computer application, especially in terms of how easy or pleasing it is to use, and the **user interface (UI)**, the point at which human users interact with a computer, website, or application. Windows-based UIs were adopted as new paradigms, which drove the creation of standards, perspectives, and sets of ideas that should be used to describe the structure and methodologies of an HLL. This resulted in the adoption of the OOP paradigm and the creation of OOP languages. The shift to OOP allowed software to focus on data and objects.

The C++ programming language extended the middle-level language features of C with OOP features that facilitated the expression of real-world requirements in programs, including in particular the support of graphical user interfaces (GUIs). Microsoft adopted C++ as the programming language for its Windows systems.

The syntax of the code in C++ for basic computation is exactly the same as the C code shown previously. There are major syntactical additions in C++ to support OOP. It became the preferred language during the mid-1980s and early 1990s and for the programming of GUIs. The code in Figure 7.4 illustrates the basics of Windows GUI programming using C++ and the Win32 API to create a simple "Hello World!" application with a graphical user interface (GUI).